BASIC II för ABC80

Kommenterad disassemblering

med jämförelse mot ABC800 M

Z80-kod, $0000–$77FF (32 KB-ROM)

Källtexten återassembleras till exakt samma bytes som ROM:en.

Innehåll

1 Inledning **1**

2 Systemvariabler och externa adresser **4**

3 Listning **6**

3.1 RST-vektorer och fasta ingångar . . . 6
3.2 Kallstart . . . 7
3.3 Kommandoloopen . . . 9
3.4 RAMINI: kopieras till $FF7B-$FF99 vid RESET . . . 9
3.5 CON: drivrutin. A = funktion (se filhuvudet). . . . 10
3.6 SCRTXT: skriv BC tecken från HL på skärmen . . . 10
3.7 CONINP: radinmatning från tangentbordet . . . 14
3.8 KEYWT: vänta på tangent (GETKYV pekar hit) . . . 15
3.9 KEYINT: avbrott från PIO A (vektor $B4) . . . 16
3.10 Nyckelordstabeller . . . 17
3.11 Tabellhuvuden (10 bytes) för funktioner (FNKWP) och satser (STKWP) . . . 18
3.12 JPTABL: den stora hopptabellen (204 ord) för kompilering och körning . . . 21
3.13 FNTAB: hopptabell (101 ord) för funktioner och uttryckspref ix . . . 24
3.14 CLRALL/CLRVAR/CLSALL: nollställ variablerna och stäng filerna . . . 26
3.15 RUNCLR: nollställ före körningen (från LINK) . . . 29
3.16 OPEN: öppna en fil ur en text. HL = texten, A = sätt (se OPENIX). . . . 34
3.17 DOLINE: utför en inmatad rad (HL = bufferten) . . . 39
3.18 INSLIN: lägg in (eller ersätt/ta bort) en programrad . . . 41
3.19 RUNLP: kör program- eller direktkod från DE . . . 70
3.20 EXECA: utför uttryckskoden A (RST $20 hamnar i EXECOD) . . . 72
3.21 Strängar . . . 89
3.22 VARLNK: slå upp (eller skapa) en variabel, DE = namnet . . . 121
3.23 PSCAN: gå igenom programmet och anropa IX för varje referens . . . 139
3.24 CEXPR: kompilera ett uttryck (prioritetsklättring) . . . 145
3.25 Filsatserna vid körning . . . 147
3.26 NMICLK: realtidsklocka (NMI, 50 Hz) . . . 153
3.27 ERRHND: felhantering. A = felnummer (0 = BREAK/Stop). . . . 155
3.28 DECOMP: LIST, översätt uttryckskoden tillbaka till text . . . 160
3.29 Direktkommandon . . . 168
3.30 DLINE: LIST, en programrad till text . . . 169
3.31 ADD$, SUB$, MUL$, DIV$ (a$,b$,n), COMP%(a$,b$), VAL(a$), NUM$(x) . . . 178
3.32 Talutskrift: tal till text (PRINT, PRINT USING, NUM$, LIST, ADD$ m.fl.) . . . 181
3.33 DVCALL: anropa drivrutinen för FCB:n IX med funktionen A . . . 193
3.34 Flyttal . . . 201
3.35 FTODEC: flyttal -> decimal form (för PRINT, LIST och NUM$) . . . 214
3.36 DECTOF: decimal form -> flyttal (för VAL, INPUT/READ och kompileringen) . . . 218
3.37 SERIES: beräkna ett polynom med Hornerschema . . . 240
3.38 ADD$, SUB$, MUL$, DIV$: räkning med decimala tal på stacken . . . 241
3.39 Tilläggsprogrammet på $6000: DOS (skivminnen via ABC-bussen) . . . 250
3.40 SECIO: läs eller skriv en sektor via ABC-bussen . . . 252
3.41 Skivans format (MF-diskett, 4 sektorer per kluster) . . . 257
3.42 DOS-fel . . . 262
3.43 DOSDRV: drivrutinen för alla DOS-enheter . . . 285
3.44 Tilläggsprogrammet på $7000: enheten PR: (skrivare) . . . 289
3.45 PRDRV: drivrutinen för PR:. A = funktion, IX = FCB. . . . 291
3.46 PRPRT: skriv BC tecken från HL. Efter varje tecken PRMARG. . . . 294
3.47 PROUT: skicka tecknet E (med paritet enligt PARITY). . . . 296

A Felkoder **299**

B Tokens och tabeller **301**

C Fel och egenheter **307**

D Oklara ställen **311**

E Källor **312**

F Symbolförteckning **313**

Inledning

Z80-disassemblering av BASIC II för ABC80 (`"BASICII 80/40"`). Återassemblerad ger listningen exakt samma bytes som ROM:en.

Koden är ABC800 M:s BASIC II anpassad till ABC80: skärmen, tangentbordet, ljudet och klockan är utbytta och kassetten borttagen, allt på plats utan att någon adress har flyttats. Tolken `$06AD`-`$5FFF` är byte för byte densamma utom NUL:-länken (`$11B2`), `POKE` (`$3A34`) och klockan (`$3ABA`). DOS:en på `$6000` är en äldre version än ABC800 M:s (1984-03-02 mot 1984-04-03), och `$7000` har PR: i stället för ABC800 M:s grafik och V24:. Se bilagan om ABC800 M.

ROM:en skriver själv `ABC800` som prompt.

Minneskarta (som ROM-koden använder den)

```
$0000-$5FFF  BASIC-tolken
$6000-$77FF  tilläggsprogram; vid RESET anropas varje sida (256 bytes)
             i $6000-$77FF som börjar med JP ($C3). Här finns två:
             $6000 (DOS för ABC800:s skivminnen, JP-tabell med 35
             ingångar) och $7000 (enheten PR:, skrivare via V24 eller
             Centronics; $7370-$77FF är tomma, $FF)
$7800-$7BFF  oanvänt ($FF i ROM-filen)
$7C00-$7FFF  bildminne, 24 rader x 40 tecken (se ROWTAB)
$8000-       RAM (BOTRAM): nedifrån COMMON-området och variablerna
             (VARBEG, FREPTR), överst under MEMTOP (= STKTOP - $276)
             programmet (PRGBEG-PRGEND, se NEW)
$F500-$FCFF  DOS: sektorbuffertar, 256 bytes per kanal 0-7
             (stacken flyttas till $F500)
$FD00        stackens topp vid start (STKTOP)
$FD00-$FE18  DOS: arbetsvariabler, kanaltabell, kopior av tabeller
$FF00-$FFFF  systemvariabler, IY = $FF00
$FF7B-$FF99  RAM-vektorer, kopierade från RAMINI ($0147)
$FFB0-$FFD7  IM 2-vektortabell (I = $FF)
$FFEF-$FFF5  realtidsklocka (nollställs inte vid RESET)
```

RST-vektorer

```
RST $08  hopp till $FF9A (krok, sätts inte av denna ROM)
RST $10  fel; efter RST-instruktionen följer felnumret
RST $18  INC HL och hoppa över blanktecken, A = nästa tecken
RST $20  utför nästa interna kod vid DE (EXEC)
RST $28  kastar returadressen och fortsätter som RST $20
RST $30  hopp till $FF9D (krok, sätts inte heller)
RST $38  hopp till $FFA0 (krok, sätts inte heller)
```

Felutskrift

```
"Error n in line m." (eller "Stop"), där n är felnumret efter RST $10.
ROM:en har inga feltexter; kroken $FFAC kan ge en text i stället för
numret (sätts av tilläggsprogrammet på $6000).
Felen ges på tre sätt: RST $10 med numret efter, A = numret och
ERRJP (återvänder vid CON eller RESUME utan radnummer), eller A = numret och
L3B6B (stacken sätts om). Kompilatorn och länkningen lämnar i stället
CY = 1 och numret i A. Felnummer vars orsak framgår av koden
(decimalt, tolkning; ställena i koden är kommenterade):
Vid körning
   21 felaktigt filnamn / okänd enhet   32 filen är inte öppen
   38 POSIT: för stor position         130 spill i flyttalsräkning
  131 index utanför fältet             132 heltalsspill
  133 ADD$ m.fl.: division med 0, för många siffror
  134 fel position/längd i LEFT$, RIGHT$, MID$
  135 negativt antal i STRING$/SPACE$; TAB med 0 eller negativt,
```

```
      eller utanför PRINT
  136 MID$-sats: texten längre än fönstret; CHAIN: namnet för långt
  137 strängen längre än variabelns maxlängd
  138 ON-index utanför listan          140 RETURN utan GOSUB / fel typ
  143 SYS med okänt nummer             144 felrad körd ($8B)
  141 READ: slut på DATA               145 FNEND utan RETURN
  146 PRINT USING: tom mall, mall utan fält, tal till strängfält,
      värdena tar slut mitt i mallen
  147 READ/INPUT: "," väntas
  148 INPUT: för få värden             149 RESTORE till rad utan DATA
  150 INPUT: för många värden          151 RESUME utan fel
  183 NEXT vars FOR inte har körts (som vid länkningen)
  200 odefinierad kod                  201 minnet räcker inte
  202 programmet förstört/skyddat      205 COMMON passar inte (ALLOCC)
  207 CON utan avbrott/fel              209 fel radintervall / RENUMBER
  210 felaktigt tal eller radnummer    211 SINGLE/DOUBLE: flyttal finns
   52 funktionen finns inte för enheten (CON:, PR:)
DOS ($6000, se $63F5): 19 ingen ledig kanal, 21 filen finns inte,
   35-48 fel från skivminnet, 64 nya namnet finns (NAME), 231 AS
   saknas i NAME
Vid länkningen (RUN)
  180 raden finns inte           181 hopp in i/ut ur DEF FN
  182 FOR/WHILE utan NEXT/WEND   183 NEXT/WEND utan FOR/WHILE
  184 NEXT/WEND passar inte      185 FOR med öppen styrvariabel
  186 NEXT med lokal variabel    187 FN utan DEF
  188 DEF FN två gånger          189 DEF/FNEND i fel ordning
  190 fält med fel antal index   191 strängparameter tilldelas
  206 FN i direktläge före RUN   208 otillåtet i direktläge
  223 fel antal/typ av argument till FN
Vid kompileringen av en rad (syntaxfel)
  220 okänd sats                 221 radslut väntas (CR, "!")
  222 satsen måste stå först på raden (END, DATA, DEF, FNEND)
  223 sträng väntas, fel antal   224 tal väntas (eller strängvariabel)
  225 variabel väntas            226 fel sats efter ON
  227 "," väntas                 228 "=" väntas
  229 ")" väntas                 230 "AS FILE" väntas
  232 "TO" väntas                233 radnummer väntas
  234 bokstav väntas (operand saknas: "A=", "()", "A=1:")
```

Enhetsdrivare

```
Enhetslistan börjar i DEVLST. Varje post: +0,+1 länk (0 = sista),
+2..+4 namn (3 tecken), +5,+6 drivrutinen. Drivrutinen anropas med
A = funktion och IX = filblocket (FCB):
  0 OPEN  1 PREPARE  2 CLOSE  3 INPUT (rad)  4 PRINT  5 GET  6 PUT
CY = 1 vid fel, A = felnumret.
```

Filblock (FCB)

```
+0,+1 länk till nästa FCB (FCBLST är början; nya läggs sist),
+2 filnummer, +3 flaggor: bit 0 öppen, 1 CON: (stängs inte av
CLSF1), 3 bufferten ändrad, 4 fel vid CLOSE, 5 SAVE,
+4,+5 adressen till drivrutinens adress i enhetsposten, +6 kolumn,
+7 rad, +8 radlängd, +10 senaste funktion, +$0D-$14 bufferten (se
DVCALL). CON: har sitt FCB på $FF4C, programfilen på $FF63.
```

I/O-portar

```
$06        ljudgeneratorn (BEL: OUT 0 och sedan $83)
$07        IN vid RESET: återställ korten på ABC-bussen
$38/$39    PIO A data/styr: tangentbord, bit 7 = tangent nedtryckt,
           bit 0-6 = teckenkod; avbrott via vektor $B4
$3A/$3B    PIO B data/styr: läge 3, bit 0-2 och 7 ingångar ($CF, $87
           vid RESET). PR: använder bit 3 (V24 ut), bit 4 och bit 1
           (skrivaren inte klar)
$00-$03    ABC-bussen: PR: väljer Centronics-kortet med OUT $BC till
           $01 och $03, läser status på $01 och skriver data på $00;
           tilläggsprogrammet $6000 använder också $00-$04
```

Länkning vid RUN (se LINK $2E27, PSCAN $3552)

Namn och radnummer i koden byts mot adresser före körningen: variabler mot 16-bitarsreferenser (REFADR), radnummer och FN-anrop mot relativa adresser, FOR/NEXT och WHILE/WEND kopplas ihop via block i variabelområdet. UNLINK gör det ogjort. Så var det också i ABC80-BASIC:en (Finnved 1979, kap. 3).

Intern kod (översikt, se tabellen JPTABL $0AC5)

Varje rad kompileras vid inmatningen till intern kod: satskoder följda av uttryck i omvänd polsk notation (postfix). Värdena ligger på Z80-stacken; varje rutin avslutas med RST $28 och hämtar nästa kod. Parenteser sparas som en egen kod ($A7) som bara används av LIST. Samma uppläggning beskrivs för ABC80:s BASIC (Finnved 1979): kompilering till postfix, typkontroll och typomvandling vid inmatningen, fixup av variabel- och radadresser vid RUN, och LIST som översätter postfix tillbaka till infix. BASIC II är ändå en annan kodbas än ABC80-BASIC:en (gemensamma block: bara 296 bytes).

Att läsa listningen

Varje rad visar adress, bytes (högst fyra; fler markeras med …), etikett, instruktion och kommentar. Etiketter, variabelnamn och adresser i ROM är länkar i PDF-filen. Längre kommentarer före en rutin står som löptext; rubricerade block ger avsnitten i innehållsförteckningen. Tolkningar som är osäkra är markerade med (?). Allt är härlett ur koden i ROM:en; uppgifter från andra källor har källan inom parentes, t.ex. “(Finnved 1979)”, och källorna står i kapitlet Källor. Symbolförteckningen sist ger adress och sida för varje etikett.

Systemvariabler och externa adresser

Namn	Adress	Beskrivning
DSLOT	$FD00	DOS: kopia av den aktuella kanalposten (SLTCPY)
DUNIT	$FD01	DOS: enhetskod (låga 5 bitarna)
VFD02	$FD02	DOS: kanalpost +2/+3, aktuell post
VFD04	$FD04	DOS: kanalpost +4/+5, första posten i den aktuella utsträckningen
VFD06	$FD06	DOS: kanalpost +6, utsträckningens första kluster, höga byten
VFD07	$FD07	DOS: kanalpost +7, klustrets bit 7-5 och antal kluster - 1 (bit 4-0)
VFD08	$FD08	DOS: kanalpost +8/+9, indexsektorn
VFD0A	$FD0A	DOS: kanalpost +A/+B, storleken (högsta posten)
VFD0C	$FD0C	DOS: kanalpost +C/+D, sektorer räknade i indexet (DFCLS)
VFD0E	$FD0E	DOS: kanalpost +E, index i bufferten; i DFCLS platsen i indexet
DBUFP	$FD12	DOS: pekare i sektorbufferten ($F500..)
VFD14	$FD14	DOS: aktuell kanal * 16 (RECSEC)
VFD15	$FD15	DOS: kortets status efter SECIO (bit 6 skrivskydd, $80 inget kort)
DFNA	$FD1C	DOS: katalogfunktionens nummer A (DOSFN1)
DFNBC	$FD1D	DOS: katalogfunktionens C; felindex vid fel
VFD1E	$FD1E	DOS: katalogfunktionens B = kanal * 16
DFNDE	$FD1F	DOS: katalogfunktionens DE, namn och typ
DERRX	$FD33	DOS: felutgång (DERRDV, ABORT ...)
VFD3F	$FD3F	DOS: kortadressen som SELCRD valde senast
SLTTAB	$FD40	DOS: kanaltabellen, 16 bytes per kanal 0-7
VFD41	$FD41	DOS: kanal 0:s flaggbyte i SLTTAB ($FF = ledig)
DEVLNK2	$FDD0	DOS: enhetspost i RAM (8 bytes ur $6E35), länken = förra DEVLST
DOSSTR	$FDD8	DOS: satstabellens huvud (DOSSTH), länken = förra STKWP
VFE02	$FE02	DOS: krok före varje sektor (SECIO), RET sist i kopian av CRDTAB
DSVHL	$FE05	DOS: kanal 0:s +0/+1 efter OPNINI, katalogposten och enhetsbyten för BASICINI.SYS (INIERR)
DSAVSP	$FE07	DOS: SP vid anropet av drivrutinen
INIBUF	$FE09	DOS: 16 felnummer ur BASICINI.SYS (post 0, bytes 6-21), sorterade, $FF sist
ONEFRM	$FF04	FNFRM vid ON ERROR GOTO
PRGBEG	$FF06	programmets början (20 bytes huvud, sedan $87 och raderna)
PRGEND	$FF08	programmets sista byte
FREPTR	$FF0A	första lediga byte ovanför variablerna
BOTRAM	$FF0C	första RAM-adressen ($8000); COMMON-områdets början
MEMTOP	$FF0E	minnesgräns: STKTOP - $276
CONSP	$FF12	SP för CON efter fel/STOP
RESSP	$FF14	SP för RESUME
STKTOP	$FF16	stackens topp ($FD00)
FNFRM	$FF18	ramen för aktuellt FN-anrop (lokala variabler)
VFF1A	$FF1A	gräns för koden som EMIT skriver (fel 20)
SCNPTR	$FF20	aktuell sats under programgenomgången
LNKDEF	$FF28	DEF FN som länkas just nu (0 = ingen)
LNKBLK	$FF2A	öppna FOR/WHILE-block under länkningen
VARLST	$FF2C	variabellistans huvud
VARBEG	$FF2E	variablernas början (referens 0)
COMEND	$FF30	COMMON-områdets slut; storleken är COMEND - BOTRAM
COMSUM	$FF32	COMMON-summan, prövas när ett program läses in (fel 205)
VFF36	$FF36	variabelns adress (PTRF/PTRI/PTRS, NEXT); läses inte av ROM:en
LSTARR	$FF38	senast använda fält (ARRELM), för VAROOT

Namn	Adress	Beskrivning
PRGPTR	$FF3A	raden som körs: sätts vid varje radslut (EOL) och hopp och ger radnumret vid fel
DATPTR	$FF3C	READ: läspositionen i DATA-texten
DATSTM	$FF3E	READ: aktuell DATA-sats (länken)
FCBLST	$FF40	första FCB i fillistan
ONEADR	$FF42	koden för ON ERROR GOTO-raden
FRMSP	$FF45	senaste ramen på stacken (GOSUB $FE, RUN $FD ...)
CURCOL	$FF52	markörens kolumn (0-39)
CURROW	$FF53	markörens rad (0-23)
DEVLST	$FF7B	första posten i enhetslistan
STKWP	$FF7D	pekare till satsernas nyckelord
FNKWP	$FF7F	pekare till funktionernas nyckelord
SYSOPT	$FF81	$FD00 vid RESET, $FB00 av DOS:en; läses inte av ROM:en (men av ABC 800:s)
AUTOST	$FF83	pekare till startkommandot
BRKPTR	$FF85	pekare till BREAK-flaggan
UNSVEC	$FF87	JP till UNSAVE: $0036 (fel 200) eller DOS:ens UNSRUT
VFF88	$FF88	UNSVEC:s mål
HOOK1	$FF8D	JP-krok i DOCMD före varje direktkommando (CMDKW), normalt RET
GETKYV	$FF90	JP till tangentrutinen
ERRHK	$FF93	krok efter felutskrift
HOOK3	$FF96	JP-krok för TRACE (TRCHOK), anropas vid varje ny rad
RST08V	$FF9A	krok för RST $08, sätts inte av ROM:en
RST30V	$FF9D	krok för RST $30, sätts inte av ROM:en
RST38V	$FFA0	krok för RST $38, sätts inte av ROM:en
HOOKA3	$FFA3	krok för OPEN med bit 7 i typen, sätts inte av ROM:en
HOOKA6	$FFA6	krok för uttryckskoderna $7D-$7F (EXECA), sätts inte av ROM:en
HOOKA9	$FFA9	krok för argumenttyp 3 i signaturbyten, sätts inte av ROM:en
ERRTXV	$FFAC	krok: feltext (0 = ingen)
PRDEFR	$FFC0	PR: standardinställningen, 11 tecken ur PRDEF (PRAREA); det första används inte (C eller V väljs efter kortet), och de andra kan ändras med POKE
IM2NMI	$FFD6	NMI-klockan i vektortabellen
PRUNIT	$FFDC	PR: arbetsblocket (inställning, radräknare)
TABPTR	$FFE0	mallrad för TAB vid radinmatning
KEYFLG	$FFE2	bit 7 = tangent har kommit
LASTKY	$FFE3	senaste tangentkod
PRBAUD	$FFE4	PR: hastighetssiffran som tecken
PRBDLY	$FFE5	PR: fördröjning per bit (BITDLY)
–	$FFEB	tangentrepetitionen: inre räknaren (KEYWT)
–	$FFEC	tangentrepetitionen: yttre räknaren, 70 före första repetitionen, sedan 8
YEAR	$FFEF	klockan: år 0-99 (POKE 65519,år,månad,dag,timme,minut,sekund)
MONTH	$FFF0	klockan: månad 1-12
DAY	$FFF1	klockan: dag; 0 = klockan står
–	$FFF2	klockan: timme 0-23
MINUTE	$FFF3	klockan: minut 0-59
SECOND	$FFF4	klockan: sekund 0-59
–	$FFF5	klockan: TICKS, NMI:er under sekunden (0-49, NMICLK)
DSYSUN	$FFF7	DOS: systemenhet (0 = ingen)
VFFF9	$FFF9	DOS: enhetskoden som UFD ($1E) står för

Listning

3.1 RST-vektorer och fasta ingångar $0000

Samma uppläggning som i ABC800: mellan RST-adresserna ligger JP-instruktioner som fasta ingångar för maskinkodsprogram, bl.a.

```
$0002 GETKEY  vänta på tangent (via RAM-vektorn GETKYV)
$0005 INLINE  läs en rad från tangentbordet (HL = buffert, C = max)
$000B PRTXT   skriv text på skärmen (HL = text, BC = längd)
```

Tre RST-vektorer ($08, $30, $38) går till krokar i RAM ($FF9A..). De övriga används av tolken själv: RST $00 kallstart, RST $10 fel med numret i byten efter, RST $18 nästa tecken i texten (blanka hoppas över, Z vid $00), RST $20 utför koden vid DE och RST $28 samma sak efter att returadressen har kastats. Ingångarna från $005A till $0069 används av DOS:en på $6000; $002B, $004E, $006C och $006F anropas inte av något i ROM:en.

```
0000  18 70        RESET:  JR    INIT          RST $00: kallstart

0002  C3 90 FF     GETKEY: JP    GETKYV        fast ingång: A = nästa tangent

0005  C3 FA 02     INLINE: JP    CONINP        fast ingång: läs en rad (HL, C)

0008  C3 9A FF     RST08:  JP    RST08V        RST $08: kroken i RAM

000B  C3 C3 01     PRTXT:  JP    SCRTXT        fast ingång: skriv BC tecken från HL

000E  7B FF                DEFB  $7B,$FF      ordet $FF7B (DEVLST); läses inte av ROM:en
```

RST $10*: fel. Felnumret står i byten efter* RST*-instruktionen.*

ERRHND *återvänder bara vid* CON *eller* RESUME *utan nummer; då blir det fel* $CF *(207).*

```
0010  E1           ERROR:  POP   HL            returadressen
0011  7E                   LD    A,(HL)        felnumret efter RST
0012  CD 80 3B     ERRA:   CALL  ERRHND        tillbaka bara efter CON
0015  D7                   RST   $10           ett nytt fel
0016  CF                   DEFB  $CF           fel 207

0017  00                   DEFB  $00           oanvänd
0018  C3 19 4C     SKIPBL: JP    S4C19         RST $18: ett steg fram, blanka över; Z vid $00

001B  06 02        CEXPS:  LD    B,$02         kompilera ett stränguttryck
001D  C3 AA 16             JP    CEXPT

0020  C3 A9 1D     EXEC:   JP    EXECOD        RST $20: utför koden vid DE

0023  C3 4B 4B     S0023:  JP    KWSRCH        fast ingång: KWSRCH (DOS:en)
```

Text CR LF (används av CRLF*) och på* $002E *standardkommandot vid start:* ! *CR (*AUTOST *pekar hit).*

```
0026  0D 0A        CRLFTX: DEFB  $0D,$0A
0028  F1           EXECNX: POP   AF            RST $28: kasta returadressen
0029  18 F5                 JR    EXEC          och utför nästa kod

002B  C3 88 11     L002B:  JP    FNDFIL        fast ingång: IX = FCB för filen A

002E  21 0D        AUTOTX: DEFB  $21,$0D
0030  C3 9D FF     L0030:  JP    RST30V        RST $30: kroken i RAM

0033  C3 80 3B     ERRJP:  JP    ERRHND        fel A som går att fortsätta

0036  D7           UNSDEF: RST   $10           fel 200: odefinierad kod
0037  C8                   DEFB  $C8

0038  C3 A0 FF     L0038:  JP    RST38V        RST $38: kroken i RAM

003B  2A 06 FF     S003B:  LD    HL,(PRGBEG)   HL = programmets början
003E  C9                   RET

003F  DD 2A 06 FF  S003F:  LD    IX,(PRGBEG)   IX = programmets början
```

```
0043  C9                 RET
```

FSTLIN: HL = PRGBEG + $14, $87-byten före första raden. Förstör F.

```
0044  2A 06 FF     FSTLIN: LD   HL,(PRGBEG)
0047  D5                   PUSH DE
0048  11 14 00             LD   DE,$0014
004B  19                   ADD  HL,DE
004C  D1                   POP  DE
004D  C9                   RET

004E  C3 AF 17     L004E:  JP   CFACT          fast ingång: en operand
```

BUFADR: HL = STKTOP - $A0 = radbuffertens adress. Stacken sätts strax under bufferten. Förstör BC och F.

```
0051  2A 16 FF     BUFADR: LD   HL,(STKTOP)
0054  01 A0 00             LD   BC,$00A0
0057  ED 42                SBC  HL,BC          CY från anroparen, 0 (provat)
0059  C9                   RET

005A  C3 52 11     S005A:  JP   DRVJP          fast ingång: drivrutinen (DOS:en)

005D  C3 35 10     S005D:  JP   OPEN           fast ingång: öppna en fil ur en text

0060  C3 5F 10     S0060:  JP   OPENIX         fast ingång: öppna med längden i C

0063  C3 B6 4A     S0063:  JP   PARSFN         fast ingång: dela upp ett filnamn

0066  C3 B3 3A     NMI:    JP   NMICLK         ABC800: JP $FF8A (RAM-vektor, JP $0000)

0069  C3 05 11     S0069:  JP   PRGCLS         fast ingång: stäng programfilen nu och efteråt

006C  C3 D3 0F     L006C:  JP   ALLOC          fast ingång: reservera minne

006F  C3 26 2A     L006F:  JP   L2A26          mitt i FNCALL; ROM:en går inte hit
```

3.2 Kallstart $0072

Nollställer RAM från $FFEE ned till $8000 (klockan $FFEF-$FFF5 lämnas orörd så att tiden överlever RESET). CPIR letar sedan efter första nollbyten från $8000 och uppåt; där RAM saknas blir det ingen nolla, så BOTRAM blir den lägsta adressen med RAM ($8000 med fullt minne).

RAMINI (31 bytes) kopieras till $FF7B.. och IM 2-tabellen byggs på $FFB0.., I = $FF. PIO B och PIO A (tangentbordet, KBINIT) initieras. NMI-vektorn i tabellen ($FFD6) pekar på klockan.

```
0072  DB 07        INIT:   IN   A,($07)        port 7: återställ ABC-bussen (Markesjö 1978, s. 122)
0074  01 EE 7F              LD   BC,$7FEE       $8000-$FFEE nollas
0077  21 EE FF              LD   HL,$FFEE
007A  11 ED FF              LD   DE,$FFED
007D  F3                    DI                  avbrotten av
007E  AF                    XOR  A
007F  77                    LD   (HL),A         $FFEE = 0, som LDDR för nedåt
0080  ED B8                 LDDR
0082  ED B1                 CPIR                första nollbyte från $8000
0084  2B                    DEC  HL             HL = första nollan
0085  22 0C FF              LD   (BOTRAM),HL
0088  21 47 01              LD   HL,$0147       RAMINI -> $FF7B
008B  11 7B FF              LD   DE,$FF7B
008E  01 1F 00              LD   BC,$001F
0091  ED B0                 LDIR
0093  7A                    LD   A,D            A = $FF
0094  ED 47                 LD   I,A
0096  ED 5E                 IM   2
0098  0E 0A                 LD   C,$0A          IM 2-tabellen från $0166
009A  1E B0                 LD   E,$B0
009C  ED B0                 LDIR                $FFB0–$FFB9
009E  21 B8 FF              LD   HL,$FFB8       $FFBA-$FFD7 blir $03DD
00A1  0E 1E                 LD   C,$1E
00A3  ED B0                 LDIR
00A5  3E CF                 LD   A,$CF
00A7  D3 3B                 OUT  ($3B),A        PIO B: läge 3
```

```
00A9  3E 87              LD   A,$87
00AB  D3 3B              OUT  ($3B),A        bit 0-2 och 7 in
00AD  CD 16 04           CALL KBINIT         tangentbordet
00B0  00                 NOP                 ABC800 initierar här seriekrets och CTC
00B1  00                 NOP
00B2  00                 NOP
00B3  00                 NOP
00B4  00                 NOP
00B5  21 B3 3A           LD   HL,$3AB3       NMI -> klockan
00B8  22 D6 FF           LD   (IM2NMI),HL
00BB  FD 21 00 FF        LD   IY,$FF00       systemvariablerna
00BF  31 00 FD           LD   SP,$FD00
```

256 st `RETI` *(B = 0 efter* `LDIR`*) avslutar eventuella avbrott som är under behandling i PIO:erna.*

```
00C2  CD DD 03     RETIS: CALL DORETI
00C5  10 FB               DJNZ RETIS
00C7  00                  NOP               ABC800: CALL $0634 (kassettens seriekrets)
00C8  00                  NOP
00C9  00                  NOP
```

`CON`*: öppnas som fil 0 med FCB på* `$FF4C`*, skärmen töms.*

```
00CA  21 4C FF            LD   HL,$FF4C
00CD  22 40 FF            LD   (FCBLST),HL  CON: först i FCB-listan
00D0  E5                  PUSH HL
00D1  21 7B 01            LD   HL,$017B     CON:
00D4  AF                  XOR  A            A = 0: sätt 0 (OPEN)
00D5  DD E1               POP  IX           IX = FCB:n
00D7  CD 35 10            CALL OPEN
00DA  DD CB 03 CE         SET  1,(IX+$03)   bit 1: stängs inte av CLSF1
00DE  CD B2 02            CALL CLS          ABC800: CALL $0299 (bildkretsen, sedan CLS)
```

Tilläggsprogram: varje sida `$6000`*,* `$6100` *..* `$7700` *som börjar med* `$C3` *(*`JP`*) anropas (*`JP (HL)` *i* `$1DBC`*).*

```
00E1  21 00 60            LD   HL,$6000
00E4  3E C3        EXTSCN: LD   A,$C3
00E6  BE                   CP   (HL)
00E7  CC BC 1D             CALL Z,S1DBC      starta tilläggsprogrammet
00EA  24                   INC  H            nästa sida
00EB  3E 78                LD   A,$78
00ED  BC                   CP   H
00EE  20 F4                JR   NZ,EXTSCN    till och med $7700
```

`STKTOP` *= SP:* `$FD00`*, eller lägre om ett tilläggsprogram har flyttat stacken (med DOS:en på* `$6000` *blir det* `$F4DE`*).* `MEMTOP` *= SP -* `$276`*.* `NEW`*. Sedan kopieras startkommandot (*`AUTOST`*, normalt* `!` *CR) till radbufferten och körs som om det hade skrivits in.*

```
00F0  ED 73 16 FF         LD   (STKTOP),SP
00F4  21 8A FD            LD   HL,$FD8A     HL = SP - $276
00F7  39                  ADD  HL,SP
00F8  22 0E FF            LD   (MEMTOP),HL
00FB  CD C6 0E            CALL NEW
00FE  ED 5B 83 FF         LD   DE,(AUTOST)
0102  CD 51 00            CALL BUFADR       HL = radbufferten
0105  F9                  LD   SP,HL
0106  EB                  EX   DE,HL
0107  D5                  PUSH DE
0108  ED B0               LDIR
010A  18 26               JR   L0132        utför startkommandot
```

3.3 Kommandoloopen $010C

READY: återställ stacken, ny rad om markören inte står i kolumn 0, skriv ABC800 (6 tecken från PROMPT). CMDLP läser en rad från fil 0 (CON:) till bufferten (BUFADR) och utför den med S11DC. CY tillbaka = fel.

```
010C  FD 36 27 00  READY0: LD   (IY+$27),$00     CON-läget och ON ERROR av
0110  2A 16 FF     READY:  LD   HL,(STKTOP)
0113  F9                   LD   SP,HL
0114  CD E8 3C             CALL CRLFC            ny rad om kolumnen inte är 0
0117  21 46 3D             LD   HL,$3D46         ABC800
011A  01 06 00             LD   BC,$0006
011D  CD 96 11             CALL PRCON
0120  FD CB 23 86  CMDLP:  RES  0,(IY+$23)       BREAK-biten nollställs
0124  CD 0F 11             CALL CLSPRG           stäng programfilen
0127  CD ED 3C             CALL CRLF
012A  CD 51 00             CALL BUFADR
012D  F9                   LD   SP,HL            stacken under bufferten
012E  E5                   PUSH HL
012F  CD 4D 49             CALL CONLIN           läs en rad från CON:
0132  E1           L0132:  POP  HL               HL = bufferten
0133  F9                   LD   SP,HL
0134  FD CB 23 D6          SET  2,(IY+$23)       direktläge
0138  FD CB 23 46          BIT  0,(IY+$23)
013C  C4 7B 3B             CALL NZ,STOP          BREAK under inmatningen
013F  CD DC 11             CALL DOLINE           kompilera och utför raden
0142  DA 6B 3B             JP   C,ERRSP          CY: fel
0145  18 D9                JR   CMDLP
```

3.4 RAMINI: kopieras till $FF7B-$FF99 vid RESET $0147

```
$FF7B DEVLST  $11B2 (NUL: -> CON: -> slut)
$FF7D STKWP   $092E satsernas nyckelordstabell
$FF7F FNKWP   $0704 funktionernas nyckelordstabell
$FF81 SYSOPT  $FD00 (läses inte av ROM:en)
$FF83 AUTOST  $002E startkommando
$FF85 BRKPTR  $FF23 (bit 0 sätts vid BREAK)
$FF87 UNSVEC  UNSAVE: JP $0036 (fel 200), DOS:en sätter UNSRUT
$FF8A RSTVEC  JP $0000
$FF8D HOOK1   JP $19CC (RET)
$FF90 GETKYV  JP $0378
$FF93 ERRHK   JP $19CC (RET), anropas efter felutskrift
$FF96 HOOK3   JP $2CC7
$FF99         $28
```

Därefter (IM2INI, $0166) 10 bytes till $FFB0: $03DD, $03DD, $03C1, $03DF, $03DD. $FFB8 kopieras sedan framåt så att resten av tabellen till $FFD7 också blir $03DD (RETI).

```
0147  B2 11 2E 09… RAMINI: DEFB $B2,$11,$2E,$09,$04,$07,$00,$FD
014F  2E 00 23 FF…         DEFB $2E,$00,$23,$FF,$C3,$36,$00,$C3
0157  00 00 C3 CC…         DEFB $00,$00,$C3,$CC,$19,$C3,$78,$03
015F  C3 CC 19 C3…         DEFB $C3,$CC,$19,$C3,$C7,$2C,$28
0166  DD 03 DD 03… IM2INI: DEFB $DD,$03,$DD,$03,$C1,$03,$DF,$03
016E  DD 03                DEFB $DD,$03
```

Bytes $0170-$017A används inte. De är en rest från ABC800: 8 bytes till tangentbordets seriekrets (port $23), sedan CTC:n: vektor $D0 och $A5 $7D (kanal 3, förskalning 256, tidskonstant 125 = klockan).

```
0170  31 1C 02 B0…         DEFB $31,$1C,$02,$B0,$03,$C1,$04,$44
0178  D0 A5 7D              DEFB $D0,$A5,$7D
```

Namnet som öppnas vid start, och enhetsposten för CON: (länk 0, CON, drivrutin $0190).

```
017B  43 4F 4E 3A  CONNAM: DEFM "CON:"
017F  0D                   DEFB $0D
0180  00 00 43 4F… CONDEV: DEFB $00,$00,$43,$4F,$4E,$90,$01
```

KEYSET: KEYFLG = $80, som när en tangent har kommit. Anropas av KEYINT, $1565 och $44EB. Förstör A.

```
0187  3E 80        KEYSET: LD   A,$80
0189  32 E2 FF             LD   (KEYFLG),A
018C  C9                   RET

018D  C3 FD 02     INLIN2: JP   CONIN2          fast ingång: radinmatning med mallen DE
```

3.5 CON: drivrutin. A = funktion (se filhuvudet). $0190

OPEN/PREPARE: flagga $04 i FCB+3, C = 40 (radlängd). CLOSE: inget (RET Z när A = 2). INPUT -> CONINP, PRINT och PUT -> CONPRT, GET -> CONGET.

Annan funktion ger fel 52 ($34). In: A = funktionen, IX = FCB:n, HL och BC = bufferten och dess längd. Ut: CY och A = felet vid fel.

```
0190  D6 02        CONDRV: SUB  $02
0192  C8                   RET  Z
0193  38 12                JR   C,CONOPN        0, 1: OPEN, PREPARE
0195  3D                   DEC  A
0196  CA FA 02             JP   Z,CONINP        3: INPUT
0199  3D                   DEC  A
019A  28 1F                JR   Z,CONPRT        4: PRINT
019C  3D                   DEC  A
019D  CA 6D 03             JP   Z,CONGET        5: GET
01A0  3D                   DEC  A
01A1  28 18                JR   Z,CONPRT        6: PUT
01A3  3E 34                LD   A,$34           okänd funktion
01A5  37                   SCF
01A6  C9                   RET
```

CONOPN: OPEN och PREPARE på CON:. FCB+3 = $04, C = 40 (radlängden) till OPENIX, CY = 0.

```
01A7  DD 36 03 04  CONOPN: LD   (IX+$03),$04
01AB  0E 28                LD   C,$28           40 tecken per rad
01AD  A7                   AND  A               CY = 0: inget fel
01AE  C9                   RET
```

Tomrum (nollor) från anpassningen till ABC80. ABC800 prövar här om bildminnet på $7801 går att skriva och väljer då 80 tecken per rad.

```
01AF  00 00 00 00…         DEFB $00,$00,$00,$00,$00,$00,$00,$00
01B7  00 00 00 C9          DEFB $00,$00,$00,$C9
```

CONPRT: skriv BC tecken från HL, sätt sedan FCB+6 = CURCOL. CURCOL är FCB+6 i CON:s egen FCB ($FF4C); för en annan fil som är öppnad på CON: förs kolumnen över till den filens FCB. In: HL, BC, IX = FCB:n. Förstör A och DE.

```
01BB  CD C3 01     CONPRT: CALL SCRTXT
01BE  3A 52 FF             LD   A,(CURCOL)
01C1  18 73                JR   L0236
```

3.6 SCRTXT: skriv BC tecken från HL på skärmen $01C3

SCRCHR behandlar ett tecken A (HL pekar efter det):

```
7 BEEP, 8 vänster, $0A radmatning, $0D CR, $0C töm skärmen,
ESC "=" rad kolumn (+$20) flyttar markören. Övriga < $20 ignoreras.
```

Vanliga tecken: CURADR ger DE = skärmadress, och SCRPUT skriver sedan så många tecken som möjligt i följd direkt (CPI räknar ned BC), tills ett styrtecken eller radslut. Tecken >= $80 skrivs som de är (bit 7 som markören). Efter radens sista kolumn står markören kvar på kolumn 40; ny rad blir det först vid nästa tecken (CURADR). ESC = verkar bara när hela sekvensen står i samma text: PRINT CHR$(27);=.. skriver =.. som vanliga tecken.

```
01C3  DD E5        SCRTXT: PUSH IX
01C5  DD 21 4C FF          LD   IX,$FF4C        FCB för CON:
01C9  79           L01C9:  LD   A,C
01CA  B0                   OR   B
01CB  CA 5B 03             JP   Z,L035B         BC = 0: klart
01CE  7E                   LD   A,(HL)
```

```
01CF  23                INC  HL
01D0  0B                DEC  BC
01D1  CD D6 01          CALL SCRCHR
01D4  18 F3             JR   L01C9
```

SCRCHR: ett tecken A. HL pekar efter tecknet och BC räknar tecknen som är kvar efter det (ESC = läser dem, SCRPUT skriver vidare så långt det går). In: A, HL, BC, IX = CON:s FCB. Ut: HL och BC efter det som är behandlat. Förstör A, DE och F.

```
01D6  FE 07       SCRCHR: CP   $07
01D8  28 50               JR   Z,BEEP          7: ljud
01DA  FE 08               CP   $08
01DC  28 79               JR   Z,CURLFT        8: vänster
01DE  FE 0A               CP   $0A
01E0  CA 7B 02            JP   Z,LINFD         $0A: ny rad
01E3  FE 0D               CP   $0D
01E5  28 4E               JR   Z,CR            $0D: kolumn 0
01E7  FE 0C               CP   $0C
01E9  CA B2 02            JP   Z,CLS           $0C: töm skärmen
01EC  FE 1B               CP   $1B
01EE  28 4A               JR   Z,ESCPOS        ESC
01F0  FE 20               CP   $20
01F2  D8                  RET  C               andra styrtecken ignoreras
01F3  CD D2 02            CALL CURADR          DE = skärmadress
01F6  03                  INC  BC              BC och HL tillbaka ett tecken
01F7  2B                  DEC  HL
01F8  00                  NOP                  tomrum till $0214
01F9  00                  NOP
01FA  00                  NOP
01FB  00                  NOP
01FC  00                  NOP
01FD  00                  NOP
01FE  00                  NOP
01FF  00                  NOP
0200  00                  NOP
0201  00                  NOP
0202  00                  NOP
0203  00                  NOP
0204  00                  NOP
0205  00                  NOP
0206  00                  NOP
0207  00                  NOP
0208  00                  NOP
0209  00                  NOP
020A  00                  NOP
020B  00                  NOP
020C  00                  NOP
020D  00                  NOP
020E  00                  NOP
020F  00                  NOP
0210  00                  NOP
0211  00                  NOP
0212  00                  NOP
0213  00                  NOP
0214  00                  NOP
```

SCRPUT: skriv vanliga tecken från HL i följd i bildminnet från DE, tills ett styrtecken kommer, alla BC tecken är skrivna eller raden är full (FCB+6, kolumnen, når FCB+8). SCRCHR kommer hit genom tomrummet ovanför, med HL och BC ett tecken tillbaka. In: HL = texten, BC = antalet, DE = skärmadressen, IX = FCB:n. Ut: HL vid första tecknet som inte är skrivet, BC = antalet kvar, FCB+6 = ny kolumn. Förstör A och DE.

```
0215  7E          SCRPUT: LD   A,(HL)          nästa tecken
0216  FE 20               CP   $20
0218  D8                  RET  C               styrtecken: tillbaka
0219  DD 34 06            INC  (IX+$06)        kolumnen + 1
021C  12                  LD   (DE),A          till bildminnet
021D  13                  INC  DE
021E  ED A1               CPI                  HL++, BC–, PO när BC = 0
0220  E0                  RET  PO              alla tecken skrivna
0221  DD 7E 06            LD   A,(IX+$06)      kolumnen
```

```
0224  DD BE 08          CP   (IX+$08)        radens slut?
0227  38 EC             JR   C,SCRPUT        plats kvar på raden
0229  C9                RET                  raden full
```

BEEP: OUT 6,0 och sedan OUT 6,131, samma pip som ABC80-BASIC:en ger för BEL. Förstör A.

```
022A  AF          BEEP:  XOR  A
022B  D3 06              OUT  ($06),A         ljudet av
022D  3E 83              LD   A,$83
022F  D3 06              OUT  ($06),A         pip
0231  C9                RET

0232  00 00 00           DEFB $00,$00,$00
0235  AF          CR:    XOR  A               kolumn 0
0236  DD 77 06    L0236: LD   (IX+$06),A      ny kolumn
0239  C9                RET
```

ESC = r k: kräver minst 3 tecken kvar. Raden begränsas till 0-23 och kolumnen till 0..radlängd-1.

```
023A  7E          ESCPOS: LD   A,(HL)
023B  FE 3D               CP   $3D
023D  C0                  RET  NZ
023E  EB                  EX   DE,HL
023F  21 FD FF            LD   HL,$FFFD       BC >= 3?
0242  09                  ADD  HL,BC
0243  EB                  EX   DE,HL
0244  D0                  RET  NC
0245  23                  INC  HL
0246  0B                  DEC  BC
0247  1E 18               LD   E,$18          raden < 24
0249  CD 6B 02            CALL ESCARG         IX+7 = raden
024C  DD 77 07            LD   (IX+$07),A
024F  DD 5E 08            LD   E,(IX+$08)     kolumnen < radlängden
0252  CD 6B 02            CALL ESCARG
0255  18 DF               JR   L0236          kolumnen
```

CURLFT: markören ett steg vänster. Från kolumn 0 till sista kolumnen på raden ovanför, från rad 0 till rad 23 (ingen rullning). Förstör A och F.

```
0257  DD 35 06    CURLFT: DEC  (IX+$06)
025A  F0                  RET  P              kolumnen >= 0
025B  DD 7E 08            LD   A,(IX+$08)
025E  3D                  DEC  A
025F  DD 77 06            LD   (IX+$06),A
0262  DD 35 07            DEC  (IX+$07)
0265  F0                  RET  P
0266  DD 36 07 17         LD   (IX+$07),$17   från rad 0 till rad 23
026A  C9                  RET
```

ESCARG: nästa tecken - $20, begränsat till 0..E-1 (under $20 blir 0). HL ett steg fram, BC ett mindre. Ut: A. Förstör F.

```
026B  7E          ESCARG: LD   A,(HL)
026C  23                  INC  HL
026D  0B                  DEC  BC
026E  D6 20               SUB  $20
0270  30 01               JR   NC,L0273       under $20: 0
0272  AF                  XOR  A
0273  BB          L0273:  CP   E
0274  D8                  RET  C
0275  7B                  LD   A,E            för stort: gränsen - 1
0276  3D                  DEC  A
0277  C9                  RET
```

NEWLIN: kolumn 0 och ny rad (CR, sedan LINFD).

```
0278  CD 35 02    NEWLIN: CALL CR
```

LINFD: markören en rad ned; på sista raden rullas skärmen upp med hjälp av ROWTAB och sista raden töms. Förstör A, DE och

F; SCROLL bevarar HL, BC och IX.

```
027B  3A 53 FF     LINFD:  LD   A,(CURROW)
027E  3C                   INC  A
027F  FE 18                CP   $18              sista raden?
0281  30 04                JR   NC,SCROLL
0283  32 53 FF             LD   (CURROW),A
0286  C9                   RET

0287  E5           SCROLL: PUSH HL
0288  C5                   PUSH BC
0289  DD E5                PUSH IX
028B  DD 21 E6 03          LD   IX,$03E6         ROWTAB
028F  3E 18                LD   A,$18            24 rader
0291  01 28 00     L0291:  LD   BC,$0028         40 tecken
0294  DD 5E 00             LD   E,(IX+$00)       DE = rad n
0297  DD 56 01             LD   D,(IX+$01)
029A  3D                   DEC  A
029B  28 0E                JR   Z,L02AB          sista: töm den
029D  DD 6E 02             LD   L,(IX+$02)       HL = rad n+1
02A0  DD 66 03             LD   H,(IX+$03)
02A3  ED B0                LDIR                  rad n+1 -> rad n
02A5  DD 23                INC  IX               nästa post
02A7  DD 23                INC  IX
02A9  18 E6                JR   L0291

02AB  DD E1        L02AB:  POP  IX
02AD  C3 C0 02             JP   BLANK            BC = 40, DE = sista raden

02B0  00 00                DEFB $00,$00
```

CLS: markören till 0,0 och $7C00-$7FFF fylls med blanktecken. Bevarar HL och BC; förstör DE och F.

```
02B2  E5           CLS:    PUSH HL
02B3  C5                   PUSH BC
02B4  21 00 00             LD   HL,$0000
02B7  22 52 FF             LD   (CURCOL),HL      kolumn och rad = 0
02BA  11 00 7C             LD   DE,$7C00
02BD  01 00 04             LD   BC,$0400         hela bildminnet, även de osynliga byten
```

BLANK: fyll BC byte från DE med blanktecken och ta sedan BC och HL från stacken (anroparen har lagt dem där).

```
02C0  62           BLANK:  LD   H,D
02C1  6B                   LD   L,E
02C2  36 20                LD   (HL),$20         första byten
02C4  13                   INC  DE
02C5  0B                   DEC  BC
02C6  ED B0                LDIR                  resten: kopian går framåt
02C8  C1                   POP  BC
02C9  E1                   POP  HL
02CA  C9                   RET

02CB  00 00 00 00…         DEFB $00,$00,$00,$00,$00,$00,$00
```

CURADR: DE = skärmadressen för markören. Är kolumnen >= radlängden görs först ny rad. In: IX = CON:s FCB. Förstör A och F.

```
02D2  3A 52 FF     CURADR: LD   A,(CURCOL)
02D5  DD BE 08             CP   (IX+$08)
02D8  D4 78 02             CALL NC,NEWLIN        ny rad först nu
02DB  E5                   PUSH HL
02DC  2A 52 FF             LD   HL,(CURCOL)      L = kolumnen, H = raden
02DF  7D                   LD   A,L
02E0  6C                   LD   L,H
02E1  26 00                LD   H,$00
02E3  29                   ADD  HL,HL            ROWTAB + 2 x rad
02E4  11 E6 03             LD   DE,$03E6
02E7  19                   ADD  HL,DE
02E8  86                   ADD  A,(HL)           + kolumnen (ryms: högst $D0 + 39)
02E9  23                   INC  HL
02EA  66                   LD   H,(HL)
02EB  6F                   LD   L,A
```

```
02EC  EB                  EX   DE,HL
02ED  E1                  POP  HL
02EE  C9                  RET

02EF  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
02F7  00 00 00            DEFB $00,$00,$00
```

3.7 CONINP: radinmatning från tangentbordet $02FA

HL = buffert, C = maxlängd. DE = mallrad för TAB (CONINP tar $0026, dvs. ingen mall). Tangenter:

```
3 (BREAK) eller CR  avslutar, CR läggs sist i bufferten
$18 eller $0C       raderar hela raden
8                   raderar senaste tecknet (RUBOUT)
9 (TAB)             kopierar nästa tecken från mallen
< $20               ignoreras
>= $80              fel 53 ($35)
```

Tecknen ekas med SCRCHR. Bufferten rymmer C-1 tecken och CR: det C:te tecknet ekas och raderas igen. $0C ekas efter raderingen och tömmer då hela skärmen. Ett skrivet tecken läggs in utan att mallen flyttas, och TAB stannar vid mallens CR. BREAK avslutar som CR, men KEYINT har satt BREAK-biten, så kommandoslingan kör STOP och raden utförs inte. Ut: HL vid CR:en i bufferten, CY och A = 53 vid ett tecken >= $80. Bevarar C och IX; förstör A, B och DE.

```
02FA  11 26 00     CONINP: LD   DE,$0026        ingen mall (CR LF)
02FD  ED 53 E0 FF  CONIN2: LD   (TABPTR),DE
0301  DD E5                PUSH IX              IX sparas
0303  DD 21 4C FF          LD   IX,$FF4C        CON:s FCB
0307  41                   LD   B,C             B = platser kvar
0308  CD 5E 03     L0308:  CALL RUBOUT          8: radera ett tecken (tom rad: inget)
030B  CD 90 FF     L030B:  CALL GETKYV
030E  FE 03                CP   $03
0310  28 47                JR   Z,L0359         CTRL-C
0312  FE 0D                CP   $0D
0314  28 43                JR   Z,L0359
0316  FE 18                CP   $18
0318  28 32                JR   Z,L034C         CTRL-X
031A  FE 0C                CP   $0C
031C  28 2E                JR   Z,L034C         CTRL-L
031E  FE 08                CP   $08
0320  28 E6                JR   Z,L0308         vänsterpil
0322  FE 09                CP   $09
0324  20 0E                JR   NZ,L0334
0326  ED 5B E0 FF          LD   DE,(TABPTR)     9: nästa tecken ur mallen
032A  1A                   LD   A,(DE)
032B  FE 0D                CP   $0D             mallens slut
032D  28 DC                JR   Z,L030B
032F  13                   INC  DE              mallen ett steg
0330  ED 53 E0 FF          LD   (TABPTR),DE
0334  FE 20        L0334:  CP   $20
0336  38 D3                JR   C,L030B         andra styrtecken ignoreras
0338  FE 80                CP   $80
033A  30 1A                JR   NC,L0356
033C  77                   LD   (HL),A          in i bufferten
033D  23                   INC  HL
033E  C5            L033E: PUSH BC
033F  D5                   PUSH DE
0340  01 00 00             LD   BC,$0000        BC = 0: bara detta tecken
0343  CD D6 01             CALL SCRCHR          eka
0346  D1                   POP  DE
0347  C1                   POP  BC
0348  10 C1                DJNZ L030B           B = platser kvar
034A  18 BC                JR   L0308           full: radera tecknet igen

034C  F5            L034C: PUSH AF
034D  CD 5E 03      L034D: CALL RUBOUT
0350  20 FB                JR   NZ,L034D        tills raden är tom
0352  F1                   POP  AF
0353  04                   INC  B               $0C ekas: töm skärmen
```

```
0354  18 E8              JR   L033E

0356  3E 35       L0356:  LD   A,$35
0358  37                  SCF
0359  36 0D       L0359:  LD   (HL),$0D
035B  DD E1       L035B:  POP  IX
035D  C9                  RET
```

RUBOUT: radera senaste tecknet. Markören ett steg vänster, blankt på skärmen, HL ett tillbaka och B + 1. Z och inget gjort om raden är tom (B = C), annars NZ. Förstör A och DE.

```
035E  78          RUBOUT: LD   A,B
035F  B9                  CP   C
0360  C8                  RET  Z               raden tom
0361  CD 57 02            CALL CURLFT
0364  2B                  DEC  HL
0365  CD D2 02            CALL CURADR
0368  3E 20                LD   A,$20
036A  12                  LD   (DE),A          blankt på skärmen
036B  04                  INC  B
036C  C9                  RET
```

CONGET (GET): BC tangenter till bufferten HL, utan eko. Ut: HL efter tecknen, BC = 0. Förstör A och F.

```
036D  78          CONGET: LD   A,B
036E  B1                  OR   C
036F  C8                  RET  Z
0370  CD 90 FF            CALL GETKYV
0373  77                  LD   (HL),A
0374  23                  INC  HL
0375  0B                  DEC  BC
0376  18 F5                JR   CONGET
```

3.8 KEYWT: vänta på tangent (GETKYV pekar hit) $0378

Markören visas som inverterat tecken (bit 7). Väntar tills KEYINT har satt bit 7 i KEYFLG, eller tills en nedtryckt tangent (IN $38 bit 7) har hållits tillräckligt länge: autorepetition. Den inre räknaren ($FFEB) går 256 varv för varje steg i den yttre ($FFEC), som börjar på 70 när tangenten är släppt och sätts till 8 efter varje repetition. A = tangentkoden (bit 0-6). Bevarar BC, DE, HL och IX.

```
0378  DD E5        KEYWT:  PUSH IX
037A  DD 21 4C FF         LD   IX,$FF4C
037E  D5                  PUSH DE              DE sparas
037F  CD D2 02            CALL CURADR          DE = markörens plats
0382  DD 21 E2 FF         LD   IX,$FFE2        IX = KEYFLG
0386  E5                  PUSH HL
0387  C5                  PUSH BC
0388  EB                  EX   DE,HL
0389  CB FE               SET  7,(HL)          visa markören
038B  FB                  EI                   tangentavbrotten på
038C  DD CB 00 7E L038C:  BIT  7,(IX+$00)
0390  20 1B               JR   NZ,L03AD        tangent via avbrottet
0392  DB 38               IN   A,($38)         bit 7: en tangent är nere
0394  87                  ADD  A,A
0395  38 06               JR   C,L039D         tangenten hålls nere: räkna
0397  DD 36 0A 46         LD   (IX+$0A),$46    släppt: fördröjningen börjar om
039B  18 EF               JR   L038C

039D  DD 35 09    L039D:  DEC  (IX+$09)        inre räknaren ($FFEB), 256 varv
03A0  20 EA               JR   NZ,L038C
03A2  DD 35 0A            DEC  (IX+$0A)        yttre räknaren ($FFEC)
03A5  20 E5               JR   NZ,L038C
03A7  DD 36 0A 08         LD   (IX+$0A),$08    sedan snabbare
03AB  18 04               JR   L03B1

03AD  DD 36 00 00 L03AD:  LD   (IX+$00),$00    KEYFLG nollställs
03B1  DB 38       L03B1:  IN   A,($38)         tangentkoden
03B3  E6 7F               AND  $7F
```

```
03B5  CB BE              RES  7,(HL)          ta bort markören
03B7  EB                 EX   DE,HL
03B8  C1                 POP  BC
03B9  E1                 POP  HL
03BA  D1                 POP  DE
03BB  DD E1              POP  IX
03BD  C9                 RET

03BE  00 00 00           DEFB $00,$00,$00
```

3.9 KEYINT: avbrott från PIO A (vektor $B4) $03C1

Koden sparas i LASTKY. Kod 3 sätter bit 0 i byten som BRKPTR pekar på (BREAK). KEYFLG får bit 7 satt. Bevarar alla register.

```
03C1  F5           KEYINT: PUSH AF
03C2  E5                   PUSH HL
03C3  DB 38                IN   A,($38)       tangentkoden
03C5  E6 7F                AND  $7F
03C7  32 E3 FF             LD   (LASTKY),A
03CA  FE 03                CP   $03           CTRL-C?
03CC  20 09                JR   NZ,L03D7
03CE  2A 85 FF             LD   HL,(BRKPTR)
03D1  7D                   LD   A,L
03D2  B4                   OR   H
03D3  28 02                JR   Z,L03D7       ingen BREAK-byte
03D5  CB C6                SET  0,(HL)        BREAK
03D7  CD 87 01     L03D7:  CALL KEYSET        KEYFLG = $80
03DA  E1                   POP  HL
03DB  F1           L03DB:  POP  AF
03DC  FB                   EI
03DD  ED 4D        DORETI: RETI
```

PIOINT: *avbrott med vektor* $B6*: läser porten och ignorerar. En rest från ABC 800, där* $B6 *är DART:ens avbrott för mottagningsfel på tangentbordets kanal; på ABC80 ger inget avbrott vektorn* $B6.

```
03DF  F5           PIOINT: PUSH AF
03E0  DB 38                IN   A,($38)
03E2  E6 7F                AND  $7F
03E4  18 F5                JR   L03DB
```

ROWTAB: *skärmradernas adresser. ABC80:s bildminne är sammanflätat: rad 0-7 på* $7C00+$80**n*, *rad 8-15 +*$28, *rad 16-23 +*$50.

```
03E6  00 7C        ROWTAB: DEFW $7C00
03E8  80 7C                DEFW $7C80
03EA  00 7D                DEFW $7D00
03EC  80 7D                DEFW $7D80
03EE  00 7E                DEFW $7E00
03F0  80 7E                DEFW $7E80
03F2  00 7F                DEFW $7F00
03F4  80 7F                DEFW $7F80
03F6  28 7C                DEFW $7C28
03F8  A8 7C                DEFW $7CA8
03FA  28 7D                DEFW $7D28
03FC  A8 7D                DEFW $7DA8
03FE  28 7E                DEFW $7E28
0400  A8 7E                DEFW $7EA8
0402  28 7F                DEFW $7F28
0404  A8 7F                DEFW $7FA8
0406  50 7C                DEFW $7C50
0408  D0 7C                DEFW $7CD0
040A  50 7D                DEFW $7D50
040C  D0 7D                DEFW $7DD0
040E  50 7E                DEFW $7E50
0410  D0 7E                DEFW $7ED0
0412  50 7F                DEFW $7F50
0414  D0 7F                DEFW $7FD0
```

KBINIT: PIO A: vektor `$B4`*, avbrott när bit 7 blir 1* (`$B7`*, mask* `$7F`)*, läge 3 med alla bitar som ingångar. Ljudet stängs av* (*OUT* (`$06`)*,0*)*. Förstör A.*

```
0416  3E B4        KBINIT: LD   A,$B4
0418  D3 39                OUT  ($39),A          vektor $B4
041A  3E B7                LD   A,$B7            avbrott, OR, hög nivå, mask följer
041C  D3 39                OUT  ($39),A
041E  3E 7F                LD   A,$7F            bara bit 7 bevakas
0420  D3 39                OUT  ($39),A
0422  3E FF                LD   A,$FF            läge 3
0424  D3 39                OUT  ($39),A
0426  D3 39                OUT  ($39),A          alla bitar in
0428  3C                   INC  A                A = 0
0429  D3 06                OUT  ($06),A
042B  C9                   RET

042C  00 00 00 00…         DEFB $00,$00,$00,$00,$00
```

Starttext med ESC = (*rad 10, kolumn 10*) *och tre BEL. Ingen kod i ROM:en refererar till den, och den visas inte vid start* (*bara ABC800*)*.*

```
0431  0C 1B 3D 2A…         DEFB $0C,$1B,"=*** BASICII 80/40 *",$07,$07,$07
044A  00                   DEFB $00
044B  00                   DEFB $00
```

Tomt område (*nollor*) *till* `$06A9`*. I ABC800 ligger här kassetten CAS:* (*drivrutin* `$044A`*, enhetsposter* `$0670` *och* `$0677`).

```
044C  00 00 00 00…         DEFB $00,$00,$00,$00,$00,$00,$00,$00
```

$0454–$06A9: bara $00 (*598 bytes*)*, utelämnas här.*

3.10 Nyckelordstabeller $06AA

Varje nyckelord är en kodbyte (>= `$80`) följd av namnet. `$FF` avslutar en tabell; `KWSRCH` (`$4B4B`) söker i en tabell med DE = början. Står `$00` efter ett namn godtas det utan kontroll av nästa tecken.

`OPGRP` (`$06AA`, IX i `$37B0`): grupper av binära operatorer, 3 bytes per grupp: pekare till gruppens nyckelord och klass (se `CEXPR`; bit 7 = tecken eller `NOT` får stå först). Lägst prioritet först; `CEXLVL` ökar IX innan den läser, så första gruppen är `$06AD`, och klass 0 (`$06C5`) ger en faktor.

```
06AA  00 00 00 C8… OPGRP:  DEFB $00,$00,$00,$C8,$06,$03,$CD,$06
06B2  03 D2 06 03…         DEFB $03,$D2,$06,$03,$DA,$06,$03,$E4
06BA  06 82 F4 06…         DEFB $06,$82,$F4,$06,$81,$F9,$06,$01
06C2  FE 06 04 00…         DEFB $FE,$06,$04,$00,$00,$00
06C8  84 45 51 56  OPKW:   DEFB $84,"EQV"
06CC  FF                   DEFB $FF
06CD  85 49 4D 50          DEFB $85,"IMP"
06D1  FF                   DEFB $FF
06D2  86 4F 52             DEFB $86,"OR"
06D5  87 58 4F 52          DEFB $87,"XOR"
06D9  FF                   DEFB $FF
06DA  88 41 4E 44          DEFB $88,"AND"
06DE  FF                   DEFB $FF
06DF  AF 4E 4F 54          DEFB $AF,"NOT"
06E3  FF                   DEFB $FF
06E4  98 3C 3D             DEFB $98,"<="
06E7  8C 3C 3E             DEFB $8C,"<>"
06EA  8F 3C                DEFB $8F,"<"
06EC  92 3E 3D             DEFB $92,">="
06EF  95 3E                DEFB $95,">"
06F1  89 3D                DEFB $89,"="
06F3  FF                   DEFB $FF
06F4  9B 2B                DEFB $9B,"+"
06F6  9E 2D                DEFB $9E,"-"
06F8  FF                   DEFB $FF
06F9  A0 2A                DEFB $A0,"*"
06FB  A2 2F                DEFB $A2,"/"
```

```
06FD  FF                  DEFB $FF
06FE  A4 5E               DEFB $A4,"^"
0700  A4 2A 2A            DEFB $A4,"**"
0703  FF                  DEFB $FF
```

3.11 Tabellhuvuden (10 bytes) för funktioner (FNKWP) och satser (STKWP) $0704

```
+0,+1  nästa huvud (0 = sista)
+2     första koden, +3 antal koder
+4,+5  rutintabell (0 = koderna utförs av ROM:en via $0ABF)
+6,+7  nyckelordslistan
+8,+9  funktioner: signaturtabellen (FNARG); hög byte 0 = alla
       koderna har signaturen för koden i låga byten (färgorden:
       $0080, sträng utan argument). Satser: kompileringens
       hopptabell för CXSTM ($0B0F; 0 = ingen, se $0938).
```

Huvudena bildar en kedja från RAM-pekaren, så att ett tilläggsprogram kan lägga in egna nyckelord först (se $6CA0). FNDTAB ($1DEB) letar upp huvudet där kod - första < antal.

```
$0704: färgord $80-$9F -> $070E: funktioner $00-$35
$092E: satser (grupp 2) $00-$17 -> $0938: satser $00-$24
```

```
0704  0E 07        FNHDR:  DEFW $070E
0706  80 20                DEFW $2080
0708  00 00                DEFW $0000
070A  B7 08                DEFW $08B7
070C  80 00                DEFW $0080
070E  00 00                DEFW $0000
0710  00 36                DEFW $3600
0712  00 00                DEFW $0000
0714  18 07                DEFW $0718
0716  41 08                DEFW $0841
0718  FE 58 46 4E  FNKW:   DEFB $FE,"XFN"
071C  00 9A 41 42…         DEFB $00,$9A,"ABS"
0721  86 41 54 4E          DEFB $86,"ATN"
0725  84 43 4F 53          DEFB $84,"COS"
0729  A3 45 58 50          DEFB $A3,"EXP"
072D  81 46 49 58          DEFB $81,"FIX"
0731  82 49 4E 54          DEFB $82,"INT"
0735  A5 4C 4F 47…         DEFB $A5,"LOG10"
073B  A4 4C 4F 47          DEFB $A4,"LOG"
073F  88 50 49             DEFB $88,"PI"
0742  89 52 4E 44          DEFB $89,"RND"
0746  8A 53 47 4E          DEFB $8A,"SGN"
074A  83 53 49 4E          DEFB $83,"SIN"
074E  87 53 51 52          DEFB $87,"SQR"
0752  85 54 41 4E          DEFB $85,"TAN"
0756  8B 41 53 43…         DEFB $8B,"ASCII"
075C  8B 41 53 43          DEFB $8B,"ASC"
0760  8C 43 48 52…         DEFB $8C,"CHR$"
0765  8D 4C 45 46…         DEFB $8D,"LEFT$"
076B  8D 4C 45 46…         DEFB $8D,"LEFT"
0770  8E 52 49 47…         DEFB $8E,"RIGHT$"
0777  8E 52 49 47…         DEFB $8E,"RIGHT"
077D  8F 4D 49 44…         DEFB $8F,"MID$"
0782  8F 4D 49 44          DEFB $8F,"MID"
0786  90 4C 45 4E          DEFB $90,"LEN"
078A  91 49 4E 53…         DEFB $91,"INSTR"
0790  92 53 50 41…         DEFB $92,"SPACE$"
0797  93 53 54 52…         DEFB $93,"STRING$"
079F  9C 4E 55 4D…         DEFB $9C,"NUM$"
07A4  A7 56 41 4C          DEFB $A7,"VAL"
07A8  94 53 57 41…         DEFB $94,"SWAP%"
07AE  AB 56 41 52…         DEFB $AB,"VARPTR"
07B5  AC 56 41 52…         DEFB $AC,"VAROOT"
07BC  AD 50 45 45…         DEFB $AD,"PEEK2"
07C2  AE 4D 4F 44          DEFB $AE,"MOD"
07C6  AF 43 56 54…         DEFB $AF,"CVT%$"
07CC  B0 43 56 54…         DEFB $B0,"CVT$%"
```

```
07D2  B1 43 56 54…          DEFB $B1,"CVTF$"
07D8  B2 43 56 54…          DEFB $B2,"CVT$F"
07DE  B3 48 45 58…          DEFB $B3,"HEX$"
07E3  B4 4F 43 54…          DEFB $B4,"OCT$"
07E8  95 50 45 45…          DEFB $95,"PEEK"
07ED  96 49 4E 50           DEFB $96,"INP"
07F1  97 43 41 4C…          DEFB $97,"CALL"
07F6  98 45 52 52…          DEFB $98,"ERRCODE"
07FE  9D 54 41 42           DEFB $9D,"TAB"
0802  9F 43 55 52           DEFB $9F,"CUR"
0806  A1 54 58 50…          DEFB $A1,"TXPOINT"
080E  9B 41 44 44…          DEFB $9B,"ADD$"
0813  9E 53 55 42…          DEFB $9E,"SUB$"
0818  A0 4D 55 4C…          DEFB $A0,"MUL$"
081D  A2 44 49 56…          DEFB $A2,"DIV$"
0822  A6 43 4F 4D…          DEFB $A6,"COMP%"
0828  99 53 59 53           DEFB $99,"SYS"
082C  A9 54 49 4D…          DEFB $A9,"TIME$"
0832  AA 50 4F 53…          DEFB $AA,"POSIT"
0838  B5 44 4F 54           DEFB $B5,"DOT"
083C  80 46 4E              DEFB $80,"FN"
083F  00 FF                 DEFB $00,$FF
```

FNARG: *funktionernas argumentsignaturer (+8 i huvudet $070E). Funktionskoder (bit 7 satt) följda av signaturen: en byte för resultatet (bit 0-1 typen, bit 5 = får sakna argument) och en per argument (se CARGS $188D); flera koder i följd delar signatur. Sist* $FF *och de tre signaturerna för variabelindex* ($08B4), DIM ($08B5) *och* FN ($08B6).

Kod $80 *(med* TIME$, $22*: sträng utan argument) är färgordens signatur (huvudet $0704, +8 =* $0080*);* FN *själv har kod 0 och egen väg.*

```
0841  9A 86 84 A3… FNARG:  DEFB $9A,$86,$84,$A3,$81,$82,$A4,$A5
0849  83 87 85 00…          DEFB $83,$87,$85,$00,$20,$88,$20,$89
0851  20 20 8A 01…          DEFB $20,$20,$8A,$01,$20,$8B,$90,$B0
0859  01 22 8C 02…          DEFB $01,$22,$8C,$02,$29,$8D,$8E,$02
0861  02 21 8F 02…          DEFB $02,$21,$8F,$02,$02,$01,$21,$91
0869  01 01 02 22…          DEFB $01,$01,$02,$22,$9D,$92,$AF,$B3
0871  B4 02 21 9F…          DEFB $B4,$02,$21,$9F,$93,$02,$01,$21
0879  9C B1 02 20…          DEFB $9C,$B1,$02,$20,$A7,$B2,$00,$22
0881  95 AD 94 96…          DEFB $95,$AD,$94,$96,$99,$01,$21,$AE
0889  01 01 21 98…          DEFB $01,$01,$21,$98,$21,$A1,$B5,$01
0891  01 21 A9 80…          DEFB $01,$21,$A9,$80,$22,$9B,$9E,$A0
0899  A2 02 06 02…          DEFB $A2,$02,$06,$02,$21,$A6,$05,$06
08A1  22 97 01 21…          DEFB $22,$97,$01,$21,$21,$AA,$00,$21
08A9  AB AC 01 70…          DEFB $AB,$AC,$01,$70,$A8,$A8,$02,$12
08B1  01 21 FF 29…          DEFB $01,$21,$FF,$29,$2D,$6C
08B7  81 52 45 44 COLKW:    DEFB $81,"RED"
08BB  82 47 52 4E           DEFB $82,"GRN"
08BF  83 59 45 4C           DEFB $83,"YEL"
08C3  84 42 4C 55           DEFB $84,"BLU"
08C7  85 4D 41 47           DEFB $85,"MAG"
08CB  86 43 59 41           DEFB $86,"CYA"
08CF  87 57 48 54           DEFB $87,"WHT"
08D3  88 46 4C 53…          DEFB $88,"FLSH"
08D8  89 53 54 44…          DEFB $89,"STDY"
08DD  8C 4E 52 4D…          DEFB $8C,"NRML"
08E2  8D 44 42 4C…          DEFB $8D,"DBLE"
08E7  91 47 52 45…          DEFB $91,"GRED"
08EC  92 47 47 52…          DEFB $92,"GGRN"
08F1  93 47 59 45…          DEFB $93,"GYEL"
08F6  94 47 42 4C…          DEFB $94,"GBLU"
08FB  95 47 4D 41…          DEFB $95,"GMAG"
0900  96 47 43 59…          DEFB $96,"GCYA"
0905  97 47 57 48…          DEFB $97,"GWHT"
090A  98 48 49 44…          DEFB $98,"HIDE"
090F  99 47 43 4F…          DEFB $99,"GCON"
0914  9A 47 53 45…          DEFB $9A,"GSEP"
0919  9C 42 4C 42…          DEFB $9C,"BLBG"
091E  9D 4E 57 42…          DEFB $9D,"NWBG"
0923  9E 47 48 4F…          DEFB $9E,"GHOL"
```

```
0928  9F 47 52 45…          DEFB $9F,"GREL"
092D  FF                    DEFB $FF
092E  38 09        STHDR:   DEFW $0938              huvud för satstabellen STKW
0930  00 18                 DEFW $1800
0932  00 00                 DEFW $0000
0934  21 0A                 DEFW $0A21
0936  0F 0B                 DEFW $0B0F
0938  00 00                 DEFW $0000(fel)
093A  00 25                 DEFW $2500
093C  00 00                 DEFW $0000
093E  42 09                 DEFW $0942
0940  00 00                 DEFW $0000
0942  86 58 53 54… STKW:    DEFB $86,"XSTM"          satsernas nyckelord
0947  00 91 4C 45…          DEFB $00,$91,"LET"
094C  92 45 4E 44           DEFB $92,"END"
0950  80 47 4F 54…          DEFB $80,"GOTO"
0955  93 50 52 49…          DEFB $93,"PRINT"
095B  94 3B                 DEFB $94,";"
095D  95 49 46              DEFB $95,"IF"
0960  8F 52 45 4D           DEFB $8F,"REM"
0964  90 21                 DEFB $90,"!"
0966  97 49 4E 50…          DEFB $97,"INPUT LINE"
0971  96 49 4E 50…          DEFB $96,"INPUT"
0977  98 46 4F 52           DEFB $98,"FOR"
097B  99 4E 45 58…          DEFB $99,"NEXT"
0980  9A 57 48 49…          DEFB $9A,"WHILE"
0986  9B 57 45 4E…          DEFB $9B,"WEND"
098B  9C 52 45 41…          DEFB $9C,"READ"
0990  83 52 45 53…          DEFB $83,"RESTORE"
0998  84 52 45 53…          DEFB $84,"RESUME"
099F  82 47 4F 53…          DEFB $82,"GOSUB"
09A5  A0 52 45 54…          DEFB $A0,"RETURN"
09AC  85 4F 4E 20…          DEFB $85,"ON ERROR GOTO"
09BA  A1 4F 4E              DEFB $A1,"ON"
09BD  8E 44 41 54…          DEFB $8E,"DATA"
09C2  8C 44 45 46…          DEFB $8C,"DEF FN"
09C9  00 8D 46 4E…          DEFB $00,$8D,"FNEND"
09D0  A2 4D 49 44…          DEFB $A2,"MID$"
09D5  A2 4D 49 44           DEFB $A2,"MID"
09D9  A3 53 49 4E…          DEFB $A3,"SINGLE"
09E0  A4 44 4F 55…          DEFB $A4,"DOUBLE"
09E7  FF                    DEFB $FF
```

Småtabeller som söks var för sig (*DE* = $09ED, $09F3 …)*:*

ELSE, STEP, TO, AS FILE, *COUNT*, USING, LOCAL, THEN. *Före dem tre koder som bara* LIST *letar upp* (CPIR *från* STKW *går förbi* $FF, *se* DLINE)*:* $81 *utan text* (GOTO *efter* THEN), $8B ? (*märket för en felrad, se* DOLINE) *och* $88 :.

```
09E8  81 8B 3F              DEFB $81,$8B,"?"
09EB  88 3A                 DEFB $88,":"
09ED  8A 45 4C 53…          DEFB $8A,"ELSE"
09F2  FF                    DEFB $FF
09F3  BE 53 54 45…          DEFB $BE,"STEP"
09F8  FF                    DEFB $FF
09F9  BD 54 4F              DEFB $BD,"TO"
09FC  FF                    DEFB $FF
09FD  BB 41 53 20…          DEFB $BB,"AS FILE"
0A05  FF                    DEFB $FF
0A06  B9 43 4F 55…          DEFB $B9,"COUNT"
0A0C  FF                    DEFB $FF
0A0D  B8 55 53 49…          DEFB $B8,"USING"
0A13  FF                    DEFB $FF
0A14  B6 4C 4F 43…          DEFB $B6,"LOCAL"
0A1A  FF                    DEFB $FF
0A1B  BD 54 48 45…          DEFB $BD,"THEN"
0A20  FF                    DEFB $FF
```

Andra satsgruppen (DIM .. *CLR DOT*)*, huvud* $092E.

```
0A21  80 44 49 4D  STKW2:  DEFB $80,"DIM"
0A25  81 43 4F 4D…         DEFB $81,"COMMON"
0A2C  82 50 4F 4B…         DEFB $82,"POKE"
0A31  83 4F 55 54          DEFB $83,"OUT"
0A35  84 4F 50 45…         DEFB $84,"OPEN"
0A3A  85 50 52 45…         DEFB $85,"PREPARE"
0A42  86 43 4C 4F…         DEFB $86,"CLOSE"
0A48  87 52 41 4E…         DEFB $87,"RANDOMIZE"
0A52  88 53 54 4F…         DEFB $88,"STOP"
0A57  89 54 58 50…         DEFB $89,"TXPOINT"
0A5F  8A 47 45 54          DEFB $8A,"GET"
0A63  8B 43 48 41…         DEFB $8B,"CHAIN"
0A69  8C 54 52 41…         DEFB $8C,"TRACE"
0A6F  8D 4E 4F 20…         DEFB $8D,"NO TRACE"
0A78  8E 50 55 54          DEFB $8E,"PUT"
0A7C  8F 49 4E 54…         DEFB $8F,"INTEGER"
0A84  90 46 4C 4F…         DEFB $90,"FLOAT"
0A8A  91 45 58 54…         DEFB $91,"EXTEND"
0A91  92 4E 4F 20…         DEFB $92,"NO EXTEND"
0A9B  93 44 49 47…         DEFB $93,"DIGITS"
0AA2  94 4F 50 54…         DEFB $94,"OPTION BASE"
0AAE  95 50 4F 53…         DEFB $95,"POSIT"
0AB4  96 53 45 54…         DEFB $96,"SET DOT"
0ABC  97 43 4C 52…         DEFB $97,"CLR DOT"
0AC4  FF                   DEFB $FF
```

3.12 JPTABL: den stora hopptabellen (204 ord) för kompilering och körning

$0AC5

Tabellen används med fyra olika baser, så att samma ord nås via olika koder:

index $00-$24 kompilering av satserna i grupp 1 (kod $80-$A4, index = kod & $7F; CMPSTM med BC = $0AC5)
index $25-$3C kompilering av grupp 2 (DIM .. CLR DOT, $0B0F = +8 i huvudet $092E, index = $25 + kod)
index $3D-$61 körning av satserna i grupp 1 (STMLP: $0A3F + 2*kod)
index $62-$79 körning av grupp 2, efter $86 (XSTM: $0B89 + 2*kod)
index $7A-$7C uttryckskoderna $7D-$7F (EXECA): krok HOOKA6, och fältelementets värde $7E+typ (adressen på stacken): $7E flyttal (FPUSH), $7F heltal (PEEK2)
index $7D-$CB uttryckskoder $80-$CE (EXECA: $0ABF + 2*kod)

Satser utan egen kompileringsrutin ($87-$8B, $9D-$9F) har 0.

```
0AC5  BC 19        JPTABL: DEFW CLNR
0AC7  BC 19                DEFW CLNR
0AC9  BC 19                DEFW CLNR
0ACB  7D 1A                DEFW CLNR0
0ACD  7D 1A                DEFW CLNR0
0ACF  7D 1A                DEFW CLNR0
0AD1  8A 1B                DEFW CXSTM
0AD3  00 00                DEFW RESET
0AD5  00 00                DEFW RESET
0AD7  00 00                DEFW RESET
0AD9  00 00                DEFW RESET
0ADB  00 00                DEFW RESET
0ADD  FD 1A                DEFW CDEF
0ADF  CC 19                DEFW NOHOOK
0AE1  70 1A                DEFW CDATA
0AE3  72 1A                DEFW CTEXT
0AE5  72 1A                DEFW CTEXT
0AE7  B7 16                DEFW CLET
0AE9  21 1A                DEFW CEND
0AEB  BA 1B                DEFW CPRINT
0AED  BA 1B                DEFW CPRINT
0AEF  D4 19                DEFW CIF
0AF1  16 1C                DEFW CINPUT
0AF3  7E 1C                DEFW CINLIN
```

```
0AF5  88 1A               DEFW CFOR
0AF7  BC 1A               DEFW CNEXT
0AF9  C6 1A               DEFW CWHILE
0AFB  BF 1A               DEFW CWEND
0AFD  67 1C               DEFW CREAD
0AFF  00 00               DEFW RESET
0B01  00 00               DEFW RESET
0B03  00 00               DEFW RESET
0B05  CD 19               DEFW CRETRN
0B07  33 1A               DEFW CON
0B09  93 16               DEFW CMID
0B0B  73 3A               DEFW SINGLE
0B0D  76 3A               DEFW DOUBLE
0B0F  E8 16               DEFW CDIM
0B11  EB 16               DEFW CCOMMN
0B13  DF 1A               DEFW CPOKE
0B15  E9 1A               DEFW COUT
0B17  BC 1C               DEFW COPEN
0B19  BC 1C               DEFW COPEN
0B1B  D7 1C               DEFW CCLOSE
0B1D  CC 19               DEFW NOHOOK
0B1F  CC 19               DEFW NOHOOK
0B21  F6 1A               DEFW CTXPNT
0B23  8B 1C               DEFW CGET
0B25  1B 00               DEFW CEXPS
0B27  A7 1C               DEFW CFILNR
0B29  CC 19               DEFW NOHOOK
0B2B  84 1C               DEFW CPUT
0B2D  59 3A               DEFW INTEGR
0B2F  5D 3A               DEFW FLOAT
0B31  69 3A               DEFW EXTEND
0B33  6D 3A               DEFW NOEXT
0B35  A4 16               DEFW CEXPI
0B37  A4 16               DEFW CEXPI
0B39  A0 1C               DEFW CPOSIT
0B3B  CB 1A               DEFW CXY
0B3D  CB 1A               DEFW CXY
0B3F  9D 1D               DEFW GOTO
0B41  9D 1D               DEFW GOTO
0B43  34 2C               DEFW GOSUB
0B45  C6 28               DEFW RESTOR
0B47  22 3C               DEFW RESUME
0B49  13 2C               DEFW ONERR
0B4B  71 1D               DEFW XSTM
0B4D  64 1D               DEFW EOL
0B4F  CC 19               DEFW NOHOOK
0B51  10 01               DEFW READY
0B53  8D 1D               DEFW SKIP
0B55  00 1E               DEFW ERR90
0B57  99 1D               DEFW DEF
0B59  E4 2B               DEFW FNEND
0B5B  8B 1D               DEFW SKIP2
0B5D  8D 1D               DEFW SKIP
0B5F  8D 1D               DEFW SKIP
0B61  A9 1D               DEFW EXECOD
0B63  6B 1D               DEFW ENDST
0B65  02 1E               DEFW PRINT
0B67  02 1E               DEFW PRINT
0B69  A9 1D               DEFW EXECOD
0B6B  02 1E               DEFW PRINT
0B6D  1D 39               DEFW INPLIN
0B6F  83 27               DEFW FOR
0B71  09 28               DEFW NEXT
0B73  A7 28               DEFW WHILE
0B75  9D 1D               DEFW GOTO
0B77  EE 28               DEFW READ
0B79  B7 28               DEFW RESTR0
0B7B  13 3C               DEFW RESUM0
0B7D  28 2C               DEFW ONERR0
```

```
0B7F  46 2C           DEFW RETURN
0B81  65 2C           DEFW ON
0B83  EC 2B           DEFW MID
0B85  CC 19           DEFW NOHOOK
0B87  CC 19           DEFW NOHOOK
0B89  A9 1D           DEFW EXECOD
0B8B  A9 1D           DEFW EXECOD
0B8D  2D 3A           DEFW POKE
0B8F  3F 3A           DEFW OUTST
0B91  B0 38           DEFW SOPEN
0B93  B2 38           DEFW PREPAR
0B95  FB 38           DEFW SCLOSE
0B97  37 35           DEFW RANDOM
0B99  7B 3B           DEFW STOP
0B9B  36 00           DEFW UNSDEF
0B9D  56 39           DEFW GET
0B9F  93 2C           DEFW CHAIN
0BA1  EA 2C           DEFW TRACE
0BA3  03 20           DEFW NOTRAC
0BA5  7E 39           DEFW PUT
0BA7  CC 19           DEFW NOHOOK
0BA9  CC 19           DEFW NOHOOK
0BAB  CC 19           DEFW NOHOOK
0BAD  CC 19           DEFW NOHOOK
0BAF  A8 3A           DEFW DIGITS
0BB1  AE 3A           DEFW OPTBAS
0BB3  9A 39           DEFW POSIT
0BB5  36 00           DEFW UNSDEF
0BB7  36 00           DEFW UNSDEF
0BB9  A6 FF           DEFW HOOKA6
0BBB  ED 22           DEFW FPUSH
0BBD  AA 22           DEFW PEEK2
0BBF  26 23           DEFW L2326
0BC1  62 21           DEFW FTOI
0BC3  70 21           DEFW ITOF
0BC5  82 44           DEFW STRNUM
0BC7  BE 21           DEFW IEQV
0BC9  CA 21           DEFW IIMP
0BCB  D6 21           DEFW IOR
0BCD  EE 21           DEFW IXOR
0BCF  E0 21           DEFW IAND
0BD1  19 21           DEFW FCMP
0BD3  F8 21           DEFW IEQ
0BD5  0A 26           DEFW SCMP
0BD7  19 21           DEFW FCMP
0BD9  03 22           DEFW INE
0BDB  0A 26           DEFW SCMP
0BDD  19 21           DEFW FCMP
0BDF  0F 22           DEFW ILT
0BE1  0A 26           DEFW SCMP
0BE3  19 21           DEFW FCMP
0BE5  1B 22           DEFW IGE
0BE7  0A 26           DEFW SCMP
0BE9  19 21           DEFW FCMP
0BEB  2B 22           DEFW IGT
0BED  0A 26           DEFW SCMP
0BEF  19 21           DEFW FCMP
0BF1  27 22           DEFW ILE
0BF3  0A 26           DEFW SCMP
0BF5  78 21           DEFW FARITH
0BF7  2F 22           DEFW IADD
0BF9  75 25           DEFW SCAT
0BFB  78 21           DEFW FARITH
0BFD  34 22           DEFW ISUB
0BFF  78 21           DEFW FARITH
0C01  3E 22           DEFW IMUL
0C03  78 21           DEFW FARITH
0C05  3D 22           DEFW IDIV
0C07  01 2E           DEFW FPOW
```

```
0C09  61 2D               DEFW FPOWI
0C0B  AB 2D               DEFW IPOW
0C0D  A9 1D               DEFW EXECOD
0C0F  FB 20               DEFW FNEG
0C11  B8 21               DEFW INEG
0C13  0A 2B               DEFW FNAMRK
0C15  0A 2B               DEFW FNAMRK
0C17  0A 2B               DEFW FNAMRK
0C19  61 21               DEFW FTOI2
0C1B  6F 21               DEFW ITOF2
0C1D  EA 21               DEFW INOT
0C1F  09 25               DEFW COLARG
0C21  D5 22               DEFW SFARR
0C23  A2 22               DEFW SIARR
0C25  16 23               DEFW SSARR
0C27  B0 22               DEFW STFVAR
0C29  98 22               DEFW STIVAR
0C2B  F4 22               DEFW STSVAR
0C2D  56 2B               DEFW FRETF
0C2F  84 2B               DEFW FRETI
0C31  99 2B               DEFW FRETS
0C33  C8 22               DEFW PTRF
0C35  CE 22               DEFW PTRI
0C37  CB 22               DEFW PTRS
0C39  92 1D               DEFW THEN
0C3B  92 1D               DEFW THEN
0C3D  CC 19               DEFW NOHOOK
0C3F  E5 22               DEFW LDFVAR
0C41  A6 22               DEFW LDIVAR
0C43  22 23               DEFW LDSVAR
0C45  2D 23               DEFW L232D
0C47  93 22               DEFW VARADR
0C49  93 22               DEFW VARADR
0C4B  93 22               DEFW VARADR
0C4D  B0 21               DEFW INTCON
0C4F  32 23               DEFW ARRELM
0C51  32 23               DEFW ARRELM
0C53  32 23               DEFW ARRELM
0C55  10 25               DEFW STRCON
0C57  10 25               DEFW STRCON
0C59  AC 24               DEFW XDIM
0C5B  AD 24               DEFW XCOMMN
```

3.13 FNTAB: hopptabell (101 ord) för funktioner och uttryckspref ix $0C5D

Index 0-$35: funktionerna, index = funktionens kod - $80 (FNKW). En funktion kompileras till $D0+n (n = antal argument, 0-3) eller $D4 $D0+n (n >= 4) och sedan index. EXECA hoppar över $D4/$D0+n och tar index härifrån.

Index $36-: rutiner som nås med andra baser, t.ex. FNFLT2 ($0CC9 = tabell + $6C): för SIN, COS, TAN, ATN, SQR, EXP, LOG, LOG10 och NUM$ gör FNFLT först det gemensamma och hoppar sedan till index + $36. FCMP/FDISP använder basen $0BC7 = JPTABL + $102: jämförelsens kod $89.. ger resultatrutinerna $2139-$215C, och +, -, *, / (kod $9B, $9E, $A0, $A2) ger FADD, FSUB, FMUL, FDIV.

```
0C5D  1B 2A       FNTAB:  DEFW FNCALL
0C5F  02 2D               DEFW FIX
0C61  06 2D               DEFW INT
0C63  85 21               DEFW FNFLT
0C65  85 21               DEFW FNFLT
0C67  85 21               DEFW FNFLT
0C69  85 21               DEFW FNFLT
0C6B  85 21               DEFW FNFLT
0C6D  E9 22               DEFW PI
0C6F  B1 34               DEFW RND
0C71  02 21               DEFW SGN
0C73  23 25               DEFW ASC
0C75  E2 25               DEFW CHR
0C77  3C 25               DEFW LEFT
```

```
0C79  54 25               DEFW RIGHT
0C7B  32 25               DEFW MIDF
0C7D  1C 25               DEFW LEN
0C7F  6D 26               DEFW INSTR
0C81  D4 26               DEFW SPACE
0C83  D9 26               DEFW STRNG
0C85  48 22               DEFW SWAP
0C87  A6 21               DEFW PEEK
0C89  0C 3A               DEFW INP
0C8B  18 3A               DEFW FCALL
0C8D  A2 21               DEFW ERRCOD
0C8F  BF 44               DEFW SYS
0C91  F6 20               DEFW ABS
0C93  F3 43               DEFW FNBCD
0C95  85 21               DEFW FNFLT
0C97  B7 1F               DEFW TAB
0C99  F3 43               DEFW FNBCD
0C9B  15 27               DEFW CUR
0C9D  F3 43               DEFW FNBCD
0C9F  36 00               DEFW UNSDEF
0CA1  F3 43               DEFW FNBCD
0CA3  85 21               DEFW FNFLT
0CA5  85 21               DEFW FNFLT
0CA7  85 21               DEFW FNFLT
0CA9  FC 43               DEFW FNSTR1
0CAB  FC 43               DEFW FNSTR1
0CAD  32 25               DEFW MIDF
0CAF  1F 3B               DEFW CLKSTR
0CB1  E6 39               DEFW FPOSIT
0CB3  F9 36               DEFW VARPTR
0CB5  06 37               DEFW VAROOT
0CB7  AA 22               DEFW PEEK2
0CB9  3A 22               DEFW MOD
0CBB  44 37               DEFW CVTIS
0CBD  1F 37               DEFW CVTSI
0CBF  40 37               DEFW CVTFS
0CC1  1B 37               DEFW CVTSF
0CC3  4F 37               DEFW HEX
0CC5  52 37               DEFW OCT
0CC7  36 00               DEFW UNSDEF
0CC9  F3 4C               DEFW FTOIR
0CCB  C0 4C               DEFW ITOFS
0CCD  00 00               DEFW RESET
0CCF  80 56               DEFW SIN
0CD1  84 56               DEFW COS
0CD3  88 56               DEFW TAN
0CD5  50 5B               DEFW ATN
0CD7  89 5A               DEFW SQR
0CD9  39 21               DEFW L2139
0CDB  39 21               DEFW L2139
0CDD  39 21               DEFW L2139
0CDF  41 21               DEFW L2141
0CE1  41 21               DEFW L2141
0CE3  41 21               DEFW L2141
0CE5  57 21               DEFW L2157
0CE7  57 21               DEFW L2157
0CE9  57 21               DEFW L2157
0CEB  5C 21               DEFW L215C
0CED  5C 21               DEFW L215C
0CEF  5C 21               DEFW L215C
0CF1  49 21               DEFW L2149
0CF3  49 21               DEFW L2149
0CF5  49 21               DEFW L2149
0CF7  50 21               DEFW L2150
0CF9  50 21               DEFW L2150
0CFB  50 21               DEFW L2150
0CFD  41 4F               DEFW FADD
0CFF  D3 5C               DEFW ADDS
0D01  39 44               DEFW NUMS
```

```
0D03  39 4F               DEFW FSUB
0D05  CB 5C               DEFW SUBS
0D07  EA 4F               DEFW FMUL
0D09  EB 5D               DEFW MULS
0D0B  5F 50               DEFW FDIV
0D0D  8A 5E               DEFW DIVS
0D0F  37 59               DEFW EXP
0D11  28 58               DEFW LOG
0D13  60 5A               DEFW LOG10
0D15  8D 44               DEFW COMPS
0D17  8C 21               DEFW VAL
0D19  00 00               DEFW RESET
0D1B  00 00               DEFW RESET
0D1D  00 00               DEFW RESET
0D1F  00 00               DEFW RESET
0D21  00 00               DEFW RESET
0D23  F3 4C               DEFW FTOIR
0D25  C0 4C               DEFW ITOFS
```

3.14 CLRALL/CLRVAR/CLSALL: nollställ variablerna och stäng filerna $0D27

DE = gränsen: allt som ligger från DE och uppåt kastas. LSTWLK kapar listan vid första posten på eller ovanför DE (utom poster ovanför MEMTOP, t.ex. CON:s FCB) och ger IX den posten med CY = 1.

CLRALL kapar först namnlistan (se nedan), CLRVAR går igenom variabellistan (VARLST) med CLRV1 och sätter FREPTR = DE, CLSALL går igenom filerna (FCBLST) med CLSF1. INCOMN avgör vad som behålls. CLRV1: enkla variabler nollas (heltal 2 bytes, flyttal 4/8), och IY+$26 bit 3 sätts om en flyttalsvariabel finns (då ger SINGLE/DOUBLE fel 211). För strängar och fält nollas de 6 byten i posten (strängen blir otilldelad, fältet odimensionerat); en behållen sträng förlorar ändå sin buffert om den ligger ovanför DE. I ett behållet strängfält prövas varje element så. Ett talfälts data rörs inte.

CLSF1: filer ovanför DE stängs alltid; annars stängs de om INCOMN säger det och FCB+3 bit 1 är 0. S0D4A (END och CLOSE utan nummer, $3918) stänger allt utom CON:. Namnlistan: med EXTEND får variablerna långa namn. Varje namn får ett nummer ($0400 + avståndet i bytes från listans början) och står i en lista med $FE före varje namn (första bokstaven versal, resten gemener, som LIST skriver dem) och $FF sist. Ordet sist i programmet är en relativ länk till listan. Nya namn tas från variabelområdet (ALLOC, $3376) och kedjas där; CLEAR flyttar in dem i programmets svans (NAMPAK, se VARNAM).

In: DE = gränsen, IY+$1D bit 3 och 4 (se INCOMN). Ut: FREPTR = DE; ett fel från en CLOSE går till ERRA. Förstör AF, BC, HL och IX.

```
0D27  2A 08 FF     CLRALL: LD   HL,(PRGEND)
0D2A  46                   LD   B,(HL)          BC = ordet sist i programmet
0D2B  2B                   DEC  HL
0D2C  4E                   LD   C,(HL)
0D2D  DD 21 0F 0E          LD   IX,L0E0F        bara kapning (L0E0F = RET)
0D31  CD 68 0D             CALL LSTWK2          namnlistan, huvud vid PRGEND-1+BC
0D34  DD 21 8F 0D  CLRVAR: LD   IX,CLRV1        IX = rutinen för varje variabel
0D38  01 2C 00             LD   BC,$002C        VARLST = IY+$2C
0D3B  FD CB 26 9E          RES  3,(IY+$26)      ingen flyttalsvariabel ännu; CLRV1 sätter biten
0D3F  CD 65 0D             CALL LSTWLK          variablerna
0D42  CD 50 0D             CALL CLSALL          sedan filerna
0D45  ED 53 0A FF          LD   (FREPTR),DE     FREPTR = DE
0D49  C9                   RET
```

S0D4A: stäng alla filer utom CON: (END, $1D6B, och CLOSE utan nummer, $3918). DE = $FFFF kapar ingenting, och S0F40 sätter IY+$1D bit 3 och 4, så att INCOMN säger nollställ om varje post. CLSALL: stäng filerna i FCBLST med CLSF1; IY+$20 samlar det första felet (ERRSUM). In: DE = gränsen. Ut: NC, eller felet via ERRA. Förstör AF, BC, HL och IX.

```
0D4A  11 FF FF  S0D4A:  LD   DE,$FFFF        inget kapas
0D4D  CD 40 0F          CALL S0F40           och ingen COMMON-variabel behålls
0D50  01 40 00  CLSALL: LD   BC,$0040        FCBLST = IY+$40
0D53  DD 21 F9 0D       LD   IX,CLSF1        IX = rutinen för varje FCB
0D57  FD 36 20 00       LD   (IY+$20),$00    inget fel ännu
0D5B  CD 65 0D          CALL LSTWLK
0D5E  CD 45 11          CALL ERRSUM          fel från någon CLOSE?
```

```
0D61  D0                RET  NC
0D62  C3 12 00          JP   ERRA
```

LSTWLK: gå igenom listan med huvud vid IY+BC och anropa IX för varje post (HL = posten). Används av CLRVAR (variabler, CLRV1) och CLSALL (filer, CLSF1). Ligger länken på eller ovanför DE (men under MEMTOP) kapas listan där och IX får CY = 1; annars NC. Ger IX tillbaka CY slutar genomgången. In: BC = huvudets avstånd från IY, DE = gränsen, IX = rutinen. Förstör AF, BC och HL.

```
0D65  FD E5       LSTWLK: PUSH IY
0D67  E1                  POP  HL
0D68  09          LSTWK2: ADD  HL,BC          som LSTWLK, HL = listhuvudet
0D69  4E          L0D69:  LD   C,(HL)         BC = länken till nästa post
0D6A  23                  INC  HL
0D6B  46                  LD   B,(HL)
0D6C  78                  LD   A,B
0D6D  B1                  OR   C              0: listan slut
0D6E  C8                  RET  Z
0D6F  EB                  EX   DE,HL          jämför länken med DE - 1
0D70  2B                  DEC  HL
0D71  ED 42               SBC  HL,BC
0D73  09                  ADD  HL,BC
0D74  23                  INC  HL
0D75  EB                  EX   DE,HL
0D76  30 0B               JR   NC,L0D83       länken < DE: behåll
0D78  78                  LD   A,B
0D79  FD BE 0F            CP   (IY+$0F)
0D7C  30 05               JR   NC,L0D83       över MEMTOP (systemets FCB:er): behåll
0D7E  36 00               LD   (HL),$00       kapa listan här (fel)
0D80  2B                  DEC  HL
0D81  36 00               LD   (HL),$00
0D83  60          L0D83:  LD   H,B            HL = posten
0D84  69                  LD   L,C
0D85  E5          L0D85:  PUSH HL
0D86  D5                  PUSH DE
0D87  CD A8 35            CALL SCNCB          IX, CY = posten ligger ovanför DE
0D8A  D1                  POP  DE
0D8B  E1                  POP  HL
0D8C  30 DB               JR   NC,L0D69       IX gav CY: sluta
0D8E  C9                  RET
```

CLRV1: nollställ variabelposten HL (från LSTWLK). CY in: posten ligger på eller ovanför DE och är redan bortkapad; RET C, och CY avslutar genomgången. Talvariabler nollas om INCOMN säger det. Strängar och fält går till S0DD6; i ett strängfält som behålls prövas varje element (6 bytes) med S0DDC. In: HL = posten (+0/+1 länken, +2 sorten, +4 värdet), DE = gränsen. Ut: NC. Förstör AF, BC och HL.

```
0D8F  D8          CLRV1:  RET  C
0D90  23                  INC  HL
0D91  23                  INC  HL
0D92  7E                  LD   A,(HL)         C = sorten
0D93  4F                  LD   C,A
0D94  23                  INC  HL
0D95  23                  INC  HL             HL = värdet (+4)
0D96  E6 06               AND  $06            sträng eller fält?
0D98  20 17               JR   NZ,L0DB1
0D9A  06 02               LD   B,$02          heltal: 2 bytes
0D9C  CB 41               BIT  0,C            bit 0 = 1: heltal
0D9E  20 07               JR   NZ,L0DA7
0DA0  FD CB 26 DE         SET  3,(IY+$26)     flyttalsvariabel finns
0DA4  FD 46 1E            LD   B,(IY+$1E)     B = 4 eller 8 (SINGLE/DOUBLE)
0DA7  CD 10 0E    L0DA7:  CALL INCOMN         Z: behålls
0DAA  C8                  RET  Z
0DAB  AF          L0DAB:  XOR  A              B bytes nollas från HL
0DAC  77          L0DAC:  LD   (HL),A
0DAD  23                  INC  HL
0DAE  10 FC               DJNZ L0DAC
0DB0  C9                  RET

0DB1  E5          L0DB1:  PUSH HL
```

```
0DB2  F5                PUSH AF
0DB3  CD D6 0D          CALL S0DD6          posten själv (6 bytes)
0DB6  F1                POP  AF
0DB7  E1                POP  HL
0DB8  EE 06             XOR  $06            bara strängfält går vidare
0DBA  C0                RET  NZ
0DBB  4E                LD   C,(HL)         BC = fältets storlek i bytes
0DBC  23                INC  HL
0DBD  46                LD   B,(HL)
0DBE  79                LD   A,C
0DBF  B0                OR   B              0: odimensionerat
0DC0  C8                RET  Z
0DC1  23                INC  HL             HL = fältets data
0DC2  7E                LD   A,(HL)
0DC3  23                INC  HL
0DC4  66                LD   H,(HL)
0DC5  6F                LD   L,A
0DC6  09                ADD  HL,BC          HL = fältets slut
0DC7  E5                PUSH HL
0DC8  ED 42             SBC  HL,BC
0DCA  C1          L0DCA: POP  BC            BC = slutet
0DCB  A7                AND  A
0DCC  ED 42             SBC  HL,BC
0DCE  09                ADD  HL,BC          HL når slutet: klart
0DCF  D0                RET  NC
0DD0  C5                PUSH BC
0DD1  CD DC 0D          CALL S0DDC          strängen vid BC
0DD4  18 F4             JR   L0DCA
```

S0DD6: posten för en sträng eller ett fält (6 bytes vid HL) nollställs om INCOMN säger det (NZ). S0DDC: posten nollställs om bufferten eller fältets data (+2/+3) ligger på eller ovanför DE, eller om adressen är 0; annars behålls den. Ut: HL = posten + 6. Förstör AF och BC.

```
0DD6  AF          S0DD6: XOR  A
0DD7  CD 10 0E           CALL INCOMN        NZ: nollställ
0DDA  20 19              JR   NZ,L0DF5
0DDC  E5          S0DDC: PUSH HL
0DDD  23                 INC  HL
0DDE  23                 INC  HL
0DDF  4E                 LD   C,(HL)        BC = buffertens adress
0DE0  23                 INC  HL
0DE1  46                 LD   B,(HL)
0DE2  23                 INC  HL
0DE3  23                 INC  HL
0DE4  23                 INC  HL
0DE5  78                 LD   A,B
0DE6  B1                 OR   C
0DE7  28 0B              JR   Z,L0DF4       ingen buffert
0DE9  EB                 EX   DE,HL         DE - BC
0DEA  ED 42              SBC  HL,BC
0DEC  09                 ADD  HL,BC
0DED  EB                 EX   DE,HL
0DEE  38 04              JR   C,L0DF4       bufferten ovanför DE:
0DF0  28 02              JR   Z,L0DF4       eller på DE: nollställ
0DF2  F1                 POP  AF            annars behålls den (HL = posten + 6)
0DF3  C9                 RET

0DF4  E1          L0DF4: POP  HL
0DF5  06 06       L0DF5: LD   B,$06         6 bytes nollas
0DF7  18 B2              JR   L0DAB
```

CLSF1: stäng FCB:n HL (från CLSALL via LSTWLK). CY in: FCB:n ligger på eller ovanför DE och stängs alltid. Annars stängs den om INCOMN säger det (NZ) och FCB+3 bit 1 är 0. Ett fel från FCLOS1 samlas av ERRSUM i IY+$20, så CLSF1 lämnar alltid NC och genomgången fortsätter. In: HL = FCB:n, DE = gränsen. IX bevaras.

```
0DF9  E5          CLSF1: PUSH HL
0DFA  DD E3              EX   (SP),IX       IX = FCB:n, den gamla IX på stacken
0DFC  38 0B              JR   C,L0E09       ovanför DE: stäng
0DFE  CD 10 0E           CALL INCOMN
```

```
0E01  28 09              JR   Z,L0E0C          COMMON: behåll
0E03  DD CB 03 4E        BIT  1,(IX+$03)
0E07  20 03              JR   NZ,L0E0C         bit 1 (bara CON:, $00DA): stängs aldrig här
0E09  CD 19 11     L0E09: CALL FCLOS1
0E0C  DD E1        L0E0C: POP  IX
0E0E  A7                  AND  A               NC: nästa FCB
0E0F  C9           L0E0F: RET
```

INCOMN: *ska posten vid HL nollställas? NZ = ja, Z = behåll.*

```
IY+$1D bit 3 = 0          behåll allt
bit 3 = 1, bit 4 = 1      nollställ allt
bit 3 = 1, bit 4 = 0      nollställ bara det som ligger ovanför
                          COMEND (COMMON-variablerna behålls)
```

CY måste vara 0 vid anropet (SBC *tar med den*).

```
0E10  FD CB 1D 5E  INCOMN: BIT  3,(IY+$1D)
0E14  C8                   RET  Z               bit 3 = 0: behåll allt
0E15  FD CB 1D 66          BIT  4,(IY+$1D)
0E19  C0                   RET  NZ              bit 4: nollställ allt
0E1A  D5                   PUSH DE
0E1B  EB                   EX   DE,HL
0E1C  2A 30 FF             LD   HL,(COMEND)
0E1F  ED 52                SBC  HL,DE           CY om HL > COMEND
0E21  EB                   EX   DE,HL
0E22  D1                   POP  DE
0E23  9F                   SBC  A,A             A = $FF -> NZ
0E24  A7                   AND  A
0E25  C9                   RET
```

3.15 RUNCLR: nollställ före körningen (från LINK) $0E26

Programmet och variablerna byter plats beroende på läget:

```
inmatning     programmet nere från BOTRAM, variablerna efter det
körning       programmet överst under MEMTOP (IY+$26 bit 5),
              variablerna nedifrån BOTRAM, stacken under programmet
```

Är programmet redan länkat (bit 0, oförändrat sedan förra RUN) nollställs bara variablerna, och COMMON-variablerna behålls. Annars nollställs allt, även COMMON, och programmet flyttas upp (LDDR). Därefter skrivs storlekarna i programmets huvud (S0E7A):

```
+6  variablernas storlek (referens till FREPTR)
+12 COMMON-områdets storlek (COMEND - BOTRAM)
+14 COMMON-summan (COMSUM)
```

(+2 flaggorna, +4 längden och +10 antalet variabler skrivs av SAVE.) S0EB1 nollar IY+$27 och OPTION BASE.

NEW: nollställ allt, stäng filerna och kopiera PRGINI (29 bytes) till minnets topp. CLEAR: avlänka, nollställ allt och flytta ned programmet till BOTRAM; variablerna börjar efter det. Namnlistan flyttas då in i programmet.

```
0E26  CD 3F 00     RUNCLR: CALL S003F
0E29  CD 3A 27             CALL IXBC6           BC = variabelområdets storlek (referens)
0E2C  FD CB 1D A6          RES  4,(IY+$1D)
0E30  CD 2E 0F             CALL S0F2E           DE = COMEND, COMMON behålls
0E33  CD 68 0F             CALL REFADR
0E36  EB                   EX   DE,HL           DE = variablernas slut vid länkningen
0E37  CD B1 0E             CALL S0EB1
0E3A  FD CB 26 46          BIT  0,(IY+$26)
0E3E  C2 27 0D             JP   NZ,CLRALL       länkat: bara nollställ
0E41  CD 38 0F             CALL S0F38           annars allt, även COMMON
0E44  FD CB 26 6E          BIT  5,(IY+$26)
0E48  20 6F                JR   NZ,L0EB9        programmet redan överst
0E4A  D5                   PUSH DE
0E4B  ED 5B 08 FF          LD   DE,(PRGEND)
0E4F  D5                   PUSH DE
0E50  13                   INC  DE
0E51  CD 27 0D             CALL CLRALL          DE = efter programmet
0E54  ED 5B 06 FF          LD   DE,(PRGBEG)
0E58  E1                   POP  HL
```

```
0E59  A7                AND  A
0E5A  ED 52             SBC  HL,DE
0E5C  44                LD   B,H
0E5D  4D                LD   C,L
0E5E  03                INC  BC              BC = programmets storlek
0E5F  19                ADD  HL,DE
0E60  ED 5B 0E FF  L0E60: LD  DE,(MEMTOP)
0E64  1B                DEC  DE
0E65  CD 29 0F          CALL S0F29
0E68  ED B8             LDDR                 programmet överst, under MEMTOP
0E6A  13                INC  DE
0E6B  ED 53 06 FF       LD   (PRGBEG),DE
0E6F  D1                POP  DE
0E70  FD 7E 26     L0E70: LD  A,(IY+$26)
0E73  E6 44             AND  $44
0E75  F6 20             OR   $20             bit 5: programmet överst
0E77  CD 91 3A          CALL S3A91           storlekarna (4/8 bytes, DIGITS)
0E7A  ED 53 0A FF  S0E7A: LD  (FREPTR),DE
0E7E  ED 53 2E FF       LD   (VARBEG),DE
0E82  C5           L0E82: PUSH BC
0E83  D5                PUSH DE
0E84  E5                PUSH HL
0E85  2A 0A FF          LD   HL,(FREPTR)
0E88  CD 49 0F          CALL ADRREF
0E8B  EB                EX   DE,HL
0E8C  2A 06 FF          LD   HL,(PRGBEG)
0E8F  01 06 00          LD   BC,$0006
0E92  09                ADD  HL,BC
0E93  CD 3F 32          CALL STDEHL          huvudet +6: variablernas storlek
0E96  EB                EX   DE,HL
0E97  CD 58 0F          CALL S0F58
0E9A  EB                EX   DE,HL
0E9B  CD DE 30          CALL S30DE           +12: COMMON-områdets storlek
0E9E  CD 3F 32          CALL STDEHL
0EA1  20 04             JR   NZ,L0EA7        ingen COMMON: summan = 0
0EA3  ED 53 32 FF       LD   (COMSUM),DE
0EA7  ED 5B 32 FF  L0EA7: LD  DE,(COMSUM)
0EAB  CD 3F 32          CALL STDEHL          +14: COMMON-summan
0EAE  E1                POP  HL
0EAF  D1                POP  DE
0EB0  C1                POP  BC
0EB1  AF           S0EB1: XOR  A
0EB2  FD 77 27          LD   (IY+$27),A
0EB5  FD 77 22          LD   (IY+$22),A      OPTION BASE 0
0EB8  C9                RET

0EB9  CD 27 0D     L0EB9: CALL CLRALL
0EBC  18 B2             JR   L0E70
```

L0EBE: CLEAR när programmet redan ligger nere: nollställ från DE (efter programmet), packa namnen (NAMPAK) och skriv storlekarna (S0E7A).

```
0EBE  CD 34 0D     L0EBE: CALL CLRVAR
0EC1  CD 16 34          CALL NAMPAK
0EC4  18 B4             JR   S0E7A
```

NEW: nollställ allt (S0F40), även COMMON, och stäng filerna. Ett skyddat program (IY+$26 bit 1) skrivs över med nollor, så att det inte kan läsas med PEEK efteråt; ett oskyddat ligger kvar i minnet tills något skriver över det. TRACE stängs av (IY+$23 bit 1). Sedan kopieras PRGINI till minnets topp, under MEMTOP, och storlekarna skrivs i huvudet (S0E7A). S0EC9 är samma väg utan S0F40 (LOAD, $14B3). Förstör AF, BC, DE och HL.

```
0EC6  CD 40 0F     NEW:   CALL S0F40           nollställ allt
0EC9  CD 09 10     S0EC9: CALL PRGSIZ          HL = PRGBEG, BC = DE = storleken
0ECC  FD CB 26 4E         BIT  1,(IY+$26)
0ED0  C4 D6 0F            CALL NZ,ZERO         skyddat program: skrivs över med nollor
0ED3  CD 2E 0F            CALL S0F2E           DE = COMEND (BOTRAM om bit 4)
0ED6  CD 34 0D            CALL CLRVAR
0ED9  FD CB 23 8E         RES  1,(IY+$23)      TRACE av
0EDD  D5                  PUSH DE              variablerna börjar vid DE
```

```
0EDE  21 34 10            LD   HL,$1034        HL = PRGINI + 28 (LDDR bakifrån)
0EE1  01 1D 00            LD   BC,$001D        tomt program, 29 bytes
0EE4  C3 60 0E            JP   L0E60
```

CLEAR: avlänka, nollställ alla variabler (även COMMON) och stäng filerna. Ligger programmet överst (IY+$26 bit 5) flyttas det ned till BOTRAM, och namnen utanför svansen flyttas in i den (NAMPAK); variablerna börjar sedan direkt efter programmet. Ligger det redan nere packas bara namnen (L0EBE). Förstör AF, BC, DE, HL och IX.

```
0EE7  FD CB 26 46  CLEAR:  BIT  0,(IY+$26)
0EEB  C4 FD 2E             CALL NZ,UNLINK       avlänka
0EEE  CD 38 0F             CALL S0F38           COMEND = BOTRAM
0EF1  2A 08 FF             LD   HL,(PRGEND)
0EF4  EB                   EX   DE,HL
0EF5  13                   INC  DE              DE = efter programmet
0EF6  FD CB 26 6E          BIT  5,(IY+$26)      programmet överst?
0EFA  28 C2                JR   Z,L0EBE         nej: det ligger redan nere
0EFC  EB                   EX   DE,HL           DE = BOTRAM: allt nollställs
0EFD  CD 34 0D             CALL CLRVAR
0F00  D5                   PUSH DE
0F01  CD 74 34             CALL NAMCNT          storleken på namnbitarna utanför svansen
0F04  D9                   EXX
0F05  7A                   LD   A,D
0F06  A3                   AND  E
0F07  3C                   INC  A               inga nya namn
0F08  28 0B                JR   Z,L0F15
0F0A  CD 3B 00             CALL S003B
0F0D  19                   ADD  HL,DE
0F0E  EB                   EX   DE,HL
0F0F  CD 16 0F             CALL S0F16           programmet ned till BOTRAM
0F12  CD 16 34             CALL NAMPAK          namnen in i programmets svans
0F15  D1           L0F15:  POP  DE
0F16  D5           S0F16:  PUSH DE
0F17  CD 09 10             CALL PRGSIZ          HL = PRGBEG, BC = storleken
0F1A  D1                   POP  DE
0F1B  FD CB 26 AE          RES  5,(IY+$26)      programmet nere, variablerna efter
0F1F  ED 53 06 FF          LD   (PRGBEG),DE
0F23  ED B0        L0F23:  LDIR                 flytta programmet
0F25  CD 7A 0E     S0F25:  CALL S0E7A           FREPTR = VARBEG = DE, storlekarna i huvudet
0F28  1B                   DEC  DE              DE = programmets sista byte
0F29  ED 53 08 FF  S0F29:  LD   (PRGEND),DE
0F2D  C9                   RET
```

S0F2E: DE = COMEND och IY+$1D bit 3 = 1; är bit 4 = 0 behålls COMMON, är den 1 går S0F2E vidare som S0F38. S0F38: DE = COMEND = BOTRAM (COMMON-området tas bort), sedan som S0F40. S0F40: IY+$1D bit 3 och 4 = 1, nollställ allt. Förstör DE (utom S0F40).

```
0F2E  ED 5B 30 FF  S0F2E:  LD   DE,(COMEND)     DE = COMEND
0F32  FD CB 1D 66          BIT  4,(IY+$1D)      bit 4 = 0: COMMON behålls
0F36  28 0C                JR   Z,L0F44
0F38  ED 5B 0C FF  S0F38:  LD   DE,(BOTRAM)     DE = BOTRAM
0F3C  ED 53 30 FF          LD   (COMEND),DE     inget COMMON-område
0F40  FD CB 1D E6  S0F40:  SET  4,(IY+$1D)      bit 4: COMMON nollställs också
0F44  FD CB 1D DE  L0F44:  SET  3,(IY+$1D)      bit 3: nollställ
0F48  C9                   RET
```

ADRREF: adress HL -> referens (se REFADR).

```
0F49  ED 4B 2E FF  ADRREF: LD   BC,(VARBEG)
0F4D  A7                   AND  A
0F4E  ED 42                SBC  HL,BC           HL >= VARBEG: relativt VARBEG
0F50  D0                   RET  NC
0F51  09                   ADD  HL,BC
0F52  CD 5B 0F             CALL S0F5B
0F55  CB FC                SET  7,H             annars relativt BOTRAM, bit 7
0F57  C9                   RET
```

S0F58: HL = COMEND - BOTRAM, COMMON-områdets storlek (Z = inget område). S0F5B: HL = HL - BOTRAM. Förstör BC och F.

```
0F58  2A 30 FF     S0F58:  LD   HL,(COMEND)
0F5B  ED 4B 0C FF  S0F5B:  LD   BC,(BOTRAM)
0F5F  A7                   AND  A
0F60  ED 42                SBC  HL,BC
0F62  C9                   RET
```

VARREF: *BC = ordet vid DE (DE flyttas 2 steg), sedan som* REFADR: *HL = adressen.*

```
0F63  EB           VARREF: EX   DE,HL
0F64  CD E0 30             CALL LDBCHL
0F67  EB                   EX   DE,HL
```

REFADR: *variabelreferens BC -> adress HL.*

```
    $0000-$7FFF  relativt VARBEG (vanliga variabler)
    $8000-$FEFF  relativt BOTRAM (COMMON-området under VARBEG)
    $FFxx        lokal i DEF FN: ordet vid FNFRM + BC
```

```
0F68  2A 2E FF     REFADR: LD   HL,(VARBEG)
0F6B  09                   ADD  HL,BC
0F6C  CB 78                BIT  7,B              bit 7 = 0: klar
0F6E  C8                   RET  Z
0F6F  C5                   PUSH BC
0F70  CB B8                RES  7,B
0F72  2A 0C FF             LD   HL,(BOTRAM)
0F75  09                   ADD  HL,BC
0F76  C1                   POP  BC
0F77  04                   INC  B                $FFxx?
0F78  20 08                JR   NZ,L0F82
0F7A  2A 18 FF             LD   HL,(FNFRM)       lokal: ordet i FN-ramen
0F7D  09                   ADD  HL,BC
0F7E  7E                   LD   A,(HL)
0F7F  23                   INC  HL
0F80  66                   LD   H,(HL)
0F81  6F                   LD   L,A
0F82  05           L0F82:  DEC  B
0F83  C9                   RET
```

S0F84/ALLOCC: *plats för* COMMON*-variabler (IY+*$1D *bit 5). Går bara så länge inga andra variabler finns (*FREPTR = VARBEG*), annars fel 205 (*`10 A=1 : 20 COMMON B` *ger fel 205 vid* RUN*). *COMEND*, *VARBEG *och* FREPTR *flyttas fram, och* COMSUM *= 2 x* COMSUM *+ storleken (+ spill), en summa över* COMMON*-områdets indelning. Vid* LOAD *av ett länkat program (*$15C2*) måste* COMMON*-områdets storlek och summan stämma med dem som står i filen, annars fel 205; finns inget* COMMON*-område tas det med* ALLOCC*.*

```
0F84  C5           S0F84:  PUSH BC
0F85  2A 0A FF             LD   HL,(FREPTR)
0F88  CD 49 0F             CALL ADRREF
0F8B  C1                   POP  BC
0F8C  3E CD                LD   A,$CD            fel 205: andra variabler finns redan
0F8E  37                   SCF
0F8F  C0                   RET  NZ
0F90  CD B3 0F             CALL MEMCHK           ryms det?
0F93  D8                   RET  C
0F94  E5                   PUSH HL
0F95  D5                   PUSH DE
0F96  19                   ADD  HL,DE
0F97  22 30 FF             LD   (COMEND),HL      COMMON-området växer
0F9A  EB                   EX   DE,HL
0F9B  CD 7A 0E             CALL S0E7A            VARBEG = FREPTR = COMEND
0F9E  D1                   POP  DE
0F9F  2A 32 FF             LD   HL,(COMSUM)
0FA2  29                   ADD  HL,HL            summa = 2 x summa + storlek
0FA3  ED 5A                ADC  HL,DE
0FA5  30 01                JR   NC,L0FA8
0FA7  23                   INC  HL
0FA8  22 32 FF     L0FA8:  LD   (COMSUM),HL
0FAB  AF                   XOR  A
0FAC  E1                   POP  HL
0FAD  C9                   RET
```

```
0FAE  CD 84 0F     ALLOCC: CALL S0F84
0FB1  18 23                JR   ZERO
```

MEMCHK: *ryms DE bytes till från* FREPTR (*med 100 bytes marginal under SP*)? *Annars CY och A =* $C9 (*201*).

```
0FB3  3E C9        MEMCHK: LD   A,$C9
0FB5  CD CF 0F             CALL S0FCF          HL = FREPTR
0FB8  19                   ADD  HL,DE          HL = FREPTR + DE
0FB9  D8                   RET  C              spill
0FBA  D5                   PUSH DE
0FBB  11 64 00             LD   DE,$0064
0FBE  19                   ADD  HL,DE          100 bytes marginal
0FBF  D1                   POP  DE
0FC0  D8                   RET  C
0FC1  ED 72                SBC  HL,SP          under SP?
0FC3  3F                   CCF
0FC4  D8                   RET  C
0FC5  CD CF 0F             CALL S0FCF          HL = den gamla FREPTR
0FC8  E5                   PUSH HL
0FC9  19                   ADD  HL,DE
0FCA  22 0A FF             LD   (FREPTR),HL    FREPTR flyttas
0FCD  E1                   POP  HL
0FCE  C9                   RET

0FCF  2A 0A FF     S0FCF:  LD   HL,(FREPTR)
0FD2  C9                   RET
```

ALLOC: *ta DE bytes från* FREPTR *och nollställ dem* (ZERO). *HL = det nya området.*

```
0FD3  CD B3 0F     ALLOC:  CALL MEMCHK
0FD6  D8           ZERO:   RET  C              nollställ DE bytes från HL
0FD7  7A                   LD   A,D            0 bytes: inget
0FD8  B3                   OR   E
0FD9  C8                   RET  Z
0FDA  C5                   PUSH BC
0FDB  D5                   PUSH DE
0FDC  E5                   PUSH HL
0FDD  42                   LD   B,D
0FDE  4B                   LD   C,E
0FDF  54                   LD   D,H
0FE0  5D                   LD   E,L
0FE1  0B                   DEC  BC             BC = DE - 1, DE = HL + 1
0FE2  13                   INC  DE
0FE3  36 00                LD   (HL),$00       första byten = 0, LDIR sprider den
0FE5  78                   LD   A,B            1 byte: klart
0FE6  B1                   OR   C
0FE7  28 02                JR   Z,L0FEB
0FE9  ED B0                LDIR
0FEB  E1           L0FEB:  POP  HL
0FEC  D1                   POP  DE
0FED  C1                   POP  BC
0FEE  C9                   RET
```

SPCHK0/SPCHK: *ligger SP* (SPCHK0) *resp. HL* (SPCHK) *minst 100 bytes över* FREPTR? *Annars CY och A =* $C9 (*201*).

```
0FEF  E5           SPCHK0: PUSH HL
0FF0  21 04 00             LD   HL,$0004
0FF3  39                   ADD  HL,SP          SP före anropet
0FF4  18 01                JR   L0FF7

0FF6  E5           SPCHK:  PUSH HL
0FF7  D5           L0FF7:  PUSH DE
0FF8  11 9C FF             LD   DE,$FF9C       HL - 100
0FFB  19                   ADD  HL,DE
0FFC  7D                   LD   A,L
0FFD  FD 96 0A             SUB  (IY+$0A)
1000  7C                   LD   A,H
1001  FD 9E 0B             SBC  A,(IY+$0B)     CY: för lite plats
1004  D1                   POP  DE
1005  E1                   POP  HL
```

```
1006  3E C9              LD   A,$C9
1008  C9                 RET
```

PRGSIZ: *HL = PRGBEG, BC = DE = programmets storlek (från PRGEND). Förstör AF.*

```
1009  ED 5B 06 FF  PRGSIZ: LD   DE,(PRGBEG)      DE = PRGBEG
100D  2A 08 FF             LD   HL,(PRGEND)
1010  23                   INC  HL
1011  A7                   AND  A
1012  ED 52                SBC  HL,DE            HL = storleken
1014  44                   LD   B,H
1015  4D                   LD   C,L
1016  EB                   EX   DE,HL            HL = PRGBEG, DE = storleken
1017  C9                   RET
```

PRGINI: *tomt program, 29 bytes, som NEW kopierar till minnets topp. Huvudet (20 bytes): +0 $8F (märket som LOAD prövar, $15C9), +1 $FF, +2 flaggorna, +4 längden ($001D), +6 variablernas storlek, +10 antalet variabler, +12 och +14 COMMON-områdets storlek och summa; resten 0. Sedan +20 $87 (slutet på raden före den första) och +21 slutraden (längden 1). Svansen är en tom namnbit: länk 0, antal 0 och $FF; sist ordet $FFFB, en relativ länk från ordets första byte (+27) till namnbiten (+22).*

```
1018  8F FF 00 00… PRGINI: DEFB $8F,$FF,$00,$00,$1D,$00,$00,$00
1020  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
1028  00 00 00 00…        DEFB $00,$00,$00,$00,$87,$01,$00,$00
1030  00 00 FF FB…        DEFB $00,$00,$FF,$FB,$FF
```

3.16 OPEN: öppna en fil ur en text. HL = texten, A = sätt (se OPENIX). $1035

Namnet står inom " eller ' eller sträcker sig till nästa komma eller CR. C = namnets längd; sedan OPENIX. Ut: HL på första icke-blanka tecknet efter namnet. Utan citattecken hoppas kommat också över (HL efter kommat), med citattecken inte (HL på kommat).

Anropas av LOAD ($14A4, sätt 2), SAVE ($1428, sätt 4), LIST till en fil ($13E9, sätt 3), RUN fil och CHAIN ($2CB2, sätt 2), vid starten för CON: ($00D7, texten vid $017B) och av den fasta ingången $005D; OPEN AS FILE går direkt till OPENIX. C är namnets längd utan slutttecknet. In: HL, A och IX = FCB:n. Förstör AF, BC och DE.

```
1035  F5           OPEN:   PUSH AF              sättet undan
1036  7E                   LD   A,(HL)           första tecknet
1037  23                   INC  HL
1038  FE 22                CP   $22              citattecken?
103A  28 07                JR   Z,L1043
103C  FE 27                CP   $27              eller apostrof?
103E  28 03                JR   Z,L1043
1040  2B                   DEC  HL               nej: namnet slutar vid kommat
1041  3E 2C                LD   A,$2C            slutttecknet blir kommat
1043  47           L1043:  LD   B,A              B = slutttecknet
1044  0E FF                LD   C,$FF            C räknar tecknen
1046  E5                   PUSH HL               namnets början
1047  7E           L1047:  LD   A,(HL)
1048  23                   INC  HL
1049  0C                   INC  C
104A  B8                   CP   B                slutttecknet?
104B  28 05                JR   Z,L1052
104D  FE 0D                CP   $0D              CR slutar också
104F  20 F6                JR   NZ,L1047
1051  2B                   DEC  HL               HL på CR
1052  06 00        L1052:  LD   B,$00            BC = längden
1054  2B                   DEC  HL
1055  DF                   RST  $18              första icke-blanka efter slutet
1056  EB                   EX   DE,HL            DE = efter namnet
1057  E1                   POP  HL               HL = namnet
1058  F1                   POP  AF
1059  D5                   PUSH DE               efter namnet sparas
105A  CD 5F 10             CALL OPENIX
105D  E1                   POP  HL               HL = efter namnet
105E  C9                   RET
```

OPENIX: *öppna filen i FCB:n IX. HL = namnet, C = längden, A = sätt:*

```
0     OPEN
1     PREPARE (ny fil, FCB+$0D = 0 block)
2     programfil för läsning: utan filtyp först .BAC, och finns den
      inte (fel 21) .BAS
3     PREPARE med filtypen .BAS som standard
4-    PREPARE med filtypen .BAC som standard
bit 7 via kroken HOOKA3
```

Namnet delas upp (PARSFN) i en buffert på 14 bytes på stacken: namn (8), filtyp (3) och enhet (3). Enheten söks i enhetslistan (DEVLST); FCB+4 pekar sedan på postens drivrutinsadress. Finns den inte: fel 21.

Utan enhet är enhetsnamnet blankt och ingen post passar. Blanka i namnet, eller ett namn som inte börjar med en bokstav, ger också fel 21.

Drivrutinen anropas med DE = bufferten och svarar med C = radlängden (FCB+8) och D = buffertens sida (FCB+$14). Därefter är filen öppen (FCB+3 bit 0). FCB+3 nollas när namnet är godkänt, så ett misslyckat OPEN lämnar filen stängd. Sätt 1, 3 och 4 nollar FCB+$0D/$0E och ger drivrutinen funktion 1 (PREPARE); 0 och 2 lämnar dem på $FFFF (okänt) och ger funktion 0. Efter kroken (bit 7) blir filen öppen utan att något fel prövas (L10EF). Sätt 2 med en egen filtyp prövas bara en gång. Vid .BAC -> .BAS används DE igen, så drivrutinen måste bevara DE när den svarar med fel.

```
105F  EB           OPENIX: EX   DE,HL          DE = namnet
1060  21 F2 FF             LD   HL,$FFF2       14 bytes på stacken
1063  39                   ADD  HL,SP
1064  F9                   LD   SP,HL
1065  EB                   EX   DE,HL          HL = namnet, DE = bufferten
1066  F5                   PUSH AF             sättet
1067  D5                   PUSH DE             bufferten
1068  CD B6 4A             CALL PARSFN         dela upp namnet
106B  DA 03 11             JP   C,L1103        felaktigt namn: fel 21
106E  ED 62                SBC  HL,HL          FCB+3 = 0: stängd
1070  DD 75 03             LD   (IX+$03),L
1073  1E 03                LD   E,$03          position: block 0, plats 3
1075  CD 92 49             CALL POSST
1078  2B                   DEC  HL             HL = $FFFF
1079  CD A8 4A             CALL SETBLK         inget block i bufferten
107C  CD AF 4A             CALL SETCNT         antalet block okänt
107F  D1                   POP  DE             DE = bufferten
1080  2A 7B FF             LD   HL,(DEVLST)    sök enheten
1083  7C           L1083:  LD   A,H
1084  B5                   OR   L
1085  28 7C                JR   Z,L1103        slut på listan: fel 21
1087  4E                   LD   C,(HL)         BC = länken till nästa post
1088  23                   INC  HL
1089  46                   LD   B,(HL)
108A  23                   INC  HL
108B  C5                   PUSH BC
108C  D5                   PUSH DE
108D  EB                   EX   DE,HL          HL = bufferten, DE = postens namn
108E  01 0B 00             LD   BC,$000B
1091  09                   ADD  HL,BC          HL = buffertens enhet (+11)
1092  0E 03                LD   C,$03          jämför enhetens namn med buffertens +11
1094  1A           L1094:  LD   A,(DE)
1095  13                   INC  DE
1096  ED A1                CPI
1098  20 07                JR   NZ,L10A1       olika
109A  EA 94 10             JP   PE,L1094       tre tecken
109D  EB                   EX   DE,HL          HL = drivrutinens adress i posten
109E  CD 7C 27             CALL IXST4          FCB+4 = adressen till drivrutinens adress
10A1  D1           L10A1:  POP  DE             bufferten
10A2  E1                   POP  HL             nästa post
10A3  20 DE                JR   NZ,L1083       inte den: vidare
10A5  F1                   POP  AF             sättet
10A6  A7                   AND  A
10A7  FA C7 10             JP   M,L10C7        bit 7: kroken
10AA  28 3B                JR   Z,L10E7        0: OPEN
10AC  3D                   DEC  A
10AD  28 0F                JR   Z,L10BE        1: PREPARE
10AF  3D                   DEC  A
10B0  28 1A                JR   Z,L10CC        2: programfil
```

```
10B2  3D                DEC  A
10B3  21 76 11          LD   HL,$1176      3: .BAS
10B6  28 03             JR   Z,L10BB
10B8  21 73 11          LD   HL,$1173      4-: .BAC
10BB  CD 5E 11    L10BB: CALL DEFEXT
10BE  AF          L10BE: XOR  A            PREPARE: FCB+$0D = 0
10BF  67                LD   H,A
10C0  6F                LD   L,A
10C1  3C                INC  A             A = 1: PREPARE
10C2  CD AF 4A          CALL SETCNT
10C5  18 22             JR   L10E9         till drivrutinen

10C7  CD A3 FF    L10C7: CALL HOOKA3       sätt med bit 7: kroken
10CA  18 23             JR   L10EF

10CC  21 73 11    L10CC: LD   HL,$1173     standard .BAC
10CF  CD 5E 11          CALL DEFEXT
10D2  20 13             JR   NZ,L10E7      egen filtyp: ett försök
10D4  3E 00             LD   A,$00
10D6  CD 52 11          CALL DRVJP         OPEN .BAC
10D9  30 14             JR   NC,L10EF
10DB  FE 15             CP   $15           fel 21: prova .BAS
10DD  C2 12 00          JP   NZ,ERRA
10E0  21 76 11          LD   HL,$1176
10E3  7E                LD   A,(HL)        A = B: .BAC ...
10E4  CD 60 11          CALL DEFEX1        ... blir .BAS
10E7  3E 00       L10E7: LD   A,$00        funktion 0: OPEN
10E9  CD 52 11    L10E9: CALL DRVJP
10EC  DA 12 00          JP   C,ERRA        fel
10EF  DD CB 03 C6 L10EF: SET  0,(IX+$03)   öppen
10F3  DD 36 06 00       LD   (IX+$06),$00  kolumn 0
10F7  DD 71 08          LD   (IX+$08),C    radlängd från drivrutinen
10FA  DD 72 14          LD   (IX+$14),D    buffertens sida från drivrutinen
10FD  21 0E 00          LD   HL,$000E      släpp bufferten
1100  39                ADD  HL,SP
1101  F9                LD   SP,HL
1102  C9                RET

1103  D7          L1103: RST  $10          fel 21: felaktigt filnamn / okänd enhet
1104  15                 DEFB $15
```

PRGCLS: *stäng programfilen (PRGFCB) nu, och en gång till när anroparen återvänder. Returadressen byts:* S13A8 *lägger* CLSPRG *under anroparens egen returadress.* LOAD *och* SAVE *behöver alltså inte stänga filen själva, inte heller vid fel.*

Anropas av LIST/ED ($13D1), SAVE ($1423) *och* LOAD/MERGE ($149E). *AF, BC, DE och HL är som vid ingången; BC', DE' och HL' förstörs. Ett fel när den gamla filen stängs vid ingången försvinner (*POP *AF).*

```
1105  D9          PRGCLS: EXX              registren undan
1106  F5                  PUSH AF          A undan
1107  CD 0F 11            CALL CLSPRG      stäng nu
110A  F1                  POP  AF          A tillbaka
110B  C1                  POP  BC          anroparens returadress
110C  CD A8 13            CALL S13A8       lägger $110F på stacken
```

CLSPRG: *stäng PRGFCB ($FF63). CY = 1: A är ett fel som redan har inträffat och som ska rapporteras (IY+$20). Anropas också av* READY ($0124) *och* RUN/CHAIN ($2CAC).

FCLOSE: *stäng FCB:n IX och nollställ IY+$20 först.*

FCLOS1: *stäng FCB:n IX. Är den öppen: är det* TRACE*-filen (IY+$35) stängs* TRACE *av; en påbörjad* PRINT*-fil avslutas (*DVEND*), bufferten skrivs (*BUFFLU*) och drivrutinen får funktion 2 (*CLOSE*). FCB+3 = 0.*

Ett fel i DVEND *eller* BUFFLU *sätter FCB+3 bit 4 (*CLSERR*) och räknas in med* ERRSUM*;* CLOSE *skickas ändå, och sedan nollas FCB+3. Ingenting i ROM:en (inte heller DOS:en) prövar bit 4, så bara drivrutinen ser den, under* CLOSE*. Ut: CY och A = det första felet; DE bevaras.*

ERRSUM: *IY+$20 håller det första felet när flera filer stängs; det är det som rapporteras. In: CY och A = ett nytt fel. Ut: CY och A = det första felet, NC om inget.*

Fel: CLSALL *(*RUN*,* CLEAR*,* NEW*, en ny programrad) kapar FCB-listan före* FCLOS1*. Är* TRACE *riktat till en fil (*TRACE *#n) hittar* NOTRAC *inte filen när radslutet ska skrivas, och det blir fel 32. Drivrutinen får då ingen* CLOSE *(bufferten skrivs inte),* TRACE *är avslaget och filen borta. En gång till går bra.* CLOSE *n,* CLOSE *och* END *stänger rätt: radslut, sedan* CLOSE*.*

```
110F  DD 21 63 FF  CLSPRG: LD   IX,$FF63
1113  38 01               JR   C,L1116          CY: felet A står kvar
1115  AF           FCLOSE: XOR  A
1116  FD 77 20     L1116:  LD   (IY+$20),A      det första felet
1119  AF           FCLOS1: XOR  A               A = 0: inget fel
111A  DD CB 03 46          BIT  0,(IX+$03)
111E  28 1B                JR   Z,L113B          inte öppen
1120  D5                   PUSH DE
1121  DD B6 02             OR   (IX+$02)
1124  28 06                JR   Z,L112C          fil 0 är aldrig TRACE-filen
1126  FD AE 35             XOR  (IY+$35)         TRACE-filen?
1129  CC 03 20             CALL Z,NOTRAC
112C  CD 82 49     L112C:  CALL DVEND            PRINT-filen avslutas
112F  D4 5C 4A             CALL NC,BUFFLU        bufferten skrivs
1132  DC 41 11             CALL C,CLSERR         fel: bit 4
1135  3E 02                LD   A,$02            CLOSE
1137  CD 52 11             CALL DRVJP
113A  D1                   POP  DE
113B  DD 36 03 00  L113B:  LD   (IX+$03),$00     stängd
113F  18 04                JR   ERRSUM           felet räknas in

1141  DD CB 03 E6  CLSERR: SET  4,(IX+$03)      fel vid stängningen
1145  08           ERRSUM: EX   AF,AF'           det nya felet undan
1146  FD 7E 20             LD   A,(IY+$20)
1149  A7                   AND  A                redan ett fel?
114A  37                   SCF                   ja: det står kvar, CY
114B  C0                   RET  NZ
114C  08                   EX   AF,AF'           det nya felet
114D  D0                   RET  NC               NC: inget nytt fel
114E  FD 77 20             LD   (IY+$20),A       blir det första
1151  C9                   RET
```

DRVJP: anropa drivrutinen för FCB:n IX med funktionen A.

FCB+4 pekar på drivrutinens adress i enhetsposten. Drivrutinen får HL och DE som de var och återvänder direkt till anroparen av DRVJP; vad som bevaras efter det bestämmer drivrutinen. Ut (från drivrutinen): CY och A = felet.

```
1152  E5           DRVJP:  PUSH HL
1153  D5                   PUSH DE
1154  CD 67 27             CALL IXHL4            HL = FCB+4
1157  5E                   LD   E,(HL)           DE = drivrutinens adress
1158  23                   INC  HL
1159  56                   LD   D,(HL)
115A  EB                   EX   DE,HL            HL = adressen
115B  D1                   POP  DE               DE tillbaka
115C  E3                   EX   (SP),HL          hoppa dit, HL tillbaka
115D  C9                   RET
```

DEFEXT: HL = standardfiltypen, DE = bufferten från PARSFN. Är filtypen (+8) tom (första tecknet blank) läggs HL:s tre tecken in. DEFEX1 jämför med A i stället (används för att byta .BAC mot .BAS). Ut: Z = inlagd, då HL efter de tre tecknen och BC = 0. DE bevaras; förstör A och F.

```
115E  3E 20        DEFEXT: LD   A,$20
1160  D5           DEFEX1: PUSH DE
1161  E5                   PUSH HL
1162  21 08 00             LD   HL,$0008         HL = filtypen (+8)
1165  19                   ADD  HL,DE
1166  BE                   CP   (HL)             filtypens första tecken
1167  EB                   EX   DE,HL            DE = filtypen
1168  E1                   POP  HL               HL = standardtypen
1169  20 06                JR   NZ,L1171         inte tom: NZ
116B  01 03 00             LD   BC,$0003
116E  ED B0                LDIR                  lägg in
1170  AF                   XOR  A                Z: inlagd
1171  D1           L1171:  POP  DE
1172  C9                   RET

1173  42 41 43 42… EXTBAC: DEFM "BACBAS"
```

GETPS0: kolumn och radlängd för fil 0. GETPOS: för filen A. Ut: A = kolumnen (FCB+6), E = radlängden (FCB+8). IX bevaras; förstör F.

```
1179  AF           GETPS0: XOR  A
117A  DD E5        GETPOS: PUSH IX              IX bevaras
117C  CD 88 11             CALL FNDFIL
117F  DD 7E 06             LD   A,(IX+$06)
1182  DD 5E 08             LD   E,(IX+$08)
1185  DD E1                POP  IX
1187  C9                   RET
```

FNDFIL: IX = FCB:n för filnummer A (FCBSRC). Fel 32 om den inte finns eller inte är öppen. A och BC bevaras; förstör F.

PRCON, PRFILE, DVPRT, DVPUT och DVGET anropar drivrutinen via DVCALL med funktion 4 (PRINT), 6 (PUT) eller 5 (GET). Vid fel: ERRJP; kommer den tillbaka (CON, eller RESUME efter ON ERROR GOTO) görs anropet om. In: IX = FCB:n (PRCON: fil 0, PRFILE: filen A), HL = texten, BC = längden. Ut: NC, HL och BC som vid ingången. Förstör A, DE (D = funktionen) och F.

```
1188  C5           FNDFIL: PUSH BC              BC bevaras
1189  CD C4 11             CALL FCBSRC
118C  20 06                JR   NZ,L1194        finns inte
118E  DD CB 03 46          BIT  0,(IX+$03)      öppen?
1192  C1                   POP  BC
1193  C0                   RET  NZ
1194  D7           L1194:  RST  $10             fel 32: filen är inte öppen
1195  20                   DEFB $20

1196  AF           PRCON:  XOR  A               fil 0
1197  CD 88 11     PRFILE: CALL FNDFIL          IX = filen A
119A  3E 04        DVPRT:  LD   A,$04           PRINT
119C  11                   DEFB $11             LD DE,$063E (överlappar nästa)
119D  3E 06        DVPUT:  LD   A,$06           PUT
119F  11                   DEFB $11             LD DE,$053E (överlappar nästa)
11A0  3E 05        DVGET:  LD   A,$05           GET
11A2  F5           L11A2:  PUSH AF              funktionen sparas
11A3  E5                   PUSH HL
11A4  C5                   PUSH BC
11A5  CD 53 49             CALL DVCALL          till drivrutinen
11A8  C1                   POP  BC
11A9  E1                   POP  HL
11AA  D1                   POP  DE              D = funktionen
11AB  D0                   RET  NC              NC: inget fel
11AC  CD 33 00             CALL ERRJP           fel som går att fortsätta
11AF  7A                   LD   A,D             efter CON/RESUME: funktionen igen
11B0  18 F0                JR   L11A2           samma anrop igen
```

NUL:-enheten: länk till CON: ($0180), NUL, drivrutin $11B9. INPUT och GET ger fel 34 ($22, filslut), allt annat går bra (CY = 0). LOAD NUL: ger alltså ett tomt program, MERGE NUL: ingenting. Vid OPEN lämnar drivrutinen C och D som de är: radlängden (FCB+8) blir 0, som C står efter sökningen i OPENIX, och buffertens sida (FCB+$14) blir D, buffertens sida på stacken ($F2 efter OPEN "NUL:" AS FILE 2 i emulatorn). I ABC800 pekar länken på CAS: ($0677).

```
11B2  80 01 4E 55… NULDEV: DEFB $80,$01,$4E,$55,$4C,$B9,$11
11B9  FE 03        NULDRV: CP   $03
11BB  28 03                JR   Z,L11C0         INPUT
11BD  EE 05                XOR  $05             GET
11BF  C0                   RET  NZ              övriga: NC, inget fel
11C0  3E 22        L11C0:  LD   A,$22           fel 34: filslut
11C2  37                   SCF
11C3  C9                   RET
```

FCBSRC: sök filnummer A i FCB-listan från FCBLST. Ut: Z och IX = FCB:n om den finns; annars NZ (B = 1) och IX = det sista blocket i listan, där SOPEN länkar in ett nytt. Listan slutar med en länk vars höga byte är 0. A bevaras; förstör BC och F.

```
11C4  DD 21 40 FF  FCBSRC: LD   IX,$FF40        listans början
11C8  DD 4E 00     L11C8:  LD   C,(IX+$00)      BC = länken
11CB  DD 46 01             LD   B,(IX+$01)
11CE  04                   INC  B               höga byten 0: slut, NZ
11CF  10 02                DJNZ L11D3
11D1  04                   INC  B               B = 1: NZ
11D2  C9                   RET
```

```
11D3  C5          L11D3:  PUSH BC             IX = nästa FCB
11D4  DD E1               POP  IX
11D6  DD BE 02            CP   (IX+$02)       filnumret?
11D9  20 ED               JR   NZ,L11C8
11DB  C9                  RET
```

3.17 DOLINE: utför en inmatad rad (HL = bufferten) $11DC

Tom rad: inget. I kommandoläge (IY+$23 bit 2) söks först bland direktkommandona (CMDKW); träff -> DOCMD via CMDTAB. Är programmet skyddat (IY+$26 bit 1) går bara RUN, NEW, LOAD och CON; allt annat, även satser och programrader, ger fel 202. Biten kommer från programhuvudets +2 vid LOAD (SAVE skriver IY+$26 dit) och nollas när programmet länkas om vid RUN (RUNCLR, AND $44); ett program som sparats länkat förblir alltså skyddat.

Annars läses ett radnummer (GETLNR). I kommandoläge prövas inte spillet: `65536 PRINT 6` tolkas som en direktsats (fel 234).

Utanför kommandoläget (LOAD av text) krävs ett nummer (fel 233). Tecknet ? efter numret hoppas över: så märker LIST felrader.

Raden kompileras (CMPLIN) i 270 bytes på stacken. Med radnummer läggs den in i programmet (INSLIN), med 0 eller utan nummer utförs den direkt (`0 PRINT 7` skriver 7).

Vid kompileringsfel i kommandoläge skrivs felet, och raden blir mall för nästa inmatning (TAB hämtar den tecken för tecken, radnumret inräknat); tom rad lämnar programmet orört. Utanför kommandoläget skrivs `Error n in line m`, och raden sparas som text: $8B, längd, texten efter numret, $87. LIST visar den som `10?PRINT (1`, RUN ger fel 144.

Efter en RUN som fyllt minnet ligger variablerna kvar tills INSLIN kör CLEAR, men kompileringen sker före: CEXPR:s stackkontroll (SPCHK0) ger fel 201 för varje rad med ett uttryck (`"20 PRINT 1"`). Rader utan uttryck (REM, PRINT, GOTO 10, END) går in och frigör minnet, liksom CLEAR.

In: HL = bufferten. Ut: CY och A = felkoden vid fel. Förstör alla register.

```
11DC  2B          DOLINE: DEC  HL(egenhet)
11DD  DF                  RST  $18            RST $18: nästa tecken
11DE  FE 0D               CP   $0D
11E0  C8                  RET  Z
11E1  FD CB 23 56         BIT  2,(IY+$23)     kommandoläge?
11E5  28 20               JR   Z,L1207
11E7  11 3F 40            LD   DE,$403F       DE = CMDKW, direktkommandona
11EA  CD 4B 4B            CALL KWSRCH         A = kod $80-$8E, $FF = inget
11ED  FD CB 26 4E         BIT  1,(IY+$26)     skyddat program?
11F1  28 04               JR   Z,L11F7
11F3  FE 84               CP   $84            LIST och framåt, satser och rader
11F5  30 0C               JR   NC,L1203
11F7  3C          L11F7:  INC  A
11F8  01 99 40            LD   BC,$4099       CMDTAB via JPTAB (H nollas)
11FB  C2 AB 13            JP   NZ,DOCMD
11FE  CD C2 4B            CALL GETLNR         spill (CY) prövas inte (egenhet)
1201  18 0A               JR   L120D

1203  3E CA        L1203:  LD   A,$CA          fel 202: skyddat program
1205  37                  SCF
1206  C9                  RET

1207  CD C2 4B    L1207:  CALL GETLNR         utan kommandoläge: nummer krävs
120A  3E E9               LD   A,$E9          fel 233: radnummer väntas
120C  D8                  RET  C
120D  7E          L120D:  LD   A,(HL)         bara radnummer: radera raden
120E  FE 0D               CP   $0D
1210  CA B5 12           JP   Z,L12B5
1213  CC 19 4C    L1213:  CALL Z,S4C19(egenhet)
1216  FE 3F               CP   $3F            ? (felradens märke) hoppas över
1218  28 F9               JR   Z,L1213
121A  42                  LD   B,D            BC = radnumret
121B  4B                  LD   C,E
121C  EB                  EX   DE,HL          DE = texten
121D  21 EC FF            LD   HL,$FFEC       gräns för EMIT
1220  39                  ADD  HL,SP
```

```
1221  22 1A FF            LD   (VFF1A),HL
1224  21 F2 FE            LD   HL,$FEF2       270 bytes på stacken
1227  39                  ADD  HL,SP
1228  F9                  LD   SP,HL
1229  E5                  PUSH HL             bufferten, texten och radnumret undan
122A  D5                  PUSH DE
122B  C5                  PUSH BC
122C  23                  INC  HL
122D  71                  LD   (HL),C         radnumret först
122E  23                  INC  HL
122F  70                  LD   (HL),B
1230  23                  INC  HL
1231  EB                  EX   DE,HL
1232  CD 2A 13            CALL CMPLIN
1235  D4 21 1A            CALL NC,CEND        sedan bara CR eller !
1238  38 43               JR   C,L127D
123A  3E 89       L123A:  LD   A,$89          $89 sist; INSLIN byter mot $87
123C  CD 87 16            CALL EMIT
123F  43                  LD   B,E
1240  D1                  POP  DE             DE = radnumret
1241  E1                  POP  HL             texten behövs inte
1242  E1                  POP  HL             HL = bufferten
1243  38 0A               JR   C,L124F        CY: raden ryms inte (fel 20)
1245  78                  LD   A,B            längden i +0
1246  95                  SUB  L
1247  77                  LD   (HL),A
1248  7B                  LD   A,E
1249  B2                  OR   D              radnummer 0: utför direkt
124A  28 0B               JR   Z,L1257
124C  CD C1 12            CALL INSLIN         lägg in raden
124F  08          L124F:  EX   AF,AF'
1250  21 0E 01            LD   HL,$010E       SP tillbaka
1253  39                  ADD  HL,SP
1254  F9                  LD   SP,HL
1255  08                  EX   AF,AF'
1256  C9                  RET

1257  FD CB 23 56 L1257:  BIT  2,(IY+$23)
125B  3E E9               LD   A,$E9          fel 233
125D  37                  SCF
125E  28 EF               JR   Z,L124F
1260  23                  INC  HL
1261  23                  INC  HL
1262  23                  INC  HL             HL = koden efter längden och numret
1263  01 4F 12            LD   BC,$124F       tillbaka via $124F
1266  C5                  PUSH BC
1267  CD 75 13            CALL NEWSTK
126A  E5                  PUSH HL
126B  CD 0B 2F            CALL LINK1          länka vid behov
126E  E1                  POP  HL
126F  D8                  RET  C              fel vid länkningen
1270  CD E8 3C            CALL CRLFC
1273  EB                  EX   DE,HL
1274  CD 07 1D            CALL RUNLP          kör raden
1277  AF                  XOR  A
1278  ED 5B 3A FF S1278:  LD   DE,(PRGPTR)    DE = PRGPTR
127C  C9                  RET

127D  E1          L127D:  POP  HL             kompileringsfel
127E  C1                  POP  BC
127F  D1                  POP  DE
1280  FD CB 23 56         BIT  2,(IY+$23)
1284  20 14               JR   NZ,L129A       kommandoläge: rättning
1286  D5                  PUSH DE
1287  C5                  PUSH BC
1288  E5                  PUSH HL
1289  C5                  PUSH BC
128A  D5                  PUSH DE
128B  CD 8D 3C            CALL S3C8D          Error n in line m
```

```
128E  D1              POP  DE
128F  13              INC  DE
1290  13              INC  DE
1291  13              INC  DE
1292  E1              POP  HL
1293  3E 8B           LD   A,$8B          raden som text: $8B, längd
1295  CD 2E 1A        CALL S1A2E
1298  18 A0           JR   L123A

129A  D5        L129A:  PUSH DE
129B  CD 8A 3C          CALL S3C8A        felet utan radnummer
129E  CD 51 00          CALL BUFADR
12A1  D1                POP  DE
12A2  D5                PUSH DE
12A3  CD 72 1A          CALL CTEXT        raden blir mall
12A6  ED A0             LDI
12A8  E1                POP  HL
12A9  23                INC  HL
12AA  EB        L12AA:  EX   DE,HL
12AB  CD 51 00          CALL BUFADR
12AE  E5                PUSH HL
12AF  CD 8D 01          CALL INLIN2       TAB hämtar mallens tecken
12B2  C3 32 01          JP   L0132

12B5  7A        L12B5:  LD   A,D          bara radnummer: radera
12B6  B3                OR   E
12B7  C8                RET  Z
12B8  ED 62             SBC  HL,HL        längd 0 på stacken
12BA  E5                PUSH HL
12BB  39                ADD  HL,SP
12BC  CD C1 12          CALL INSLIN
12BF  E1                POP  HL
12C0  C9                RET
```

3.18 INSLIN: lägg in (eller ersätt/ta bort) en programrad $12C1

Programrad: +0 längd (hela raden), +1,+2 radnummer, sedan de interna koderna, sist $87. Programmet slutar med en rad med längden 1.

CLEAR först (variablerna tas bort). FNDLIN letar upp numret; finns raden är dess längd den gamla, annars 0. Lika lång: bara kopiera. Längre: MEMCHK (fel 201, programmet orört) och flytta med LDDR. Kortare: flytta ihop med LDIR. Längd 0 (bara radnummer) tar bort raden; ett nummer som inte finns ändrar ingenting.

In: HL = den kompilerade raden (+0 längden, 0 = ta bort), DE = radnumret. Ut: CY och A = 201 om raden inte ryms. Förstör alla register.

```
12C1  E5        INSLIN: PUSH HL
12C2  D5                PUSH DE
12C3  CD E7 0E          CALL CLEAR        variablerna bort först
12C6  D1                POP  DE
12C7  CD 8C 32          CALL FNDLIN       Z = raden finns
12CA  C1                POP  BC
12CB  C5                PUSH BC
12CC  0A                LD   A,(BC)       A = nya radens längd
12CD  01 00 00          LD   BC,$0000
12D0  20 01             JR   NZ,L12D3
12D2  4E                LD   C,(HL)       C = gamla längden
12D3  91        L12D3:  SUB  C
12D4  28 27             JR   Z,L12FD      lika långa
12D6  38 32             JR   C,L130A      kortare: flytta ihop
12D8  E5                PUSH HL
12D9  50                LD   D,B
12DA  47                LD   B,A
12DB  C5                PUSH BC
12DC  58                LD   E,B
12DD  CD 24 13          CALL S1324        ryms den? annars fel 201
12E0  C1                POP  BC
12E1  38 3E             JR   C,L1321
```

```
12E3  78                LD   A,B
12E4  06 00             LD   B,$00
12E6  EB                EX   DE,HL
12E7  E1                POP  HL
12E8  E5                PUSH HL
12E9  09                ADD  HL,BC
12EA  D5                PUSH DE
12EB  EB                EX   DE,HL
12EC  ED 52             SBC  HL,DE
12EE  4D                LD   C,L
12EF  44                LD   B,H
12F0  D1                POP  DE
12F1  6F                LD   L,A
12F2  26 00             LD   H,$00
12F4  19                ADD  HL,DE
12F5  1B                DEC  DE
12F6  EB                EX   DE,HL
12F7  CD 25 0F          CALL S0F25          PRGEND = DE - 1, FREPTR = VARBEG = DE
12FA  ED B8             LDDR                gör plats
12FC  E1          L12FC: POP  HL
12FD  EB          L12FD: EX   DE,HL
12FE  E1                POP  HL
12FF  4E                LD   C,(HL)
1300  79                LD   A,C
1301  A7                AND  A
1302  C8                RET  Z              längd 0: bara raderad
1303  ED B0             LDIR                den nya raden in
1305  1B                DEC  DE
1306  3E 87             LD   A,$87          $87 sist
1308  12                LD   (DE),A
1309  C9                RET

130A  E5          L130A: PUSH HL            kortare: HL = platsen för raden
130B  09                ADD  HL,BC
130C  EB                EX   DE,HL
130D  2A 08 FF          LD   HL,(PRGEND)
1310  ED 52             SBC  HL,DE
1312  44                LD   B,H
1313  4D                LD   C,L
1314  03                INC  BC
1315  26 FF             LD   H,$FF
1317  6F                LD   L,A
1318  19                ADD  HL,DE
1319  EB                EX   DE,HL
131A  ED B0             LDIR                flytta ihop
131C  CD 25 0F          CALL S0F25          PRGEND = DE - 1, FREPTR = VARBEG = DE
131F  18 DB             JR   L12FC

1321  C1          L1321: POP  BC            fel 201: stacken rensas
1322  C1                POP  BC
1323  C9                RET
```

S1324: ryms DE bytes till? MEMCHK med SP på den stack som NEWSTK väljer (MEMTOP när programmet ligger nere), inte vid kodbufferten på den nuvarande stacken. Ut: CY och A = 201 om inte. Förstör AF och HL.

```
1324  CD 75 13    S1324: CALL NEWSTK
1327  C3 B3 0F           JP   MEMCHK
```

CMPLIN: kompilera en rad till interna koder vid DE. Satser skiljs åt av : ($88). Varje sats börjar med $86 (CMPSTM) följt av satsens kod från nyckelordstabellerna (STKWP-kedjan). Kompileringen görs av satsens rutin via hopptabellen $0AC5 (JPTAB). Körs med SP flyttad (NEWSTK). In: HL = texten, DE = var koden skrivs. Ut: HL efter raden, DE efter koden, CY och A = felet. Förstör alla register.

```
132A  CD 75 13  CMPLIN: CALL NEWSTK
132D  CD 51 13          CALL CMPSTM         en sats
1330  D8                RET  C
1331  7E          L1331: LD   A,(HL)        : -> $88, nästa sats
1332  EE 3A             XOR  $3A
1334  C0                RET  NZ
```

```
1335  3E 88              LD   A,$88
1337  12                 LD   (DE),A
1338  13                 INC  DE
1339  DF                 RST  $18
133A  D5           CMPST2: PUSH DE             nästa sats; fel 222 om den bara får stå först (egenhet)
133B  CD 51 13             CALL CMPSTM
133E  C1                   POP  BC
133F  D8                   RET  C
1340  0A                   LD   A,(BC)         satsens kod
1341  E5                   PUSH HL
1342  21 C5 13             LD   HL,$13C5
1345  01 05 00             LD   BC,$0005
1348  ED B1                CPIR
134A  E1                   POP  HL
134B  20 E4                JR   NZ,L1331
134D  3E DE                LD   A,$DE          fel 222: satsen får inte stå efter :
134F  37                   SCF
1350  C9                   RET
```

CMPSTM: *kompilera en sats vid HL till koden vid DE: först* $86, *sedan satsens kod. Ett nyckelord ur* STKWP*-kedjan går till satsens rutin via* JPTAB ($0AC5 *för ROM:ens satser); annars prövas en tilldelning utan LET* (CLET). *Ut: HL efter satsen, DE efter koden, CY och A = felet* (220 *för en okänd sats*). *Förstör alla register.*

```
1351  3E 86        CMPSTM: LD   A,$86          $86: satsens början
1353  12                   LD   (DE),A
1354  13                   INC  DE
1355  D9                   EXX
1356  2A 7D FF             LD   HL,(STKWP)     satsernas tabellkedja
1359  CD A2 19             CALL S19A2
135C  1B                   DEC  DE
135D  30 0B                JR   NC,L136A       NC = nyckelord
135F  CD B7 16             CALL CLET           annars tilldelning utan LET
1362  D0                   RET  NC
1363  FE E4                CP   $E4            fel 228 (= saknas) -> 220
1365  37                   SCF
1366  C0                   RET  NZ
1367  3E DC                LD   A,$DC          fel 220: okänd sats
1369  C9                   RET

136A  20 05        L136A:  JR   NZ,L1371
136C  01 C5 0A             LD   BC,$0AC5       ROM:ens satser: JPTABL
136F  1B                   DEC  DE
1370  12                   LD   (DE),A
1371  13           L1371:  INC  DE
1372  C3 B7 13             JP   JPTAB          satsens kompilering
```

NEWSTK: *kör resten av anroparen med SP flyttad: till* MEMTOP (*IY+*$0E), *eller om IY+*$26 *bit 5 är satt till* PRGBEG (*IY+*$06), *och om dessutom IY+*$27 *bit 0* (CON *möjligt*) *till* CONSP (*IY+*$12), *så att ett avbrutet programs stack står kvar. Ligger den nya SP inte 100 bytes över* FREPTR *blir det fel 201* (ERRSP). *Den gamla SP läggs på den nya stacken; när anroparen gör* RET *återställs den och* RET *går till anroparens anropare. Registren* (EXX) *och flaggorna går igenom.*

```
1375  D9           NEWSTK: EXX
1376  C1                   POP  BC
1377  11 0E 00             LD   DE,$000E       MEMTOP
137A  FD CB 26 6E          BIT  5,(IY+$26)
137E  28 0C                JR   Z,L138C
1380  11 06 00             LD   DE,$0006       PRGBEG
1383  FD CB 27 46          BIT  0,(IY+$27)
1387  28 03                JR   Z,L138C
1389  11 12 00             LD   DE,$0012       CONSP
138C  FD E5        L138C:  PUSH IY
138E  E1                   POP  HL
138F  19                   ADD  HL,DE
1390  5E                   LD   E,(HL)
1391  23                   INC  HL
1392  56                   LD   D,(HL)
1393  21 00 00             LD   HL,$0000
1396  39                   ADD  HL,SP
1397  EB                   EX   DE,HL
```

```
1398  CD F6 0F            CALL SPCHK          100 bytes över FREPTR?
139B  DA 6B 3B            JP   C,ERRSP
139E  F9                  LD   SP,HL
139F  D5                  PUSH DE             gamla SP på nya stacken
13A0  CD A8 13            CALL S13A8          resten av anroparen
13A3  D9                  EXX
13A4  E1                  POP  HL             gamla SP tillbaka
13A5  F9                  LD   SP,HL
13A6  D9                  EXX
13A7  C9                  RET

13A8  C5            S13A8: PUSH BC
13A9  D9                   EXX
13AA  C9                   RET
```

DOCMD: *utför ett direktkommando (A = index i* CMDTAB*). Först kroken* HOOK1*, sedan kommandot; ett fel skrivs utan radnummer. Återvänder aldrig: hoppar alltid till* READY*.*

```
13AB  CD 8D FF      DOCMD: CALL HOOK1          RAM-krok
13AE  CD B7 13             CALL JPTAB
13B1  DC 8A 3C             CALL C,S3C8A        fel utan radnummer
13B4  C3 10 01             JP   READY          sedan alltid READY
```

JPTAB: *hoppa via tabell. BC = tabell, A = index; HL = BC + (2*A &* $FF*), så bit 7 i A försvinner. Hoppet görs med* RET*.*

```
13B7  E5            JPTAB: PUSH HL
13B8  6F                   LD   L,A
13B9  29                   ADD  HL,HL
13BA  26 00                LD   H,$00
13BC  09                   ADD  HL,BC
13BD  D5                   PUSH DE
13BE  5E                   LD   E,(HL)
13BF  23                   INC  HL
13C0  56                   LD   D,(HL)
13C1  EB                   EX   DE,HL
13C2  D1                   POP  DE
13C3  E3                   EX   (SP),HL
13C4  C9                   RET
```

STMEND: *satser som måste stå först på raden.* CMPST2 *prövar varje sats efter* : *mot tabellen (den första satsen, från* CMPLIN*, prövas inte) och ger fel 222 vid träff:* A=1:END*,* A=1:DATA 5*,* A=1:DEF FNA(X)=X *och* A=1:FNEND *är fel.* $8C = DEF*,* $8D = FNEND*,* $8E = DATA*;* END *ger också fel 222 (*$92 *(?)); *$00 *(?). Samma prov gäller satsen efter* THEN *och* ELSE *(*CIF *anropar* CMPST2*):* IF A THEN END *och* IF A THEN DATA 5 *ger fel 222.*

```
13C5  00 92 8E 8C… STMEND: DEFB $00,$92,$8E,$8C,$8D
```

ED [n]: som LIST*, men med 0 rader kvar (PRGFCB+*$16 *= 0): första raden i intervallet skrivs, och sedan blir den mall för en ny inmatningsrad (*L12AA*,* TAB *hämtar mallens tecken, numret ingår). ED utan nummer tar första raden; finns raden inte händer ingenting.*

LIST *[fil] [n1] [- n2]: PRGFCB används som tillstånd (+*$16 *= rader kvar, 20 på skärmen). En fil anges med citattecken eller som ett namn som börjar med en bokstav (*LIST X *= filen X); efter filen kan* , *och ett intervall följa. Till en fil skrivs allt utan väntan. Är programmet länkat (IY+*$26 *bit 0) letas* DEF*-blocket för första raden upp (*INDEF*,* SETDEF*), så att lokala variabler får sina namn även när listningen börjar inne i en* DEF *(*LIST *20).*

```
13CA  AF            ED:    XOR  A
13CB  4F                   LD   C,A            C = 0 rader kvar: ED
13CC  18 03                JR   L13D1

13CE  7E            LIST:  LD   A,(HL)
13CF  0E 14                LD   C,$14          C = 20 rader: LIST
13D1  CD 05 11      L13D1: CALL PRGCLS
13D4  DD 71 16             LD   (IX+$16),C     PRGFCB+$16: rader kvar
13D7  DD 36 17 00          LD   (IX+$17),$00
13DB  FE 22                CP   $22
13DD  28 08                JR   Z,L13E7
13DF  FE 27                CP   $27
13E1  28 04                JR   Z,L13E7
13E3  FE 41                CP   $41            bokstav: filnamn utan citattecken
```

```
13E5  38 0B              JR   C,L13F2
13E7  3E 03       L13E7: LD   A,$03
13E9  CD 35 10           CALL OPEN          OPEN sätt 3
13EC  7E                 LD   A,(HL)
13ED  FE 2C              CP   $2C
13EF  CC 19 4C           CALL Z,S4C19       , efter filen: intervallet
13F2  E5          L13F2: PUSH HL
13F3  CD ED 3C           CALL CRLF
13F6  E1                 POP  HL
13F7  CD 8B 15           CALL LNRNG         DE = första, BC = sista
13FA  C5                 PUSH BC
13FB  CD 8C 32           CALL FNDLIN
13FE  FD CB 26 46        BIT  0,(IY+$26)    länkat program?
1402  28 08              JR   Z,L140C
1404  E5                 PUSH HL
1405  CD 01 31           CALL INDEF
1408  CD 59 31           CALL SETDEF
140B  E1                 POP  HL
140C  C1          L140C: POP  BC
140D  7E                 LD   A,(HL)
140E  EE 01              XOR  $01           längd 1 = programslut
1410  C8                 RET  Z
1411  54                 LD   D,H
1412  5D                 LD   E,L
1413  23                 INC  HL
1414  7E                 LD   A,(HL)
1415  23                 INC  HL
1416  66                 LD   H,(HL)
1417  6F                 LD   L,A
1418  37                 SCF
1419  ED 42              SBC  HL,BC         radnummer > sista: klart
141B  D0                 RET  NC
141C  EB                 EX   DE,HL
141D  C5                 PUSH BC
141E  CD 2B 15           CALL LSTLIN        skriv raden
1421  18 E9              JR   L140C
```

SAVE *fil: alltid binärt (.BAC). Ett olänkat program länkas först. Huvudet får +2 = IY+$26 (flaggorna), +4 = längden (PRGBEG till PRGEND) och +10 = antalet variabler i VARLST; efter programmet följer en post på 4 bytes per variabel: referens (ADRREF) och namn. Skyddat program: fel 202 redan i kommandoläget, ingen fil skrivs.*

```
1423  CD 05 11    SAVE:  CALL PRGCLS
1426  3E 04              LD   A,$04
1428  CD 35 10           CALL OPEN
142B  DD CB 03 EE        SET  5,(IX+$03)
142F  DD E5              PUSH IX
1431  FD CB 26 46        BIT  0,(IY+$26)
1435  CC 27 2E           CALL Z,LINK
1438  DD 2A 06 FF        LD   IX,(PRGBEG)
143C  01 00 00           LD   BC,$0000
143F  FD 7E 26           LD   A,(IY+$26)
1442  DD 77 02           LD   (IX+$02),A    +2 = flaggorna
1445  E6 01              AND  $01
1447  58                 LD   E,B
1448  C4 6E 14           CALL NZ,S146E      E = 0: räkna bara variablerna
144B  DD 71 0A           LD   (IX+$0A),C
144E  DD 70 0B           LD   (IX+$0B),B
1451  2A 08 FF           LD   HL,(PRGEND)
1454  23                 INC  HL
1455  DD E5              PUSH IX
1457  D1                 POP  DE
1458  ED 52              SBC  HL,DE
145A  DD 75 04           LD   (IX+$04),L    +4 = längden
145D  DD 74 05           LD   (IX+$05),H
1460  DD E1              POP  IX
1462  C5                 PUSH BC
1463  4D                 LD   C,L
1464  44                 LD   B,H
```

```
1465  EB              EX   DE,HL
1466  CD 9D 11        CALL DVPUT          programmet
1469  C1              POP  BC
146A  78              LD   A,B
146B  B1              OR   C
146C  C8              RET  Z
146D  5F              LD   E,A            E <> 0: skriv posterna
```

S146E: gå igenom VARLST. E = 0: räkna bara posterna (BC); E <> 0: skriv för varje post 4 bytes till filen (DVPUT): värdets referens (ADRREF) och namnet. In: BC = 0, IX = FCB:n. Ut: BC = antalet poster. Förstör AF, DE och HL.

```
146E  2A 2C FF  S146E:  LD   HL,(VARLST)
1471  7D                LD   A,L
1472  B4        L1472:  OR   H
1473  C8                RET  Z
1474  1C                INC  E            E = 0?
1475  1D                DEC  E
1476  28 1A             JR   Z,L1492
1478  D5                PUSH DE
1479  E5                PUSH HL
147A  23                INC  HL
147B  23                INC  HL
147C  4E                LD   C,(HL)       BC = namnet
147D  23                INC  HL
147E  46                LD   B,(HL)
147F  23                INC  HL
1480  C5                PUSH BC
1481  CD 49 0F          CALL ADRREF       värdets adress -> referens
1484  E5                PUSH HL
1485  01 04 00          LD   BC,$0004
1488  60                LD   H,B
1489  68                LD   L,B
148A  39                ADD  HL,SP
148B  CD 9D 11          CALL DVPUT        referens och namn, 4 bytes
148E  E1                POP  HL
148F  E1                POP  HL
1490  E1                POP  HL
1491  D1                POP  DE
1492  03        L1492:  INC  BC           en post till
1493  7E                LD   A,(HL)
1494  23                INC  HL
1495  66                LD   H,(HL)
1496  6F                LD   L,A
1497  18 D9             JR   L1472
```

LOAD fil / MERGE fil: LOAD sätter IY+$1D bit 3 och 4 (S0F40): allt nollställs, även COMMON. Filen öppnas som .BAC och annars .BAS. Första raden läses som text (S164A): lyckas det är filen text, och varje rad matas in som från tangentbordet (DOLINE); en felaktig rad skrivs ut och sparas som felrad, rad utan nummer ger fel 233 och avbryter, ett tecken med bit 7 ger fel 58 och avbryter. Ger första raden fel 58 är filen binär: MERGE fel 204, LOAD läser programmet (S15C2) och bygger upp VARLST ur posterna efter det. Programmet kommer länkat, med variablerna nollställda.

```
1499  CD 40 0F  LOAD:   CALL S0F40
149C  F6                DEFB $F6          OR $AF (överlappar nästa)
149D  AF          MERGE:  XOR  A
149E  CD 05 11            CALL PRGCLS
14A1  F5                  PUSH AF
14A2  3E 02               LD   A,$02
14A4  CD 35 10            CALL OPEN
14A7  F1                  POP  AF
14A8  F5          S14A8:  PUSH AF
14A9  FD 36 27 00         LD   (IY+$27),$00 IY+$27 = 0
14AD  FD CB 23 D6         SET  2,(IY+$23)   kommandoläge medan första raden läses
14B1  DD E5               PUSH IX           FCB:n undan
14B3  C4 C9 0E            CALL NZ,S0EC9     LOAD: NEW först
14B6  DD E1               POP  IX
14B8  CD 4A 16            CALL S164A        första raden
14BB  C1                  POP  BC           C = flaggorna (Z för MERGE)
14BC  38 15               JR   C,L14D3
14BE  FD CB 23 96         RES  2,(IY+$23)   text: raderna matas in
```

```
14C2  DD E5       L14C2:  PUSH IX             nästa rad
14C4  CD DC 11            CALL DOLINE
14C7  DD E1               POP  IX
14C9  D4 4A 16            CALL NC,S164A
14CC  30 F4               JR   NC,L14C2
14CE  FE 22       L14CE:  CP   $22            fel 34 (filslut) = klart
14D0  C8                  RET  Z
14D1  37                  SCF
14D2  C9                  RET

14D3  FE 3A       L14D3:  CP   $3A            fel 58: binär fil?
14D5  20 F7               JR   NZ,L14CE
14D7  CB 71               BIT  6,C            MERGE (Z i flaggorna)?
14D9  3E CC               LD   A,$CC          fel 204: MERGE av binär fil
14DB  20 F1               JR   NZ,L14CE
14DD  CD 75 13            CALL NEWSTK         programmet under NEWSTK:s stack
14E0  CD C2 15            CALL S15C2
14E3  D8                  RET  C
14E4  22 06 FF            LD   (PRGBEG),HL    PRGBEG = HL
14E7  D5                  PUSH DE
14E8  E5                  PUSH HL
14E9  CD 42 34            CALL NAMSET         PRGEND, namnlistan
14EC  E1                  POP  HL
14ED  11 0A 00            LD   DE,$000A       +10: antalet variabler
14F0  19                  ADD  HL,DE
14F1  4E                  LD   C,(HL)
14F2  23                  INC  HL
14F3  46                  LD   B,(HL)
14F4  21 2C FF            LD   HL,$FF2C       HL = VARLST, länken som fylls
14F7  78          L14F7:  LD   A,B
14F8  B1                  OR   C
14F9  28 22               JR   Z,L151D
14FB  C5                  PUSH BC
14FC  E5                  PUSH HL
14FD  ED 62               SBC  HL,HL          två ord 0 på stacken: referens och namn
14FF  E5                  PUSH HL
1500  E5                  PUSH HL
1501  39                  ADD  HL,SP
1502  01 04 00            LD   BC,$0004
1505  CD A0 11            CALL DVGET          referens och namn
1508  C1                  POP  BC             BC = referensen
1509  CD 68 0F            CALL REFADR         referens -> adress
150C  C1                  POP  BC             BC = namnet
150D  2B                  DEC  HL
150E  70                  LD   (HL),B
150F  2B                  DEC  HL
1510  71                  LD   (HL),C
1511  2B                  DEC  HL
1512  2B                  DEC  HL
1513  EB                  EX   DE,HL          DE = posten (länken)
1514  E1                  POP  HL
1515  73                  LD   (HL),E         länka in posten i VARLST
1516  23                  INC  HL
1517  72                  LD   (HL),D
1518  EB                  EX   DE,HL
1519  C1                  POP  BC
151A  0B                  DEC  BC
151B  18 DA               JR   L14F7

151D  71          L151D:  LD   (HL),C         sista länken = 0
151E  23                  INC  HL
151F  70                  LD   (HL),B
1520  E1                  POP  HL             HL = slutet på programmet
1521  0E 06               LD   C,$06
1523  09                  ADD  HL,BC
1524  7E                  LD   A,(HL)
1525  23                  INC  HL
1526  66                  LD   H,(HL)
1527  6F                  LD   L,A
```

```
1528  F9              LD   SP,HL          SP = den gamla SP, som NEWSTK sparade vid programmets slut
1529  AF              XOR  A
152A  C9              RET
```

LSTLIN: avkoda raden HL till text i en buffert på stacken ($10E bytes, DLINE) och skriv den, till filen (DVPRT) eller till skärmen (PRCON). På skärmen räknas skärmraderna (radlängd FCB+8) ned i +$16. Vid 0 väntas på en tangent: blanksteg ger en rad till, en annan tangent avbryter listningen. Tangenten sparas (KEYSET) och blir första tecknet på en ny inmatningsrad med den sist skrivna raden som mall (L12AA, samma väg som ED). Stacken lämnas; kommandoslingan återställer den.

```
152B  EB        LSTLIN: EX   DE,HL
152C  21 F2 FE          LD   HL,$FEF2
152F  39                ADD  HL,SP
1530  F9                LD   SP,HL
1531  EB                EX   DE,HL
1532  D5                PUSH DE
1533  DD E5             PUSH IX
1535  DD 46 17          LD   B,(IX+$17)
1538  CD BB 40          CALL DLINE
153B  DD E1             POP  IX
153D  DD 70 17          LD   (IX+$17),B
1540  E3                EX   (SP),HL
1541  EB                EX   DE,HL
1542  A7                AND  A
1543  ED 52             SBC  HL,DE
1545  EB                EX   DE,HL
1546  4B                LD   C,E
1547  42                LD   B,D
1548  DD CB 03 46       BIT  0,(IX+$03)     öppen fil?
154C  28 05             JR   Z,L1553
154E  CD 9A 11          CALL DVPRT
1551  18 30             JR   L1583

1553  E5        L1553:  PUSH HL
1554  C5                PUSH BC
1555  DD E5             PUSH IX
1557  CD 96 11          CALL PRCON
155A  DD 5E 08          LD   E,(IX+$08)
155D  DD E1             POP  IX
155F  C1                POP  BC
1560  E1                POP  HL
1561  79                LD   A,C
1562  D6 02             SUB  $02            skärmrader i radens längd
1564  BF                CP   A
1565  C4 87 01  L1565:  CALL NZ,KEYSET
1568  DD 35 16  L1568:  DEC  (IX+$16)
156B  FA AA 12          JP   M,L12AA        under 0: ny inmatningsrad med mall
156E  20 10             JR   NZ,L1580
1570  DD E5             PUSH IX
1572  CD 90 FF          CALL GETKYV         vänta på tangent
1575  1E FF             LD   E,$FF
1577  DD E1             POP  IX
1579  FE 20             CP   $20            blanksteg?
157B  20 E8             JR   NZ,L1565       nej: KEYSET, sedan avbrott
157D  DD 34 16          INC  (IX+$16)       en rad till
1580  93        L1580:  SUB  E
1581  30 E5             JR   NC,L1568
1583  D1        L1583:  POP  DE
1584  21 0E 01          LD   HL,$010E       bufferten bort från stacken
1587  39                ADD  HL,SP
1588  F9                LD   SP,HL
1589  EB                EX   DE,HL
158A  C9                RET
```

LNRNG: radintervall vid HL -> DE = första, BC = sista (0 och 65535 om de saknas). Skiljetecken -, , eller TO ($09F9). Annat skiljetecken, text efter intervallet eller första > sista ger fel 209. Används av LIST/ED, ERASE och RENUM.

```
158B  7E        LNRNG:  LD   A,(HL)
158C  11 00 00          LD   DE,$0000
158F  01 FF FF          LD   BC,$FFFF
```

```
1592  FE 0D              CP   $0D
1594  C8                 RET  Z
1595  CD C2 4B           CALL GETLNR
1598  D5                 PUSH DE
1599  7E                 LD   A,(HL)
159A  FE 0D              CP   $0D
159C  28 16              JR   Z,L15B4
159E  11 F9 09           LD   DE,$09F9       TO
15A1  CD 4B 4B           CALL KWSRCH
15A4  28 08              JR   Z,L15AE
15A6  7E                 LD   A,(HL)
15A7  D6 2C              SUB  $2C            - eller ,
15A9  E6 FE              AND  $FE
15AB  20 13              JR   NZ,L15C0
15AD  DF                 RST  $18
15AE  CD C2 4B     L15AE: CALL GETLNR
15B1  30 01              JR   NC,L15B4
15B3  1B                 DEC  DE             utan andra nummer: 65535
15B4  42           L15B4: LD   B,D
15B5  4B                 LD   C,E
15B6  7E                 LD   A,(HL)
15B7  FE 0D              CP   $0D
15B9  20 05              JR   NZ,L15C0
15BB  EB                 EX   DE,HL
15BC  D1                 POP  DE
15BD  ED 52              SBC  HL,DE          första > sista: fel
15BF  D0                 RET  NC
15C0  D7           L15C0: RST  $10          fel 209: fel i radintervallet
15C1  D1                 DEFB $D1
```

S15C2: *läs ett binärt program (IX = FCB, första blocket i bufferten). Första byten* $8F*, annars fel 203. Har filen ett* COMMON*-område (länkat program, +12 <> 0) och finns redan ett, måste storleken (+12) och summan* COMSUM *(+14) stämma, annars fel 205; finns inget tas det med* ALLOCC*. Summan räknas ur storlekarna, inte namnen:* COMMON *B i filen tar emot* COMMON *A:s värde,* COMMON *A% gör det inte. Flaggorna:* DOUBLE *(bit 2) tas ur filen, fel 211 om det skiljer och flyttalsvariabler finns (*S3A78*); bit 0, 1, 6, 7 ORas in. Program (+4) och variabler (+6) måste rymmas under SP, annars fel 201. Vid fel finns inget program kvar (*LOAD *och* CHAIN *har redan raderat det).*

```
15C2  DD 5E 13     S15C2: LD   E,(IX+$13)(egenhet)
15C5  DD 56 14            LD   D,(IX+$14)
15C8  1A                  LD   A,(DE)
15C9  FE 8F               CP   $8F
15CB  37                  SCF
15CC  3E CB               LD   A,$CB          fel 203: inte ett BASIC II-program
15CE  C0                  RET  NZ
15CF  D5                  PUSH DE
15D0  DD E3               EX   (SP),IX
15D2  DD CB 02 46         BIT  0,(IX+$02)
15D6  28 2E               JR   Z,L1606
15D8  DD 5E 0C            LD   E,(IX+$0C)
15DB  DD 56 0D            LD   D,(IX+$0D)
15DE  7A                  LD   A,D
15DF  B3                  OR   E
15E0  28 24               JR   Z,L1606
15E2  CD 58 0F            CALL S0F58          COMMON-område redan?
15E5  DD 4E 0E            LD   C,(IX+$0E)
15E8  DD 46 0F            LD   B,(IX+$0F)
15EB  28 10               JR   Z,L15FD
15ED  ED 52               SBC  HL,DE
15EF  3E CD               LD   A,$CD          fel 205: storleken skiljer
15F1  20 07               JR   NZ,L15FA
15F3  2A 32 FF            LD   HL,(COMSUM)
15F6  ED 42               SBC  HL,BC          summan
15F8  28 0C               JR   Z,L1606
15FA  E1           L15FA: POP  HL
15FB  37                  SCF
15FC  C9                  RET

15FD  CD AE 0F     L15FD: CALL ALLOCC         inget: ta det
1600  38 F8               JR   C,L15FA
```

```
1602  ED 43 32 FF          LD   (COMSUM),BC
1606  DD 4E 02     L1606:  LD   C,(IX+$02)
1609  3E 18                LD   A,$18
160B  A1                   AND  C
160C  79                   LD   A,C
160D  C4 78 3A             CALL NZ,S3A78          DOUBLE ur filen
1610  3E C3                LD   A,$C3
1612  A1                   AND  C
1613  CD 8E 3A             CALL S3A8E             bit 0, 1, 6, 7
1616  CD 33 27             CALL IXBC4             BC = +4, programmets längd
1619  DD 5E 06             LD   E,(IX+$06)        DE = +6, variablernas storlek
161C  DD 56 07             LD   D,(IX+$07)
161F  21 00 00             LD   HL,$0000
1622  39                   ADD  HL,SP
1623  3E C9                LD   A,$C9             fel 201: ryms inte
1625  ED 42                SBC  HL,BC
1627  38 D1                JR   C,L15FA
1629  ED 52                SBC  HL,DE
162B  38 CD                JR   C,L15FA
162D  CD F6 0F             CALL SPCHK
1630  38 C8                JR   C,L15FA
1632  19                   ADD  HL,DE
1633  E5                   PUSH HL
1634  CD D3 0F             CALL ALLOC             variablerna nollställda
1637  22 2E FF             LD   (VARBEG),HL
163A  E1                   POP  HL
163B  DD E1                POP  IX
163D  D1                   POP  DE
163E  F9                   LD   SP,HL             programmet överst, under stacken
163F  D5                   PUSH DE
1640  E5                   PUSH HL
1641  C5                   PUSH BC
1642  CD A0 11             CALL DVGET             läs programmet
1645  E1                   POP  HL
1646  D1                   POP  DE
1647  19                   ADD  HL,DE
1648  EB                   EX   DE,HL
1649  C9                   RET
```

S164A: läs en textrad från filen IX till bufferten (BUFADR, DVINP). *Ut: HL = bufferten, CY och A = felet* (*34 vid filslut, 58 vid ett tecken med bit 7*). *Förstör AF, BC och DE.*

```
164A  CD 51 00     S164A:  CALL BUFADR
164D  E5                   PUSH HL
164E  CD 51 49             CALL DVINP
1651  E1                   POP  HL
1652  C9                   RET
```

ERASE *intervall: radera raderna i intervallet* (*som* LIST, LNRNG); *utan argument ingenting. Variablerna nollställs* (CLEAR).

```
1653  7E           ERASE:  LD   A,(HL)
1654  FE 0D                CP   $0D
1656  C8                   RET  Z
1657  CD 8B 15             CALL LNRNG
165A  C5                   PUSH BC
165B  D5                   PUSH DE
165C  CD E7 0E             CALL CLEAR             CLEAR
165F  D1                   POP  DE
1660  CD 8C 32             CALL FNDLIN
1663  D1                   POP  DE
1664  E5                   PUSH HL
1665  CD 8C 32             CALL FNDLIN
1668  20 04                JR   NZ,L166E          finns sista raden: ta med den
166A  4E                   LD   C,(HL)
166B  06 00                LD   B,$00
166D  09                   ADD  HL,BC
166E  EB           L166E:  EX   DE,HL
166F  2A 08 FF             LD   HL,(PRGEND)
1672  A7                   AND  A
```

```
1673  ED 52              SBC  HL,DE
1675  23                 INC  HL
1676  4D                 LD   C,L
1677  44                 LD   B,H
1678  EB                 EX   DE,HL
1679  D1                 POP  DE
167A  C3 23 0F           JP   L0F23          flytta resten ned (L0F23)
```

LINLNG: *fel 20, koden ryms inte i radbufferten (från* EMIT*, med CY satt).*

```
167D  3E 14       LINLNG: LD   A,$14         fel 20: koden ryms inte (raden för lång)
167F  C9                  RET
```

CEXPA: *uttryck av valfri typ följt av koden* $B7+*typ.* $B7-$B9 *är "returnera värde"* (FRETF/FRETI/FRETS)*; RETURN uttryck behåller den* (RETURN *5 -> A0 D6 83 20 B7), de flesta andra anropare tar bort den igen* (DEC *DE). Parentes blir* $A7*, unärt minus* $A8*. In: HL = texten, DE = platsen i koden. Ut: NC: B = typen, HL efter uttrycket, DE efter* $B7+*typ; CY: A = felet. Förstör AF, C, IX och de alternativa registren.*

```
1680  CD B0 37    CEXPA:  CALL CEXPR         B = typen
1683  D8          L1683:  RET  C             L1683: fel: CY och A = felet
1684  3E B7       S1684:  LD   A,$B7         S1684: slutkoden $B7+typ
1686  80          L1686:  ADD  A,B           L1686: A = koden + typen
```

EMIT: *lägg koden A vid DE; kontrollera att utrymmet* (VFF1A) *inte är fullt, annars fel 20* ($14)*. Gränsen* VFF1A *sätts i* DOLINE*: en rad med 250 bytes kod går, en med 254 ger fel 20. In: A = koden, DE = platsen, CY = 0 (SBC). Ut: DE + 1. Förstör F.*

```
1687  E5          EMIT:   PUSH HL
1688  2A 1A FF            LD   HL,(VFF1A)    sista byte som får skrivas
168B  ED 52               SBC  HL,DE         CY: DE ligger efter gränsen
168D  E1                  POP  HL
168E  38 ED               JR   C,LINLNG      fel 20
1690  12                  LD   (DE),A
1691  13                  INC  DE
1692  C9                  RET
```

CMID: MID$(*a$,s*[,*n*]) = *b$. Koden:* $28 *och argumenten som för funktionen* MID$ (FNARG*-posten* $A8)*,* $B9*, =* ($3D)*, strängen och* $B9*. Ett fel i argumenten ger ett felnummer som är tecknets kod vid HL i stället för det riktiga felet (se Fel och egenheter). In: HL efter* MID$*, DE = koden. Ut: som* CEXPT*, B = 2. Förstör som* CEXPA*.*

```
1693  3E 28       CMID:   LD   A,$28          $A8 = MID$-satsens post i FNARG
1695  CD 1E 18            CALL S181E         $28, sedan a$, s, n och $D3 (CARGS), B = 2
1698  D4 84 16            CALL NC,S1684      $B9
169B  7E                  LD   A,(HL)        = läggs in; vid fel skrivs A (felet) över (fel)
169C  12                  LD   (DE),A
169D  13                  INC  DE
169E  D4 A5 17            CALL NC,CEQUAL     = hoppas över; B = 2 från MID$-posten
16A1  D8                  RET  C
16A2  18 06               JR   CEXPT         strängen och $B9
```

Kompilera ett uttryck av viss typ: CEXPI *heltal (B=1),* CEXPF *flyttal (B=0),* CEXPS ($001B) *sträng (B=2).* CEXPT*: typen i B,* CTYPE *lägger in typomvandling* ($81 FTOI*,* $82 ITOF*) eller ger fel 223 (sträng väntas, tal givet: A$=B,* LET *A$=1) eller 224 (tal väntas, sträng given: A=B$,* IF *A$* THEN*). Sist läggs* $B7+*typ (se* CEXPA*). In: HL = texten, DE = koden;* CEXPT *också B = typen som behövs. Ut: NC: B = den typen, DE efter* $B7+*typ; CY: A = felet, B oförändrad. Förstör AF, C, IX och de alternativa registren.*

```
16A4  06 01       CEXPI:  LD   B,$01
16A6  18 02               JR   CEXPT

16A8  06 00       CEXPF:  LD   B,$00
16AA  C5          CEXPT:  PUSH BC            typen som behövs
16AB  CD B0 37            CALL CEXPR
16AE  DA 2F 17            JP   C,L172F       fel: släpp BC
16B1  F1                  POP  AF            A = typen som behövs
16B2  CD C1 43            CALL CTYPE         $81/$82 eller fel 223/224
16B5  18 CC               JR   L1683         $B7+typ
```

CLET: *variabel = uttryck. Enkel variabel: uttrycket följt av* $B4+*typ och variabelns referens (A=1 -> D5 81 B4 .. ..). Fältelement: indexuttrycken,* $C8+*typ, antal index, referensen, uttrycket och* $B1+*typ. Fel 228 utan =, fel 221 för mer efter uttrycket (*"A=B,C"*). In: HL vid variabeln, DE = koden. Ut: NC: DE efter koden, HL efter uttrycket; CY: A = felet. Förstör AF, BC, IX och*

de alternativa registren.

```
16B7  CD 93 17     CLET:   CALL CVAR          BC = namnets kod, A = antal index
16BA  D8           L16BA:  RET  C
16BB  20 1A                JR   NZ,L16D7      NZ: fältelement
16BD  CD A5 17             CALL CEQUAL        = hoppas över, annars fel 228
16C0  D8                   RET  C
16C1  C5                   PUSH BC            namnets kod
16C2  41                   LD   B,C           B = C, CTYPE tar typen ur bit 0-1
16C3  CD AA 16             CALL CEXPT
16C6  38 67                JR   C,L172F
16C8  1B                   DEC  DE            stryk $B7+typ
16C9  3E B4                LD   A,$B4         $B4+typ och referensen
16CB  80                   ADD  A,B
16CC  12                   LD   (DE),A
16CD  13                   INC  DE
16CE  E3                   EX   (SP),HL       HL = namnets kod, texten vid SP
16CF  EB                   EX   DE,HL
16D0  73                   LD   (HL),E        namnets kod, C först
16D1  23                   INC  HL
16D2  72                   LD   (HL),D
16D3  23                   INC  HL
16D4  EB                   EX   DE,HL
16D5  E1                   POP  HL            texten
16D6  C9                   RET

16D7  CD 85 17     L16D7:  CALL CVREFX        $C8+typ, antal index, referens
16DA  CD A5 17             CALL CEQUAL        = ?
16DD  D8                   RET  C
16DE  41                   LD   B,C           typen ur C
16DF  CD AA 16             CALL CEXPT
16E2  D8                   RET  C
16E3  1B                   DEC  DE            stryk $B7+typ, lägg $B1+typ
16E4  3E B1                LD   A,$B1
16E6  18 9E                JR   L1686         $B1+typ (EMIT)
```

CDIM/CCOMMN: *per variabel (med , mellan) först gränserna (*CARGS*, signatur* $08B5*): ett index ger uttrycket och* $B0*,* a:b *två uttryck utan* $B0*. För en sträng följer längden efter = (heltal), utan = $C3. Sedan* $CD *(*DIM*) eller* $CE *(*COMMON*) och referensen (*$C4+*typ eller* $C8+*typ, antal); *COMMON *har två bytes till efter referensen. Exempel:* DIM *A(2:5) -> D5 82 81 D6 83 20 81 CD C8 01 .. ..;* COMMON *A(2:5) -> ... CE C8 01 .. .. 00 00.* DIM A(5)=3 *fel 221. In: HL efter* DIM *eller* COMMON*, DE = koden. Ut: NC: HL och DE efter den sista variabeln; CY: A = felet. Förstör AF, BC, IX, AF' och de alternativa registren.*

```
16E8  3E CD        CDIM:   LD   A,$CD
16EA  01                   DEFB $01           LD BC,$CE3E (överlappar nästa)
16EB  3E CE        CCOMMN: LD   A,$CE
16ED  FD 77 20             LD   (IY+$20),A    $CD eller $CE
16F0  CD 3E 33     L16F0:  CALL VNAME         L16F0: namnet; en funktion ger fel 234
16F3  D8                   RET  C
16F4  C5                   PUSH BC            namnets kod
16F5  DD 21 B5 08          LD   IX,$08B5      index med : (signatur $08B5)
16F9  CD 8D 18             CALL CARGS         gränserna, C = antalet
16FC  38 31                JR   C,L172F       fel
16FE  E3                   EX   (SP),HL
16FF  CB 4D                BIT  1,L           strängvariabel?
1701  E3                   EX   (SP),HL
1702  28 09                JR   Z,L170D       tal: ingen längd
1704  CD A5 17             CALL CEQUAL        = efter en sträng?
1707  30 1E                JR   NC,L1727      =: längden (L1727)
1709  3E C3                LD   A,$C3         utan =: $C3 (standardlängden)
170B  12                   LD   (DE),A
170C  13                   INC  DE
170D  FD 7E 20     L170D:  LD   A,(IY+$20)    $CD/$CE
1710  12                   LD   (DE),A
1711  13                   INC  DE
1712  08                   EX   AF,AF'        spara $CD/$CE
1713  79                   LD   A,C           A = antalet gränser
1714  C1                   POP  BC            namnets kod
1715  A7                   AND  A             Z: inga gränser, $C4+typ
1716  CD 70 17             CALL CVREF         $C4+typ eller $C8+typ, antal, referens
```

```
1719  08              EX   AF,AF'
171A  FE CE           CP   $CE          COMMON: två bytes till
171C  20 02           JR   NZ,L1720
171E  13              INC  DE           de två bytes efter referensen (00 00 före RUN)
171F  13              INC  DE
1720  7E       L1720: LD   A,(HL)       ,: nästa variabel
1721  EE 2C           XOR  $2C
1723  C0              RET  NZ
1724  DF              RST  $18          hoppa över ,
1725  18 C9           JR   L16F0

1727  C5       L1727: PUSH BC           =: längden, ett heltalsuttryck
1728  CD A4 16        CALL CEXPI        längden, ett heltalsuttryck
172B  1B              DEC  DE           stryk $B7+typ
172C  C1              POP  BC
172D  30 DE           JR   NC,L170D     vidare med $CD/$CE
172F  C1       L172F: POP  BC           fel: släpp BC
1730  37              SCF
1731  C9              RET
```

S1732: enkel variabel som inte är en sträng (FOR, NEXT), annars fel 225 (FOR A(1), FOR A$, NEXT A(1)). NEXT 5 och INPUT 5 ger fel 234 (bokstav väntas) redan i CVAR. In: HL vid variabeln, DE = koden. Ut: NC: BC = namnets kod, A = 0; CY: A = felet. Förstör F, IX och de alternativa registren. Referensen läggs inte ut (se S173F, CVREF).

```
1732  CD 93 17  S1732: CALL CVAR          BC = namnets kod, NZ: index
1735  D8               RET  C
1736  37               SCF
1737  3E E1            LD   A,$E1         fel 225: variabel väntas
1739  C0               RET  NZ            index: fel 225
173A  CB 49            BIT  1,C           strängvariabel?
173C  C0               RET  NZ            sträng: fel 225 (CY från SCF)
173D  AF               XOR  A             NC
173E  C9               RET

173F  CD 32 17  S173F: CALL S1732         S173F: FOR-variabeln = uttryck (som CLET)
1742  C3 BA 16         JP   L16BA
```

$1745: en talvariabel att läsa till, som S174A men med S1732 (CALL S1732; JR $174D). Ingen kod i ROM:en hänvisar hit.

```
1745  CD 32 17 18…     DEFB $CD,$32,$17,$18,$03
```

S174A: variabel att läsa till (INPUT, READ): referensen och $BA+typ. S175F: dito, men bara strängar (INPUT LINE, GET); annars fel 224 (INPUT LINE A, GET A, GET A%). In: HL vid variabeln, DE = koden. Ut: NC: B = typen, DE efter $BA+typ; CY: A = felet. Förstör AF, C och IX.

```
174A  CD 93 17  S174A: CALL CVAR          S174A: variabel att läsa till (INPUT, READ)
174D  D8               RET  C
174E  CD 70 17         CALL CVREF         referensen, B = typen
1751  3E BA    L1751: LD   A,$BA         $BA+typ
1753  C3 86 16         JP   L1686
```

$1756: en heltalsvariabel att läsa till, annars fel 224 (CALL S176C; RET C; LD A,B; CP 1; JR $1766). Ingen kod i ROM:en hänvisar hit.

```
1756  CD 6C 17 D8…     DEFB $CD,$6C,$17,$D8,$78,$FE,$01,$18
175E  07               DEFB $07
175F  CD 6C 17  S175F: CALL S176C         S175F: strängvariabel (INPUT LINE, GET)
1762  D8               RET  C
1763  78               LD   A,B
1764  FE 02            CP   $02           strängvariabel?
1766  3E E0            LD   A,$E0         fel 224: strängvariabel väntas
1768  37               SCF
1769  C0               RET  NZ
176A  18 E5            JR   L1751         $BA+2
```

S176C: variabel (CVAR) och dess referens (CVREF). CVREF: lägg ut referensen för namnet BC efter CVAR: utan index (Z) $C4+typ och namnets kod, med index (NZ) CVREFX. CVREFX: $C8+typ, antalet index A och namnets kod med bit 2 i C satt (A(1) -> C8 01 7C 01). L1778: A vid DE och namnets kod efter. In: HL vid variabeln, DE = koden. Ut: NC: B = typen, DE efter

referensen; CY: A = felet. Förstör AF, C och IX.

```
176C  CD 93 17     S176C:  CALL  CVAR          S176C: variabelreferens
176F  D8                   RET   C
1770  C2 85 17     CVREF:  JP    NZ,CVREFX     NZ: fältelement
1773  3E 03                LD    A,$03
1775  A1                   AND   C
1776  C6 C4                ADD   A,$C4         $C4+typ, referensen
1778  12           L1778:  LD    (DE),A        L1778: koden, sedan namnets kod
1779  EB           L1779:  EX    DE,HL
177A  23           L177A:  INC   HL
177B  71                   LD    (HL),C
177C  23                   INC   HL
177D  70                   LD    (HL),B
177E  23                   INC   HL
177F  EB                   EX    DE,HL
1780  3E 03                LD    A,$03         B = typen
1782  A1                   AND   C
1783  47                   LD    B,A
1784  C9                   RET

1785  13           CVREFX: INC   DE            CVREFX: $C8+typ, antal index (A), referens
1786  12                   LD    (DE),A
1787  1B                   DEC   DE
1788  3E 03                LD    A,$03
178A  A1                   AND   C
178B  C6 C8                ADD   A,$C8
178D  12                   LD    (DE),A
178E  13                   INC   DE
178F  CB D1                SET   2,C           C bit 2: fältelement
1791  18 E6                JR    L1779
```

CVAR: variabelnamn (VNAME) och index inom parentes (CARGS med signaturen $08B4: heltal, valfritt antal). CVREF lägger ut referensen: $C4+typ + 2 bytes för en enkel variabel, indexuttrycken + $C8+typ + antal + 2 bytes för ett fältelement. Som värde (S17CF) blir det $C0+typ + 2 bytes, för element $7E+typ efter referensen. In: HL vid namnet, DE = koden (indexuttrycken läggs där). Ut: NC: BC = namnets kod (C bit 0-1 = typen), A = antalet index, Z = inga, HL efter namnet och indexen; CY: A = felet. Förstör IX och de alternativa registren. Före RUN är referensen i koden namnets kod, C först (A = 78 01, B9% = 49 02); RUN gör om den till ett avstånd (REFADR).

```
1793  CD 3E 33     CVAR:   CALL  VNAME         BC = namnets kod; en funktion ger fel 234
1796  D8                   RET   C
1797  C5                   PUSH  BC            namnets kod
1798  DD 21 B4 08          LD    IX,$08B4      index: heltal, valfritt antal (signatur $08B4)
179C  CD 8D 18             CALL  CARGS
179F  38 8E                JR    C,L172F       fel: släpp BC
17A1  79                   LD    A,C           Z = inga index, C = typ
17A2  C1                   POP   BC
17A3  A7                   AND   A
17A4  C9                   RET
```

CEQUAL: = vid HL hoppas över (med blanka efter); annars CY och A = 228. Förstör AF.

```
17A5  7E           CEQUAL: LD    A,(HL)        CEQUAL: =?
17A6  FE 3D                CP    $3D
17A8  37                   SCF
17A9  3E E4                LD    A,$E4         fel 228: = väntas
17AB  C0                   RET   NZ
17AC  C3 19 4C             JP    S4C19         = hoppas över
```

CFACT: en operand. Siffra eller punkt -> tal (L1926), (-> uttryck +) ($A7), citattecken -> strängkonstant, funktionsnamn -> CFUNC, annars variabel (S17CF).

Strängkonstant: $CB (") eller $CC (') + längd + tecknen. Ett dubblerat avgränsningstecken blir ett tecken ("A""B" -> CB 03 41 22 42), och konstanten får sluta vid radslutet utan avslutande tecken. "" = CB 00. In: HL vid operanden, DE = koden. Ut: NC: B = typen, HL efter operanden, DE efter koden; CY: A = felet. Förstör AF, C, IX och de alternativa registren.

```
17AF  7E           CFACT:  LD    A,(HL)
17B0  D6 2E                SUB   $2E           punkt: 0
17B2  28 02                JR    Z,L17B6
17B4  C6 FE                ADD   A,$FE         siffror: 0-9
```

```
17B6  FE 0A       L17B6:  CP   $0A
17B8  DA 26 19            JP   C,L1926          siffra eller punkt: tal
17BB  FE F8               CP   $F8              (
17BD  28 27               JR   Z,L17E6
17BF  0E CB               LD   C,$CB            '"': $CB
17C1  FE F2               CP   $F2
17C3  28 35               JR   Z,L17FA
17C5  0C                  INC  C
17C6  FE F7               CP   $F7              ': $CC
17C8  28 30               JR   Z,L17FA
17CA  CD 9E 19            CALL S199E            ett funktionsnamn?
17CD  30 53               JR   NC,CFUNC
17CF  CD 93 17    S17CF:  CALL CVAR             S17CF: variabelns värde; BC = namnets kod
17D2  D8                  RET  C
17D3  20 08               JR   NZ,L17DD         NZ: fältelement
17D5  3E 03               LD   A,$03
17D7  A1                  AND  C
17D8  C6 C0               ADD  A,$C0            $C0+typ, referensen
17DA  C3 78 17            JP   L1778            och namnets kod (L1778)

17DD  CD 85 17    L17DD:  CALL CVREFX           $C8+typ, antal, namnets kod
17E0  3E 7E               LD   A,$7E            fältelement: $7E+typ
17E2  80                  ADD  A,B
17E3  12                  LD   (DE),A
17E4  13                  INC  DE
17E5  C9                  RET

17E6  DF          L17E6:  RST  $18              hoppa över ( och blanka
17E7  CD 80 16            CALL CEXPA            uttrycket
17EA  1B                  DEC  DE               stryk $B7+typ
17EB  D8                  RET  C
17EC  7E                  LD   A,(HL)
17ED  FE 29               CP   $29
17EF  3E E5               LD   A,$E5            fel 229: ) väntas
17F1  37                  SCF
17F2  C0                  RET  NZ
17F3  3E A7               LD   A,$A7            ): $A7
17F5  12                  LD   (DE),A
17F6  13                  INC  DE
17F7  C3 19 4C            JP   S4C19            hoppa över ) och blanka

17FA  79          L17FA:  LD   A,C              $CB/$CC, längd, tecknen
17FB  12                  LD   (DE),A
17FC  13                  INC  DE
17FD  4E                  LD   C,(HL)           C = avgränsningstecknet, B = längden
17FE  06 00               LD   B,$00
1800  23                  INC  HL
1801  D5                  PUSH DE               plats för längden
1802  13                  INC  DE
1803  7E          L1803:  LD   A,(HL)
1804  FE 0D               CP   $0D              CR: slutet citattecken behövs inte
1806  28 0E               JR   Z,L1816
1808  23                  INC  HL
1809  B9                  CP   C                avgränsningstecknet?
180A  28 05               JR   Z,L1811
180C  12          L180C:  LD   (DE),A
180D  13                  INC  DE
180E  04                  INC  B
180F  18 F2               JR   L1803

1811  BE          L1811:  CP   (HL)             dubblerat avgränsningstecken = ett tecken
1812  23                  INC  HL
1813  28 F7               JR   Z,L180C
1815  2B                  DEC  HL
1816  78          L1816:  LD   A,B              L1816: längden
1817  C1                  POP  BC
1818  02                  LD   (BC),A           längden
1819  06 02               LD   B,$02            B = 2: sträng
181B  C3 1A 4C            JP   SKIPB0           hoppa över blanka efter strängen
```

S181E: koden A vid DE, sedan funktionsargumenten för FNARG-posten A + $80 (som CFUNC med Z). Används av CMID. In: A = koden, HL vid (, DE = koden. Ut: som CFUNC.

```
181E  12          S181E:  LD   (DE),A        S181E: koden A, sedan argumenten enligt FNARG
181F  13                  INC  DE
1820  18 03               JR   L1825
```

CFUNC: funktionsanrop. S199E har lagt ut funktionskoden; CFUNC letar upp koden i huvudets signaturtabell (NZ; $0841 för de vanliga funktionerna) eller, när B = 0 (färgorden), koden C i FNARG, och kompilerar argumenten med CARGS. Koden blir argumenten, $D0+antal och funktionsnumret: PI = D0 08, SIN(X) = <X> D1 03. Med fyra eller fler argument läggs $D4 före (CHR$ med 4 argument: D4 D4 0C). FN (kod 0): argumenten, vart och ett följt av $AA+typ och ett avstånd bakåt till förra märket (0 för det första), sedan $CF, antal, avstånd till sista märket och referensen: FNX(1,B$) -> D5 81 AA 00 C2 .. .. AC FB CF 02 FD .. ..; FNX utan argument -> CF 00 00 .. .. In: Z och BC från S199E, funktionskoden vid DE - 1, HL efter namnet. Ut: NC: B = resultattypen, DE efter koden; CY: A = felet. Förstör AF, C, IX och de alternativa registren.

```
1822  20 04       CFUNC:  JR   NZ,L1828      NZ: BC = signaturtabellen
1824  79                  LD   A,C           Z: koden C ...
1825  01 41 08    L1825:  LD   BC,$0841      ... söks i FNARG
1828  E5          L1828:  PUSH HL            L1828: HL = tabellen
1829  60                  LD   H,B
182A  69                  LD   L,C
182B  F6 80               OR   $80           kodbytes har bit 7
182D  ED B1               CPIR               sök funktionskoden
182F  2B                  DEC  HL
1830  23          L1830:  INC  HL            hoppa över koderna ($7F-$FE) till signaturen
1831  4E                  LD   C,(HL)
1832  0C                  INC  C
1833  FA 30 18            JP   M,L1830
1836  E3                  EX   (SP),HL       HL = texten, IX = signaturen
1837  DD E1               POP  IX
1839  1B                  DEC  DE
183A  1A                  LD   A,(DE)        koden som S199E lade ut
183B  A7                  AND  A
183C  28 2F               JR   Z,L186D       0: FN
183E  FE 7E               CP   $7E           $7E: XFN n
1840  28 21               JR   Z,L1863
1842  4F                  LD   C,A           C = koden
1843  DD 7E 00            LD   A,(IX+$00)    resultattypen
1846  E6 03               AND  $03
1848  47                  LD   B,A
1849  C5                  PUSH BC            C = koden, B = resultattypen
184A  CD 8D 18            CALL CARGS         argumenten, C = antalet
184D  DA 23 19            JP   C,L1923
1850  79                  LD   A,C           A = antalet
1851  EB                  EX   DE,HL
1852  FE 04               CP   $04           4 eller fler argument: $D4 före
1854  38 03               JR   C,L1859
1856  36 D4               LD   (HL),$D4
1858  23                  INC  HL
1859  C6 D0       L1859:  ADD  A,$D0         $D0+antal, funktionskoden
185B  A7                  AND  A
185C  77                  LD   (HL),A
185D  23                  INC  HL
185E  C1                  POP  BC
185F  71                  LD   (HL),C
1860  23                  INC  HL
1861  EB                  EX   DE,HL
1862  C9                  RET

1863  ED 4B 7F FF L1863:  LD   BC,(FNKWP)    XFN: +8 tas som tabell (fel)
1867  CD 95 1B            CALL XNUM          n (låga byten) i koden
186A  30 BC               JR   NC,L1828      NC: sök som för en funktion
186C  C9                  RET

186D  CD 43 33    L186D:  CALL VARNAM        FN-namnet
1870  D8                  RET  C             fel
1871  C5                  PUSH BC            namnets kod
1872  DD 21 B6 08         LD   IX,$08B6      argument, valfritt antal (signatur $08B6)
1876  CD 8D 18            CALL CARGS
```

```
1879  DA 23 19            JP   C,L1923          fel: släpp BC
187C  EB                  EX   DE,HL
187D  36 CF               LD   (HL),$CF         $CF, antal, avstånd till sista $AA-märket
187F  23                  INC  HL
1880  71                  LD   (HL),C
1881  23                  INC  HL
1882  79                  LD   A,C
1883  B7                  OR   A
1884  28 02               JR   Z,L1888
1886  78                  LD   A,B              B = sista märkets låga byte (CARGS)
1887  95                  SUB  L
1888  77            L1888: LD   (HL),A           L1888: 0 utan argument
1889  C1                  POP  BC
188A  C3 7A 17            JP   L177A            referensen
```

CARGS: *argument inom parentes enligt signaturbytes från IX och framåt (C = antalet). Före första argumentet och efter varje argument utan bit 3 flyttas IX en byte. En signaturbyte:*

```
bit 0-1  typ (0 flyttal, 1 heltal, 2 sträng)
bit 2    specialargument: typ 0 -> $AA+typ (FN-märke), typ 1 -> $B0
         och efter ":" ett heltalsuttryck (DIM), typ 2 -> $83,
         typ 3 -> tilläggets krok HOOKA9
bit 3    upprepas (valfritt antal)
bit 4    variabel (S17CF för sträng, annars S176C)
bit 5    ")" (eller ingen parentes) får komma här
bit 6    ingen typomvandling (CTYPE)
bit 7    nästa post: för många argument
```

Fel 223 för fel antal argument och utan (när argument krävs (SIN 1, PI(), SIN(B,C)); fel 229 när) saknas. Typen prövas av CTYPE: fel 223 när en sträng väntas, fel 224 när ett tal väntas (LEN(1), SIN(B$)). In: IX = signaturens första byte (före argumenten), HL vid (eller det som följer, DE = koden. Ut: NC: C = antalet argument, HL efter), DE efter koden; B oförändrad utom vid FN-märken (B = det sista märkets låga byte); CY: A = felet. Förstör AF, IX och de alternativa registren.

```
188D  0E 00         CARGS: LD   C,$00          C = antalet argument
188F  3E 28                LD   A,$28          utan (: bit 5 avgör
1891  BE                  CP   (HL)
1892  C2 1A 19            JP   NZ,L191A
1895  DF            L1895: RST  $18            L1895: hoppa över ( eller , och blanka
1896  DD CB 00 5E         BIT  3,(IX+$00)      bit 3: samma signatur igen
189A  20 02               JR   NZ,L189E
189C  DD 23               INC  IX
189E  0C            L189E: INC  C              L189E: ett argument till
189F  DD 7E 00            LD   A,(IX+$00)
18A2  CB 7F               BIT  7,A             bit 7 (nästa post): fel 223
18A4  20 7A               JR   NZ,L1920
18A6  C5                  PUSH BC              antalet och B
18A7  DD E5               PUSH IX              signaturbyten
18A9  CB 67               BIT  4,A             bit 4: variabel
18AB  28 10               JR   Z,L18BD
18AD  E6 03               AND  $03             typen
18AF  EE 02               XOR  $02             sträng?
18B1  20 05               JR   NZ,L18B8
18B3  CD CF 17            CALL S17CF           strängvariabel: värdet ($C0+typ)
18B6  18 09               JR   L18C1

18B8  CD 6C 17      L18B8: CALL S176C          L18B8: talvariabel: referensen ($C4+typ)
18BB  18 04               JR   L18C1

18BD  CD 80 16      L18BD: CALL CEXPA          uttryck, utan $B7+typ
18C0  1B                  DEC  DE
18C1  DD E1         L18C1: POP  IX             L18C1: IX = signaturbyten
18C3  38 5E               JR   C,L1923         fel
18C5  DD 7E 00            LD   A,(IX+$00)      signaturbyten igen
18C8  4F                  LD   C,A             C = signaturbyten
18C9  CB 77               BIT  6,A             bit 6: ingen typomvandling
18CB  CC C1 43            CALL Z,CTYPE
18CE  38 53               JR   C,L1923         fel 223/224
18D0  79                  LD   A,C             A = signaturbyten
18D1  CB 57               BIT  2,A             bit 2: specialargument efter typen
18D3  28 37               JR   Z,L190C
18D5  E6 03               AND  $03             typ 1-3
```

```
18D7  20 1B              JR   NZ,L18F4
18D9  78                 LD   A,B          B = argumentets typ
18DA  C1                 POP  BC           C = antalet, B = förra märket
18DB  C6 AA              ADD  A,$AA        typ 0: FN-märke $AA+typ
18DD  12                 LD   (DE),A
18DE  13                 INC  DE
18DF  79                 LD   A,C          det första märket?
18E0  3D                 DEC  A
18E1  28 02              JR   Z,L18E5      ja: avstånd 0
18E3  78                 LD   A,B          avstånd bakåt till förra märket
18E4  93                 SUB  E
18E5  43          L18E5: LD   B,E          L18E5: B = märkets plats
18E6  12          L18E6: LD   (DE),A       L18E6: byten
18E7  13                 INC  DE
18E8  18 24              JR   L190E

18EA  3D          L18EA: DEC  A            L18EA: typ 2 eller 3
18EB  3E 83              LD   A,$83        typ 2: $83
18ED  28 F7              JR   Z,L18E6
18EF  CD A9 FF           CALL HOOKA9       typ 3: tilläggets krok
18F2  18 19              JR   L190D

18F4  C1          L18F4: POP  BC           L18F4: C = antalet
18F5  3D                 DEC  A            typ 1?
18F6  20 F2              JR   NZ,L18EA
18F8  3E B0              LD   A,$B0        typ 1: $B0
18FA  12                 LD   (DE),A
18FB  13                 INC  DE
18FC  3E 3A              LD   A,$3A        : : övre gränsen följer
18FE  AE                 XOR  (HL)
18FF  20 0D              JR   NZ,L190E
1901  DF                 RST  $18          hoppa över : och blanka
1902  1B                 DEC  DE           skriv över $B0
1903  C5                 PUSH BC
1904  DD E5              PUSH IX
1906  CD A4 16           CALL CEXPI        övre gränsen
1909  1B                 DEC  DE           stryk $B7+typ
190A  DD E1              POP  IX
190C  C1          L190C: POP  BC           L190C: C = antalet
190D  D8          L190D: RET  C            L190D: fel
190E  7E          L190E: LD   A,(HL)       ,: nästa argument
190F  FE 2C              CP   $2C
1911  28 82              JR   Z,L1895
1913  FE 29              CP   $29
1915  3E E5              LD   A,$E5        fel 229: ) väntas
1917  37                 SCF
1918  C0                 RET  NZ           fel 229
1919  DF                 RST  $18          hoppa över ) och blanka
191A  DD 7E 00    L191A: LD   A,(IX+$00)   bit 5: ) får komma här
191D  E6 20              AND  $20
191F  C0                 RET  NZ           NZ: inget fel, NC
1920  3E DF       L1920: LD   A,$DF        fel 223: fel antal argument
1922  C5                 PUSH BC           för L1923
1923  C1          L1923: POP  BC           L1923: släpp BC, CY
1924  37                 SCF
1925  C9                 RET
```

L1926: *talkonstant. Talet läses först till slutet (blanka hoppas över). Med % sist, eller i* INTEGER*-läge utan punkt, blir det ett heltal: 0-16 -> $DD+n, annars $C7 lo hi; över 65535 fel 210. Annars flyttal: talet tolkas alltid med 8 bytes (IY+$1E = 8 tillfälligt) och nollbytes sist stryks; koden blir $D4+n och n bytes (1 = D5 81, 0 = D5 00, 1.5 = D6 81 40; 123456789012 får 6 bytes även i* SINGLE*). Ett tal utan % är flyttal även när det tilldelas ett heltal (A%=1 -> D5 81 81).* 1.5% *ger fel 221, 1E40 fel 130. In: HL vid första siffran eller punkten, DE = koden. Ut: NC: B = 0 (flyttal) eller 1 (heltal), HL efter talet (och % och blanka), DE efter koden; CY: A = felet. Förstör AF, C och IX.*

```
1926  E5          L1926: PUSH HL           talets början
1927  2B                 DEC  HL
1928  23          L1928: INC  HL           L1928: blanka hoppas över
1929  7E                 LD   A,(HL)
192A  FE 20              CP   $20
```

```
192C  28 FA              JR   Z,L1928
192E  FE 25              CP   $25          %: heltal
1930  28 3A              JR   Z,L196C
1932  FE 2E              CP   $2E          punkt: flyttal
1934  28 0C              JR   Z,L1942
1936  D6 30              SUB  $30          siffra?
1938  FE 0A              CP   $0A
193A  38 EC              JR   C,L1928
193C  FD CB 1D 46        BIT  0,(IY+$1D)   INTEGER: tal utan punkt blir heltal
1940  20 2A              JR   NZ,L196C
1942  E1           L1942: POP  HL          L1942: HL = talets början
1943  D5                 PUSH DE           platsen för $D4+n
1944  13                 INC  DE           flyttalet efter den
1945  FD 46 1E           LD   B,(IY+$1E)   B = längden (SINGLE/DOUBLE)
1948  C5                 PUSH BC
1949  FD 36 1E 08        LD   (IY+$1E),$08 alltid 8 bytes
194D  CD 1D 48           CALL TXTNUM
1950  C1                 POP  BC
1951  FD 70 1E           LD   (IY+$1E),B   längden tillbaka
1954  DA 23 19           JP   C,L1923      fel 210 eller 130
1957  2B                 DEC  HL           tecknet efter talet
1958  DF                 RST  $18          hoppa över blanka
1959  06 08              LD   B,$08        stryk nollbytes sist (minst en kvar)
195B  1B           L195B: DEC  DE          L195B: sista byten
195C  1A                 LD   A,(DE)
195D  A7                 AND  A
195E  20 02              JR   NZ,L1962
1960  10 F9              DJNZ L195B        tom?: en till
1962  7B           L1962: LD   A,E         L1962: E = sista byten som inte är noll (eller den första)
1963  13                 INC  DE
1964  C1                 POP  BC           BC = platsen för $D4+n
1965  91                 SUB  C            $D4 + antalet bytes
1966  C6 D4              ADD  A,$D4
1968  02                 LD   (BC),A
1969  06 00              LD   B,$00
196B  C9                 RET               B = 0: flyttal

196C  E3           L196C: EX   (SP),HL     L196C: HL = talets början, SP = där det slutade
196D  D5                 PUSH DE
196E  CD C2 4B           CALL GETLNR       heltal: DE = talet
1971  42                 LD   B,D          BC = talet
1972  4B                 LD   C,E
1973  D1                 POP  DE
1974  3E D2              LD   A,$D2        fel 210: över 65535
1976  38 AB              JR   C,L1923      fel 210
1978  EB                 EX   DE,HL
1979  78                 LD   A,B          över 255?
197A  B0                 OR   B
197B  20 09              JR   NZ,L1986     ja: $C7 lo hi
197D  79                 LD   A,C          0-16: $DD+n
197E  C6 DD              ADD  A,$DD
1980  77                 LD   (HL),A       $DD+n (skrivs över om n är 17 eller mer)
1981  79                 LD   A,C
1982  D6 11              SUB  $11
1984  38 06              JR   C,L198C      under 17: klar
1986  36 C7        L1986: LD   (HL),$C7     $C7, lågbyte, högbyte
1988  23                 INC  HL
1989  71                 LD   (HL),C
198A  23                 INC  HL
198B  70                 LD   (HL),B
198C  23           L198C: INC  HL
198D  EB                 EX   DE,HL
198E  C1                 POP  BC           C = där talet slutade
198F  79                 LD   A,C
1990  BD                 CP   L            stämmer slutet? (låga byten) Med EXTEND stannar GETLNR vid en blank: 1
                                           2% ger fel 210
1991  3E D2              LD   A,$D2
1993  37                 SCF
1994  C0                 RET  NZ
```

```
1995  06 01              LD   B,$01          B = 1: heltal
1997  7E                 LD   A,(HL)         % hoppas över
1998  EE 25              XOR  $25
199A  C0                 RET  NZ
199B  C3 19 4C           JP   S4C19
```

S199E/S19A2: sök namnet vid HL bland funktionerna (S199E: kedjan från FNKWP*). In: HL vid namnet, DE = koden; S19A2 med tabellkedjan i HL'. Ut: NC = en funktion: koden skriven vid DE (DE + 1), HL efter namnet, BC = ordet efter tabellhuvudet (signaturtabellen, eller B = 0 och C = koden i FNARG), Z när B = 0; CY: inget funktionsnamn. Förstör AF och de alternativa registren.*

```
199E  D9           S199E:  EXX                 S199E: sök namnet bland funktionerna (FNKWP)
199F  2A 7F FF             LD   HL,(FNKWP)
19A2  D9           S19A2:  EXX                 S19A2: samma, tabellkedjan i HL'
19A3  D5                   PUSH DE             DE' sparas
19A4  D9                   EXX
19A5  CD 8F 34             CALL FNKSRC         sök i tabellerna
19A8  D9                   EXX
19A9  D1                   POP  DE
19AA  3F                   CCF                 CY: inte funnen
19AB  D8                   RET  C
19AC  D9                   EXX
19AD  23                   INC  HL
19AE  23                   INC  HL
19AF  F5                   PUSH AF
19B0  E6 7F                AND  $7F
19B2  86                   ADD  A,(HL)         koden: + tabellens första kod
19B3  C5                   PUSH BC             BC' ...
19B4  D9                   EXX
19B5  C1                   POP  BC             ... = BC
19B6  12                   LD   (DE),A         koden
19B7  13                   INC  DE
19B8  F1                   POP  AF
19B9  04                   INC  B              Z: B = 0
19BA  05                   DEC  B
19BB  C9                   RET
```

CLNR*: radnummer vid HL (*GETLNR*) som två bytes vid DE, låga först. Ut: NC: BC = numret, DE + 2, HL efter numret; CY och A = 233 (inget radnummer eller över 65535), DE oförändrad. Förstör AF.* NOHOOK*: en krok som inte används (bara RET).*

```
19BC  D5           CLNR:   PUSH DE
19BD  CD C2 4B             CALL GETLNR         DE = numret
19C0  42                   LD   B,D            BC = numret
19C1  4B                   LD   C,E
19C2  D1                   POP  DE
19C3  3E E9                LD   A,$E9          fel 233
19C5  D8                   RET  C
19C6  EB                   EX   DE,HL          två bytes
19C7  71                   LD   (HL),C
19C8  23                   INC  HL
19C9  70                   LD   (HL),B
19CA  23                   INC  HL
19CB  EB                   EX   DE,HL
19CC  C9           NOHOOK: RET
```

CRETRN: RETURN [*uttryck*]*. In: HL efter* RETURN*, DE = koden. Ut: som* CEXPA*; bara* RETURN *(Z) lägger inget.*

```
19CD  CD 01 1C     CRETRN: CALL EOSTM          CRETRN: RETURN [uttryck]
19D0  C8                   RET  Z              bara RETURN
19D1  C3 80 16             JP   CEXPA          uttrycket och $B7+typ
```

CIF*:* IF *villkor* THEN *radnr|satser* [ELSE *radnr|satser*]*. Koden:*

$95*, villkoret som heltal (efter ett flyttal* $81*,* FTOI*;* IF A% THEN 20 *-> 95 C1 .. .. BD 04 81 14 00), sedan* $BD *(*THEN*) eller* $BE *(utan* THEN*), en avståndsbyte, räknad från sig själv, till* ELSE*-delens kod (efter* $8A *och dess avståndsbyte) eller till radslutets* $87*, och* $81 *lo hi eller satserna (*CMPST2*). Utan* THEN *följer alltid satser:* IF A GOTO 20 *blir satsen* GOTO *(*$80 *lo hi), och* IF A B=1 *går.* ELSE*:* $8A*, ett avstånd till radslutet, radnummer eller satser.* : *efter* IF*-satsen ger fel 221 (*"IF A THEN 20 : B=1"*). Exempel:* IF *A* THEN *20* ELSE *30 -> 95 C0 .. .. 81 BD 06 81 14 00 8A 04 81 1E 00. In: HL efter* IF*, DE = koden. Ut: NC: HL efter*

satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.

```
19D4  CD A4 16     CIF:    CALL  CEXPI          villkoret: heltal
19D7  D8                   RET   C
19D8  D5                   PUSH  DE
19D9  11 1B 0A             LD    DE,$0A1B       THEN?
19DC  CD 4B 4B             CALL  KWSRCH         Z: hittat, A = $BD
19DF  D1                   POP   DE
19E0  28 02                JR    Z,L19E4
19E2  3E BE                LD    A,$BE          nej: $BE, satser följer
19E4  1B           L19E4:  DEC   DE             L19E4: över $B8 från CEXPI
19E5  12                   LD    (DE),A
19E6  13                   INC   DE
19E7  D5                   PUSH  DE             avståndet fylls i sist
19E8  13                   INC   DE
19E9  3E 81                LD    A,$81          $81 + radnummer
19EB  12                   LD    (DE),A
19EC  13                   INC   DE
19ED  CD BC 19             CALL  CLNR           radnumret
19F0  30 04                JR    NC,L19F6       fanns: klart
19F2  1B                   DEC   DE             inget radnummer: satser
19F3  CD 3A 13             CALL  CMPST2
19F6  C1           L19F6:  POP   BC             L19F6: BC = avståndsbyten
19F7  D8                   RET   C
19F8  C5                   PUSH  BC
19F9  D5                   PUSH  DE
19FA  11 ED 09             LD    DE,$09ED       ELSE?
19FD  CD 4B 4B             CALL  KWSRCH         Z: hittat, A = $8A
1A00  D1                   POP   DE
1A01  C1                   POP   BC             BC = THEN-delens avståndsbyte
1A02  20 16                JR    NZ,L1A1A       inget ELSE
1A04  12                   LD    (DE),A         $8A
1A05  13                   INC   DE
1A06  D5                   PUSH  DE             ELSE-delens avståndsbyte
1A07  13                   INC   DE
1A08  7B                   LD    A,E            THEN-delens avstånd: till ELSE-delens kod
1A09  91                   SUB   C
1A0A  02                   LD    (BC),A
1A0B  3E 81                LD    A,$81          $81 + radnummer
1A0D  12                   LD    (DE),A
1A0E  13                   INC   DE
1A0F  CD BC 19             CALL  CLNR           radnumret
1A12  30 04                JR    NC,L1A18
1A14  1B                   DEC   DE             eller satser
1A15  CD 3A 13             CALL  CMPST2
1A18  C1           L1A18:  POP   BC             L1A18: BC = ELSE-delens avståndsbyte
1A19  D8                   RET   C
1A1A  7B           L1A1A:  LD    A,E            inget ELSE: avståndet till slutet
1A1B  91                   SUB   C
1A1C  02                   LD    (BC),A
1A1D  7E                   LD    A,(HL)         : efter IF: fel 221 (CEND)
1A1E  EE 3A                XOR   $3A
1A20  C0                   RET   NZ
```

CEND: efter satsen får bara radslutet eller `!` *(kommentar,* `$90` *+ texten) komma, annars fel 221.* `END` *(*`$92`*) har* `CEND` *som hela sin kompilering:* `END ! x` *går,* `END 5` *ger fel 221. In: HL efter satsen, DE = koden. Ut: vid radslutet Z och NC, HL vid CR; efter ! som* `CTEXT`*; annars CY och A = 221. Förstör AF och BC.*

```
1A21  7E           CEND:   LD    A,(HL)
1A22  FE 0D                CP    $0D            radslutet: klart (Z, NC)
1A24  C8                   RET   Z
1A25  FE 21                CP    $21            ! (kommentar)?
1A27  37                   SCF
1A28  3E DD                LD    A,$DD          fel 221: radslut väntas
1A2A  C0                   RET   NZ
1A2B  DF                   RST   $18            ! hoppas över
1A2C  3E 90                LD    A,$90
1A2E  12           S1A2E:  LD    (DE),A         S1A2E: koden A, sedan texten
1A2F  13                   INC   DE
```

```
1A30  C3 72 1A           JP    CTEXT
```

CON: ON *uttryck* GOTO|GOSUB|RESTORE|RESUME|ON ERROR GOTO *nr,nr... Koden:* $A1, *heltalsuttrycket,* $BF, *antalet radnummer, satsens kod* ($80, $82-$85) *och numren (2 bytes vardera, utan* $81*). Andra satser ger fel 226* (ON *A* PRINT, ON *A* XSTM *1), saknat nummer fel 233 (även efter ett sista komma och för 65536). Blanka mellan siffrorna hoppas över:* ON A GOTO 10 20 *hoppar till 1020. I direktläge fel 208 när satsen körs. In: HL efter* ON, *DE = koden. Ut: NC: HL efter det sista numret, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1A33  CD A4 16     CON:    CALL  CEXPI          uttrycket: heltal
1A36  D8                   RET   C
1A37  3E BF                LD    A,$BF          $BF efter heltalet
1A39  12                   LD    (DE),A
1A3A  13                   INC   DE
1A3B  D5                   PUSH  DE
1A3C  11 42 09             LD    DE,$0942       satsen ur STKW
1A3F  CD 4B 4B             CALL  KWSRCH
1A42  D1                   POP   DE
1A43  20 04                JR    NZ,L1A49       inte funnen: fel 226
1A45  FE 86                CP    $86            GOTO, GOSUB, RESTORE, RESUME, ON ERROR GOTO
1A47  38 04                JR    C,L1A4D
1A49  3E E2        L1A49:  LD    A,$E2          fel 226: fel sats efter ON
1A4B  37                   SCF
1A4C  C9                   RET

1A4D  D5           L1A4D:  PUSH  DE             antalet fylls i sist
1A4E  13                   INC   DE
1A4F  12                   LD    (DE),A         satsens kod, platsen för antalet före
1A50  13                   INC   DE
1A51  06 00                LD    B,$00          B = antalet nummer
1A53  C5           L1A53:  PUSH  BC
1A54  D5                   PUSH  DE
1A55  CD C2 4B             CALL  GETLNR         radnummer (blanka hoppas över)
1A58  DA 83 1B             JP    C,L1B83        inget nummer: fel 233
1A5B  E3                   EX    (SP),HL        HL = koden, texten vid SP
1A5C  73                   LD    (HL),E         numret, låga byten först
1A5D  23                   INC   HL
1A5E  72                   LD    (HL),D
1A5F  23                   INC   HL
1A60  D1                   POP   DE
1A61  EB                   EX    DE,HL          HL = texten, DE = koden
1A62  C1                   POP   BC
1A63  04                   INC   B              ett nummer till
1A64  7E                   LD    A,(HL)         ,: ett till
1A65  EE 2C                XOR   $2C
1A67  20 03                JR    NZ,L1A6C
1A69  DF                   RST   $18
1A6A  18 E7                JR    L1A53

1A6C  78           L1A6C:  LD    A,B
1A6D  C1                   POP   BC             BC = platsen för antalet
1A6E  02                   LD    (BC),A         antalet
1A6F  C9                   RET
```

CDATA/CTEXT: DATA ($8E) *får två bytes för* DATA*-kedjan (fylls vid* RUN*),* REM ($8F) *och* ! ($90) *inte. Sedan en längdbyte (texten + 1) och texten oförändrad till radslutet:* DATA 1,2,"AB" *-> 8E .. .. 09 31 2C 32 2C 22 41 42 22;* DATA 1,2,AB *-> 8E .. .. 07 31 2C 32 2C 41 42;* REM hej *-> 8F 04 68 65 6A; 70* DATA *-> 8E .. .. 01. In: HL vid texten, DE = koden. Ut: NC och NZ: HL vid CR, DE efter texten. Förstör AF och BC.*

```
1A70  13           CDATA:  INC   DE             DATA: två bytes länk
1A71  13                   INC   DE
1A72  D5           CTEXT:  PUSH  DE
1A73  13                   INC   DE             längdbyten (texten + 1)
1A74  3E 0D                LD    A,$0D          A = CR
1A76  BE           L1A76:  CP    (HL)
1A77  28 9F                JR    Z,L1A18        radslutet: längden (L1A18)
1A79  ED A0                LDI                  texten till radslutet
1A7B  18 F9                JR    L1A76
```

CLNR0: RESTORE, RESUME och ON ERROR GOTO med radnummer; utan radnummer blir koden + $1A ($9D, $9E, $9F). Ett tal över 65535 eller något annat än ett nummer lämnar texten kvar, och då ger CEND fel 221 (RESTORE 70000, RESTORE X). In: HL vid numret, DE = koden efter satsens kod. Ut: NC: DE efter numret eller samma DE med den nya koden. Förstör AF och BC.

```
1A7D  CD BC 19     CLNR0:  CALL CLNR
1A80  D0                   RET  NC              numret: som CLNR
1A81  1B                   DEC  DE              inget nummer: satsens kod
1A82  1A                   LD   A,(DE)
1A83  C6 1A                ADD  A,$1A           $83-$85 -> $9D-$9F
1A85  12                   LD   (DE),A
1A86  13                   INC  DE
1A87  C9                   RET
```

CFOR: FOR v = a TO b [STEP c]. Koden: $98, tilldelningen (som CLET), $BD och gränsen, med STEP $BE och steget, båda omvandlade till variabelns typ; sist $BC och två bytes som länkas till NEXT vid RUN. FOR I=1 STEP 2 ger fel 232, STEP utan uttryck fel 234. FOR I=1 TO 5 STEP 2 -> 98 D5 81 B4 78 09 BD D6 83 20 B7 BE D5 82 B7 BC .. .. In: HL efter FOR, DE = koden. Ut: NC: HL efter satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.

CNEXT: $99 och variabelns referens (S1732); NEXT utan variabel fel 234, NEXT I,J fel 221. CWHILE: $9A, två bytes länk och villkoret (heltal). CWEND: $9B och två bytes (länken till WHILE; vid kompileringen står JPTABL:s adress där). Exempel: NEXT I -> 99 78 09; WHILE A% -> 9A .. .. C1 79 01 B8; WEND -> 9B C5 0A. In: HL efter satsens namn, DE = koden; CWEND också BC = de två bytena. Ut: NC: DE efter koden; CY: A = felet. Förstör som CEXPI.

```
1A88  CD 3F 17     CFOR:   CALL S173F           variabel = uttryck
1A8B  D8                   RET  C
1A8C  C5                   PUSH BC
1A8D  D5                   PUSH DE
1A8E  11 F9 09             LD   DE,$09F9
1A91  CD 4B 4B             CALL KWSRCH
1A94  D1                   POP  DE
1A95  C1                   POP  BC
1A96  4F                   LD   C,A
1A97  3E E8                LD   A,$E8           fel 232: TO väntas
1A99  37                   SCF
1A9A  C0                   RET  NZ
1A9B  79                   LD   A,C
1A9C  12                   LD   (DE),A
1A9D  13                   INC  DE
1A9E  CD AA 16             CALL CEXPT           gränsen, variabelns typ
1AA1  D8                   RET  C
1AA2  D5                   PUSH DE
1AA3  C5                   PUSH BC
1AA4  11 F3 09             LD   DE,$09F3        STEP?
1AA7  CD 4B 4B             CALL KWSRCH
1AAA  C1                   POP  BC
1AAB  D1                   POP  DE
1AAC  20 06                JR   NZ,L1AB4
1AAE  12                   LD   (DE),A
1AAF  13                   INC  DE
1AB0  CD AA 16             CALL CEXPT
1AB3  D8                   RET  C
1AB4  3E BC        L1AB4:  LD   A,$BC           $BC + två bytes (länk)
1AB6  12                   LD   (DE),A
1AB7  13                   INC  DE
1AB8  13                   INC  DE
1AB9  13                   INC  DE
1ABA  A7                   AND  A
1ABB  C9                   RET

1ABC  CD 32 17     CNEXT:  CALL S1732           enkel variabel, inte sträng
1ABF  EB           CWEND:  EX   DE,HL           två bytes (länk)
1AC0  71                   LD   (HL),C
1AC1  23                   INC  HL
1AC2  70                   LD   (HL),B
1AC3  23                   INC  HL
1AC4  EB                   EX   DE,HL
1AC5  C9                   RET

1AC6  13           CWHILE: INC  DE              två bytes (länk), villkoret
1AC7  13                   INC  DE
```

```
1AC8  C3 A4 16            JP   CEXPI
```

CXY: *två heltalsuttryck med* , *emellan; kommat läggs in i koden* ($2C). CXY2: , *och ett uttryck till, annars fel 227;* CXY1: *ett uttryck, Z om* , *följer.* POKE *a,b*[*,c...*] *tar valfritt antal värden,* OUT *p,v*[*,p,v...*] *bara par* (OUT *1,2,3 fel 227*), TXPOINT *x,y*[*,z*] *två eller tre och* SET DOT/CLR DOT *exakt två* (*mer ger fel 221*). *In: HL vid det första uttrycket, DE = koden. Ut: NC: Z om* , *följer* (*HL vid kommat*), *NZ annars; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1ACB  CD D7 1A    CXY:    CALL CXY1
1ACE  D8                  RET  C
1ACF  3E E3       CXY2:   LD   A,$E3          fel 227: , väntas
1AD1  37                  SCF
1AD2  C0                  RET  NZ
1AD3  ED A0               LDI                 , in i koden
1AD5  2B                  DEC  HL             HL tillbaka till kommat
1AD6  DF                  RST  $18            och förbi det och blanka efter
1AD7  CD A4 16    CXY1:   CALL CEXPI          Z: , följer
1ADA  D8                  RET  C
1ADB  7E                  LD   A,(HL)
1ADC  EE 2C               XOR  $2C
1ADE  C9                  RET

1ADF  CD CB 1A    CPOKE:  CALL CXY
1AE2  D8          L1AE2:  RET  C
1AE3  C0                  RET  NZ             inget mer: klart
1AE4  CD CF 1A            CALL CXY2           POKE: valfritt antal till
1AE7  18 F9               JR   L1AE2

1AE9  CD CB 1A    COUT:   CALL CXY
1AEC  D8          L1AEC:  RET  C
1AED  C0                  RET  NZ
1AEE  CD CF 1A            CALL CXY2           OUT: parvis
1AF1  D4 CF 1A            CALL NC,CXY2
1AF4  18 F6               JR   L1AEC

1AF6  CD CB 1A    CTXPNT: CALL CXY
1AF9  D8                  RET  C
1AFA  C0                  RET  NZ             bara två: klart
1AFB  18 D2               JR   CXY2           TXPOINT: högst ett till
```

CDEF: DEF *FNn*[(*p,...*)] [= *uttryck* | LOCAL *v,...*]. *Koden:* $8C, *två bytes länk, namnets referens* (*samma post i variabellistan som variabeln med samma namn, men FNA och A hålls isär vid körningen*), *två bytes till* (*fylls vid* RUN), *antalet parametrar och deras referenser. Med = följer uttrycket och* $B7+*typ* (*enradig funktion*). *Med* LOCAL *följer* $B6, *antalet och referenserna; en lokal sträng måste ha = längd* (*2 bytes efter referensen, fel 228 utan =, fel 210 utan tal*). *Utan någotdera är funktionen flerradig, och kroppen följer på nästa rader till* FNEND *med* RETURN *uttryck.* DEF *FNA*(*X*)=*X*2 -> 8C .. .. 78 01 .. .. 01 78 18 C0 78 18 D5 82 A0 B7. Fel:*) *saknas 229, parameter som inte är en variabel 234, mer efter*) *221. In: HL efter* DEF, *DE = koden. Ut: NC: HL efter satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1AFD  13          CDEF:   INC  DE             två bytes länk
1AFE  13                  INC  DE
1AFF  CD 43 33            CALL VARNAM         namnet, som en variabel
1B02  D8                  RET  C
1B03  EB                  EX   DE,HL          namnets referens
1B04  71                  LD   (HL),C
1B05  23                  INC  HL
1B06  70                  LD   (HL),B
1B07  23                  INC  HL
1B08  23                  INC  HL             två bytes till
1B09  23                  INC  HL
1B0A  36 00               LD   (HL),$00       inga parametrar
1B0C  23                  INC  HL
1B0D  EB                  EX   DE,HL
1B0E  7E                  LD   A,(HL)
1B0F  FE 28               CP   $28            ( : parametrar
1B11  20 12               JR   NZ,L1B25
1B13  DF                  RST  $18
1B14  1B                  DEC  DE             DE vid antalet
1B15  C5                  PUSH BC
1B16  0E 00               LD   C,$00          C = 0: parametrar
1B18  CD 3C 1B            CALL S1B3C          parametrarna
```

```
1B1B  C1                POP  BC
1B1C  D8                RET  C
1B1D  7E                LD   A,(HL)
1B1E  FE 29             CP   $29
1B20  3E E5             LD   A,$E5          fel 229: ) väntas
1B22  37                SCF
1B23  C0                RET  NZ
1B24  DF                RST  $18
1B25  7E          L1B25: LD   A,(HL)        L1B25: = : enradig funktion
1B26  EE 3D             XOR  $3D
1B28  20 05             JR   NZ,L1B2F
1B2A  DF                RST  $18            =: uttryck av funktionens typ
1B2B  41                LD   B,C            B = funktionens typ
1B2C  C3 AA 16          JP   CEXPT

1B2F  D5          L1B2F: PUSH DE
1B30  11 14 0A          LD   DE,$0A14       LOCAL?
1B33  CD 4B 4B          CALL KWSRCH
1B36  D1                POP  DE
1B37  C0                RET  NZ             varken = eller LOCAL: flerradig
1B38  12                LD   (DE),A         $B6 (LOCAL)
1B39  13                INC  DE
1B3A  0E 02             LD   C,$02          lokala: strängar med längd
```

S1B3C: *namn med , emellan, tills något annat följer; antalet läggs i byten vid DE. Varje namn blir sin referens (namnets kod, 2 bytes). Med C = 2 (LOCAL) får en strängvariabel = längd efter sig, 2 bytes. In: HL vid det första namnet, DE = platsen för antalet, C = 0 (parametrar) eller 2 (LOCAL). Ut: NC: HL efter det sista namnet, DE efter koden; CY: A = felet. Förstör AF, BC och IX.*

```
1B3C  D5          S1B3C: PUSH DE
1B3D  13                 INC  DE
1B3E  06 00              LD   B,$00
1B40  04          L1B40: INC  B              L1B40: B = antalet hittills
1B41  C5                 PUSH BC
1B42  CD 3E 33           CALL VNAME         namn (inte en funktion)
1B45  38 3F              JR   C,L1B86
1B47  EB                 EX   DE,HL         BC = namnets kod
1B48  71                 LD   (HL),C
1B49  23                 INC  HL
1B4A  70                 LD   (HL),B
1B4B  23                 INC  HL
1B4C  EB                 EX   DE,HL
1B4D  CB 49              BIT  1,C           strängvariabel?
1B4F  28 1F              JR   Z,L1B70
1B51  C1                 POP  BC            C = 0 eller 2
1B52  C5                 PUSH BC
1B53  CB 49              BIT  1,C           och LOCAL?
1B55  28 19              JR   Z,L1B70
1B57  7E                 LD   A,(HL)        lokal sträng: = längd
1B58  FE 3D              CP   $3D
1B5A  3E E4              LD   A,$E4
1B5C  20 28              JR   NZ,L1B86
1B5E  DF                 RST  $18
1B5F  D5                 PUSH DE
1B60  CD C2 4B           CALL GETLNR        längden
1B63  4B                 LD   C,E           BC = längden
1B64  42                 LD   B,D
1B65  D1                 POP  DE
1B66  3E D2              LD   A,$D2         fel 210: inget tal
1B68  38 1C              JR   C,L1B86
1B6A  EB                 EX   DE,HL         längden efter referensen
1B6B  71                 LD   (HL),C
1B6C  23                 INC  HL
1B6D  70                 LD   (HL),B
1B6E  23                 INC  HL
1B6F  EB                 EX   DE,HL
1B70  C1          L1B70: POP  BC             L1B70: B = antalet
1B71  CD 87 16           CALL EMIT          bara utrymmesprovet
1B74  38 11              JR   C,L1B87
1B76  1B                 DEC  DE            DE tillbaka
```

```
1B77  7E                LD   A,(HL)          , : ett namn till
1B78  EE 2C             XOR  $2C
1B7A  20 03             JR   NZ,L1B7F
1B7C  DF                RST  $18
1B7D  18 C1             JR   L1B40

1B7F  E3          L1B7F: EX   (SP),HL        L1B7F: HL = platsen för antalet
1B80  70                LD   (HL),B          antalet
1B81  E1                POP  HL              texten tillbaka
1B82  C9                RET

1B83  3E E9       L1B83: LD   A,$E9          L1B83: fel 233 (från CON)
1B85  C1                POP  BC
1B86  C1          L1B86: POP  BC             L1B86: rensa stacken, CY
1B87  C1          L1B87: POP  BC
1B88  37                SCF
1B89  C9                RET
```

CXSTM: XSTM *n, där n är numret på en sats i grupp 2 eller i ett tilläggsprogram (tabellerna i* STKWP*-kedjan,* FNDTAB*) och sedan satsens egna argument. Koden blir* $86 *n, alltså samma som satsen själv;* LIST *visar satsens namn (*XSTM 4 "A" AS FILE 1 -> OPEN "A" AS FILE 1, XSTM 6 -> CLOSE*). Inget tal, eller ett n som ingen tabell har (*XSTM 128*), ger fel 200. Över 255 räknas bara den låga byten:* XSTM 262 *blir* CLOSE*. Blanka i talet hoppas inte över (*XSTM 6 2 *är* CLOSE 2*,* XSTM 62 *fel 200).* XSTM *24-36 hittas i huvudtabellens huvud (*STKW*), som saknar hopptabell, och hoppar därför via ROM:ens början. In: HL efter* XSTM*, DE = koden. Ut: som satsens kompilering; CY och A = 200 vid okänt n.*

```
1B8A  ED 4B 7D FF CXSTM: LD   BC,(STKWP)
1B8E  CD 95 1B           CALL XNUM
1B91  D2 B7 13           JP   NC,JPTAB
1B94  C9                 RET
```

XNUM*: läs n (*GETLNR *med IY+*$1D *bit 2 satt, som med* EXTEND*: blanka inne i talet hoppas inte över), lägg den låga byten i koden och sök n i tabellkedjan med början vid BC (*FNDTAB*). In: HL vid talet, DE = koden, BC = första tabellhuvudet. Ut: NC: A = n minus tabellens första kod, BC = kompileringsrutinernas tabell (huvudet + 8), DE + 1, HL efter talet; CY: A = 200. Förstör F.*

```
1B95  C5          XNUM:  PUSH BC
1B96  D5                 PUSH DE
1B97  FD 46 1D           LD   B,(IY+$1D)     spara IY+$1D, sätt bit 2
1B9A  C5                 PUSH BC
1B9B  FD CB 1D D6        SET  2,(IY+$1D)
1B9F  CD C2 4B           CALL GETLNR
1BA2  C1                 POP  BC
1BA3  FD 70 1D           LD   (IY+$1D),B
1BA6  7B                 LD   A,E            n (låga byten) i koden
1BA7  D1                 POP  DE
1BA8  12                 LD   (DE),A
1BA9  13                 INC  DE
1BAA  C1                 POP  BC
1BAB  38 0A              JR   C,L1BB7        inget tal: fel 200
1BAD  E5                 PUSH HL
1BAE  CD EB 1D           CALL FNDTAB         finns n i någon tabell?
1BB1  CD DC 30           CALL S30DC          BC = kompileringstabellen (huvudet + 8)
1BB4  E1                 POP  HL
1BB5  3F                 CCF                 NC = funnen -> JPTAB
1BB6  D0                 RET  NC
1BB7  3E C8       L1BB7: LD   A,$C8
1BB9  C9                 RET
```

CPRINT*: [#fil,] [*USING *sträng,] och sedan uttryck;* , *och* ; *läggs in oförändrade i koden (*$2C/$3B*). #fil (*CFILNR*) blir* $23*, heltalet och* $B8*,* USING *blir* $B8*, strängen och* $B9*; varje uttryck slutar med* $B7*+typ (*CEXPA*). Exempel:* PRINT *1;X -> 93 D5 81 B7 3B C0 78 18 B7. Uttryck utan skiljetecken skrivs var för sig (*PRINT 1 2 *ger 1 och 2); bara när uttrycket är en enda bokstav och nästa tecken en bokstav blir det fel 220 (*"PRINT AB"*). Blanka i namnet hoppas över (A 1 = A1). Kommat efter #fil får fattas (*"PRINT #1 A"*). In: HL efter* PRINT*, DE = koden. Ut: NC: HL vid satsens slut, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1BBA  CD A7 1C    CPRINT: CALL CFILNR         #fil
1BBD  D8                  RET  C
1BBE  D5                  PUSH DE
1BBF  11 0D 0A            LD   DE,$0A0D
```

```
1BC2  CD 4B 4B            CALL KWSRCH          USING?
1BC5  D1                  POP  DE
1BC6  20 06               JR   NZ,L1BCE
1BC8  12                  LD   (DE),A
1BC9  13                  INC  DE              $B8 (USING)
1BCA  CD 1B 00            CALL CEXPS           formatsträngen
1BCD  D8                  RET  C
1BCE  7E            L1BCE: LD   A,(HL)        L1BCE: nästa del
1BCF  FE 2C               CP   $2C             , och ; kopieras
1BD1  28 28               JR   Z,L1BFB
1BD3  FE 3B               CP   $3B
1BD5  28 24               JR   Z,L1BFB
1BD7  CD 01 1C            CALL EOSTM           slut på satsen?
1BDA  C8                  RET  Z
1BDB  E5                  PUSH HL              uttryckets början
1BDC  CD 80 16            CALL CEXPA           ett uttryck
1BDF  C1                  POP  BC              BC = början
1BE0  D8                  RET  C
1BE1  7D                  LD   A,L             uttrycket ett tecken långt
1BE2  91                  SUB  C
1BE3  3D                  DEC  A
1BE4  20 E8               JR   NZ,L1BCE
1BE6  0A                  LD   A,(BC)          och en bokstav?
1BE7  FE 40               CP   $40
1BE9  38 E3               JR   C,L1BCE
1BEB  CD 01 1C            CALL EOSTM           uttrycket sist i satsen: klart
1BEE  C8                  RET  Z
1BEF  7E                  LD   A,(HL)          nästa tecken en bokstav?
1BF0  E6 5F               AND  $5F
1BF2  D6 41               SUB  $41
1BF4  FE 5E               CP   $5E
1BF6  30 D6               JR   NC,L1BCE
1BF8  3E DC               LD   A,$DC           fel 220: en bokstav följd av en bokstav
1BFA  C9                  RET

1BFB  ED A0         L1BFB: LDI                  L1BFB: skiljetecknet in i koden
1BFD  2B                  DEC  HL              HL tillbaka till tecknet
1BFE  DF                  RST  $18             och förbi det
1BFF  18 CD               JR   L1BCE
```

EOSTM: *slut på satsen? Z vid* :, !, *CR eller* ELSE (KWSRCH *i tabellen vid* $09ED, *som bara har* ELSE)*; NZ annars. In: HL vid tecknet. Ut: Z eller NZ, HL och DE oförändrade. Förstör AF.*

```
1C01  7E            EOSTM: LD   A,(HL)        : ! CR eller ELSE
1C02  FE 3A               CP   $3A
1C04  C8                  RET  Z
1C05  FE 21               CP   $21
1C07  C8                  RET  Z
1C08  FE 0D               CP   $0D
1C0A  C8                  RET  Z
1C0B  D5                  PUSH DE
1C0C  E5                  PUSH HL
1C0D  11 ED 09            LD   DE,$09ED        ELSE?
1C10  CD 4B 4B            CALL KWSRCH
1C13  E1                  POP  HL
1C14  D1                  POP  DE
1C15  C9                  RET
```

CINPUT: [*#fil,*] [*"text"* [;|,]], *sedan* $B7, *en typbyte per variabel (0 flyttal, 1 heltal, 2 sträng; den sista +* $0C*) och sist referenserna med* $BA+*typ.* DMERGE *flyttar varje typbyte fram till de andra, så att alla typbytes står före referenserna:*

INPUT "T";A,B$ -> *96 CB 01 54 B9 3B B7 00 0E C4 .. .. BA C6 .. .. BC;* INPUT A%,B$,C -> *96 B7 01 02 0C C5 .. .. BB C6 .. .. BC C4 .. .. BA. Ett* ; *eller* , *sist står kvar efter referenserna (ingen ny rad efter svaret). Utan text skrivs* ? , *med text och* ; *(eller inget tecken) inget frågetecken, med* , *går markören till nästa zon. Blanka i en inläst sträng behålls.* CREAD*: per variabel* $B7, *typ + 4 och referensen.* CINLIN (INPUT LINE)*: [#fil,] och en strängvariabel. In: HL efter satsens namn, DE = koden. Ut: NC: HL efter satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1C16  CD A7 1C      CINPUT: CALL CFILNR
1C19  D8                    RET  C
```

```
1C1A  7E              LD   A,(HL)          text i ' eller "
1C1B  FE 27           CP   $27
1C1D  28 04           JR   Z,L1C23
1C1F  FE 22           CP   $22
1C21  20 0F           JR   NZ,L1C32
1C23  CD 1B 00  L1C23: CALL CEXPS
1C26  D8              RET  C
1C27  7E              LD   A,(HL)          ; eller , efter texten
1C28  FE 3B           CP   $3B
1C2A  28 04           JR   Z,L1C30
1C2C  FE 2C           CP   $2C
1C2E  20 02           JR   NZ,L1C32
1C30  ED A0     L1C30: LDI                 L1C30: skiljetecknet in i koden
1C32  2B        L1C32: DEC  HL             L1C32: blanka hoppas över
1C33  DF              RST  $18
1C34  3E B7           LD   A,$B7           $B7 före variablerna
1C36  12              LD   (DE),A
1C37  13              INC  DE
1C38  D5              PUSH DE              där typbytena börjar
1C39  CD 4A 17  L1C39: CALL S174A          L1C39: referensen och $BA+typ
1C3C  DA 2F 17        JP   C,L172F         fel: stacken rensas (L172F)
1C3F  D5              PUSH DE              efter referensen
1C40  78              LD   A,B             typen
1C41  12              LD   (DE),A
1C42  13              INC  DE
1C43  CD CF 3F        CALL DMERGE          flytta typbyten före referenserna
1C46  C1              POP  BC              BC = där typbyten lades in
1C47  C1              POP  BC
1C48  7E              LD   A,(HL)          fler variabler?
1C49  FE 3B           CP   $3B
1C4B  28 04           JR   Z,L1C51
1C4D  EE 2C           XOR  $2C
1C4F  20 10           JR   NZ,L1C61
1C51  ED A0     L1C51: LDI                 L1C51: skiljetecknet; LDI räknar ned BC (egenhet)
1C53  03              INC  BC              BC tillbaka
1C54  1B              DEC  DE              skrivs över av nästa
1C55  2B              DEC  HL
1C56  DF              RST  $18
1C57  03              INC  BC              BC = platsen för nästa typbyte
1C58  C5              PUSH BC
1C59  CD 01 1C        CALL EOSTM           ; eller , sist
1C5C  20 DB           JR   NZ,L1C39        mer: en variabel till
1C5E  13              INC  DE              ... står kvar i koden
1C5F  C1              POP  BC
1C60  0B              DEC  BC              BC = den sista typbyten
1C61  0A        L1C61: LD  A,(BC)          sista variabeln: + $0C
1C62  F6 0C           OR   $0C
1C64  02              LD   (BC),A
1C65  C9              RET

1C66  DF        L1C66: RST  $18            L1C66: , hoppas över
1C67  3E B7     CREAD: LD   A,$B7          per variabel: $B7, typ + 4
1C69  12              LD   (DE),A
1C6A  13              INC  DE
1C6B  D5              PUSH DE              platsen för typbyten
1C6C  13              INC  DE              typbyten före referensen
1C6D  CD 4A 17        CALL S174A
1C70  DA 2F 17        JP   C,L172F
1C73  78              LD   A,B
1C74  C1              POP  BC
1C75  F6 04           OR   $04             typen + 4 före referensen
1C77  02              LD   (BC),A
1C78  7E              LD   A,(HL)
1C79  EE 2C           XOR  $2C             , : en variabel till
1C7B  28 E9           JR   Z,L1C66
1C7D  C9              RET
```

Filsatserna har koden $86 + *nummer* (*grupp 2*): OPEN *4*, PREPARE *5*, CLOSE *6*, GET $0A, PUT $0E, POSIT $15. CFILNR: # ($23),

heltalet ($B8) *och* , *– allt valfritt.* GET*: strängvariabel (fel 224 annars), sedan ev. COUNT* ($B9) *och ett heltal.* PUT*: en sträng* ($B9). POSIT*: ett flyttal* ($B7). OPEN/PREPARE*: namnet,* AS FILE ($BB*, fel 230 utan) och filnumret, där ett # hoppas över.* CLOSE*: inget (alla filer) eller nummer med* , *emellan, # valfritt;* CLOSE 1 2 *fel 221.*

Exempel: GET *#1,A$ COUNT 5 -> 86 0A 23 D5 81 81 B8 2C C6 .. .. BC B9 D6 83 20 81 B8.* CINLIN, CPUT, CGET *och* CPOSIT*: In: HL efter satsens namn, DE = koden. Ut: NC: HL efter satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1C7E  CD A7 1C     CINLIN: CALL CFILNR
1C81  C3 5F 17             JP   S175F          strängvariabel

1C84  CD A7 1C     CPUT:   CALL CFILNR
1C87  D4 1B 00             CALL NC,CEXPS       strängen
1C8A  C9                   RET

1C8B  CD A7 1C     CGET:   CALL CFILNR
1C8E  D4 5F 17             CALL NC,S175F       strängvariabel
1C91  D8                   RET  C
1C92  D5                   PUSH DE             COUNT?
1C93  11 06 0A             LD   DE,$0A06
1C96  CD 4B 4B             CALL KWSRCH
1C99  D1                   POP  DE
1C9A  C0                   RET  NZ
1C9B  12                   LD   (DE),A
1C9C  13                   INC  DE
1C9D  C3 A4 16             JP   CEXPI          antalet tecken

1CA0  CD A7 1C     CPOSIT: CALL CFILNR
1CA3  D4 A8 16             CALL NC,CEXPF       positionen (flyttal)
1CA6  C9                   RET
```

CFILNR*: # och filnumret (heltal), sedan ev.* ,*; allt valfritt* (GET A$ -> *86 0A C6 .. .. BC). Kommat kopieras in i koden. In: HL vid texten, DE = koden. Ut: NC: HL efter numret och kommat, DE efter koden; utan # ingenting gjort (NZ); CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1CA7  7E           CFILNR: LD   A,(HL)          # och heltal, , efter
1CA8  EE 23                XOR  $23
1CAA  C0                   RET  NZ
1CAB  ED A0                LDI                 # in i koden
1CAD  2B                   DEC  HL             HL tillbaka till #
1CAE  DF                   RST  $18            och förbi det
1CAF  CD A4 16             CALL CEXPI
1CB2  D8                   RET  C
1CB3  7E                   LD   A,(HL)         , efter?
1CB4  EE 2C                XOR  $2C
1CB6  C0                   RET  NZ
1CB7  ED A0                LDI                 , in i koden
1CB9  C3 1A 4C             JP   SKIPB0         och blanka efter
```

COPEN: OPEN/PREPARE *"namn"* AS FILE *n: strängen,* $BB *och heltalet* (OPEN "A" AS FILE 1 -> *86 04 CB 01 41 B9 BB D5 81 81 B8).* S1CCE*: filnumret, där ett # hoppas över (även i* CLOSE*).* CCLOSE*: utan nummer alla filer (86 06), annars numren med* , *emellan och kommat i koden. In: HL efter satsens namn, DE = koden. Ut: NC: HL efter satsen, DE efter koden; CY: A = felet. Förstör AF, BC, IX och de alternativa registren.*

```
1CBC  CD 1B 00     COPEN:  CALL CEXPS          filnamnet
1CBF  D8                   RET  C
1CC0  D5                   PUSH DE
1CC1  11 FD 09             LD   DE,$09FD       AS FILE = $BB
1CC4  CD 4B 4B             CALL KWSRCH
1CC7  D1                   POP  DE
1CC8  12                   LD   (DE),A         $BB (A är fel om inte funnen)
1CC9  3E E6                LD   A,$E6          fel 230: AS FILE väntas
1CCB  37                   SCF
1CCC  C0                   RET  NZ
1CCD  13                   INC  DE
1CCE  7E           S1CCE:  LD   A,(HL)         # hoppas över
1CCF  EE 23                XOR  $23
1CD1  CC 19 4C             CALL Z,S4C19
1CD4  C3 A4 16             JP   CEXPI

1CD7  CD 01 1C     CCLOSE: CALL EOSTM          CLOSE utan nummer: alla
```

```
1CDA  C8                RET  Z
1CDB  CD CE 1C    L1CDB: CALL S1CCE          L1CDB: nästa nummer
1CDE  D8                RET  C
1CDF  7E                LD   A,(HL)        fler nummer med ,
1CE0  EE 2C             XOR  $2C
1CE2  C0                RET  NZ
1CE3  ED A0             LDI                , in i koden
1CE5  2B                DEC  HL            HL tillbaka till kommat
1CE6  DF                RST  $18           och förbi det
1CE7  18 F2             JR   L1CDB
```

RUN [*fil*]: *med argument först* LOAD *fil* (RUN TST); *någon radnummerform finns inte,* RUN 20 *söker filen 20 (fel 21). Sedan länka* (LINK), RESTORE, *nollställ slumptalsfröet, SP = PRGBEG och börja på första raden* (RUNLP). RUN0 *är samma utan* LOAD, *också för* CHAIN. *In: HL efter* RUN. *Ut: återvänder bara vid fel i* LOAD (*CY, A = felet*).

```
1CE9  7E          RUN:  LD   A,(HL)
1CEA  FE 0D             CP   $0D
1CEC  C4 99 14          CALL NZ,LOAD       argument: RUN fil
1CEF  D8                RET  C
1CF0  CD ED 3C          CALL CRLF          ny rad
1CF3  CD 27 2E    RUN0: CALL LINK
1CF6  DA 6B 3B          JP   C,ERRSP       fel vid länkningen
1CF9  CD B7 28          CALL RESTR0        DATA-pekaren till början
1CFC  CD 3A 35          CALL RNDCLR        slumptalsfröet
1CFF  CD 3B 00          CALL S003B         HL = PRGBEG
1D02  F9                LD   SP,HL         SP = PRGBEG
1D03  CD 44 00          CALL FSTLIN        DE = $87 före första raden
1D06  EB                EX   DE,HL         DE vid $87, sedan RUNLP
```

3.19 RUNLP: kör program- eller direktkod från DE $1D07

Programmet är kompilerat till en bytekod som körs trådat: varje kod är ett index i JPTABL, och varje rutin avslutas med RST $28 (nästa kod) eller RET. Värden ligger på Z80-stacken.

STMLP ($1D56) tar en sats i taget: kod >= $C0 är ett uttryck som körs via EXECA, annars är koden en sats (JPTABL-index kod - $43). Före varje sats testas IY+$23:

```
    bit 0  BREAK (BRKPTR pekar på $FF23, KEYINT sätter biten vid
           CTRL-C): programmet väntar på en ny tangent; CTRL-C igen
           ger STOP, CTRL-S ($13) sätter bit 3, annan tangent fortsätter
    bit 1  TRACE: HOOK3 ($2CC7) skriver radnumret vid varje ny rad
    bit 2  direktläge (satt i CMDLP)
    bit 3  paus vid radslut: CTRL-S kör en rad i taget. Pausen kommer
           vid $87 före raden; ett hopp landar på föregående rads $87
           (som EOL behöver för radnumret), så även efter ett hopp
           kommer pausen före målraden. Bit 0 och 3 nollas vid varje
           paus.
```

```
1D07  0E FD       RUNLP: LD   C,$FD          C = $FD: ramen för RUN (GOSUB $FE)
1D09  E5          L1D09: PUSH HL             L1D09: ramen: HL, gamla FRMSP, BC
1D0A  2A 45 FF           LD   HL,(FRMSP)
1D0D  E5                 PUSH HL
1D0E  06 00       L1D0E: LD   B,$00          L1D0E: B = 0, C = ramens sort
1D10  C5                 PUSH BC
1D11  ED 73 45 FF L1D11: LD   (FRMSP),SP     L1D11: FRMSP = ramen
1D15  ED 53 3A FF        LD   (PRGPTR),DE    PRGPTR = koden som körs
1D19  37                 SCF                 CY: inget uttryck första gången
1D1A  D4 AA 1D    L1D1A: CALL NC,EXECA       L1D1A: uttryckskoden vid DE
1D1D  FD 7E 23    L1D1D: LD   A,(IY+$23)     L1D1D: BREAK, TRACE eller paus?
1D20  A7                 AND  A
1D21  28 33              JR   Z,STMLP        inget att pröva
1D23  47                 LD   B,A            B = bitarna
1D24  E6 01              AND  $01            bit 0: BREAK
1D26  20 10              JR   NZ,L1D38
1D28  C5                 PUSH BC
1D29  CB 48              BIT  1,B
1D2B  C4 96 FF           CALL NZ,HOOK3       TRACE
1D2E  C1                 POP  BC
```

```
1D2F  1A              LD   A,(DE)
1D30  FE 87           CP   $87          radslut?
1D32  20 22           JR   NZ,STMLP
1D34  CB 58           BIT  3,B
1D36  28 1E           JR   Z,STMLP      en rad i taget?
1D38  13        L1D38: INC  DE          L1D38: PRGPTR = DE + 1
1D39  CD A3 1D        CALL S1DA3
1D3C  1B              DEC  DE
1D3D  78              LD   A,B          bit 0 och 3 nollas
1D3E  E6 F6           AND  $F6
1D40  47              LD   B,A
1D41  AF              XOR  A            ingen gammal tangent
1D42  32 E2 FF        LD   (KEYFLG),A
1D45  CD 90 FF        CALL GETKYV       vänta på en ny tangent
1D48  FE 13           CP   $13
1D4A  20 02           JR   NZ,L1D4E
1D4C  CB D8           SET  3,B          CTRL-S: paus före nästa rad
1D4E  FE 03     L1D4E: CP  $03          L1D4E: CTRL-C?
1D50  FD 70 23        LD   (IY+$23),B   bitarna tillbaka
1D53  CC 7B 3B        CALL Z,STOP       CTRL-C igen: STOP
1D56  1A        STMLP: LD  A,(DE)       STMLP: nästa kod
1D57  FE C0           CP   $C0
1D59  30 BF           JR   NC,L1D1A     uttryckskod
1D5B  13              INC  DE           satsens kod: DE efter den
1D5C  21 3F 0A        LD   HL,$0A3F     satsernas körtabell, $0A3F + 2*kod
1D5F  CD B3 1D        CALL S1DB3        satsens rutin
1D62  18 B9           JR   L1D1D        sedan nästa sats
```

EOL ($87, radslut): nästa rads längd; längd 1 = programmets slut -> ENDST, som END. Annars hoppa över radnumret och fortsätt. ENDST stänger alla filer (S0D4A); STOP gör det inte, där står filerna öppna och CON fortsätter. In: DE vid längdbyten efter $87. Ut: DE vid radens första kod, PRGPTR = DE. Förstör AF.

```
1D64  1A        EOL:   LD  A,(DE)       radens längd
1D65  13               INC  DE          längden och numret hoppas över
1D66  13               INC  DE
1D67  13               INC  DE
1D68  3D               DEC  A           längd 1: slutmärket
1D69  20 38            JR   NZ,S1DA3
1D6B  CD 4A 0D  ENDST: CALL S0D4A       stäng filerna
1D6E  C3 0C 01         JP   READY0      READY0: inget fortsätter
```

XSTM ($86 n): satser i grupp 2 (n < $80, via $0B89) eller från tilläggsprogram (n >= $80, sökes i STKWP-kedjan med FNDTAB; ingen rutin -> fel 200). In: DE vid n. Ut: som satsens rutin.

```
1D71  21 89 0B  XSTM:  LD  HL,$0B89      grupp 2: $0B89 + 2*n
1D74  1A               LD  A,(DE)        n
1D75  13               INC  DE
1D76  FE 80            CP   $80
1D78  38 39            JR   C,S1DB3      n < $80: ROM:ens sats
1D7A  ED 4B 7D FF      LD   BC,(STKWP)   tilläggens satser
1D7E  CD EB 1D  L1D7E: CALL FNDTAB       L1D7E: för funktioner BC = FNKWP
1D81  CA 36 00         JP   Z,UNSDEF     ingen tabell: fel 200
1D84  4E               LD   C,(HL)       HL = körtabellen (huvudet + 4)
1D85  23               INC  HL
1D86  46               LD   B,(HL)
1D87  60               LD   H,B
1D88  69               LD   L,C
1D89  18 28            JR   S1DB3        A = n minus första koden
```

SKIP: hoppa framåt; byten efter koden är avståndet (REM, !, DATA, ELSE). GOTO och WEND har ett 16-bitars relativt avstånd (via $1D9D). SKIP2 hoppar först över två bytes (DATA-länken). In: DE vid avståndsbyten, B = 0 (från S1DB3, där C fick koden). Ut: DE + avståndet. Förstör F och C.

```
1D8B  13        SKIP2: INC  DE          SKIP2: DATA-länken
1D8C  13               INC  DE
1D8D  EB        SKIP:  EX   DE,HL
1D8E  4E               LD   C,(HL)       avståndet
1D8F  09               ADD  HL,BC        B = 0
```

```
1D90  EB                EX   DE,HL
1D91  C9                RET
```

THEN ($BD) *och* STEP*-varianten* ($BE)*: värdet på stacken är villkoret; 0 = falskt -> hoppa* (SKIP)*, annars fortsätt efter byten. Villkoret är ett heltal, och flyttal avrundas till närmaste (.5 och -.5 sant, .4 och -.4 falskt, 65536 fel 132). In: villkoret vid SP, DE vid avståndsbyten, B = 0. Ut: DE efter avståndsbyten eller där avståndet pekar. Förstör AF, HL och C.*

```
1D92  E1          THEN:   POP  HL              villkoret (heltal) (egenhet)
1D93  7C                  LD   A,H
1D94  B5                  OR   L
1D95  28 F6               JR   Z,SKIP          falskt: hoppa
1D97  13                  INC  DE              sant: förbi avståndsbyten
1D98  C9                  RET
```

DEF ($8C) *vid körning: funktionen hoppas över. Länken, referensen och två bytes till ligger före avståndet, som* RUN *fyllt i.* GOTO*: 16 bitar efter koden, räknade från den andra av dem, och målet är* $87 *före raden* (10 GOTO 30 *-> 80 09 00 när rad 20 är 8 bytes). Samma rutin för* WEND*, och för* $81 *efter* THEN. S1DA3*: PRGPTR = DE (används på 20 ställen). In: DE vid avståndet. Ut: DE = målet, PRGPTR = DE. Förstör HL.*

```
1D99  13          DEF:    INC  DE              länken och referensen
1D9A  13                  INC  DE
1D9B  13                  INC  DE
1D9C  13                  INC  DE
1D9D  EB          GOTO:   EX   DE,HL
1D9E  5E                  LD   E,(HL)          avståndet, låga byten först
1D9F  23                  INC  HL
1DA0  56                  LD   D,(HL)
1DA1  19                  ADD  HL,DE           räknat från den andra byten
1DA2  EB                  EX   DE,HL
1DA3  ED 53 3A FF S1DA3:  LD   (PRGPTR),DE
1DA7  C9                  RET

1DA8  E5          L1DA8:  PUSH HL              L1DA8: HL på stacken, sedan nästa kod
1DA9  1A          EXECOD: LD   A,(DE)          RST $20: nästa kod
```

3.20 EXECA: utför uttryckskoden A (RST $20 hamnar i EXECOD) $1DAA

```
$00-$CF  via JPTABL (bas $0ABF)
$D0-$D4  funktionsanrop, se FNTAB ($D4 följs av $D0+n)
         kod $80-$9F efter prefixet: färgord, en sträng med ett
         tecken = koden (RED = CHR$(129), LEN 1); RED+1 fel 224
         $A0-: funktion från ett tilläggsprogram (FNKWP)
$D5-$DC  talkonstant med 1-8 bytes (NUMCON)
$DD-$FF  litet heltal 0-$22 (SMLINT)
```

```
1DAA  FE D0       EXECA:  CP   $D0
1DAC  30 0F               JR   NC,L1DBD        $D0-: funktion, tal
1DAE  21 BF 0A            LD   HL,$0ABF        ; koderna $00-$CF: JPTABL - 6
1DB1  1A          L1DB1:  LD   A,(DE)          ; L1DB1: koden ur programmet
1DB2  13                  INC  DE
1DB3  4F          S1DB3:  LD   C,A             ; S1DB3: hoppa via ordet HL + 2A
1DB4  06 00               LD   B,$00
1DB6  09                  ADD  HL,BC           två bytes per kod
1DB7  09                  ADD  HL,BC
1DB8  7E                  LD   A,(HL)
1DB9  23                  INC  HL
1DBA  66                  LD   H,(HL)
1DBB  6F                  LD   L,A
1DBC  E9          S1DBC:  JP   (HL)            ; S1DBC: hoppa till HL

1DBD  FE DD       L1DBD:  CP   $DD
1DBF  13                  INC  DE
1DC0  D2 A9 21            JP   NC,SMLINT       $DD-: litet heltal
1DC3  FE D5               CP   $D5
1DC5  D2 EE 20            JP   NC,NUMCON       $D5-$DC: talkonstant
1DC8  FE D4               CP   $D4
1DCA  20 01               JR   NZ,L1DCD        $D4: hoppa över en byte
1DCC  13                  INC  DE
```

```
1DCD  21 5D 0C     L1DCD:  LD   HL,$0C5D        ; FNTAB
1DD0  1A                   LD   A,(DE)
1DD1  13                   INC  DE
1DD2  FE 80                CP   $80
1DD4  38 DD                JR   C,S1DB3         funktion ur FNTAB
1DD6  FE A0                CP   $A0
1DD8  ED 4B 7F FF          LD   BC,(FNKWP)
1DDC  30 A0                JR   NC,L1D7E        $A0-: tilläggens funktioner
1DDE  F5                   PUSH AF              färgord: sträng med ett tecken
1DDF  33                   INC  SP
1DE0  21 01 00             LD   HL,$0001
1DE3  E5                   PUSH HL
1DE4  E5                   PUSH HL
1DE5  2E 04                LD   L,$04
1DE7  39                   ADD  HL,SP
1DE8  E3                   EX   (SP),HL         ; adressen; L1DA8 lägger 1 överst
1DE9  18 BD                JR   L1DA8
```

FNDTAB*: sök i en kedja av tabellhuvuden (BC = första) efter koden i A. Huvudet = länk, första kod, antal, tabellerna; koden hör till huvudet när kod - första < antal. Ut: HL = tabellerna, BC = 0 om ingen träff. Används för satser och funktioner från tilläggen (*STKWP/FNKWP*); utanför alla tabeller blir det fel 200 (*XSTM*). In: A = koden, BC = första huvudet. Förstör A, BC.*

```
1DEB  D5           FNDTAB: PUSH DE
1DEC  5F                   LD   E,A
1DED  78           L1DED:  LD   A,B
1DEE  B1                   OR   C
1DEF  28 0C                JR   Z,L1DFD
1DF1  60                   LD   H,B
1DF2  69                   LD   L,C
1DF3  4E                   LD   C,(HL)
1DF4  23                   INC  HL
1DF5  46                   LD   B,(HL)
1DF6  23                   INC  HL
1DF7  7B                   LD   A,E
1DF8  96                   SUB  (HL)            kod - första
1DF9  23                   INC  HL
1DFA  BE                   CP   (HL)            utanför: nästa huvud
1DFB  30 F0                JR   NC,L1DED
1DFD  D1           L1DFD:  POP  DE
1DFE  23                   INC  HL              ; HL = tabellerna (efter antalet)
1DFF  C9                   RET

1E00  D7           ERR90:  RST  $10             ; fel 144: rad med syntaxfel ($8B)
1E01  90                   DEFB $90
```

PRINT *[#n,] [*USING *mall,] lista.* , *går till nästa zon om 14 tecken (IY+$1F + 8),* ; *skriver inget. Med* USING *tas mallens fält i tur och ordning, och mallen börjar om när den tar slut; text mellan fälten skrivs som den är. Talfält: # . , (blanksteg mellan tusentalen,* 1 234.50*) ^^^^ (exponent) ¤¤ ** + och - sist; ett tal som inte ryms skrivs helt med % före. Strängfält: ! (första tecknet), & (hela strängen), \ \ (n+2 tecken, utfyllt med blanka). En sträng till ett talfält läses som tal (*"12", 1E1*); är den inget tal blir det fel 210, och efter* RESUME *skrivs 0. Fel 146: tom mall, mall utan fält, tal till strängfält, och när värdena tar slut mitt i mallen (texten skrivs fram till nästa fält först). En mall som inte är en sträng ger fel 223.*

```
1E02  CD 24 20     PRINT:  CALL S2024           ; L = filnumret (0 = skärmen)
1E05  FD CB 27 DE          SET  3,(IY+$27)      ; tal med tecken eller blank
1E09  4D           S1E09:  LD   C,L             ; S1E09: TRACE kommer in här
1E0A  21 00 00             LD   HL,$0000        ; HL = SP att återvända till
1E0D  39                   ADD  HL,SP
1E0E  1A                   LD   A,(DE)
1E0F  DD 21 F2 FF          LD   IX,$FFF2        ; ramen 14 bytes utan USING
1E13  EE B8                XOR  $B8             ; $B8: USING
1E15  3E 00                LD   A,$00           ; A = 0: ingen mall
1E17  20 1D                JR   NZ,L1E36
1E19  13                   INC  DE
1E1A  C5                   PUSH BC
1E1B  E7                   RST  $20             ; mallen som sträng på stacken
1E1C  E1                   POP  HL              ; HL = bytes text på stacken
1E1D  DD E1                POP  IX              ; IX = mallens adress
1E1F  C1                   POP  BC              ; BC = mallens längd
```

```
1E20  DD E5              PUSH IX             ; adressen blir ramens +C,+D
1E22  AF                 XOR  A
1E23  B0                 OR   B              ; 1-255 tecken, annars fel 146
1E24  20 03              JR   NZ,L1E29
1E26  B1                 OR   C
1E27  20 02              JR   NZ,L1E2B
1E29  D7           L1E29: RST  $10           ; fel 146: ingen mall eller inget fält
1E2A  92                 DEFB $92

1E2B  0E 02        L1E2B: LD   C,$02         ; förbi mallens text på stacken
1E2D  09                 ADD  HL,BC
1E2E  39                 ADD  HL,SP
1E2F  4E                 LD   C,(HL)         ; C = filnumret
1E30  23                 INC  HL
1E31  23                 INC  HL             ; HL = SP före PRINT
1E32  DD 21 F4 FF        LD   IX,$FFF4       ; 12 bytes + mallens adress
```

Ramen för PRINT, 14 bytes vid IX = SP: +0 fältets typ med bit 7 (0 = fri form), +1 största antal siffror (7, 17, 126), +2 decimaler + 1 eller strängfältets bredd, +3 platser före punkten, +4,+5 SP att återvända till, +6 flaggor (2 = ett värde skrivet, bit 0 = ; eller , sist, bit 1 = INPUT:s prompt skriven), +7,+8 listans början (INPUT börjar om där efter fel), +9 tecken kvar i mallen (0 = från början), +A mallens längd (0 = ingen USING), +B filnumret, +C,+D mallens adress (bara med USING).

```
1E36  DD 39        L1E36: ADD  IX,SP         ; IX = SP = ramen
1E38  DD F9               LD   SP,IX
1E3A  DD 71 0B            LD   (IX+$0B),C   ; +B filen
1E3D  DD 75 04            LD   (IX+$04),L   ; +4,+5 SP att återvända till
1E40  DD 74 05            LD   (IX+$05),H
1E43  DD 77 0A            LD   (IX+$0A),A   ; +A mallens längd (0 = ingen)
1E46  AF                  XOR  A
1E47  DD 77 00            LD   (IX+$00),A   ; +0 inget fält
1E4A  DD 77 09            LD   (IX+$09),A   ; +9 mallen från början
1E4D  DD 73 07            LD   (IX+$07),E   ; +7,+8 listans början (INPUT)
1E50  DD 72 08            LD   (IX+$08),D
1E53  DD 36 06 00  L1E53: LD   (IX+$06),$00 ; L1E53: +6 = 0
```

Listan: ; skriver inget, , går till nästa zon om DIGITS + 8 = 14 tecken (med USING som ;), och en zon som hamnar på eller utanför radlängden ger ny rad (PRINT "A",,"B" skriver B på nästa rad). $B7 är INPUT-delen. Ett led är omvänd polsk kod och börjar alltid med en kod >= $C0; en lägre kod är listans slut. Ett tal som inte ryms på raden ger ny rad först (TAB(37);12345 på 40 tecken), men en sträng gör det inte, den bryts mitt i.

```
1E57  1A           L1E57: LD   A,(DE)
1E58  FE 3B               CP   $3B           ; ;
1E5A  CA 82 1F     L1E5A: JP   Z,L1F82
1E5D  FE 2C               CP   $2C           ; , ger CY
1E5F  37                  SCF
1E60  28 F8               JR   Z,L1E5A
1E62  FE B7               CP   $B7           ; $B7: INPUT-delen
1E64  CA 32 20            JP   Z,L2032
1E67  FE C0               CP   $C0           ; ingen uttryckskod: slut
1E69  DA EA 1F            JP   C,L1FEA
1E6C  DD 36 06 02         LD   (IX+$06),$02  ; +6 = 2: ett värde skrivs
1E70  DD 7E 0A            LD   A,(IX+$0A)    ; USING?
1E73  A7                  AND  A
1E74  28 0F               JR   Z,L1E85
1E76  D5                  PUSH DE
1E77  DD 7E 09            LD   A,(IX+$09)
1E7A  A7                  AND  A
1E7B  37                  SCF
1E7C  C4 D7 20            CALL NZ,S20D7      ; texten fram till fältet
1E7F  DC D7 20            CALL C,S20D7       ; inget fält: mallen från början
1E82  38 A5               JR   C,L1E29       ; inget fält alls: fel 146
1E84  D1                  POP  DE
1E85  E7           L1E85: RST  $20           ; ledet räknas ut, B = typ
1E86  CD A3 1D            CALL S1DA3         ; hit kommer ledet tillbaka (TAB)
1E89  78                  LD   A,B
1E8A  A7                  AND  A
1E8B  28 2B               JR   Z,L1EB8       ; flyttal
1E8D  3D                  DEC  A
1E8E  C2 40 1F            JP   NZ,L1F40      ; sträng
1E91  EB           L1E91: EX   DE,HL         ; L1E91: heltalet i HL
```

```
1E92  21 FA FF            LD   HL,$FFFA        ; 6 bytes för siffrorna
1E95  39                  ADD  HL,SP
1E96  F9                  LD   SP,HL
1E97  EB                  EX   DE,HL
1E98  D5                  PUSH DE
1E99  CD 00 45            CALL INTSTR          ; tecken eller blank, siffror
1E9C  E1                  POP  HL
1E9D  7B                  LD   A,E
1E9E  95                  SUB  L
1E9F  47                  LD   B,A             ; B = antal tecken
1EA0  3D                  DEC  A
1EA1  4F                  LD   C,A             ; C = exponenten = antal siffror
1EA2  CB 46               BIT  0,(HL)          ; '-' har bit 0
1EA4  28 02               JR   Z,L1EA8
1EA6  CB F8               SET  7,B             ; negativt
1EA8  1B          L1EA8:  DEC  DE              ; nollorna sist räknas bort
1EA9  05                  DEC  B
1EAA  1A                  LD   A,(DE)
1EAB  FE 30               CP   $30
1EAD  28 F9               JR   Z,L1EA8
1EAF  71                  LD   (HL),C          ; decimalform som från FTODEC
1EB0  2B                  DEC  HL
1EB1  F9                  LD   SP,HL
1EB2  70                  LD   (HL),B
1EB3  2E 07               LD   L,$07           ; 7 siffror
1EB5  7D                  LD   A,L
1EB6  18 12               JR   L1ECA

1EB8  CD 53 51    L1EB8:  CALL FTODEC          ; flyttalet i decimalform
1EBB  FD 7E 1E            LD   A,(IY+$1E)
1EBE  FE 05               CP   $05
1EC0  DE 00               SBC  A,$00
1EC2  87                  ADD  A,A
1EC3  3C                  INC  A               ; 7 eller 17 siffror
1EC4  E1          L1EC4:  POP  HL              ; L1EC4: talets storlek
1EC5  E5                  PUSH HL
1EC6  CB BD               RES  7,L
1EC8  2C                  INC  L
1EC9  2C                  INC  L
1ECA  26 00       L1ECA:  LD   H,$00           ; IX = ramen
1ECC  39                  ADD  HL,SP
1ECD  E5                  PUSH HL
1ECE  DD E1               POP  IX
1ED0  21 00 00            LD   HL,$0000
1ED3  DD 77 01            LD   (IX+$01),A      ; +1 största antal siffror
1ED6  39                  ADD  HL,SP
1ED7  47                  LD   B,A
1ED8  DD 4E 00            LD   C,(IX+$00)      ; C = fältet (0 = fri form)
1EDB  0C                  INC  C
1EDC  0D                  DEC  C
1EDD  28 0A               JR   Z,L1EE9
1EDF  DD 7E 03            LD   A,(IX+$03)
1EE2  DD 86 02            ADD  A,(IX+$02)
1EE5  B8                  CP   B
1EE6  30 01               JR   NC,L1EE9
1EE8  78                  LD   A,B
1EE9  C6 09       L1EE9:  ADD  A,$09           ; buffert för texten
1EEB  ED 44               NEG
1EED  16 FF               LD   D,$FF
1EEF  5F                  LD   E,A
1EF0  EB                  EX   DE,HL
1EF1  39                  ADD  HL,SP
1EF2  F9                  LD   SP,HL
1EF3  EB                  EX   DE,HL
1EF4  D5                  PUSH DE
1EF5  79                  LD   A,C
1EF6  B7                  OR   A
1EF7  28 0F               JR   Z,L1F08         ; fri form
1EF9  E6 40               AND  $40
```

```
1EFB  20 41              JR   NZ,L1F3E        ; strängfält: fel 146
1EFD  E5                 PUSH HL
1EFE  CD 9F 45           CALL FMUSE           ; enligt fältet
1F01  E1                 POP  HL
1F02  30 07              JR   NC,L1F0B
1F04  3E 25              LD   A,$25           ; ryms inte: % och fri form
1F06  12                 LD   (DE),A
1F07  13                 INC  DE
1F08  CD 2E 45     L1F08: CALL PRFREE         ; fri form
1F0B  E1           L1F0B: POP  HL             ; BC = textens längd
1F0C  7B                 LD   A,E
1F0D  95                 SUB  L
1F0E  06 00              LD   B,$00
1F10  4F                 LD   C,A
1F11  DD 7E 0B           LD   A,(IX+$0B)
1F14  CD 7A 11           CALL GETPOS          ; A = kolumnen, E = radlängden
1F17  1C                 INC  E
1F18  81                 ADD  A,C
1F19  BB                 CP   E               ; ryms inte på raden: ny rad
1F1A  E5                 PUSH HL
1F1B  C5                 PUSH BC
1F1C  D4 14 20           CALL NC,S2014
1F1F  C1                 POP  BC
1F20  E1                 POP  HL
1F21  CD 17 20     L1F21: CALL S2017          ; L1F21: texten till filen
1F24  DD F9        L1F24: LD   SP,IX          ; L1F24: värdet bort
1F26  CD 78 12           CALL S1278           ; DE = PRGPTR
1F29  C3 57 1E           JP   L1E57           ; nästa led

1F2C  CD 67 48     L1F2C: CALL USEDEC         ; strängen läses som tal
1F2F  3E 7E              LD   A,$7E           ; högst 126 siffror
1F31  30 91              JR   NC,L1EC4
1F33  3E D2              LD   A,$D2           fel 210: inget tal, efter RESUME 0
1F35  CD 33 00           CALL ERRJP
1F38  21 00 00           LD   HL,$0000        ; 0 efter RESUME
1F3B  C3 91 1E           JP   L1E91

1F3E  D7           L1F3E: RST  $10            fel 146: fältet passar inte
1F3F  92                 DEFB $92

1F40  DD E1        L1F40: POP  IX             ; L1F40: sträng
1F42  E1                 POP  HL
1F43  C1                 POP  BC
1F44  DD 39              ADD  IX,SP           ; IX = ramen
1F46  DD 7E 00           LD   A,(IX+$00)
1F49  5F                 LD   E,A
1F4A  A7                 AND  A
1F4B  28 D4              JR   Z,L1F21         ; inget fält: hela strängen
1F4D  FE C1              CP   $C1             ; & ($C1): hela strängen
1F4F  28 D0              JR   Z,L1F21
1F51  78                 LD   A,B
1F52  B7                 OR   A
1F53  20 E9              JR   NZ,L1F3E        ; 256 tecken eller fler: fel 146
1F55  CB 73              BIT  6,E
1F57  28 D3              JR   Z,L1F2C         ; talfält
1F59  DD 46 02           LD   B,(IX+$02)      ; B = fältets bredd
1F5C  79                 LD   A,C
1F5D  B8                 CP   B
1F5E  38 01              JR   C,L1F61         ; kapas till bredden
1F60  48                 LD   C,B
1F61  78           L1F61: LD   A,B             ; A = blanka som fattas
1F62  91                 SUB  C
1F63  F5                 PUSH AF
1F64  06 00              LD   B,$00
1F66  CD 17 20           CALL S2017
1F69  F1                 POP  AF
1F6A  DD F9              LD   SP,IX
1F6C  A7           L1F6C: AND  A              ; L1F6C: A blanka
1F6D  28 B5              JR   Z,L1F24
1F6F  4F                 LD   C,A
```

```
1F70  ED 44              NEG
1F72  6F                 LD   L,A
1F73  26 FF              LD   H,$FF
1F75  39                 ADD  HL,SP
1F76  F9                 LD   SP,HL
1F77  41                 LD   B,C
1F78  3E 20              LD   A,$20
1F7A  54                 LD   D,H
1F7B  5D                 LD   E,L
1F7C  12           L1F7C: LD   (DE),A
1F7D  13                 INC  DE
1F7E  10 FC              DJNZ L1F7C
1F80  18 9F              JR   L1F21

1F82  13           L1F82: INC  DE              ; L1F82: ; eller , (CY)
1F83  DD CB 06 C6        SET  0,(IX+$06)       ; +6 bit 0: ingen ny rad sist
1F87  D2 57 1E     L1F87: JP   NC,L1E57        ; ; går vidare
1F8A  DD 7E 0A           LD   A,(IX+$0A)
1F8D  A7                 AND  A
1F8E  20 F7              JR   NZ,L1F87         ; med USING som ;
1F90  CD A3 1D           CALL S1DA3
1F93  DD 7E 0B           LD   A,(IX+$0B)
1F96  CD 7A 11           CALL GETPOS           ; A = kolumnen, E = radlängden
1F99  47                 LD   B,A
1F9A  FD 7E 1F           LD   A,(IY+$1F)       ; zonen = DIGITS + 8
1F9D  C6 08              ADD  A,$08
1F9F  4F                 LD   C,A
1FA0  78                 LD   A,B
1FA1  91           L1FA1: SUB  C               ; kolumnen modulo zonen
1FA2  30 FD              JR   NC,L1FA1
1FA4  ED 44              NEG
1FA6  4F                 LD   C,A              ; C = blanka till nästa zon
1FA7  80                 ADD  A,B
1FA8  BB                 CP   E                ; utanför raden?
1FA9  79                 LD   A,C
1FAA  1E 00              LD   E,$00
1FAC  3F                 CCF
1FAD  30 BD        L1FAD: JR   NC,L1F6C        ; L1FAD: ryms: A blanka
1FAF  D5                 PUSH DE
1FB0  CD 14 20           CALL S2014            ; ny rad, sedan E blanka
1FB3  D1                 POP  DE
1FB4  7B                 LD   A,E
1FB5  18 B5              JR   L1F6C
```

TAB(n) är en strängfunktion som bara får stå som ett eget led i PRINT: koden efter måste vara slutkoden $B9 och anropet komma från PRINT, annars fel 135 (LEN(TAB(5)), A$=TAB(5), A+TAB(5)). n avrundas till närmaste; 0 och negativt ger fel 135. Kolumnen är (n-1) modulo radlängden (TAB(85) = TAB(5) på 40 tecken), och en kolumn som redan är passerad ger först en ny rad.

```
1FB7  1A           TAB:  LD   A,(DE)
1FB8  FE B9              CP   $B9              ; ledet måste sluta här ($B9)
1FBA  20 07              JR   NZ,L1FC3
1FBC  E1                 POP  HL
1FBD  E3                 EX   (SP),HL
1FBE  01 86 1E           LD   BC,$1E86         ; och ha kommit från PRINT
1FC1  ED 42              SBC  HL,BC
1FC3  3E 87        L1FC3: LD   A,$87           fel 135: TAB inte eget led
1FC5  C2 6B 3B           JP   NZ,ERRSP
1FC8  E1                 POP  HL               ; HL = n
1FC9  13                 INC  DE
1FCA  CD A3 1D           CALL S1DA3
1FCD  2B                 DEC  HL
1FCE  CB 7C              BIT  7,H              ; n < 1: fel 135
1FD0  C4 33 00           CALL NZ,ERRJP
1FD3  DD 21 00 00        LD   IX,$0000
1FD7  DD 39              ADD  IX,SP
1FD9  DD 7E 0B           LD   A,(IX+$0B)
1FDC  CD 7A 11           CALL GETPOS           ; A = kolumnen, E = radlängden
1FDF  16 00              LD   D,$00
```

```
1FE1  F5                PUSH AF
1FE2  CD 2D 4B          CALL DIV16          ; DE = (n - 1) MOD radlängden
1FE5  C1                POP  BC
1FE6  7B                LD   A,E
1FE7  90                SUB  B              ; blanka till kolumnen, CY: passerad
1FE8  18 C3             JR   L1FAD
```

Listans slut: med USING skrivs mallens text fram till nästa fält; finns ett fält kvar blir det fel 146 (PRINT USING "_##=###";5 skriver #5= och sedan Error 146). Ny rad om listan inte slutade med ; eller ,. Sedan bort med ramen och mallen.

```
1FEA  D5          L1FEA:  PUSH DE             ; L1FEA: listans slut (egenhet)
1FEB  DD 7E 09            LD   A,(IX+$09)
1FEE  A7                  AND  A
1FEF  37                  SCF
1FF0  C4 D7 20            CALL NZ,S20D7       ; resten av mallens text
1FF3  D2 3E 1F            JP   NC,L1F3E       ; fält kvar: fel 146
1FF6  DD CB 06 46         BIT  0,(IX+$06)     ; slutade med ; eller ,?
1FFA  CC 14 20            CALL Z,S2014        ; annars ny rad
1FFD  D1                  POP  DE
1FFE  E1                  POP  HL             ; ramen bort
1FFF  E1                  POP  HL
2000  E1                  POP  HL
2001  F9                  LD   SP,HL          ; SP före PRINT
2002  C9                  RET

2003  FD CB 23 4E NOTRAC: BIT  1,(IY+$23)     NO TRACE
2007  C8                  RET  Z
2008  FD CB 23 8E         RES  1,(IY+$23)
200C  FD 7E 35            LD   A,(IY+$35)
200F  CD FC 3C            CALL S3CFC          ; ny rad till TRACE-filen
2012  18 06               JR   L201A

2014  CD FC 3C  S2014:    CALL S3CFC          ; S2014: ny rad till filen +B
2017  DD 7E 0B  S2017:    LD   A,(IX+$0B)     ; S2017: HL, BC till filen +B
201A  DD E5      L201A:   PUSH IX             ; L201A: filen i A; IX, DE sparas
201C  D5                  PUSH DE
201D  CD 97 11            CALL PRFILE
2020  D1                  POP  DE
2021  DD E1               POP  IX
2023  C9                  RET

2024  2E 00       S2024:  LD   L,$00          ; S2024: #n, ger L = n (0 utan)
2026  1A                  LD   A,(DE)
2027  FE 23               CP   $23
2029  C0                  RET  NZ
202A  13                  INC  DE
202B  E7                  RST  $20            ; filnumret
202C  1A                  LD   A,(DE)
202D  FE 2C               CP   $2C
202F  C0                  RET  NZ
2030  13                  INC  DE             ; DE efter kommat
2031  C9                  RET
```

INPUT-delen ($B7 i PRINT-listan, efter prompttexten): "? " skrivs bara till CON: och bara om ingen prompt skrivits. DVINP läser raden; BREAK ger Stop (CTRL-C under INPUT, och efter CON börjar INPUT om). RDVALS tolkar värdena. Andra enheter än CON: får ingen ny rad efteråt; CON: får det om inte ; eller , följer. Vid fel (t.ex. bokstäver till ett tal, fel 210) körs efter RESUME hela satsen om med prompten, utan ny rad före felmeddelandet.

```
2032  21 60 FF  L2032:  LD   HL,$FF60       ; L2032: INPUT, 160 bytes buffert
2035  39                 ADD  HL,SP
2036  F9                 LD   SP,HL
2037  DD E5              PUSH IX
2039  D5                 PUSH DE
203A  E5                 PUSH HL
203B  DD CB 06 4E        BIT  1,(IX+$06)     ; prompttext skriven?
203F  DD 36 06 00        LD   (IX+$06),$00
2043  DD 7E 0B           LD   A,(IX+$0B)
2046  F5                 PUSH AF
2047  CD 88 11           CALL FNDFIL         ; IX = filens FCB
204A  F1                 POP  AF
```

```
204B  20 0D              JR   NZ,L205A          ; skriven: inget frågetecken
204D  21 EC 20           LD   HL,$20EC          ; "? "
2050  01 02 00           LD   BC,$0002
2053  DD CB 03 56        BIT  2,(IX+$03)        ; bara på CON: (FCB+3 bit 2)
2057  C4 9A 11           CALL NZ,DVPRT
205A  E1           L205A: POP  HL
205B  01 A0 00           LD   BC,$00A0
205E  E5                 PUSH HL
205F  CD 51 49           CALL DVINP             ; läs raden
2062  38 07              JR   C,L206B
2064  FD 7E 23           LD   A,(IY+$23)        ; BREAK: CY och A = 0 (Stop)
2067  E6 01              AND  $01
2069  C6 FF              ADD  A,$FF
206B  E1           L206B: POP  HL
206C  D1                 POP  DE
206D  DD 46 03           LD   B,(IX+$03)
2070  C5                 PUSH BC
2071  D4 12 29           CALL NC,RDVALS         ; tolka värdena, B = FCB+3
2074  08                 EX   AF,AF'
2075  F1                 POP  AF
2076  DD E1              POP  IX
2078  CB 57              BIT  2,A               ; CON:?
207A  20 04              JR   NZ,L2080
207C  DD 36 06 01        LD   (IX+$06),$01      ; annan enhet: ingen ny rad
2080  21 A0 00     L2080: LD   HL,$00A0          ; bufferten bort
2083  39                 ADD  HL,SP
2084  F9                 LD   SP,HL
2085  08                 EX   AF,AF'
2086  D2 57 1E           JP   NC,L1E57          ; fortsätt listan
2089  CD 33 00           CALL ERRJP             ; fel; efter RESUME hela
208C  DD 5E 07           LD   E,(IX+$07)        ; satsen om, prompten också
208F  DD 56 08           LD   D,(IX+$08)
2092  C3 53 1E           JP   L1E53
```

Mallen: S2096 letar från där förra fältet slutade (+9 tecken kvar; 0 och A = 0 börjar om från början) tecken för tecken med USEFLD efter nästa fält och lägger in det i ramen (+0, +2, +3, +9). Ut: HL = textens början, A = fältets början (låg byte), D = $FF när inget fält fanns. S20D7 skriver texten fram till fältet och ger NC när ett fält hittades, CY annars; _ gör nästa tecken till text.

```
2095  AF           S2095: XOR  A                 ; S2095: A = 0, mallen får börja om
2096  DD 6E 0C     S2096: LD   L,(IX+$0C)        ; S2096: HL = mallen
2099  DD 66 0D            LD   H,(IX+$0D)
209C  DD 4E 09            LD   C,(IX+$09)
209F  DD 46 0A            LD   B,(IX+$0A)
20A2  B1                  OR   C                ; tecken kvar?
20A3  28 0A               JR   Z,L20AF
20A5  79                  LD   A,C              ; förbi de använda tecknen
20A6  F5                  PUSH AF
20A7  ED 44               NEG
20A9  80                  ADD  A,B
20AA  4F                  LD   C,A
20AB  06 00               LD   B,$00
20AD  09                  ADD  HL,BC
20AE  C1                  POP  BC
20AF  E5           L20AF: PUSH HL
20B0  78                  LD   A,B
20B1  B7                  OR   A
20B2  28 0C               JR   Z,L20C0
20B4  C5           L20B4: PUSH BC
20B5  E5                  PUSH HL
20B6  CD 3D 47            CALL USEFLD           ; fält här?
20B9  30 09               JR   NC,L20C4
20BB  E1                  POP  HL
20BC  C1                  POP  BC
20BD  23                  INC  HL
20BE  10 F4               DJNZ L20B4            ; nästa tecken
20C0  C5           L20C0: PUSH BC
20C1  E5                  PUSH HL
```

```
20C2  16 FF              LD   D,$FF          ; D = $FF: inget fält
20C4  DD 70 09     L20C4: LD   (IX+$09),B    ; +9 tecken kvar efter fältet
20C7  DD 71 02            LD   (IX+$02),C    ; +2 decimaler + 1 eller bredd
20CA  DD 73 03            LD   (IX+$03),E    ; +3 platser före punkten
20CD  CB FA               SET  7,D
20CF  DD 72 00            LD   (IX+$00),D    ; +0 fältets typ, bit 7
20D2  E1                  POP  HL
20D3  C1                  POP  BC
20D4  7D                  LD   A,L           ; A = fältets början (låg byte)
20D5  E1                  POP  HL            ; HL = textens början
20D6  C9                  RET

20D7  CD 95 20     S20D7: CALL S2095         ; S20D7: texten före fältet
20DA  95           L20DA: SUB  L
20DB  4F                  LD   C,A           ; C = textens längd
20DC  06 00               LD   B,$00
20DE  C4 17 20            CALL NZ,S2017
20E1  7A                  LD   A,D
20E2  C6 02               ADD  A,$02         ; $FE: _, $FF: inget fält (CY)
20E4  C0                  RET  NZ
20E5  7A                  LD   A,D
20E6  CD 96 20            CALL S2096         ; _: nästa tecken som text
20E9  2B                  DEC  HL
20EA  18 EE               JR   L20DA

20EC  3F 20               DEFB $3F,$20       ; "? " för INPUT
```

Koder $D5-$DC*: talkonstant, antal bytes = kod -* $D4*. Konstanten är ett 8-bytestal där nollorna sist är borttagna (.5 =* $D5 $80*, .1 =* $DC + *8 bytes);* NUMPSH *fyller ut med nollor på stacken och avrundar till 4 bytes (*FRND4*) när* DOUBLE *inte gäller. Konstanterna sparas alltså alltid med full noggrannhet. Ett tal utan % är en flyttalskonstant även när det tilldelas ett heltal (A%=5 =* $D6 $83 $20 + $81 FTOI*), och uttryck med konstanter räknas inte ut vid kompileringen (1/3, -7).*

```
20EE  D6 D4        NUMCON: SUB  $D4
20F0  EB                   EX   DE,HL
20F1  CD 00 48             CALL NUMPSH
20F4  EB                   EX   DE,HL
20F5  EF                   RST  $28
20F6  E1           ABS:    POP  HL
20F7  CB BC                RES  7,H           ; tecknet i det översta ordet
20F9  18 1C                JR   L2117

20FB  E1           FNEG:   POP  HL
20FC  3E 80                LD   A,$80         ; byt tecken
20FE  AC                   XOR  H
20FF  67                   LD   H,A
2100  18 15                JR   L2117

2102  C1           SGN:    POP  BC            ; C = exponenten, B = tecknet
2103  E1                   POP  HL
2104  FD CB 1E 5E          BIT  3,(IY+$1E)    ; DOUBLE: två ord till
2108  28 02                JR   Z,L210C
210A  E1                   POP  HL
210B  E1                   POP  HL
210C  79           L210C:  LD   A,C
210D  A7                   AND  A             ; exponent 0: talet är 0
210E  78                   LD   A,B
210F  07                   RLCA               ; CY = tecknet
2110  28 2A        L2110:  JR   Z,L213C
2112  ED 62                SBC  HL,HL         ; HL = -1 eller
2114  38 01                JR   C,L2117
2116  23                   INC  HL            ; 1
2117  E5           L2117:  PUSH HL            ; L2117: HL tillbaka
2118  EF                   RST  $28
```

FCMP: jämför två flyttal; koderna är $89*,* $8C*,* $8F*,* $92*,* $95 *och* $98 *för =, <>, <, >=, > och <=.* FCOMP *ger CY när a < b och Z när talen är lika. Flaggorna sparas i AF', båda talen tas bort, och koden slås upp en gång till med basen* $0BC7*, som ger* L2139 *(=),* L2141 *(<>),* L2157 *(<),* L215C *(>=),* L2149 *(>) och* L2150 *(<=). Svaret är ett heltal, -1 eller 0.* FDISP *gör samma omslag för + - * / (koderna* $9B*,* $9E*,* $A0*,* $A2 *ger FADD, FSUB, FMUL, FDIV) och* FNFLT2 *för de matematiska funktionerna med basen* $0CC9*. CY efter räkningen ger fel 130 (*FARITH*) eller 142 (*FNFLT*: LOG(-1), SQR(-1), EXP(1000)).*

```
2119  ED 53 3A FF  FCMP:   LD   (PRGPTR),DE
211D  CD 61 4C             CALL FCOMP
2120  08                   EX   AF,AF'          ; flaggorna i AF'
2121  FD 6E 1E             LD   L,(IY+$1E)
2124  26 00                LD   H,$00
2126  29                   ADD  HL,HL
2127  39                   ADD  HL,SP
2128  F9                   LD   SP,HL           ; båda talen bort
2129  CD 78 12             CALL S1278           ; DE = PRGPTR
212C  18 04                JR   L2132

212E  ED 53 3A FF  FDISP:  LD   (PRGPTR),DE
2132  1B          L2132:   DEC  DE             ; L2132: koden en gång till
2133  21 C7 0B             LD   HL,$0BC7        ; bas $0BC7 = JPTABL + $102
2136  C3 B1 1D             JP   L1DB1

2139  08          L2139:   EX   AF,AF'          ; =
213A  28 08                JR   Z,L2144
213C  21 00 00    L213C:   LD   HL,$0000       ; L213C: falskt (0)
213F  E5                   PUSH HL
2140  EF                   RST  $28
2141  08          L2141:   EX   AF,AF'          ; <>
2142  28 F8                JR   Z,L213C
2144  21 FF FF    L2144:   LD   HL,$FFFF       ; L2144: sant (-1)
2147  E5                   PUSH HL
2148  EF                   RST  $28
2149  08          L2149:   EX   AF,AF'          ; >
214A  28 F0                JR   Z,L213C
214C  38 EE                JR   C,L213C
214E  18 F4                JR   L2144

2150  08          L2150:   EX   AF,AF'          ; <=
2151  28 F1                JR   Z,L2144
2153  38 EF                JR   C,L2144
2155  18 E5                JR   L213C

2157  08          L2157:   EX   AF,AF'          ; <
2158  38 EA                JR   C,L2144
215A  18 E0                JR   L213C

215C  08          L215C:   EX   AF,AF'          ; >=
215D  38 DD                JR   C,L213C
215F  18 E3                JR   L2144

2161  13          FTOI2:   INC  DE             ; $AD: förbi avståndsbyten
```

Typomvandling: FTOI ($81) *flyttal -> heltal,* ITOF ($82) *heltal -> flyttal. FTOI avrundar till närmaste (2.5 -> 3, -2.5 -> -3) och räknar modulo 65536 när |x| < 65536 (A%=40000 ger -25536, 32767.6 ger -32768, 65535.5 ger -1); fel 132* ($84) *först från 65536.* $AD (FTOI2) *och* $AE (ITOF2) *gör samma sak men hoppar först över en byte. De står i argumentlistan till en egen funktion: kompilatorn lägger* $AA-$AC *efter varje argument, följt av avståndet bakåt till förra argumentets byte (0 för det första), och när ett argument har en annan typ än parametern i DEF byter RUN koden till* $AD *eller* $AE*. Med DEF FNB(X,Y%) blir A%=FNB(2%,3)* DF AB 00 D6 82 40 AA FB... *före RUN och* DF AE 00 D6 82 40 AD FB... *efter.*

```
2162  CD A3 1D    FTOI:    CALL S1DA3(gränsfall)
2165  CD F3 4C    L2165:   CALL FTOIR
2168  3E 84                LD   A,$84           fel 132: |x| >= 65536
216A  DC 33 00             CALL C,ERRJP
216D  18 11                JR   L2180

216F  13          ITOF2:   INC  DE             ; $AE: förbi avståndsbyten
2170  CD A3 1D    ITOF:    CALL S1DA3           heltal -> flyttal
2173  CD C0 4C             CALL ITOFS
2176  18 03                JR   L217B

2178  CD 2E 21    FARITH:  CALL FDISP           ; + - * / för flyttal
217B  3E 82       L217B:   LD   A,$82           fel 130: spill i flyttalsräkningen
217D  DC 33 00    L217D:   CALL C,ERRJP
2180  ED 5B 3A FF L2180:   LD   DE,(PRGPTR)     ; L2180: DE = PRGPTR, nästa kod
2184  EF                   RST  $28
2185  CD 97 21    FNFLT:   CALL FNFLT2          ; SIN, COS, TAN, ATN, SQR, EXP, LOG ...
2188  3E 8E                LD   A,$8E           ; fel 142: utanför definitionsmängden
218A  18 F1                JR   L217D
```

```
218C  E1           VAL:    POP  HL
218D  CD 90 52             CALL DECTOF          inledande blanka går, annat tecken fel 210
2190  18 E9                JR   L217B

2192  CD 78 12     S2192:  CALL S1278           ; S2192: DE = PRGPTR
2195  18 04                JR   L219B

2197  ED 53 3A FF FNFLT2: LD   (PRGPTR),DE
219B  1B           L219B:  DEC  DE              ; L219B: koden en gång till
219C  21 C9 0C             LD   HL,$0CC9        ; bas $0CC9 = FNTAB + $6C
219F  C3 B1 1D             JP   L1DB1
```

ERRCOD (ERR), PEEK och SMLINT har samma svans: värdet (felnumret i IY+$44, byten på adressen vid SP, koden - $DD) läggs som heltal vid SP, och RST $28 (EXECNX) tar nästa kod. Byten $01 på $21A5 och $21A8 är början på LD BC,nn, som slukar de två byten efter, så att ingången ovanför hoppar förbi dem utan JR. Förstör BC.

```
21A2  FD 7E 44     ERRCOD: LD   A,(IY+$44)      ERR
21A5  01                   DEFB $01             LD BC,$7EE1 (överlappar nästa)
21A6  E1           PEEK:   POP  HL              adressen
21A7  7E                   LD   A,(HL)          PEEK
21A8  01                   DEFB $01             LD BC,$DDD6 (överlappar nästa)
```

SMLINT: kod $DD-$FF ger heltalet kod - $DD på stacken. Kompilatorn gör bara $DD-$ED (0%-16%, $197D); större heltalskonstanter blir INTCON ($C7) med ett 16-bitars heltal efter koden.

```
21A9  D6 DD        SMLINT: SUB  $DD             koden $DD–$FF ger 0–34
21AB  6F                   LD   L,A
21AC  26 00                LD   H,$00
21AE  E5                   PUSH HL              heltalet på stacken
21AF  EF                   RST  $28             nästa kod
21B0  EB           INTCON: EX   DE,HL           heltalskonstant, 2 bytes
21B1  5E                   LD   E,(HL)
21B2  23                   INC  HL
21B3  56                   LD   D,(HL)
21B4  23                   INC  HL
21B5  EB                   EX   DE,HL
21B6  E5                   PUSH HL
21B7  EF                   RST  $28
21B8  E1           INEG:   POP  HL
21B9  CD 6D 22             CALL NEGHL           -(-32768%) = -32768, utan fel
21BC  E5                   PUSH HL
21BD  EF                   RST  $28
```

Heltalsoperatorer. Operatorernas koder i OPKW används direkt som körkoder; varje jämförelse har tre koder i följd: flyttal (FCMP), heltal och sträng (SCMP), t.ex. = $89/$8A/$8B. Sant = -1, falskt = 0. + - * / har flyttal på grundkoden och heltal på koden + 1 ($9C IADD, $9D strängsammanslagning SCAT, $9F ISUB, $A1 IMUL, $A3 IDIV).

```
21BE  E1           IEQV:   POP  HL
21BF  C1                   POP  BC
21C0  78                   LD   A,B
21C1  AC                   XOR  H
21C2  2F                   CPL                  EQV = NOT (a XOR b)
21C3  67                   LD   H,A
21C4  79                   LD   A,C
21C5  AD                   XOR  L
21C6  2F                   CPL
21C7  6F                   LD   L,A
21C8  E5                   PUSH HL
21C9  EF                   RST  $28
21CA  E1           IIMP:   POP  HL
21CB  C1                   POP  BC
21CC  78                   LD   A,B
21CD  2F                   CPL                  IMP = NOT a OR b
21CE  B4                   OR   H
21CF  67                   LD   H,A
21D0  79                   LD   A,C
21D1  2F                   CPL
21D2  B5                   OR   L
21D3  6F                   LD   L,A
```

```
21D4  E5                PUSH HL
21D5  EF                RST  $28
21D6  E1          IOR:  POP  HL
21D7  C1                POP  BC
21D8  79                LD   A,C
21D9  B5                OR   L
21DA  6F                LD   L,A
21DB  78                LD   A,B
21DC  B4                OR   H
21DD  67                LD   H,A
21DE  E5                PUSH HL
21DF  EF                RST  $28
21E0  E1          IAND: POP  HL
21E1  C1                POP  BC
21E2  79                LD   A,C
21E3  A5                AND  L
21E4  6F                LD   L,A
21E5  78                LD   A,B
21E6  A4                AND  H
21E7  67                LD   H,A
21E8  E5                PUSH HL
21E9  EF                RST  $28
21EA  21 FF FF    INOT: LD   HL,$FFFF       NOT = XOR -1
21ED  E5                PUSH HL
21EE  E1          IXOR: POP  HL
21EF  C1                POP  BC
21F0  79                LD   A,C
21F1  AD                XOR  L
21F2  6F                LD   L,A
21F3  78                LD   A,B
21F4  AC                XOR  H
21F5  67                LD   H,A
21F6  E5                PUSH HL
21F7  EF                RST  $28
21F8  E1          IEQ:  POP  HL
21F9  C1                POP  BC
21FA  ED 42             SBC  HL,BC          CY = 0 från EXECA:s ADD
21FC  28 0C             JR   Z,TRUE
21FE  21 00 00    FALSE: LD  HL,$0000
2201  E5                PUSH HL
2202  EF                RST  $28
2203  E1          INE:  POP  HL
2204  C1                POP  BC
2205  ED 42             SBC  HL,BC
2207  CA A8 1D          JP   Z,L1DA8        lika: HL = 0 = falskt
220A  21 FF FF    TRUE: LD   HL,$FFFF
220D  E5                PUSH HL
220E  EF                RST  $28
220F  C1          ILT:  POP  BC
2210  E1                POP  HL
2211  ED 42       L2211: SBC  HL,BC
2213  EA 22 22          JP   PE,L2222       spill: tecknet omvänt
2216  F2 FE 21    L2216: JP   P,FALSE
2219  18 EF             JR   TRUE

221B  C1          IGE:  POP  BC
221C  E1                POP  HL
221D  ED 42       L221D: SBC  HL,BC
221F  EA 16 22          JP   PE,L2216
2222  FA FE 21    L2222: JP   M,FALSE
2225  18 E3             JR   TRUE

2227  E1          ILE:  POP  HL             a <= b som b >= a
2228  C1                POP  BC
2229  18 F2             JR   L221D

222B  E1          IGT:  POP  HL             a > b som b < a
222C  C1                POP  BC
222D  18 E2             JR   L2211
```

```
222F  C1           IADD:   POP  BC          heltal + (gränsfall)
2230  E1                   POP  HL
2231  09                   ADD  HL,BC       slår runt utan fel
2232  E5                   PUSH HL
2233  EF                   RST  $28
2234  C1           ISUB:   POP  BC
2235  E1                   POP  HL
2236  ED 42                SBC  HL,BC
2238  E5                   PUSH HL
2239  EF                   RST  $28
```

*MOD, IDIV och IMUL delar IMULDV; A väljer (0 = *, 1 = MOD, bit 7 = /). Beloppen räknas utan tecken (MUL16, DIV16) och tecknet sätts efteråt. Produkten får bli upp till 65535, så 32768-65535 slår runt utan fel (200%*200% = -25536); större och division med 0 ger fel 132. Division avrundar nedåt, inte mot noll: 7%/(-2%) = -4. MOD(a,b) är resten till den kvoten och får divisorns tecken: MOD(-7%,2%) = 1, MOD(7%,-2%) = -1. Unärt minus binder svagare än / (-7%/2% = -3, men (-7%)/2% = -4). -32768%/(-1%) = -32768 utan fel.*

```
223A  AF           MOD:    XOR  A
223B  3C                   INC  A           A = 1: MOD
223C  21                   DEFB $21         LD HL,$AFF6 (överlappar nästa)
223D  F6           IDIV:   DEFB $F6         A har bit 7: division; OR $AF (överlappar nästa)
223E  AF           IMUL:   XOR  A           A = 0: multiplikation
223F  E1                   POP  HL
2240  EB                   EX   DE,HL
2241  E3                   EX   (SP),HL     HL = a, DE = b, stacken = kodpekaren
2242  CD 4F 22             CALL IMULDV
2245  D1                   POP  DE
2246  E5                   PUSH HL
2247  EF                   RST  $28
2248  E1           SWAP:   POP  HL          ; byt höga och låga byten
2249  7C                   LD   A,H
224A  65                   LD   H,L
224B  6F                   LD   L,A
224C  E5                   PUSH HL
224D  EF                   RST  $28
224E  AF           S224E:  XOR  A           ; S224E: HL x DE
224F  08           IMULDV: EX   AF,AF'
2250  06 02                LD   B,$02
2252  7C                   LD   A,H
2253  F5                   PUSH AF
2254  AA                   XOR  D           resultatets tecken
2255  F5                   PUSH AF
2256  EB           L2256:  EX   DE,HL       beloppen, även $8000
2257  7C                   LD   A,H
2258  A7                   AND  A
2259  FC 6D 22             CALL M,NEGHL
225C  10 F8                DJNZ L2256
225E  08                   EX   AF,AF'
225F  20 14                JR   NZ,L2275
2261  08                   EX   AF,AF'
2262  CD 19 4B             CALL MUL16       |a| * |b|, CY över 65535
2265  3E 84        L2265:  LD   A,$84       fel 132: heltalsspill
2267  DC 33 00             CALL C,ERRJP
226A  F1                   POP  AF
226B  C1                   POP  BC
226C  F0                   RET  P           positivt: klart
226D  AF           NEGHL:  XOR  A           HL = -HL
226E  95                   SUB  L
226F  6F                   LD   L,A
2270  3E 00                LD   A,$00
2272  9C                   SBC  A,H
2273  67                   LD   H,A
2274  C9                   RET

2275  08           L2275:  EX   AF,AF'
2276  CD 2D 4B             CALL DIV16       HL = kvot, DE = rest; CY vid b = 0
2279  38 EA                JR   C,L2265
227B  F1                   POP  AF
227C  F2 8B 22             JP   P,L228B
```

```
227F  7A                LD   A,D              negativt: avrunda nedåt
2280  B3                OR   E
2281  28 05             JR   Z,L2288
2283  EB                EX   DE,HL(egenhet)
2284  13                INC  DE
2285  ED 42             SBC  HL,BC            rest - |b|
2287  EB                EX   DE,HL
2288  CD 6D 22    L2288: CALL NEGHL
228B  C1          L228B: POP  BC
228C  08                EX   AF,AF'
228D  F8                RET  M                division: kvoten
228E  EB                EX   DE,HL
228F  B0                OR   B                MOD: restens tecken efter a
2290  F0                RET  P
2291  18 DA             JR   NEGHL
```

Variabler. Variabeltypen står i de två lägsta bitarna av koden:

0 = flyttal, 1 = heltal, 2 = sträng. Efter koden kommer variabelns 2-bytesreferens (S0F63 räknar ut adressen).

```
    $C0-$C2  hämta värdet (LDFVAR, LDIVAR, LDSVAR)
    $C4-$C6  lägg adressen på stacken (VARADR)
    $C8-$CA  vektorelement (ARRELM)
    Tio sätt att avsluta uttrycksberäkningen (RET till anroparen,
    B = typ 0/1/2, variabelns adress sparas i VFF36):
    $B1-$B3  tilldela värdet till adress på stacken, t.ex. vektorelement
             (SFARR, SIARR, SSARR)
    $B4-$B6  tilldela enkel variabel, referensen följer (STFVAR, STIVAR,
             STSVAR)
    $B7-$B9  lämna värdet (FRETF, FRETI, FRETS)
    $BA-$BC  lämna en pekare till variabeln (PTRF, PTRI, PTRS), för READ,
             INPUT o.d.
    $BF      bara RET utan värde (sist i DIM)
    Samma indelning (tre typer x värde/pekare/tilldelning + ett tomt slut,
    alltså tio slutkoder) beskrivs för ABC80-BASIC:en (Finnved 1979,
    kap. 5), och i ABC80 lades adressen för A(4%) i A(4%)=... också ut
    först, med tilldelningen sist.
2293  CD 63 0F    VARADR: CALL VARREF
2296  E5                PUSH HL
2297  EF                RST  $28
2298  CD 63 0F    STIVAR: CALL VARREF
229B  C1                POP  BC
229C  71          L229C: LD   (HL),C
229D  E5                PUSH HL
229E  23                INC  HL
229F  70                LD   (HL),B
22A0  18 2C             JR   PTRI

22A2  C1          SIARR: POP  BC
22A3  E1                POP  HL
22A4  18 F6             JR   L229C

22A6  CD 63 0F    LDIVAR: CALL VARREF
22A9  E5                PUSH HL
22AA  E1          PEEK2: POP  HL
22AB  4E                LD   C,(HL)
22AC  23                INC  HL
22AD  46                LD   B,(HL)
22AE  C5                PUSH BC
22AF  EF                RST  $28
22B0  CD 63 0F    STFVAR: CALL VARREF
22B3  FD 7E 1E            LD   A,(IY+$1E)
22B6  0F                RRCA                  2 eller 4 ord, CY = 0
22B7  C1          L22B7: POP  BC
22B8  71                LD   (HL),C
22B9  23                INC  HL
22BA  70                LD   (HL),B
22BB  23                INC  HL
22BC  3D                DEC  A
22BD  20 F8             JR   NZ,L22B7
```

```
22BF  47                LD   B,A
22C0  38 06             JR   C,PTRF
22C2  FD 4E 1E          LD   C,(IY+$1E)
22C5  ED 42             SBC  HL,BC          HL = variabelns början
22C7  E5                PUSH HL
22C8  06 00       PTRF: LD   B,$00
22CA  21                DEFB $21            LD HL,$0206 (överlappar nästa)
22CB  06 02       PTRS: LD   B,$02
22CD  21                DEFB $21            LD HL,$0106 (överlappar nästa)
22CE  06 01       PTRI: LD   B,$01
22D0  E1                POP  HL
22D1  22 36 FF          LD   (VFF36),HL     variabelns adress
22D4  C9                RET

22D5  FD 4E 1E    SFARR: LD   C,(IY+$1E)
22D8  69                LD   L,C
22D9  26 00             LD   H,$00
22DB  39                ADD  HL,SP
22DC  7E                LD   A,(HL)
22DD  23                INC  HL
22DE  66                LD   H,(HL)
22DF  6F                LD   L,A
22E0  79                LD   A,C
22E1  0F                RRCA
22E2  37                SCF                 CY: adressen ligger kvar
22E3  18 D2             JR   L22B7

22E5  CD 63 0F    LDFVAR: CALL VARREF
22E8  DD                DEFB $DD            LD IX,$57E0 (överlappar nästa)
22E9  21 E0 57    PI:     LD   HL,$57E0
22EC  E5                PUSH HL
22ED  D9          FPUSH: EXX
22EE  E1                POP  HL
22EF  CD 25 4C          CALL FLOAD
22F2  D9                EXX
22F3  EF                RST  $28
22F4  CD 63 0F    STSVAR: CALL VARREF       HL = variabeln
22F7  DD E1       L22F7: POP  IX
22F9  DD 39             ADD  IX,SP          IX = ovanför strängens text
22FB  C1                POP  BC
22FC  E3                EX   (SP),HL        HL = längden
22FD  DD E3             EX   (SP),IX        IX = variabeln
22FF  D5                PUSH DE
2300  C5                PUSH BC
2301  44                LD   B,H
2302  4D                LD   C,L
2303  CD 50 24          CALL SVSET          buffert vid behov, annars fel 137
2306  E1                POP  HL
2307  78                LD   A,B
2308  B1                OR   C
2309  28 02             JR   Z,L230D
230B  ED B0             LDIR                texten till bufferten
230D  D1          L230D: POP  DE
230E  E1                POP  HL
230F  F9                LD   SP,HL          strängen bort från stacken
2310  C1                POP  BC
2311  C1                POP  BC
2312  DD E5             PUSH IX
2314  18 B5             JR   PTRS

2316  E1          SSARR: POP  HL            tilldela strängvektorelement
2317  23                INC  HL
2318  23                INC  HL
2319  E5                PUSH HL
231A  39                ADD  HL,SP
231B  CD DC 30          CALL S30DC
231E  60                LD   H,B
231F  69                LD   L,C
2320  18 D5             JR   L22F7
```

```
2322  CD 63 0F     LDSVAR: CALL VARREF
2325  E5                   PUSH HL
2326  DD E1        L2326:  POP  IX
2328  CD 5D 27             CALL IXBCHL
232B  C5                   PUSH BC
232C  E5                   PUSH HL
232D  21 00 00     L232D:  LD   HL,$0000       0: texten ligger inte på stacken
2330  E5                   PUSH HL
2331  EF                   RST  $28
```

ARRELM ($C8-$CA)*: fältelement. Koden följs av antalet index och variabelreferensen; indexen ligger på stacken.* VDSC0 *läser in fältet (IX) och* AUTODM *dimensionerar det till 10 om det inte finns. För varje dimension: index - undre gräns måste vara < antalet, annars fel* $83 *(131); adressen = data (+2,+3) + summan av (index - undre) x steg. Fältets adress sparas i* LSTARR *(för* VAROOT*). Lämnar elementets adress. Ett fält (variabelns post): +0,+1 storlek (0 = inte dimensionerat), +2,+3 dataadress, +4 antal dimensioner, +5 elementets storlek (4/8, 2 eller 6 bytes), och för varje dimension 6 bytes: +6 undre gräns, +8 antal element, +A steg i bytes.*

```
2332  1B           ARRELM: DEC  DE
2333  CD 5C 24             CALL VDSC0
2336  CD 7E 23             CALL AUTODM           odimensionerat: (10)
2339  DD 22 38 FF          LD   (LSTARR),IX
233D  DD 6E 04             LD   L,(IX+$04)
2340  45                   LD   B,L
2341  2C                   INC  L
2342  26 00                LD   H,$00
2344  29                   ADD  HL,HL
2345  39                   ADD  HL,SP
2346  E5                   PUSH HL
2347  CD 48 27             CALL IXDE2
234A  EB                   EX   DE,HL
234B  C5           L234B:  PUSH BC
234C  E5                   PUSH HL
234D  EB                   EX   DE,HL
234E  CD 46 32             CALL S3246
2351  E5                   PUSH HL
2352  EB                   EX   DE,HL
2353  CD 3A 27             CALL IXBC6
2356  AF                   XOR  A
2357  ED 42                SBC  HL,BC            index - undre gräns
2359  AF                   XOR  A
235A  CD 41 27             CALL IXBC8
235D  ED 42                SBC  HL,BC            - antalet
235F  3E 83                LD   A,$83            fel 131: index utanför fältet
2361  D2 6B 3B             JP   NC,ERRSP
2364  09                   ADD  HL,BC
2365  EB                   EX   DE,HL
2366  CD 75 27             CALL IXHLA
2369  CD 19 4B             CALL MUL16            x steget
236C  D1                   POP  DE
236D  C1                   POP  BC
236E  09                   ADD  HL,BC
236F  01 06 00             LD   BC,$0006
2372  DD 09                ADD  IX,BC
2374  C1                   POP  BC
2375  10 D4                DJNZ L234B            nästa dimension
2377  DD E1                POP  IX
2379  D1                   POP  DE
237A  DD F9                LD   SP,IX
237C  E5                   PUSH HL
237D  EF                   RST  $28
```

AUTODM*: fält som används utan* DIM *får dimensionerna* OPTION BASE *(IY+$22) till 10, i varje dimension; A(1,1,1) utan* DIM *ger 11^3 element. Utanför gränserna fel 131, även A(0) efter* OPTION BASE *1.*

```
237E  DD 7E 00     AUTODM: LD   A,(IX+$00)
2381  DD B6 01             OR   (IX+$01)
2384  C0                   RET  NZ               redan dimensionerat
2385  DD 46 04             LD   B,(IX+$04)
```

```
2388  48              LD   C,B
2389  57              LD   D,A
238A  FD 5E 22        LD   E,(IY+$22)      OPTION BASE
238D  21 0A 00        LD   HL,$000A        övre gräns 10
2390  D5        L2390: PUSH DE
2391  E5               PUSH HL
2392  10 FC            DJNZ L2390
2394  6C               LD   L,H
2395  39               ADD  HL,SP
2396  CD A3 23         CALL MKARR
2399  DD 6E 04         LD   L,(IX+$04)
239C  26 00            LD   H,$00
239E  29               ADD  HL,HL
239F  29               ADD  HL,HL
23A0  39               ADD  HL,SP
23A1  F9               LD   SP,HL
23A2  C9               RET
```

MKARR: *skapa fältet IX med B dimensioner; gränserna (undre, övre) ligger vid HL. Antalet element per dimension multipliceras upp (MUL16); negativt antal eller spill ger DIMERR (fel $C9 (201) med CY). Antalet = övre + 1 - undre (undre = OPTION BASE; DIM A(2 TO 5) finns inte, fel 229). Övre = undre - 1 ger 0 element och storlek 0, alltså ett odimensionerat fält som AUTODM sedan gör till 10 (DIM A(-1) och sedan A(10)=1 går). Elementen ligger radvis: sista index ändras snabbast. In: IX = fältet, C = antal dimensioner, HL = gränserna. Ut: CY och A = $C9 vid fel. Anropas från AUTODM och DIM ($24B4).*

```
23A3  DD 5E 05  MKARR: LD   E,(IX+$05)
23A6  AF               XOR  A
23A7  57               LD   D,A
23A8  41               LD   B,C
23A9  D9               EXX
23AA  67               LD   H,A
23AB  6F               LD   L,A
23AC  39               ADD  HL,SP
23AD  D9               EXX
23AE  DD E5            PUSH IX
23B0  D5        L23B0: PUSH DE
23B1  C5               PUSH BC
23B2  D5               PUSH DE
23B3  5E               LD   E,(HL)
23B4  23               INC  HL
23B5  56               LD   D,(HL)
23B6  CD DF 30         CALL S30DF
23B9  E3               EX   (SP),HL
23BA  EB               EX   DE,HL
23BB  23               INC  HL
23BC  A7               AND  A
23BD  ED 42            SBC  HL,BC           antalet = övre + 1 - undre (egenhet)
23BF  FA 02 24         JP   M,DIMERR        negativt: fel 201
23C2  EA 02 24         JP   PE,DIMERR
23C5  CD 19 4B         CALL MUL16
23C8  38 38            JR   C,DIMERR        över 65535: fel 201
23CA  EB               EX   DE,HL
23CB  E1               POP  HL
23CC  C1               POP  BC
23CD  10 E1            DJNZ L23B0
23CF  E5               PUSH HL
23D0  CD 2C 24         CALL SALLOC          större än förut: fel 137 (egenhet)
23D3  38 2D            JR   C,DIMERR
23D5  41               LD   B,C
23D6  E1               POP  HL
23D7  C5        L23D7: PUSH BC
23D8  CD 90 30         CALL S3090
23DB  CD 46 32         CALL S3246
23DE  13               INC  DE
23DF  EB               EX   DE,HL
23E0  A7               AND  A
23E1  ED 42            SBC  HL,BC
23E3  EB               EX   DE,HL
23E4  DD 73 08         LD   (IX+$08),E
```

```
23E7  DD 72 09           LD   (IX+$09),D
23EA  DD 71 06           LD   (IX+$06),C
23ED  DD 70 07           LD   (IX+$07),B
23F0  C1                 POP  BC
23F1  D1                 POP  DE
23F2  DD 73 0A           LD   (IX+$0A),E
23F5  DD 72 0B           LD   (IX+$0B),D
23F8  11 06 00           LD   DE,$0006
23FB  DD 19              ADD  IX,DE
23FD  10 D8              DJNZ L23D7            nästa dimension
23FF  DD E1              POP  IX
2401  C9                 RET

2402  D9          DIMERR: EXX
2403  3E C9              LD   A,$C9            fel 201
2405  37                 SCF
2406  F9                 LD   SP,HL
2407  C9                 RET
```

3.21 Strängar $2408

Ett strängvärde under beräkningen är tre ord på stacken (överst först):

```
antal bytes som strängens text själv tar på stacken (0 om texten
ligger någon annanstans), textens adress, längden.
```

Konstanter (STRCON) och variabler (LDSVAR) pekar på texten där den står; nya strängar (SCAT, CHR$, LEFT$ m.fl.) byggs på stacken under beskrivningen. LEN tar bort alltihop och lämnar längden som heltal.

En strängvariabel har 6 bytes: +0,+1 max längd (0 = inte tilldelad), +2,+3 textens adress, +4,+5 aktuell längd. Vid första tilldelningen får den 80 bytes (eller mer om texten är längre) med ALLOC (SVALOC); längre text än max längd ger fel $89 (137). Utrymmet ligger sedan kvar hos variabeln och växer aldrig, så ingen skräpsamling behövs. En tom sträng som första värde tar ingen buffert (adressen förblir 0).

SVALOC: se till att strängvariabeln (IX) rymmer BC tecken. Har den ingen buffert (+0,+1 = 0) tas max(BC, 80) bytes med ALLOC och storlek och adress skrivs in; annars fel $89 om BC > storleken. A$ = 100 tecken först ger alltså en buffert på 100, och A$ + X sedan fel 137. In: IX = strängvariabeln, BC = antalet tecken. Ut: DE = buffertens adress; BC som förut. Förstör AF och HL.

```
2408  CD 56 27    SVALOC: CALL IXHL0            HL = max längd (0: ingen buffert)
240B  50                  LD   D,B              DE = antalet tecken
240C  59                  LD   E,C
240D  7C                  LD   A,H
240E  B5                  OR   L
240F  20 37               JR   NZ,L2448         en buffert finns: räcker den?
2411  EB                  EX   DE,HL            HL = antalet, DE = 0
2412  11 50 00            LD   DE,$0050         minst 80 bytes
2415  ED 52               SBC  HL,DE            antalet - 80
2417  38 02               JR   C,L241B          färre: 80 bytes
2419  50                  LD   D,B              annars antalet
241A  59                  LD   E,C
241B  CD D3 0F    L241B:  CALL ALLOC            HL = det nya området, DE = storleken
241E  DA 12 00            JP   C,ERRA           minnet räcker inte
2421  CD 6E 25    L2421:  CALL IXST2            +2,+3 = adressen
2424  EB                  EX   DE,HL            DE = adressen, HL = storleken
2425  DD 75 00    IXST0:  LD   (IX+$00),L       (IX+0) = HL
2428  DD 74 01            LD   (IX+$01),H
242B  C9                  RET
```

SALLOC: DE bytes till strängvariabeln eller fältet IX (en strängbuffert eller ett fälts element). Under länkningen av en COMMON-sats (IY+$1D bit 5, satt av SCOMMN) reserveras platsen bara i COMMON-området med ALLOCC, och variabeln får inga värden. När COMMON-satsen sedan körs pekar SCNPTR på slutet av variabelns utrymme (VDSC), och platsen tas nedåt därifrån: fältets element överst och elementens buffertar under dem, i omvänd ordning. Med `COMMON B$(2)=3` *ligger B$(0):s buffert högst och B$(2):s lägst, alla under elementen (prövat i emulatorn). Annars (SCNPTR = 0) används en befintlig buffert, och är den mindre än DE blir det fel 137; utan buffert tas en ny med ALLOC. Därför ger DIM igen av ett större fält (*`DIM A(3)`* och sedan* `DIM A(20)`*) eller en längre sträng (*`DIM A$=5`* och sedan* `DIM A$=10`*) fel 137, medan lika eller mindre godtas: datat står kvar och läses efter den nya formen (*`DIM A(3)`*, A(1)=7,* `DIM A(1)` *ger A(1) = 7), och bufferten krymper inte (*`DIM B$=10`*,*

B$="HEJSAN", DIM B$=5 och B$ är fortfarande HEJSAN). In: IX = variabeln, DE = antalet bytes. Ut: DE = adressen, +0,+1 = storleken och +2,+3 = adressen; under länkningen i stället HL = det reserverade området och CY = 1 om det inte ryms. Förstör AF och HL.

```
242C  FD CB 1D 6E  SALLOC: BIT  5,(IY+$1D)      COMMON-länkningen?
2430  C2 AE 0F           JP   NZ,ALLOCC        ja: plats i COMMON-området
2433  2A 20 FF           LD   HL,(SCNPTR)      HL = SCNPTR (slutet när COMMON körs)
2436  7C                 LD   A,H
2437  B5                 OR   L
2438  28 07              JR   Z,L2441          0: DIM, vanlig plats
243A  ED 52              SBC  HL,DE            DE bytes nedåt
243C  22 20 FF           LD   (SCNPTR),HL      nästa tas under den här
243F  18 E0              JR   L2421            adressen och storleken in

2441  CD 56 27  L2441:  CALL IXHL0            HL = storleken
2444  7D                 LD   A,L
2445  B4                 OR   H
2446  28 D3              JR   Z,L241B          ingen buffert: ALLOC
2448  ED 52        L2448: SBC  HL,DE           storleken - antalet
244A  CD 48 27           CALL IXDE2            DE = adressen
244D  D0                 RET  NC               ryms: behåll bufferten
244E  D7                 RST  $10              fel 137: längre än maxlängden
244F  89                 DEFB $89
```

SVSET: sätt strängvariabelns längd (+4,+5) till BC, efter SVALOC om BC ≠ 0; en tom sträng behöver ingen buffert. L2455 skriver bara BC i +4,+5 och används också av VDSC (antalet index och elementets storlek i ett fält). In: IX = variabeln, BC = längden. Ut: DE = buffertens adress när BC ≠ 0. Förstör AF och HL.

```
2450  78           SVSET: LD   A,B              BC = 0?
2451  B1                  OR   C
2452  C4 08 24            CALL NZ,SVALOC        tom sträng: ingen buffert
2455  DD 71 04     L2455: LD   (IX+$04),C       +4,+5 = BC
2458  DD 70 05            LD   (IX+$05),B
245B  C9                  RET
```

VDSC0/VDSC: läs variabelkoden vid DE och referensen (VARREF), IX = variabeln. Fältkoder (bit 3) följs av antalet index, som tillsammans med elementstorleken skrivs i +4,+5 (via L2455); kodpekaren läggs då under returadressen på stacken och HL = 2 (4 för strängfält, som också har en längd på stacken). För en enkel variabel tar rutinen bort sin egen returadress och går direkt till XDIM:s slut (L24C9), för en strängvariabel först via SALLOC med längden från stacken.

CY = 0 (VDSC0, DIM): SCNPTR = 0. CY = 1 (COMMON): ordet efter referensen är avståndet från variabelns värde till slutet på dess utrymme i COMMON-området (räknat vid länkningen, LCOMMN), och SCNPTR = variabeln + ordet. SALLOC tar sedan platsen nedåt därifrån. In: DE vid koden, CY som ovan. Ut (bara fält): IX = variabeln, kodpekaren under returadressen, HL = 2 eller 4, A = koden, C = antalet index. Förstör B och DE.

```
245C  A7           VDSC0: AND  A                CY = 0: inte COMMON
245D  1A           VDSC:  LD   A,(DE)           A = koden
245E  F5                  PUSH AF               koden och CY sparas
245F  13                  INC  DE
2460  CB 5F                BIT  3,A              fält?
2462  28 03               JR   Z,L2467
2464  1A                  LD   A,(DE)
2465  13                  INC  DE
2466  4F                  LD   C,A              C = antalet index
2467  C5           L2467: PUSH BC
2468  CD 63 0F            CALL VARREF           HL = variabeln, DE efter referensen
246B  C1                  POP  BC
246C  E5                  PUSH HL
246D  DD E1               POP  IX               IX = variabeln
246F  F1                  POP  AF               A = koden, CY tillbaka
2470  38 04               JR   C,L2476          CY: COMMON
2472  ED 62               SBC  HL,HL            SCNPTR = 0
2474  18 08               JR   L247E

2476  C5           L2476: PUSH BC
2477  EB                  EX   DE,HL
2478  CD E0 30            CALL LDBCHL           BC = ordet efter referensen
247B  EB                  EX   DE,HL
247C  09                  ADD  HL,BC            HL = slutet på utrymmet
247D  C1                  POP  BC
```

```
247E  22 20 FF     L247E:  LD   (SCNPTR),HL     SCNPTR = 0 eller slutet
2481  EB                 EX   DE,HL             HL = kodpekaren, DE = SCNPTR
2482  CB 5F              BIT  3,A
2484  28 18              JR   Z,L249E           enkel variabel
2486  E3                 EX   (SP),HL           kodpekaren under returadressen
2487  E5                 PUSH HL
2488  21 02 00           LD   HL,$0002          gränserna vid SP + 2
248B  FD 46 1E           LD   B,(IY+$1E)        flyttal: 4 eller 8 bytes
248E  FE C8              CP   $C8
2490  28 0A              JR   Z,L249C           $C8: flyttalsfält
2492  06 02              LD   B,$02             heltal: 2 bytes
2494  FE C9              CP   $C9               $C9: heltalsfält
2496  28 04              JR   Z,L249C
2498  06 06              LD   B,$06             sträng: 6 bytes
249A  2E 04              LD   L,$04             längden vid SP + 2, gränserna vid SP + 4
249C  18 B7        L249C:  JR   L2455          +4 = antalet index, +5 = storleken

249E  D1           L249E:  POP  DE             kasta returadressen
249F  C6 3A              ADD  A,$3A             $C6 (strängvariabel) ger 0
24A1  20 06              JR   NZ,L24A9          tal: inget mer
24A3  D1                 POP  DE                DE = längden ur DIM A$=n (0 utan)
24A4  E5                 PUSH HL                kodpekaren
24A5  CD 2C 24           CALL SALLOC
24A8  E1           L24A8:  POP  HL             HL = kodpekaren
24A9  EB           L24A9:  EX   DE,HL          DE = kodpekaren
24AA  18 1D              JR   L24C9             vidare som XDIM
```

XDIM ($CD) / XCOMMN ($CE): *DIM resp. COMMON för en variabel (knep:* $F6 = OR $37 *nollar CY för* XDIM, XCOMMN *börjar på SCF). Fält skapas med* MKARR*; för strängfält tas sedan en buffert per element (längden från* DIM A$(..)=n*; utan =n lägger* $C3 *längden 0, och elementen får ingen buffert utan uppför sig som enkla strängvariabler, med 80 bytes vid första tilldelningen:* DIM C$(1) *och sedan C$(0) med 81 tecken går). En enkel strängvariabel (*DIM A$=n*) får sin buffert med* SALLOC *redan i* VDSC*, och en enkel talvariabel behöver ingenting; för båda går* VDSC *direkt till* L24C9*. Under länkningen (IY+*$1D *bit 5) slutar rutinen efter en variabel och går tillbaka till* LCOMMN *med CY från* SALLOC *eller* MKARR*. Annars körs nästa kod med RST* $28 *om den är >=* $C0 *(nästa variabels gränser eller* $CD/$CE*), och en lägre kod avslutar satsen. In: DE efter* $CD/$CE*; gränserna och för en sträng längden på stacken. Ut: DE efter variablerna. Fel 137 och 201 (via* ERRA*).*

```
24AC  F6           XDIM:   DEFB $F6            OR $37 (överlappar nästa)
24AD  37           XCOMMN: SCF                 COMMON: CY = 1
24AE  CD 5D 24             CALL VDSC           IX = variabeln; HL = 2 eller 4 för fält
24B1  39                   ADD  HL,SP          HL = gränserna
24B2  47                   LD   B,A            B = koden, C = antalet index
24B3  C5                   PUSH BC
24B4  CD A3 23             CALL MKARR          skapa fältet
24B7  08                   EX   AF,AF'         MKARR:s CY sparas
24B8  E1                   POP  HL             H = koden, L = antalet index
24B9  7C                   LD   A,H
24BA  26 00                LD   H,$00
24BC  D1                   POP  DE             DE = kodpekaren
24BD  FE CA                CP   $CA            strängfält?
24BF  20 01                JR   NZ,L24C2
24C1  C1                   POP  BC             BC = elementens längd
24C2  29           L24C2:  ADD  HL,HL          4 bytes per dimension
24C3  29                   ADD  HL,HL
24C4  39                   ADD  HL,SP
24C5  F9                   LD   SP,HL          ta bort gränserna
24C6  28 0E                JR   Z,L24D6        strängfält: buffertarna
24C8  08                   EX   AF,AF'         CY från MKARR
24C9  FD CB 1D 6E  L24C9:  BIT  5,(IY+$1D)
24CD  C0                   RET  NZ             länkningen: tillbaka till LCOMMN
24CE  DA 12 00             JP   C,ERRA         fel (A = koden)
24D1  1A                   LD   A,(DE)         nästa kod
24D2  FE C0                CP   $C0
24D4  D8                   RET  C              lägre: satsens slut
24D5  EF                   RST  $28            nästa variabel
24D6  08           L24D6:  EX   AF,AF'         CY från MKARR
24D7  38 F0                JR   C,L24C9        fel
24D9  D5                   PUSH DE             kodpekaren
24DA  C5                   PUSH BC             elementens längd
```

```
24DB  CD 48 27           CALL IXDE2          DE = elementen
24DE  CD 41 27           CALL IXBC8          BC = antalet i första dimensionen
24E1  CD 75 27           CALL IXHLA          HL = steget i första dimensionen
24E4  D5                 PUSH DE
24E5  DD E1              POP  IX             IX = första elementet
24E7  D1                 POP  DE             DE = längden
24E8  C5                 PUSH BC             räknaren sparas
24E9  01 FA FF  L24E9:   LD   BC,$FFFA       steget / 6 varv
24EC  09                 ADD  HL,BC
24ED  C1                 POP  BC
24EE  30 B8              JR   NC,L24A8       slut: alla element har fått sin buffert
24F0  C5                 PUSH BC
24F1  78        L24F1:   LD   A,B
24F2  B1                 OR   C
24F3  28 F4              JR   Z,L24E9        inre varvet slut: nästa yttre
24F5  0B                 DEC  BC             ett element till
24F6  E5                 PUSH HL
24F7  D5                 PUSH DE
24F8  CD 2C 24           CALL SALLOC         buffert för elementet IX
24FB  08                 EX   AF,AF'         CY sparas
24FC  11 06 00           LD   DE,$0006
24FF  DD 19              ADD  IX,DE          IX = nästa element
2501  D1                 POP  DE
2502  E1                 POP  HL
2503  08                 EX   AF,AF'
2504  30 EB              JR   NC,L24F1       nästa element
2506  E1                 POP  HL
2507  18 9F              JR   L24A8
```

COLARG (`$B0`): *läggs ut av* CARGS *efter en fältgräns utan :* (`DIM A(5)`, *signatur* `$08B5`). *Lägger OPTION BASE (IY+$22; B = 0 från* EXECOD*) under den övre gränsen som undre gräns, så att* `DIM A(5)` *blir* `DIM A(b:5)`. *Med* `a:b` *läggs båda gränserna ut och* `$B0` *behövs inte. In: den övre gränsen vid SP. Ut: den undre och den övre gränsen på stacken, den övre överst. Förstör C och HL.*

```
2509  E1        COLARG: POP  HL             övre gränsen
250A  FD 4E 22          LD   C,(IY+$22)     undre = OPTION BASE
250D  C5                PUSH BC
250E  E5                PUSH HL
250F  EF                RST  $28
```

STRCON (`$CB`/`$CC`): *strängkonstant; längden (1 byte) och texten följer koden. Texten kopieras inte: beskrivningen pekar på texten i programmet, och avståndet är 0 (ingen text på stacken). In: DE vid längdbyten. Ut: DE efter texten, beskrivningen vid SP. Förstör HL.*

```
2510  EB        STRCON: EX   DE,HL          HL = längdbyten
2511  5E                LD   E,(HL)         DE = längden
2512  16 00             LD   D,$00
2514  23                INC  HL             HL = texten
2515  D5                PUSH DE             längden
2516  E5                PUSH HL             adressen
2517  19                ADD  HL,DE          HL = efter texten
2518  5A                LD   E,D            DE = 0
2519  EB                EX   DE,HL          DE = efter texten, HL = 0
251A  E5                PUSH HL             beskrivning: avstånd 0
251B  EF                RST  $28
```

LEN: *ta bort strängen från stacken och lämna längden (heltal).* L2521 *lägger BC och tar nästa kod, också för* ASC. *Förstör BC och HL.*

```
251C  E1        LEN:    POP  HL             HL = textens bytes på stacken
251D  C1                POP  BC             adressen
251E  C1                POP  BC             BC = längden
251F  39                ADD  HL,SP          ta bort strängens text
2520  F9                LD   SP,HL
2521  C5        L2521:  PUSH BC             BC som heltal
2522  EF                RST  $28
```

ASC: *första tecknets kod, 0 för en tom sträng* (`LEN("") = 0` *likaså*). *Förstör AF, BC och HL.*

```
2523  E1           ASC:   POP  HL            HL = textens bytes på stacken
2524  C1                  POP  BC            BC = adressen
2525  0A                  LD   A,(BC)        första tecknet
2526  C1                  POP  BC            BC = längden
2527  39                  ADD  HL,SP
2528  F9                  LD   SP,HL
2529  6F                  LD   L,A
252A  78                  LD   A,B           tom sträng?
252B  B1                  OR   C
252C  28 F3               JR   Z,L2521       tom: 0
252E  26 00               LD   H,$00         H = 0: koden som heltal
2530  E5                  PUSH HL
2531  EF                  RST  $28
```

MIDF: MID$(A$,S,N): *först som* RIGHT$(A$,S) (RIGHT1)*, sedan som* LEFT$(..,N)*. Alla tre argumenten krävs* (MID$(A$,2) *ger fel 223*). MID$("ABCDE",2,3) = "BCD", MID$("ABCDE",6,0) = "" *och* MID$("ABCDE",2,5) *fel 134. Talargumenten avrundas till närmaste heltal* (FTOI)*, också i* LEFT *och* RIGHT*:* LEFT$("ABCDE",2.7) = "ABC"*. In: N vid SP, S under och strängen under dem.*

```
2532  E1           MIDF:  POP  HL            HL = N
2533  C1                  POP  BC            BC = S
2534  E5                  PUSH HL            N tillbaka
2535  DD 21 04 00         LD   IX,$0004      RIGHT$(A$,S)
2539  CD 5D 25            CALL RIGHT1
```

LEFT: LEFT$(A$,N)*: bara längden ändras till N; texten flyttas inte. N = 0 ger "", N större än längden eller negativt fel* $86 *(134). In: N vid SP, strängen under. Ut: strängen vid SP. Förstör AF, BC, HL och IX.*

```
253C  C1           LEFT:  POP  BC            BC = N
253D  21 04 00            LD   HL,$0004      längden vid SP + 4
2540  39                  ADD  HL,SP
2541  7E                  LD   A,(HL)
2542  23                  INC  HL
2543  66                  LD   H,(HL)
2544  6F                  LD   L,A           HL = längden
2545  ED 42               SBC  HL,BC         N > längden?
2547  38 09               JR   C,L2552
2549  E1                  POP  HL            HL = textens bytes
254A  DD E1               POP  IX            IX = adressen
254C  F1                  POP  AF            den gamla längden bort
254D  C5                  PUSH BC            längden = N
254E  DD E5               PUSH IX
2550  E5                  PUSH HL
2551  EF                  RST  $28
2552  D7           L2552: RST  $10           fel 134: position/längd utanför strängen
2553  86                  DEFB $86
```

RIGHT: RIGHT$(A$,N)*: delsträngen från och med tecken N (som i ABC 800, inte de N sista tecknen).* RIGHT1 *(BC = N, IX = beskrivningens avstånd från SP inne i* RIGHT1*, efter returadressen) drar N - 1 från längden och lägger N - 1 till adressen; N = längden + 1 ger "", N = 0 eller N > längden + 1 fel* $86 *(134). Ut: IX = beskrivningen. Förstör AF, BC och HL.*

```
2554  C1           RIGHT: POP  BC            BC = N
2555  DD 21 02 00         LD   IX,$0002      beskrivningen efter returadressen
2559  CD 5D 25            CALL RIGHT1
255C  EF                  RST  $28
255D  0B           RIGHT1: DEC  BC           BC = N - 1
255E  DD 39               ADD  IX,SP         IX = beskrivningen
2560  CD 67 27            CALL IXHL4         HL = längden
2563  ED 42               SBC  HL,BC         längd - (N - 1)
2565  38 EB               JR   C,L2552       N = 0 eller för stort: fel 134
2567  CD 7C 27            CALL IXST4         den nya längden
256A  CD 60 27            CALL IXHL2         HL = adressen
256D  09                  ADD  HL,BC         adress + N - 1
256E  DD 75 02     IXST2: LD   (IX+$02),L    IXST2: (IX+2) = HL
2571  DD 74 03            LD   (IX+$03),H
2574  C9                  RET
```

SCAT (+ för strängar): *stacken har beskrivning 2 med text, sedan beskrivning 1 med text. Den nya strängen (längd1 + längd2) byggs under allt (*SPCHK *kontrollerar att det ryms, annars* MEMERR*) och flyttas sedan med* LDDR *upp, så att den ersätter båda de gamla; resultatets text ligger alltså på stacken. Står operandernas text också på stacken behövs plats för dubbla längden samtidigt. Med BASIC II på ABC80 (variablerna från* $C000*) går* SPACE$(12500)*, men* STRING$(3200,65)+STRING$(3200,65) *ger fel 201; A$+A$ med A$ på 3200 tecken går, eftersom en variabel bara ligger som pekare på stacken, men A$+A$+A$ ger fel 201 igen, eftersom A$+A$ då redan ligger där. In: sträng 2 vid SP, sträng 1 under. Ut: summan vid SP, med texten. Förstör AF, BC, HL och IX.*

```
2575  DD 21 06 00  SCAT:   LD   IX,$0006(egenhet)
2579  DD 39                ADD  IX,SP            IX = SP + 6: sträng 2:s text
257B  DD 6E FE             LD   L,(IX-$02)       HL = längd2
257E  DD 66 FF             LD   H,(IX-$01)
2581  C1                   POP  BC               BC = sträng 2:s textbytes
2582  C5                   PUSH BC
2583  DD 09                ADD  IX,BC            IX = beskrivning 1
2585  CD 33 27             CALL IXBC4            BC = längd1
2588  09                   ADD  HL,BC            längd1 + längd2
2589  44                   LD   B,H              BC = den nya längden
258A  4D                   LD   C,L
258B  ED 62                SBC  HL,HL
258D  39                   ADD  HL,SP            HL = SP
258E  ED 42                SBC  HL,BC            ryms under SP?
2590  DA 69 3B             JP   C,MEMERR         längre än SP: fel 201
2593  CD F6 0F             CALL SPCHK            100 bytes över FREPTR?
2596  DA 69 3B             JP   C,MEMERR
2599  F9                   LD   SP,HL            SP = den nya strängen
259A  D5                   PUSH DE               kodpekaren
259B  C5                   PUSH BC               den nya längden
259C  EB                   EX   DE,HL            DE = platsen
259D  CD 33 27             CALL IXBC4            BC = längd1
25A0  79                   LD   A,C
25A1  B0                   OR   B
25A2  28 05                JR   Z,L25A9          tom
25A4  CD 60 27             CALL IXHL2            HL = text1
25A7  ED B0                LDIR                  sträng 1
25A9  DD E1        L25A9:  POP  IX               IX = den nya längden
25AB  DD E5                PUSH IX
25AD  DD 39                ADD  IX,SP            beskrivning 2 vid IX+4
25AF  CD 41 27             CALL IXBC8            BC = längd2
25B2  79                   LD   A,C
25B3  B0                   OR   B
25B4  28 05                JR   Z,L25BB
25B6  CD 6E 27             CALL IXHL6            HL = text2
25B9  ED B0                LDIR                  sträng 2 efter
25BB  1B           L25BB:  DEC  DE               DE = den nya strängens sista byte
25BC  C1                   POP  BC               BC = den nya längden
25BD  21 02 00             LD   HL,$0002         två beskrivningar
25C0  7D                   LD   A,L
25C1  39                   ADD  HL,SP
25C2  09                   ADD  HL,BC            HL = beskrivning 2
25C3  C5                   PUSH BC
25C4  4E           L25C4:  LD   C,(HL)           hoppa över båda
25C5  23                   INC  HL
25C6  46                   LD   B,(HL)
25C7  09                   ADD  HL,BC            + textens bytes
25C8  01 05 00             LD   BC,$0005
25CB  09                   ADD  HL,BC            + 5: nästa beskrivning
25CC  3D                   DEC  A
25CD  20 F5                JR   NZ,L25C4
25CF  2B                   DEC  HL               HL = sista byten som ersätts
25D0  C1                   POP  BC
25D1  C5                   PUSH BC
25D2  EB                   EX   DE,HL            DE = den, HL = den nya strängens sista
25D3  78                   LD   A,B
25D4  B1                   OR   C
25D5  28 02                JR   Z,L25D9
25D7  ED B8                LDDR                  flytta upp resultatet
25D9  C1           L25D9:  POP  BC
25DA  13                   INC  DE               DE = resultatets text
```

```
25DB  EB                EX   DE,HL          HL = texten
25DC  D1                POP  DE             kodpekaren
25DD  F9                LD   SP,HL          SP = texten
25DE  C5                PUSH BC             längden
25DF  E5                PUSH HL             adressen
25E0  C5                PUSH BC             textens bytes = längden
25E1  EF                RST  $28
```

CHR: CHR$(a,b,...): *valfritt antal argument; antalet står i koden* $D0+*n före funktionsindexet* (*DE* - 2). *Heltalen* (*2 bytes var*) *packas först till en byte var och vänds sedan, så att det första argumentet blir det första tecknet; texten hamnar i den övre halvan av argumentens plats. Bara den låga byten används* (CHR$(321) = CHR$(-191) = "A"), *flyttal avrundas* (CHR$(65.6) = "B"), *och* CHR$() *ger fel 234. In: argumenten vid SP, det sista överst; DE efter funktionsindexet. Ut: strängen vid SP. Förstör AF, BC och HL.*

```
25E2  D5          CHR:  PUSH DE             kodpekaren
25E3  1B                DEC  DE
25E4  1B                DEC  DE
25E5  1A                LD   A,(DE)         $D0+n före indexet
25E6  D6 D0             SUB  $D0            antal argument
25E8  4F                LD   C,A
25E9  06 00             LD   B,$00          BC = n
25EB  21 02 00          LD   HL,$0002
25EE  39                ADD  HL,SP          HL = argumenten (efter DE)
25EF  54                LD   D,H            DE = samma plats
25F0  5D                LD   E,L
25F1  ED A0       L25F1:LDI                 låga byten
25F3  23                INC  HL             hoppa över den höga byten
25F4  EA F1 25          JP   PE,L25F1
25F7  4F                LD   C,A            C = n igen
25F8  D5                PUSH DE             slutet på de packade
25F9  62                LD   H,D            HL = samma
25FA  6B                LD   L,E
25FB  2B          L25FB:DEC  HL             vänd ordningen
25FC  ED A0             LDI                 bakifrån, framåt i DE
25FE  2B                DEC  HL
25FF  EA FB 25          JP   PE,L25FB
2602  E1                POP  HL             HL = texten
2603  D1                POP  DE             kodpekaren
2604  F9                LD   SP,HL          SP = texten (övre halvan)
2605  4F                LD   C,A
2606  C5                PUSH BC             längden
2607  E5                PUSH HL             adressen
2608  C5                PUSH BC             textens bytes = n
2609  EF                RST  $28
```

SCMP: *jämför sträng 1* (*djupare*) *med sträng 2* (*överst*) *tecken för tecken; vid lika början avgör längden. Båda tas bort från stacken och flaggorna* (*Z lika, CY sträng 1 < sträng 2*) *går via* EX AF,AF' *till jämförelsens resultatrutin* (L2132, *se* FNTAB). *Bytevärdena jämförs utan tecken, och blanka på slutet räknas:* "a">"Z", CHR$(200)>"A", ""<"A" *och* "AB"<"ABC" *är sanna,* "AB"="AB " *är falskt. In: sträng 2 vid SP, sträng 1 under. Ut: AF' = flaggorna, båda strängarna borta.*

```
260A  D5          SCMP: PUSH DE             kodpekaren
260B  21 07 00          LD   HL,$0007       beskrivning 2 efter DE
260E  39                ADD  HL,SP
260F  46                LD   B,(HL)
2610  2B                DEC  HL
2611  4E                LD   C,(HL)         BC = längd2
2612  2B                DEC  HL
2613  56                LD   D,(HL)
2614  2B                DEC  HL
2615  5E                LD   E,(HL)         DE = adress2
2616  2B                DEC  HL
2617  7E                LD   A,(HL)
2618  2B                DEC  HL
2619  6E                LD   L,(HL)
261A  67                LD   H,A            HL = sträng 2:s textbytes
261B  39                ADD  HL,SP
261C  E5                PUSH HL
261D  DD E1             POP  IX             IX: beskrivning 1 vid IX+8
```

```
261F  DD 6E 0C          LD   L,(IX+$0C)     HL = längd1
2622  DD 66 0D          LD   H,(IX+$0D)
2625  ED 42             SBC  HL,BC          längd1 - längd2
2627  CD 75 27          CALL IXHLA          HL = adress1
262A  EB                EX   DE,HL          DE = adress1, HL = adress2
262B  38 29             JR   C,L2656        1 kortare
262D  20 1A             JR   NZ,L2649       1 längre
262F  78          L262F: LD   A,B           lika långa: BC tecken
2630  B1               OR   C
2631  28 07            JR   Z,L263A        alla lika: Z, NC
2633  1A               LD   A,(DE)         tecken ur 1
2634  13               INC  DE
2635  BE               CP   (HL)           jämför med 2: CY om 1 är mindre
2636  23               INC  HL
2637  0B               DEC  BC
2638  28 F5            JR   Z,L262F        lika: nästa
263A  08          L263A: EX   AF,AF'        flaggorna till AF'
263B  D1               POP  DE             kodpekaren
263C  E1               POP  HL             sträng 2 bort
263D  C1               POP  BC
263E  C1               POP  BC
263F  39               ADD  HL,SP
2640  F9               LD   SP,HL
2641  E1               POP  HL             sträng 1 bort
2642  C1               POP  BC
2643  C1               POP  BC
2644  39               ADD  HL,SP
2645  F9               LD   SP,HL
2646  C3 32 21         JP   L2132          resultatet: -1 eller 0

2649  78          L2649: LD   A,B           1 längre: längd2 tecken
264A  B1               OR   C
264B  28 1D            JR   Z,L266A        lika början: 1 > 2
264D  1A               LD   A,(DE)
264E  13               INC  DE
264F  BE               CP   (HL)
2650  23               INC  HL
2651  0B               DEC  BC
2652  28 F5            JR   Z,L2649
2654  18 E4            JR   L263A

2656  DD 4E 0C    L2656: LD   C,(IX+$0C)    jämför längd1 tecken
2659  DD 46 0D           LD   B,(IX+$0D)
265C  78          L265C: LD   A,B
265D  B1               OR   C
265E  28 09            JR   Z,L2669        lika början
2660  1A               LD   A,(DE)
2661  13               INC  DE
2662  BE               CP   (HL)
2663  23               INC  HL
2664  0B               DEC  BC
2665  28 F5            JR   Z,L265C
2667  18 D1            JR   L263A

2669  37          L2669: SCF                lika början: 1 < 2
266A  3C          L266A: INC  A             A = 0 -> 1: NZ, CY som förut
266B  18 CD            JR   L263A
```

INSTR: `INSTR(S,A$,B$)`: *position (1..) för B$ i A$ med början vid tecken S, 0 om den inte finns. Varje tecken i B$ söks med* `CPIR` *från förra träffen; ligger träffarna inte i följd börjar sökningen om från den möjliga början (sista träffen + 1 minus B$:s längd) tills en träff börjar där sökningen började. Den möjliga början ligger aldrig efter en verklig träff, så ingen hoppas över (*`INSTR(1,"AXAB","AB")` *= 3,* `INSTR(1,"AXBAB","AB")` *= 4). A$ och B$ tas bort och S ersätts med resultatet. S avrundas till närmaste (*`INSTR(2.6,"ABCABC","A")` *= 4). Ett S utanför strängen ger 0 utan fel (S = 0, negativt eller bortom slutet). En tom B$ ger S för S upp till längden + 1 (*`INSTR(3,"AB","")` *= 3,* `INSTR(1,"","")` *= 1), annars 0. In: B$ vid SP, A$ under och S under den. Ut: positionen som heltal i S:s plats. Förstör AF, BC, HL, IX och BC', DE', HL'; DE (kodpekaren) står kvar, eftersom rutinen räknar i den andra registeruppsättningen.*

```
266D  DD E1       INSTR: POP  IX            IX = B$:s textbytes
266F  E1                 POP  HL            HL = B$:s adress
```

```
2670  C1              POP  BC          BC = B$:s längd
2671  DD 39           ADD  IX,SP       IX = A$:s beskrivning
2673  D9              EXX              B$ i HL' och BC'
2674  CD 2C 27        CALL IXBC0       BC = A$:s textbytes
2677  CD 60 27        CALL IXHL2       HL = A$:s adress
267A  CD 4F 27        CALL IXDE4       DE = A$:s längd
267D  DD E5           PUSH IX
267F  DD 09           ADD  IX,BC
2681  CD 3A 27        CALL IXBC6       BC = S (efter A$)
2684  DD E1           POP  IX
2686  0B              DEC  BC          S - 1
2687  09              ADD  HL,BC       HL = tecken S
2688  EB              EX   DE,HL       HL = längden, DE = tecken S
2689  ED 42           SBC  HL,BC       tecken kvar från S
268B  38 41           JR   C,L26CE     S utanför: 0
268D  EB              EX   DE,HL       HL = tecken S
268E  42              LD   B,D         BC = tecknen kvar
268F  4B              LD   C,E
2690  E5        L2690: PUSH HL          sökningens början
2691  D9              EXX
2692  E5              PUSH HL          B$:s adress och längd
2693  C5              PUSH BC
2694  78              LD   A,B         tom B$: träff vid S
2695  B1              OR   C
2696  28 12           JR   Z,L26AA
2698  7E        L2698: LD   A,(HL)     nästa tecken i B$
2699  D9              EXX
269A  5F              LD   E,A         E = tecknet
269B  78              LD   A,B
269C  B1              OR   C
269D  28 2F           JR   Z,L26CE     A$ slut: ingen träff
269F  7B              LD   A,E
26A0  ED B1           CPIR             B$:s tecken i A$
26A2  20 2A           JR   NZ,L26CE    finns inte: 0
26A4  D9              EXX
26A5  ED A1           CPI              bara steget i B$ (alltid lika)
26A7  EA 98 26        JP   PE,L2698    fler tecken i B$
26AA  C1        L26AA: POP  BC         BC = B$:s längd
26AB  E1              POP  HL          HL = B$:s adress
26AC  C5              PUSH BC
26AD  D9              EXX
26AE  D1              POP  DE          DE = B$:s längd
26AF  ED 52           SBC  HL,DE       möjlig början (sista träffen + 1 - längden)
26B1  EB              EX   DE,HL       DE = den
26B2  09              ADD  HL,BC       HL = tecknen kvar från den
26B3  E3              EX   (SP),HL     HL = sökningens början
26B4  C1              POP  BC          BC = tecknen kvar
26B5  ED 52           SBC  HL,DE       började träffen där sökningen började?
26B7  EB              EX   DE,HL       HL = den möjliga början
26B8  20 D6           JR   NZ,L2690    nej: sök om därifrån
26BA  E5              PUSH HL          träffens adress till HL
26BB  D9              EXX
26BC  E1              POP  HL
26BD  DD F9       L26BD: LD   SP,IX    SP = A$:s beskrivning
26BF  DD E1           POP  IX          IX = A$:s textbytes
26C1  C1              POP  BC          BC = A$:s adress
26C2  38 02           JR   C,L26C6     ingen träff: HL = -1
26C4  ED 42           SBC  HL,BC       HL = avståndet från början
26C6  23          L26C6: INC  HL       position 1..
26C7  C1              POP  BC          A$:s längd
26C8  DD 39           ADD  IX,SP       IX = S
26CA  DD F9           LD   SP,IX
26CC  E3              EX   (SP),HL     S ersätts med positionen
26CD  EF              RST  $28
26CE  D9          L26CE: EXX           ingen träff: HL = -1 -> 0
26CF  37              SCF
26D0  ED 62           SBC  HL,HL
26D2  18 E9           JR   L26BD
```

SPACE: SPACE$(N) = STRING$(N," "); STRNG: STRING$(N,C), *N tecken C, byggs på stacken. Negativt N ger fel* $87 *(135) via* ERRJP, *och fortsätter man efter felet med CON blir strängen tom* (A$=STRING$(-1,65), *CON, LEN(A$) = 0). N från 65536 ger fel 132* (FTOI), *och strängen måste rymmas på stacken med 100 bytes över* FREPTR (*med BASIC II på ABC80 går* SPACE$(12500) *men inte* SPACE$(12750)). *C är ett tal, och bara den låga byten används* (STRING$(2,321) = "AA"); STRING$(3,"X") *ger fel 224. In: N vid SP* (SPACE), *C vid SP och N under* (STRNG). *Ut: strängen vid SP. Förstör AF, BC och HL.*

```
26D4  C1          SPACE:  POP  BC          BC = N
26D5  2E 20               LD   L,$20       blanktecken
26D7  18 02               JR   L26DB

26D9  E1          STRNG:  POP  HL          HL = C
26DA  C1                  POP  BC          BC = N
26DB  CB 78       L26DB:  BIT  7,B         N < 0?
26DD  3E 87               LD   A,$87       fel 135: negativt antal
26DF  28 06               JR   Z,L26E7
26E1  CD 33 00            CALL ERRJP       fel 135 (går att fortsätta)
26E4  01 00 00            LD   BC,$0000    efter fel: tom
26E7  7D          L26E7:  LD   A,L         A = tecknet
26E8  21 00 00            LD   HL,$0000
26EB  39                  ADD  HL,SP       HL = SP
26EC  ED 42               SBC  HL,BC       ryms på stacken?
26EE  DA 69 3B            JP   C,MEMERR    fel 201
26F1  F5                  PUSH AF
26F2  CD F6 0F            CALL SPCHK       100 bytes över FREPTR?
26F5  DA 69 3B            JP   C,MEMERR
26F8  F1                  POP  AF
26F9  F9                  LD   SP,HL       SP = texten
26FA  C5                  PUSH BC          längden
26FB  E5                  PUSH HL          adressen
26FC  C5                  PUSH BC          textens bytes = N
26FD  F5                  PUSH AF          tecknet
26FE  78                  LD   A,B
26FF  B1                  OR   C
2700  CA 13 27            JP   Z,L2713     N = 0: bara beskrivningen
2703  F1                  POP  AF          A = tecknet
2704  77                  LD   (HL),A      första tecknet
2705  0B                  DEC  BC
2706  78                  LD   A,B
2707  B1                  OR   C
2708  CA A9 1D            JP   Z,EXECOD    ett tecken
270B  D5                  PUSH DE          kodpekaren
270C  54                  LD   D,H
270D  5D                  LD   E,L
270E  13                  INC  DE          DE = HL + 1
270F  ED B0               LDIR             fyll resten
2711  D1                  POP  DE
2712  EF                  RST  $28
2713  C1          L2713:  POP  BC          kasta tecknet
2714  EF                  RST  $28
```

CUR: CUR(R,K): *strängen ESC = R+32 K+32 (4 bytes på stacken), alltså markörstyrningen som PRINT skickar till skärmen. R och K prövas inte; bara den låga byten av R+32 och K+32 används* (CUR(257,2) = CUR(1,2)). *Utanför skärmen hamnar markören vid kanten, eftersom skärmrutinen begränsar ESC =:* CUR(30,50) *ger sista raden och kolumnen,* CUR(-1,-1) *övre vänstra hörnet. In: K vid SP, R under. Ut: strängen vid SP. Förstör BC och HL.*

```
2715  C1          CUR:    POP  BC          BC = K
2716  E1                  POP  HL          L = R
2717  61                  LD   H,C         H = kolumn
2718  01 20 20            LD   BC,$2020
271B  09                  ADD  HL,BC       +32 på båda
271C  E5                  PUSH HL          R+32 och K+32 i den ordningen
271D  21 1B 3D            LD   HL,$3D1B    ESC =
2720  E5                  PUSH HL          ESC och = före
2721  21 00 00            LD   HL,$0000
2724  39                  ADD  HL,SP       HL = SP: texten
2725  01 04 00            LD   BC,$0004    4 tecken
2728  C5                  PUSH BC          längden
2729  E5                  PUSH HL          adressen
272A  C5                  PUSH BC          textens bytes = 4
```

```
272B  EF                  RST  $28
```

IX-hjälpare: läs ett ord i posten vid IX till BC, DE eller HL, t.ex. `IXBC4` *= BC :=* (IX+4). `IXBCHL` *ger BC =* (IX+4) *och HL =* (IX+2)*, alltså längden och adressen i en strängvariabel.* `IXST0`/`IXST2`/`IXST4` *skriver HL. Ingen ändrar något annat register eller flaggorna.*

```
272C  DD 4E 00     IXBC0:  LD   C,(IX+$00)
272F  DD 46 01             LD   B,(IX+$01)
2732  C9                   RET

2733  DD 4E 04     IXBC4:  LD   C,(IX+$04)
2736  DD 46 05             LD   B,(IX+$05)
2739  C9                   RET

273A  DD 4E 06     IXBC6:  LD   C,(IX+$06)
273D  DD 46 07             LD   B,(IX+$07)
2740  C9                   RET

2741  DD 4E 08     IXBC8:  LD   C,(IX+$08)
2744  DD 46 09             LD   B,(IX+$09)
2747  C9                   RET

2748  DD 5E 02     IXDE2:  LD   E,(IX+$02)
274B  DD 56 03             LD   D,(IX+$03)
274E  C9                   RET

274F  DD 5E 04     IXDE4:  LD   E,(IX+$04)
2752  DD 56 05             LD   D,(IX+$05)
2755  C9                   RET

2756  DD 6E 00     IXHL0:  LD   L,(IX+$00)
2759  DD 66 01             LD   H,(IX+$01)
275C  C9                   RET

275D  CD 33 27     IXBCHL: CALL IXBC4
2760  DD 6E 02     IXHL2:  LD   L,(IX+$02)
2763  DD 66 03             LD   H,(IX+$03)
2766  C9                   RET

2767  DD 6E 04     IXHL4:  LD   L,(IX+$04)        HL = (IX+4)
276A  DD 66 05             LD   H,(IX+$05)
276D  C9                   RET

276E  DD 6E 06     IXHL6:  LD   L,(IX+$06)
2771  DD 66 07             LD   H,(IX+$07)
2774  C9                   RET

2775  DD 6E 0A     IXHLA:  LD   L,(IX+$0A)
2778  DD 66 0B             LD   H,(IX+$0B)
277B  C9                   RET

277C  DD 75 04     IXST4:  LD   (IX+$04),L        (IX+4) = HL
277F  DD 74 05             LD   (IX+$05),H
2782  C9                   RET
```

Korsreferenstabell för strängarna. Vad rutinerna tar emot (I), lämnar (U) och använder tillfälligt eller förstör (X), i registren, på stacken och i systemvariablerna; IY-kolumnerna är flaggorna och inställningarna vid IY+$1D (bit 5, COMMON-länkningen), IY+$1E (flyttalens längd) och IY+$22 (OPTION BASE). Skugg. är de skuggade registren (AF' och BC', DE', HL'). Det som en anropad rutin gör räknas med: `SVALOC` flyttar `FREPTR` genom `ALLOC`, `SCAT` och `STRNG` läser den genom `SPCHK`. `MIDF` är `RIGHT1` följd av `LEFT`, `RIGHT` samma som `RIGHT1`, `XCOMMN` samma som `XDIM` och `SPACE` samma som `STRNG`. Ett tomt fält: orört. Tabellen är gjord ur koden och kommentarerna ovan.

	register och stack								variabler		IY+		
Rutin	A	CY	BC	DE	HL	IX	skugg.	stack	SCNPTR	FREPTR	$1D	$1E	$22
SVALOC	X	X	I	U	X	I				IU			
SALLOC	X	U		IU	X	I			IU	IU	I		
SVSET	X	X	I	U	X	I				IU			
VDSC	U	I	U	IX	U	U		U	U			I	
XDIM	X	U	X	IU	X	X	X	I	IU	IU	I	I	
COLARG			IX		X			IU					I
STRCON				IU	X			U					

	register och stack								variabler		IY+		
Rutin	A	CY	BC	DE	HL	IX	skugg.	stack	SCNPTR	FREPTR	$1D	$1E	$22
LEN			X		X			IU					
ASC	X	X	X		X			IU					
MIDF	X	X	X		X	X		IU					
LEFT	X	X	X		X	X		IU					
RIGHT1	X	X	IX		X	IU		IU					
SCAT	X	X	X		X	X		IU		I			
CHR	X	X	X	I	X			IU					
SCMP	X	X	X		X	X	U	IU					
INSTR	X	X	X		X	X	X	IU					
STRNG	X	X	X		X			IU		I			
CUR			X		X			IU					

Rutin	**Anropar**	**Anropas från**
SVALOC	IXHL0, ALLOC, IXST2, IXDE2	SVSET
SALLOC	ALLOCC (länkningen), ALLOC, IXST2, IXHL0, IXDE2	VDSC, XDIM, MKARR
SVSET	SVALOC	STSVAR, GET, RDVALS
VDSC	VARREF, LDBCHL, SALLOC; går vidare till L2455 eller L24C9	XDIM (VDSC), ARRELM (VDSC0)
XDIM	VDSC, MKARR, SALLOC, IXDE2, IXBC8, IXHLA, ERRA	JPTABL ($CD, $CE); LCOMMN vid länkningen
COLARG		JPTABL ($B0, läggs ut av CARGS)
STRCON		JPTABL ($CB, $CC)
LEN, ASC, CHR, INSTR, CUR		FNTAB
MIDF	RIGHT1, sedan LEFT	FNTAB
LEFT		FNTAB, MIDF
RIGHT1	IXHL4, IXST4, IXHL2	RIGHT, MIDF
SCAT	IXBC4, IXHL2, IXBC8, IXHL6, SPCHK, MEMERR	JPTABL (+ för strängar)
SCMP	IXHLA; går vidare till L2132	JPTABL (jämförelserna)
SPACE, STRNG	ERRJP, SPCHK, MEMERR	FNTAB

FOR *v* = *a* TO *b* [STEP *c*]: *v* = *a utförs först* (RST $20), *och referensen till v står före* $BD. *Gränsen och steget hamnar på stacken* (*ett heltal pushas, ett flyttal ligger redan där*); *utan* STEP *läggs 1 dit. Efter* $BC *följer en relativ länk till* NEXT*-satsens ord, och det ordet är* FOR*-blockets referens* (*relativ* VARBEG). *Blocket: +0 adressen efter* NEXT (*utgången*), *+2 variabelns referens, +4 slingans början* (*efter* FOR), *+6 typ* (*0 = flyttal*), *+7 steget, sedan gränsen. Gränsen och steget räknas en gång. Slingan prövas genast, så att* FOR *J=5* TO *1 inte kör kroppen* (*J förblir 5*). *Ett flyttalsvärde till en heltalsvariabel avrundas till närmaste* (FOR *A%=1.6* TO *3.5 ger 2 3 4*). *In: DE efter FOR-koden, vid tilldelningen. Ut: DE = slingans början, eller adressen efter NEXT om slingan inte ska köras alls. Förstör de övriga registren.*

```
2783  E7           FOR:    RST  $20          tilldelningen; HL = variabeln
2784  E5                   PUSH HL
2785  EB                   EX   DE,HL
2786  2B                   DEC  HL           referensen före DE
2787  56                   LD   D,(HL)
2788  2B                   DEC  HL
2789  5E                   LD   E,(HL)
278A  23                   INC  HL
278B  23                   INC  HL
278C  EB                   EX   DE,HL
278D  E5                   PUSH HL
278E  13                   INC  DE           förbi $BD
278F  E7                   RST  $20          gränsen
2790  78                   LD   A,B
2791  FE 00                CP   $00
2793  28 01                JR   Z,L2796
2795  E5                   PUSH HL           heltal på stacken
2796  1A           L2796:  LD   A,(DE)
2797  FE BE                CP   $BE          STEP?
2799  20 58                JR   NZ,L27F3
279B  13                   INC  DE
279C  E7                   RST  $20          steget
279D  78             L279D:  LD   A,B
279E  FE 00                CP   $00
```

```
27A0  28 01              JR   Z,L27A3
27A2  E5                 PUSH HL
27A3  13           L27A3: INC  DE              förbi $BC
27A4  EB                 EX   DE,HL
27A5  5E                 LD   E,(HL)
27A6  23                 INC  HL
27A7  56                 LD   D,(HL)
27A8  EB                 EX   DE,HL
27A9  19                 ADD  HL,DE            HL = NEXT:s blockreferens
27AA  13                 INC  DE               DE = slingans början
27AB  4E                 LD   C,(HL)
27AC  23                 INC  HL
27AD  46                 LD   B,(HL)
27AE  23                 INC  HL
27AF  C5                 PUSH BC
27B0  DD E1              POP  IX
27B2  ED 4B 2E FF        LD   BC,(VARBEG)
27B6  DD 09              ADD  IX,BC            IX = FOR-blocket
27B8  DD 73 04           LD   (IX+$04),E       +4 slingans början
27BB  DD 72 05           LD   (IX+$05),D
27BE  DD 75 00           LD   (IX+$00),L       +0 efter NEXT
27C1  DD 74 01           LD   (IX+$01),H
27C4  DD E5              PUSH IX
27C6  E1                 POP  HL
27C7  11 06 00           LD   DE,$0006
27CA  19                 ADD  HL,DE
27CB  EB                 EX   DE,HL
27CC  12                 LD   (DE),A           +6 typ (0 = flyttal)
27CD  13                 INC  DE
27CE  FE 00              CP   $00
27D0  01 04 00           LD   BC,$0004
27D3  20 05              JR   NZ,L27DA
27D5  FD 4E 1E           LD   C,(IY+$1E)       flyttal: 2 x storleken
27D8  CB 21              SLA  C
27DA  21 00 00     L27DA: LD   HL,$0000
27DD  39                 ADD  HL,SP
27DE  ED B0              LDIR                  steg och gräns från stacken
27E0  F9                 LD   SP,HL
27E1  E1                 POP  HL
27E2  DD 75 02           LD   (IX+$02),L       +2 variabelns referens
27E5  DD 74 03           LD   (IX+$03),H
27E8  E1                 POP  HL
27E9  FE 00              CP   $00
27EB  28 78              JR   Z,L2865          flyttal: jämför
27ED  5E                 LD   E,(HL)
27EE  23                 INC  HL
27EF  56                 LD   D,(HL)
27F0  C3 3E 28           JP   L283E            heltal: jämför direkt

27F3  21 01 00     L27F3: LD   HL,$0001        inget STEP: steget 1
27F6  78                 LD   A,B
27F7  A7                 AND  A
27F8  20 A3              JR   NZ,L279D
27FA  2B                 DEC  HL               flyttalet 1: nollord ...
27FB  E5                 PUSH HL
27FC  FD CB 1E 5E        BIT  3,(IY+$1E)
2800  28 02              JR   Z,L2804
2802  E5                 PUSH HL
2803  E5                 PUSH HL
2804  2E 81        L2804: LD   L,$81           ... och exponenten $81
2806  E5                 PUSH HL
2807  18 94              JR   L279D
```

NEXT: IX = FOR-blocket. Har FOR inte körts (+1 = 0, t.ex. efter GOTO in i slingan) blir det fel 183 vid körningen. Heltal: v + steget utan spillkontroll, sedan jämförs med gränsen (omvänt vid negativt steg); förbi gränsen -> utgången, annars slingans början. Flyttal: NXTADD lägger v och steget på stacken, FADD och tillbaka i v, sedan FCOMP2. Blocken är fasta: ett FOR i en rekursiv GOSUB börjar om samma slinga, och STEP 0 tar aldrig slut. Fel 183 går via ERRSP: ON ERROR fångar det inte. In: DE vid blockets referens. Ut: DE = slingans början, eller efter NEXT när slingan är slut; VFF36 = variabelns adress. Förstör de övriga registren.

```
2809  EB          NEXT:  EX   DE,HL
280A  5E                 LD   E,(HL)
280B  23                 INC  HL
280C  56                 LD   D,(HL)
280D  D5                 PUSH DE
280E  DD E1              POP  IX
2810  ED 5B 2E FF        LD   DE,(VARBEG)
2814  DD 19              ADD  IX,DE
2816  DD 7E 01           LD   A,(IX+$01)       FOR inte utfört?
2819  A7                 AND  A
281A  3E B7              LD   A,$B7            fel 183
281C  CA 6B 3B           JP   Z,ERRSP
281F  DD 6E 02           LD   L,(IX+$02)
2822  DD 66 03           LD   H,(IX+$03)
2825  19                 ADD  HL,DE
2826  22 36 FF           LD   (VFF36),HL       variabelns adress
2829  DD CB 06 46        BIT  0,(IX+$06)
282D  28 2F              JR   Z,L285E
282F  DD 4E 07           LD   C,(IX+$07)       steget
2832  DD 46 08           LD   B,(IX+$08)
2835  5E                 LD   E,(HL)
2836  23                 INC  HL
2837  56                 LD   D,(HL)
2838  EB                 EX   DE,HL
2839  09                 ADD  HL,BC            variabeln + steget, utan spillkontroll (egenhet)
283A  EB                 EX   DE,HL
283B  72                 LD   (HL),D
283C  2B                 DEC  HL
283D  73                 LD   (HL),E
283E  DD 6E 09     L283E:LD   L,(IX+$09)       gränsen
2841  DD 66 0A           LD   H,(IX+$0A)
2844  DD 7E 08           LD   A,(IX+$08)
2847  A7                 AND  A
2848  F2 4C 28           JP   P,L284C          negativt steg: vänd
284B  EB                 EX   DE,HL
284C  ED 52        L284C:SBC  HL,DE
284E  E2 54 28           JP   PO,L2854         spill: rätta tecknet
2851  3E 80              LD   A,$80
2853  AC                 XOR  H
2854  FA 7E 28     L2854:JP   M,L287E          förbi gränsen: ut
2857  DD 5E 04     L2857:LD   E,(IX+$04)       nästa varv: slingans början
285A  DD 56 05           LD   D,(IX+$05)
285D  C9                 RET

285E  DD E5        L285E:PUSH IX
2860  CD 85 28           CALL NXTADD           variabeln + steget
2863  DD E1              POP  IX
2865  EB           L2865:EX   DE,HL
2866  DD E5              PUSH IX
2868  C1                 POP  BC
2869  FD 7E 1E           LD   A,(IY+$1E)
286C  C6 07              ADD  A,$07
286E  6F                 LD   L,A
286F  26 00              LD   H,$00
2871  09                 ADD  HL,BC
2872  DD CB 08 7E        BIT  7,(IX+$08)       negativt steg: vänd
2876  20 01              JR   NZ,L2879
2878  EB                 EX   DE,HL
2879  CD 64 4C     L2879:CALL FCOMP2
287C  30 D9              JR   NC,L2857
287E  DD 5E 00     L287E:LD   E,(IX+$00)       efter NEXT
2881  DD 56 01           LD   D,(IX+$01)
2884  C9                 RET
```

NXTADD: *flyttalsvariabeln + steget. Variabelns adress läggs på stacken och plats för två flyttal görs under den; variabeln och steget (blockets +7) kopieras dit,* FADD *lägger ihop dem och* SFARR *skriver summan till adressen. In: HL = variabelns adress, IX = FOR-blocket. Ut: HL = variabelns adress (även i* VFF36*), B = 0, stacken som före. Förstör AF, C och DE, och det som* FADD *förstör.*

```
2885  E5          NXTADD:PUSH HL
```

```
2886  EB                EX   DE,HL
2887  21 00 00          LD   HL,$0000
288A  44                LD   B,H
288B  39                ADD  HL,SP
288C  FD 4E 1E          LD   C,(IY+$1E)
288F  ED 42             SBC  HL,BC
2891  ED 42             SBC  HL,BC
2893  F9                LD   SP,HL
2894  EB                EX   DE,HL
2895  79                LD   A,C
2896  ED B0             LDIR                 variabeln på stacken
2898  0E 07             LD   C,$07
289A  DD E5             PUSH IX
289C  E1                POP  HL
289D  09                ADD  HL,BC
289E  4F                LD   C,A
289F  ED B0             LDIR                 steget
28A1  CD 41 4F          CALL FADD
28A4  C3 D5 22          JP   SFARR           summan till variabeln
```

WHILE *villkor: länken pekar efter* WEND. *Villkoret är ett heltal (avrundat till närmaste, .4 falskt, .5 sant); falskt -> efter* WEND. WEND *hoppar tillbaka till* WHILE *med en relativ länk (via* $1D9D*). Ett strängvillkor ger fel 224 redan vid kompileringen. Villkoret blir heltal som i IF: 40000 och -32769 är sanna, och från 65536 blir det fel 132. In: DE vid länken. Ut: DE efter villkoret (sant) eller efter WEND (falskt). Förstör de övriga registren.*

```
28A7  EB          WHILE:  EX   DE,HL
28A8  5E                  LD   E,(HL)
28A9  23                  INC  HL
28AA  56                  LD   D,(HL)
28AB  EB                  EX   DE,HL
28AC  13                  INC  DE
28AD  23                  INC  HL
28AE  19                  ADD  HL,DE           HL = efter WEND
28AF  E5                  PUSH HL
28B0  E7                  RST  $20             villkoret (heltal)
28B1  7C                  LD   A,H
28B2  B5                  OR   L
28B3  E1                  POP  HL
28B4  C0                  RET  NZ              sant: fortsätt
28B5  EB                  EX   DE,HL           falskt: efter WEND
28B6  C9                  RET
```

RESTORE*: utan nummer läsposition =* $28F3*, en byte* $87 *i ROM:en, så att nästa* READ *går vidare till första* DATA*-satsen i kedjan (*PRGBEG+$12*). Med nummer måste raden börja med* DATA ($8E)*, annars fel 149 (ABC80-BASIC:en gick vidare till nästa* DATA*-rad).* DATSTM *= länkordet i aktuell* DATA*-sats,* DATPTR *= läspositionen i dess text. In: DE efter koden, för* RESTOR *vid avståndet till raden (ON n RESTORE kommer till* $28CD *med HL = raden). Ut: DE efter satsen,* DATSTM *och* DATPTR *satta. Förstör AF, BC och HL.*

```
28B7  CD 3B 00    RESTR0: CALL S003B
28BA  01 12 00            LD   BC,$0012        DATA-kedjans huvud
28BD  09                  ADD  HL,BC
28BE  CD DF 28            CALL S28DF
28C1  21 F3 28            LD   HL,$28F3        $87 i ROM:en: nästa READ tar första DATA
28C4  18 1F               JR   S28E5

28C6  EB          RESTOR: EX   DE,HL           RESTORE till rad, relativ adress
28C7  5E                  LD   E,(HL)
28C8  23                  INC  HL
28C9  56                  LD   D,(HL)
28CA  EB                  EX   DE,HL
28CB  19                  ADD  HL,DE           HL = raden
28CC  13                  INC  DE
28CD  7E          L28CD:  LD   A,(HL)
28CE  FE 87               CP   $87             radhuvudet?
28D0  20 05               JR   NZ,L28D7
28D2  23                  INC  HL
28D3  23                  INC  HL
28D4  23                  INC  HL
```

```
28D5  23              INC  HL
28D6  7E              LD   A,(HL)
28D7  EE 8E     L28D7:XOR  $8E(egenhet)
28D9  3E 95           LD   A,$95          fel 149: raden är inte DATA ($8E)
28DB  C2 12 00        JP   NZ,ERRA
28DE  23              INC  HL
28DF  22 3E FF  S28DF:LD   (DATSTM),HL    länkordet i DATA-satsen
28E2  23              INC  HL
28E3  23              INC  HL
28E4  23              INC  HL
28E5  22 3C FF  S28E5:LD   (DATPTR),HL    läsposition
28E8  C9              RET
```

L28E9: felutgång för READ. Kodpekaren tas tillbaka till DE och felet A går via ERRJP, så ON ERROR kan fånga det.

```
28E9  D1        L28E9:POP  DE
28EA  CD 33 00        CALL ERRJP          fel A
28ED  EB              EX   DE,HL
```

READ: är läspositionen slut (bit 7) tas nästa DATA-sats via länken; länk 0 = fel 141. RDVALS läser ett värde per variabel; följer $B7 igen läses nästa grupp. Varje variabel blir en egen grupp (åtskilda av $B7), så READ tar ett värde i taget och kan fortsätta i nästa DATA-sats mitt i satsen: READ E,F tog 8 ur en sats och 9 ur nästa. Felen 148 och 150 kommer därför aldrig från READ. In: DE vid variabellistan. Ut: DE efter satsen, DATPTR flyttad. Förstör de övriga registren.

```
28EE  D5        READ: PUSH DE
28EF  2A 3C FF        LD   HL,(DATPTR)
28F2  7E              LD   A,(HL)
28F3  87              ADD  A,A            slut på satsen (bit 7)?
28F4  30 10           JR   NC,L2906
28F6  2A 3E FF        LD   HL,(DATSTM)
28F9  4E              LD   C,(HL)
28FA  23              INC  HL
28FB  46              LD   B,(HL)
28FC  78              LD   A,B
28FD  B1              OR   C
28FE  3E 8D           LD   A,$8D          fel 141: inga fler DATA
2900  28 E7           JR   Z,L28E9
2902  09              ADD  HL,BC
2903  CD DF 28        CALL S28DF          nästa DATA-sats
2906  CD 12 29  L2906:CALL RDVALS        värdena
2909  38 DE           JR   C,L28E9
290B  E1              POP  HL
290C  1A              LD   A,(DE)
290D  EE B7           XOR  $B7            fler variabler?
290F  28 DD           JR   Z,READ
2911  C9              RET
```

RDVALS, gemensam för READ och INPUT: först reserveras plats på stacken för alla värden (märke med typ, sist märket $80), sedan tolkas texten: heltal (GETINT, fel 210 från 65536, . och E ger fel 147), flyttal (TXTNUM), sträng med " eller ' (dubblat = ett tecken) eller utan citattecken till "," eller radslut, med blanka först och sist borttagna. Efter ett värde ska "," eller radslut följa, annars fel 147. För få värden ger fel 148, för många vid INPUT (typbytens bit 3) fel 150; READ sparar i stället läspositionen. Först när allt är tolkat tilldelas variablerna, så vid fel är ingen ändrad. INPUT frågar inte om utan ger felet. In: HL = textens position, DE vid variabellistan. Ut: NC: värdena tilldelade, DE efter variablerna (för READ även DATPTR = positionen efter värdena); CY: A = felnumret och ingen variabel ändrad. Förstör de övriga registren.

```
2912  E5        RDVALS:PUSH HL
2913  13        L2913: INC  DE            till sista typbyten (>= 4)
2914  1A               LD   A,(DE)
2915  FE 04            CP   $04
2917  38 FA            JR   C,L2913
2919  D5               PUSH DE
291A  21 02 00         LD   HL,$0002
291D  39               ADD  HL,SP
291E  47               LD   B,A
291F  0E 80            LD   C,$80         slutmärke
2921  C5               PUSH BC
2922  E6 03     L2922: AND  $03
2924  4F               LD   C,A
```

```
2925  C5                PUSH BC             märke + plats för värdet
2926  C5                PUSH BC
2927  CB 47             BIT  0,A
2929  20 0D             JR   NZ,L2938
292B  C5                PUSH BC
292C  CB 4F             BIT  1,A
292E  20 08             JR   NZ,L2938
2930  FD CB 1E 5E       BIT  3,(IY+$1E)
2934  28 02             JR   Z,L2938
2936  C5                PUSH BC
2937  C5                PUSH BC
2938  1B          L2938: DEC  DE             föregående typbyte
2939  1A                LD   A,(DE)
293A  CB 7F             BIT  7,A
293C  28 E4             JR   Z,L2922
293E  E5                PUSH HL
293F  5E                LD   E,(HL)
2940  23                INC  HL
2941  56                LD   D,(HL)
2942  21 02 00          LD   HL,$0002
2945  39                ADD  HL,SP
2946  EB                EX   DE,HL         HL = texten, DE = platserna
2947  2B          L2947: DEC  HL
2948  DF                RST  $18           blanka hoppas över
2949  13                INC  DE
294A  13                INC  DE
294B  1A                LD   A,(DE)
294C  CB 47             BIT  0,A
294E  20 0B             JR   NZ,L295B
2950  CB 4F             BIT  1,A
2952  20 18             JR   NZ,L296C
2954  CD 1D 48          CALL TXTNUM         flyttal
2957  38 70             JR   C,L29C9
2959  18 52             JR   L29AD

295B  D5          L295B: PUSH DE
295C  CD FA 4B          CALL GETINT         heltal
295F  3E D2             LD   A,$D2          fel 210
2961  E3                EX   (SP),HL
2962  73                LD   (HL),E
2963  23                INC  HL
2964  72                LD   (HL),D
2965  23                INC  HL
2966  EB                EX   DE,HL
2967  E1                POP  HL
2968  38 5F             JR   C,L29C9
296A  18 41             JR   L29AD

296C  7E          L296C: LD   A,(HL)         sträng
296D  01 00 2C          LD   BC,$2C00       slutar vid ,
2970  FE 22             CP   $22
2972  28 04             JR   Z,L2978
2974  FE 27             CP   $27
2976  20 02             JR   NZ,L297A
2978  23          L2978: INC  HL             inom citattecken
2979  47                LD   B,A
297A  EB          L297A: EX   DE,HL
297B  73                LD   (HL),E         +0 textens adress
297C  23                INC  HL
297D  72                LD   (HL),D
297E  23                INC  HL
297F  EB                EX   DE,HL
2980  7E          L2980: LD   A,(HL)
2981  FE 0D             CP   $0D
2983  28 16             JR   Z,L299B
2985  B8                CP   B              avgränsaren?
2986  28 07             JR   Z,L298F
2988  87                ADD  A,A
2989  38 10             JR   C,L299B
```

```
298B  23          L298B:  INC  HL
298C  0C                  INC  C
298D  18 F1               JR   L2980

298F  FE 2C       L298F:  CP   $2C
2991  28 08               JR   Z,L299B
2993  23                  INC  HL
2994  BE                  CP   (HL)          dubbelt = ett tecken
2995  28 F4               JR   Z,L298B
2997  2B                  DEC  HL
2998  DF                  RST  $18           blanka efter citatet
2999  18 0C               JR   L29A7

299B  0C          L299B:  INC  C             blanka sist stryks
299C  3E 20               LD   A,$20
299E  E5                  PUSH HL
299F  0D          L299F:  DEC  C
29A0  28 04               JR   Z,L29A6
29A2  2B                  DEC  HL
29A3  BE                  CP   (HL)
29A4  28 F9               JR   Z,L299F
29A6  E1          L29A6:  POP  HL
29A7  EB          L29A7:  EX   DE,HL
29A8  71                  LD   (HL),C        +2 längd, +3 avgränsare
29A9  23                  INC  HL
29AA  70                  LD   (HL),B
29AB  23                  INC  HL
29AC  EB                  EX   DE,HL
29AD  7E          L29AD:  LD   A,(HL)
29AE  CB 7F               BIT  7,A
29B0  20 1C               JR   NZ,L29CE
29B2  FE 0D               CP   $0D
29B4  28 18               JR   Z,L29CE
29B6  FE 2C               CP   $2C           , efter värdet?
29B8  3E 93               LD   A,$93         fel 147
29BA  20 0D               JR   NZ,L29C9
29BC  23                  INC  HL
29BD  1A                  LD   A,(DE)        fler variabler?
29BE  87                  ADD  A,A
29BF  30 86               JR   NC,L2947
29C1  13                  INC  DE
29C2  1A                  LD   A,(DE)
29C3  E6 08               AND  $08           INPUT (bit 3)?
29C5  28 11               JR   Z,L29D8
29C7  3E 96               LD   A,$96         fel 150: för många värden
29C9  E1          L29C9:  POP  HL            SP tillbaka, CY (egenhet)
29CA  F9                  LD   SP,HL
29CB  D1                  POP  DE
29CC  37                  SCF
29CD  C9                  RET

29CE  1A          L29CE:  LD   A,(DE)
29CF  87                  ADD  A,A
29D0  3E 94               LD   A,$94         fel 148: för få värden
29D2  30 F5               JR   NC,L29C9
29D4  13                  INC  DE
29D5  1A                  LD   A,(DE)
29D6  E6 08               AND  $08
29D8  CC E5 28    L29D8:  CALL Z,S28E5       READ: spara läspositionen
29DB  E1                  POP  HL
29DC  2B                  DEC  HL
29DD  56                  LD   D,(HL)
29DE  2B                  DEC  HL
29DF  5E                  LD   E,(HL)
29E0  13                  INC  DE            DE = första referensen
29E1  E1                  POP  HL
29E2  E7          L29E2:  RST  $20           variabelns adress, B = typ
29E3  78                  LD   A,B
29E4  05                  DEC  B
29E5  28 07               JR   Z,L29EE
```

```
29E7  05              DEC  B
29E8  28 13           JR   Z,L29FD
29EA  FD 7E 1E        LD   A,(IY+$1E)      flyttal: 2 eller 4 ord
29ED  0F              RRCA
29EE  C1        L29EE: POP  BC
29EF  71              LD   (HL),C          värdet till variabeln
29F0  23              INC  HL
29F1  70              LD   (HL),B
29F2  23              INC  HL
29F3  3D              DEC  A
29F4  20 F8           JR   NZ,L29EE
29F6  F1        L29F6: POP  AF             märket: fler?
29F7  F2 E2 29        JP   P,L29E2
29FA  E1              POP  HL
29FB  E1              POP  HL
29FC  C9              RET

29FD  E3        L29FD: EX   (SP),HL
29FE  DD E1           POP  IX
2A00  C1              POP  BC
2A01  D5              PUSH DE
2A02  C5              PUSH BC
2A03  E5              PUSH HL
2A04  06 00           LD   B,$00
2A06  CD 50 24        CALL SVSET           plats i strängvariabeln
2A09  E1              POP  HL
2A0A  F1              POP  AF
2A0B  0C              INC  C
2A0C  0D              DEC  C
2A0D  28 09           JR   Z,L2A18
2A0F  BE        L2A0F: CP   (HL)           avgränsaren?
2A10  20 01           JR   NZ,L2A13
2A12  23              INC  HL              dubbelt: ta ett
2A13  ED A0     L2A13: LDI
2A15  EA 0F 2A        JP   PE,L2A0F
2A18  D1        L2A18: POP  DE
2A19  18 DB           JR   L29F6
```

FNCALL ($CF, *antal, avstånd, referens*): *anrop av en egen* DEF FN. *Referensen pekar (relativt) på* DEF*-satsens antal parametrar; två bytes före står funktionsnamnet, vars låga bitar ger typen. Argumentvärdena ligger redan på stacken (vart och ett med* $AA+*typ, omvandlade till parametrarnas typ: FNA(2.6) med X% ger X% = 3). Efter dem läggs de lokala variablerna (0, tom sträng med bufferten på stacken), sedan en pekartabell med ett ord per namn (FNFRM pekar dit; lokala referenser $FFxx läses via FNFRM, se REFADR) och ramen: SP före argumenten, fortsättningen efter anropet, FRMSP, FNFRM, IY+$27, ONEADR och ONEFRM. En* ON ERROR GOTO *i funktionen gäller alltså bara där. Enradig funktion: uttrycket körs (*RST $28*) och slutar med* $B7+*typ; flerradig: satsslingan körs från raden efter* DEF *till* RETURN *uttryck. Ett märke med B = 0 och C = typ ligger överst. Varje nivå tar ca 26 bytes stack (FNS(N)=N+FNS(N-1) ger fel 201 efter ca 1100 nivåer med BASIC II på ABC80 med 32 kilobyte, ett heltals-FN ca 1200). Fel 201 går via* MEMERR *och fångas inte av* ON ERROR. *Fel antal argument eller fel typ ger fel 223 redan vid länkningen. In: DE vid antalet (efter* $CF*), argumenten vid SP. Ut: kommer inte tillbaka hit; funktionens kropp körs, och värdet kommer tillbaka via* FRETF, FRETI *eller* FRETS. *Förstör alla register.*

```
2A1B  13        FNCALL: INC  DE            hoppa över antal och avstånd
2A1C  13                INC  DE
2A1D  EB                EX   DE,HL
2A1E  4E                LD   C,(HL)
2A1F  23                INC  HL
2A20  46                LD   B,(HL)
2A21  E5                PUSH HL            IX = efter referensen (fortsättningen)
2A22  DD E1             POP  IX
2A24  DD 23             INC  IX
2A26  09        L2A26:  ADD  HL,BC         HL = DEF:s antal parametrar
2A27  EB                EX   DE,HL
2A28  21 FC FF          LD   HL,$FFFC
2A2B  19                ADD  HL,DE
2A2C  7E                LD   A,(HL)        namnets låga byte: typ
2A2D  EB                EX   DE,HL
2A2E  E5                PUSH HL
2A2F  08                EX   AF,AF'
2A30  D9                EXX
```

```
2A31  E1                POP  HL
2A32  4E                LD   C,(HL)
2A33  06 00             LD   B,$00
2A35  23                INC  HL
2A36  09                ADD  HL,BC
2A37  09                ADD  HL,BC
2A38  7E                LD   A,(HL)          efter parametrarna: LOCAL?
2A39  FE B6             CP   $B6
2A3B  20 40             JR   NZ,L2A7D
2A3D  23                INC  HL
2A3E  46                LD   B,(HL)
2A3F  79                LD   A,C
2A40  80                ADD  A,B
2A41  4F                LD   C,A
2A42  23                INC  HL
2A43  11 00 00    L2A43: LD   DE,$0000        lokal: 0 på stacken
2A46  CB 46             BIT  0,(HL)
2A48  20 0D             JR   NZ,L2A57
2A4A  CB 4E             BIT  1,(HL)
2A4C  20 10             JR   NZ,L2A5E
2A4E  FD CB 1E 5E       BIT  3,(IY+$1E)      flyttal: 2 eller 4 ord
2A52  28 02             JR   Z,L2A56
2A54  D5                PUSH DE
2A55  D5                PUSH DE
2A56  D5          L2A56: PUSH DE
2A57  D5          L2A57: PUSH DE
2A58  23                INC  HL
2A59  23                INC  HL
2A5A  10 E7             DJNZ L2A43
2A5C  18 1F             JR   L2A7D

2A5E  23          L2A5E: INC  HL              sträng: maxlängden efter namnet
2A5F  23                INC  HL
2A60  5E                LD   E,(HL)
2A61  23                INC  HL
2A62  56                LD   D,(HL)
2A63  E5                PUSH HL
2A64  21 02 00          LD   HL,$0002
2A67  39                ADD  HL,SP
2A68  ED 52             SBC  HL,DE
2A6A  CD F6 0F          CALL SPCHK           bufferten på stacken
2A6D  DA 69 3B          JP   C,MEMERR        fel 201
2A70  D1                POP  DE
2A71  F9                LD   SP,HL
2A72  E5                PUSH HL
2A73  ED 62             SBC  HL,HL           +0 maxlängd, +2 adress, +4 längd 0
2A75  E3                EX   (SP),HL
2A76  E5                PUSH HL
2A77  EB                EX   DE,HL
2A78  56                LD   D,(HL)
2A79  2B                DEC  HL
2A7A  5E                LD   E,(HL)
2A7B  18 DA             JR   L2A57

2A7D  21 00 00    L2A7D: LD   HL,$0000        ett ord per namn: pekartabellen
2A80  39                ADD  HL,SP
2A81  CB 21             SLA  C
2A83  ED 42             SBC  HL,BC
2A85  CD F6 0F          CALL SPCHK
2A88  DA 69 3B          JP   C,MEMERR
2A8B  F9                LD   SP,HL
2A8C  E5                PUSH HL
2A8D  D9                EXX
2A8E  D1                POP  DE
2A8F  4E                LD   C,(HL)
2A90  23                INC  HL
2A91  06 00             LD   B,$00
2A93  79                LD   A,C
2A94  CB 21             SLA  C
```

```
2A96  28 03              JR   Z,L2A9B
2A98  ED B0              LDIR                 parametrarnas namn till tabellen
2A9A  4F                 LD   C,A
2A9B  7E           L2A9B:LD   A,(HL)
2A9C  FE B6              CP   $B6
2A9E  20 14              JR   NZ,L2AB4
2AA0  23                 INC  HL
2AA1  46                 LD   B,(HL)
2AA2  79                 LD   A,C
2AA3  80                 ADD  A,B
2AA4  4F                 LD   C,A
2AA5  23                 INC  HL
2AA6  7E           L2AA6:LD   A,(HL)          de lokalas typbyte
2AA7  12                 LD   (DE),A
2AA8  13                 INC  DE
2AA9  13                 INC  DE
2AAA  23                 INC  HL
2AAB  23                 INC  HL
2AAC  E6 02              AND  $02
2AAE  28 02              JR   Z,L2AB2
2AB0  23                 INC  HL
2AB1  23                 INC  HL
2AB2  10 F2        L2AB2:DJNZ L2AA6
2AB4  E5           L2AB4:PUSH HL
2AB5  6B                 LD   L,E
2AB6  62                 LD   H,D
2AB7  79                 LD   A,C
2AB8  41                 LD   B,C
2AB9  B7                 OR   A
2ABA  28 1D              JR   Z,L2AD9
2ABC  2B           L2ABC:DEC  HL               bakifrån: ordet = värdets adress
2ABD  72                 LD   (HL),D
2ABE  2B                 DEC  HL
2ABF  7E                 LD   A,(HL)
2AC0  73                 LD   (HL),E
2AC1  CB 47              BIT  0,A
2AC3  20 10              JR   NZ,L2AD5
2AC5  CB 4F              BIT  1,A              sträng: 6 bytes + bufferten
2AC7  20 43              JR   NZ,L2B0C
2AC9  FD CB 1E 5E        BIT  3,(IY+$1E)
2ACD  28 04              JR   Z,L2AD3
2ACF  13                 INC  DE
2AD0  13                 INC  DE
2AD1  13                 INC  DE
2AD2  13           L2AD2:INC  DE
2AD3  13           L2AD3:INC  DE
2AD4  13                 INC  DE
2AD5  13           L2AD5:INC  DE
2AD6  13                 INC  DE
2AD7  10 E3              DJNZ L2ABC
2AD9  EB           L2AD9:EX   DE,HL           DE = kroppen, HL = tabellen
2ADA  E3                 EX   (SP),HL
2ADB  EB                 EX   DE,HL
2ADC  DD E5              PUSH IX               ramen: fortsättningen, FRMSP,
2ADE  ED 4B 45 FF        LD   BC,(FRMSP)
2AE2  C5                 PUSH BC
2AE3  ED 4B 18 FF        LD   BC,(FNFRM)       FNFRM (= tabellen), IY+$27,
2AE7  22 18 FF           LD   (FNFRM),HL
2AEA  C5                 PUSH BC
2AEB  FD 4E 27           LD   C,(IY+$27)
2AEE  C5                 PUSH BC
2AEF  ED 4B 42 FF        LD   BC,(ONEADR)      ONEADR och ONEFRM
2AF3  C5                 PUSH BC
2AF4  ED 4B 04 FF        LD   BC,(ONEFRM)
2AF8  C5                 PUSH BC
2AF9  CD A3 1D           CALL S1DA3
2AFC  08                 EX   AF,AF'
2AFD  E6 03              AND  $03              funktionens typ
2AFF  4F                 LD   C,A
```

```
2B00  1A                LD   A,(DE)
2B01  FE C0             CP   $C0            flerradig: satsslingan
2B03  DA 0E 1D          JP   C,L1D0E
2B06  06 00             LD   B,$00
2B08  C5                PUSH BC             märke B = 0, C = typ
2B09  1B                DEC  DE
```

FNAMRK ($AA-$AC): *markör för ett argument till en egen* DEF FN*, typ i de låga bitarna, följd av en byte med avståndet bakåt till förra argumentets markör (0 för det första). Har argumentet en annan typ än parametern byter* RUN *markören mot* $AD *eller* $AE *(se* FTOI*). Läggs ut av* CARGS ($18DB) *och ses över vid fixup (*$3021*); vid körning hoppas koden och byten över.* LIST *hoppar också över den (*DSKIP*).*

```
2B0A  13          FNAMRK: INC  DE           hoppa över avståndsbyten
2B0B  EF                  RST  $28
2B0C  EB          L2B0C:  EX   DE,HL        (FNCALL) sträng: förbi bufferten och beskrivningen
2B0D  C5                  PUSH BC
2B0E  4E                  LD   C,(HL)
2B0F  23                  INC  HL
2B10  46                  LD   B,(HL)
2B11  09                  ADD  HL,BC
2B12  C1                  POP  BC
2B13  EB                  EX   DE,HL
2B14  18 BC               JR   L2AD2
```

S2B16*: prövar typen vid retur ur en funktion. A = märkets C: 2 (strängfunktion) eller mer (*$FD/$FE*, en RUN- eller GOSUB-ram) ger fel 140 via* L2BE7*.* S2B1B*: lägger tillbaka det som* FNCALL *sparade under pekartabellen:* PRGPTR *= fortsättningen,* FRMSP*,* FNFRM*, IY+*$27*,* ONEADR *och* ONEFRM*. In: A = funktionens typ,* FNFRM *= pekartabellen. Ut: HL = SP före argumenten, Z om A = 0 (flyttal), CY = 0; A och BC som vid ingången. Förstör F.*

```
2B16  FE 02       S2B16:  CP   $02            sträng/tal fel: fel 140
2B18  D2 E7 2B            JP   NC,L2BE7
2B1B  C5          S2B1B:  PUSH BC            lägg tillbaka det FNCALL sparade
2B1C  2A 18 FF            LD   HL,(FNFRM)
2B1F  2B                  DEC  HL
2B20  46                  LD   B,(HL)
2B21  2B                  DEC  HL
2B22  4E                  LD   C,(HL)
2B23  C5                  PUSH BC
2B24  2B                  DEC  HL
2B25  46                  LD   B,(HL)
2B26  2B                  DEC  HL
2B27  4E                  LD   C,(HL)
2B28  ED 43 3A FF         LD   (PRGPTR),BC
2B2C  2B                  DEC  HL
2B2D  46                  LD   B,(HL)
2B2E  2B                  DEC  HL
2B2F  4E                  LD   C,(HL)
2B30  ED 43 45 FF         LD   (FRMSP),BC
2B34  2B                  DEC  HL
2B35  46                  LD   B,(HL)
2B36  2B                  DEC  HL
2B37  4E                  LD   C,(HL)
2B38  ED 43 18 FF         LD   (FNFRM),BC
2B3C  2B                  DEC  HL
2B3D  2B                  DEC  HL
2B3E  4E                  LD   C,(HL)
2B3F  FD 71 27            LD   (IY+$27),C
2B42  2B                  DEC  HL
2B43  46                  LD   B,(HL)
2B44  2B                  DEC  HL
2B45  4E                  LD   C,(HL)
2B46  ED 43 42 FF         LD   (ONEADR),BC
2B4A  2B                  DEC  HL
2B4B  46                  LD   B,(HL)
2B4C  2B                  DEC  HL
2B4D  4E                  LD   C,(HL)
2B4E  ED 43 04 FF         LD   (ONEFRM),BC
2B52  E1                  POP  HL
```

```
2B53  C1                POP  BC
2B54  A7                AND  A
2B55  C9                RET
```

FRETF/FRETI/FRETS ($B7-$B9)*: lämna ett värde. Är det översta ordet funktionens märke (B = 0) är det retur från* DEF FN*: typen prövas (*S2B16*, sträng mot tal fel 140),* S2B1B *lägger tillbaka* FNCALL*:s sparade värden, SP sätts till läget före argumenten och värdet läggs dit, omvandlat efter funktionens typ (FN% avrundar och slår runt: FNI%(40000) = -25536).* FRETS *flyttar texten ned om den ligger i den frigjorda ramen (en lokal sträng). Annars är det ett vanligt slut på ett uttryck och värdet lämnas till anroparen.* RETURN *uttryck utanför en funktion, eller i en* GOSUB *inuti den, hittar inte märket: fel 140, samma nummer som* RETURN *utan* GOSUB*. In: värdet vid SP (för FRETS beskrivningen), under det funktionens märke eller anroparens returadress. Ut: vid funktionsretur ligger värdet där argumenten började och körningen fortsätter efter anropet (*$2180*); annars får anroparen värdet (FRETF: på stacken, B = 0; FRETI: HL = heltalet, B = 1). Förstör alla register.*

```
2B56  E1          FRETF:  POP  HL
2B57  C1                  POP  BC
2B58  D9                  EXX
2B59  FD CB 1E 5E         BIT  3,(IY+$1E)      DOUBLE: 8 bytes
2B5D  28 02               JR   Z,L2B61
2B5F  C1                  POP  BC
2B60  D1                  POP  DE
2B61  08          L2B61:  EX   AF,AF'
2B62  F1                  POP  AF
2B63  F5                  PUSH AF
2B64  A7                  AND  A               A = märkets B = 0: funktionsretur
2B65  28 0E               JR   Z,L2B75
2B67  DD E1       S2B67:  POP  IX              annars: värdet tillbaka, vidare
2B69  08                  EX   AF,AF'
2B6A  28 02               JR   Z,L2B6E
2B6C  D5                  PUSH DE
2B6D  C5                  PUSH BC
2B6E  D9          L2B6E:  EXX
2B6F  C5                  PUSH BC
2B70  E5                  PUSH HL
2B71  06 00               LD   B,$00
2B73  DD E9               JP   (IX)

2B75  E1          L2B75:  POP  HL
2B76  7D                  LD   A,L
2B77  CD 16 2B            CALL S2B16
2B7A  F9                  LD   SP,HL           SP = före argumenten
2B7B  CD 67 2B            CALL S2B67
2B7E  08                  EX   AF,AF'
2B7F  28 15               JR   Z,L2B96
2B81  C3 65 21            JP   L2165           FN%: avrunda till heltal

2B84  06 01       FRETI:  LD   B,$01
2B86  E1                  POP  HL
2B87  F1                  POP  AF              märket (B = 0)? annars RET
2B88  F5                  PUSH AF
2B89  A7                  AND  A
2B8A  C0                  RET  NZ
2B8B  EB                  EX   DE,HL
2B8C  C1                  POP  BC
2B8D  79                  LD   A,C
2B8E  CD 16 2B            CALL S2B16
2B91  F9                  LD   SP,HL
2B92  D5                  PUSH DE
2B93  CC C0 4C            CALL Z,ITOFS         talfunktion: ITOF
2B96  C3 80 21    L2B96:  JP   L2180

2B99  E1          FRETS:  POP  HL              lämna strängvärde (egenhet)
2B9A  23                  INC  HL
2B9B  23                  INC  HL
2B9C  E5                  PUSH HL
2B9D  39                  ADD  HL,SP
2B9E  23                  INC  HL
2B9F  23                  INC  HL
2BA0  23                  INC  HL
2BA1  23                  INC  HL
```

```
2BA2  7E                LD   A,(HL)
2BA3  23                INC  HL
2BA4  66                LD   H,(HL)
2BA5  6F                LD   L,A
2BA6  06 02             LD   B,$02
2BA8  24                INC  H
2BA9  25                DEC  H
2BAA  C2 BC 1D          JP   NZ,S1DBC          inte i en funktion
2BAD  FE 02             CP   $02
2BAF  20 36             JR   NZ,L2BE7
2BB1  CD 1B 2B          CALL S2B1B
2BB4  D1                POP  DE
2BB5  D1                POP  DE
2BB6  C1                POP  BC
2BB7  E5                PUSH HL
2BB8  78                LD   A,B               tom sträng
2BB9  B1                OR   C
2BBA  28 1B             JR   Z,L2BD7
2BBC  ED 52             SBC  HL,DE             texten i ramen? flytta ned den
2BBE  38 1A             JR   C,L2BDA
2BC0  62                LD   H,D
2BC1  6B                LD   L,E
2BC2  ED 72             SBC  HL,SP
2BC4  38 14             JR   C,L2BDA
2BC6  EB                EX   DE,HL
2BC7  D1                POP  DE
2BC8  1B                DEC  DE
2BC9  09                ADD  HL,BC
2BCA  2B                DEC  HL
2BCB  C5                PUSH BC
2BCC  ED B8             LDDR
2BCE  C1                POP  BC
2BCF  EB                EX   DE,HL
2BD0  23                INC  HL
2BD1  F9                LD   SP,HL
2BD2  C5                PUSH BC
2BD3  E5                PUSH HL
2BD4  C5                PUSH BC
2BD5  18 BF             JR   L2B96

2BD7  11 00 00    L2BD7: LD   DE,$0000
2BDA  E1          L2BDA: POP  HL
2BDB  F9                 LD   SP,HL
2BDC  C5                 PUSH BC
2BDD  D5                 PUSH DE
2BDE  01 00 00           LD   BC,$0000
2BE1  C5                 PUSH BC
2BE2  18 B2              JR   L2B96
```

FNEND: *fel 145, en flerradig funktion kom till FNEND utan RETURN.* L2BE7: *fel 140. Båda går via* ERRSP: *ON ERROR fångar dem inte och CON går inte.*

```
2BE4  3E 91       FNEND: LD   A,$91           fel 145: FNEND nådd (ingen RETURN)
2BE6  21                 DEFB $21             LD HL,$8C3E (överlappar nästa)
2BE7  3E 8C       L2BE7: LD   A,$8C           fel 140
2BE9  C3 6B 3B           JP   ERRSP
```

MID: MID$(*a$,s,n*) = *b$. s och n avrundas till närmaste och prövas som i* MID$*-funktionen (fel 134 utanför strängen). b$ skrivs in från position s: en kortare text ersätter bara sina egna tecken* (MID$(*A$,2,3*)="XY" *ger AXYDEF av ABCDEF*)*, en tom ingenting, en längre än n ger fel 136. Går även på fältelement. In: DE vid koden för a$, s och n. Ut: DE efter satsen. Förstör de övriga registren.*

```
2BEC  E7          MID:   RST  $20
2BED  13                 INC  DE
2BEE  E7                 RST  $20
2BEF  DD E1              POP  IX
2BF1  E1                 POP  HL
2BF2  C1                 POP  BC
2BF3  DD 39              ADD  IX,SP
```

```
2BF5  78                LD   A,B
2BF6  B1                OR   C
2BF7  28 10             JR   Z,L2C09          tom text: ingenting ändras
2BF9  E5                PUSH HL
2BFA  CD 67 27          CALL IXHL4
2BFD  ED 42             SBC  HL,BC
2BFF  38 10             JR   C,L2C11          texten längre än fönstret
2C01  E1                POP  HL
2C02  D5                PUSH DE
2C03  CD 48 27          CALL IXDE2
2C06  ED B0             LDIR                  texten in i strängen
2C08  D1                POP  DE
2C09  DD F9       L2C09: LD   SP,IX
2C0B  E1                POP  HL
2C0C  C1                POP  BC
2C0D  C1                POP  BC
2C0E  39                ADD  HL,SP
2C0F  F9                LD   SP,HL
2C10  C9                RET

2C11  D7          L2C11: RST  $10              fel 136: texten för lång (även CHAIN)
2C12  88                 DEFB $88
```

ONERR: ON ERROR GOTO nr. ONEADR = radens adress, ONEFRM = FNFRM (en ON ERROR i en funktion gäller bara där), IY+$27 bit 1 sätts. ONERR0: ON ERROR GOTO utan nummer stänger av. I felrutinen gör den dessutom att felet skrivs ut och programmet stannar, som om det inte hade fångats, men bara om programmet har någon RESUME. Utan RESUME är felläget redan släppt när felrutinen börjar (se ERRHND), och då stängs bara ON ERROR av. In: DE vid avståndet till raden (ONERR0: efter koden). Ut: DE efter satsen. Förstör F och HL (ONERR0: AF och HL, och i felrutinen kommer den inte tillbaka).

```
2C13  EB          ONERR: EX   DE,HL
2C14  5E                 LD   E,(HL)
2C15  23                 INC  HL
2C16  56                 LD   D,(HL)
2C17  EB                 EX   DE,HL
2C18  19                 ADD  HL,DE
2C19  13                 INC  DE
2C1A  22 42 FF           LD   (ONEADR),HL     ON ERROR GOTO: radens adress
2C1D  2A 18 FF           LD   HL,(FNFRM)
2C20  22 04 FF           LD   (ONEFRM),HL
2C23  FD CB 27 CE        SET  1,(IY+$27)
2C27  C9                 RET

2C28  FD CB 27 8E ONERR0: RES  1,(IY+$27)
2C2C  CD 49 3C           CALL RESCHK
2C2F  D8                 RET  C               utanför felrutinen: bara avstängning
2C30  F9                 LD   SP,HL
2C31  C3 C8 3B           JP   L3BC8           i felrutinen: felet skrivs ut
```

GOSUB: SPCHK0 (stacken full: fel 201 via ERRSP), sedan en ram med returadressen, FRMSP och C = $FE. RETURN tar senaste ramen i FRMSP-kedjan. FOR- och WHILE-block ligger utanför kedjan och hindrar inte RETURN (FOR I=1 TO 3 : RETURN går, I = 1). Är ramen ingen GOSUB blir det fel 140 via ERRSP: ON ERROR fångar det inte och CON går inte. In: DE vid avståndet till raden (RETURN: efter koden, eller vid uttrycket i RETURN uttryck). Ut: GOSUB: DE = målet; RETURN: DE efter GOSUB-satsen, FRMSP = föregående ram. Förstör AF, BC och HL.

```
2C34  CD EF 0F    GOSUB: CALL SPCHK0
2C37  38 29              JR   C,L2C62         stacken full: fel 201, fångas inte
2C39  EB                 EX   DE,HL
2C3A  5E                 LD   E,(HL)
2C3B  23                 INC  HL
2C3C  56                 LD   D,(HL)
2C3D  EB                 EX   DE,HL
2C3E  19                 ADD  HL,DE
2C3F  13                 INC  DE
2C40  0E FE       L2C40: LD   C,$FE           ram: retur, FRMSP, C = $FE (GOSUB)
2C42  EB                 EX   DE,HL
2C43  C3 09 1D           JP   L1D09

2C46  1A          RETURN: LD   A,(DE)
2C47  C6 40               ADD  A,$40
```

```
2C49  2A 45 FF            LD   HL,(FRMSP)
2C4C  F9                  LD   SP,HL          senaste ramen; FOR/WHILE ligger utanför kedjan
2C4D  30 01               JR   NC,L2C50
2C4F  EF                  RST  $28
2C50  E1            L2C50: POP  HL
2C51  3E FE               LD   A,$FE
2C53  BD                  CP   L
2C54  20 07               JR   NZ,L2C5D       ingen GOSUB-ram: fel 140, fångas inte
2C56  E1                  POP  HL
2C57  22 45 FF            LD   (FRMSP),HL
2C5A  D1                  POP  DE
2C5B  18 26               JR   L2C83

2C5D  87            L2C5D: ADD  A,A(egenhet)
2C5E  3E 8B               LD   A,$8B          blir alltid $8C (140): CY = 1 efter ADD A,A med A = $FE
2C60  CE 00               ADC  A,$00
2C62  C3 6B 3B      L2C62: JP   ERRSP
```

ON: index (avrundat till närmaste; ON 2.5 tar tredje numret) minus antalet; 0 eller för stort ger fel 138, som ON ERROR fångar. Koden efter listan väljer: GOTO, GOSUB (återvänder efter listan), RESTORE, annars RESUME. ON n ON ERROR GOTO ($85) blir alltså samma sak som ON n RESUME: fel 151 utanför felrutinen, och i den RESUME till raden. In: DE vid indexuttrycket. Ut: som GOTO, GOSUB, RESTORE eller RESUME till det valda numret. Förstör de övriga registren.

```
2C65  E7            ON:   RST  $20
2C66  13                  INC  DE
2C67  1A                  LD   A,(DE)
2C68  13                  INC  DE
2C69  D5                  PUSH DE
2C6A  13                  INC  DE
2C6B  95                  SUB  L
2C6C  4F                  LD   C,A
2C6D  2B                  DEC  HL
2C6E  3E 00               LD   A,$00
2C70  9C                  SBC  A,H
2C71  38 1E               JR   C,L2C91        index 0 eller större än antalet
2C73  44                  LD   B,H
2C74  29                  ADD  HL,HL
2C75  19                  ADD  HL,DE
2C76  5E                  LD   E,(HL)
2C77  23                  INC  HL
2C78  56                  LD   D,(HL)
2C79  EB                  EX   DE,HL
2C7A  19                  ADD  HL,DE
2C7B  EB                  EX   DE,HL
2C7C  23                  INC  HL
2C7D  09                  ADD  HL,BC
2C7E  09                  ADD  HL,BC
2C7F  C1                  POP  BC
2C80  0A                  LD   A,(BC)
2C81  D6 80               SUB  $80            $80 GOTO, $82 GOSUB, $83 RESTORE
2C83  28 59         L2C83: JR   Z,L2CDE
2C85  EB                  EX   DE,HL
2C86  3D                  DEC  A
2C87  3D                  DEC  A
2C88  28 B6               JR   Z,L2C40
2C8A  3D                  DEC  A
2C8B  CA CD 28            JP   Z,L28CD
2C8E  C3 27 3C            JP   L3C27          $84 RESUME, även $85 ON ERROR GOTO (fel)

2C91  D7            L2C91: RST  $10           fel 138: ON-index utanför listan
2C92  8A                   DEFB $8A
```

CHAIN fil: namnet kopieras till BUFADR (högst 160 tecken, annars fel 136), programmet stängs och filen laddas som vid LOAD (S14A8) utan att COMMON-variablerna nollställs; sedan RUN. Ett fel när filen öppnas går via ERRJP: CHAIN "FINNSEJ" ger fel 21, som ON ERROR fångar, och programmet finns kvar. Först när filen är öppen gör S14A8 NEW, och ett fel efter det (201, 203, 205) går via ERRSP; då finns inget program kvar. COMMON-värdena följer med (COMMON A,B$=10: det nya programmet såg A = 5 och B$ = HEJ). In: DE vid namnets uttryck. Ut: kommer inte tillbaka; RUN0 startar det nya programmet. Förstör alla register.

```
2C93  E7            CHAIN: RST  $20
```

```
2C94  CD 51 00        CALL BUFADR
2C97  D1              POP  DE
2C98  D1              POP  DE
2C99  E3              EX   (SP),HL
2C9A  AF              XOR  A
2C9B  7D              LD   A,L
2C9C  2B              DEC  HL
2C9D  ED 42           SBC  HL,BC
2C9F  E1              POP  HL
2CA0  E5              PUSH HL
2CA1  EB              EX   DE,HL
2CA2  4F              LD   C,A
2CA3  D2 11 2C        JP   NC,L2C11      namnet längre än 160 tecken
2CA6  ED B0           LDIR
2CA8  3E 0D           LD   A,$0D
2CAA  12              LD   (DE),A
2CAB  E1              POP  HL
2CAC  CD 0F 11        CALL CLSPRG        programfilen stängs
2CAF  3E 02           LD   A,$02
2CB1  E5              PUSH HL
2CB2  CD 35 10        CALL OPEN          fel här (21, filen finns inte) fångas, programmet kvar
2CB5  E1              POP  HL
2CB6  B4              OR   H             NZ: S14A8 gör NEW
2CB7  F9              LD   SP,HL
2CB8  FD CB 1D A6     RES  4,(IY+$1D)    COMMON behålls
2CBC  CD A8 14        CALL S14A8
2CBF  CD 0F 11        CALL CLSPRG
2CC2  38 9E           JR   C,L2C62       fel efter NEW: via ERRSP, fångas inte
2CC4  C3 F3 1C        JP   RUN0          kör det nya programmet
```

TRCHOK: TRACE-kroken (HOOK3), nås från EOL vid varje $87 när IY+$23 bit 1 är satt. Den kör minisatsen TRCCOD ($2CE1), som är kompilerad kod för PRINT CALL($2CFC) med PEEK2 och ;: TRCLNR ger adressen till nästa rads nummer. Utskriften går till TRACE-filen (IY+$35, 0 = skärmen) och skrivs utan tecken, så rad 40000 blir inte negativ. Varje nummer får ett blanksteg före och efter (" 10 20"). Ett hopp landar på $87 före målet och visar alltså målraden; en slinga inom en rad syns inte. Programslut utan END ger 0 (slutmärkets "radnummer"). Direktrader skrivs inte. In: DE vid koden efter satsen. Ut: DE som vid ingången; stod DE vid $87 är numret skrivet och PRGPTR = DE. Förstör de övriga registren.

```
2CC7  1A           TRCHOK: LD   A,(DE)          radslut från EOL via HOOK3 (egenhet)
2CC8  FE 87                CP   $87
2CCA  C0                   RET  NZ
2CCB  21 E1 2C             LD   HL,$2CE1       minisatsen TRCCOD
2CCE  D5                   PUSH DE
2CCF  EB                   EX   DE,HL
2CD0  22 20 FF             LD   (SCNPTR),HL    SCNPTR = $87 före raden
2CD3  FD 6E 35             LD   L,(IY+$35)     L = TRACE-filen (0 = skärmen)
2CD6  FD CB 27 9E          RES  3,(IY+$27)     utan tecken: 40000 blir inte negativt
2CDA  CD 09 1E             CALL S1E09          som PRINT #fil
2CDD  D1                   POP  DE
2CDE  C3 A3 1D     L2CDE:  JP   S1DA3

2CE1  C7 FC 2C D1… TRCCOD: DEFB $C7,$FC,$2C,$D1,$17,$7F,$B8,$3B  INTCON $2CFC, CALL, PEEK2, heltal, ;
2CE9  00                   DEFB $00            satsens slut
```

TRACE [#n]: en tidigare TRACE stängs av (NOTRAC), filen n måste vara öppen (annars fel 32); TRACE 1 ger fel 221. NOTRACE ($2003) slår av. In: DE efter koden, vid #n om det finns. Ut: DE efter satsen, IY+$35 = filen, IY+$23 bit 1 satt. Förstör de övriga registren.

```
2CEA  CD 03 20     TRACE:  CALL NOTRAC         en tidigare TRACE stängs av
2CED  CD 24 20             CALL S2024          #n, (L = 0 utan)
2CF0  7D                   LD   A,L
2CF1  CD 88 11             CALL FNDFIL         filen måste vara öppen: fel 32
2CF4  FD 75 35             LD   (IY+$35),L
2CF7  FD CB 23 CE          SET  1,(IY+$23)
2CFB  C9                   RET
```

TRCLNR: anropas av TRCCOD (CALL($2CFC)). In: SCNPTR vid $87 före raden. Ut: HL = adressen till radens nummer, två bytes längre fram. Förstör inget annat.

```
2CFC  2A 20 FF     TRCLNR: LD   HL,(SCNPTR)      HL = radnumrets adress (efter längden)
2CFF  23                   INC  HL
2D00  23                   INC  HL
2D01  C9                   RET
```

FIX och INT. FFRAC letar efter bråkbitar i talet vid HL (exponent $80 + antal heltalsbitar, mantissan efter): |x| < 1 är bara bråk, annars räknas de hela byten fram och en mask för den första byte som har bråkbitar. FTRUNC nollar bråkbitarna i talet på stacken. FIX avkortar alltså mot noll (FIX(-2.5) = -2, FIX(-.5) = 0), och INT drar dessutom ifrån 1 för negativa tal med bråk (INT(-2.5) = -3, INT(-.5) = -1). Med DOUBLE gäller alla 8 bytes (INT(123456789.9) = 123456789). In: x vid SP (nås via FNTAB från körslingan, DE efter koden). Ut: talet utan bråk vid SP, DE kvar, nästa kod (EXECNX). Förstör A, BC och HL.

```
2D02  CD 4F 2D     FIX:    CALL FTRUNC          bråkbitarna bort: mot noll
2D05  EF           L2D05:  RST  $28
2D06  CD 4F 2D     INT:    CALL FTRUNC
2D09  28 FA                JR   Z,L2D05         inget bråk: klart
2D0B  E1                   POP  HL
2D0C  E5                   PUSH HL
2D0D  CB 7C                BIT  7,H             positivt: klart
2D0F  28 F4                JR   Z,L2D05
2D11  CD A3 1D             CALL S1DA3
2D14  21 F2 4D             LD   HL,$4DF2        negativt med bråk: minus 1
2D17  CD 25 4C             CALL FLOAD
2D1A  CD 39 4F             CALL FSUB
2D1D  C3 80 21             JP   L2180
```

FFRAC: har talet vid HL bråkbitar? In: HL vid exponenten, IY+$1E = längden. Ut: Z = nej (heltal eller 0). NZ: HL = första byten med bråkbitar, C = deras mask och B = antalet bytes därifrån till slutet; är beloppet under 1 är det exponenten själv (C = $FF, B = 1, CY satt). DE orört; förstör A.

```
2D20  7E           FFRAC:  LD   A,(HL)          HL = exponenten, sedan mantissan
2D21  A7                   AND  A
2D22  C8                   RET  Z
2D23  01 FF 01             LD   BC,$01FF
2D26  FE 81                CP   $81             |x| < 1: allt är bråk
2D28  D8                   RET  C
2D29  0F                   RRCA                 C = hela bytes i heltalsdelen
2D2A  0F                   RRCA
2D2B  0F                   RRCA
2D2C  E6 0F                AND  $0F
2D2E  4F                   LD   C,A
2D2F  2F                   CPL
2D30  3D                   DEC  A
2D31  FD 86 1E             ADD  A,(IY+$1E)      IY+$1E = talets längd
2D34  30 17                JR   NC,L2D4D        inga bråkbitar: Z
2D36  3C                   INC  A
2D37  F5                   PUSH AF
2D38  7E                   LD   A,(HL)
2D39  E6 07                AND  $07
2D3B  47                   LD   B,A
2D3C  3E FF                LD   A,$FF           mask för bråkbitarna i första byten
2D3E  28 04                JR   Z,L2D44
2D40  A7           L2D40:  AND  A
2D41  1F                   RRA
2D42  10 FC                DJNZ L2D40
2D44  09           L2D44:  ADD  HL,BC
2D45  C1                   POP  BC
2D46  23           L2D46:  INC  HL
2D47  4F                   LD   C,A
2D48  A6                   AND  (HL)            bråkbit? NZ, C = masken
2D49  C0                   RET  NZ
2D4A  3D                   DEC  A
2D4B  10 F9                DJNZ L2D46
2D4D  AF           L2D4D:  XOR  A
2D4E  C9                   RET
```

FTRUNC: nollar bråkbitarna i talet under returadressen; ett tal med belopp under 1 blir 0 genom att exponenten nollas. Ut: Z = talet var redan ett heltal, NZ = ändrat. DE orört; förstör A, BC och HL.

```
2D4F  21 02 00     FTRUNC: LD   HL,$0002       talet under returadressen
2D52  39                   ADD  HL,SP
2D53  CD 20 2D             CALL FFRAC
2D56  C8                   RET  Z
2D57  79                   LD   A,C             bråkbitarna bort
2D58  2F                   CPL
2D59  A6                   AND  (HL)
2D5A  77           L2D5A:  LD   (HL),A
2D5B  23                   INC  HL
2D5C  AF                   XOR  A               resten av bytena 0
2D5D  10 FB                DJNZ L2D5A
2D5F  3C                   INC  A               NZ: talet ändrat
2D60  C9                   RET
```

*FPOWI: heltal upphöjt till heltal, genom kvadrering och IMUL (S224E): slår runt som * (200%^2% = -25536, 2%^15% = -32768) och ger fel 132 när ett mellanresultat är över 65535 (2%^16%). Negativ exponent: basen 1 ger 1, -1 ger -1 eller 1 efter exponentens paritet, 0 ger fel 132, annan bas 0 (3%^(-1%) = 0). In: exponenten vid SP, basen under (heltal; nås via FNTAB, DE efter koden). Ut: resultatet vid SP, DE kvar, nästa kod. Förstör A, BC och HL.*

```
2D61  C1           FPOWI:  POP  BC              BC = exponenten, HL = basen
2D62  E1                   POP  HL
2D63  D5                   PUSH DE
2D64  78                   LD   A,B
2D65  87                   ADD  A,A
2D66  38 21                JR   C,L2D89         negativ exponent
2D68  11 01 00             LD   DE,$0001
2D6B  EB                   EX   DE,HL
2D6C  CB 38        L2D6C:  SRL  B               exponentens bitar nedifrån
2D6E  CB 19                RR   C
2D70  30 07                JR   NC,L2D79
2D72  D5                   PUSH DE
2D73  C5                   PUSH BC
2D74  CD 4E 22             CALL S224E           resultatet * basen (IMUL)
2D77  C1                   POP  BC
2D78  D1                   POP  DE
2D79  78           L2D79:  LD   A,B
2D7A  B1                   OR   C
2D7B  28 1D                JR   Z,L2D9A
2D7D  C5                   PUSH BC
2D7E  E5                   PUSH HL
2D7F  62                   LD   H,D
2D80  6B                   LD   L,E
2D81  CD 4E 22             CALL S224E           basen kvadreras
2D84  D1                   POP  DE
2D85  C1                   POP  BC
2D86  EB                   EX   DE,HL
2D87  18 E3                JR   L2D6C

2D89  7C           L2D89:  LD   A,H             basen 0, 1 eller -1?
2D8A  B7                   OR   A
2D8B  20 10                JR   NZ,L2D9D
2D8D  B5                   OR   L
2D8E  3D                   DEC  A
2D8F  28 09                JR   Z,L2D9A
2D91  3C                   INC  A
2D92  3E 84                LD   A,$84           fel 132: 0 upphöjt till negativt tal
2D94  CC 33 00             CALL Z,ERRJP
2D97  21 00 00     L2D97:  LD   HL,$0000        annan bas: 0
2D9A  D1           L2D9A:  POP  DE
2D9B  E5                   PUSH HL
2D9C  EF                   RST  $28
2D9D  A5           L2D9D:  AND  L               -1?
2D9E  3C                   INC  A
2D9F  20 F6                JR   NZ,L2D97
2DA1  3E 01                LD   A,$01           udda exponent: -1, jämn: 1
2DA3  A1                   AND  C
2DA4  20 F4                JR   NZ,L2D9A
2DA6  21 01 00             LD   HL,$0001
2DA9  18 EF                JR   L2D9A
```

IPOW: flyttal upphöjt till heltal, genom kvadrering. Exponenten (beloppet) ligger i IY+$20/$21, och vid negativ exponent tas först 1/basen (0^(-1) ger fel 130). Resultatet är exakt så långt talen räcker: (-2)^3 = -8, 0^0 = 1. Spill ger fel 130, underspill 0. Exponenten hålls under tiden i SCNPTR. In: heltalsexponenten vid SP, flyttalsbasen under (DE efter koden). Ut: resultatet vid SP, DE = PRGPTR, nästa kod; CY ger fel 130. Förstör de övriga registren.

```
2DAB  CD A3 1D     IPOW:   CALL S1DA3           flyttal upphöjt till heltal
2DAE  E1           L2DAE:  POP  HL
2DAF  7C                   LD   A,H
2DB0  87                   ADD  A,A
2DB1  DC 6D 22             CALL C,NEGHL         negativ: beloppet
2DB4  22 20 FF             LD   (SCNPTR),HL
2DB7  30 0E                JR   NC,L2DC7
2DB9  21 F2 4D             LD   HL,$4DF2        negativ: basen = 1/basen
2DBC  CD 25 4C             CALL FLOAD
2DBF  CD 52 4C             CALL FSWAP
2DC2  CD 5F 50             CALL FDIV
2DC5  38 37                JR   C,L2DFE
2DC7  21 F2 4D     L2DC7:  LD   HL,$4DF2        resultatet = 1
2DCA  CD 25 4C             CALL FLOAD
2DCD  FD CB 21 3E  L2DCD:  SRL  (IY+$21)        exponentens bitar nedifrån
2DD1  FD CB 20 1E          RR   (IY+$20)
2DD5  30 0E                JR   NC,L2DE5
2DD7  FD 6E 1E             LD   L,(IY+$1E)
2DDA  26 00                LD   H,$00
2DDC  39                   ADD  HL,SP
2DDD  CD 25 4C             CALL FLOAD
2DE0  CD EA 4F             CALL FMUL            resultatet * basen
2DE3  38 11                JR   C,L2DF6
2DE5  CD 52 4C     L2DE5:  CALL FSWAP
2DE8  FD 7E 20             LD   A,(IY+$20)      fler bitar? basen kvadreras
2DEB  FD B6 21             OR   (IY+$21)
2DEE  28 0D                JR   Z,L2DFD
2DF0  CD 21 4C             CALL FDUP
2DF3  CD EA 4F             CALL FMUL
2DF6  08           L2DF6:  EX   AF,AF'
2DF7  CD 52 4C             CALL FSWAP
2DFA  08                   EX   AF,AF'
2DFB  30 D0                JR   NC,L2DCD
2DFD  F9           L2DFD:  LD   SP,HL
2DFE  C3 7B 21     L2DFE:  JP   L217B           CY: fel 130
```

*FPOW: x^y med flyttal. Är y ett heltal med |y| < 16384 (exponenten under $8F) går det till IPOW; annars EXP(y*LOG(x)). LOG-felet för x <= 0 (fel 142 i LOG) blir här fel 130, så (-1)^16383 = -1 men (-1)^16384, (-2)^.5 och 0^.5 ger fel 130. ^ räknas från vänster (2^3^2 = 64) och före unärt minus (-2^2 = -4). In: y vid SP, x under (flyttal; DE efter koden). Ut: x^y vid SP, DE = PRGPTR, nästa kod; CY ger fel 130. Förstör de övriga registren.*

```
2E01  CD A3 1D     FPOW:   CALL S1DA3
2E04  21 00 00             LD   HL,$0000
2E07  39                   ADD  HL,SP
2E08  7E                   LD   A,(HL)
2E09  FE 8F                CP   $8F             |y| >= 16384: via LOG (gränsfall)
2E0B  30 0A                JR   NC,L2E17
2E0D  CD 20 2D             CALL FFRAC           bråkdel: via LOG
2E10  20 05                JR   NZ,L2E17
2E12  CD F7 4C             CALL FTOIT           heltal: IPOW
2E15  18 97                JR   L2DAE

2E17  CD 52 4C     L2E17:  CALL FSWAP           x^y = EXP(y*LOG(x))
2E1A  CD 28 58             CALL LOG
2E1D  38 D7                JR   C,L2DF6         LOG-fel (x <= 0) blir fel 130
2E1F  CD EA 4F             CALL FMUL
2E22  D4 37 59             CALL NC,EXP
2E25  18 D7                JR   L2DFE
```

LINK: länkning (fixup) före körning. Anropas av RUN och när ett direktkommando behöver ett länkat program (IY+$26 bit 0 = länkat). Samma princip som i ABC80-BASIC:en (Finnved 1979): koden som lagras är oberoende av var programmet och variablerna hamnar; först vid RUN byts namn och radnummer ut mot adresser.

Pass 1 (radvis, bara satsen först på raden):

```
DEF FN ($8C)  läggs in i DEF-listan (PRGBEG+$10) med relativa
              länkar; samma namn två gånger -> fel $BC (188),
              som anges på den FÖRSTA DEF:ens rad (S2EDE byter
              DE och HL, och HL pekar då på den funna posten).
              En enradig DEF (uttryck direkt) gås igenom med ESCAN.
FNEND ($8D)   utan DEF -> fel $BD (189)
DATA ($8E)    kedjas i DATA-listan (PRGBEG+$12) för READ
```

Pass 2: PSCAN *går igenom hela programmet och anropar* LNKJP *för varje referens;* LNKTAB *väljer rutin efter referensens sort (se* PSCAN*). Blir det fel görs det redan länkade ogjort fram till felet (*ULKPRT*) och* PRGPTR *pekar på felet. Öppen* FOR/WHILE *vid slutet -> fel* $B6*,* DEF *utan* FNEND *-> fel* $BD *(på* DEF*-raden). Till sist* SET *0,(IY+*$26*). In: programmet vid* PRGBEG*. Ut: NC och A = 0 = länkat; CY = fel, A = felkoden och* PRGPTR *= felets plats. Förstör de övriga registren, även IX.*

```
2E27  CD 26 0E     LINK:   CALL RUNCLR
2E2A  FD CB 23 96          RES  2,(IY+$23)      IY+$23 bit 2 av
2E2E  FD CB 26 46          BIT  0,(IY+$26)      redan länkat: klart
2E32  C0                   RET  NZ
2E33  CD 75 13             CALL NEWSTK
2E36  CD 4B 31             CALL LNKCLR          inget DEF eller block öppet
2E39  CD 44 00             CALL FSTLIN
2E3C  E5                   PUSH HL
2E3D  06 13                LD   B,$13           nollställ programhuvudet PRGBEG+1..+$13
2E3F  2B           L2E3F:  DEC  HL
2E40  77                   LD   (HL),A
2E41  10 FC                DJNZ L2E3F
2E43  E1                   POP  HL
2E44  CD B5 32             CALL PRGCHK
2E47  4E           L2E47:  LD   C,(HL)          radens längd
2E48  0D                   DEC  C
2E49  28 75                JR   Z,L2EC0         längd 1 = programmets slut
2E4B  23                   INC  HL              förbi längd och radnummer
2E4C  23                   INC  HL
2E4D  23                   INC  HL
2E4E  0D                   DEC  C
2E4F  0D                   DEC  C
2E50  06 00                LD   B,$00
2E52  7E                   LD   A,(HL)          satsen först på raden
2E53  FE 8C                CP   $8C
2E55  28 0B                JR   Z,L2E62
2E57  FE 8E                CP   $8E
2E59  28 56                JR   Z,L2EB1
2E5B  FE 8D                CP   $8D
2E5D  28 47                JR   Z,L2EA6
2E5F  09           L2E5F:  ADD  HL,BC
2E60  18 E5                JR   L2E47

2E62  CD 5D 31     L2E62:  CALL CURDEF          DEF FN: redan inne i en DEF?
2E65  20 77                JR   NZ,S2EDE
2E67  C5                   PUSH BC
2E68  E5                   PUSH HL
2E69  23                   INC  HL
2E6A  AF                   XOR  A               nolla länken
2E6B  77                   LD   (HL),A
2E6C  23                   INC  HL
2E6D  77                   LD   (HL),A
2E6E  CD DF 30             CALL S30DF
2E71  CD 59 31             CALL SETDEF          LNKDEF = denna DEF
2E74  C5                   PUSH BC
2E75  23                   INC  HL
2E76  23                   INC  HL
2E77  5E                   LD   E,(HL)
2E78  23                   INC  HL
2E79  19                   ADD  HL,DE
2E7A  19                   ADD  HL,DE
2E7B  7E                   LD   A,(HL)
2E7C  FE C0                CP   $C0             enradig DEF (uttryck)?
2E7E  38 11                JR   C,L2E91
2E80  DD 21 B6 31          LD   IX,L31B6
2E84  CD 7C 35             CALL ESCAN           länka uttrycket direkt
2E87  CD 5D 31             CALL CURDEF
```

```
2E8A  CD 4B 31           CALL LNKCLR          och avsluta DEF:en
2E8D  EB                 EX   DE,HL
2E8E  CD 7C 32           CALL SETREL
2E91  C1           L2E91: POP  BC
2E92  11 10 00           LD   DE,$0010        DEF-listan PRGBEG+$10
2E95  CD 4B 32     L2E95: CALL FNSRC           finns namnet redan?
2E98  3E BC              LD   A,$BC
2E9A  D1                 POP  DE
2E9B  C1                 POP  BC
2E9C  30 40              JR   NC,S2EDE        ja: fel 188 (på den funna DEF:en) (fel)
2E9E  13           L2E9E: INC  DE
2E9F  0B                 DEC  BC
2EA0  CD 7C 32           CALL SETREL          länka in posten sist
2EA3  EB                 EX   DE,HL
2EA4  18 B9              JR   L2E5F

2EA6  CD 5D 31     L2EA6: CALL CURDEF          FNEND: utan DEF fel 189
2EA9  28 33              JR   Z,S2EDE
2EAB  EB                 EX   DE,HL
2EAC  CD 4B 31           CALL LNKCLR
2EAF  18 ED              JR   L2E9E

2EB1  C5           L2EB1: PUSH BC              DATA: länk = 0
2EB2  E5                 PUSH HL
2EB3  23                 INC  HL
2EB4  AF                 XOR  A
2EB5  77                 LD   (HL),A
2EB6  23                 INC  HL
2EB7  77                 LD   (HL),A
2EB8  01 00 00           LD   BC,$0000
2EBB  11 12 00           LD   DE,$0012        DATA-kedjan PRGBEG+$12
2EBE  18 D5              JR   L2E95

2EC0  CD 5D 31     L2EC0: CALL CURDEF          slutet: DEF utan FNEND?
2EC3  20 1A              JR   NZ,L2EDF
2EC5  DD 21 C2 32        LD   IX,LNKJP
2EC9  CD 52 35           CALL PSCAN           pass 2: alla referenser
2ECC  38 21              JR   C,L2EEF         fel i pass 2
2ECE  CD 66 31           CALL OPNBLK          FOR/WHILE öppen: fel 182
2ED1  30 10              JR   NC,L2EE3
2ED3  EB                 EX   DE,HL
2ED4  CD 2F 32           CALL S322F
2ED7  D5           L2ED7: PUSH DE
2ED8  F5                 PUSH AF
2ED9  CD FD 2E           CALL UNLINK          gör länkningen ogjord
2EDC  F1                 POP  AF
2EDD  E1                 POP  HL
2EDE  EB           S2EDE: EX   DE,HL          fel: DE = felets plats
2EDF  37           L2EDF: SCF
2EE0  C3 A3 1D           JP   S1DA3

2EE3  CD 5D 31     L2EE3: CALL CURDEF
2EE6  20 EF              JR   NZ,L2ED7
2EE8  FD CB 26 C6        SET  0,(IY+$26)      programmet är länkat
2EEC  C3 82 0E           JP   L0E82

2EEF  F5           L2EEF: PUSH AF
2EF0  CD DE 2E           CALL S2EDE
2EF3  DD 21 DD 32        LD   IX,ULKPRT       ogjort fram till felet
2EF7  CD 05 2F           CALL UNLNK0
2EFA  F1                 POP  AF
2EFB  37                 SCF
2EFC  C9                 RET
```

UNLINK: gör länkningen ogjord (vid ändring av programmet, CLEAR m.m.): PSCAN med ULKTAB byter tillbaka adresser mot namn/radnummer. In: programmet vid PRGBEG. Ut: NC = hela programmet avlänkat, IY+$26 bit 0 av; LNKBLK och LNKDEF = 0. Förstör alla register utom IX:s nya värde och IY.

```
2EFD  FD CB 26 86  UNLINK: RES  0,(IY+$26)
2F01  DD 21 E6 32          LD   IX,ULKJP      avlänkrutinerna (ULKTAB)
2F05  CD 4B 31     UNLNK0: CALL LNKCLR        från LINK:s felväg med IX = ULKPRT
```

```
2F08  C3 52 35            JP   PSCAN
```

LINK1: *länka bara en rad (direktkommando, från* DOLINE*). LNKONE länkar variabler och* FN*-anrop (bara om programmet är länkat, annars fel* $CE *(206)); SINGLE/DOUBLE lämnas; övriga sorter (hopp,* FOR/NEXT*,* DEF*) ger fel* $D0 *(208). In: HL = raden. Ut: NC = länkad; ett fel går till ERRSP med A = felet och kommer inte tillbaka. GOTO ensamt på direktraden fångas före LINK1 (fel 207), GOSUB och FOR ger fel 208. Förstör alla register utom IY.*

```
2F0B  CD 4B 31     LINK1:  CALL LNKCLR
2F0E  DD 21 C8 32          LD   IX,LNKONE
2F12  CD 55 35             CALL PSCAN1          bara raden från HL
2F15  D0                   RET  NC
2F16  C3 6B 3B             JP   ERRSP
```

LVAR: *sort* $0A*, en variabel i ett uttryck. VARLNK ger referensen i BC, och L306C skriver den i ordet före HL. In: DE = namnet ur koden, HL efter ordet. Ut: NC och A = 0, HL kvar; CY: A = felet från VARLNK. Förstör BC och DE.*

```
2F19  CD 3A 2F     LVAR:   CALL VARLNK
2F1C  D8           L2F1C:  RET  C
2F1D  C3 6C 30             JP   L306C           skriv referensen BC i ordet
```

LCOMMN: COMMON*-satsens kod utförs redan vid länkningen (*RST $20 *vid* SCNPTR*), så att* COMMON*-variablerna får sin plats i* COMMON*-området. Ordet efter referensen (00 00 i den olänkade koden) blir avståndet från variabelns värde till* FREPTR *efteråt, alltså värdet med strängbuffert och fältets element:* COMMON *A,B$=10,C(2) ger 4,* $10 *och* $18*. Ett fel när satsen utförs (CY) avbryter länkningen. In: DE =* FREPTR *+ 4 före satsen (*SCOMMN*), HL efter ordet,* SCNPTR *vid satsens kod. Ut: NC, ordet = avståndet; CY: A = felet från satsen. IX kvar; förstör de övriga registren.*

```
2F20  DD E5        LCOMMN: PUSH IX
2F22  E5                   PUSH HL
2F23  D5                   PUSH DE
2F24  ED 5B 20 FF          LD   DE,(SCNPTR)
2F28  E7                   RST  $20             utför COMMON-koden nu
2F29  C1                   POP  BC
2F2A  38 09                JR   C,L2F35
2F2C  CD CF 0F             CALL S0FCF
2F2F  ED 42                SBC  HL,BC           avstånd från värdet till FREPTR
2F31  3E CD                LD   A,$CD
2F33  44                   LD   B,H
2F34  4D                   LD   C,L
2F35  E1           L2F35:  POP  HL
2F36  DD E1                POP  IX
2F38  18 E2                JR   L2F1C
```

3.22 VARLNK: slå upp (eller skapa) en variabel, DE = namnet $2F3A

Namnet är två bytes: B = bokstaven (1-29), C = siffran * 8 eller $78 utan siffra; de låga bitarna ger sorten: bit 0 heltal (%), bit 1 sträng ($), bit 2 fält. Inuti en DEF FN söks först bland parametrarna (PARSRC); de blir lokala ($FFxx, relativt FNFRM). Annars söks variabellistan (VARSRC, huvud VARLST). Saknas variabeln skapas en post med ALLOC: +0,+1 länk, +2,+3 namn, +4.. värdet (heltal 2 bytes, flyttal 4/8 bytes enligt IY+$1E, sträng 6 bytes, fält se nedan). Referensen som skrivs in i koden är ADRREF av adressen. Fält med annat antal index -> fel $BE. Posten för ett fält är 4 + 6 bytes och 6 bytes per dimension (16 för A(2), 22 för A(2,3); provat med VARPTR mellan två variabler), en heltalspost 6 och en flyttalspost 8 (12 med DOUBLE). In: DE = namnet (D = bokstaven, E = siffra och sort), HL efter referensen i koden, LNKDEF = parameterlistan när länkningen är inne i en DEF. Ut: NC: BC = referensen (för en parameter B = $FF och C = dess nummer x 2), HL kvar; CY: A = felet (190, 191 eller minnet slut). Förstör A och DE.

```
2F3A  E5           VARLNK: PUSH HL
2F3B  EB                   EX   DE,HL
2F3C  CD 5D 31             CALL CURDEF
2F3F  EB                   EX   DE,HL
2F40  28 05                JR   Z,L2F47         inne i en DEF: parametrarna först
2F42  CD 2D 36             CALL PARSRC
2F45  28 5E                JR   Z,L2FA5
2F47  CD 51 32     L2F47:  CALL VARSRC
2F4A  D1           L2F4A:  POP  DE
2F4B  D5                   PUSH DE
```

```
2F4C  1B              DEC  DE          koden före referensen
2F4D  1B              DEC  DE
2F4E  1B              DEC  DE
2F4F  1A              LD   A,(DE)
2F50  CB 7F           BIT  7,A
2F52  28 07           JR   Z,L2F5B
2F54  3E 02           LD   A,$02
2F56  CB 41           BIT  0,C
2F58  28 01           JR   Z,L2F5B
2F5A  3D              DEC  A
2F5B  30 60     L2F5B: JR  NC,L2FBD
2F5D  E5              PUSH HL          ny post: storleken ur sorten
2F5E  5F              LD   E,A
2F5F  87              ADD  A,A
2F60  83              ADD  A,E
2F61  87              ADD  A,A
2F62  C6 06           ADD  A,$06
2F64  16 00           LD   D,$00
2F66  CB 51           BIT  2,C
2F68  20 1D           JR   NZ,L2F87
2F6A  3E 06           LD   A,$06
2F6C  CB 49           BIT  1,C
2F6E  20 17           JR   NZ,L2F87
2F70  3E 02           LD   A,$02
2F72  CB 41           BIT  0,C
2F74  20 11           JR   NZ,L2F87
2F76  FD CB 26 DE     SET  3,(IY+$26)  flyttal: bredden IY+$1E
2F7A  FD 7E 1E        LD   A,(IY+$1E)
2F7D  FD CB 1D 6E     BIT  5,(IY+$1D)
2F81  28 04           JR   Z,L2F87
2F83  FD CB 26 E6     SET  4,(IY+$26)
2F87  C6 04     L2F87: ADD A,$04
2F89  5F              LD   E,A
2F8A  FD CB 1D 6E     BIT  5,(IY+$1D)  COMMON (IY+$1D bit 5)?
2F8E  20 05           JR   NZ,L2F95
2F90  CD D3 0F        CALL ALLOC
2F93  18 03           JR   L2F98

2F95  CD AE 0F  L2F95: CALL ALLOCC
2F98  D1        L2F98: POP  DE
2F99  38 3D           JR   C,L2FD8
2F9B  EB              EX   DE,HL       länka in posten sist i listan
2F9C  73              LD   (HL),E
2F9D  23              INC  HL
2F9E  72              LD   (HL),D
2F9F  EB              EX   DE,HL
2FA0  CD 38 32        CALL S3238
2FA3  18 A5           JR   L2F4A

2FA5  06 FF     L2FA5: LD  B,$FF       parameter: B = $FF, lokal
2FA7  E1              POP  HL
2FA8  CB 79           BIT  7,C         bit 7: LOCAL, får tilldelas och läsas in
2FAA  CB B9           RES  7,C
2FAC  C0              RET  NZ
2FAD  CB 4B           BIT  1,E         talparameter går också
2FAF  C8              RET  Z
2FB0  E5              PUSH HL
2FB1  2B              DEC  HL
2FB2  2B              DEC  HL
2FB3  2B              DEC  HL
2FB4  7E              LD   A,(HL)
2FB5  E1              POP  HL
2FB6  D6 C0           SUB  $C0         sträng: bara värdekod $C0-$C2, annars fel 191 (egenhet)
2FB8  C6 FD           ADD  A,$FD
2FBA  3E BF           LD   A,$BF
2FBC  C9              RET

2FBD  CB 51     L2FBD: BIT 2,C
2FBF  28 12           JR   Z,L2FD3
2FC1  5F              LD   E,A
```

```
2FC2  01 04 00           LD   BC,$0004
2FC5  09                 ADD  HL,BC
2FC6  7E                 LD   A,(HL)          antal index i posten
2FC7  A7                 AND  A
2FC8  28 06              JR   Z,L2FD0
2FCA  BB                 CP   E
2FCB  3E BE              LD   A,$BE           annat antal: fel 190
2FCD  C2 B4 31           JP   NZ,L31B4
2FD0  73           L2FD0: LD   (HL),E
2FD1  ED 42              SBC  HL,BC
2FD3  CD 49 0F     L2FD3: CALL ADRREF
2FD6  4D                 LD   C,L
2FD7  44                 LD   B,H
2FD8  E1           L2FD8: POP  HL
2FD9  C9                 RET

2FDA  FD CB 26 46  L2FDA: BIT  0,(IY+$26)      FN i direktrad kräver länkat program
2FDE  3E CE              LD   A,$CE
2FE0  37                 SCF
2FE1  C8                 RET  Z
```

LFNCAL: FN-*anrop: sök* DEF (FNSRC)*; saknas -> fel* $BB (*187*). *Argumentens antal och typer jämförs med parametrarna (fel* $DF (*223*)*), sedan skrivs relativ adress till* DEF*:en in. Ett tal går till en heltalsparameter och tvärtom: omvandlingskoden läggs in i argumentets slutkod (*FNA(2.7) *med* DEF FNA(X%) *ger 3); tal mot sträng, för få och för många argument ger fel 223. In: DE = namnet, HL efter ordet. Ut: NC: ordet = relativ länk till DEF:en (*SETREL*); CY: A = felet (187, 223). Förstör BC och DE.*

```
2FE2  E5           LFNCAL: PUSH HL
2FE3  42                 LD   B,D
2FE4  4B                 LD   C,E
2FE5  11 10 00           LD   DE,$0010
2FE8  CD 4B 32           CALL FNSRC
2FEB  23                 INC  HL
2FEC  23                 INC  HL
2FED  3E BB              LD   A,$BB           saknas: fel 187
2FEF  DA B4 31           JP   C,L31B4
2FF2  D1                 POP  DE
2FF3  D5                 PUSH DE
2FF4  1B                 DEC  DE
2FF5  1B                 DEC  DE
2FF6  1B                 DEC  DE
2FF7  4E                 LD   C,(HL)          antal parametrar
2FF8  06 00              LD   B,$00
2FFA  09                 ADD  HL,BC
2FFB  09                 ADD  HL,BC
2FFC  EB                 EX   DE,HL
2FFD  7E           L2FFD: LD   A,(HL)
2FFE  A7                 AND  A
2FFF  28 26              JR   Z,L3027
3001  0D                 DEC  C
3002  1B                 DEC  DE
3003  85                 ADD  A,L
3004  6F                 LD   L,A
3005  38 01              JR   C,L3008
3007  25                 DEC  H
3008  2B           L3008: DEC  HL
3009  7E                 LD   A,(HL)          argumentets typ
300A  D6 AD              SUB  $AD
300C  30 02              JR   NC,L3010
300E  C6 03              ADD  A,$03
3010  47           L3010: LD   B,A
3011  1A                 LD   A,(DE)          parameterns typ
3012  E6 03              AND  $03
3014  1B                 DEC  DE
3015  A8                 XOR  B
3016  28 08              JR   Z,L3020
3018  3D                 DEC  A
3019  3E DF              LD   A,$DF           tal/sträng blandat: fel 223
301B  C2 B4 31           JP   NZ,L31B4
301E  3E 03              LD   A,$03
```

```
3020  80          L3020:  ADD  A,B
3021  C6 AA               ADD  A,$AA
3023  77                  LD   (HL),A          omvandlingskod $AA-$AC
3024  23                  INC  HL
3025  18 D6               JR   L2FFD

3027  B1          L3027:  OR   C               argument kvar: fel 223
3028  3E DF               LD   A,$DF
302A  18 26               JR   L3052
```

LLINE: *radnummer DE -> relativ adress till raden* (SETREL). *Saknas raden: fel* $B4 (*180*). *Hopp mellan en* DEF FN *och resten av programmet: fel* $B5 (*181*), *utom för* RESTORE. *Raden med DEF räknas till programmet utanför:* GOTO *till den går* (*och körningen hoppar över funktionen*), GOTO *till en rad inne i funktionen eller ut ur den ger fel 181. In: DE = radnumret, HL efter ordet,* SCNPTR *vid satsens kod. Ut: NC: ordet = relativ länk till byten före raden; CY: A = felet* (*180, 181*). *Förstör BC och DE.*

```
302C  E5          LLINE:  PUSH HL
302D  CD 8C 32            CALL FNDLIN          finns raden?
3030  3E B4      L3030:  LD   A,$B4
3032  C2 B4 31           JP   NZ,L31B4
3035  CD 01 31           CALL INDEF
3038  CD 5D 31           CALL CURDEF          i samma DEF som hoppet?
303B  ED 52               SBC  HL,DE
303D  28 0E               JR   Z,L304D
303F  2A 20 FF            LD   HL,(SCNPTR)
3042  7E          L3042:  LD   A,(HL)
3043  FE 83               CP   $83             RESTORE får gå ut ur DEF
3045  28 06               JR   Z,L304D
3047  23                  INC  HL
3048  23                  INC  HL
3049  FE BF               CP   $BF
304B  28 F5               JR   Z,L3042
304D  3E B5       L304D:  LD   A,$B5           annars fel 181
304F  50                  LD   D,B
3050  59                  LD   E,C
3051  1B          L3051:  DEC  DE
3052  E1          L3052:  POP  HL
3053  37                  SCF
3054  C0                  RET  NZ
3055  2B                  DEC  HL
3056  2B                  DEC  HL
3057  C3 7C 32            JP   SETREL
```

RNLINE (*RENUMBER,* $42E7): *som* LLINE *för sort 9 men utan prövningen av DEF: radnumret blir en relativ länk. Andra sorter lämnas* (*NC*). *Saknas raden: CY och A = 180.*

```
305A  D6 09       RNLINE: SUB  $09
305C  A7                  AND  A
305D  C0                  RET  NZ
305E  E5                  PUSH HL
305F  CD 8C 32            CALL FNDLIN
3062  20 CC               JR   NZ,L3030
3064  EB                  EX   DE,HL
3065  18 EA               JR   L3051
```

ULREF: *avlänka sort 7* (*NEXT*), *9* (*radnummer*), $0A (*variabel*) *och* $0B (*FN-anrop*): ULVAL *tar fram det ursprungliga ordet, och det skrivs tillbaka i koden. In: A = sorten, DE = det länkade ordet, HL efter ordet. Ut: NC och A = 0, HL kvar. Förstör BC.*

```
3067  42          ULREF:  LD   B,D             BC = det länkade ordet
3068  4B                  LD   C,E
3069  CD 7A 30            CALL S307A
306C  2B          L306C:  DEC  HL
306D  2B                  DEC  HL
306E  AF                  XOR  A
306F  C3 3A 32            JP   STBCHL
```

S3072 (*LIST,* $41A7): *BC = ordet vid HL, HL efter. Är programmet länkat* (*IY+*$26 *bit 0*) *blir ordet det ursprungliga, som i* ULREF (*A = sorten*).

```
3072  CD E0 30     S3072:  CALL LDBCHL
```

```
3075  FD CB 26 46          BIT  0,(IY+$26)      olänkat: ordet är redan numret eller namnet
3079  C8                   RET  Z
307A  E5             S307A:  PUSH HL
307B  CD 80 30               CALL ULVAL
307E  E1                     POP  HL
307F  C9                     RET
```

ULVAL: det ursprungliga ordet ur en länk. In: A = sorten, BC = länken, HL efter ordet. Ut: BC = ordet. Radnummer (9): länken är relativ, och numret läses ur målraden. FN-anrop ($0B): namnet läses ur DEF:en. Variabel ($0A): länken är en referens (REFADR) och namnet står före värdet; en lokal variabel (B = $FF) slås upp i den aktuella DEF:ens parameterlista. NEXT (7): länken gäller FOR-blocket, och styrvariabeln läses därifrån. Förstör A, DE och HL.

```
3080  FE 0A          ULVAL:  CP   $0A             sort $0A: variabel
3082  28 30                  JR   Z,L30B4
3084  FE 07                  CP   $07             sort 7: NEXT
3086  28 0D                  JR   Z,L3095
3088  09                     ADD  HL,BC           relativ länk: målet
3089  EE 0B                  XOR  $0B             sort $0B: FN-anrop, namnet före målet
308B  20 52                  JR   NZ,S30DF        annars numret efter målradens längdbyte
308D  2B                     DEC  HL
308E  2B               S308E:  DEC  HL
308F  2B                     DEC  HL
3090  2B               S3090:  DEC  HL
3091  46                     LD   B,(HL)
3092  2B                     DEC  HL
3093  4E                     LD   C,(HL)
3094  C9                     RET

3095  CD 68 0F       L3095:  CALL REFADR          NEXT: FOR-blocket
3098  23                     INC  HL
3099  23                     INC  HL
309A  E5                     PUSH HL
309B  DD E5                  PUSH IX
309D  D5                     PUSH DE
309E  DD 21 E5 30            LD   IX,L30E5
30A2  7E                     LD   A,(HL)
30A3  23                     INC  HL
30A4  B6                     OR   (HL)
30A5  CC 52 35               CALL Z,PSCAN
30A8  D1                     POP  DE
30A9  DD E1                  POP  IX
30AB  E1                     POP  HL
30AC  4E                     LD   C,(HL)
30AD  23                     INC  HL
30AE  46                     LD   B,(HL)
30AF  FD CB 26 46            BIT  0,(IY+$26)
30B3  C8                     RET  Z
30B4  04               L30B4:  INC  B               B = $FF: lokal variabel
30B5  28 06                  JR   Z,L30BD
30B7  05                     DEC  B
30B8  CD 68 0F               CALL REFADR          variabelns post: namnet före värdet
30BB  18 D3                  JR   S3090

30BD  EB               L30BD:  EX   DE,HL           lokal: sök i DEF:ens parameterlista
30BE  CD 5D 31               CALL CURDEF
30C1  EB                     EX   DE,HL
30C2  7E                     LD   A,(HL)
30C3  3C                     INC  A
30C4  CB 19                  RR   C
30C6  0C                     INC  C
30C7  3D               L30C7:  DEC  A
30C8  28 0A                  JR   Z,L30D4
30CA  23                     INC  HL
30CB  0D                     DEC  C
30CC  28 12                  JR   Z,LDBCHL
30CE  CB 4E                  BIT  1,(HL)
30D0  23                     INC  HL
30D1  28 F4                  JR   Z,L30C7
30D3  3C                     INC  A
```

```
30D4  3D          L30D4:  DEC  A
30D5  F2 C7 30            JP   P,L30C7
30D8  23                  INC  HL
30D9  23                  INC  HL
30DA  18 EB               JR   L30C7
```

Hjälprutiner: `LDBCHL` *läser BC ur ordet vid HL och flyttar HL förbi det;* `S30DF`, `S30DE` *och* `S30DC` *hoppar först över 1, 2 resp. 4 bytes.* `S3090` *läser BC ur de två byten före HL (HL - 2),* `S308E` *ur de två före dem.*

```
30DC  23          S30DC:  INC  HL
30DD  23                  INC  HL
30DE  23          S30DE:  INC  HL
30DF  23          S30DF:  INC  HL
30E0  4E          LDBCHL: LD   C,(HL)
30E1  23                  INC  HL
30E2  46                  LD   B,(HL)
30E3  23                  INC  HL
30E4  C9                  RET

30E5  EE 05       L30E5:  XOR  $05           bara FOR (sort 5)
30E7  C0                  RET  NZ
```

`UFOR`*: avlänka FOR. Ordet efter FOR är länken till FOR-blocket (0 = ingen, inget att göra); styrvariabelns ord (DE, redan avlänkat) skrivs i blockets nyckel, så att NEXT kan avlänkas efter den (*`ULVAL`*). In: DE = ordet, HL efter länken. Ut: HL kvar. Förstör A och BC.*

```
30E8  D5          UFOR:   PUSH DE
30E9  2B                  DEC  HL
30EA  2B                  DEC  HL
30EB  CD 33 32            CALL LDDEHL
30EE  7B                  LD   A,E
30EF  B2                  OR   D
30F0  CA B7 31            JP   Z,L31B7     länk 0: inget att göra
30F3  EB                  EX   DE,HL
30F4  19                  ADD  HL,DE
30F5  46                  LD   B,(HL)
30F6  2B                  DEC  HL
30F7  4E                  LD   C,(HL)
30F8  CD 68 0F            CALL REFADR
30FB  23                  INC  HL
30FC  23                  INC  HL
30FD  C1                  POP  BC
30FE  C3 AF 31            JP   L31AF
```

`INDEF`*: ligger adressen HL inne i en DEF FN? DEF-listan (*`PRGBEG` + `$10`*, relativa länkar i adressordning) gås igenom. Ut: NZ: inne, HL = DEF:ens parameterlista (samma värde som* `LNKDEF` *under länkningen) och DE = DEF:ens plats i listan; Z: utanför, HL = 0. Raden med DEF ligger utanför, eftersom listan pekar in i raden. Förstör A och BC.*

```
3101  44          INDEF:  LD   B,H           ligger HL i en DEF FN?
3102  4D                  LD   C,L
3103  11 10 00            LD   DE,$0010
3106  CD 3B 00            CALL S003B
3109  19                  ADD  HL,DE
310A  E5                  PUSH HL
310B  E1          L310B:  POP  HL            gå igenom DEF-listan
310C  5E                  LD   E,(HL)
310D  23                  INC  HL
310E  56                  LD   D,(HL)
310F  7A                  LD   A,D
3110  B3                  OR   E
3111  EB                  EX   DE,HL
3112  C8                  RET  Z
3113  19                  ADD  HL,DE
3114  ED 42               SBC  HL,BC
3116  09                  ADD  HL,BC
3117  30 0D               JR   NC,L3126
3119  E5                  PUSH HL
311A  CD 2F 32            CALL S322F
311D  EB                  EX   DE,HL
```

```
311E  19                ADD  HL,DE
311F  ED 42             SBC  HL,BC
3121  38 E8             JR   C,L310B        HL innanför en DEF?
3123  E1                POP  HL
3124  EB                EX   DE,HL
3125  C0                RET  NZ
3126  21 00 00  L3126:  LD   HL,$0000
3129  AF         NOLINK: XOR  A              inget att länka
312A  C9                RET
```

LDEF: DEF FN. *Öppen FOR/WHILE -> fel* $B6 *(182), DEF i DEF -> fel* $BD *(189). LNKDEF = DEF:en; en enradig DEF avslutas direkt (LFNEND). LFNEND (FNEND, och slutet på en enradig DEF) kräver en öppen DEF och ingen öppen FOR/WHILE (fel 189, 182) och nollar LNKBLK och LNKDEF (LNKCLR). Samma rutiner går också vid avlänkningen (ULKTAB), så att lokala variabler kan avlänkas. In: DE = parameterlistan (koden + 7), HL efter listan och LOCAL-listan (SDEF). Ut: NC; CY: A = felet. Förstör A och DE.*

```
312B  D5           LDEF:   PUSH DE
312C  CD 66 31             CALL OPNBLK         DEF i FOR/WHILE: fel 182
312F  D4 5D 31             CALL NC,CURDEF
3132  D1                   POP  DE
3133  D8                   RET  C
3134  37                   SCF
3135  C0                   RET  NZ
3136  EB                   EX   DE,HL
3137  CD 59 31             CALL SETDEF
313A  EB                   EX   DE,HL
313B  7E                   LD   A,(HL)         kod $C0-: enradig DEF
313C  C6 40                ADD  A,$40
313E  D0                   RET  NC
313F  CD 7C 35             CALL ESCAN          enradig: uttrycket, sedan som FNEND
3142  D4 66 31    LFNEND:  CALL NC,OPNBLK      FNEND: ingen FOR/WHILE öppen, en DEF öppen
3145  D4 5D 31             CALL NC,CURDEF
3148  D8                   RET  C
3149  37                   SCF
314A  C8                   RET  Z
314B  AF           LNKCLR: XOR  A              LNKBLK och LNKDEF = 0
314C  FD 77 2A             LD   (IY+$2A),A
314F  FD 77 2B             LD   (IY+$2B),A
3152  FD 77 28             LD   (IY+$28),A
3155  FD 77 29             LD   (IY+$29),A
3158  C9                   RET
```

SETDEF: LNKDEF = *HL.* CURDEF: *DE =* LNKDEF*, Z = ingen DEF öppen, A =* $BD *(189).* OPNBLK: *DE =* LNKBLK*; Z och NC = inget block öppet, annars CY och A =* $B6 *(182).*

```
3159  22 28 FF  SETDEF: LD   (LNKDEF),HL
315C  C9                RET

315D  ED 5B 28 FF CURDEF: LD   DE,(LNKDEF)
3161  7B                 LD   A,E
3162  B2                 OR   D
3163  3E BD               LD   A,$BD
3165  C9                 RET

3166  ED 5B 2A FF OPNBLK: LD   DE,(LNKBLK)
316A  7A                 LD   A,D
316B  B3                 OR   E
316C  C8                 RET  Z
316D  3E B6              LD   A,$B6
316F  37                 SCF
3170  C9                 RET
```

LFOR: *länka* FOR*. Samma styrvariabel redan öppen -> fel* $B9 *(185). Ett* FOR*-block (11 bytes för heltal, 7 + 2 * IY+*$1E *för flyttal) tas från minnet och läggs överst i* LNKBLK*-kedjan;* NEXT *kopplas till det. In: DE = styrvariabelns referens, HL efter FOR-satsens ord. Ut: NC: blocket överst i* LNKBLK *med styrvariabeln som nyckel, ordet efter FOR nollat (NEXT fyller i det); CY: A = felet (185 eller minnet slut). Förstör A, BC och DE.*

```
3171  E5           LFOR:   PUSH HL
3172  42                   LD   B,D
```

```
3173  4B                LD   C,E
3174  11 2A 00          LD   DE,$002A
3177  CD 56 32          CALL LSTSRC          styrvariabeln redan öppen?
317A  3E B9             LD   A,$B9
317C  30 36             JR   NC,L31B4
317E  CD 68 0F          CALL REFADR
3181  2B                DEC  HL
3182  2B                DEC  HL
3183  CB 46             BIT  0,(HL)          bit 0 i namnet: heltal
3185  11 0B 00          LD   DE,$000B        11 bytes för heltal
3188  20 07             JR   NZ,L3191
318A  FD 7E 1E          LD   A,(IY+$1E)
318D  87                ADD  A,A
318E  C6 07             ADD  A,$07           7 + 2 x bredden för flyttal
3190  5F                LD   E,A
3191  CD CC 31    L3191: CALL BLKPSH
3194  38 1F             JR   C,L31B5
3196  EB                EX   DE,HL
3197  E1                POP  HL              länken efter FOR = 0 tills NEXT
3198  2B                DEC  HL
3199  2B                DEC  HL
319A  AF                XOR  A
319B  77                LD   (HL),A
319C  23                INC  HL
319D  77                LD   (HL),A
319E  23                INC  HL
319F  EB                EX   DE,HL
31A0  CD 3F 32    L31A0: CALL STDEHL
31A3  E5                PUSH HL
31A4  CD 49 0F          CALL ADRREF
31A7  01 06 00          LD   BC,$0006
31AA  ED 42             SBC  HL,BC
31AC  4D                LD   C,L
31AD  44                LD   B,H
31AE  E1                POP  HL
31AF  71          L31AF: LD   (HL),C
31B0  23                INC  HL
31B1  70                LD   (HL),B
31B2  EB                EX   DE,HL
31B3  C9                RET

31B4  37          L31B4: SCF
31B5  E1          L31B5: POP  HL
31B6  C9          L31B6: RET

31B7  C1          L31B7: POP  BC
31B8  C9                 RET
```

LWHILE: *som* FOR, *men nyckeln är* $FFFF *och blocket 7 bytes.* LWEND (*WEND*) *går direkt till* BLKPOP *med nyckeln* $FFFF. *Ut: som* LFOR.

```
31B9  E5          LWHILE: PUSH HL
31BA  11 07 00            LD   DE,$0007
31BD  01 FF FF            LD   BC,$FFFF        WHILE: nyckel $FFFF
31C0  CD CC 31            CALL BLKPSH
31C3  D1                  POP  DE
31C4  30 DA               JR   NC,L31A0
31C6  C9                  RET

31C7  01 FF FF    LWEND:  LD   BC,$FFFF
31CA  18 1A               JR   BLKPOP
```

BLKPSH: *ta DE bytes* (ALLOC) *till ett block och lägg det överst i* LNKBLK: *+0 länken till förra blocket, +2 nyckeln BC. Ut: NC, HL efter nyckeln; CY = minnet slut.*

```
31CC  CD D3 0F    BLKPSH: CALL ALLOC
31CF  D8                  RET  C
31D0  ED 5B 2A FF         LD   DE,(LNKBLK)
31D4  22 2A FF            LD   (LNKBLK),HL     blocket överst
31D7  CD 3F 32            CALL STDEHL
```

```
31DA  18 5E              JR   STBCHL
```

LNEXT: *NEXT. Styrvariabeln länkas (VARLNK); en lokal variabel i en DEF får inte vara styrvariabel (fel 186). Sedan BLKPOP med variabelns referens i BC.*

```
31DC  CD 3A 2F     LNEXT: CALL VARLNK          NEXT: länka styrvariabeln
31DF  D8                  RET  C
31E0  3E BA               LD   A,$BA           lokal styrvariabel (B = $FF): fel 186 (egenhet)
31E2  04                  INC  B
31E3  37                  SCF
31E4  C8                  RET  Z
31E5  05                  DEC  B
```

BLKPOP: *NEXT och WEND tar översta blocket i LNKBLK. Tom kedja: fel $B7 (183); annan sorts block eller annan variabel: fel $B8 (184). Blockets länk till förra blocket blir LNKBLK igen (kedjans slut prövas bara på den höga byten, som nollas i blocket). NEXT måste ha en variabel:* NEXT *ensamt ger fel 234 redan när raden skrivs in. Avståndet till blocket skrivs in efter NEXT/WEND, för WEND också länken tillbaka till WHILE. In: BC = nyckeln (styrvariabelns referens,* $FFFF *för WEND), DE = ordet efter koden, HL efter ordet. Ut: NC; CY: A = felet. Förstör A, BC och DE.*

```
31E6  ED 5B 2A FF  BLKPOP: LD   DE,(LNKBLK)
31EA  7A                  LD   A,D
31EB  A7                  AND  A
31EC  3E B7               LD   A,$B7           inget block: fel 183
31EE  37                  SCF
31EF  C8                  RET  Z
31F0  1A                  LD   A,(DE)
31F1  FD 77 2A            LD   (IY+$2A),A
31F4  13                  INC  DE
31F5  1A                  LD   A,(DE)
31F6  FD 77 2B            LD   (IY+$2B),A
31F9  AF                  XOR  A
31FA  12                  LD   (DE),A
31FB  13                  INC  DE
31FC  1A                  LD   A,(DE)
31FD  13                  INC  DE
31FE  B9                  CP   C               rätt variabel?
31FF  20 03               JR   NZ,L3204
3201  1A                  LD   A,(DE)
3202  13                  INC  DE
3203  B8                  CP   B
3204  3E B8        L3204: LD   A,$B8           annars fel 184
3206  37                  SCF
3207  C0                  RET  NZ
3208  78                  LD   A,B             WEND: B = C = $FF, A = $FF
3209  A1                  AND  C
320A  E5                  PUSH HL
320B  EB                  EX   DE,HL
320C  CD 33 32            CALL LDDEHL
320F  4E                  LD   C,(HL)
3210  23                  INC  HL
3211  46                  LD   B,(HL)
3212  E1                  POP  HL
3213  3C                  INC  A               WEND: länk tillbaka till WHILE
3214  20 0A               JR   NZ,L3220
3216  D5                  PUSH DE
3217  EB                  EX   DE,HL
3218  37                  SCF
3219  ED 52               SBC  HL,DE
321B  2B                  DEC  HL
321C  44                  LD   B,H
321D  4D                  LD   C,L
321E  EB                  EX   DE,HL
321F  D1                  POP  DE
3220  2B          L3220: DEC  HL
3221  2B                  DEC  HL
3222  1B                  DEC  DE
3223  A7                  AND  A
3224  ED 52               SBC  HL,DE
```

```
3226  EB                EX   DE,HL
3227  72                LD   (HL),D
3228  2B                DEC  HL
3229  73                LD   (HL),E
322A  23                INC  HL
322B  EB                EX   DE,HL
322C  19                ADD  HL,DE
322D  18 0B             JR   STBCHL
```

Hjälprutiner: LDDEHL *läser DE ur ordet vid HL och flyttar HL förbi det (*S322F *och* S3232 *hoppar först över 4 resp. 1 byte);* STBCHL *och* STDEHL *skriver BC resp. DE på samma sätt (*S3238 *hoppar först över 2 bytes);* S3246 *läser DE ur de två byten före HL (HL - 2),* S3244 *ur de två före dem.*

```
322F  23          S322F:  INC  HL
3230  23                  INC  HL
3231  23                  INC  HL
3232  23          S3232:  INC  HL
3233  5E          LDDEHL: LD   E,(HL)
3234  23                  INC  HL
3235  56                  LD   D,(HL)
3236  23                  INC  HL
3237  C9                  RET

3238  23          S3238:  INC  HL
3239  23                  INC  HL
323A  71          STBCHL: LD   (HL),C
323B  23                  INC  HL
323C  70                  LD   (HL),B
323D  23                  INC  HL
323E  C9                  RET

323F  73          STDEHL: LD   (HL),E
3240  23                  INC  HL
3241  72                  LD   (HL),D
3242  23                  INC  HL
3243  C9                  RET

3244  2B          S3244:  DEC  HL
3245  2B                  DEC  HL
3246  2B          S3246:  DEC  HL
3247  56                  LD   D,(HL)
3248  2B                  DEC  HL
3249  5E                  LD   E,(HL)
324A  C9                  RET

324B  CD 3B 00    FNSRC:  CALL S003B
324E  37                  SCF
324F  18 09               JR   L325A

3251  4B          VARSRC: LD   C,E
3252  42                  LD   B,D
3253  11 2C 00            LD   DE,$002C
```

LSTSRC*: sök i en länkad lista med huvud vid IY+DE efter namnet BC.*

Z = hittad, HL efter namnet; CY = slut. FNSRC *söker i* DEF*-listan vid* PRGBEG+$10*, där länkarna är relativa (CY' i AF').* VARSRC*: namnet i DE, variabellistans huvud vid IY+*$2C*. Förstör A och DE.*

```
3256  AF          LSTSRC: XOR  A
3257  FD E5               PUSH IY
3259  E1                  POP  HL
325A  08          L325A:  EX   AF,AF'
325B  19                  ADD  HL,DE
325C  23                  INC  HL
325D  56          L325D:  LD   D,(HL)          länken
325E  2B                  DEC  HL
325F  5E                  LD   E,(HL)
3260  7B                  LD   A,E
3261  B2                  OR   D
3262  37                  SCF                  listans slut: CY
3263  C8                  RET  Z
```

```
3264  23                INC  HL
3265  EB                EX   DE,HL
3266  08                EX   AF,AF'
3267  30 02             JR   NC,L326B        FNSRC (CY'): relativ länk
3269  19                ADD  HL,DE
326A  37                SCF
326B  08          L326B:  EX   AF,AF'
326C  23                INC  HL
326D  23                INC  HL
326E  79                LD   A,C             jämför namnet
326F  BE                CP   (HL)
3270  20 07             JR   NZ,L3279
3272  23                INC  HL
3273  78                LD   A,B
3274  BE                CP   (HL)
3275  23                INC  HL
3276  C8                RET  Z               hittad: Z, NC
3277  2B                DEC  HL
3278  2B                DEC  HL
3279  2B          L3279:  DEC  HL
327A  18 E1             JR   L325D
```

SETREL: *skriv DE - (HL + 1) i ordet vid HL (relativ länk, räknad från ordets höga byte). Ut: HL efter ordet, DE = målet, NC. Förstör A.*

```
327C  23          SETREL: INC  HL
327D  EB                EX   DE,HL
327E  A7                AND  A
327F  ED 52             SBC  HL,DE           mål - position
3281  1B                DEC  DE
3282  7D                LD   A,L
3283  12                LD   (DE),A
3284  13                INC  DE
3285  7C                LD   A,H
3286  12                LD   (DE),A
3287  19                ADD  HL,DE
3288  EB                EX   DE,HL
3289  23                INC  HL
328A  A7          L328A:  AND  A
328B  C9                RET
```

FNDLIN: *sök rad nummer DE. Z = hittad (HL = raden), annars HL = första raden med högre nummer eller slutraden. DE ut = numret på raden före (0 = ingen). In: DE = numret. BC = det sökta numret efteråt. Förstör A.*

```
328C  CD B5 32    FNDLIN: CALL PRGCHK
328F  42                LD   B,D
3290  4B                LD   C,E
3291  11 00 00          LD   DE,$0000
3294  D5                PUSH DE              föregående rad = 0
3295  7E          L3295:  LD   A,(HL)
3296  FE 01             CP   $01             längd 1 = slutraden: inte hittad
3298  28 18             JR   Z,L32B2
329A  23                INC  HL
329B  5E                LD   E,(HL)
329C  23                INC  HL
329D  56                LD   D,(HL)
329E  2B                DEC  HL
329F  2B                DEC  HL
32A0  EB                EX   DE,HL
32A1  A7                AND  A
32A2  ED 42             SBC  HL,BC
32A4  EB                EX   DE,HL
32A5  28 0C             JR   Z,L32B3         hittad: Z
32A7  30 09             JR   NC,L32B2        större nummer: inte hittad
32A9  EB                EX   DE,HL
32AA  09                ADD  HL,BC
32AB  E3                EX   (SP),HL         radnumret sparas som föregående
32AC  6F                LD   L,A
```

```
32AD  26 00              LD   H,$00
32AF  19                 ADD  HL,DE
32B0  18 E3              JR   L3295

32B2  B7           L32B2: OR   A
32B3  D1           L32B3: POP  DE              DE = föregående radnummer (0 = ingen)
32B4  C9                 RET
```

PRGCHK: *HL = första raden. Står inte* $87 *före den är programmet förstört: fel 202* ($CA).

```
32B5  CD 44 00     PRGCHK: CALL FSTLIN
32B8  7E                 LD   A,(HL)
32B9  FE 87              CP   $87
32BB  3E CA              LD   A,$CA             fel 202: programmet slutar inte med $87
32BD  C2 6B 3B           JP   NZ,ERRSP
32C0  23                 INC  HL
32C1  C9                 RET
```

LNKJP *och* ULKJP *väljer rutin ur* LNKTAB *resp.* ULKTAB *efter sorten i A* (JPTAB). LNKONE (LINK1, *direktraden*): *variabler som vid RUN, FN-anrop bara när programmet är länkat (fel 206), SINGLE och DOUBLE (sort 2, 3) lämnas, allt annat ger fel 208.* ULKPRT *avlänkar bara fram till* PRGPTR *(felets plats efter ett fel i pass 2): där avbryter CY genomgången.* RNUNLK (*RENUMBER*) *gör samma sak men bara för radnummer (sort 9).*

```
32C2  01 05 33     LNKJP:  LD   BC,$3305       länkrutinerna (LNKTAB)
32C5  C3 B7 13     L32C5:  JP   JPTAB

32C8  FE 0A        LNKONE: CP   $0A
32CA  28 F6                JR   Z,LNKJP        variabler: som vid RUN
32CC  FE 0B                CP   $0B
32CE  CA DA 2F             JP   Z,L2FDA        FN-anrop: kräver länkat program
32D1  D6 02                SUB  $02
32D3  FE 03                CP   $03            SINGLE/DOUBLE (sort 2-4) godtas
32D5  3E D0                LD   A,$D0          allt annat: fel 208
32D7  3F                   CCF
32D8  C9                   RET

32D9  FE 09        RNUNLK: CP   $09
32DB  20 AD                JR   NZ,L328A
32DD  ED 4B 3A FF  ULKPRT: LD   BC,(PRGPTR)    bara fram till felet (PRGPTR)
32E1  ED 42                SBC  HL,BC
32E3  09                   ADD  HL,BC
32E4  3F                   CCF
32E5  D8                   RET  C
32E6  01 EB 32     ULKJP:  LD   BC,$32EB       avlänkrutinerna (ULKTAB)
32E9  18 DA                JR   L32C5
```

ULKTAB: *avlänkrutin för varje sort* (NOLINK = *inget att göra*).

```
32EB  2B 31        ULKTAB: DEFW LDEF
32ED  42 31                DEFW LFNEND
32EF  29 31                DEFW NOLINK
32F1  29 31                DEFW NOLINK
32F3  29 31                DEFW NOLINK
32F5  E8 30                DEFW UFOR
32F7  29 31                DEFW NOLINK
32F9  67 30                DEFW ULREF
32FB  29 31                DEFW NOLINK
32FD  67 30                DEFW ULREF
32FF  67 30                DEFW ULREF
3301  67 30                DEFW ULREF
3303  29 31                DEFW NOLINK
```

LNKTAB: *länkrutin för varje sort från* PSCAN (*0-*$0C).

```
3305  2B 31        LNKTAB: DEFW LDEF
3307  42 31                DEFW LFNEND
3309  73 3A                DEFW SINGLE
330B  76 3A                DEFW DOUBLE
330D  36 00                DEFW UNSDEF
330F  71 31                DEFW LFOR
```

```
3311  B9 31              DEFW LWHILE
3313  DC 31              DEFW LNEXT
3315  C7 31              DEFW LWEND
3317  2C 30              DEFW LLINE
3319  19 2F              DEFW LVAR
331B  E2 2F              DEFW LFNCAL
331D  20 2F              DEFW LCOMMN
```

Variabelnamn. Ett namn kodas i två bytes B,C. Korta namn (SHNAME): B = bokstavens nummer + 1 (A = 1 ... Z = 26, Ä Ö Å = 27-29; andra tecken fel 234), C = siffran x 8 eller 15 x 8 utan siffra, plus typen: 0 flyttal, 1 heltal (%), 2 sträng ($). A = $01,$78, B9% = $02,$49, C$ = $03,$7A. Utan typtecken blir namnet heltal i INTEGER-läget (IY+$1D bit 0); . gör det till flyttal ändå. Blanka mellan bokstav och siffra hoppas över (A 1 = A1). Fler bokstäver är fel utan EXTEND: AB=3 och TOP=3 ger fel 220.

Långa namn (VARNAM, bara med EXTEND, IY+$1D bit 1): följer ännu en bokstav eller siffra efter bokstav och siffra är namnet långt. Det söks i namnlistan (NAMSRC). Ett nytt namn får en egen bit i variabelområdet (ALLOC): länk, antal bytes, $FE, namnet med första bokstaven versal och resten gemener, $FF. Koden blir ($0400 + namnets avstånd från listans början) x 8 + typen; den första bitens namn Alpha får $2000, nästa namn Be $2030 och ALPHA% $2001. Versaler och gemener är alltså samma namn, och siffror får stå inne i namnet (A1B). Med EXTEND är blanka betydelsefulla (NXTCHR): PRINT A 1 är två led, och ett nyckelord känns bara igen om det inte följs av en bokstav eller siffra (KWSRCH): PRINTALPHA ger fel 220, FORI=1TO2 fel 221.

VNAME prövar först att namnet inte är en funktion (S199E): SIN=1 och SIN1=2 ger fel 234, SINUS går med EXTEND.

NAMTXT skriver koden som text för LIST: korta namn som bokstav, siffra och typtecken, i INTEGER-läget heltal utan % och flyttal med .; långa namn ur listan. Då slås EXTEND också på, så att ett program med långa namn kan matas in igen efter LIST fast NO EXTEND har getts. SHNAME: In: HL vid bokstaven. Ut: NC: B = bokstaven (1-29), C = siffran x 8 eller $78, HL efter (NXTCHR); CY: A = 234. VARNAM: In: HL = texten. Ut: NC: BC = namnets kod med typen, HL efter namnet och typtecknet; CY: A = felet (234, eller minnet slut för ett nytt långt namn). DE kvar; förstör A och de alternativa registren. NAMTXT/NAMTX1: In: HL vid koden (NAMTXT) eller BC = koden (NAMTX1), DE i textbufferten. Ut: DE efter texten, HL och BC kvar, A = 0.

```
331F  7E           SHNAME: LD   A,(HL)
3320  E6 DF                 AND  $DF
3322  D6 41                 SUB  $41            bokstavens nummer, Ä Ö Å = $1A-$1C
3324  47                   LD   B,A
3325  FE 1D                CP   $1D
3327  3F           L3327:  CCF
3328  3E EA                LD   A,$EA          fel 234: bokstav väntas
332A  D8                   RET  C
332B  04                   INC  B              B = 1..29
332C  CD 10 4C             CALL NXTCHR
332F  0E 78                LD   C,$78          ingen siffra: 15 x 8
3331  7E                   LD   A,(HL)
3332  D6 30                SUB  $30
3334  FE 0A                CP   $0A
3336  D0                   RET  NC
3337  87                   ADD  A,A            siffran x 8
3338  87                   ADD  A,A
3339  87                   ADD  A,A
333A  4F                   LD   C,A
333B  C3 10 4C             JP   NXTCHR

333E  CD 9E 19     VNAME:  CALL S199E
3341  30 E4                JR   NC,L3327       funktionsnamn: fel 234
3343  D5           VARNAM: PUSH DE
3344  E5                   PUSH HL
3345  CD 1F 33             CALL SHNAME
3348  38 73                JR   C,L33BD
334A  FD CB 1D 4E          BIT  1,(IY+$1D)
334E  28 51                JR   Z,L33A1        utan EXTEND: kort namn
3350  7E                   LD   A,(HL)
3351  CD AF 4B             CALL ALNUM
3354  38 4B                JR   C,L33A1        inget tecken till: kort namn
3356  E1                   POP  HL
3357  01 00 04             LD   BC,$0400
335A  CD 57 34             CALL NAMSRC
335D  D9                   EXX
335E  C5                   PUSH BC             namnets nummer
335F  38 38                JR   C,L3399        funnet
3361  5D                   LD   E,L
3362  54                   LD   D,H
3363  7E           L3363:  LD   A,(HL)         namnets längd
```

```
3364  CD AF 4B            CALL ALNUM
3367  23                  INC  HL
3368  30 F9               JR   NC,L3363
336A  ED 52               SBC  HL,DE
336C  45                  LD   B,L
336D  7D                  LD   A,L
336E  C6 06               ADD  A,$06
3370  6F                  LD   L,A
3371  EB                  EX   DE,HL
3372  E5                  PUSH HL
3373  D9                  EXX
3374  E5                  PUSH HL
3375  D9                  EXX
3376  CD D3 0F            CALL ALLOC          ny bit i variabelområdet
3379  EB                  EX   DE,HL
337A  E1                  POP  HL
337B  38 3F               JR   C,L33BC
337D  72                  LD   (HL),D         kedja in den
337E  2B                  DEC  HL
337F  73                  LD   (HL),E
3380  EB                  EX   DE,HL
3381  23                  INC  HL
3382  23                  INC  HL
3383  70                  LD   (HL),B
3384  34                  INC  (HL)           antal = längden + 1
3385  D1                  POP  DE
3386  23                  INC  HL
3387  23                  INC  HL
3388  36 FE               LD   (HL),$FE
338A  23                  INC  HL
338B  1A                  LD   A,(DE)
338C  E6 5F               AND  $5F            första bokstaven versal
338E  77          L338E:  LD   (HL),A
338F  23                  INC  HL
3390  13                  INC  DE
3391  1A                  LD   A,(DE)
3392  F6 20               OR   $20            resten gemener
3394  10 F8               DJNZ L338E
3396  36 FF               LD   (HL),$FF       listans slut
3398  EB                  EX   DE,HL
3399  E3          L3399:  EX   (SP),HL
339A  29                  ADD  HL,HL          nummer x 8
339B  29                  ADD  HL,HL
339C  29                  ADD  HL,HL
339D  4D                  LD   C,L
339E  44                  LD   B,H
339F  E1                  POP  HL
33A0  E5                  PUSH HL
33A1  F1          L33A1:  POP  AF
33A2  D1                  POP  DE
33A3  7E                  LD   A,(HL)
33A4  FE 24               CP   $24            typtecknet
33A6  28 10               JR   Z,L33B8
33A8  FE 2E               CP   $2E
33AA  28 0E               JR   Z,L33BA
33AC  EE 25               XOR  $25
33AE  28 09               JR   Z,L33B9
33B0  2B                  DEC  HL
33B1  FD CB 1D 46         BIT  0,(IY+$1D)     inget: heltal i INTEGER-läget
33B5  28 03               JR   Z,L33BA
33B7  0D                  DEC  C
33B8  0C          L33B8:  INC  C
33B9  0C          L33B9:  INC  C
33BA  DF          L33BA:  RST  $18            nästa tecken
33BB  C9                  RET

33BC  C1          L33BC:  POP  BC
33BD  C1          L33BD:  POP  BC
33BE  C1                  POP  BC
```

```
33BF  C9                RET

33C0  CD E0 30    NAMTXT: CALL LDBCHL         koden ur programmet
33C3  E5          NAMTX1: PUSH HL           variabelnamn till text, kod i BC
33C4  C5                  PUSH BC
33C5  D5                  PUSH DE
33C6  69                  LD   L,C
33C7  60                  LD   H,B
33C8  06 03               LD   B,$03
33CA  CB 3C       L33CA:  SRL  H              koden / 8
33CC  CB 1D               RR   L
33CE  10 FA               DJNZ L33CA
33D0  01 FF 03            LD   BC,$03FF
33D3  A7                  AND  A
33D4  ED 42               SBC  HL,BC
33D6  38 2A               JR   C,L3402        under $3FF: kort namn
33D8  EB                  EX   DE,HL
33D9  CD 7F 34            CALL NAMOFS         biten med namnet
33DC  D9                  EXX
33DD  09                  ADD  HL,BC
33DE  19                  ADD  HL,DE
33DF  D1                  POP  DE
33E0  CD CC 3F            CALL S3FCC          namnet ur listan
33E3  CD 69 3A            CALL EXTEND         LIST slår på EXTEND (egenhet)
33E6  EB                  EX   DE,HL
33E7  C1                  POP  BC
33E8  36 24       L33E8:  LD   (HL),$24       sträng: $
33EA  CB 49               BIT  1,C
33EC  20 0F               JR   NZ,L33FD
33EE  36 25               LD   (HL),$25
33F0  79                  LD   A,C
33F1  0F                  RRCA
33F2  FD CB 1D 46         BIT  0,(IY+$1D)     INTEGER: heltal utan tecken, flyttal med .
33F6  28 03               JR   Z,L33FB
33F8  36 2E               LD   (HL),$2E
33FA  3F                  CCF
33FB  30 01       L33FB:  JR   NC,L33FE
33FD  23          L33FD:  INC  HL
33FE  EB          L33FE:  EX   DE,HL
33FF  E1                  POP  HL
3400  AF                  XOR  A
3401  C9                  RET

3402  09          L3402:  ADD  HL,BC
3403  7D                  LD   A,L
3404  E1                  POP  HL
3405  C1                  POP  BC
3406  CB F0               SET  6,B            bokstaven: nummer + $40
3408  70                  LD   (HL),B
3409  23                  INC  HL
340A  E6 0F               AND  $0F
340C  FE 0F               CP   $0F            15: ingen siffra
340E  28 D8               JR   Z,L33E8
3410  F6 30               OR   $30
3412  77                  LD   (HL),A
3413  23                  INC  HL
3414  18 D2               JR   L33E8
```

NAMPAK: *flytta namnen in i programmets svans* (CLEAR $0F12, $0EC1). *Programmets sista ord är en relativ länk till den första biten* (NAMHD *läser den och nollar bitens länk*). *De andra bitarnas namn kopieras efter dess* (LDIR), *så att svansen blir en enda bit med länk 0, antal och* $FE *namn* $FE *namn ...* $FF; *sista ordet sätts om* (SETREL). *Ett namn tas aldrig bort: namn ur raderade rader och namn som bara skrivits i direktläge står kvar, även vid* SAVE. *Bara* NEW *tömmer listan.* NAMSET (LOAD, $14E9) *sätter först* PRGEND = *DE - 1. Förstör A, BC, DE och HL.*

```
3416  CD 47 34    NAMPAK: CALL NAMHD(egenhet)
3419  C5                  PUSH BC
341A  E3                  EX   (SP),HL
341B  7D          L341B:  LD   A,L
341C  B4                  OR   H
```

```
341D  28 0C              JR   Z,L342B          slut på kedjan
341F  CD E0 30           CALL LDBCHL
3422  C5                 PUSH BC
3423  CD E0 30           CALL LDBCHL
3426  ED B0              LDIR                  bitens namn efter de förra
3428  E1                 POP  HL
3429  18 F0              JR   L341B

342B  3D           L342B:  DEC  A
342C  12                   LD   (DE),A         $FF sist
342D  13                   INC  DE
342E  E1                   POP  HL
342F  EB                   EX   DE,HL
3430  CD 7C 32             CALL SETREL         bitens antal
3433  E5                   PUSH HL
3434  2B                   DEC  HL
3435  22 08 FF             LD   (PRGEND),HL    nytt programslut
3438  2B                   DEC  HL
3439  2B                   DEC  HL
343A  2B                   DEC  HL
343B  13                   INC  DE
343C  EB                   EX   DE,HL
343D  CD 7C 32             CALL SETREL         sista ordet: länk till listan
3440  D1                   POP  DE
3441  C9                   RET

3442  1B           NAMSET: DEC  DE
3443  ED 53 08 FF          LD   (PRGEND),DE
3447  2A 08 FF     NAMHD:  LD   HL,(PRGEND)
344A  56                   LD   D,(HL)
344B  2B                   DEC  HL
344C  5E                   LD   E,(HL)
344D  EB                   EX   DE,HL
344E  19                   ADD  HL,DE          listans huvud
344F  1B                   DEC  DE
3450  4E                   LD   C,(HL)
3451  AF                   XOR  A
3452  77                   LD   (HL),A         länken till nästa bit nollas
3453  23                   INC  HL
3454  46                   LD   B,(HL)
3455  77                   LD   (HL),A
3456  C9                   RET
```

NAMSRC: *sök namnet vid HL i namnlistan med* KWSRCH*, bit för bit.*

CY = funnet, med numret ($0400 *+ avståndet till namnets* $FE*) i BC. Hittas det inte ligger avståndet till listans slut i BC.* NAMWLK/CHKWLK*: gå igenom bitarna med början vid* NAMWLK*:s huvud (programmets sista ord) eller HL; koden efter* CALL *tas som IX (*POP *IX) och anropas för varje bit via* LSTWLK*:s slinga (*$0D85*). CY från den avbryter.* NAMCNT*: DE =* $FFFF *minus storleken på bitarna utanför svansen (*CLEAR*).* NAMOFS*: sök biten som håller avståndet DE (*NAMTXT*).* FNKSRC*: samma sökning i funktionstabellerna (kedjan* FNKWP*).* NAMSRC*: In: HL = namnet i texten. Förstör A, DE, IX och de alternativa registren.*

```
3457  CD A0 34     NAMSRC: CALL NAMWLK
345A  23                   INC  HL             hoppa över länk och antal
345B  23                   INC  HL
345C  23                   INC  HL
345D  23                   INC  HL
345E  E5                   PUSH HL
345F  D9           L345F:  EXX
3460  D1                   POP  DE
3461  C5                   PUSH BC
3462  D5                   PUSH DE
3463  CD 4B 4B             CALL KWSRCH         sök namnet i biten
3466  C1                   POP  BC
3467  EB                   EX   DE,HL
3468  A7                   AND  A
3469  ED 42                SBC  HL,BC
346B  EB                   EX   DE,HL
346C  E3                   EX   (SP),HL
346D  19                   ADD  HL,DE          avståndet läggs till numret
346E  E3                   EX   (SP),HL
```

```
346F  C1                POP  BC
3470  D9                EXX
3471  FE FF             CP   $FF            CY = funnet
3473  C9                RET

3474  11 FF FF    NAMCNT: LD   DE,$FFFF
3477  CD A0 34            CALL NAMWLK
347A  DD 21 82 34          LD   IX,$3482     första biten (svansen): bara byt IX
347E  C9                   RET

347F  CD A0 34    NAMOFS: CALL NAMWLK
3482  E5                  PUSH HL
3483  D9                  EXX
3484  E1                  POP  HL
3485  CD DE 30            CALL S30DE        bitens antal
3488  EB                  EX   DE,HL
3489  A7                  AND  A
348A  ED 42                 SBC  HL,BC       CY: namnet ligger i den här biten
348C  EB                    EX   DE,HL
348D  D9                    EXX
348E  C9                    RET

348F  CD A8 34    FNKSRC: CALL CHKWLK
3492  01 06 00            LD   BC,$0006     huvudet +6: nyckelorden, BC = +8
3495  09                  ADD  HL,BC
3496  5E                  LD   E,(HL)
3497  23                  INC  HL
3498  56                  LD   D,(HL)
3499  23                  INC  HL
349A  4E                  LD   C,(HL)
349B  23                  INC  HL
349C  46                  LD   B,(HL)
349D  D5                  PUSH DE
349E  18 BF               JR   L345F

34A0  D9          NAMWLK: EXX
34A1  2A 08 FF            LD   HL,(PRGEND)  programmets sista ord: länken till namnlistan
34A4  46                  LD   B,(HL)
34A5  2B                  DEC  HL
34A6  4E                  LD   C,(HL)
34A7  09                  ADD  HL,BC
34A8  DD E1       CHKWLK: POP  IX           IX = koden efter CALL
34AA  11 FF FF            LD   DE,$FFFF
34AD  AF                  XOR  A
34AE  C3 85 0D            JP   L0D85
```

RND: *slumptal. Fröet RNDSED* ($FF47-$FF4B) *är 40 bitar:*

```
frö = frö * $00BB40E64D + $1440B60CA0  (mod 2^40)
```

Faktorn och tillägget står i RNDMUL/RNDADD ($3548-$3551). *Fröet blir mantissan i ett tal mellan 0 och 1.* RUN *nollställer fröet* (RNDCLR), RANDOMIZE *sätter det från R-registret. Ett argument används inte: RND(5) och RND(-1) ger samma följd som RND. Med DOUBLE blir alla 40 bitarna mantissa: första RND efter RUN är .079112413484836 (7.91124E-02 med 4 bytes). RANDOMIZE lägger R-registret i fröets fyra höga byte och nollar den lägsta; R räknar bara med 7 bitar, så det finns 128 olika följder. In: DE efter koden. Ut: talet vid SP, nästa kod. Förstör alla register utom IY, även IX.*

```
34B1  CD A3 1D    RND:    CALL S1DA3
34B4  1B                  DEC  DE
34B5  1B                  DEC  DE
34B6  1A                  LD   A,(DE)
34B7  FE D0               CP   $D0
34B9  C4 C3 4D            CALL NZ,FDROP     RND(x): argumentet tas bort
34BC  DD 21 00 00         LD   IX,$0000
34C0  DD E5               PUSH IX
34C2  DD E5               PUSH IX
34C4  DD E5               PUSH IX
34C6  DD E5               PUSH IX
34C8  DD 39               ADD  IX,SP
34CA  21 4B FF            LD   HL,$FF4B     fröets lägsta byte
34CD  EB                  EX   DE,HL
34CE  21 4C 35            LD   HL,$354C     RNDMUL:s lägsta byte
```

```
34D1  DD 36 00 05          LD   (IX+$00),$05
34D5  0E 08          L34D5: LD   C,$08
34D7  7E                    LD   A,(HL)
34D8  E5                    PUSH HL
34D9  D5             L34D9: PUSH DE
34DA  0F                    RRCA
34DB  F5                    PUSH AF
34DC  30 1A                 JR   NC,L34F8
34DE  DD E5                 PUSH IX
34E0  26 00                 LD   H,$00
34E2  DD 6E 00              LD   L,(IX+$00)
34E5  45                    LD   B,L
34E6  EB                    EX   DE,HL
34E7  DD 19                 ADD  IX,DE
34E9  7E             L34E9: LD   A,(HL)
34EA  DD 8E 00              ADC  A,(IX+$00)
34ED  DD 77 00              LD   (IX+$00),A
34F0  2B                    DEC  HL
34F1  DD 2B                 DEC  IX
34F3  10 F4                 DJNZ L34E9
34F5  DD E1                 POP  IX
34F7  AF                    XOR  A
34F8  21 4B FF       L34F8: LD   HL,$FF4B          fröet x 2
34FB  06 05                 LD   B,$05
34FD  CB 16          L34FD: RL   (HL)
34FF  2B                    DEC  HL
3500  10 FB                 DJNZ L34FD
3502  F1                    POP  AF
3503  D1                    POP  DE
3504  0D                    DEC  C
3505  20 D2                 JR   NZ,L34D9
3507  E1                    POP  HL
3508  1B                    DEC  DE
3509  2B                    DEC  HL
350A  DD 35 00              DEC  (IX+$00)
350D  20 C6                 JR   NZ,L34D5
350F  06 05                 LD   B,$05
3511  11 51 35              LD   DE,$3551         RNDADD:s lägsta byte
3514  21 4B FF              LD   HL,$FF4B
3517  DD 36 00 80           LD   (IX+$00),$80
351B  AF                    XOR  A
351C  1A             L351C: LD   A,(DE)
351D  DD 8E 05              ADC  A,(IX+$05)       produkten + tillägget: nya fröet
3520  77                    LD   (HL),A
3521  DD 77 05              LD   (IX+$05),A
3524  2B                    DEC  HL
3525  1B                    DEC  DE
3526  DD 2B                 DEC  IX
3528  10 F2                 DJNZ L351C
352A  CD 5B 4D       L352A: CALL FNRM8            normalisera: ett tal mellan 0 och 1
352D  FD CB 1E 5E           BIT  3,(IY+$1E)
3531  CC 99 4C              CALL Z,FRND4
3534  C3 80 21              JP   L2180

3537  ED 5F        RANDOM: LD   A,R              RANDOMIZE: A = R (0-127)
3539  06                    DEFB $06              LD B,$AF (överlappar nästa)
353A  AF             RNDCLR: XOR  A
353B  21 4B FF              LD   HL,$FF4B
353E  4F                    LD   C,A
353F  36 00                 LD   (HL),$00         lägsta byten 0, de fyra andra = A
3541  06 04                 LD   B,$04
3543  2B             L3543: DEC  HL
3544  71                    LD   (HL),C
3545  10 FC                 DJNZ L3543
3547  C9                    RET

3548  00 BB 40 E6… RNDMUL: DEFB $00,$BB,$40,$E6,$4D
354D  14 40 B6 0C… RNDADD: DEFB $14,$40,$B6,$0C,$A0
```

3.23 PSCAN: gå igenom programmet och anropa IX för varje referens $3552

PSCAN börjar på första raden, PSCAN1 vid HL. PSTEP tar ett steg: koder >= $C0 är uttryck (ESCAN), andra är satskoder som slås upp i SCNSTM (25 par: satskod, index i SCNSJP). Satskoder som inte står där hoppas över (deras uttryck kommer i följande steg). IX anropas med A = sort och HL efter referensen; DE beror på sorten:

```
0  DEF FN ($8C)  DE = parameterlistan (koden + 7); HL efter listan
                 och LOCAL-listan ($B6)
1  FNEND ($8D), 2 ($A3), 3 ($A4): DE utan betydelse
5  FOR ($98)     DE = styrvariabelns ord (variabeln har redan gått
                 som $0A); TO och STEP hoppas över
6  WHILE ($9A), 7 NEXT ($99), 8 WEND ($9B): DE = ordet efter koden
                 (NEXT: variabeln, WEND: länken)
9  radnummer efter $80-$85 (GOTO, GOSUB, RESTORE, RESUME, ON ERROR
   GOTO ...) och i listan efter ON ($BF antal, kod, nummer ...);
   DE = numret
$0A variabel: $C0-$C2, $C4-$C6, fält $C8-$CA (antalet index hoppas
    över), tilldelning $B4-$B6 (avslutar uttrycket); DE = referensen
$0B FN-anrop ($CF + 2 bytes), DE = namnet
$0C COMMON ($CE): DE = FREPTR + 4, BC = ordet på FREPTR + 2;
    variabeln före går som $0A med IY+$1D bit 5 satt
Sort 4 används inte.
```

Satskoderna $8B, $8F och $90 har längdbyte + text, $8E längdbyte + 2 bytes, $86/$8A en byte, $BC två; $B7 hoppas över till en byte med bit 2 satt. I uttrycken hoppas konstanterna över ($CB/$CC strängar, $D5-$DC tal, $DD- små heltal), operatorerna ($AA och lägre) har ingen operand, och $B1-$B3, $B7-$BF avslutar uttrycket.

Rad: SLINE ($87) hoppar över längd och radnummer; längd 1 = slut. Direktradens slutkod $89 avslutar också genomgången.

RESUME som sats ($9E) eller som kod i ON-listan ($84) sätter IY+$26 bit 7 (SCNCOD): programmet har RESUME, och ON ERROR står kvar efter ett fel; RESUME i en REM räknas inte.

IX returnerar CY = 1 vid fel (A = felnummer), då avbryts genomgången. Samma genomgång används för länkning (LINK pass 2, LINK1), avlänkning (UNLINK), RENUMBER ($42EB) och ULVAL ($30A5: NEXT utan länk, IX = L30E5 letar efter FOR-satserna). In: IX = rutinen. Ut: NC = slutet nått (slutraden eller direktradens $89); CY: felet från IX, A = felet, HL vid felet. IX kvar; förstör de övriga registren.

```
3552  CD 44 00     PSCAN:   CALL FSTLIN
3555  CD 5F 35     PSCAN1:  CALL PSTEP          ett steg; CY = fel eller slut
3558  FD CB 1D AE           RES  5,(IY+$1D)     COMMON-läget av (SCOMMN)
355C  30 F7                 JR   NC,PSCAN1
355E  C9                    RET

355F  7E           PSTEP:   LD   A,(HL)
3560  FE C0                 CP   $C0
3562  30 18                 JR   NC,ESCAN       uttryck
3564  22 20 FF              LD   (SCNPTR),HL
3567  CD D8 35              CALL SCNCOD          satskoden; RESUME sätter IY+$26 bit 7
356A  EB                    EX   DE,HL
356B  21 57 36              LD   HL,$3657
356E  01 32 00              LD   BC,$0032        25 par
3571  ED B1                 CPIR                 A = koden (> $11, träffar aldrig ett index)
3573  EB                    EX   DE,HL
3574  1A                    LD   A,(DE)
3575  01 89 36              LD   BC,$3689
3578  28 23                 JR   Z,L359D         funnen: JPTAB med A = index (NC)
357A  A7           L357A:   AND  A               annars hoppa över koden
357B  C9                    RET
```

ESCAN: gå igenom ett uttryck till och med slutkoden ($B1-$BF). ESTEP tar en kod: $DD- små heltal, $D5-$DC talkonstanter (1-8 bytes), $D0-$D4 funktioner med argumentbyte, $AA-$CF via SCNEJP, lägre koder (operatorer) saknar operand.

```
357C  22 20 FF     ESCAN:   LD   (SCNPTR),HL
357F  CD 86 35     L357F:   CALL ESTEP
3582  30 FB                 JR   NC,L357F        tills slutkoden (POP BC; RET NC) eller fel
3584  C9                    RET

3585  C1           L3585:   POP  BC              XDIM: nästa kod avslutar uttrycket
3586  7E           ESTEP:   LD   A,(HL)
3587  23                    INC  HL
```

```
3588  FE DD              CP   $DD
358A  D0                 RET  NC             $DD-: litet heltal
358B  FE D5              CP   $D5
358D  30 27              JR   NC,L35B6
358F  FE D4              CP   $D4
3591  28 30              JR   Z,L35C3
3593  FE D0              CP   $D0
3595  30 2D              JR   NC,L35C4
3597  C6 56              ADD  A,$56          operator: ingen operand
3599  D0                 RET  NC
359A  01 AD 36           LD   BC,$36AD
359D  C3 B7 13  L359D:   JP   JPTAB

35A0  C1        L35A0:   POP  BC             tilldelning: referensen avslutar uttrycket
35A1  2B                 DEC  HL
35A2  23        L35A2:   INC  HL             fält: hoppa över antalet index
35A3  3E 0A     L35A3:   LD   A,$0A          variabel
35A5  CD 33 32  SCNREF:  CALL LDDEHL
35A8  DD E9     SCNCB:   JP   (IX)

35AA  23        L35AA:   INC  HL             FN-anrop: hoppa över 2 bytes
35AB  23                 INC  HL
35AC  3E 0B              LD   A,$0B
35AE  18 F5              JR   SCNREF

35B0  23        L35B0:   INC  HL
35B1  23                 INC  HL
35B2  3F                 CCF                 CY = 0: längdbyten + 2 bytes
35B3  3F        L35B3:   CCF                 CY = 1: strängkonstant, längdbyten + texten
35B4  7E        L35B4:   LD   A,(HL)
35B5  01                 DEFB $01            DEFB $01 slukar SUB $D4; LD BC,$D4D6 (överlappar nästa)
35B6  D6 D4     L35B6:   SUB  $D4
35B8  4F                 LD   C,A
35B9  06 00              LD   B,$00
35BB  ED 4A              ADC  HL,BC          HL + A + CY
35BD  C9                 RET

35BE  7E        SLINE:   LD   A,(HL)
35BF  23                 INC  HL
35C0  3D                 DEC  A
35C1  28 03              JR   Z,L35C6        längd 1 = programmets slut
35C3  23        L35C3:   INC  HL
35C4  C5        L35C4:   PUSH BC
35C5  23        L35C5:   INC  HL
35C6  C1        L35C6:   POP  BC             POP BC: avsluta ESCAN, eller PSCAN vid $89
35C7  C9                 RET

35C8  46        SONGO:   LD   B,(HL)         antal radnummer
35C9  23                 INC  HL
35CA  CD D8 35           CALL SCNCOD         koden efter ON (GOTO, RESUME ...)
35CD  C5        L35CD:   PUSH BC
35CE  3E 09              LD   A,$09
35D0  CD A5 35           CALL SCNREF
35D3  C1                 POP  BC
35D4  D8                 RET  C              fel: avbryt
35D5  10 F6              DJNZ L35CD
35D7  C9                 RET

35D8  7E        SCNCOD:  LD   A,(HL)         RESUME i koden sätter IY+$26 bit 7
35D9  23                 INC  HL
35DA  FE 84              CP   $84
35DC  28 04              JR   Z,L35E2
35DE  FE 9E              CP   $9E
35E0  20 98              JR   NZ,L357A
35E2  FD CB 26 FE L35E2: SET  7,(IY+$26)     programmet har RESUME
35E6  C9                 RET

35E7  11 06 00  SDEF:    LD   DE,$0006       DE = parameterlistan (koden + 7)
35EA  19                 ADD  HL,DE
35EB  E5                 PUSH HL
35EC  5A                 LD   E,D            E = D = $36: namnet finns aldrig
35ED  CD 2D 36           CALL PARSRC         -> HL efter listan (och LOCAL-listan)
```

```
35F0  3E 00              LD   A,$00
35F2  18 20              JR   L3614

35F4  CB 56       L35F4: BIT  2,(HL)
35F6  23                 INC  HL
35F7  28 FB              JR   Z,L35F4
35F9  C9                 RET

35FA  CD 7C 35    SFOR:  CALL ESCAN
35FD  D8                 RET  C              startvärdet (variabeln med sort $0A)
35FE  2B                 DEC  HL
35FF  2B                 DEC  HL
3600  CD E0 30           CALL LDBCHL         BC = styrvariabelns ord
3603  C5                 PUSH BC
3604  23          L3604: INC  HL
3605  CD 7C 35           CALL ESCAN          hoppa över $BD/$BE, TO-/STEP-värdet
3608  38 BC              JR   C,L35C6
360A  7E                 LD   A,(HL)
360B  EE BE              XOR  $BE            STEP ($BE) följer?
360D  28 F5              JR   Z,L3604
360F  3E 05              LD   A,$05          FOR: sort 5, DE = variabeln
3611  23                 INC  HL
3612  23          L3612: INC  HL
3613  23                 INC  HL
3614  D1          L3614: POP  DE
3615  D8                 RET  C
3616  DD E9              JP   (IX)

3618  E3          SCOMMN: EX  (SP),HL        HL = FREPTR (EX med returadressen)
3619  CD CF 0F           CALL S0FCF
361C  CD DE 30           CALL S30DE          BC = (FREPTR + 2), stacken: FREPTR + 4
361F  E3                 EX   (SP),HL
3620  FD CB 1D EE        SET  5,(IY+$1D)     COMMON-läget (ALLOCC)
3624  CD 86 35           CALL ESTEP          variabeln (sort $0A)
3627  38 EB              JR   C,L3614
3629  3E 0C              LD   A,$0C          sort $0C, DE = FREPTR + 4
362B  18 E5              JR   L3612
```

PARSRC: *sök namnet DE i en DEF:s parameterlista vid HL (antal, namn ..., och efter* $B6 *LOCAL-listan på samma sätt; en lokal sträng har 2 bytes längd efter namnet). Ut: Z = funnet, C = numret x 2 (bit 7 satt för LOCAL); NZ: HL efter listorna. Förstör A och B.*

```
362D  0E 00       PARSRC: LD   C,$00        parameterlistan: antal, namn ...
362F  7E          L362F:  LD   A,(HL)
3630  A7                  AND  A
3631  28 1A                JR   Z,L364D
3633  47                   LD   B,A
3634  23          L3634:  INC  HL
3635  7E                   LD   A,(HL)
3636  23                   INC  HL
3637  BB                   CP   E              namnet = DE?
3638  20 03                JR   NZ,L363D
363A  7E                   LD   A,(HL)
363B  BA                   CP   D
363C  C8                   RET  Z              funnet: Z
363D  CB 79       L363D:  BIT  7,C
363F  28 08                JR   Z,L3649
3641  2B                   DEC  HL             lokal sträng: 2 bytes längd till
3642  CB 4E                BIT  1,(HL)
3644  23                   INC  HL
3645  28 02                JR   Z,L3649
3647  23                   INC  HL
3648  23                   INC  HL
3649  0C          L3649:  INC  C              C = 2 x nummer, bit 7 = LOCAL
364A  0C                   INC  C
364B  10 E7                DJNZ L3634
364D  CB F9       L364D:  SET  7,C
364F  23                   INC  HL
3650  7E                   LD   A,(HL)
3651  EE B6                XOR  $B6            LOCAL ($B6) följer?
```

```
3653  C0                RET  NZ
3654  23                INC  HL
3655  18 D8             JR   L362F
```

SCNSTM: *satskod -> index i* SCNSJP *(25 par).*

```
3657  8C 00 98 05…  SCNSTM: DEFB $8C,$00,$98,$05,$8D,$01,$9A,$06
365F  9B 08 99 07…           DEFB $9B,$08,$99,$07,$80,$09,$81,$09
3667  82 09 83 09…           DEFB $82,$09,$83,$09,$84,$09,$85,$09
366F  8F 0C 8A 0A…           DEFB $8F,$0C,$8A,$0A,$86,$0A,$BC,$0B
3677  90 0C 8B 0C…           DEFB $90,$0C,$8B,$0C,$8E,$0D,$BF,$0E
367F  B7 0F 89 11…           DEFB $B7,$0F,$89,$11,$87,$10,$A3,$02
3687  A4 03                  DEFB $A4,$03
3689  E7 35          SCNSJP: DEFW SDEF
368B  A8 35                  DEFW SCNCB
368D  A8 35                  DEFW SCNCB
368F  A8 35                  DEFW SCNCB
3691  A8 35                  DEFW SCNCB
3693  FA 35                  DEFW SFOR
3695  A5 35                  DEFW SCNREF
3697  A5 35                  DEFW SCNREF
3699  A5 35                  DEFW SCNREF
369B  A5 35                  DEFW SCNREF
369D  C4 35                  DEFW L35C4
369F  C3 35                  DEFW L35C3
36A1  B4 35                  DEFW L35B4
36A3  B0 35                  DEFW L35B0
36A5  C8 35                  DEFW SONGO
36A7  F4 35                  DEFW L35F4
36A9  BE 35                  DEFW SLINE
36AB  C6 35                  DEFW L35C6
36AD  C4 35          SCNEJP: DEFW L35C4
36AF  C4 35                  DEFW L35C4
36B1  C4 35                  DEFW L35C4
36B3  C4 35                  DEFW L35C4
36B5  C4 35                  DEFW L35C4
36B7  CC 19                  DEFW NOHOOK
36B9  CC 19                  DEFW NOHOOK
36BB  C6 35                  DEFW L35C6
36BD  C6 35                  DEFW L35C6
36BF  C6 35                  DEFW L35C6
36C1  A0 35                  DEFW L35A0
36C3  A0 35                  DEFW L35A0
36C5  A0 35                  DEFW L35A0
36C7  C6 35                  DEFW L35C6
36C9  C6 35                  DEFW L35C6
36CB  C6 35                  DEFW L35C6
36CD  C6 35                  DEFW L35C6
36CF  C6 35                  DEFW L35C6
36D1  C6 35                  DEFW L35C6
36D3  C5 35                  DEFW L35C5
36D5  C5 35                  DEFW L35C5
36D7  C6 35                  DEFW L35C6
36D9  A3 35                  DEFW L35A3
36DB  A3 35                  DEFW L35A3
36DD  A3 35                  DEFW L35A3
36DF  CC 19                  DEFW NOHOOK
36E1  A3 35                  DEFW L35A3
36E3  A3 35                  DEFW L35A3
36E5  A3 35                  DEFW L35A3
36E7  C3 35                  DEFW L35C3
36E9  A2 35                  DEFW L35A2
36EB  A2 35                  DEFW L35A2
36ED  A2 35                  DEFW L35A2
36EF  B3 35                  DEFW L35B3
36F1  B3 35                  DEFW L35B3
36F3  85 35                  DEFW L3585
36F5  18 36                  DEFW SCOMMN
36F7  AA 35                  DEFW L35AA
```

VARPTR(v) och VAROOT(v): adresser i variabelområdet (heltal, som PEEK/PEEK2 tar). VARPTR ger värdets adress: flyttal och heltal själva talet, en sträng texten, ett fältelement elementet. VAROOT ger för en sträng variabelns post (+0 största längd, +2 textens adress, +4 längden) och för ett fältelement fältets post (ARRELM, LSTARR: +0 storlek, +2 data, +4 antal dimensioner, +5 elementets storlek, +6 undre gräns, +8 antal ...); för ett tal samma som VARPTR. Utan index avses den enkla variabeln (D% och D%(1) är olika variabler). Argumentet måste vara en variabel (VARPTR(1) fel 234). In: argumentets adress vid SP, DE efter koden. Ut: adressen som heltal vid SP, nästa kod.

```
36F9  CD 11 37     VARPTR: CALL S3711
36FC  CB 4F                BIT  1,A            sträng?
36FE  28 05                JR   Z,L3705
3700  E1                   POP  HL
3701  CD DE 30             CALL S30DE          då textens adress (variabeln +2)
3704  C5                   PUSH BC
3705  EF             L3705: RST  $28
3706  CD 11 37       VAROOT: CALL S3711
3709  FA A9 1D             JP   M,EXECOD       enkel variabel: som VARPTR
370C  2A 38 FF             LD   HL,(LSTARR)    fältelement: fältets post (LSTARR)
370F  E3                   EX   (SP),HL
3710  EF                   RST  $28
3711  21 FB FF       S3711: LD   HL,$FFFB      argumentets kod (DE-5, fält DE-6)
3714  19                   ADD  HL,DE
3715  7E                   LD   A,(HL)
3716  A7                   AND  A
3717  F8                   RET  M
3718  2B                   DEC  HL
3719  7E                   LD   A,(HL)
371A  C9                   RET
```

CVT%$(i) och CVTF$(x) ger talets bytes som en sträng (2 resp. 4 eller 8 bytes med DOUBLE, lägsta byten först: CVT%$(258) = CHR$(2,1)). CVT$%(s) och CVT$F(s) gör tvärtom med strängens första 2/4/8 bytes. Strängens längd prövas inte: en kortare sträng läser vidare i minnet (CVT$%("") ger ett skräpvärde). In: argumentet vid SP, DE efter koden. Ut: resultatet i dess ställe, nästa kod.

```
371B  FD 4E 1E     CVTSF:  LD   C,(IY+$1E)     CVT$F: flyttalets storlek (4/8)
371E  21                   DEFB $21            LD HL,$020E (överlappar nästa)
371F  0E 02        CVTSI:  LD   C,$02          CVT$%: 2 bytes text -> heltal
3721  06 00                LD   B,$00
3723  CD A3 1D             CALL S1DA3
3726  CD CF 0F             CALL S0FCF
3729  EB                   EX   DE,HL
372A  F1                   POP  AF
372B  E1                   POP  HL
372C  DD E1                POP  IX
372E  F5                   PUSH AF
372F  C5                   PUSH BC
3730  D5                   PUSH DE
3731  ED B0                LDIR                C bytes ur texten, längden prövas inte (egenhet)
3733  D1                   POP  DE
3734  C1                   POP  BC
3735  E1                   POP  HL
3736  39                   ADD  HL,SP
3737  ED 42                SBC  HL,BC
3739  F9                   LD   SP,HL
373A  EB                   EX   DE,HL
373B  ED B0                LDIR
373D  C3 80 21             JP   L2180

3740  FD 4E 1E     CVTFS:  LD   C,(IY+$1E)     CVTF$: flyttalets storlek (4/8)
3743  21                   DEFB $21            LD HL,$020E (överlappar nästa)
3744  0E 02        CVTIS:  LD   C,$02          CVT%$: 2 bytes
3746  21 00 00             LD   HL,$0000
3749  44                   LD   B,H
374A  39                   ADD  HL,SP          strängen = värdets bytes på stacken
374B  C5                   PUSH BC
374C  E5                   PUSH HL
374D  C5                   PUSH BC
374E  EF                   RST  $28
```

`HEX$(i)` *och* `OCT$(i)`*: heltalet utan tecken (0-65535) som text i bas 16 resp. 8, utan inledande nollor:* `HEX$(255) = "FF"`*,* `HEX$(-1) = "FFFF"`*,* `OCT$(65535) = "177777"`*,* `HEX$(0) = "0"`*. Argumentet avrundas* (`HEX$(1.6) = "2"`)*; från 65536 fel 132.*

```
374F  0E 10       HEX:    LD   C,$10          HEX$: bas 16
3751  21                  DEFB $21            LD HL,$080E (överlappar nästa)
3752  0E 08       OCT:    LD   C,$08          OCT$: bas 8
3754  E3                  EX   (SP),HL
3755  E5                  PUSH HL
3756  E5                  PUSH HL
3757  EB                  EX   DE,HL
3758  E5                  PUSH HL
3759  21 02 00            LD   HL,$0002
375C  39                  ADD  HL,SP
375D  E5                  PUSH HL
375E  EB                  EX   DE,HL
375F  CD 15 45            CALL BASSTR         siffrorna, utan inledande nollor
3762  E1                  POP  HL
3763  EB                  EX   DE,HL
3764  ED 52               SBC  HL,DE
3766  E3                  EX   (SP),HL
3767  D5                  PUSH DE
3768  0E 06               LD   C,$06
376A  C5                  PUSH BC
376B  EB                  EX   DE,HL
376C  EF                  RST  $28
```

`L376D` *(från* `CUNARY`*, klass* `$81`*): unärt + eller - först på nivån + -. + hoppas över; efter - kompileras operanden på nivån * / och högre* (`CEXLVL`)*, och INEG* (`$A9`) *eller FNEG* (`$A8`) *läggs ut; en sträng ger fel 224* (`-"A"`)*.* `CUNARY`*: på relationsnivån prövas NOT* (`KWSRCH` *med tabellen vid* `$06DF`)*; NOT gäller hela relationen, som görs om till heltal* (`CTOINT`) *och får koden* `$AF`*. In: HL = texten, DE = koden, IX = nivån. Ut: NC: B = typen; CY: A = felet.*

```
376D  7E          L376D:  LD   A,(HL)          unärt tecken först på nivån + -
376E  FE 2D               CP   $2D
3770  28 07               JR   Z,L3779
3772  FE 2B               CP   $2B
3774  CC 19 4C            CALL Z,S4C19        + hoppas över
3777  18 4B               JR   S37C4

3779  DF          L3779:  RST  $18
377A  DD E5               PUSH IX
377C  CD B4 37            CALL CEXLVL         operanden: nivån * / och högre
377F  DD E1               POP  IX
3781  D8                  RET  C
3782  3E 01               LD   A,$01
3784  B8                  CP   B
3785  3E A9               LD   A,$A9          heltal: INEG ($A9)
3787  28 09               JR   Z,L3792
3789  3E 00               LD   A,$00
378B  B8                  CP   B
378C  3E E0               LD   A,$E0
378E  37                  SCF                 sträng: fel 224
378F  C0                  RET  NZ
3790  3E A8               LD   A,$A8          flyttal: FNEG ($A8)
3792  12          L3792:  LD   (DE),A
3793  13                  INC  DE
3794  18 39               JR   L37CF

3796  FE 81       CUNARY: CP   $81            klass $81: + - med unärt tecken
3798  28 D3               JR   Z,L376D
379A  D5                  PUSH DE
379B  11 DF 06            LD   DE,$06DF
379E  CD 4B 4B            CALL KWSRCH         relationsnivån: NOT först?
37A1  D1                  POP  DE
37A2  20 20               JR   NZ,S37C4
37A4  CD C4 37            CALL S37C4          NOT gäller hela relationen
37A7  D4 D1 43            CALL NC,CTOINT
37AA  D8                  RET  C
37AB  3E AF               LD   A,$AF          NOT = $AF (heltal)
37AD  C3 87 16            JP   EMIT
```

3.24 CEXPR: kompilera ett uttryck (prioritetsklättring) $37B0

IX går igenom OPGRP (3 bytes per nivå, lägst prioritet först):

```
EQV, IMP, OR/XOR, AND  klass 3 (bara heltal)
relationer             klass 2, bit 7: NOT kan stå före (CUNARY)
+ -                    klass 1, bit 7: unärt minus
* /                    klass 1
^                      klass 4
```

Klassen avgör vilken körkod som läggs ut: grundkoden för flyttal, +1 för heltal, +2 för strängar (jämförelser och +). Blandas heltal och flyttal läggs FTOI/ITOF ($81/$82) in (CTOINT, CTOFLT). Längst ned (klass 0) kompilerar CFACT en faktor: tal, sträng, variabel, funktion eller parentes. B = typen på resultatet (0 flyttal, 1 heltal, 2 sträng). Fel $E0 (224): sträng där ett tal behövs, eller tal mot sträng (A$-Y, A$*2, A$ AND 1, -A$, A$=1, A$+1).

Varje nivå är vänsterassociativ: 2^3^2 = 64, 3<2<1 = -1, 8/2/2 = 2. Unärt minus bara först på nivån + -: -2^2 = -4, men 2*-3 och 2^-1 ger fel 234. NOT bara först på relationsnivån och gäller hela relationen: NOT 1=2 = -1, NOT 2+3 = -6, 1 AND NOT 0 = 1, men i 2=NOT 1 och NOT NOT 1 blir det andra NOT variabeln N följd av OT: fel 220 i PRINT (en bokstav efter en bokstav) och 221 efter X= (radslut väntas). De logiska operatorerna avrundar flyttal till heltal (7.9 AND 3 = 0). Flyttal ^ heltal behålls (kod + 2, FPOWI). In: HL = texten, DE = koden. Ut: NC: B = typen, HL efter uttrycket, DE efter koden (utan $B7+typ, se CEXPA); CY: A = felet. Förstör AF, C, IX och de alternativa registren. Medan den högra operanden kompileras ligger vid SP (IX = SP efter CALL CEXLVL): +0/+1 IX (nivån), +2/+3 AF (+3 = operatorns kod), +4/+5 BC (+4 = klassen, som räknas ned för att välja fall, +5 = den vänstra typen) och +6/+7 DE, där den högra operandens kod börjar.

```
37B0  DD 21 AA 06  CEXPR:   LD   IX,$06AA        IX = OPGRP; CEXLVL ökar först (egenhet)
37B4  DD 23        CEXLVL:  INC  IX              en nivå i CEXPR
37B6  DD 23                 INC  IX
37B8  DD 23                 INC  IX
37BA  DD 7E 02              LD   A,(IX+$02)       klassen
37BD  A7                    AND  A
37BE  CA AF 17              JP   Z,CFACT          klass 0: en faktor
37C1  FA 96 37              JP   M,CUNARY         bit 7: tecken eller NOT först
37C4  CD EF 0F     S37C4:   CALL SPCHK0           stacken räcker? (fel 201)
37C7  DD E5                 PUSH IX               nivån
37C9  D4 B4 37              CALL NC,CEXLVL        vänstra operanden, nästa nivå
37CC  DD E1                 POP  IX
37CE  D8           L37CE:   RET  C                L37CE: fel
37CF  D5           L37CF:   PUSH DE               L37CF: nästa operator på nivån (vänster först)
37D0  DD 5E 00              LD   E,(IX+$00)       nivåns nyckelord
37D3  DD 56 01              LD   D,(IX+$01)
37D6  CD 4B 4B              CALL KWSRCH           operator på den här nivån?
37D9  D1                    POP  DE
37DA  C0                    RET  NZ               ingen: klar, B = typen
37DB  DD 4E 02              LD   C,(IX+$02)       C = klassen utan bit 7
37DE  CB B9                 RES  7,C
37E0  D5                    PUSH DE               där den högra operanden börjar
37E1  C5                    PUSH BC               klassen och den vänstra typen
37E2  F5                    PUSH AF               A = operatorns kod
37E3  DD E5                 PUSH IX
37E5  CD B4 37              CALL CEXLVL           högra operanden, B = dess typ
37E8  38 16                 JR   C,L3800          fel
37EA  DD 21 00 00           LD   IX,$0000         IX = SP: ramen
37EE  DD 39                 ADD  IX,SP
37F0  DD 7E 03              LD   A,(IX+$03)       operatorns kod
37F3  FE 9B                 CP   $9B              + ($9B): även strängar
37F5  20 12                 JR   NZ,L3809
37F7  CD 62 38              CALL S3862            kod + typ, också för strängar
37FA  38 04        L37FA:   JR   C,L3800          L37FA: fel
37FC  79           L37FC:   LD   A,C              L37FC: koden C
37FD  CD 87 16              CALL EMIT
3800  DD E1        L3800:   POP  IX               L3800: släpp ramen, AF bevaras
3802  08                    EX   AF,AF'
3803  F1                    POP  AF
3804  F1                    POP  AF
3805  F1                    POP  AF
3806  08                    EX   AF,AF'
3807  18 C5                 JR   L37CE            nästa operator
```

```
3809  DD 35 04     L3809:  DEC  (IX+$04)        klass 1: + - * /
380C  20 0B                JR   NZ,L3819        inte klass 1
380E  78           L380E:  LD   A,B             sträng: fel 224
380F  FE 02                CP   $02
3811  3E E0                LD   A,$E0
3813  37                   SCF
3814  C4 62 38             CALL NZ,S3862        tal: kod + typ
3817  18 E1                JR   L37FA

3819  DD 35 04     L3819:  DEC  (IX+$04)        klass 2: relation, resultat heltal
381C  20 07                JR   NZ,L3825
381E  CD 62 38             CALL S3862           kod + typ, också för strängar
3821  06 01                LD   B,$01           resultatet heltal
3823  18 D5                JR   L37FA

3825  DD 35 04     L3825:  DEC  (IX+$04)        klass 3: bara heltal
3828  20 1F                JR   NZ,L3849
382A  CD D1 43             CALL CTOINT          höger till heltal
382D  38 D1                JR   C,L3800         fel
382F  DD 7E 05             LD   A,(IX+$05)      den vänstra typen
3832  FE 01                CP   $01             heltal: klar
3834  28 0E                JR   Z,L3844
3836  FE 00                CP   $00             flyttal?
3838  3E E0                LD   A,$E0
383A  37                   SCF
383B  20 C3                JR   NZ,L3800        sträng: fel 224
383D  3E 81                LD   A,$81           vänster flyttal: FTOI in
383F  CD 97 38             CALL S3897
3842  06 01                LD   B,$01           B = 1
3844  DD 4E 03     L3844:  LD   C,(IX+$03)      L3844: koden utan typ
3847  18 B3                JR   L37FC

3849  DD 35 04     L3849:  DEC  (IX+$04)        klass 4: ^
384C  20 FE        L384C:  JR   NZ,L384C        annan klass: slinga (finns inte)
384E  78                   LD   A,B             höger heltal?
384F  FE 01                CP   $01
3851  20 BB                JR   NZ,L380E        nej: som klass 1
3853  DD 7E 05             LD   A,(IX+$05)      vänster flyttal?
3856  FE 00                CP   $00
3858  20 B4                JR   NZ,L380E        nej: som klass 1
385A  DD 4E 03             LD   C,(IX+$03)      flyttal ^ heltal: kod + 2
385D  0C                   INC  C
385E  0C                   INC  C
385F  47                   LD   B,A             B = 0: flyttal
3860  18 98                JR   L37FA
```

S3862: gör typerna lika för operatorn i CEXPR:s ram. In: B = den högra typen, IX = ramen (+3 grundkoden, +5 den vänstra typen). Lika typer: C = koden + typen. Höger heltal, vänster flyttal: ITOF efter den högra (CTOFLT), C = koden, B = 0. Vänster heltal, höger flyttal: ITOF in efter den vänstra (S3897), C = koden, B = 0. Sträng mot tal: CY och A = 224. Förstör AF.

```
3862  78           S3862:  LD   A,B             typerna lika: kod + typ
3863  DD BE 05             CP   (IX+$05)        den vänstra typen
3866  28 17                JR   Z,L387F         lika
3868  FE 00                CP   $00             höger flyttal
386A  28 19                JR   Z,L3885
386C  FE 01                CP   $01             höger heltal?
386E  3E E0                LD   A,$E0
3870  37                   SCF
3871  C0                   RET  NZ              höger sträng: fel 224
3872  DD 7E 05             LD   A,(IX+$05)      den vänstra typen
3875  FE 00                CP   $00
3877  3E E0                LD   A,$E0           vänster sträng: fel 224
3879  37                   SCF
387A  C0                   RET  NZ
387B  CD E2 43             CALL CTOFLT          höger heltal: ITOF efter
387E  D8                   RET  C
387F  78           L387F:  LD   A,B             L387F: C = koden + typen
3880  DD 86 03             ADD  A,(IX+$03)
3883  4F                   LD   C,A
3884  C9                   RET
```

```
3885  DD 7E 05     L3885:  LD   A,(IX+$05)      L3885: höger flyttal; vänster heltal?
3888  FE 01                CP   $01
388A  3E E0                LD   A,$E0
388C  37                   SCF
388D  C0                   RET  NZ              vänster sträng: fel 224
388E  3E 82                LD   A,$82           vänster heltal: ITOF in
3890  CD 97 38             CALL S3897
3893  06 00                LD   B,$00           B = 0: flyttal
3895  18 E8                JR   L387F
```

S3897: lägg in byten A efter den vänstra operanden (IX+6/IX+7 i CEXPR:s ram). INSCHR: lägg in byten A vid BC; koden från BC till DE - 1 flyttas en byte upp. Det måste finnas minst en byte att flytta, annars flyttar LDDR 64 kilobyte. In: A = byten, BC = platsen, DE = kodens slut. Ut: DE + 1. Förstör BC och F.

```
3897  DD 4E 06     S3897:  LD   C,(IX+$06)      S3897: där den högra operanden börjar (= efter den vänstra)
389A  DD 46 07               LD   B,(IX+$07)
389D  E5           INSCHR: PUSH HL
389E  62                   LD   H,D
389F  6B                   LD   L,E
38A0  A7                   AND  A
38A1  ED 42                SBC  HL,BC           BC = antalet bytes att flytta
38A3  44                   LD   B,H
38A4  4D                   LD   C,L
38A5  62                   LD   H,D
38A6  6B                   LD   L,E
38A7  2B                   DEC  HL              HL = kodens sista byte
38A8  D5                   PUSH DE
38A9  ED B8                LDDR                 en byte upp
38AB  12                   LD   (DE),A          byten in
38AC  D1                   POP  DE
38AD  13                   INC  DE
38AE  E1                   POP  HL
38AF  C9                   RET
```

3.25 Filsatserna vid körning $38B0

SOPEN/PREPAR: OPEN resp. PREPARE sträng AS FILE n. Finns inget FCB för n tas 23 bytes (ALLOC) och länkas in sist i listan, där FCBSRC slutade (prövat: efter OPEN "NUL:" AS FILE 2 och sedan FILE 1 går kedjan CON: -> fil 2 ($C000) -> fil 1 ($C017)); finns det stängs filen först (FCLOSE). Sedan OPENIX med A = 0 resp. 1. Fil 0 (CON:) kan inte öppnas om: OPEN/PREPARE ... AS FILE 0 gör ingenting. SCLOSE: CLOSE n, m, ... stänger de uppräknade filerna; fil 0 och filer som inte är öppna hoppas över. CLOSE utan argument stänger alla (S0D4A).

INPLIN: INPUT LINE: läs en rad i en buffert på stacken, lägg LF efter CR och tilldela strängen (CR och LF ingår i längden). Raden får ha 157 tecken; med 158 eller fler fel 20, och filen står kvar på radens början. Filslutet ger fel 34.

GET: GET #n, a$ [COUNT k] läser k bytes (standard 1, 0 ger tom sträng) som de står i blocket, även ETX; k större än variabelns maxlängd fel 137. Bortom filens sista block ger drivrutinens fel (37/38) vid INPUT LINE och GET.

PUT: PUT #n, a$ skriver strängen utan CR.

POSIT: POSIT #n, p ställer filen på byte p, 0 <= p < 2^24 (decimaler huggs av, annars fel 38): block p / 253, plats p MOD 253 + 3 (se DVCALL). Med DOUBLE tas 8 bytes av stacken. FPOSIT: funktionen POSIT(n) räknar tillbaka. Positionen räknas så för alla filer, men PRINT lämnar plats 255 åt ETX, så i en textfil från PRINT motsvarar POSIT 253 och framåt inte textens byte 253 ...

```
38B0  97           SOPEN:  SUB  A
38B1  21                   DEFB $21             LD HL,$013E (överlappar nästa)
38B2  3E 01        PREPAR: LD   A,$01
38B4  F5                   PUSH AF              A = 0 OPEN, 1 PREPARE
38B5  E7                   RST  $20             strängen på stacken
38B6  13                   INC  DE
38B7  E7                   RST  $20             L = filnumret
38B8  D5                   PUSH DE              programpekaren
38B9  7D                   LD   A,L
38BA  A7                   AND  A
38BB  28 36                JR   Z,L38F3         fil 0: ingenting
38BD  CD C4 11             CALL FCBSRC
```

```
38C0  28 16              JR   Z,L38D8          finns: stäng först
38C2  F5                 PUSH AF               filnumret
38C3  11 17 00           LD   DE,$0017         nytt FCB, 23 bytes
38C6  CD D3 0F           CALL ALLOC
38C9  DA 69 3B           JP   C,MEMERR
38CC  CD 25 24           CALL IXST0            länkas in sist: länken i det sista blocket (IX från FCBSRC)
38CF  E5                 PUSH HL               IX = det nya blocket
38D0  DD E1              POP  IX
38D2  F1                 POP  AF
38D3  DD 77 02           LD   (IX+$02),A       filnumret
38D6  18 03              JR   L38DB

38D8  CD 15 11     L38D8: CALL FCLOSE
38DB  21 02 00     L38DB: LD   HL,$0002        SP+2: strängens n
38DE  39                  ADD  HL,SP
38DF  5E                  LD   E,(HL)
38E0  23                  INC  HL
38E1  56                  LD   D,(HL)
38E2  23                  INC  HL
38E3  D5                  PUSH DE
38E4  5E                  LD   E,(HL)           strängens adress
38E5  23                  INC  HL
38E6  56                  LD   D,(HL)
38E7  CD DF 30            CALL S30DF            strängens adress och längd
38EA  EB                  EX   DE,HL
38EB  E3                  EX   (SP),HL
38EC  19                  ADD  HL,DE
38ED  23                  INC  HL
38EE  7E                  LD   A,(HL)           A = sättet, bakom strängen
38EF  E1                  POP  HL
38F0  CD 5F 10            CALL OPENIX           öppna
38F3  D1           L38F3: POP  DE               stacken tillbaka förbi strängen
38F4  E1                  POP  HL
38F5  C1                  POP  BC
38F6  C1                  POP  BC
38F7  39                  ADD  HL,SP
38F8  F9                  LD   SP,HL
38F9  E1                  POP  HL
38FA  C9                  RET
```

SCLOSE: `CLOSE` [*n, m, ...*]. *In: DE = programpekaren. Ut: DE efter satsen. Fel vid stängningen går till* `ERRA`. *Förstör AF, BC, HL och IX.*

```
38FB  1A           SCLOSE: LD   A,(DE)
38FC  FE C0                CP   $C0
38FE  38 17                JR   C,L3917          inget nummer: stäng alla
3900  E7           L3900:  RST  $20              filnumret
3901  7D                   LD   A,L
3902  A7                   AND  A
3903  28 0B                JR   Z,L3910          fil 0 hoppas över
3905  CD C4 11             CALL FCBSRC
3908  20 06                JR   NZ,L3910         inget FCB: hoppa över
390A  CD 15 11             CALL FCLOSE
390D  DA 12 00             JP   C,ERRA           fel vid stängningen
3910  1A           L3910:  LD   A,(DE)           fler efter komma?
3911  EE 2C                XOR  $2C
3913  C0                   RET  NZ
3914  13                   INC  DE
3915  18 E9                JR   L3900

3917  D5           L3917:  PUSH DE               utan nummer: stäng alla (CLSALL)
3918  CD 4A 0D             CALL S0D4A
391B  D1                   POP  DE
391C  C9                   RET
```

INPLIN: `INPUT LINE` #*n, a$. Bufferten* (`$A0` *bytes*) *läggs på stacken och blir en strängs data: n =* `$A0`, *adressen, längden.* `SSARR` *tilldelar och tar bort den. In: DE = programpekaren.*

```
391D  CD 24 20     INPLIN: CALL S2024            #n,
```

```
3920  E5              PUSH HL             filnumret
3921  E7              RST  $20
3922  E3              EX   (SP),HL
3923  7D              LD   A,L            A = filnumret
3924  CD 88 11        CALL FNDFIL
3927  21 60 FF        LD   HL,$FF60       buffert $A0 bytes på stacken
392A  39              ADD  HL,SP
392B  F9              LD   SP,HL
392C  AF              XOR  A              NC: första försöket
392D  DC 33 00  L392D: CALL C,ERRJP       fel: ERRJP, sedan ett nytt försök
3930  01 9F 00        LD   BC,$009F       högst 159 bytes inkl. CR
3933  E5              PUSH HL             bufferten
3934  D5              PUSH DE
3935  CD 51 49        CALL DVINP          läs raden
3938  D1              POP  DE
3939  E1              POP  HL
393A  38 F1           JR   C,L392D        fel: ERRJP
393C  E5              PUSH HL
393D  3E 0D           LD   A,$0D
393F  01 00 00        LD   BC,$0000       BC = 0: sök utan gräns
3942  ED B1           CPIR                leta upp CR
3944  36 0A           LD   (HL),$0A       LF efter
3946  79              LD   A,C
3947  ED 44           NEG
3949  3C              INC  A              längden = till och med LF
394A  6F              LD   L,A
394B  26 00           LD   H,$00
394D  E3              EX   (SP),HL        längden in, buffertens adress ut
394E  E5              PUSH HL             adressen
394F  21 A0 00        LD   HL,$00A0
3952  E5              PUSH HL             n
3953  C3 16 23        JP   SSARR          tilldela, bufferten städas
```

GET: GET *#n, a$* [COUNT *k*]. *Med k = 0 flyttar* SVSET *ingenting (BC = 0) och strängen blir tom. In: DE = programpekaren.*

```
3956  CD 24 20  GET:  CALL S2024          #n,
3959  65              LD   H,L            H = filnumret
395A  E5              PUSH HL
395B  E7              RST  $20
395C  E3              EX   (SP),HL
395D  E5              PUSH HL
395E  1A              LD   A,(DE)
395F  FE B9           CP   $B9            COUNT?
3961  01 01 00        LD   BC,$0001       standard 1
3964  20 04           JR   NZ,L396A
3966  13              INC  DE             förbi COUNT
3967  E7              RST  $20            k
3968  44              LD   B,H            BC = k
3969  4D              LD   C,L
396A  F1        L396A: POP  AF            A = filnumret
396B  CD 88 11        CALL FNDFIL
396E  DD E3           EX   (SP),IX
3970  D5              PUSH DE
3971  CD 50 24        CALL SVSET          variabeln tar BC bytes (fel 137)
3974  E1              POP  HL
3975  DD E1           POP  IX             IX = FCB:n
3977  E5              PUSH HL
3978  EB              EX   DE,HL
3979  CD A0 11        CALL DVGET          läs bytes rakt in
397C  D1              POP  DE
397D  C9              RET
```

PUT: PUT *#n, a$. Strängen ligger på stacken: n, adressen och längden; bakom dess n bytes data ligger filnumret. In: DE = programpekaren.*

```
397E  CD 24 20  PUT:  CALL S2024          #n,
3981  E5              PUSH HL             filnumret
3982  E7              RST  $20            strängen på stacken
3983  DD E1           POP  IX             IX = n
```

```
3985  E1              POP  HL          HL = strängens adress
3986  C1              POP  BC          BC = längden
3987  DD 39           ADD  IX,SP       IX = SP + n: filnumret
3989  DD 7E 00        LD   A,(IX+$00)  A = filnumret
398C  DD E5           PUSH IX          stacken efter strängen
398E  D5              PUSH DE
398F  CD 88 11        CALL FNDFIL
3992  CD 9D 11        CALL DVPUT       skriv strängen
3995  D1              POP  DE
3996  E1              POP  HL
3997  F9              LD   SP,HL       strängen bort
3998  C1              POP  BC          filnumret bort
3999  C9              RET
```

POSIT: `POSIT` *#n, p. p tas som flyttal: C = exponenten, B, H och L mantissan (24 bitar, den dolda ettan i bit 7 av B). Felet från* `DVEND` *prövas inte. Ut via* `S1278` *(programpekaren tillbaka).*

```
399A  CD 24 20     POSIT:  CALL S2024          #n, (egenhet)
399D  E5                   PUSH HL           filnumret
399E  E7                   RST  $20          p
399F  CD A3 1D             CALL S1DA3        programpekaren undan
39A2  C1                   POP  BC           C = exponenten, B = mantissans höga byte
39A3  D1                   POP  DE
39A4  63                   LD   H,E          HL = mantissans låga 16 bitar
39A5  6A                   LD   L,D
39A6  FD CB 1E 5E          BIT  3,(IY+$1E)   DOUBLE: 8 bytes
39AA  28 02                JR   Z,L39AE
39AC  F1                   POP  AF
39AD  F1                   POP  AF
39AE  79           L39AE:  LD   A,C          exponent 0: talet 0
39AF  A7                   AND  A
39B0  28 06                JR   Z,L39B8
39B2  CB 78                BIT  7,B          negativ: fel
39B4  20 11                JR   NZ,L39C7
39B6  CB F8                SET  7,B          den dolda ettan
39B8  FE 98        L39B8:  CP   $98          exponent $98: heltal i 24 bitar
39BA  28 0D                JR   Z,L39C9
39BC  30 09                JR   NC,L39C7
39BE  CB 38                SRL  B            skifta ned, decimalerna faller bort
39C0  CB 1C                RR   H
39C2  CB 1D                RR   L
39C4  3C                   INC  A
39C5  18 F1                JR   L39B8

39C7  D7           L39C7:  RST  $10          fel 38: negativ eller minst 2^24
39C8  26                   DEFB $26

39C9  58           L39C9:  LD   E,B          E = den höga byten, D = 0
39CA  01 FD 00             LD   BC,$00FD
39CD  50                   LD   D,B
39CE  CD 36 4B             CALL DIV32        dela med 253: block, plats
39D1  C1                   POP  BC           C = filnumret
39D2  E5                   PUSH HL           blocket undan
39D3  D5                   PUSH DE           platsen undan
39D4  79                   LD   A,C
39D5  CD 88 11             CALL FNDFIL
39D8  CD 82 49             CALL DVEND        en PRINT-fil avslutas; felet prövas inte
39DB  D1                   POP  DE
39DC  E1                   POP  HL
39DD  1C                   INC  E            plats + 3
39DE  1C                   INC  E
39DF  1C                   INC  E
39E0  CD 92 49             CALL POSST        ny position
39E3  C3 78 12             JP   S1278        programpekaren tillbaka
```

FPOSIT: funktionen `POSIT`*(n) = blocket * 253 + platsen - 3. Plats 0 (efter ett fullt block) ger blocket * 253 + 253, samma tal som nästa block, plats 3. Talet läggs som flyttal på stacken (8 bytes) och normaliseras i* `$352A`*.*

```
39E6  CD A3 1D     FPOSIT: CALL S1DA3        programpekaren undan
```

```
39E9  E1              POP  HL          L = filnumret
39EA  7D              LD   A,L
39EB  CD 88 11        CALL FNDFIL
39EE  CD 9C 49        CALL POSLD       HL = blocket, E = platsen
39F1  1D              DEC  E
39F2  1D              DEC  E
39F3  1D              DEC  E           platsen - 3
39F4  EB              EX   DE,HL       DE = blocket, L = platsen
39F5  63              LD   H,E
39F6  7A              LD   A,D         AHL = blocket * 256 + platsen
39F7  01 00 03        LD   BC,$0300    B = 3 varv, C = 0
39FA  A7        L39FA: AND  A           carry nollas
39FB  ED 52           SBC  HL,DE
39FD  99              SBC  A,C         lånet till den höga byten
39FE  10 FA           DJNZ L39FA       tre gånger blocket bort: blocket * 253 + platsen - 3
3A00  C5              PUSH BC          två nollord: de låga byten
3A01  C5              PUSH BC
3A02  4C              LD   C,H
3A03  45              LD   B,L
3A04  C5              PUSH BC          mantissans låga bytes
3A05  47              LD   B,A
3A06  0E 98           LD   C,$98       tillbaka som flyttal
3A08  C5              PUSH BC
3A09  C3 2A 35        JP   L352A       normalisera (avrundat till 4 bytes utan DOUBLE)
```

INP, CALL, POKE *och* OUT. INP *och* OUT *skriver kortvalet IY+$34 till port 1 före varje portåtkomst;* OUT *p,v med p:s låga byte = 1 (även 257) sparar v där. IY+$34 är 0 efter start och står kvar vid* RUN, CLEAR *och* NEW. OUT *tar par åtskilda med komma.*

CALL(a[,x])*: x i DE (0 utan argument, koden* $D2 *två bytes bakåt visar att det finns ett), hopp via* S1DBC*, HL tillbaka som heltal med tecken (*CALL(a,40000) *= -25536, 1.6 -> 2, 70000 fel 132, sträng fel 224).* POKE *a,v[,v...]: värdena till a, a+1 ..., bara låga byten (-1 -> 255, 256.7 -> 1); adressen -1 går, 65536 ger fel 132. DI/*EI *kring skrivningen.*

```
3A0C  C1        INP:  POP  BC          BC = porten
3A0D  FD 7E 34        LD   A,(IY+$34)  kortvalet först, sedan porten
3A10  D3 01           OUT  ($01),A
3A12  ED 68           IN   L,(C)       B = adressens höga byte
3A14  26 00           LD   H,$00
3A16  E5              PUSH HL          som heltal
3A17  EF              RST  $28
3A18  D5        FCALL:PUSH DE
3A19  1B              DEC  DE
3A1A  1B              DEC  DE
3A1B  1A              LD   A,(DE)
3A1C  FE D2           CP   $D2         två argument?
3A1E  C1              POP  BC          BC = programpekaren
3A1F  11 00 00        LD   DE,$0000    inget argument: DE = 0
3A22  20 01           JR   NZ,L3A25
3A24  D1              POP  DE          DE = x
3A25  E1        L3A25:POP  HL          HL = a
3A26  C5              PUSH BC
3A27  CD BC 1D        CALL S1DBC       JP (HL); HL tillbaka
3A2A  D1              POP  DE
3A2B  E5              PUSH HL          HL som heltal
3A2C  EF              RST  $28
3A2D  E7        POKE: RST  $20         adressen som heltal, 65536 fel 132
3A2E  13        L3A2E:INC  DE          förbi kommat
3A2F  E5              PUSH HL          adressen undan
3A30  E7              RST  $20         värdet
3A31  7D              LD   A,L         låga byten
3A32  E1              POP  HL
3A33  F3              DI
3A34  00              NOP              ABC800: OUT ($AA),A
3A35  00              NOP
3A36  77              LD   (HL),A
3A37  FB              EI
3A38  23              INC  HL
3A39  1A              LD   A,(DE)
3A3A  FE 2C           CP   $2C         fler värden till nästa adress
3A3C  28 F0           JR   Z,L3A2E
```

```
3A3E  C9                RET

3A3F  E7          OUTST: RST  $20             porten
3A40  13                 INC  DE              förbi kommat
3A41  E5                 PUSH HL
3A42  E7                 RST  $20             värdet
3A43  C1                 POP  BC              BC = porten
3A44  79                 LD   A,C
3A45  3D                 DEC  A               port 1 (låga byten)?
3A46  20 03              JR   NZ,L3A4B
3A48  FD 75 34           LD   (IY+$34),L      kortvalet sparas
3A4B  FD 7E 34    L3A4B: LD   A,(IY+$34)
3A4E  D3 01              OUT  ($01),A         port 1: kortvalet (OUT 1,v skriver alltså v två gånger)
3A50  ED 69              OUT  (C),L           värdet, B = adressens höga byte
3A52  1A                 LD   A,(DE)
3A53  FE 2C              CP   $2C
3A55  C0                 RET  NZ              fler par?
3A56  13                 INC  DE
3A57  18 E6              JR   OUTST
```

Lägessatserna verkar vid kompileringen (JPTABL grupp 2 $0B2D-$0B33, SINGLE/DOUBLE grupp 1 och vid länkningen $3309). 10 INTEGER gäller alltså från inmatningen av raden, inte från RUN: rader efter den lagras med heltalskonstanter (LIST visar 7%/2%). NEW och CLEAR behåller INTEGER och EXTEND.

SINGLE/DOUBLE: fel 211 om IY+$26 bit 3 (flyttalsvariabel finns) eller bit 4 (flyttals-COMMON) är satt, också när raden matas in; A=1 : DOUBLE går eftersom A skapas efter kompileringen. Sätter IY+$1E = 4/8 bytes och IY+$1F = 6/16 siffror.

DIGITS n: IY+$1F = låga byten, 0 ändrar inget; i praktiken högst 7 (SINGLE) eller 17 siffror (DOUBLE). NEW och SINGLE/DOUBLE återställer, CLEAR inte. OPTION BASE n: IY+$22 = låga byten (258 -> 2), nollas av RUN, CLEAR och NEW. In: INTEGR, FLOAT, EXTEND och NOEXT tar inget (C = bitarna som sätts i IY+$1D); SINGLE och DOUBLE läser IY+$26; DIGITS och OPTBAS körs i programmet, och RST $20 utför argumentets kod (HL = värdet). Ut: NC; SINGLE/DOUBLE vid fel CY och A = $D3 (211). Förstör AF, B och C; DIGITS och OPTBAS dessutom det som argumentets kod förstör.

```
3A59  0E 01       INTEGR: LD   C,$01(egenhet)
3A5B  18 02               JR   L3A5F

3A5D  0E 00       FLOAT:  LD   C,$00
3A5F  3E FE       L3A5F:  LD   A,$FE           IY+$1D bit 0 nollas och C läggs i (L3A61)
3A61  FD A6 1D    L3A61:  AND  (IY+$1D)
3A64  B1                  OR   C
3A65  FD 77 1D            LD   (IY+$1D),A
3A68  C9                  RET

3A69  0E 06       EXTEND: LD   C,$06
3A6B  18 02               JR   L3A6F

3A6D  0E 00       NOEXT:  LD   C,$00
3A6F  3E F9       L3A6F:  LD   A,$F9           IY+$1D bit 1-2
3A71  18 EE               JR   L3A61

3A73  AF          SINGLE: XOR  A               A = bit 2 som den ska bli
3A74  18 02               JR   S3A78

3A76  3E 04       DOUBLE: LD   A,$04
3A78  FD 46 26    S3A78:  LD   B,(IY+$26)
3A7B  4F                  LD   C,A
3A7C  A8                  XOR  B               skiljer bit 2 sig från nuvarande?
3A7D  E6 04               AND  $04             redan i det läget
3A7F  C8                  RET  Z
3A80  78                  LD   A,B             bit 3 och 4: flyttalsvariabler eller -COMMON
3A81  E6 18               AND  $18
3A83  3E D3               LD   A,$D3           fel 211: flyttalsvariabler finns
3A85  37                  SCF
3A86  C0                  RET  NZ
3A87  FD CB 26 96         RES  2,(IY+$26)      IY+$26 bit 2 = DOUBLE
3A8B  79                  LD   A,C
3A8C  E6 1C               AND  $1C
3A8E  FD B6 26    S3A8E:  OR   (IY+$26)
3A91  FD 77 26    S3A91:  LD   (IY+$26),A
3A94  CB 57               BIT  2,A             bytes per flyttal: 4 eller 8
3A96  3E 04               LD   A,$04
3A98  28 02               JR   Z,L3A9C
```

```
3A9A  3E 08              LD   A,$08
3A9C  FD 77 1E     L3A9C: LD   (IY+$1E),A
3A9F  FE 05              CP   $05            4 -> 6 siffror, 8 -> 16
3AA1  DE 00              SBC  A,$00
3AA3  87                 ADD  A,A
3AA4  FD 77 1F     L3AA4: LD   (IY+$1F),A
3AA7  C9                 RET

3AA8  E7           DIGITS: RST  $20(egenhet)
3AA9  7D                 LD   A,L
3AAA  A7                 AND  A
3AAB  C8                 RET  Z              0: ingen ändring
3AAC  18 F6              JR   L3AA4

3AAE  E7           OPTBAS: RST  $20
3AAF  FD 75 22           LD   (IY+$22),L     låga byten, ingen kontroll
3AB2  C9                 RET
```

3.26 NMICLK: realtidsklocka (NMI, 50 Hz) $3AB3

TICKS räknas upp; vid 50 (eller redan vid 49 när sekunderna & 31 = 0, dvs. sekund 0 och 32) nollställs den och CLKTIK stegar klockan. Klockan går bara om DAY <> 0 (är ställd). Alltså 2998 NMI per minut (49,97 Hz). ABC800 har samma rutin för CTC:n (93,75 Hz): 94 avbrott per sekund, 93 när sekunderna & 3 = 0, och slutar med EI / RETI i stället för RETN. Bevarar alla register.

```
3AB3  F5           NMICLK: PUSH AF(fel)
3AB4  E5                   PUSH HL
3AB5  21 F5 FF             LD   HL,$FFF5       TICKS ($FFF5)
3AB8  34                   INC  (HL)
3AB9  7E                   LD   A,(HL)
3ABA  FE 31                CP   $31            49 eller 50 NMI per sekund (ABC800: 93/94)
3ABC  38 12                JR   C,L3AD0
3ABE  20 07                JR   NZ,L3AC7       50: ny sekund
3AC0  3A F4 FF             LD   A,(SECOND)
3AC3  E6 1F                AND  $1F            49 räcker vid sekund 0 och 32
3AC5  20 09                JR   NZ,L3AD0
3AC7  C5           L3AC7:  PUSH BC
3AC8  01 00 1E             LD   BC,$1E00       B = 30 (dagarna), C = 0
3ACB  71                   LD   (HL),C         TICKS = 0
3ACC  CD D5 3A             CALL CLKTIK
3ACF  C1                   POP  BC
3AD0  E1           L3AD0:  POP  HL
3AD1  F1                   POP  AF
3AD2  ED 45                RETN

3AD4  00                   DEFB $00            rest: RETN är en byte kortare än EI / RETI
```

CLKTIK: sekund 0-59, minut 0-59, timme 0-23, dag 1-dagar i månaden (februari 28/29 med skottår när YEAR AND 3 = 0), månad 1-12, år 0-99. In: HL = TICKS ($FFF5), B = 30, C = 0. Förstör A, BC, HL och F.

```
3AD5  3A F1 FF     CLKTIK: LD   A,(DAY)
3AD8  A7                   AND  A
3AD9  C8                   RET  Z              klockan inte ställd
3ADA  2B                   DEC  HL             SECOND
3ADB  34                   INC  (HL)
3ADC  3E 3B                LD   A,$3B          59
3ADE  BE                   CP   (HL)
3ADF  D0                   RET  NC
3AE0  71                   LD   (HL),C         0
3AE1  2B                   DEC  HL             MINUTE
3AE2  34                   INC  (HL)
3AE3  BE                   CP   (HL)
3AE4  D0                   RET  NC
3AE5  71                   LD   (HL),C
3AE6  21 F2 FF             LD   HL,$FFF2       timmen ($FFF2)
3AE9  3E 17                LD   A,$17          23
3AEB  34                   INC  (HL)
3AEC  BE                   CP   (HL)
```

```
3AED  D0                RET  NC
3AEE  71                LD   (HL),C
3AEF  3A F0 FF          LD   A,(MONTH)      B = 30 här
3AF2  FE 02             CP   $02            februari?
3AF4  20 0B             JR   NZ,L3B01
3AF6  05                DEC  B              februari: 29
3AF7  4F                LD   C,A
3AF8  3A EF FF          LD   A,(YEAR)
3AFB  E6 03             AND  $03            delbart med 4?
3AFD  79                LD   A,C            månaden igen, Z orörd
3AFE  28 01             JR   Z,L3B01
3B00  05                DEC  B              inte skottår: 28
3B01  FE 08       L3B01: CP   $08            CY för månad 1–7
3B03  CE FF             ADC  A,$FF          8–12: en mindre
3B05  E6 01             AND  $01            1 = 31 dagar: udda 1–7, jämn 8–12
3B07  80                ADD  A,B            dagar i månaden
3B08  2B                DEC  HL             DAY
3B09  34                INC  (HL)
3B0A  BE                CP   (HL)
3B0B  D0                RET  NC
3B0C  36 01             LD   (HL),$01       dag 1
3B0E  2B                DEC  HL             MONTH
3B0F  34                INC  (HL)
3B10  3E 0C             LD   A,$0C          12
3B12  BE                CP   (HL)
3B13  D0                RET  NC
3B14  36 01             LD   (HL),$01       månad 1
3B16  2B                DEC  HL             YEAR
3B17  3E 63             LD   A,$63          99
3B19  34                INC  (HL)
3B1A  BE                CP   (HL)
3B1B  D0                RET  NC
3B1C  36 00             LD   (HL),$00       år 0
3B1E  C9                RET
```

CLKSTR (TIME$): *skriver* ÅÅÅÅ-MM-DD TT.MM.SS *på stacken, alltid 19 tecken; seklet är 19 för år 78-99 och 20 för 0-77. Läses om ifall minuten ändras under tiden (ett sekund- eller timskifte mitt i läsningen ändrar också minuten).*

Klockan ställs med POKE *65519,år,månad,dag,timme,minut,sekund (inget eget kommando);* DAY *= 0 stoppar den. År 99 går över till 0 (2000), och år 0 räknas som skottår. Värden utanför gränserna rättas först när fältet stegas.*

```
3B1F  21 EC FF    CLKSTR: LD   HL,$FFEC       20 byte för texten på stacken
3B22  01 13 00            LD   BC,$0013
3B25  39                  ADD  HL,SP
3B26  F9                  LD   SP,HL
3B27  C5                  PUSH BC             längden 19
3B28  E5                  PUSH HL             textens adress
3B29  03                  INC  BC
3B2A  C5                  PUSH BC             20 byte på stacken
3B2B  D5                  PUSH DE             koden undan
3B2C  E5          L3B2C:  PUSH HL             textens början
3B2D  3A F3 FF            LD   A,(MINUTE)
3B30  F5                  PUSH AF             minuten, jämförs efteråt
3B31  11 EF FF            LD   DE,$FFEF
3B34  01 63 3B            LD   BC,$3B63       BC = CLKSEP
3B37  1A                  LD   A,(DE)
3B38  C6 B2               ADD  A,$B2          år >= 78?
3B3A  9F                  SBC  A,A            19 eller 20
3B3B  C6 14               ADD  A,$14
3B3D  CD 56 3B            CALL PR2DIG
3B40  1A          L3B40:  LD   A,(DE)         nästa fält
3B41  13                  INC  DE
3B42  CD 56 3B            CALL PR2DIG
3B45  0A                  LD   A,(BC)         skiljetecknet efter fältet
3B46  03                  INC  BC
3B47  77                  LD   (HL),A
3B48  23                  INC  HL
3B49  A7                  AND  A
3B4A  20 F4               JR   NZ,L3B40       till och med nollan
```

```
3B4C  D1                POP  DE            D = minuten före
3B4D  E1                POP  HL
3B4E  3A F3 FF          LD   A,(MINUTE)
3B51  BA                CP   D             minuten ändrad: om igen
3B52  20 D8             JR   NZ,L3B2C
3B54  D1                POP  DE            koden
3B55  EF                RST  $28           nästa kod
```

PR2DIG: A (0-99) som två siffror vid HL; HL två steg fram. Förstör A och F.

```
3B56  36 2F        PR2DIG: LD   (HL),$2F
3B58  34           L3B58:  INC  (HL)
3B59  D6 0A                SUB  $0A
3B5B  30 FB                JR   NC,L3B58
3B5D  23                   INC  HL
3B5E  C6 3A                ADD  A,$3A
3B60  77                   LD   (HL),A
3B61  23                   INC  HL
3B62  C9                   RET

3B63  2D 2D 20 2E… CLKSEP: DEFM "-- .."
3B68  00                   DEFB $00
```

MEMERR: fel 201 (minnet räcker inte) när stacken inte går att lita på (SCAT, STRING$/SPACE$, FNCALL, PREPAR): SP sätts om till STKTOP. ERRSP: fel A som inte fångas och inte går att fortsätta. Meddelandet får radnummer (PRGPTR); i ett program blir det READY0, som nollar IY+$27 (CON-läget och ON ERROR), i direktläge READY, som behåller det.

```
3B69  3E C9        MEMERR: LD   A,$C9         fel 201
3B6B  2A 16 FF     ERRSP:  LD   HL,(STKTOP)
3B6E  F9                   LD   SP,HL         stacken om
3B6F  CD 81 3C             CALL S3C81         DE = PRGPTR: med radnummer
3B72  FD CB 23 56          BIT  2,(IY+$23)    direktläge?
3B76  20 62                JR   NZ,L3BDA
3B78  C3 0C 01             JP   READY0        program: CON och ON ERROR bort

3B7B  AF           STOP:   XOR  A             A = 0: Stop
3B7C  FD CB 23 86          RES  0,(IY+$23)    BREAK-biten
```

3.27 ERRHND: felhantering. A = felnummer (0 = BREAK/Stop). $3B80

STOP ($3B7B) kommer hit med A = 0 och nollar först BREAK-biten.

I direktläge blir det genast ERRSP (utskrift, READY). Annars sparas IX, BC, HL, DE, FRMSP, FNFRM, PRGPTR och AF på stacken.

Med ON ERROR GOTO aktivt (IY+$27 bit 1) och inte redan i felrutinen (bit 2): ERRNUM = A (ERR), bit 2 sätts, RESSP = SP (för RESUME), och körningen fortsätter vid ONEADR med FNFRM = ONEFRM (sparade vid ON ERROR GOTO). Det gäller när programmet har någon RESUME (IY+$26 bit 7, satt vid länkningen av SCNCOD); då står felläget kvar och ON ERROR gäller vidare. Utan RESUME går det som RESUME till ONEADR: felläget släpps, stacken sätts om och ON ERROR stängs av, så att nästa fel inte fångas om inte felrutinen gör ON ERROR GOTO igen. BREAK (A = 0) fångas aldrig.

Annars: CONSP = SP, IY+$27 bit 0 (går att fortsätta), meddelandet skrivs (ERRMSG), kroken ERRHK anropas och det blir READY.

CON (CONT, via CONCHK) sätter SP = CONSP, tar tillbaka registren och gör RET, dvs. ERRHND återvänder till den som anropade. Efter ERRJP fortsätter då körningen efter felet (fel som går att fortsätta); efter RST $10 kommer man till ERRA+3, där RST $10 $CF ger fel 207.

RESUME (RESCHK: fel 151 om bit 2 inte är satt) går tillbaka till RESSP; utan radnummer fortsätter körningen efter felet, som vid CON (X=1/0 ger X = det största talet), men efter ett RST $10-fel blir det fel 207; med radnummer dit (FNDLIN, fel 180; INDEF, fel 181). Fel som går via ERRSP (RETURN utan GOSUB, GOSUB med full stack, RESUME utan fel) fångas inte.

```
3B80  DD E5        ERRHND: PUSH IX
3B82  C5                   PUSH BC
3B83  FD CB 23 56          BIT  2,(IY+$23)
3B87  20 E2                JR   NZ,ERRSP      direktläge: inget att fortsätta
3B89  E5                   PUSH HL            registren och lägena sparas
3B8A  D5                   PUSH DE
```

```
3B8B  2A 45 FF            LD   HL,(FRMSP)
3B8E  E5                  PUSH HL
3B8F  2A 18 FF            LD   HL,(FNFRM)
3B92  E5                  PUSH HL
3B93  2A 3A FF            LD   HL,(PRGPTR)
3B96  E5                  PUSH HL
3B97  F5                  PUSH AF                 felnumret vid SP
3B98  FD 4E 27            LD   C,(IY+$27)
3B9B  CB 51               BIT  2,C
3B9D  20 29               JR   NZ,L3BC8           redan i felrutinen
3B9F  CB 49               BIT  1,C
3BA1  28 25               JR   Z,L3BC8            ingen ON ERROR
3BA3  A7                  AND  A
3BA4  28 22               JR   Z,L3BC8            Stop fångas inte
3BA6  FD 77 44            LD   (IY+$44),A         ERR = felnumret
3BA9  FD CB 27 D6         SET  2,(IY+$27)         i felrutinen
3BAD  ED 73 14 FF         LD   (RESSP),SP         för RESUME
3BB1  ED 5B 42 FF         LD   DE,(ONEADR)        dit ON ERROR GOTO pekar
3BB5  2A 04 FF            LD   HL,(ONEFRM)        FN-ramen vid ON ERROR GOTO
3BB8  22 18 FF            LD   (FNFRM),HL
3BBB  FD CB 26 7E         BIT  7,(IY+$26)(egenhet)
3BBF  C2 07 1D            JP   NZ,RUNLP           programmet har RESUME: felläget kvar
3BC2  FD CB 27 8E         RES  1,(IY+$27)         annars: ON ERROR av, som RESUME nr
3BC6  18 60               JR   L3C28

3BC8  F1            L3BC8: POP  AF
3BC9  D1                   POP  DE                DE = PRGPTR (radnumret)
3BCA  D5                   PUSH DE
3BCB  F5                   PUSH AF
3BCC  ED 73 12 FF          LD   (CONSP),SP        för CON
3BD0  FD CB 27 C6          SET  0,(IY+$27)        går att fortsätta
3BD4  CD 84 3C             CALL ERRMSG            skriv felet
3BD7  CD 93 FF             CALL ERRHK             kroken (normalt RET)
3BDA  C3 10 01      L3BDA: JP   READY
```

CONT (CON): CONCHK ger HL = CONSP. Registren och lägena som ERRHND sparade tas tillbaka, och ERRHND återvänder till den som anropade den. RESUME utan nummer kommer till L3BE0 med HL = RESSP.

```
3BDD  CD F4 3B      CONT:  CALL CONCHK
3BE0  F9            L3BE0: LD   SP,HL              SP = CONSP eller RESSP
3BE1  F1                   POP  AF                 felnumret
3BE2  E1                   POP  HL
3BE3  22 3A FF             LD   (PRGPTR),HL
3BE6  E1                   POP  HL                 FN-ramen
3BE7  22 18 FF             LD   (FNFRM),HL
3BEA  E1                   POP  HL                 FRMSP
3BEB  22 45 FF             LD   (FRMSP),HL
3BEE  D1                   POP  DE
3BEF  E1                   POP  HL
3BF0  C1                   POP  BC
3BF1  DD E1                POP  IX
3BF3  C9                   RET                     tillbaka efter felet
```

CONCHK: kräver CON-läget (IY+$27 bit 0, annars fel 207 via ERRSP), släpper det och lämnar direktläget (IY+$23 bit 2 = 0). Ut: HL = CONSP. Förstör A, E och F.

```
3BF4  CD E8 3C      CONCHK: CALL CRLFC
3BF7  FD CB 27 46           BIT  0,(IY+$27)
3BFB  3E CF                 LD   A,$CF              fel 207: inget att fortsätta
3BFD  CA 6B 3B              JP   Z,ERRSP
3C00  FD CB 27 86           RES  0,(IY+$27)         CON-läget av
3C04  2A 12 FF              LD   HL,(CONSP)
3C07  FD CB 23 96           RES  2,(IY+$23)         inte längre direktläge
3C0B  C9                    RET
```

Direktkommandona RESUME och GOTO. RESUME utan nummer: CONCHK (programmet måste stå vid STOP eller ett fel, annars 207), sedan RESCHK (felrutinen måste vara igång, annars 151) och tillbaka som CON. RESUME n och GOTO n: GETLNR (210), FNDLIN (180), INDEF (181, raden ligger i en DEF), sedan CONCHK; RESUME n prövar därefter RESCHK och går vidare som satsen RESUME,

GOTO n tar SP = CONSP och fortsätter vid raden med variablerna kvar. Fel 180/181/210 lämnar CON-läget orört, men ett RESUME som faller på RESCHK har redan släppt det: `Error 151 in line m` *och sedan går inte CON.*

```
3C0C  7E           CMDRES: LD   A,(HL)
3C0D  D6 30                SUB  $30
3C0F  D6 0A                SUB  $0A              CY för en siffra
3C11  38 4D                JR   C,L3C60          siffra: RESUME rad
3C13  FD CB 23 56  RESUM0: BIT  2,(IY+$23)
3C17  C4 F4 3B             CALL NZ,CONCHK        direktläge: CON-läget krävs
3C1A  CD 49 3C             CALL RESCHK(egenhet)
3C1D  30 C1                JR   NC,L3BE0         som CON
3C1F  C3 6B 3B     L3C1F:  JP   ERRSP
```

RESUME n (satsen): DE pekar på radens relativa adress i koden. RESCHK (fel 151) ger HL = RESSP; stacken sätts dit, och har felet kommit i en FN-funktion som anropats efter ON ERROR GOTO tas dess ramar bort tills FNFRM:s ram är nådd. Sedan sätts SP = FRMSP och körningen fortsätter vid raden (L1D11). GOTO n i CON-läget går samma väg från CONSP.

```
3C22  EB           RESUME: EX   DE,HL
3C23  5E                   LD   E,(HL)
3C24  23                   INC  HL
3C25  56                   LD   D,(HL)
3C26  19                   ADD  HL,DE            radens adress (relativ)
3C27  EB           L3C27:  EX   DE,HL
3C28  CD 49 3C     L3C28:  CALL RESCHK(egenhet)
3C2B  38 F2                JR   C,L3C1F          inget fel väntar: 151
3C2D  F9           L3C2D:  LD   SP,HL
3C2E  E1                   POP  HL               AF
3C2F  E1                   POP  HL               PRGPTR
3C30  C1           L3C30:  POP  BC               FN-ramar på stacken tas bort
3C31  2A 18 FF             LD   HL,(FNFRM)
3C34  ED 42                SBC  HL,BC
3C36  20 05                JR   NZ,L3C3D         ramen till ON ERROR nådd
3C38  E1                   POP  HL               FRMSP
3C39  F9                   LD   SP,HL
3C3A  C3 11 1D             JP   L1D11            till raden DE

3C3D  04           L3C3D:  INC  B
3C3E  05                   DEC  B
3C3F  28 DE                JR   Z,L3C1F          ingen ram kvar: fel 151
3C41  21 F8 FF             LD   HL,$FFF8         SP = ramens adress - 8
3C44  09                   ADD  HL,BC
3C45  F9                   LD   SP,HL
3C46  BF                   CP   A                CY = 0
3C47  18 E7                JR   L3C30
```

RESCHK: felrutinen måste vara igång (IY+$27 bit 2). Då släpps felläget och HL = RESSP, NC. Annars CY. A = 151 i båda fallen. Förstör F.

```
3C49  FD CB 27 56  RESCHK: BIT  2,(IY+$27)
3C4D  3E 97                LD   A,$97            fel 151: RESUME utan fel
3C4F  37                   SCF
3C50  C8                   RET  Z
3C51  FD CB 27 96          RES  2,(IY+$27)       felläget av
3C55  2A 14 FF             LD   HL,(RESSP)
3C58  BF                   CP   A
3C59  C9                   RET
```

CMDGO (GOTO n i direktläge): S3C66, sedan som RESUME n men från CONSP, utan RESCHK. Fel: tillbaka med CY och A = felet.

```
3C5A  CD 66 3C     CMDGO:  CALL S3C66
3C5D  D8                   RET  C
3C5E  18 CD                JR   L3C2D

3C60  CD 66 3C     L3C60:  CALL S3C66
3C63  D8                   RET  C
3C64  18 C2                JR   L3C28
```

S3C66: radnumret vid HL. Ut: NC, DE = radens adress - 1 och HL = CONSP (CONCHK). Fel: CY och A = 210, 180 eller 181,

CON-läget orört. Utan CON-läge blir det fel 207 (ERRSP).

```
3C66  CD C2 4B     S3C66:  CALL GETLNR
3C69  3E D2                LD   A,$D2          fel 210: radnummer väntas
3C6B  D8                   RET  C
3C6C  CD 8C 32             CALL FNDLIN
3C6F  3E B4                LD   A,$B4          fel 180: raden finns inte
3C71  37                   SCF
3C72  C0                   RET  NZ
3C73  CD 01 31             CALL INDEF
3C76  3E B5                LD   A,$B5          fel 181: raden ligger i en DEF
3C78  37                   SCF
3C79  C0                   RET  NZ
3C7A  C5                   PUSH BC
3C7B  CD F4 3B             CALL CONCHK         CON-läget av först nu
3C7E  D1                   POP  DE
3C7F  1B                   DEC  DE             före raden
3C80  C9                   RET
```

S3C81: DE = PRGPTR, sedan ERRMSG.

```
3C81  CD 78 12     S3C81:  CALL S1278
```

ERRMSG: bygger texten på stacken: BEL, `Stop` eller `Error n` (eller text från kroken ERRTXV), `in line m` om radnummer finns, sist .. Skrivs till fil 0, sedan CRLFC. Ingångar: S3C81 radnumret ur PRGPTR, ERRMSG ur DE (LINNR), S3C8A utan, S3C8D med HL.

```
3C84  F5           ERRMSG: PUSH AF
3C85  CD 03 3D             CALL LINNR          HL = radnumret
3C88  18 04                JR   L3C8E

3C8A  21 00 00     S3C8A:  LD   HL,$0000       utan radnummer
3C8D  F5           S3C8D:  PUSH AF
3C8E  CD E8 3C     L3C8E:  CALL CRLFC
3C91  F1                   POP  AF
3C92  EB                   EX   DE,HL          DE = radnumret
3C93  21 B0 FF             LD   HL,$FFB0       80 bytes på stacken
3C96  39                   ADD  HL,SP
3C97  F9                   LD   SP,HL
3C98  E5                   PUSH HL             buffertens början
3C99  36 07                LD   (HL),$07       BEL
3C9B  23                   INC  HL
3C9C  EB                   EX   DE,HL
3C9D  E5                   PUSH HL             radnumret undan
3C9E  A7                   AND  A
3C9F  20 0A                JR   NZ,L3CAB
3CA1  21 55 3D             LD   HL,$3D55       A = 0: Stop
3CA4  01 04 00             LD   BC,$0004
3CA7  ED B0                LDIR
3CA9  18 18                JR   L3CC3

3CAB  2A AC FF     L3CAB:  LD   HL,(ERRTXV)
3CAE  24                   INC  H
3CAF  25                   DEC  H
3CB0  37                   SCF
3CB1  C4 BC 1D             CALL NZ,S1DBC       kroken ERRTXV, NC = texten skriven
3CB4  30 0D                JR   NC,L3CC3
3CB6  21 59 3D             LD   HL,$3D59       Error  och numret
3CB9  01 06 00             LD   BC,$0006
3CBC  ED B0                LDIR
3CBE  6F                   LD   L,A            HL = felnumret (B = 0)
3CBF  60                   LD   H,B
3CC0  CD 13 45             CALL DECSTR         numret som text
3CC3  E1           L3CC3:  POP  HL             radnumret
3CC4  7C                   LD   A,H
3CC5  B5                   OR   L
3CC6  28 0D                JR   Z,L3CD5        0: inget radnummer
3CC8  E5                   PUSH HL
3CC9  21 4C 3D             LD   HL,$3D4C        in line  och numret
3CCC  01 09 00             LD   BC,$0009
3CCF  ED B0                LDIR
```

```
3CD1  E1                POP  HL
3CD2  CD 13 45          CALL DECSTR
3CD5  E1          L3CD5: POP  HL            buffertens början
3CD6  EB                EX   DE,HL
3CD7  36 2E             LD   (HL),$2E       . sist
3CD9  23                INC  HL
3CDA  B7                OR   A              BC = textens längd
3CDB  ED 52             SBC  HL,DE
3CDD  4D                LD   C,L
3CDE  44                LD   B,H
3CDF  EB                EX   DE,HL
3CE0  CD 96 11          CALL PRCON
3CE3  21 50 00          LD   HL,$0050
3CE6  39                ADD  HL,SP          släpp bufferten, sedan CRLFC
3CE7  F9                LD   SP,HL
3CE8  CD 79 11    CRLFC: CALL GETPS0         A = kolumnen
3CEB  A7                AND  A
3CEC  C8                RET  Z              kolumn 0: ingen ny rad
```

CRLF: skriv CR LF på fil 0. CRLFC gör det bara om kolumnen <> 0. Bevarar DE, HL och IX; förstör A, BC och F. CRLFC förstör också E.

```
3CED  D5          CRLF: PUSH DE
3CEE  E5                PUSH HL
3CEF  DD E5             PUSH IX
3CF1  CD FC 3C          CALL S3CFC
3CF4  CD 96 11          CALL PRCON
3CF7  DD E1             POP  IX
3CF9  E1                POP  HL
3CFA  D1                POP  DE
3CFB  C9                RET

3CFC  21 26 00    S3CFC: LD   HL,$0026       CR LF på $0026
3CFF  01 02 00           LD   BC,$0002
3D02  C9                 RET
```

LINNR: HL = radnumret för raden som innehåller adressen DE (0 i direktläge). Raderna stegas från FSTLIN med längdbyten; $87 före varje rad. Ligger DE utanför programmet följs FN-ramarna från FNFRM till anroparens adress (ingen ram: 0). Ett fel i en enradig DEF anges alltså på DEF-raden, i en flerradig på raden i funktionen. Förstör A, BC, DE och F.

```
3D03  ED 4B 18 FF LINNR: LD   BC,(FNFRM)
3D07  FD CB 23 56        BIT  2,(IY+$23)
3D0B  21 00 00    L3D0B: LD   HL,$0000
3D0E  C0                 RET  NZ            direktläge: 0
3D0F  2A 08 FF    L3D0F: LD   HL,(PRGEND)
3D12  A7                 AND  A
3D13  ED 52              SBC  HL,DE
3D15  38 22              JR   C,L3D39       efter programmet
3D17  13                 INC  DE
3D18  CD 44 00           CALL FSTLIN
3D1B  A7                 AND  A
3D1C  ED 52              SBC  HL,DE
3D1E  19                 ADD  HL,DE
3D1F  30 18              JR   NC,L3D39      före första raden
3D21  7E                 LD   A,(HL)
3D22  EE 87       L3D22: XOR  $87           $87 = radstart
3D24  23                 INC  HL
3D25  20 E4              JR   NZ,L3D0B
3D27  47                 LD   B,A
3D28  4E          L3D28: LD   C,(HL)
3D29  09                 ADD  HL,BC         nästa sats/rad
3D2A  0D                 DEC  C
3D2B  28 F5              JR   Z,L3D22
3D2D  ED 52              SBC  HL,DE
3D2F  19                 ADD  HL,DE
3D30  38 F6              JR   C,L3D28       inte förbi DE än
3D32  ED 42              SBC  HL,BC
3D34  7E                 LD   A,(HL)        radnumret före
3D35  23                 INC  HL
```

```
3D36  66              LD   H,(HL)
3D37  6F              LD   L,A
3D38  C9              RET

3D39  60        L3D39: LD   H,B
3D3A  69               LD   L,C
3D3B  78               LD   A,B
3D3C  B1               OR   C
3D3D  C8               RET  Z              ingen FN-ram: 0
3D3E  CD 44 32         CALL S3244
3D41  CD 8E 30         CALL S308E
3D44  18 C9            JR   L3D0F          anroparens adress i ramen
```

PROMPT: fyra texter i följd: ABC800 ($3D46, 6 tecken, READY), in line med blanka före och efter ($3D4C, 9), Stop ($3D55, 4) och Error med en blank efter ($3D59, 6), de tre sista för ERRMSG.

```
3D46  41 42 43 38… PROMPT: DEFM "ABC800 in line StopError "
```

DSKIP: koderna $AA-$AE i LIST (DECTAB): byten efter koden hoppas över, och DECOMP fortsätter med nästa kod.

```
3D5F  23           DSKIP:  INC  HL
```

3.28 DECOMP: LIST, översätt uttryckskoden tillbaka till text $3D60

HL = koden, DE = utbufferten. Koden är omvänd polsk notation: operanderna kommer före operatorn, så texten byggs i ordning men operatorer och namn sätts in efteråt. Varje operand lägger sin startadress i bufferten på stacken (ett ”fragment”). En operator skriver sin text sist och DMERGE roterar den på plats före det förra fragmentets början (INSCHR, en byte i taget); två fragment blir ett. Parenteser står i koden ($A7) och kommer tillbaka.

```
$DD-$FF  heltal 0-34 (DSMALL)       $D5-$DC flyttal, 8 bytes (DNUM)
$D0-$D4  funktion och (arg,...)     $A7-$CF via DECTAB ($3FED)
$89-$A6  räkneoperator, jämförelse  $84-$88 EQV IMP OR XOR AND
```

Operatorernas text ur OPKW ($06C8); en kod som inte står där (andra typen av samma operator) söks som koden - 1, ** blir därför ^.

EQV ... AND och NOT får mellanslag före och efter, men bara om inget mellanslag står där redan (även efter ; och (: ; NOT B).

DECTAB: $A7 parentes, $A8/$A9 unärt minus, $AA-$AE FN-märke (en byte hoppas över), $AF NOT, $B0/$C3 tomt fragment (DIM utan storlek), $B1-$B3 = mellan mål och värde, $B4-$B6 tilldelning till en enkel variabel, $B7-$BC och $BF uttryckets slut, $BD THEN, $BE IF utan THEN, $C0-$C6 variabel, $C7 heltal, $C8-$CA fältelement, $CB ”sträng”, $CC ’sträng’, $CD DIM- och $CE COMMON-post, $CF FN. Flyttal skrivs som PRINT gör men med 16-17 siffror, heltal med %; i INTEGER-läget (IY+$1D bit 0) heltal utan % och flyttal med . . DCHK ($4250) lägger varje tecken genom EMIT: blir raden längre än bufferten (VFF1A) ges fel 20, fast raden godtogs vid inmatningen (1E6 blir 1.E+06). In: HL = koden, DE = platsen i bufferten. Ut: HL efter uttryckets slutkod, DE efter texten (DDROP, DTHEN, DASGN; DDIM/DCOMMN vid listans slut). Förstör AF, BC, IX och de skuggade registren (talutskriften i DNUM).

```
3D60  CD 50 42     DECOMP: CALL DCHK
3D63  7E                   LD   A,(HL)
3D64  23                   INC  HL
3D65  FE DD                CP   $DD            $DD-: heltal 0-34
3D67  D2 B6 3E             JP   NC,DSMALL
3D6A  FE D5                CP   $D5            $D5-$DC: flyttal
3D6C  D2 65 3E             JP   NC,DNUM
3D6F  FE D0                CP   $D0            $D0-$D4: funktion
3D71  D2 D8 3D             JP   NC,DFUNC
3D74  D6 A7                SUB  $A7
3D76  01 ED 3F             LD   BC,$3FED       $A7-$CF via DECTAB
3D79  D2 B7 13             JP   NC,JPTAB
3D7C  C6 A7                ADD  A,$A7
3D7E  FE 89                CP   $89            $89-$A6: utan mellanslag
3D80  D2 24 3E             JP   NC,DOP
3D83  FE 84                CP   $84            $84-$88: med mellanslag
3D85  D2 1D 3E             JP   NC,DOPSP
3D88  18 D6                JR   DECOMP         under $84: hoppas över
```

Rutinerna i DECTAB*, hopp från* DECOMP *genom* JPTAB*. In: HL efter koden, DE efter texten, och vid SP de påbörjade fragmentens början, det senaste närmast SP (därunder returadressen till den som anropade* DECOMP*). En operand lägger ett fragment* (PUSH DE) *och går tillbaka till* DECOMP*; en operator gör två fragment till ett. Uttrycket slutar med ett fragment kvar:* DDROP ($B7-$BC, $BF) *tar bort det och återvänder* (RET) *till den som anropade* DECOMP*.* DTHEN ($BD) *skriver THEN först, med texten ur STKW* ($0A1C)*, och hoppar som* $BE (L3D95) *över avståndsbyten efter koden.* DASGN1 ($B1-$B3) *sätter = före värdets fragment* (S3F70) *och slutar som* DDROP*.*

```
3D8A  CD 3C 42     DTHEN:  CALL DBLANK          THEN ($0A1C), avståndet bort
3D8D  E5                   PUSH HL
3D8E  21 1C 0A             LD   HL,$0A1C
3D91  CD CC 3F             CALL S3FCC           texten fram till nästa kod
3D94  E1                   POP  HL
3D95  23           L3D95:  INC  HL
3D96  18 05                JR   DDROP

3D98  3E 3D        DASGN1: LD   A,$3D
3D9A  CD 70 3F             CALL S3F70           = vid värdets början, fragmentet bort
3D9D  C1           DDROP:  POP  BC              uttryckets slut: tillbaka
3D9E  C9                   RET
```

DPAREN ($A7)*: (och) runt det senaste fragmentet.* DARR ($C8-$CA)*: fältelement; byten efter koden är antalet index,* DCOMMA *sätter komman mellan deras fragment, och namnet och (flyttas före dem* (L3DA0)*.* DVAR ($C0-$C2, $C4-$C6)*: variabelns namn blir ett fragment.* DASGN ($B4-$B6)*: tilldelning till en enkel variabel; namn= flyttas före värdet och uttrycket slutar.* DVNAME*: namnet ur ordet vid HL (*S3072 *med sorten A = 10, så att en länk blir namnet igen), sedan* NAMTX1*; ett nollord ger ingen text (*S3DCF *med A = 7 för NEXT,* $0B *för FN). Ut: HL efter ordet, DE efter texten. Förstör A och BC.*

```
3D9F  D5           DPAREN: PUSH DE              ( ) runt senaste fragmentet
3DA0  3E 28        L3DA0:  LD   A,$28
3DA2  12                   LD   (DE),A
3DA3  13                   INC  DE
3DA4  CD CF 3F             CALL DMERGE          ( flyttas före fragmentet
3DA7  C1                   POP  BC
3DA8  3E 29                LD   A,$29           ) sist
3DAA  12                   LD   (DE),A
3DAB  13                   INC  DE
3DAC  18 B2        L3DAC:  JR   DECOMP

3DAE  7E           DARR:   LD   A,(HL)          A = antal index
3DAF  23                   INC  HL
3DB0  CD 78 3F             CALL DCOMMA
3DB3  D5                   PUSH DE
3DB4  CD CD 3D             CALL DVNAME          namnet före
3DB7  18 E7                JR   L3DA0

3DB9  D5           DVAR:   PUSH DE
3DBA  CD CD 3D             CALL DVNAME
3DBD  18 ED                JR   L3DAC

3DBF  D5           DASGN:  PUSH DE              namn= före värdet
3DC0  CD CD 3D             CALL DVNAME
3DC3  3E 3D                LD   A,$3D
3DC5  12                   LD   (DE),A
3DC6  13                   INC  DE
3DC7  CD CF 3F             CALL DMERGE          namn= före värdet
3DCA  C1                   POP  BC
3DCB  C1                   POP  BC
3DCC  C9                   RET

3DCD  3E 0A        DVNAME: LD   A,$0A           ordet vid HL: namnet som text
3DCF  CD 72 30     S3DCF:  CALL S3072
3DD2  78                   LD   A,B
3DD3  B1                   OR   C
3DD4  C8                   RET  Z
3DD5  C3 C3 33             JP   NAMTX1          ordet BC som namn
```

DFUNC ($D0-$D4)*: inbyggd funktion.* $D0-$D3 *ger antalet argument,* $D4 *tar det ur nästa byte (också* $D0 + *antal); sedan funktionens kod (*DFNAME*). Utan argument skrivs bara namnet, annars komman mellan argumenten, namnet och (före dem och) efter (*L3DA0*).* DFN ($CF)*: FN-anrop; antalet argument och en byte till, sedan ordet med namnet (A =* $0B*,* S3DCF*).* S3E09 *skriver FN och namnet.*

```
3DD8  D6 D0        DFUNC:  SUB  $D0
3DDA  FE 04                CP   $04          $D0-$D3: antal argument
3DDC  38 04                JR   C,L3DE2
3DDE  7E                   LD   A,(HL)       $D4: antalet i nästa byte
3DDF  D6 D0                SUB  $D0
3DE1  23                   INC  HL
3DE2  A7           L3DE2:  AND  A
3DE3  28 09                JR   Z,L3DEE      utan argument: bara namnet
3DE5  CD 78 3F             CALL DCOMMA
3DE8  D5                   PUSH DE
3DE9  CD A6 3F             CALL DFNAME
3DEC  18 B2                JR   L3DA0

3DEE  D5           L3DEE:  PUSH DE
3DEF  CD A6 3F             CALL DFNAME
3DF2  18 B8                JR   L3DAC

3DF4  7E           DFN:    LD   A,(HL)       antal argument, en byte till
3DF5  23                   INC  HL
3DF6  23                   INC  HL
3DF7  A7                   AND  A
3DF8  28 09                JR   Z,L3E03
3DFA  CD 78 3F             CALL DCOMMA
3DFD  D5                   PUSH DE
3DFE  CD 09 3E             CALL S3E09
3E01  18 9D                JR   L3DA0

3E03  D5           L3E03:  PUSH DE
3E04  CD 09 3E             CALL S3E09
3E07  18 A3        L3E07:  JR   L3DAC

3E09  3E 46        S3E09:  LD   A,$46        FN och namnet
3E0B  12                   LD   (DE),A
3E0C  13                   INC  DE
3E0D  3E 4E                LD   A,$4E
3E0F  12                   LD   (DE),A
3E10  13                   INC  DE
3E11  3E 0B                LD   A,$0B
3E13  18 BA                JR   S3DCF
```

DNOT ($AF), DOPSP ($84-$88) *och* DOP ($89-$A6)*: operatorerna. IX pekar på en flaggbyte före ingången: bit 0 mellanslag före och efter, bit 1 bara en operand.* DOPR *skriver operatorns text sist i bufferten* (DOPTXT *med koden vid HL - 1*)*, med mellanslag bara där det inte redan står ett, och* DMERGE *flyttar texten före den högra operanden. Med två operander tas den högras fragment bort, så att den vänstra omfattar alltsammans; NOT behåller sitt fragment. Förstör A, BC och IX.*

```
3E15  03                   DEFB $03
3E16  DD 21 15 3E  DNOT:   LD   IX,$3E15     bit 0 mellanslag, bit 1 en operand
3E1A  18 0C                JR   DOPR

3E1C  01                   DEFB $01
3E1D  DD 21 1C 3E  DOPSP:  LD   IX,$3E1C
3E21  18 05                JR   DOPR

3E23  00                   DEFB $00
3E24  DD 21 23 3E  DOP:    LD   IX,$3E23
3E28  C1           DOPR:   POP  BC           skriv operatorn, mellanslag styrs av IX
3E29  C5                   PUSH BC
3E2A  D5                   PUSH DE
3E2B  DD CB 00 46          BIT  0,(IX+$00)   mellanslag före om inget står där (egenhet)
3E2F  28 0B                JR   Z,L3E3C
3E31  C5                   PUSH BC
3E32  0B                   DEC  BC
3E33  0A                   LD   A,(BC)       tecknet före den högra operanden
3E34  FE 20                CP   $20
3E36  28 04                JR   Z,L3E3C
3E38  3E 20                LD   A,$20
3E3A  12                   LD   (DE),A
3E3B  13                   INC  DE
3E3C  2B           L3E3C:  DEC  HL
3E3D  7E                   LD   A,(HL)
3E3E  23                   INC  HL
```

```
3E3F  CD 89 3F            CALL DOPTXT          operatorns text sist
3E42  DD CB 00 46         BIT  0,(IX+$00)
3E46  28 0A               JR   Z,L3E52
3E48  C1                  POP  BC
3E49  0A                  LD   A,(BC)          högra operandens första tecken
3E4A  FE 20               CP   $20
3E4C  28 04               JR   Z,L3E52
3E4E  3E 20               LD   A,$20
3E50  12                  LD   (DE),A
3E51  13                  INC  DE
3E52  DD CB 00 4E  L3E52: BIT  1,(IX+$00)      NOT: bara ett fragment
3E56  20 07               JR   NZ,L3E5F
3E58  CD CF 3F            CALL DMERGE
3E5B  C1                  POP  BC
3E5C  C1                  POP  BC
3E5D  18 A8               JR   L3E07

3E5F  CD CF 3F     L3E5F: CALL DMERGE          NOT: operatorn först i fragmentet
3E62  C1                  POP  BC
3E63  18 A2        L3E63: JR   L3E07
```

DNUM ($D5-$DC): *flyttal med A - $D4 bytes i koden (NUMPSH fyller ut till 8). Talet skrivs med FTODEC och PRINT-rutinerna med IY+$1E = 8 under tiden, med 17 siffror när den första är 0-4 och annars 16, och avrundas som i PRINT (prövat: 0.1 -> .1, 3.14159 och 9.99 står kvar). I INTEGER-läget (C = 3) får ett tal utan decimaler en punkt efter (C=2.0 -> C=2.), så att det inte blir ett heltal när raden matas in igen.* DSMALL ($DD-$FF)*: heltalen 0-34 i en byte;* DINT ($C7)*: heltal i ett ord. Båda skrivs med DECSTR och % efter, utom i INTEGER-läget.*

```
3E65  D5           DNUM:  PUSH DE
3E66  D6 D4               SUB  $D4
3E68  0E 02               LD   C,$02           C = 3 i INTEGER-läget: . efter
3E6A  FD CB 1D 46         BIT  0,(IY+$1D)
3E6E  28 01               JR   Z,L3E71
3E70  0C                  INC  C
3E71  FD 46 1E     L3E71: LD   B,(IY+$1E)
3E74  FD 36 1E 08         LD   (IY+$1E),$08    8 bytes medan talet skrivs
3E78  C5                  PUSH BC
3E79  06 06               LD   B,$06
3E7B  E5                  PUSH HL
3E7C  D5                  PUSH DE
3E7D  C5                  PUSH BC
3E7E  0E 00               LD   C,$00
3E80  C5                  PUSH BC
3E81  CD 00 48            CALL NUMPSH          talet till stacken, 8 bytes
3E84  EB                  EX   DE,HL
3E85  21 0E 00            LD   HL,$000E
3E88  39                  ADD  HL,SP
3E89  73                  LD   (HL),E
3E8A  23                  INC  HL
3E8B  72                  LD   (HL),D
3E8C  CD 53 51            CALL FTODEC          decimal form
3E8F  CD 4D 48            CALL DECEND
3E92  23                  INC  HL
3E93  23                  INC  HL
3E94  7E                  LD   A,(HL)
3E95  2B                  DEC  HL
3E96  2B                  DEC  HL
3E97  FE 35               CP   $35             första siffran 5-9: 16 siffror
3E99  3E 10               LD   A,$10
3E9B  CE 00               ADC  A,$00
3E9D  DD 77 01            LD   (IX+$01),A
3EA0  DD 5E 04            LD   E,(IX+$04)
3EA3  DD 56 05            LD   D,(IX+$05)
3EA6  CD 45 45            CALL FMFREE
3EA9  DD F9               LD   SP,IX
3EAB  E1                  POP  HL
3EAC  E1                  POP  HL
3EAD  E1                  POP  HL
3EAE  E1                  POP  HL
3EAF  C1                  POP  BC
```

```
3EB0  FD 70 1E            LD   (IY+$1E),B      precisionen tillbaka
3EB3  1B                  DEC  DE
3EB4  18 AD               JR   L3E63

3EB6  D5          DSMALL: PUSH DE
3EB7  E5                  PUSH HL
3EB8  D6 DD               SUB  $DD             $DD = 0
3EBA  6F                  LD   L,A
3EBB  26 00               LD   H,$00
3EBD  18 07               JR   L3EC6

3EBF  D5          DINT:   PUSH DE
3EC0  D5                  PUSH DE
3EC1  CD 33 32            CALL LDDEHL          talet ur koden
3EC4  E3                  EX   (SP),HL
3EC5  EB                  EX   DE,HL
3EC6  CD 13 45    L3EC6:  CALL DECSTR          talet som text
3EC9  FD CB 1D 46         BIT  0,(IY+$1D)      % utom i INTEGER-läget
3ECD  20 04               JR   NZ,L3ED3
3ECF  3E 25               LD   A,$25
3ED1  12                  LD   (DE),A
3ED2  13                  INC  DE
3ED3  E1          L3ED3:  POP  HL
3ED4  18 8D       L3ED4:  JR   L3E63
```

DSTR ($CB): *strängkonstant inom citattecken,* DSTR2 ($CC) *inom apostrofer* ($A = \$25 + 2 = \$27$). *Längdbyten först; ett citattecken inne i strängen skrivs dubbelt* ("A""B" *listas som det skrevs*). DNEG ($A8, $A9): *- före det senaste fragmentet.*

```
3ED6  3E 20       DSTR:   LD   A,$20           " (DSTR2: A = $25 -> ')
3ED8  C6 02       DSTR2:  ADD  A,$02
3EDA  4F                  LD   C,A
3EDB  D5                  PUSH DE
3EDC  46                  LD   B,(HL)          B = längden + 1
3EDD  04                  INC  B
3EDE  23                  INC  HL
3EDF  79          L3EDF:  LD   A,C
3EE0  12                  LD   (DE),A
3EE1  13                  INC  DE
3EE2  CD 50 42    L3EE2:  CALL DCHK
3EE5  10 05               DJNZ L3EEC
3EE7  79                  LD   A,C
3EE8  12                  LD   (DE),A
3EE9  13                  INC  DE
3EEA  18 E8               JR   L3ED4

3EEC  7E          L3EEC:  LD   A,(HL)
3EED  23                  INC  HL
3EEE  12                  LD   (DE),A
3EEF  13                  INC  DE
3EF0  B9                  CP   C               avgränsaren dubbleras
3EF1  28 EC               JR   Z,L3EDF
3EF3  18 ED               JR   L3EE2

3EF5  D5          DNEG:   PUSH DE
3EF6  3E 2D               LD   A,$2D           - framför
3EF8  12                  LD   (DE),A
3EF9  13                  INC  DE
3EFA  CD CF 3F            CALL DMERGE
3EFD  C1                  POP  BC
3EFE  18 D4       L3EFE:  JR   L3ED4
```

DDIM ($CD) *och* DCOMMN ($CE): *posterna i DIM och COMMON, tills koden efter en post är under* $C0. *Postens typbyte: bit 1 sträng (storleken står först, som ett fragment), bit 3 fält (dimensionerna, var och en med en undre gräns eller ett tomt fragment). Texten blir namn(a:b,c)=storlek (prövat:* DIM A$(3)=10,B(2:4)*); COMMON har två bytes till efter posten. Mellan posterna komma (*DSEP). DEMPTY ($B0, $C3): *tomt fragment (0), när storleken eller den undre gränsen saknas.*

```
3F00  F6          DDIM:   DEFB $F6             OR $37 (överlappar nästa)
3F01  37          DCOMMN: SCF                  COMMON: CY, DIM: NC (OR $37)
3F02  7E                  LD   A,(HL)          A = postens typ
3F03  23                  INC  HL
```

```
3F04  CB 4F              BIT  1,A            bit 1 (sträng): storleken finns
3F06  20 04              JR   NZ,L3F0C
3F08  01 00 00           LD   BC,$0000
3F0B  C5                 PUSH BC
3F0C  D5           L3F0C: PUSH DE
3F0D  CB 5F              BIT  3,A            bit 3: fält
3F0F  F5                 PUSH AF
3F10  3E 29              LD   A,$29
3F12  12                 LD   (DE),A
3F13  13                 INC  DE
3F14  20 04              JR   NZ,L3F1A       fält: namnet sätts in senare (L3F4A)
3F16  1B                 DEC  DE
3F17  CD CD 3D           CALL DVNAME
3F1A  F1           L3F1A: POP  AF
3F1B  08                 EX   AF,AF'
3F1C  DD E1              POP  IX
3F1E  C1                 POP  BC
3F1F  78                 LD   A,B
3F20  B1                 OR   C
3F21  28 0C              JR   Z,L3F2F        ingen storlek
3F23  C5                 PUSH BC
3F24  DD E5              PUSH IX
3F26  3E 3D              LD   A,$3D          =storleken efter namnet
3F28  12                 LD   (DE),A
3F29  13                 INC  DE
3F2A  CD CF 3F           CALL DMERGE
3F2D  C1                 POP  BC
3F2E  C1                 POP  BC
3F2F  08           L3F2F: EX   AF,AF'
3F30  28 29              JR   Z,L3F5B
3F32  7E                 LD   A,(HL)         A = antal dimensioner
3F33  23                 INC  HL
3F34  08                 EX   AF,AF'
3F35  C1           L3F35: POP  BC
3F36  78                 LD   A,B
3F37  B1                 OR   C
3F38  28 06              JR   Z,L3F40        ingen undre gräns
3F3A  C5                 PUSH BC
3F3B  3E 3A              LD   A,$3A          a:b
3F3D  CD 70 3F           CALL S3F70
3F40  08           L3F40: EX   AF,AF'
3F41  3D                 DEC  A
3F42  28 06              JR   Z,L3F4A
3F44  08                 EX   AF,AF'
3F45  CD 6E 3F           CALL S3F6E          komma mellan dimensionerna
3F48  18 EB              JR   L3F35

3F4A  D5           L3F4A: PUSH DE
3F4B  F5                 PUSH AF
3F4C  CD CD 3D           CALL DVNAME
3F4F  3E 28              LD   A,$28          namn( före
3F51  12                 LD   (DE),A
3F52  13                 INC  DE
3F53  F1                 POP  AF
3F54  08                 EX   AF,AF'
3F55  CD CF 3F           CALL DMERGE
3F58  C1                 POP  BC
3F59  C1                 POP  BC
3F5A  08                 EX   AF,AF'
3F5B  30 02        L3F5B: JR   NC,L3F5F       COMMON: två bytes till
3F5D  23                 INC  HL
3F5E  23                 INC  HL
3F5F  7E           L3F5F: LD   A,(HL)
3F60  FE C0              CP   $C0            ingen post till: klart
3F62  D8                 RET  C
3F63  CD 47 42           CALL DSEP           komma, nästa post
3F66  18 96              JR   L3EFE

3F68  01 00 00     DEMPTY: LD   BC,$0000     tomt fragment
```

```
3F6B  C5                PUSH BC
3F6C  18 90             JR   L3EFE
```

S3F6E/S3F70: *lägg komma (S3F6E) eller A vid början av det senaste fragmentet, som tas bort.* DCOMMA: *A - 1 komman, ett vid början av vart och ett av de A - 1 senaste fragmenten, som tas bort; kvar blir det första (inget när A = 1). Förstör A (DCOMMA: A = 0), BC och IX.*

```
3F6E  3E 2C       S3F6E:  LD   A,$2C
3F70  DD E1       S3F70:  POP  IX
3F72  C1                  POP  BC
3F73  DD E5               PUSH IX
3F75  C3 9D 38            JP   INSCHR

3F78  3D          DCOMMA: DEC  A            A - 1 komman mellan fragmenten
3F79  C8                  RET  Z
3F7A  DD E1               POP  IX
3F7C  C1          L3F7C:  POP  BC
3F7D  F5                  PUSH AF
3F7E  3E 2C               LD   A,$2C
3F80  CD 9D 38            CALL INSCHR
3F83  F1                  POP  AF
3F84  3D                  DEC  A
3F85  20 F5               JR   NZ,L3F7C
3F87  DD E9               JP   (IX)
```

DOPTXT: *operatorns text för koden A ur OPKW ($06C8, 60 bytes); står koden inte där söks koden - 1 (andra typen av samma operator).* DFNAME: *funktionens kod (byten vid HL) söks i tabellhuvudena från FNKWP (FNDTAB), och namnet hämtas ur huvudets nyckelordstabell;* S3FAA *gör samma sak för en sats ur STKWP-kedjan ($86 n, DLINE). Hittas koden inte blir texten ett nyckelord och numret (L3F98).* DKWTXT: *texten efter koden A i tabellen BC (CPIR);* S3FCC: *texten vid HL. Texten kopieras fram till nästa kod (bit 7) eller $00, och DCHK prövar platsen före varje tecken (fel 20). In: DE = bufferten. Ut: DE efter texten, HL oförändrad (DFNAME/S3FAA: efter koden). Förstör A och BC.*

```
3F89  E5          DOPTXT: PUSH HL
3F8A  3C                  INC  A
3F8B  3D          L3F8B:  DEC  A
3F8C  21 C8 06            LD   HL,$06C8     sök koden i OPKW, annars koden - 1
3F8F  01 3C 00            LD   BC,$003C
3F92  ED B1               CPIR
3F94  20 F5               JR   NZ,L3F8B
3F96  18 25               JR   L3FBD

3F98  0A          L3F98:  LD   A,(BC)       okänt n: XSTM och numret
3F99  CD B8 3F            CALL DKWTXT
3F9C  E5                  PUSH HL
3F9D  2B                  DEC  HL
3F9E  6E                  LD   L,(HL)
3F9F  26 00               LD   H,$00
3FA1  CD 13 45            CALL DECSTR
3FA4  E1                  POP  HL
3FA5  C9                  RET

3FA6  ED 4B 7F FF DFNAME: LD   BC,(FNKWP)   funktionens namn ur tabellerna
3FAA  7E          S3FAA:  LD   A,(HL)       $86 n: satsen ur STKWP-kedjan
3FAB  23                  INC  HL
3FAC  E5                  PUSH HL
3FAD  CD EB 1D            CALL FNDTAB       huvudet för koden
3FB0  CD DE 30            CALL S30DE        BC = tilläggets nyckelordstabell
3FB3  E1                  POP  HL
3FB4  28 E2               JR   Z,L3F98
3FB6  C6 80               ADD  A,$80
3FB8  E5          DKWTXT: PUSH HL           texten efter koden A i tabellen BC
3FB9  60                  LD   H,B
3FBA  69                  LD   L,C
3FBB  ED B1               CPIR
3FBD  7E          L3FBD:  LD   A,(HL)
3FBE  87                  ADD  A,A          till nästa kod eller $00
3FBF  38 09               JR   C,L3FCA
3FC1  28 07               JR   Z,L3FCA
3FC3  CD 50 42            CALL DCHK
```

```
3FC6  ED A0              LDI
3FC8  18 F3              JR    L3FBD

3FCA  E1           L3FCA: POP   HL
3FCB  C9                  RET

3FCC  E5           S3FCC: PUSH  HL
3FCD  18 EE              JR    L3FBD
```

DMERGE: *gör de två senaste fragmenten till ett genom att flytta det senastes text (fram till DE) före det förras, en byte i taget från slutet (INSCHR vid det förras början). In: vid SP (efter returadressen) det senaste fragmentets början, därunder det förras. Ut: vid SP det förras början (nu början på båda) och därunder det senastes början, som anroparen tar bort; HL och DE oförändrade. Förstör A, BC och IX.*

```
3FCF  DD E1        DMERGE: POP   IX              senaste fragmentet före det förra
3FD1  C1                   POP   BC
3FD2  E3                   EX    (SP),HL
3FD3  E5                   PUSH  HL
3FD4  C5                   PUSH  BC
3FD5  D5                   PUSH  DE
3FD6  EB                   EX    DE,HL
3FD7  A7                   AND   A
3FD8  ED 42                SBC   HL,BC           L = textens längd
3FDA  4B                   LD    C,E             BC = det förra fragmentets början
3FDB  42                   LD    B,D
3FDC  D1                   POP   DE
3FDD  1B           L3FDD:  DEC   DE              rotera, en byte i taget
3FDE  1A                   LD    A,(DE)
3FDF  C5                   PUSH  BC
3FE0  CD 9D 38             CALL  INSCHR
3FE3  C1                   POP   BC
3FE4  2D                   DEC   L
3FE5  20 F6                JR    NZ,L3FDD
3FE7  C1                   POP   BC
3FE8  E1                   POP   HL
3FE9  E3                   EX    (SP),HL
3FEA  C5                   PUSH  BC
3FEB  DD E9                JP    (IX)

3FED  9F 3D        DECTAB: DEFW  DPAREN          $A7: parentes
3FEF  F5 3E                DEFW  DNEG            $A8, $A9: unärt minus
3FF1  F5 3E                DEFW  DNEG
3FF3  5F 3D                DEFW  DSKIP           $AA-$AE: FN-märke, en byte till
3FF5  5F 3D                DEFW  DSKIP
3FF7  5F 3D                DEFW  DSKIP
3FF9  5F 3D                DEFW  DSKIP
3FFB  5F 3D                DEFW  DSKIP
3FFD  16 3E                DEFW  DNOT            $AF: NOT
3FFF  68 3F                DEFW  DEMPTY          $B0: tomt fragment
4001  98 3D                DEFW  DASGN1          $B1-$B3: = mellan mål och värde
4003  98 3D                DEFW  DASGN1
4005  98 3D                DEFW  DASGN1
4007  BF 3D                DEFW  DASGN           $B4-$B6: tilldelning till en enkel variabel
4009  BF 3D                DEFW  DASGN
400B  BF 3D                DEFW  DASGN
400D  9D 3D                DEFW  DDROP           $B7-$BC: uttryckets slut
400F  9D 3D                DEFW  DDROP
4011  9D 3D                DEFW  DDROP
4013  9D 3D                DEFW  DDROP
4015  9D 3D                DEFW  DDROP
4017  9D 3D                DEFW  DDROP
4019  8A 3D                DEFW  DTHEN           $BD: THEN
401B  95 3D                DEFW  L3D95           $BE: IF utan THEN
401D  9D 3D                DEFW  DDROP           $BF: slut
401F  B9 3D                DEFW  DVAR            $C0-$C2: variabel
4021  B9 3D                DEFW  DVAR
4023  B9 3D                DEFW  DVAR
4025  68 3F                DEFW  DEMPTY          $C3: tomt fragment
4027  B9 3D                DEFW  DVAR            $C4-$C6: variabel
4029  B9 3D                DEFW  DVAR
```

```
402B  B9 3D             DEFW DVAR
402D  BF 3E             DEFW DINT           $C7: heltal i ett ord
402F  AE 3D             DEFW DARR           $C8-$CA: fältelement
4031  AE 3D             DEFW DARR
4033  AE 3D             DEFW DARR
4035  D6 3E             DEFW DSTR           $CB: sträng i citattecken
4037  D8 3E             DEFW DSTR2          $CC: sträng i apostrofer
4039  00 3F             DEFW DDIM           $CD: DIM-post
403B  01 3F             DEFW DCOMMN         $CE: COMMON-post
403D  F4 3D             DEFW DFN            $CF: FN-anrop
```

3.29 Direktkommandon $403F

CMDKW: nyckelorden, som de andra tabellerna ($FF sist). Koden $80-$8E är index i CMDTAB. SCR och NEW har samma kod, likaså RENUMBER och REN. Rutinen får HL = texten efter ordet.

```
403F  80 52 55 4E  CMDKW:  DEFB $80,"RUN"
4043  81 53 43 52          DEFB $81,"SCR"
4047  81 4E 45 57          DEFB $81,"NEW"
404B  82 4C 4F 41…         DEFB $82,"LOAD"
4050  83 43 4F 4E          DEFB $83,"CON"
4054  00 84 4C 49…         DEFB $00,$84,"LIST"
405A  85 43 4C 45…         DEFB $85,"CLEAR"
4060  86 4D 45 52…         DEFB $86,"MERGE"
4066  87 52 45 4E…         DEFB $87,"RENUMBER"
406F  87 52 45 4E          DEFB $87,"REN"
4073  88 45 52 41…         DEFB $88,"ERASE"
4079  89 41 55 54…         DEFB $89,"AUTO"
407E  8A 45 44             DEFB $8A,"ED"
4081  00 8B 53 41…         DEFB $00,$8B,"SAVE"
4087  8C 55 4E 53…         DEFB $8C,"UNSAVE"
408E  8D 47 4F 54…         DEFB $8D,"GOTO"
4093  8E 52 45 53…         DEFB $8E,"RESUME"
409A  FF                   DEFB $FF
409B  E9 1C        CMDTAB: DEFW RUN            hopptabell för direktkommandon
409D  C6 0E                DEFW NEW
409F  99 14                DEFW LOAD
40A1  DD 3B                DEFW CONT
40A3  CE 13                DEFW LIST
40A5  E7 0E                DEFW CLEAR
40A7  9D 14                DEFW MERGE
40A9  7C 42                DEFW RENUM
40AB  53 16                DEFW ERASE
40AD  36 43                DEFW AUTO
40AF  CA 13                DEFW ED
40B1  23 14                DEFW SAVE
40B3  87 FF                DEFW UNSVEC
40B5  5A 3C                DEFW CMDGO
40B7  0C 3C                DEFW CMDRES
40B9  06 00                DEFB $06,$00        ett 16:e ord; CMDKW har ingen kod $8F
```

3.30 DLINE: LIST, en programrad till text $40BB

HL = raden, DE = bufferten (gräns 159 bytes, VFF1A). Radnumret skrivs först, sedan varje sats med en blank före (DBLANK: bara om ingen står där). Koder under $80 (; , # o.d.) kopieras, $C0 och uppåt är uttryck (DECOMP), $87 avslutar raden med CR LF. Övriga koder är nyckelord: texten söks med CPIR från STKW ($0942), som går vidare in i småtabellerna efter den ($88 :, $8A ELSE, $BD TO ...; $81, underförstått GOTO efter THEN, har tom text). $9D-$9F är RESTORE, RESUME och ON ERROR GOTO utan nummer, $86 n en sats ur STKWP-kedjan, $BF (ON) antal och kod före numren. $80-$85 följs av radnummer (länkat: ur adressen), $BC (FOR:s länk) hoppas över, $B7 inleder en INPUT- eller READ-lista. DSPTAB/DSPJP ger egen avkodning för DEF FN, ELSE, LOCAL, REM, !, DATA, MID$ och felraden ($8B ?). Indrag: PRGFCB+$17 är nivån (LIST nollar den). FOR, WHILE och en flerradig DEF FN ökar den för följande rader, NEXT, WEND och FNEND minskar den, och raden med NEXT/WEND/FNEND får den lägre nivån. Indraget, 2 blanka per nivå, läggs in efter radnumret när raden är klar (INSCHR). Ryms det inte ges inget fel; i stället försvinner ett tecken från slutet för varje blank som inte fick plats, även CR LF. In: HL = raden, DE = bufferten, B = nivån före raden (PRGFCB+$17). Ut: HL = nästa rad, DE efter CR LF, B = nivån efter raden. Förstör AF, C, IX och de skuggade registren (DECOMP). Prövat: `DEF FNA(X) LOCAL Z,W$=5` listas som det skrevs, raderna efter får indrag och FNEND-raden inte; `IF A THEN 10` listas med THEN 10 ($81 utan text).

```
40BB  48           DLINE:  LD   C,B
40BC  C5                   PUSH BC
40BD  E5                   PUSH HL
40BE  21 9F 00             LD   HL,$009F          gränsen: 159 bytes (VFF1A)
40C1  19                   ADD  HL,DE
40C2  22 1A FF             LD   (VFF1A),HL
40C5  E1                   POP  HL
40C6  D5                   PUSH DE
40C7  CD 32 32             CALL S3232             radnumret
40CA  E3                   EX   (SP),HL
40CB  EB                   EX   DE,HL
40CC  CD 13 45             CALL DECSTR
40CF  E1                   POP  HL
40D0  C1                   POP  BC
40D1  D5                   PUSH DE                början efter numret (för indraget)
40D2  C5           DSTMT:  PUSH BC
40D3  CD 3C 42     L40D3:  CALL DBLANK            blank före satsen
40D6  CD 50 42     L40D6:  CALL DCHK
40D9  7E                   LD   A,(HL)
40DA  FE BC                CP   $BC               $BC: FOR:s länk, hoppas över
40DC  CA 7A 41             JP   Z,L417A
40DF  FE 87                CP   $87               $87: radslut
40E1  CA EB 41             JP   Z,DEOL
40E4  FE C0                CP   $C0               $C0-: uttryck
40E6  D2 6F 41             JP   NC,L416F
40E9  FE 80                CP   $80               under $80: tecknet som det står
40EB  DA 9D 41             JP   C,L419D
40EE  FE B7                CP   $B7               $B7: INPUT- eller READ-lista
40F0  CA 80 41             JP   Z,DVARS
40F3  CD 3C 42             CALL DBLANK
40F6  7E                   LD   A,(HL)
40F7  23                   INC  HL
40F8  06 01                LD   B,$01             B = antal radnummer
40FA  FE BF                CP   $BF               $BF (ON): antal och kod
40FC  20 04                JR   NZ,L4102
40FE  46                   LD   B,(HL)
40FF  23                   INC  HL
4100  7E                   LD   A,(HL)
4101  23                   INC  HL
4102  4F           L4102:  LD   C,A
4103  C5                   PUSH BC
4104  01 42 09             LD   BC,$0942          BC = STKW, även räknare för CPIR
4107  FE 9D                CP   $9D               $9D-$9F som $83-$85
4109  38 06                JR   C,L4111
410B  FE A0                CP   $A0
410D  30 02                JR   NC,L4111
410F  C6 E6                ADD  A,$E6
4111  FE 86           L4111:  CP   $86               $86 n: grupp 2 eller tillägg
4113  20 09                JR   NZ,L411E
```

```
4115  ED 4B 7D FF          LD   BC,(STKWP)
4119  CD AA 3F             CALL S3FAA
411C  18 03                JR   L4121

411E  CD B8 3F     L411E:  CALL DKWTXT
4121  C1           L4121:  POP  BC
4122  CD 3C 42             CALL DBLANK
4125  79                   LD   A,C
4126  FE 86                CP   $86            $80-$85: radnummer
4128  DA A4 41             JP   C,DLNUMS
412B  E5                   PUSH HL
412C  21 61 42             LD   HL,$4261       egen avkodning (DSPTAB)
412F  01 0D 00             LD   BC,$000D
4132  ED B9                CPDR
4134  79                   LD   A,C            C = platsen i DSPTAB
4135  E1                   POP  HL
4136  20 9E                JR   NZ,L40D6       annars uttrycken
4138  01 62 42             LD   BC,$4262
413B  C3 B7 13             JP   JPTAB

413E  3E 07        L413E:  LD   A,$07          NEXT: variabelns namn (0 = inget)
4140  CD CF 3D             CALL S3DCF
4143  C1           DDEDNT: POP  BC             B = nivån efter satsen, C = radens
4144  78                   LD   A,B
4145  A7                   AND  A
4146  28 06                JR   Z,L414E
4148  3D                   DEC  A              ett steg ut; radens nivå högst B
4149  47                   LD   B,A
414A  B9                   CP   C
414B  30 01                JR   NC,L414E
414D  4F                   LD   C,A
414E  18 82        L414E:  JR   DSTMT

4150  23           L4150:  INC  HL             WHILE: länken bort
4151  23                   INC  HL
4152  C1           L4152:  POP  BC             FOR: följande rader ett steg in
4153  04                   INC  B
4154  18 F8                JR   L414E

4156  23           L4156:  INC  HL             WEND: länken bort, ett steg ut
4157  23                   INC  HL
4158  18 E9                JR   DDEDNT

415A  1B           L415A:  DEC  DE             ?: tecknet över blanket före
415B  1B                   DEC  DE
415C  1A                   LD   A,(DE)
415D  1B                   DEC  DE
415E  12                   LD   (DE),A
415F  13                   INC  DE
4160  18 02                JR   DTEXT

4162  23           L4162:  INC  HL             DATA: länken bort
4163  23                   INC  HL
4164  4E           DTEXT:  LD   C,(HL)         längd och text, $87 med
4165  23                   INC  HL
4166  06 00                LD   B,$00
4168  ED B0                LDIR
416A  1B                   DEC  DE             $87 bort: radslut
416B  C3 EC 41             JP   L41EC

416E  1B           L416E:  DEC  DE             MID$ utan blank
416F  CD 60 3D     L416F:  CALL DECOMP
4172  7E                   LD   A,(HL)         uttryck till: blank emellan
4173  FE C0                CP   $C0
4175  38 06                JR   C,L417D
4177  C3 D3 40             JP   L40D3

417A  23           L417A:  INC  HL             + två bytes
417B  23                   INC  HL
417C  23           L417C:  INC  HL
417D  C3 D6 40     L417D:  JP   L40D6
```

DVARS ($B7): *INPUT- och READ-listan börjar med typbytes, en per variabel, där bit 2 märker den sista; sedan variablerna, var och en med* DECOMP *och komma emellan.* DLNUMS*: B radnummer med komma emellan, som text ur orden (*S3072 *med A = 9 ger numret också när programmet är länkat).*

```
4180  06 00        DVARS:  LD   B,$00          B = antal typbytes (bit 2 = sista)
4182  23                   INC  HL
4183  04           L4183:  INC  B
4184  CB 56                BIT  2,(HL)
4186  23                   INC  HL
4187  28 FA                JR   Z,L4183
4189  C5           L4189:  PUSH BC
418A  CD 60 3D             CALL DECOMP         en variabel
418D  C1                   POP  BC
418E  10 05                DJNZ L4195
4190  7E                   LD   A,(HL)
4191  FE B7                CP   $B7
4193  20 06                JR   NZ,L419B
4195  CD 47 42     L4195:  CALL DSEP           komma emellan
4198  04                   INC  B
4199  10 EE                DJNZ L4189
419B  18 E0        L419B:  JR   L417D

419D  ED A0        L419D:  LDI                 ; , # o.d.
419F  18 DC                JR   L417D

41A1  CD 47 42     L41A1:  CALL DSEP
41A4  C5           DLNUMS: PUSH BC
41A5  3E 09                LD   A,$09          A = 9: numret (länkat: ur adressen)
41A7  CD 72 30             CALL S3072
41AA  E5                   PUSH HL
41AB  69                   LD   L,C
41AC  60                   LD   H,B
41AD  CD 13 45             CALL DECSTR         numret som text
41B0  E1                   POP  HL
41B1  C1                   POP  BC
41B2  10 ED                DJNZ L41A1
41B4  18 C7        L41B4:  JR   L417D
```

DDEFFN*: DEF FNn(p,...); namnet, parametrarna (*S4215 *med C = 0) och sedan =uttrycket (enradig) eller LOCAL-listan (flerradig). Nivån ökas för följande rader. Under avkodningen är* LNKDEF *= parameterlistan (*SETDEF*), så att parametrarnas namn hittas;* L41E5 *nollar den (*LNKCLR*) och minskar nivån, direkt för en enradig DEF och vid FNEND för en flerradig.*

```
41B6  1B           DDEFFN: DEC  DE              DEF FN utan blank, länken bort
41B7  23                   INC  HL
41B8  23                   INC  HL
41B9  CD C0 33             CALL NAMTXT         namnet
41BC  23                   INC  HL
41BD  23                   INC  HL
41BE  CD 59 31             CALL SETDEF         LNKDEF = parameterlistan
41C1  7E                   LD   A,(HL)
41C2  23                   INC  HL
41C3  A7                   AND  A              antal parametrar
41C4  28 0E                JR   Z,L41D4
41C6  47                   LD   B,A
41C7  3E 28                LD   A,$28
41C9  12                   LD   (DE),A
41CA  13                   INC  DE
41CB  0E 00                LD   C,$00          C = 0: utan storlek
41CD  CD 15 42             CALL S4215
41D0  3E 29                LD   A,$29
41D2  12                   LD   (DE),A
41D3  13                   INC  DE
41D4  7E           L41D4:  LD   A,(HL)
41D5  E3                   EX   (SP),HL
41D6  24                   INC  H              nivån + 1
41D7  E3                   EX   (SP),HL
41D8  C6 40                ADD  A,$40          under $C0: flerradig (LOCAL, $87)
41DA  30 A1                JR   NC,L417D
41DC  3E 3D                LD   A,$3D
41DE  12                   LD   (DE),A
```

```
41DF  13              INC  DE
41E0  CD 60 3D        CALL DECOMP        =uttrycket
41E3  18 00           JR   L41E5

41E5  CD 4B 31  L41E5: CALL LNKCLR       DEF slut: ett steg ut igen
41E8  C3 43 41        JP   DDEDNT
```

DEOL: *CR LF efter raden. Sedan indraget: 2 blanka per nivå (C = radens nivå) sätts in efter radnumret (IX), en i taget med* INSCHR. EMIT *prövar platsen; när den är slut flyttas DE ändå en byte tillbaka, och för varje blank som inte ryms försvinner ett tecken i slutet.*

```
41EB  23        DEOL:  INC  HL           radslutet $87 hoppas över
41EC  3E 0D     L41EC: LD   A,$0D
41EE  12               LD   (DE),A
41EF  13               INC  DE
41F0  3E 0A            LD   A,$0A
41F2  12               LD   (DE),A
41F3  13               INC  DE
41F4  C1               POP  BC
41F5  79               LD   A,C          C = radens nivå
41F6  DD E1            POP  IX
41F8  87               ADD  A,A          2 blanka per nivå
41F9  C8        L41F9: RET  Z
41FA  C5               PUSH BC
41FB  F5               PUSH AF
41FC  DD E5            PUSH IX
41FE  C1               POP  BC
41FF  CD 87 16         CALL EMIT         plats? CY ger inget fel (fel)
4202  1B               DEC  DE           DE - 1 även utan plats
4203  0A               LD   A,(BC)
4204  D4 9D 38         CALL NC,INSCHR    blanket efter numret en gång till
4207  F1               POP  AF
4208  C1               POP  BC
4209  3D               DEC  A
420A  18 ED            JR   L41F9
```

L420C: *LOCAL-listan (*DNAMES *med C = 2).* DNAMES: *B namn (antalet i byten vid HL) med komma emellan; med C bit 1 får en strängvariabel =storlek efter sig.* S4215 *börjar med B redan satt (DEF FN:s parametrar). Ut: HL efter listan, DE efter det sista namnet (kommat bort). Förstör AF och BC.*

```
420C  0E 02     L420C: LD   C,$02         LOCAL: med strängarnas storlek
420E  CD 13 42         CALL DNAMES
4211  18 A1            JR   L41B4

4213  46        DNAMES: LD   B,(HL)       B = antal namn
4214  23               INC  HL
4215  C5        S4215: PUSH BC
4216  7E               LD   A,(HL)
4217  F5               PUSH AF
4218  CD C0 33         CALL NAMTXT
421B  F1               POP  AF
421C  C1               POP  BC
421D  E6 02            AND  $02          sträng och C bit 1:
421F  28 14            JR   Z,L4235
4221  CB 49            BIT  1,C
4223  28 10            JR   Z,L4235
4225  C5               PUSH BC
4226  CD E0 30         CALL LDBCHL
4229  E5               PUSH HL
422A  60               LD   H,B
422B  69               LD   L,C
422C  3E 3D            LD   A,$3D        =storleken
422E  12               LD   (DE),A
422F  13               INC  DE
4230  CD 13 45         CALL DECSTR
4233  E1               POP  HL
4234  C1               POP  BC
4235  CD 47 42  L4235: CALL DSEP
4238  10 DB            DJNZ S4215
```

```
423A  1B                DEC  DE             sista kommat bort
423B  C9                RET
```

DBLANK: *blank vid DE, utom när tecknet före redan är en blank; platsen prövas inte. Förstör A och F.* DSEP: *komma genom* DEMIT. DEMIT: *A vid DE genom* EMIT*; är bufferten full (*VFF1A*) blir det fel 20 genom* ERRSP. DCHK: *prövar bara platsen; byten skrivs men DE står kvar.* DECOMP, DKWTXT, DSTR *och* DLINE *anropar den före varje tecken eller kod. Ut: DE + 1 (*DCHK*: oförändrad). Förstör F.*

```
423C  3E 20       DBLANK: LD   A,$20          blank om inte en står före
423E  EB                  EX   DE,HL
423F  2B                  DEC  HL
4240  BE                  CP   (HL)
4241  23                  INC  HL
4242  EB                  EX   DE,HL
4243  C8                  RET  Z
4244  12                  LD   (DE),A
4245  13                  INC  DE
4246  C9                  RET

4247  3E 2C       DSEP:   LD   A,$2C          komma
4249  CD 87 16    DEMIT:  CALL EMIT           lägg A vid DE, fel 20 om bufferten är full
424C  D0                  RET  NC
424D  C3 6B 3B            JP   ERRSP

4250  CD 49 42    DCHK:   CALL DEMIT          som DEMIT, men DE står kvar (egenhet)
4253  1B                  DEC  DE
4254  C9                  RET
```

DSPTAB: *satserna som* DLINE *avkodar själv;* CPDR *bakifrån ger C = platsen i tabellen, och* DSPJP *har rutinerna i samma ordning.*

```
4255  8C 8A B6 8F… DSPTAB: DEFB $8C,$8A,$B6,$8F,$8E,$9A,$9B,$8D  DEF FN, ELSE, LOCAL, REM, DATA, WHILE, WEND, FNEND
425D  98 99 90 A2…         DEFB $98,$99,$90,$A2,$8B             FOR, NEXT, !, MID$, ?
4262  B6 41       DSPJP:  DEFW DDEFFN
4264  7C 41               DEFW L417C
4266  0C 42               DEFW L420C
4268  64 41               DEFW DTEXT
426A  62 41               DEFW L4162
426C  50 41               DEFW L4150
426E  56 41               DEFW L4156
4270  E5 41               DEFW L41E5
4272  52 41               DEFW L4152
4274  3E 41               DEFW L413E
4276  64 41               DEFW DTEXT
4278  6E 41               DEFW L416E
427A  5A 41               DEFW L415A
```

RENUM: RENUMBER [*start*][,*steg*][,*intervall*] (*även REN*). RNARGS *läser start och steg (10 och 10 om de saknas; , , hoppar över steget); 0 eller annat än komma efter ger fel 209. Intervallet läses av* LNRNG*, utan intervall hela programmet. Raderna i intervallet får numren start, start+steg, ... Ordningen ändras inte: start måste vara större än numret på raden före intervallet och det sista nya numret mindre än numret på raden efter, annars fel 209, liksom när ett nummer går över 65535 (även numret efter den sista raden räknas).*

Två varv (A = 2, 1). Det första prövar bara numren. Är programmet inte länkat (IY+$26 *bit 0) görs sedan hänvisningarna (sort 9 i* PSCAN: GOTO, GOSUB, ON, RESTORE, RESUME, ON ERROR GOTO*) om till relativa länkar (*RNLINE*). Saknas en rad ges fel 180 med radens nummer (*"in line"*, IY+*$23 *bit 2 av); länkarna som redan gjorts tas bort (*RNUNLK *fram till* PRGPTR*) och programmet är orört. Det andra varvet skriver de nya numren, och* RNUNLK *gör länkarna till nummer igen, nu de nya. Ett länkat program (efter* RUN*) har redan länkarna och behåller dem. In: HL = texten efter ordet. Ut:* RET *till kommandoslingan; fel genom* ERRSP *(A = 209 eller 180). Förstör alla register. Prövat:* REN 100,5 *med GOTO, GOSUB och ON GOTO;* REN 1000 *med* 40 GOTO 99 *ger* Error 180 in line 40. *och programmet står kvar;* REN 0 *och* REN 10,0 *ger fel 209.*

```
427C  CD 0D 43    RENUM:  CALL RNARGS         DE = start, BC = steg
427F  D5                  PUSH DE
4280  C5                  PUSH BC
4281  7E                  LD   A,(HL)
4282  FE 0D               CP   $0D
4284  28 05               JR   Z,L428B
4286  FE 2C               CP   $2C
4288  20 40               JR   NZ,L42CA
428A  23                  INC  HL
428B  CD 8B 15    L428B:  CALL LNRNG          DE-BC = intervallet
```

```
428E  C5                PUSH BC
428F  CD 8C 32          CALL FNDLIN         HL = första raden, DE = numret före
4292  DD E1            POP  IX
4294  C1                POP  BC
4295  E3                EX   (SP),HL
4296  EB                EX   DE,HL
4297  AF                XOR  A
4298  ED 52             SBC  HL,DE          start måste vara större
429A  30 2E             JR   NC,L42CA
429C  E1                POP  HL
429D  F6 02             OR   $02            två varv: A = 2 prövar, 1 skriver
429F  E5          L429F: PUSH HL
42A0  D5                PUSH DE
42A1  F5                PUSH AF
42A2  7E          L42A2: LD   A,(HL)
42A3  3D                DEC  A              slutraden: klart
42A4  28 32             JR   Z,L42D8
42A6  23                INC  HL
42A7  D5                PUSH DE
42A8  5E                LD   E,(HL)
42A9  23                INC  HL
42AA  56                LD   D,(HL)
42AB  2B                DEC  HL
42AC  DD E5             PUSH IX
42AE  E3                EX   (SP),HL
42AF  A7                AND  A
42B0  ED 52             SBC  HL,DE
42B2  38 1B             JR   C,L42CF        efter intervallet
42B4  E1                POP  HL
42B5  D1                POP  DE
42B6  E3                EX   (SP),HL
42B7  CB 4C             BIT  1,H            första varvet: bara prov
42B9  E3                EX   (SP),HL
42BA  20 04             JR   NZ,L42C0
42BC  73                LD   (HL),E
42BD  23                INC  HL
42BE  72                LD   (HL),D
42BF  2B                DEC  HL
42C0  85          L42C0: ADD  A,L            HL = nästa rad
42C1  6F                LD   L,A
42C2  30 01             JR   NC,L42C5
42C4  24                INC  H
42C5  EB          L42C5: EX   DE,HL
42C6  09                ADD  HL,BC
42C7  EB                EX   DE,HL
42C8  30 D8             JR   NC,L42A2       nästa nummer, spill ger fel (egenhet)
42CA  3E D1       L42CA: LD   A,$D1          fel 209
42CC  C3 6B 3B    L42CC: JP   ERRSP

42CF  E1          L42CF: POP  HL
42D0  E1                POP  HL
42D1  A7                AND  A
42D2  ED 42             SBC  HL,BC          sista nya numret < radens efter
42D4  ED 52             SBC  HL,DE
42D6  30 F2             JR   NC,L42CA
42D8  AF          L42D8: XOR  A
42D9  FD CB 26 46       BIT  0,(IY+$26)     länkat: hänvisningarna är länkar
42DD  20 27             JR   NZ,L4306
42DF  E1                POP  HL
42E0  E5                PUSH HL
42E1  C5                PUSH BC
42E2  DD E5             PUSH IX
42E4  25                DEC  H              andra varvet: avlänka
42E5  28 0D             JR   Z,L42F4
42E7  DD 21 5A 30       LD   IX,RNLINE      hänvisningarna till länkar
42EB  CD 52 35          CALL PSCAN
42EE  EB                EX   DE,HL
42EF  CD A3 1D          CALL S1DA3          PRGPTR = hur långt det gick
42F2  30 0F             JR   NC,L4303
```

```
42F4  F5           L42F4:  PUSH AF
42F5  DD 21 D9 32          LD   IX,RNUNLK       länkarna till nummer (fram till PRGPTR)
42F9  CD 52 35             CALL PSCAN
42FC  FD CB 23 96          RES  2,(IY+$23)      fel anges med in line
4300  F1                   POP  AF
4301  38 C9                JR   C,L42CC         fel 180 från RNLINE
4303  DD E1        L4303:  POP  IX
4305  C1                   POP  BC
4306  F1            L4306:  POP  AF
4307  D1                   POP  DE
4308  E1                   POP  HL
4309  3D                   DEC  A
430A  20 93                JR   NZ,L429F
430C  C9                   RET
```

RNARGS: *start och steg för RENUMBER och AUTO, 10 och 10 när de saknas. 0 som start eller steg, eller något annat än komma eller CR efter starten, ger fel 209 (ERRSP); , , lämnar steget 10 och HL vid det andra kommat. In: HL = texten. Ut: DE = start, BC = steg, HL efter det sista som lästes. Förstör A och F.*

```
430D  7E           RNARGS: LD   A,(HL)
430E  11 0A 00             LD   DE,$000A        start 10
4311  FE 0D                 CP   $0D
4313  28 08                JR   Z,L431D
4315  CD C2 4B             CALL GETLNR
4318  7A                   LD   A,D
4319  B3                   OR   E
431A  28 AE                JR   Z,L42CA         0 ger fel
431C  7E                   LD   A,(HL)
431D  01 0A 00      L431D:  LD   BC,$000A        steg 10
4320  FE 0D                 CP   $0D
4322  C8                   RET  Z
4323  FE 2C                 CP   $2C
4325  20 A3                JR   NZ,L42CA
4327  23                   INC  HL
4328  BE                   CP   (HL)            , ,: steget hoppas över
4329  C8                   RET  Z
432A  D5                   PUSH DE
432B  CD C2 4B             CALL GETLNR
432E  7A                   LD   A,D
432F  B3                   OR   E
4330  28 98                JR   Z,L42CA
4332  4B                   LD   C,E
4333  42                   LD   B,D
4334  D1                   POP  DE
4335  C9                   RET
```

AUTO [*start*][,*steg*]: *radnumren föreslås vid inmatningen. Utan argument börjar AUTO på närmaste tiotal över programmets sista rad (10 utan program) med steget 10; finns rad 65535 görs inget. Med start slutar AUTO före den första befintliga raden med nummer >= start: AUTO 10 gör inget när rad 10 finns, AUTO 12,4 före rad 20 ger 12 och 16. Numret och en blank skrivs ut och står i radbufferten, CONLIN läser resten. Tom rad slutar AUTO (READY). Börjar raden med en siffra ersätter den det föreslagna numret, som då föreslås igen. Raden går till DOLINE; ett fel avslutar AUTO (raden blir mall som annars), likaså en direktsats. In: HL = texten efter ordet. Ut: READY vid tom rad,* RET *när gränsen nås; fel genom ERRSP. Förstör alla register. Prövat:* AUTO 12,4 *före rad 20 ger 12 och 16,* AUTO *sedan 30,* AUTO 0 *fel 209.*

```
4336  7E           AUTO:   LD   A,(HL)
4337  FE 0D                CP   $0D
4339  28 13                JR   Z,L434E         utan argument: efter sista raden
433B  CD 0D 43             CALL RNARGS
433E  C5                   PUSH BC
433F  D5                   PUSH DE
4340  CD 8C 32             CALL FNDLIN          första raden >= start ...
4343  7E                   LD   A,(HL)
4344  3D                   DEC  A
4345  23                   INC  HL
4346  5E                   LD   E,(HL)
4347  23                   INC  HL
4348  56                   LD   D,(HL)
4349  E1                   POP  HL
```

```
434A  28 17              JR   Z,L4363          ... är gränsen (ingen: $FFFF)
434C  18 18              JR   L4366

434E  11 FF FF     L434E: LD   DE,$FFFF
4351  CD 8C 32           CALL FNDLIN          DE = sista radens nummer
4354  C8                 RET  Z               rad 65535 finns: inget
4355  EB                 EX   DE,HL
4356  11 0A 00           LD   DE,$000A
4359  D5                 PUSH DE
435A  CD 2D 4B           CALL DIV16           (sista / 10 + 1) * 10, spill prövas inte (fel)
435D  D1                 POP  DE
435E  D5                 PUSH DE
435F  23                 INC  HL
4360  CD 19 4B           CALL MUL16
4363  11 FF FF     L4363: LD   DE,$FFFF
4366  C1           L4366: POP  BC
4367  A7           L4367: AND  A               framme vid gränsen: klart
4368  ED 52              SBC  HL,DE
436A  19                 ADD  HL,DE
436B  D0                 RET  NC
436C  E5                 PUSH HL
436D  D5                 PUSH DE
436E  C5                 PUSH BC
436F  CD ED 3C           CALL CRLF
4372  EB                 EX   DE,HL
4373  CD 51 00           CALL BUFADR          HL = radbufferten
4376  E5                 PUSH HL
4377  C5                 PUSH BC
4378  EB                 EX   DE,HL
4379  CD 13 45           CALL DECSTR          numret och en blank i bufferten
437C  EB                 EX   DE,HL
437D  36 20              LD   (HL),$20
437F  23                 INC  HL
4380  C1                 POP  BC
4381  D1                 POP  DE
4382  7B                 LD   A,E
4383  95                 SUB  L
4384  81                 ADD  A,C             C = platsen som är kvar
4385  4F                 LD   C,A
4386  D5                 PUSH DE
4387  E5                 PUSH HL
4388  C5                 PUSH BC
4389  EB                 EX   DE,HL
438A  7B                 LD   A,E
438B  95                 SUB  L
438C  4F                 LD   C,A
438D  CD 96 11           CALL PRCON           skriv numret
4390  C1                 POP  BC
4391  E1                 POP  HL
4392  E5                 PUSH HL
4393  CD 4D 49           CALL CONLIN          resten av raden
4396  E1                 POP  HL
4397  D1                 POP  DE
4398  CD 1A 4C           CALL SKIPB0
439B  FE 0D              CP   $0D
439D  CA 10 01           JP   Z,READY         tom rad: slut
43A0  D6 30              SUB  $30
43A2  FE 0A              CP   $0A
43A4  D5                 PUSH DE
43A5  30 07              JR   NC,L43AE        siffra: eget nummer ...
43A7  7E           L43A7: LD   A,(HL)         ... flyttas till buffertens början, A = 0
43A8  ED A0              LDI
43AA  D6 0D              SUB  $0D
43AC  20 F9              JR   NZ,L43A7
43AE  E1           L43AE: POP  HL
43AF  F5                 PUSH AF
43B0  CD DC 11           CALL DOLINE          raden in i programmet
43B3  DA 6B 3B           JP   C,ERRSP         fel avslutar AUTO
43B6  F1                 POP  AF
```

```
43B7  C1                POP   BC
43B8  D1                POP   DE
43B9  E1                POP   HL
43BA  A7                AND   A
43BB  28 AA             JR    Z,L4367         eget nummer: samma föreslås igen
43BD  09                ADD   HL,BC
43BE  30 A7             JR    NC,L4367        spill: CY kvar, A = tecknet - 0 (fel)
43C0  C9                RET
```

CTYPE: *typen på uttrycket som just kompilerats (B) mot den som behövs (A; 0 flyttal, 1 heltal, 2 sträng). Lika: inget. Heltal till flyttal och tvärtom ger koden* $82 (ITOF) *respektive* $81 (FTOI, *med avrundning*), *annars fel 223 (sträng väntas) eller 224 (tal väntas). Ut B = typen efter omvandlingen. In: A = typen som behövs (bit 0-1), B = typen som uttrycket har, DE = koden. Ut: NC: B = typen som behövs, DE efter* $81/$82 *om en lades in; CY: A = felet. Förstör A och F.*

```
43C1  E6 03       CTYPE:  AND   $03              A = önskad typ, B = given
43C3  B8                  CP    B
43C4  C8                  RET   Z
43C5  FE 01               CP    $01
43C7  28 08               JR    Z,CTOINT        heltal önskas
43C9  FE 00               CP    $00
43CB  28 15               JR    Z,CTOFLT
43CD  3E DF               LD    A,$DF           sträng väntas: fel 223
43CF  37                  SCF
43D0  C9                  RET

43D1  78          CTOINT: LD    A,B             A = given typ, B = 1 ut
43D2  06 01               LD    B,$01
43D4  B8                  CP    B
43D5  C8                  RET   Z
43D6  FE 00               CP    $00
43D8  3E E0               LD    A,$E0           sträng: fel 224
43DA  37                  SCF
43DB  C0                  RET   NZ
43DC  3E 81               LD    A,$81           $81 FTOI
43DE  12                  LD    (DE),A
43DF  13                  INC   DE
43E0  A7                  AND   A
43E1  C9                  RET

43E2  78          CTOFLT: LD    A,B
43E3  06 00               LD    B,$00
43E5  B8                  CP    B
43E6  C8                  RET   Z
43E7  FE 01               CP    $01
43E9  3E E0               LD    A,$E0
43EB  37                  SCF
43EC  C0                  RET   NZ
43ED  3E 82               LD    A,$82           $82 ITOF
43EF  12                  LD    (DE),A
43F0  13                  INC   DE
43F1  A7                  AND   A
43F2  C9                  RET
```

3.31 ADD$, SUB$, MUL$, DIV$ (a$,b$,n), COMP%(a$,b$), VAL(a$), NUM$(x) $43F3

Strängarna räknas som decimala tal med upp till 124 siffror. Varje sträng görs om till decimal form på stacken (STRDEC; för det första argumentet kod $83, STRNUM): blanka före och efter, tecken, siffror, punkt och E med exponent; annat ger fel 210. FNBCD sparar n (låga byten) i IY+$20 och -n i IY+$21, FNSTR1 gör om den sista strängen och hoppar via FNTAB + $36 + index till ADDS, SUBS, MULS, DIVS, COMPS eller VAL. Fel 133: division med 0 eller fler än 124 siffror (även med decimalerna: ADD$("1","1",124)).

Resultatet skrivs med PRINT USING-rutinerna i en buffert under stacken (S44A4) och flyttas upp som sträng utan inledande blank:

```
n > 0   n decimaler, avrundat ("1.5" * "1.5" med 1 -> "2.3"); minst
        en siffra före punkten ("0.0010")
n = 0   heltal, avrundat
n < 0   -n värdesiffror i fri form som PRINT och NUM$ ("1.E+20",
        ".06")
```

ADD$ och SUB$ räknar med siffrorna fram till n + 2 decimaler (n < 0: -n + 2 siffror från den största termens första siffra, där 0 räknas från första decimalen); resten av termerna stryks utan avrundning (S5FB4, ADDS). ADD$("1.234E-5","0",-2) blir därför 0. Med n = 0 kan resultatet bli -0 (ADD$("-0.4","0",0)).

COMP% ger -1, 0 eller 1 efter talens värde (1.0 = "1", 1E3 > 999; exponenter som skiljer 128 eller mer jämförs fel, se $5FA6). NUM$ skriver talet som PRINT, utan blanka före och efter, med högst DIGITS siffror (7 eller 17). In: vid SP n som heltal (bara FNBCD), därunder b$ som sträng och a$ i decimal form (STRNUM); DE = kodpekaren. FNSTR1 (VAL och COMP%) börjar med strängen vid SP. Ut: strängen vid SP (bytes på stacken, adress, längd, med texten ovanför), för COMP% ett heltal och för VAL ett flyttal; DE = PRGPTR och nästa kod. Fel 210 och 133 går till ERRA.

```
43F3  E1           FNBCD:  POP  HL              L = n (decimaler) ...
43F4  FD 75 20             LD   (IY+$20),L
43F7  AF                   XOR  A
43F8  95                   SUB  L
43F9  FD 77 21             LD   (IY+$21),A      ... och -n
43FC  CD A3 1D     FNSTR1: CALL S1DA3           andra strängen till decimal form
43FF  CD 5E 48             CALL STRDEC
4402  3E D2        L4402:  LD   A,$D2           fel 210: texten är inget tal
4404  38 05                JR   C,L440B
4406  CD 92 21             CALL S2192           räkna (ADDS .. DIVS)
4409  3E 85                LD   A,$85           fel 133: division med 0, > 124 siffror
440B  DA 12 00     L440B:  JP   C,ERRA
440E  CD 4D 48             CALL DECEND          IX = efter talet
4411  FD 7E 20             LD   A,(IY+$20)
4414  A7                   AND  A
4415  FA 52 44             JP   M,L4452         n < 0: n värdesiffror som NUM$
4418  A7                   AND  A
4419  28 01                JR   Z,L441C
441B  3C                   INC  A
441C  4F           L441C:  LD   C,A             C = n + 1 (punkten), 0 utan
441D  D1                   POP  DE              E = längd + tecken, D = exponent
441E  D5                   PUSH DE
441F  7A                   LD   A,D
4420  3D                   DEC  A
4421  F2 25 44             JP   P,L4425
4424  AF                   XOR  A
4425  3C           L4425:  INC  A               B = heltalssiffror + 1 (+ 1 för -)
4426  3C                   INC  A
4427  CB 7B                BIT  7,E
4429  28 01                JR   Z,L442C
442B  3C                   INC  A
442C  47           L442C:  LD   B,A
442D  11 80 80             LD   DE,$8080        fast form, utan exponent
4430  81                   ADD  A,C
4431  CD A4 44             CALL S44A4
4434  CD 9F 45             CALL FMUSE
4437  18 30                JR   L4469
```

NUMS: NUM$(x), *via* FNFLT2 (*bas* $0CC9). *Talet görs om med* FTODEC *och skrivs med* FMFREE *i fri form som i PRINT, med högst 7 eller 17 siffror (IY+$1E = 4 eller 8) och högst* DIGITS. *Formen väljs däremot efter* DIGITS *platser (+3), så med* DIGITS *över 7 i* SINGLE *kan talet skrivas utan exponent där PRINT har en (se* $4457*). +2 = 2 (bit 1) tar bort blanken före ett positivt tal, och*

L4469 blanken efter. L4452 är ingången för ADD$ m.fl. med n < 0 (A = -n). In: x vid SP under returadressen. Ut: strängen vid SP som för FNBCD, och nästa kod.

```
4439  E1           NUMS:   POP  HL              returadressen bort
443A  CD 53 51               CALL FTODEC
443D  CD 4D 48               CALL DECEND
4440  FD 7E 1E               LD   A,(IY+$1E)    7 eller 17 siffror ...
4443  FE 05                  CP   $05
4445  DE 00                  SBC  A,$00
4447  87                     ADD  A,A
4448  3C                     INC  A
4449  FD 46 1F               LD   B,(IY+$1F)
444C  B8                     CP   B             ... högst DIGITS
444D  38 05                  JR   C,L4454
444F  78                     LD   A,B
4450  18 02                  JR   L4454

4452  ED 44        L4452:   NEG
4454  57          L4454:   LD   D,A
4455  1E 00                  LD   E,$00         fri form som PRINT
4457  FD 46 1F               LD   B,(IY+$1F)    +3 = DIGITS platser, +1 = siffrorna (egenhet)
445A  0E 02                  LD   C,$02         bufferten: max(siffror, DIGITS) + 8
445C  B8                     CP   B
445D  30 01                  JR   NC,L4460
445F  78                     LD   A,B
4460  C6 08        L4460:   ADD  A,$08
4462  CD A4 44               CALL S44A4
4465  CD 45 45               CALL FMFREE
4468  1B                     DEC  DE            blanken efter talet bort
4469  C1          L4469:   POP  BC            BC = bufferten, HL = gamla IX
446A  E1                     POP  HL
446B  0A                     LD   A,(BC)
446C  FE 21                  CP   $21           inledande blank bort
446E  7B                     LD   A,E
446F  99                     SBC  A,C
4470  4F                     LD   C,A
4471  06 00                  LD   B,$00
4473  EB                     EX   DE,HL         texten flyttas upp över talet
4474  2B                     DEC  HL
4475  1B                     DEC  DE
4476  ED B8                  LDDR
4478  13                     INC  DE
4479  EB                     EX   DE,HL
447A  F9                     LD   SP,HL
447B  4F                     LD   C,A           strängen: längd, adress, bytes
447C  C5                     PUSH BC
447D  E5                     PUSH HL
447E  C5                     PUSH BC
447F  C3 80 21     L447F:    JP   L2180
```

STRNUM (kod $83): det första strängargumentet till ADD$ m.fl. och COMP% görs om till decimal form direkt, så att bara det sista återstår för FNSTR1. In: strängen vid SP, DE = kodpekaren. Ut: talet i decimal form vid SP, DE = PRGPTR och nästa kod; fel 210 om texten inte är ett tal.

```
4482  CD A3 1D    STRNUM:  CALL S1DA3          kod $83: strängargument till tal
4485  CD 5E 48             CALL STRDEC
4488  DA 02 44             JP   C,L4402
448B  18 F2                JR   L447F
```

COMPS: COMP%(a$,b$). BCDCMP jämför, flaggorna sparas i AF' medan de två talen tas bort, och L2110 lägger -1 (CY), 0 (Z) eller 1. In: returadressen från S2192 vid SP, under den b$ och a$ i decimal form. Ut: heltalet vid SP, DE = PRGPTR och nästa kod.

```
448D  E1          COMPS:  POP  HL
448E  CD 73 5F            CALL BCDCMP        jämför de två talen
4491  08                  EX   AF,AF'
4492  06 02               LD   B,$02
4494  E1          L4494:  POP  HL            två tal bort från stacken
4495  26 00               LD   H,$00
4497  CB BD               RES  7,L
```

```
4499  39                ADD  HL,SP
449A  F9                LD   SP,HL
449B  10 F7             DJNZ L4494
449D  08                EX   AF,AF'
449E  CD 78 12          CALL S1278
44A1  C3 10 21          JP   L2110          -1, 0 eller 1
```

S44A4: *lägg en buffert på A bytes under stacken och en post på fyra bytes som IX pekar på: +0 E (flaggor, $80 = fast form utan exponent), +1 D (siffror), +2 C (decimaler med punkten), +3 B (platser före punkten). Under returadressen ligger buffertens början och gamla IX; DE = bufferten. In: A = buffertens storlek, B, C, D och E = postens fyra bytes. Ut: DE = bufferten, IX = posten; förstör A, DE' och HL'.*

```
44A4  D9          S44A4: EXX                  A bytes buffert under stacken
44A5  D1                 POP  DE
44A6  ED 44              NEG
44A8  6F                 LD   L,A
44A9  26 FF              LD   H,$FF
44AB  39                 ADD  HL,SP
44AC  F9                 LD   SP,HL
44AD  D9                 EXX
44AE  C5                 PUSH BC             posten B C D E ...
44AF  D5                 PUSH DE
44B0  DD E5              PUSH IX
44B2  DD 21 02 00        LD   IX,$0002       ... IX pekar på den
44B6  DD 39              ADD  IX,SP
44B8  D9                 EXX
44B9  E5                 PUSH HL
44BA  D5                 PUSH DE
44BB  E5                 PUSH HL
44BC  D9                 EXX
44BD  D1                 POP  DE             DE = bufferten
44BE  C9                 RET
```

SYS(*n*): *systemvärden, bara n:s låga byte räknas* (SYS(*258*) = SYS(*2*)):

```
  2  MEMTOP - BOTRAM      3  programmets storlek (PRGEND - PRGBEG)
  4  ledigt minne (SP - FREPTR)
  5  KEYFLG: 128 när en tangent väntar, annars 0
  6  som 5, och sätter KEYFLG: nästa GET ger den senaste tangenten
 10  IY ($FF00 = -256)   11  PRGBEG   12  VARLST ($FF2C = -212)
```

Andra n ger fel 143. In: n som heltal vid SP, DE = kodpekaren. Ut: värdet som heltal vid SP genom svansen av ISUB (*HL - BC, CY = 0), sedan nästa kod; 6 går dit genom* KEYSET. *Prövat:* SYS(10) *ger -256 och* SYS(12) *-212;* SYS(6) *ger 0 och* SYS(5) *direkt efter 128;* SYS(0) *och* SYS(7) *ger fel 143.*

```
44BF  21 36 22    SYS:   LD   HL,$2236       svaret blir HL - BC (ISUB)
44C2  E3                 EX   (SP),HL
44C3  7D                 LD   A,L            bara låga byten
44C4  2A 0E FF           LD   HL,(MEMTOP)    2: MEMTOP - BOTRAM
44C7  ED 4B 0C FF        LD   BC,(BOTRAM)
44CB  D6 02              SUB  $02
44CD  C8                 RET  Z
44CE  2A 08 FF           LD   HL,(PRGEND)    3: programmets storlek
44D1  ED 4B 06 FF        LD   BC,(PRGBEG)
44D5  3D                 DEC  A
44D6  C8                 RET  Z
44D7  21 00 00           LD   HL,$0000       4: ledigt (SP - FREPTR)
44DA  39                 ADD  HL,SP
44DB  ED 4B 0A FF        LD   BC,(FREPTR)
44DF  3D                 DEC  A
44E0  C8                 RET  Z
44E1  01 00 00           LD   BC,$0000
44E4  2A E2 FF           LD   HL,(KEYFLG)    5: KEYFLG (128 = tangent väntar)
44E7  60                 LD   H,B
44E8  3D                 DEC  A
44E9  C8                 RET  Z
44EA  3D                 DEC  A
44EB  CA 87 01           JP   Z,KEYSET       6: dito, och sätt KEYFLG
44EE  FD E5              PUSH IY             10: IY
44F0  E1                 POP  HL
```

```
44F1  D6 04              SUB  $04
44F3  C8                 RET  Z
44F4  2A 06 FF           LD   HL,(PRGBEG)      11: PRGBEG
44F7  3D                 DEC  A
44F8  C8                 RET  Z
44F9  21 2C FF           LD   HL,$FF2C         12: VARLST
44FC  3D                 DEC  A
44FD  C8                 RET  Z
44FE  D7                 RST  $10              fel 143: okänt SYS-nummer
44FF  8F                 DEFB $8F
```

3.32 Talutskrift: tal till text (PRINT, PRINT USING, NUM$, LIST, ADD$ m.fl.) $4500

INTSTR: heltalet HL som text vid DE med - eller blank före (IY+$27 bit 3; utan den, som i TRACE, utan tecken). DECSTR: HL utan tecken i bas 10, BASSTR: i bas C (HEX$, OCT$), siffrorna 0-9 och A-F utan inledande nollor. In: HL = talet, DE = bufferten; för BASSTR C = basen. Ut: DE efter texten, HL = 0, B = 0, C kvar, A = det sista tecknet och CY = 0. BASDIG anropar sig själv en gång per siffra (4 bytes stack per varv).

Flyttalen skrivs ur decimal form (FTODEC, STRDEC): B = antal siffror (bit 7 = tecken), C = exponent (talet = 0.siffrorna * 10^C) och siffrorna i ASCII. IX pekar på en post (S44A4; PRINT $1ECA):

```
    +0  flaggor: 0 = fri form; ur mallen (USEFLD) med bit 7 satt:
        bit 0 **, 1 ¤¤, 2 avskiljare, 3 +, 4 tecken sist, 5 ^^^^,
        6 strängfält ($80 utan annat: ADD$ m.fl.)
    +1  högst så många siffror (7 eller 17; fri form högst DIGITS)
    +2  decimaler + 1 för punkten (0 = ingen punkt)
    +3  platser före punkten
```

```
4500  FD CB 27 5E  INTSTR: BIT  3,(IY+$27)      bit 3 (PRINT): med tecken; TRACE: utan
4504  28 02                JR   Z,L4508
4506  CB 7C                BIT  7,H             med tecken: negativt?
4508  3E 20        L4508:  LD   A,$20
450A  28 05                JR   Z,L4511
450C  CD 6D 22             CALL NEGHL           beloppet, - före
450F  3E 2D                LD   A,$2D
4511  12           L4511:  LD   (DE),A
4512  13                   INC  DE
4513  0E 0A        DECSTR: LD   C,$0A           HL utan tecken, bas 10
4515  06 00        BASSTR: LD   B,$00           tal till text i bas C
4517  D5           BASDIG: PUSH DE              rekursivt: högsta siffran först
4518  CD 2F 4B             CALL DIVBC           HL = HL / BC, E = resten
451B  7C                   LD   A,H
451C  B5                   OR   L
451D  7B                   LD   A,E
451E  D1                   POP  DE
451F  F5                   PUSH AF
4520  C4 17 45             CALL NZ,BASDIG       siffrorna före denna
4523  F1                   POP  AF
4524  C6 90                ADD  A,$90           0-15 -> 0-9, A-F
4526  27                   DAA
4527  CE 40                ADC  A,$40
4529  27                   DAA
452A  12                   LD   (DE),A
452B  13                   INC  DE
452C  A7                   AND  A               CY = 0
452D  C9                   RET
```

PRFREE/FMFREE: fri form (PRINT utan USING, NUM$, LIST). Talet avrundas till n = +3 siffror och skrivs utan exponent när det ryms på n siffror: 0 <= C <= n, eller C < 0 och nollorna efter punkten plus siffrorna högst n (n = 6: .000001, men 1.E-07 och 1.5E-06). Ingen nolla före punkten, punkten bara om det finns decimaler. Annars E-form: en siffra före punkten, punkten alltid ("1.E+06"), exponenten med två siffror (tre över 99: ADD$ m.fl.). Före talet - eller blank (inget för positiva tal i NUM$, +2 bit 1), efter det en blank. In: HL = talet i decimal form, DE = utbufferten, IX = posten (PRFREE: +1 = högst så många siffror; FMFREE använder +1, +2 och +3 som de är). Ut: DE efter texten (efter blanken); förstör A, BC, HL och AF'.

```
452E  AF           PRFREE: XOR  A               +0 = 0: fri form, +2 = 0
452F  DD 77 00             LD   (IX+$00),A
4532  DD 77 02             LD   (IX+$02),A
```

```
4535  DD 7E 01           LD   A,(IX+$01)
4538  FD 46 1F           LD   B,(IY+$1F)       högst DIGITS siffror
453B  B8                 CP   B
453C  38 04              JR   C,L4542
453E  78                 LD   A,B
453F  DD 77 01           LD   (IX+$01),A
4542  DD 77 03     L4542: LD   (IX+$03),A       +3: lika många platser
4545  46           FMFREE: LD   B,(HL)          B = siffror (bit 7 = tecken) ...
4546  23                 INC  HL
4547  4E                 LD   C,(HL)           ... C = exponent (punktens plats)
4548  23                 INC  HL
4549  CD FC 46           CALL FMRND            avrunda till +1 siffror
454C  D5                 PUSH DE
454D  DD 5E 03           LD   E,(IX+$03)       E = platser, D = siffror
4550  50                 LD   D,B
4551  CB BA              RES  7,D
4553  79                 LD   A,C              0 <= C <= E: fast form
4554  BB                 CP   E
4555  38 0C              JR   C,L4563
4557  28 0A              JR   Z,L4563
4559  A7                 AND  A
455A  F2 75 45           JP   P,L4575          C < 0: ryms nollorna och ...
455D  83                 ADD  A,E
455E  30 15              JR   NC,L4575
4560  BA                 CP   D                ... siffrorna på E platser?
4561  38 12              JR   C,L4575
4563  D1           L4563: POP  DE               fast form
4564  0D                 DEC  C
4565  05                 DEC  B
4566  0C           L4566: INC  C                talet 0 (B = 0): en siffra
4567  04                 INC  B
4568  28 FC              JR   Z,L4566
456A  CD 8F 45           CALL FMSGN            blank eller - före
456D  EB                 EX   DE,HL
456E  AF                 XOR  A
456F  CD 92 46           CALL FMDIGS           siffrorna, punkten bara vid behov
4572  C3 1D 46           JP   L461D            och en blank efter

4575  7A           L4575: LD   A,D              E-form: siffrorna efter punkten
4576  B7                 OR   A
4577  20 01              JR   NZ,L457A
4579  3C                 INC  A
457A  D1           L457A: POP  DE
457B  CD 8F 45           CALL FMSGN
457E  CB B8              RES  7,B              tecknet bort ur B
4580  DD 77 02           LD   (IX+$02),A       +2 = siffror, +3 = 1 före punkten
4583  3E 01              LD   A,$01
4585  DD 77 03           LD   (IX+$03),A
4588  CD D1 45           CALL FMEXP            mantissa och exponent
458B  EB                 EX   DE,HL
458C  C3 1D 46           JP   L461D
```

FMSGN: *tecknet i fri form: - för ett negativt tal (B bit 7), annars en blank, eller inget alls när +2 bit 1 är satt (NUM$); biten nollas. In: DE = utbufferten. Ut: DE efter tecknet; A och HL kvar.*

```
458F  DD CB 02 4E  FMSGN: BIT  1,(IX+$02)      +2 bit 1 (NUM$): ...
4593  DD CB 02 8E       RES  1,(IX+$02)
4597  28 03              JR   Z,L459C
4599  CB 78              BIT  7,B              ... ingen blank före positivt tal
459B  C8                 RET  Z
459C  EB           L459C: EX   DE,HL
459D  18 70              JR   L460F
```

FMUSE: *talfält i PRINT USING (och ADD$ m.fl.). Med ^^^^ (bit 5) tas en plats för tecknet om mallen saknar + och -; resten är siffror före punkten. Talet avrundas till dem plus decimalerna och skrivs med exponent (FMEXP). Annars avrundas det till decimalerna (FMRFIX) och skrivs i fast form (FMFIX): utfyllnad med blank eller * (**), tecknet ("-", med + i mallen även +), ¤ (¤¤), siffrorna med avskiljare (en blank per tre siffror, den tar en plats), punkten och decimalerna. Ett negativt tal utan + eller - i mallen tar en plats för -; en nolla står före punkten om det finns plats. CY: talet ryms inte (PRINT skriver det då i fri form med %*

före). FMEXP: mantissan med A siffror före punkten (FMFIX) och E+nn eller E-nn efter; in: HL = siffrorna, DE = utbufferten. In till FMUSE: HL = talet i decimal form, DE = utbufferten, IX = posten ur USEFLD. Ut: NC och DE efter texten; CY: talet ryms inte, DE kvar. Förstör A, BC, HL och AF'.

```
459F  46           FMUSE:  LD   B,(HL)
45A0  23                   INC  HL
45A1  4E                   LD   C,(HL)
45A2  23                   INC  HL
45A3  DD 7E 00             LD   A,(IX+$00)       ^^^^ (bit 5)?
45A6  CB 6F                BIT  5,A
45A8  28 5B                JR   Z,L4605
45AA  E6 18                AND  $18              + eller - i mallen?
45AC  DD 7E 03             LD   A,(IX+$03)
45AF  20 04                JR   NZ,L45B5
45B1  B7                   OR   A                nej: en plats för tecknet
45B2  28 01                JR   Z,L45B5
45B4  3D                   DEC  A
45B5  D5           L45B5:  PUSH DE
45B6  5F                   LD   E,A
45B7  DD CB 00 56          BIT  2,(IX+$00)       med avskiljare: ...
45BB  28 06                JR   Z,L45C3
45BD  CB 3B                SRL  E                ... platser - platser/4
45BF  CB 3B                SRL  E
45C1  93                   SUB  E
45C2  5F                   LD   E,A
45C3  F5           L45C3:  PUSH AF
45C4  DD 7E 02             LD   A,(IX+$02)       decimalerna ...
45C7  A7                   AND  A
45C8  28 01                JR   Z,L45CB
45CA  3D                   DEC  A
45CB  83           L45CB:  ADD  A,E              ... + platserna = värdesiffror
45CC  CD F7 46             CALL FMRMAX           avrunda
45CF  F1                   POP  AF
45D0  D1                   POP  DE
45D1  C5           FMEXP:  PUSH BC               C = siffror före punkten
45D2  4F                   LD   C,A
45D3  C5                   PUSH BC
45D4  CD 25 46             CALL FMFIX            mantissan i fast form
45D7  C1                   POP  BC
45D8  79                   LD   A,C
45D9  C1                   POP  BC
45DA  D8                   RET  C                ryms inte: CY
45DB  04                   INC  B                talet 0: exponent 0
45DC  05                   DEC  B
45DD  20 01                JR   NZ,L45E0
45DF  79                   LD   A,C
45E0  ED 44        L45E0:  NEG                   exponent = C - siffror före punkten
45E2  81                   ADD  A,C
45E3  36 45                LD   (HL),$45         E+ eller E-
45E5  23                   INC  HL
45E6  36 2B                LD   (HL),$2B
45E8  F2 EF 45             JP   P,L45EF
45EB  36 2D                LD   (HL),$2D
45ED  ED 44                NEG
45EF  23           L45EF:  INC  HL
45F0  FE 64                CP   $64              100 och över: 1 först
45F2  38 05                JR   C,L45F9
45F4  D6 64                SUB  $64
45F6  36 31                LD   (HL),$31
45F8  23                   INC  HL
45F9  36 2F        L45F9:  LD   (HL),$2F         tiotalet: / + 1 per 10
45FB  34           L45FB:  INC  (HL)
45FC  D6 0A                SUB  $0A
45FE  30 FB                JR   NC,L45FB
4600  23                   INC  HL
4601  D6 C6                SUB  $C6              entalet: A - 10 + 0
4603  77                   LD   (HL),A
4604  23                   INC  HL
4605  CC 22 46     L4605:  CALL Z,FMRFIX         fast form: avrunda och skriv
```

```
4608  D8                   RET  C
4609  DD CB 00 66          BIT  4,(IX+$00)      tecknet sist?
460D  28 11                JR   Z,L4620
460F  36 2D        L460F:  LD   (HL),$2D        -, + eller blank
4611  CB 78                BIT  7,B
4613  20 0A                JR   NZ,L461F
4615  36 2B                LD   (HL),$2B
4617  DD CB 00 5E          BIT  3,(IX+$00)
461B  20 02                JR   NZ,L461F
461D  36 20        L461D:  LD   (HL),$20
461F  23           L461F:  INC  HL
4620  EB           L4620:  EX   DE,HL           DE = efter texten
4621  C9                   RET
```

*FMRFIX: avrunda till decimalerna (FMRDEC) och skriv i fast form. FMFIX: skriv talet med huvudet B och C (antal siffror och exponent) och siffrorna vid HL i fast form enligt posten IX: utfyllnad med blank eller *, tecknet, ¤ och siffrorna (FMDIGS). +3 räknas ned med utfyllnaden och platsen för -. In: HL = siffrorna, DE = utbufferten. Ut: NC och HL efter texten; CY: talet ryms inte, DE kvar. Förstör A och DE.*

```
4622  CD E9 46     FMRFIX: CALL FMRDEC          avrunda till decimalerna
4625  DD 7E 00     FMFIX:  LD   A,(IX+$00)      + eller - i mallen: ingen plats ...
4628  E6 18                AND  $18
462A  DD 7E 03             LD   A,(IX+$03)
462D  20 0B                JR   NZ,L463A
462F  CB 78                BIT  7,B             ... annars en för -
4631  28 07                JR   Z,L463A
4633  A7                   AND  A
4634  37                   SCF                  ingen plats: CY
4635  C8                   RET  Z
4636  3D                   DEC  A
4637  DD 77 03             LD   (IX+$03),A
463A  0C           L463A:  INC  C               C <= 0: bara decimaler
463B  0D                   DEC  C
463C  FA 59 46             JP   M,L4659
463F  28 18                JR   Z,L4659
4641  D5                   PUSH DE
4642  5F                   LD   E,A
4643  DD CB 00 56          BIT  2,(IX+$00)      med avskiljare: ...
4647  28 0A                JR   Z,L4653
4649  79                   LD   A,C
464A  D6 03        L464A:  SUB  $03             ... C + 1 per tre siffror
464C  38 05                JR   C,L4653
464E  28 03                JR   Z,L4653
4650  0C                   INC  C
4651  18 F7                JR   L464A

4653  7B           L4653:  LD   A,E             A = platser över
4654  91                   SUB  C
4655  D1                   POP  DE
4656  D8                   RET  C               ryms inte
4657  18 04                JR   L465D

4659  A7           L4659:  AND  A               en plats för 0 före punkten
465A  28 01                JR   Z,L465D
465C  3D                   DEC  A
465D  EB           L465D:  EX   DE,HL           fyll A platser ...
465E  28 14                JR   Z,L4674
4660  C5                   PUSH BC
4661  47                   LD   B,A
4662  3E 20                LD   A,$20
4664  DD CB 00 46          BIT  0,(IX+$00)      ... med blank eller * (**)
4668  28 02                JR   Z,L466C
466A  3E 2A                LD   A,$2A
466C  77           L466C:  LD   (HL),A
466D  23                   INC  HL
466E  DD 35 03             DEC  (IX+$03)        +3 = platserna som är kvar
4671  10 F9                DJNZ L466C
4673  C1                   POP  BC
4674  DD 7E 00     L4674:  LD   A,(IX+$00)      tecknet före, om inte sist
```

```
4677  CB 67              BIT  4,A
4679  20 0D              JR   NZ,L4688
467B  36 2D              LD   (HL),$2D
467D  CB 78              BIT  7,B
467F  20 06              JR   NZ,L4687
4681  CB 5F              BIT  3,A
4683  28 03              JR   Z,L4688
4685  36 2B              LD   (HL),$2B
4687  23         L4687:  INC  HL
4688  CB 4F      L4688:  BIT  1,A          ¤ efter tecknet (¤¤)
468A  28 03              JR   Z,L468F
468C  36 24              LD   (HL),$24
468E  23                 INC  HL
468F  DD 7E 03     L468F:  LD   A,(IX+$03)   A = platser före punkten
```

FMDIGS: *skriv B siffror (bit 7 = tecken) från DE med punkten och avskiljarna. Platserna räknas från A före punkten ned till den sista decimalen: plats 1 är entalet och 0 den första decimalen, med punkten före; en avskiljare tar en plats, och platser utan siffror blir 0. C = den första siffrans plats. Den sista siffran får avrundningens CY ur AF'. Punkten skrivs sist om mallen har den men inga decimaler (+2 = 1). In: HL = utbufferten, DE = siffrorna, A = platser före punkten. Ut: HL efter texten, NC.* FMZERO: *nollor från plats A ned till C + 1.* FMSEP: *före plats A: punkten när A = 0 (NC, sedan inga avskiljare), en blank vid plats 4, 8 osv. när +0 bit 2 är satt (CY, platsen tagen), annars inget (NC); A kvar.*

```
4692  CD C9 46   FMDIGS:  CALL FMZERO        nollor före siffrorna (A > C)
4695  C5                  PUSH BC
4696  CB B8               RES  7,B          B = antal utan tecken
4698  18 10               JR   L46AA

469A  79           L469A:  LD   A,C          punkt eller blank före siffran?
469B  0D                   DEC  C
469C  CD D7 46             CALL FMSEP
469F  38 F9                JR   C,L469A
46A1  10 01                DJNZ L46A4        sista siffran: ...
46A3  08                   EX   AF,AF'       ... + avrundningens CY
46A4  1A           L46A4:  LD   A,(DE)
46A5  CE 00                ADC  A,$00
46A7  13                   INC  DE
46A8  77                   LD   (HL),A
46A9  23                   INC  HL
46AA  04           L46AA:  INC  B
46AB  10 ED                DJNZ L469A
46AD  F1                   POP  AF
46AE  47                   LD   B,A
46AF  AF                   XOR  A            D = -(decimaler + punkten)
46B0  DD 96 02             SUB  (IX+$02)
46B3  57                   LD   D,A
46B4  28 01                JR   Z,L46B7
46B6  3C                   INC  A
46B7  5F           L46B7:  LD   E,A
46B8  79                   LD   A,C
46B9  4B                   LD   C,E          nollor till sista decimalen
46BA  CD C9 46             CALL FMZERO
46BD  A7                   AND  A
46BE  C0                   RET  NZ
46BF  B2                   OR   D            punkten sist, om ingen skrivits
46C0  C8                   RET  Z
46C1  36 2E        L46C1:  LD   (HL),$2E     efter punkten ingen avskiljare
46C3  23                   INC  HL
46C4  DD CB 00 96          RES  2,(IX+$00)
46C8  C9                   RET

46C9  B9           FMZERO: CP   C            nollor från plats A ned till C
46CA  C8                   RET  Z
46CB  F8                   RET  M
46CC  CD D7 46             CALL FMSEP
46CF  38 03                JR   C,L46D4
46D1  36 30                LD   (HL),$30
46D3  23                   INC  HL
46D4  3D           L46D4:  DEC  A
46D5  18 F2                JR   FMZERO
```

```
46D7  A7           FMSEP:  AND  A               plats 0: punkten
46D8  28 E7                JR   Z,L46C1
46DA  77                   LD   (HL),A          plats 4, 8 ...: ...
46DB  E6 03                AND  $03
46DD  7E                   LD   A,(HL)
46DE  C0                   RET  NZ
46DF  DD CB 00 56          BIT  2,(IX+$00)
46E3  C8                   RET  Z
46E4  36 20                LD   (HL),$20        ... blank som avskiljare
46E6  23                   INC  HL
46E7  37                   SCF
46E8  C9                   RET
```

FMRDEC: avrunda till decimalerna: siffror = decimaler + C, högst +1 (FMRMAX); under 0 blir talet 0 (B = C = 0, utan tecken). FMRND: avrunda till A siffror. Avslutande 0:or stryks; är siffran efter 5-9 stryks i stället 9:orna, och den sista som blir kvar ökas med 1 först när den skrivs (CY i AF', FMDIGS). Blir inga siffror kvar (alla 9:or, eller A = 0) ökar C med 1; vid avrundning uppåt pekar HL då på $4712, där $30 ("0") står som operand i LD A,$30, och siffran blir 1 (9.96 -> 10.0 med ##.#).

Nedåt till inga siffror står tecknet kvar: -0.04 blir -0.0 med ##.#, men -0.0004 (under 0 siffror) blir 0.0. In: B och C = huvudet, HL = siffrorna, IX = posten; L46FF: A = antal siffror. Ut: B och C avrundade, HL = siffrorna (eller $4712), AF' = CY till FMDIGS; DE kvar, A förstört.

```
46E9  DD 7E 02     FMRDEC: LD   A,(IX+$02)      decimalerna ...
46EC  A7                   AND  A
46ED  28 01                JR   Z,L46F0
46EF  3D                   DEC  A
46F0  81           L46F0:  ADD  A,C             ... + C = siffror som blir kvar
46F1  F2 F7 46             JP   P,FMRMAX
46F4  E2 2C 47             JP   PO,L472C        under 0: talet blir 0
46F7  DD BE 01     FMRMAX: CP   (IX+$01)        högst +1 siffror
46FA  38 03                JR   C,L46FF
46FC  DD 7E 01     FMRND:  LD   A,(IX+$01)
46FF  E5           L46FF:  PUSH HL              avrunda till A siffror
4700  D5                   PUSH DE
4701  50                   LD   D,B
4702  CB BA                RES  7,D
4704  BA                   CP   D               redan högst A: ingen avrundning
4705  30 1D                JR   NC,L4724
4707  5F                   LD   E,A
4708  AA                   XOR  D               B = tecknet och A
4709  A8                   XOR  B
470A  47                   LD   B,A
470B  16 00                LD   D,$00
470D  19                   ADD  HL,DE
470E  7E                   LD   A,(HL)          siffran efter: 5-9 uppåt
470F  FE 35                CP   $35
4711  3E 30                LD   A,$30           nedåt stryks 0:or före, ...
4713  38 03                JR   C,L4718
4715  3E 39                LD   A,$39           ... uppåt 9:or
4717  15                   DEC  D
4718  1C           L4718:  INC  E               A = 0: ingen siffra kvar
4719  1D           L4719:  DEC  E
471A  28 07                JR   Z,L4723
471C  2B                   DEC  HL
471D  BE                   CP   (HL)
471E  20 07                JR   NZ,L4727
4720  05                   DEC  B
4721  18 F6                JR   L4719

4723  0C           L4723:  INC  C               inga siffror kvar: C + 1 (egenhet)
4724  7E           L4724:  LD   A,(HL)          talet 0?
4725  FE 30                CP   $30
4727  7A           L4727:  LD   A,D
4728  D1                   POP  DE
4729  E1                   POP  HL
472A  20 03                JR   NZ,L472F
472C  AF           L472C:  XOR  A               talet blir 0 (utan tecken)
472D  47                   LD   B,A
472E  4F                   LD   C,A
```

```
472F  C6 01       L472F:  ADD  A,$01          AF' = CY till FMDIGS
4731  F5                PUSH AF
4732  08                EX   AF,AF'
4733  F1                POP  AF
4734  B0                OR   B
4735  E6 7F             AND  $7F
4737  C0                RET  NZ
4738  21 12 47          LD   HL,$4712         bara 9:or: 0 ($4712) + CY = "1"
473B  04                INC  B
473C  C9                RET
```

USEFLD: *läs ett fält i* PRINT USING*-mallen vid HL (B tecken kvar). NC och HL efter fältet: D = flaggor (se* $4500*), E = platser före punkten, C = decimaler + 1 (strängfält: antal tecken).*

```
#    en plats             ,   bit 2, en plats
.#   punkten och decimalerna
¤¤   bit 1: en plats, ¤ skrivs utöver
**   bit 0: två platser    **¤  dessutom bit 1
+    först bit 3; - sist bit 4, + sist bit 3 och 4
^^^^ efter siffrorna bit 5
!    D = $40, C = 1        &   D = $41
\  \ D = $40, C = n + 2    _   D = $FE, nästa tecken är text
```

*CY: inget fält, tecknet är text: ett ensamt ¤ eller *, . utan # efter, - först (-## = "-" och ##). In: HL = mallen, B = tecken kvar. Ut även B = tecken kvar efter fältet. Vid CY gäller inte HL och B: anroparen (S2096) tar dem från före anropet och prövar nästa tecken. Förstör A.*

```
473D  AF          USEFLD: XOR  A              D = flaggor, E = platser
473E  5F                  LD   E,A
473F  57          L473F:  LD   D,A
4740  7E                  LD   A,(HL)
4741  23                  INC  HL
4742  FE 21               CP   $21            ! första tecknet
4744  CA A5 47            JP   Z,L47A5
4747  FE 26               CP   $26            & hela strängen
4749  CA 9D 47            JP   Z,L479D
474C  FE 23               CP   $23            #
474E  28 35               JR   Z,L4785
4750  05                  DEC  B              sista tecknet: inget fält
4751  37                  SCF
4752  C8                  RET  Z
4753  FE 2B               CP   $2B            + först: bit 3, sedan fältet
4755  3E 08               LD   A,$08
4757  28 E6               JR   Z,L473F
4759  2B                  DEC  HL             tecknet igen
475A  7E                  LD   A,(HL)
475B  23                  INC  HL
475C  FE 2E               CP   $2E            .#
475E  28 5C               JR   Z,L47BC
4760  FE 5C               CP   $5C            \ \
4762  28 47               JR   Z,L47AB
4764  FE 5F               CP   $5F            _ nästa tecken som det är
4766  28 39               JR   Z,L47A1
4768  BE                  CP   (HL)           annars dubbelt: ¤¤ eller **
4769  37                  SCF
476A  C0                  RET  NZ
476B  FE 24               CP   $24
476D  28 13               JR   Z,L4782
476F  FE 2A               CP   $2A
4771  37                  SCF
4772  C0                  RET  NZ
4773  1C                  INC  E              **: bit 0, två platser
4774  78                  LD   A,B
4775  FE 02               CP   $02
4777  23                  INC  HL
4778  38 03               JR   C,L477D
477A  7E                  LD   A,(HL)         **¤?
477B  FE 24               CP   $24
477D  CB C2       L477D:  SET  0,D
477F  20 04               JR   NZ,L4785
```

```
4781  05              DEC  B
4782  CB CA    L4782: SET  1,D          ¤¤: bit 1, en plats
4784  23              INC  HL
4785  1C       L4785: INC  E            #: en plats
4786  0E 00           LD   C,$00
4788  A7              AND  A
4789  05              DEC  B
478A  C8              RET  Z
478B  7E              LD   A,(HL)
478C  23              INC  HL
478D  FE 2E           CP   $2E           .: decimalerna
478F  28 33           JR   Z,L47C4
4791  FE 23           CP   $23
4793  28 F0           JR   Z,L4785
4795  FE 2C           CP   $2C           ,: bit 2, tar en plats
4797  20 35           JR   NZ,L47CE
4799  CB D2           SET  2,D
479B  18 E8           JR   L4785

479D  16 41    L479D: LD   D,$41         &: $41
479F  05              DEC  B
47A0  C9              RET

47A1  16 FE    L47A1: LD   D,$FE         _: $FE
47A3  05              DEC  B
47A4  C9              RET

47A5  0E 01    L47A5: LD   C,$01         !: $40, ett tecken
47A7  16 40    L47A7: LD   D,$40
47A9  05              DEC  B
47AA  C9              RET

47AB  0E 02    L47AB: LD   C,$02         \: blanka till nästa \ ...
47AD  7E       L47AD: LD   A,(HL)
47AE  23              INC  HL
47AF  FE 5C           CP   $5C
47B1  28 F4           JR   Z,L47A7
47B3  FE 20           CP   $20
47B5  20 03           JR   NZ,L47BA
47B7  0C              INC  C             ... n + 2 tecken
47B8  10 F3           DJNZ L47AD
47BA  37       L47BA: SCF                inget fält
47BB  C9              RET

47BC  7E       L47BC: LD   A,(HL)
47BD  FE 23           CP   $23
47BF  37              SCF
47C0  C0              RET  NZ
47C1  0E 01           LD   C,$01         C = punkten + decimalerna
47C3  23              INC  HL
47C4  0C       L47C4: INC  C
47C5  A7              AND  A
47C6  05              DEC  B
47C7  C8              RET  Z
47C8  7E              LD   A,(HL)
47C9  23              INC  HL
47CA  FE 23           CP   $23
47CC  28 F6           JR   Z,L47C4
47CE  E5       L47CE: PUSH HL
47CF  FE 5E           CP   $5E           ^^^^: bit 5
47D1  20 16           JR   NZ,L47E9
47D3  BE              CP   (HL)
47D4  20 13           JR   NZ,L47E9
47D6  23              INC  HL
47D7  BE              CP   (HL)
47D8  20 0F           JR   NZ,L47E9
47DA  23              INC  HL
47DB  BE              CP   (HL)
47DC  20 0B           JR   NZ,L47E9
47DE  23              INC  HL
47DF  23              INC  HL
```

```
47E0  78                LD   A,B
47E1  D6 04             SUB  $04
47E3  38 04             JR   C,L47E9
47E5  E3                EX   (SP),HL
47E6  CB EA             SET  5,D
47E8  47                LD   B,A
47E9  E1          L47E9: POP  HL            HL = efter fältet
47EA  2B                DEC  HL
47EB  AF                XOR  A
47EC  B0                OR   B
47ED  C8                RET  Z
47EE  CB 5A             BIT  3,D            + först: inget tecken sist
47F0  C0                RET  NZ
47F1  7E                LD   A,(HL)
47F2  FE 2D             CP   $2D            - sist: bit 4
47F4  28 05             JR   Z,L47FB
47F6  EE 2B             XOR  $2B            + sist: bit 3 och 4
47F8  C0                RET  NZ
47F9  CB DA             SET  3,D
47FB  CB E2       L47FB: SET  4,D
47FD  23                INC  HL
47FE  05                DEC  B
47FF  C9                RET
```

NUMPSH: *talkonstanten (A bytes vid HL) till stacken, utfylld med nollor till 8 bytes och avrundad till 4 utan* DOUBLE (FRND4). *För* NUMCON *och* LIST (DECOMP). *In: A = antal bytes, HL = konstanten. Ut: talet vid SP, IY+*$1E *bytes, HL efter konstanten; förstör A, BC, DE och IX.*

```
4800  DD E1       NUMPSH: POP  IX            IX = returadressen
4802  11 00 00            LD   DE,$0000
4805  42                  LD   B,D
4806  D5                  PUSH DE             8 nollbytes på stacken
4807  D5                  PUSH DE
4808  D5                  PUSH DE
4809  D5                  PUSH DE
480A  EB                  EX   DE,HL
480B  39                  ADD  HL,SP
480C  EB                  EX   DE,HL
480D  4F                  LD   C,A            A bytes ur koden ...
480E  B7                  OR   A
480F  28 02               JR   Z,L4813
4811  ED B0               LDIR                ... över nollorna
4813  FD CB 1E 56 L4813:  BIT  2,(IY+$1E)     inte DOUBLE: ...
4817  DD E5               PUSH IX
4819  C8                  RET  Z
481A  C3 99 4C            JP   FRND4          ... avrunda till 4 bytes
```

TXTNUM: *talet i texten vid HL som flyttal (IY+*$1E *bytes) vid DE, för kompileringen (konstanter, alltid 8 bytes) och* INPUT/READ *(direkt i variabeln). Ut: HL efter talet, DE efter flyttalet; CY och A = fel 210 (inget tal) eller 130 (spill). Vid fel 210 är HL kvar vid talets början. Förstör BC och IX.*

```
481D  D5          TXTNUM: PUSH DE           DE = målet, HL = texten
481E  E5                  PUSH HL
481F  CD 6D 48            CALL TXTDEC       decimal form på stacken
4822  3E D2               LD   A,$D2        fel 210: inget tal
4824  38 24               JR   C,L484A
4826  EB                  EX   DE,HL
4827  CD 4D 48            CALL DECEND       IX = platsen för HL ...
482A  DD 73 00            LD   (IX+$00),E   ... = efter talet i texten
482D  DD 72 01            LD   (IX+$01),D
4830  CD 90 52            CALL DECTOF       till flyttal
4833  3E 82               LD   A,$82        fel 130: spill
4835  38 13               JR   C,L484A
4837  21 00 00            LD   HL,$0000
483A  FD 4E 1E            LD   C,(IY+$1E)   talet (IY+$1E bytes) ...
483D  44                  LD   B,H
483E  39                  ADD  HL,SP
483F  E5                  PUSH HL
```

```
4840  09                ADD  HL,BC
4841  23                INC  HL
4842  23                INC  HL
4843  5E                LD   E,(HL)          ... till målet (sparade DE)
4844  23                INC  HL
4845  56                LD   D,(HL)
4846  E1                POP  HL
4847  ED B0             LDIR
4849  F9                LD   SP,HL
484A  E1          L484A:  POP  HL            HL = efter talet i texten
484B  C1                POP  BC
484C  C9                RET
```

DECEND: talet i decimal form under returadressen. Ut: IX efter talet, HL = talet, BC = antal siffror; A kvar.

```
484D  21 02 00    DECEND: LD   HL,$0002      talet under returadressen
4850  39                ADD  HL,SP
4851  E5                PUSH HL
4852  4E                LD   C,(HL)          C = antal siffror
4853  23                INC  HL
4854  23                INC  HL
4855  CB B9             RES  7,C
4857  06 00             LD   B,$00
4859  09                ADD  HL,BC           IX = efter talet
485A  E3                EX   (SP),HL
485B  DD E1             POP  IX
485D  C9                RET
```

Text till decimal form: STRDEC (sträng på stacken: VAL, ADD$ m.fl.), USEDEC (sträng till talfält i PRINT USING), TXTDEC (text vid HL) Formen är [blanka] [+|-] siffror [. siffror] [E|e [+|-] siffror] [blanka], minst en siffra och ingen blank efter tecknet. Siffrorna räknas först och kopieras sist (utan punkten) från texten till stacken, under huvudet B (antal, bit 7 = tecken) och C (exponent = heltalssiffrorna + exponenten); DECNRM stryker sedan nollorna. Högst 126 siffror, exponenten högst 255 (128-255 slår runt, se $48F3). En sträng måste vara slut efter talet (VAL("12X") och VAL("1E") ger fel 210); i en text (TXTDEC, C = $FF) slutar talet vid första tecknet som inte hör dit. Vid fel CY, och SP hämtas ur HL i den andra registeruppsättningen. FNCHAR: nästa tecken, C tecken kvar ($FF = ingen gräns); NC = siffra, CY och Z = slut. In: STRDEC: strängen vid SP under returadressen (bytes på stacken, adress, längd). USEDEC: HL = texten, BC = längden, IX = PRINT:s ram. TXTDEC: HL = texten. Ut: NC och talet i decimal form vid SP (i strängens ställe), HL efter talet i texten; förstör A, BC, DE och IX. CY: SP som före anropet, men strängen borttagen. DIGCNT: som FNCHAR, men fel när E redan har 127 siffror.

```
485E  DD E1       STRDEC: POP  IX            strängen på stacken: ...
4860  E1                POP  HL              ... HL = längden
4861  D9                EXX
4862  E1                POP  HL              HL' = adressen, C' = längden
4863  C1                POP  BC
4864  D9                EXX
4865  18 0F             JR   L4876           HL = SP efter strängen (vid fel)

4867  DD E3       USEDEC: EX   (SP),IX        gamla IX (PRINT:s ram) ...
4869  D9                EXX
486A  E1                POP  HL              ... = SP vid fel
486B  18 0A             JR   L4877

486D  01 FF 00    TXTDEC: LD   BC,$00FF      text vid HL till decimal form (C = $FF: ingen längd)
4870  DD E1             POP  IX
4872  D9                EXX
4873  21 00 00          LD   HL,$0000        HL' = SP vid fel
4876  39          L4876:  ADD  HL,SP
4877  D9          L4877:  EXX
4878  04                INC  B               B <> 0: över 255 tecken, fel
4879  10 6C             DJNZ L48E7
487B  2B                DEC  HL
487C  58                LD   E,B             E = siffror + tecken, D = före punkten
487D  50                LD   D,B
487E  CD 3C 49    L487E:  CALL FNCHAR          blanka före hoppas över
4881  30 13             JR   NC,L4896
4883  FE 20             CP   $20
4885  28 F7             JR   Z,L487E
4887  FE 2B             CP   $2B             + eller - ...
4889  28 06             JR   Z,L4891
```

```
488B  FE 2D              CP   $2D
488D  20 0D              JR   NZ,L489C
488F  1E 80              LD   E,$80          ... -: bit 7
4891  CD 36 49     L4891: CALL DIGCNT        nästa, högst 126 siffror
4894  38 06              JR   C,L489C
4896  14           L4896: INC  D             en siffra före punkten
4897  1C                 INC  E
4898  10 F7              DJNZ L4891
489A  18 4B              JR   L48E7

489C  FE 2E        L489C: CP   $2E           .: siffrorna efter bara i E
489E  20 07              JR   NZ,L48A7
48A0  1D                 DEC  E
48A1  1C           L48A1: INC  E
48A2  CD 36 49           CALL DIGCNT
48A5  30 FA              JR   NC,L48A1
48A7  E5           L48A7: PUSH HL            HL = efter siffrorna
48A8  47                 LD   B,A
48A9  E6 5F              AND  $5F            E eller e: exponent
48AB  FE 45              CP   $45
48AD  20 52              JR   NZ,L4901
48AF  D5                 PUSH DE
48B0  C5                 PUSH BC
48B1  CD 3C 49           CALL FNCHAR
48B4  47                 LD   B,A
48B5  30 0D              JR   NC,L48C4
48B7  FE 2B              CP   $2B            + eller - ...
48B9  28 04              JR   Z,L48BF
48BB  FE 2D              CP   $2D
48BD  20 2E              JR   NZ,L48ED
48BF  CD 3C 49     L48BF: CALL FNCHAR        ... och en siffra
48C2  38 29              JR   C,L48ED
48C4  D1           L48C4: POP  DE
48C5  C5                 PUSH BC
48C6  16 00              LD   D,$00          E = exponenten
48C8  D6 30              SUB  $30
48CA  5F                 LD   E,A
48CB  CD 3C 49     L48CB: CALL FNCHAR
48CE  47                 LD   B,A
48CF  38 22              JR   C,L48F3
48D1  EB                 EX   DE,HL          DE * 10 + siffran ...
48D2  7D                 LD   A,L
48D3  29                 ADD  HL,HL
48D4  29                 ADD  HL,HL
48D5  85                 ADD  A,L
48D6  6F                 LD   L,A
48D7  38 0B              JR   C,L48E4
48D9  29                 ADD  HL,HL
48DA  7C                 LD   A,H            ... över 255: fel
48DB  B7                 OR   A
48DC  37                 SCF
48DD  20 05              JR   NZ,L48E4
48DF  78                 LD   A,B
48E0  E6 0F              AND  $0F
48E2  85                 ADD  A,L
48E3  6F                 LD   L,A
48E4  EB           L48E4: EX   DE,HL
48E5  30 E4              JR   NC,L48CB
48E7  D9           L48E7: EXX
48E8  F9                 LD   SP,HL
48E9  D9                 EXX
48EA  37                 SCF
48EB  DD E9              JP   (IX)

48ED  C1           L48ED: POP  BC            ingen exponent: E är text
48EE  D1                 POP  DE
48EF  E1                 POP  HL
48F0  E5                 PUSH HL
48F1  18 0E              JR   L4901
```

```
48F3  F1          L48F3:  POP  AF          exponenten med tecken ... (fel)
48F4  FE 2D               CP   $2D
48F6  7B                  LD   A,E
48F7  20 02               JR   NZ,L48FB
48F9  ED 44               NEG
48FB  D1          L48FB:  POP  DE
48FC  82                  ADD  A,D         ... + D (bara spill prövas)
48FD  EA E7 48            JP   PE,L48E7
4900  57                  LD   D,A
4901  78          L4901:  LD   A,B         blanka efter hoppas över
4902  EE 20       L4902:  XOR  $20
4904  CC 3C 49            CALL Z,FNCHAR
4907  38 F9               JR   C,L4902
4909  0C                  INC  C           sträng: allt måste vara läst
490A  20 DB               JR   NZ,L48E7
490C  3E 7F               LD   A,$7F       inga siffror: fel
490E  A3                  AND  E
490F  28 D6               JR   Z,L48E7
4911  D9                  EXX              C = antal siffror
4912  4F                  LD   C,A
4913  D1                  POP  DE          DE = efter siffrorna i texten
4914  06 00               LD   B,$00
4916  ED 42               SBC  HL,BC       plats under SP? ...
4918  ED 72               SBC  HL,SP
491A  39                  ADD  HL,SP
491B  30 01               JR   NC,L491E
491D  F9                  LD   SP,HL       ... annars SP ned
491E  09          L491E:  ADD  HL,BC
491F  EB                  EX   DE,HL
4920  3E 2E               LD   A,$2E       siffrorna bakifrån, utan .
4922  1B                  DEC  DE
4923  2B          L4923:  DEC  HL
4924  BE          L4924:  CP   (HL)
4925  28 FC               JR   Z,L4923
4927  ED A8               LDD
4929  EA 24 49            JP   PE,L4924
492C  EB                  EX   DE,HL
492D  23                  INC  HL
492E  F9                  LD   SP,HL       SP = siffrorna
492F  D9                  EXX
4930  D5                  PUSH DE          huvudet: antal, exponent
4931  DD E5               PUSH IX          DECNRM, sedan tillbaka via IX
4933  C3 7B 4D            JP   DECNRM

4936  3E 80       DIGCNT: LD   A,$80       127 siffror: fel
4938  B3                  OR   E
4939  3C                  INC  A
493A  28 AB               JR   Z,L48E7
493C  79          FNCHAR: LD   A,C         C = $FF: ingen gräns
493D  23                  INC  HL
493E  3C                  INC  A
493F  28 04               JR   Z,L4945
4941  0D                  DEC  C           annars C - 1, ...
4942  3D                  DEC  A
4943  37                  SCF
4944  C8                  RET  Z           ... slut: CY och Z
4945  7E          L4945:  LD   A,(HL)      NC: en siffra
4946  FE 30               CP   $30
4948  D8                  RET  C
4949  FE 3A               CP   $3A
494B  3F                  CCF
494C  C9                  RET
```

3.33 DVCALL: anropa drivrutinen för FCB:n IX med funktionen A $494D

Funktionen sparas i FCB+$0A. Klarar drivrutinen det själv (CY = 0) är det klart. CY = 1 med A = $FF betyder att drivrutinen bara flyttar hela block (DOS:en): då sköter BASIC:en själv tecknen i en buffert på 256 bytes på sidan FCB+$14 (DVBUF).

Ett block har plats 3-255 för data (253 bytes); plats 0-2 lämnas åt drivrutinen (DOS:en: sektorhuvudet). Textfiler: en rad slutar med CR, LF lagras inte; två eller fler blanka i följd lagras som $09 n (n blanka, mod 256); $03 (ETX) avslutar blockets data, och $00 är filens slut. Plats 255 får alltid ETX från PRINT; PUT fyller blocket ända ut.

FCB-fälten för bufferten:

```
+$0A      senaste funktion
+$0D,$0E  antalet block i filen ($FFFF = okänt)
+$0F,$10  blocket i bufferten ($FFFF = inget)
+$11,$12  aktuellt block
+$13      plats i blocket (3-255, 0 = nästa block)
+$14      buffertens sida (från OPEN)
+3 bit 3  bufferten är ändrad
```

CONLIN: INPUT från fil 0. DVINP: INPUT (funktion 3). In: IX = FCB:n, HL = texten, BC = längden (vid INPUT: utrymmet). Ut: CY och A = felet; går det genom DVBUF står HL efter texten.

```
494D  AF           CONLIN: XOR  A              IX = fil 0
494E  CD 88 11             CALL FNDFIL
4951  3E 03         DVINP:  LD   A,$03          funktion 3: INPUT
4953  DD 77 0A      DVCALL: LD   (IX+$0A),A     senaste funktion
4956  CD 52 11             CALL DRVJP
4959  D0                   RET  NC             drivrutinen klarade det
495A  FE FF                CP   $FF            $FF: BASIC:en sköter tecknen
495C  D8                   RET  C              annat fel: CY
```

DVBUF: flytta BC tecken mellan HL och bufferten, block för block. Nästa block hämtas med BUFBLK; plats 0 = gå till nästa block, plats 3.

```
495D  E5           DVBUF:  PUSH HL             texten
495E  CD 9C 49             CALL POSLD          HL = blocket, E = platsen
4961  7B           L4961:  LD   A,E
4962  A7                   AND  A
4963  20 03                JR   NZ,L4968       platsen 0: nästa block, plats 3
4965  23                   INC  HL
4966  1E 03                LD   E,$03
4968  C5           L4968:  PUSH BC
4969  D5                   PUSH DE
496A  CD 5F 4A             CALL BUFBLK         blocket till bufferten
496D  D1                   POP  DE
496E  C1                   POP  BC
496F  DD 56 14             LD   D,(IX+$14)     DE = platsen i bufferten
4972  E3                   EX   (SP),HL        HL = texten
4973  D4 A6 49             CALL NC,BUFXFR      flytta
4976  E3                   EX   (SP),HL
4977  38 07                JR   C,L4980        fel: positionen står kvar
4979  78                   LD   A,B            mer kvar: nästa block
497A  B1                   OR   C
497B  20 E4                JR   NZ,L4961
497D  CD 92 49             CALL POSST          spara positionen
4980  E1           L4980:  POP  HL             HL = efter texten
4981  C9                   RET
```

DVEND: var sista funktionen PRINT och är bufferten ändrad avslutas filen med funktion 0 via DVBUF: ETX efter texten och sedan ett block som börjar med sex $00 och ETX (filslut); står positionen i början av ett block blir det bara det blocket. Resten av bufferten töms inte. Efter PUT eller INPUT görs ingenting.

```
4982  DD 7E 0A     DVEND:  LD   A,(IX+$0A)     senast PRINT? ... (egenhet)
4985  EE 04                XOR  $04
4987  C0                   RET  NZ
4988  DD CB 03 5E          BIT  3,(IX+$03)     ... och ändrad: ...
498C  DD 77 0A             LD   (IX+$0A),A     ... funktion 0
498F  20 CC                JR   NZ,DVBUF
4991  C9                   RET
```

POSST: spara positionen, FCB+$11/$12 = HL (blocket) och FCB+$13 = E (platsen). POSLD *läser tillbaka den. Inget annat ändras.*

```
4992  DD 75 11     POSST:  LD   (IX+$11),L
4995  DD 74 12             LD   (IX+$12),H
4998  DD 73 13             LD   (IX+$13),E
499B  C9                   RET

499C  DD 6E 11     POSLD:  LD   L,(IX+$11)
499F  DD 66 12             LD   H,(IX+$12)
49A2  DD 5E 13             LD   E,(IX+$13)
49A5  C9                   RET
```

BUFXFR: *flytta mellan bufferten (DE) och HL efter FCB+$0A:*

```
3 BINPUT  läs en rad: $09 n blir n blanka, $03 = blockets slut,
          $00 var som helst = filslut (fel 34), bit 7 satt ger
          fel 58; raden med CR ska rymmas i C - 1 bytes, annars
          fel 20; platsen prövas inte (se $49E6)
4 BPRINT  skriv text: två eller fler blanka blir $09 n (BSPACE,
          inte från plats 254), blanka direkt före CR i samma
          anrop stryks; FCB+6 räknar kolumnen, CR nollställer den;
          LF skrivs inte; plats 255 får alltid ETX
5 BGET    kopiera BC bytes ut
6 BPUT    kopiera BC bytes in
0         filslut (DVEND)
```

Alla utom 3 och 5 markerar bufferten som ändrad (FCB+3 bit 3).

```
49A6  DD 7E 0A     BUFXFR: LD   A,(IX+$0A)
49A9  FE 03                CP   $03
49AB  28 36                JR   Z,BINPUT
49AD  FE 05                CP   $05
49AF  28 26                JR   Z,BGET
49B1  DD CB 03 DE          SET  3,(IX+$03)       bufferten ändrad
49B5  FE 04                CP   $04
49B7  28 78                JR   Z,BPRINT
49B9  30 12                JR   NC,BPUT
49BB  01 00 06             LD   BC,$0600         funktion 0: B = 6
49BE  7B                   LD   A,E              i början av blocket?
49BF  EE 03                XOR  $03
49C1  20 04                JR   NZ,L49C7         inte: bara ETX
49C3  12           L49C3:  LD   (DE),A           sex $00: filslut
49C4  13                   INC  DE
49C5  10 FC                DJNZ L49C3
49C7  3E 03        L49C7:  LD   A,$03            ETX
49C9  12                   LD   (DE),A
49CA  1E 00        L49CA:  LD   E,$00            E = 0: nästa block
49CC  C9                   RET

49CD  78           BPUT:   LD   A,B
49CE  B1                   OR   C
49CF  C8                   RET  Z                BC = 0: klart
49D0  ED A0                LDI                   in i bufferten ...
49D2  7B                   LD   A,E              ... till blockets slut
49D3  B7                   OR   A
49D4  20 F7                JR   NZ,BPUT
49D6  C9                   RET

49D7  78           BGET:   LD   A,B
49D8  B1                   OR   C
49D9  C8                   RET  Z
49DA  EB                   EX   DE,HL
49DB  ED A0                LDI                   ut ur bufferten
49DD  EB                   EX   DE,HL
49DE  7B                   LD   A,E
49DF  B7                   OR   A
49E0  20 F5                JR   NZ,BGET
49E2  C9                   RET

49E3  AF           BINPUT: XOR  A                C = utrymmet kvar
49E4  B1                   OR   C
49E5  C8                   RET  Z                C = 0: klart
```

```
49E6  1A          L49E6:  LD   A,(DE)        bit 7: fel 58 (fel)
49E7  87                  ADD  A,A
49E8  3E 3A               LD   A,$3A
49EA  D8                  RET  C
49EB  3E 22               LD   A,$22         $00: fel 34 (filslut)
49ED  37                  SCF
49EE  C8                  RET  Z
49EF  1A                  LD   A,(DE)
49F0  1C                  INC  E             nästa plats (255 blir 0 utan prövning)
49F1  FE 03               CP   $03           ETX: nästa block
49F3  28 D5               JR   Z,L49CA
49F5  06 01               LD   B,$01         ett tecken
49F7  FE 09               CP   $09           $09 n: n blanka
49F9  20 05               JR   NZ,L4A00
49FB  1A                  LD   A,(DE)        n
49FC  47                  LD   B,A
49FD  1C                  INC  E
49FE  3E 20               LD   A,$20         blank
4A00  77          L4A00:  LD   (HL),A        till texten
4A01  23                  INC  HL
4A02  0D                  DEC  C             C räknar CR:en med
4A03  28 0A               JR   Z,L4A0F
4A05  10 F9               DJNZ L4A00         n gånger
4A07  FE 0D               CP   $0D           CR: klart, BC = 0
4A09  20 DB               JR   NZ,L49E6
4A0B  01 00 00            LD   BC,$0000
4A0E  C9                  RET

4A0F  3E 14       L4A0F:  LD   A,$14         fel 20: raden för lång
4A11  37                  SCF
4A12  C9                  RET

4A13  D5          BSPACE: PUSH DE            platsen i bufferten (fel)
4A14  1E 00               LD   E,$00         E = 0 blanka
4A16  23          L4A16:  INC  HL            nästa blank
4A17  0B                  DEC  BC
4A18  1C                  INC  E             E räknar blanka (mod 256)
4A19  DD 34 06            INC  (IX+$06)      kolumnen + 1
4A1C  78                  LD   A,B
4A1D  B1                  OR   C
4A1E  28 09               JR   Z,L4A29       slut på texten: $09 n
4A20  7E                  LD   A,(HL)
4A21  FE 20               CP   $20
4A23  28 F1               JR   Z,L4A16
4A25  FE 0D               CP   $0D
4A27  28 07               JR   Z,L4A30       CR efter: blanka stryks
4A29  E3          L4A29:  EX   (SP),HL       HL = bufferten
4A2A  36 09               LD   (HL),$09      $09 n
4A2C  23                  INC  HL
4A2D  73                  LD   (HL),E
4A2E  23                  INC  HL
4A2F  E3                  EX   (SP),HL       HL = texten igen
4A30  D1          L4A30:  POP  DE
4A31  78          BPRINT: LD   A,B
4A32  B1                  OR   C
4A33  C8                  RET  Z             BC = 0: klart
4A34  7B                  LD   A,E
4A35  3C                  INC  A
4A36  28 8F               JR   Z,L49C7       plats 255: ETX
4A38  3C                  INC  A             plats 254: ingen packning
4A39  7E                  LD   A,(HL)        tecknet
4A3A  28 09               JR   Z,L4A45
4A3C  FE 20               CP   $20           blank och blank efter?
4A3E  20 05               JR   NZ,L4A45
4A40  23                  INC  HL            nästa tecken
4A41  BE                  CP   (HL)
4A42  2B                  DEC  HL
4A43  28 CE               JR   Z,BSPACE
4A45  DD 34 06    L4A45:  INC  (IX+$06)      kolumnen + 1
```

```
4A48  FE 0D              CP   $0D
4A4A  20 04              JR   NZ,L4A50
4A4C  DD 36 06 00        LD   (IX+$06),$00     CR: kolumnen = 0
4A50  ED A0        L4A50: LDI                  till bufferten
4A52  FE 0A              CP   $0A
4A54  20 DB              JR   NZ,BPRINT
4A56  1B                 DEC  DE               LF skrivs inte
4A57  DD 35 06           DEC  (IX+$06)
4A5A  18 D5              JR   BPRINT
```

BUFFLU: *skriv bufferten om den är ändrad (HL = $FFFF).*

BUFBLK: *se till att block HL ligger i bufferten. Ligger det redan där är det klart (Z, NC, A = 0). Ett ändrat block skrivs först (funktion 8, DE = blocknumret);* BUFFLU *slutar där, med NC om skrivningen gick bra. Är antalet block* $FFFF *(okänt) läses blocket alltid; drivrutinen får funktion 7 med DE = blocket och HL = antalet. Block bortom filens slut (FCB+$0D) läses inte utan blir nya; filen växer. Annars läses blocket (funktion 7); ett nytt block töms inte utan får det som låg i bufferten. Fel 37 eller 38 från drivrutinen (blocket finns inte) står kvar vid* INPUT *och* GET *(*NULDRV *ger CY, A tas tillbaka ur C); vid* PRINT *och* PUT *blir blocket nytt. Andra fel går vidare.*

```
4A5C  21 FF FF  BUFFLU: LD   HL,$FFFF         bara skriva
4A5F  DD 5E 0F  BUFBLK: LD   E,(IX+$0F)       blocket redan i bufferten?
4A62  DD 56 10          LD   D,(IX+$10)
4A65  AF                XOR  A                A = 0, NC
4A66  ED 52             SBC  HL,DE
4A68  19                ADD  HL,DE
4A69  C8                RET  Z
4A6A  E5                PUSH HL               det önskade blocket
4A6B  F6 08             OR   $08              funktion 8: skriv blocket
4A6D  DD CB 03 5E       BIT  3,(IX+$03)
4A71  C4 52 11          CALL NZ,DRVJP         ändrad: skriv blocket DE
4A74  E1                POP  HL
4A75  D8                RET  C                skrivfel
4A76  DD CB 03 9E       RES  3,(IX+$03)
4A7A  7D                LD   A,L              HL = $FFFF: klart
4A7B  A4                AND  H
4A7C  3C                INC  A
4A7D  C8                RET  Z
4A7E  DD 5E 0D          LD   E,(IX+$0D)       DE = antal block
4A81  DD 56 0E          LD   D,(IX+$0E)
4A84  ED 52             SBC  HL,DE
4A86  19                ADD  HL,DE
4A87  23                INC  HL               HL + 1 = det nya antalet
4A88  D4 AF 4A          CALL NC,SETCNT        bortom slutet: filen växer
4A8B  2B                DEC  HL               HL tillbaka
4A8C  30 1A             JR   NC,SETBLK        nytt block: läses inte
4A8E  A7                AND  A
4A8F  3E 07             LD   A,$07            funktion 7: läs blocket
4A91  E5                PUSH HL
4A92  EB                EX   DE,HL            DE = blocket, HL = antalet
4A93  CD 52 11          CALL DRVJP
4A96  E1                POP  HL
4A97  30 0F             JR   NC,SETBLK        läst
4A99  4F                LD   C,A              felet undan
4A9A  D6 25             SUB  $25              37 ger 0, 38 ger 1: filens slut
4A9C  C6 FE             ADD  A,$FE            NC bara för 37 och 38
4A9E  79                LD   A,C              felet igen
4A9F  D8                RET  C                andra fel går vidare
4AA0  DD 7E 0A          LD   A,(IX+$0A)       funktionen
4AA3  CD B9 11          CALL NULDRV           CY för 3 och 5 (INPUT, GET)
4AA6  79                LD   A,C              felet
4AA7  D8                RET  C                INPUT, GET: felet
4AA8  DD 75 0F  SETBLK: LD   (IX+$0F),L       blocket i bufferten = HL
4AAB  DD 74 10          LD   (IX+$10),H
4AAE  C9                RET

4AAF  DD 75 0D  SETCNT: LD   (IX+$0D),L       antal block = HL
4AB2  DD 74 0E          LD   (IX+$0E),H
4AB5  C9                RET
```

PARSFN: dela upp filnamnet HL (BC tecken) i bufferten DE: +0 namn (8), +8 filtyp (3), +11 enhet (3), fyllda med blanka. Formen är [enhet:]namn[.typ]. Siffror och tecknen $41-$5D räknas, och tecknen $61-$7D görs till versaler; tecken från $80 godtas med bit 7 och 5 borttagna ($C8 = H, $B1 = $11), och ‘ ($60) blir @, som själv inte godtas. Enhetens tecken efter det tredje huggs av (TSTXYZ:F = TST:F). Namnet och typen får vara tomma (TST: och TST:.Q går). CY = 1 om texten är tom eller längre än 255, börjar med ett tecken under A, har mer än 8 tecken i namnet eller 3 i typen, eller tecken över (blank, ett andra :).

*MUL16: HL = HL * DE utan tecken, CY vid spill.*

*DIV16: HL = HL / DE, DE = resten; CY om DE = 0. DIVBC: som DIV16, med delaren i BC. DIV32: (DE * 65536 + HL) / BC, DE < BC, kvoten i HL, resten i DE.*

```
4AB6  AF           PARSFN: XOR  A(egenhet)
4AB7  90                   SUB  B
4AB8  D8                   RET  C              B <> 0: för långt
4AB9  B1                   OR   C
4ABA  37                   SCF
4ABB  C8                   RET  Z              tomt
4ABC  7E                   LD   A,(HL)
4ABD  FE 41                CP   $41
4ABF  D8                   RET  C              första tecknet under A
4AC0  2B                   DEC  HL
4AC1  06 0E                LD   B,$0E          14 blanka: namn, typ, enhet
4AC3  CD 0E 4B             CALL FILSPC
4AC6  CD F0 4A             CALL FNPART         namnet eller enheten
4AC9  FE 3A                CP   $3A            enhet: flytta den till +11
4ACB  20 12                JR   NZ,L4ADF
4ACD  D5                   PUSH DE
4ACE  E5                   PUSH HL
4ACF  06 03                LD   B,$03          tre tecken, resten huggs av
4AD1  21 0B 00     L4AD1:  LD   HL,$000B
4AD4  19                   ADD  HL,DE
4AD5  1A                   LD   A,(DE)
4AD6  77                   LD   (HL),A
4AD7  13                   INC  DE
4AD8  10 F7                DJNZ L4AD1
4ADA  E1                   POP  HL
4ADB  D1                   POP  DE
4ADC  CD F0 4A             CALL FNPART         sedan namnet
4ADF  06 08        L4ADF:  LD   B,$08          DE = +8
4AE1  13           L4AE1:  INC  DE
4AE2  10 FD                DJNZ L4AE1
4AE4  FE 2E                CP   $2E            filtyp
4AE6  06 03                LD   B,$03
4AE8  CC F5 4A             CALL Z,FNPRT1
4AEB  3E 01                LD   A,$01
4AED  81                   ADD  A,C            slut (C = $FF): NC, ...
4AEE  3F                   CCF                 ... annars tecken över, CY
4AEF  C9                   RET
```

FNPART: B = 8 blanka vid DE (FILSPC), sedan högst 8 tecken ur texten. FNPRT1: högst B tecken utan blankning. Tecken räknas med FNCHAR. Ut: A = tecknet efter, DE som vid ingången; förstör B och F.

```
4AF0  06 08        FNPART: LD   B,$08          8 blanka
4AF2  CD 0E 4B             CALL FILSPC
4AF5  D5           FNPRT1: PUSH DE
4AF6  CD 3C 49     L4AF6:  CALL FNCHAR
4AF9  30 0A                JR   NC,L4B05       siffra
4AFB  FE 41                CP   $41
4AFD  38 0D                JR   C,L4B0C        under A: slut
4AFF  E6 5F                AND  $5F            versaler, bit 7 bort
4B01  FE 5E                CP   $5E
4B03  30 07                JR   NC,L4B0C       ^ och över: slut
4B05  12           L4B05:  LD   (DE),A
4B06  13                   INC  DE
4B07  10 ED                DJNZ L4AF6          högst B tecken
4B09  CD 3C 49             CALL FNCHAR         tecknet efter
4B0C  D1           L4B0C:  POP  DE
4B0D  C9                   RET
```

FILSPC: B blanka vid DE. Förstör A; BC och DE bevaras.

```
4B0E  3E 20       FILSPC: LD   A,$20          B blanka vid DE
4B10  D5                  PUSH DE
4B11  C5                  PUSH BC
4B12  12          L4B12:  LD   (DE),A
4B13  13                  INC  DE
4B14  10 FC               DJNZ L4B12
4B16  C1                  POP  BC
4B17  D1                  POP  DE
4B18  C9                  RET
```

MUL16*: HL = HL * DE utan tecken, med skift och addition (16 varv, multiplikatorn HL en bit i taget från den högsta). In: HL och DE. Ut: HL = produkten, NC; CY om den blir över 65535 (HL är då inte produkten). Förstör A, BC (= gamla DE) och DE. Används av heltalsmultiplikationen (*IMULDV*), fältindexen, DIM (antalet element; `DIM X(2000,2000)` ger fel 201) och AUTO.*

```
4B19  42          MUL16:  LD   B,D            BC = DE, DE = summan
4B1A  4B                  LD   C,E
4B1B  11 00 00            LD   DE,$0000
4B1E  3E 10               LD   A,$10
4B20  29          L4B20:  ADD  HL,HL          nästa bit ur HL ...
4B21  EB                  EX   DE,HL
4B22  30 02               JR   NC,L4B26
4B24  09                  ADD  HL,BC          ... ettställd: + BC
4B25  D8                  RET  C              spill
4B26  3D          L4B26:  DEC  A
4B27  C8                  RET  Z
4B28  29                  ADD  HL,HL          summan * 2
4B29  EB                  EX   DE,HL
4B2A  30 F4               JR   NC,L4B20       NC: igen; CY: spill
4B2C  C9                  RET
```

DIV16*: HL = HL / DE utan tecken; DIVBC: HL = HL / BC; DIV32: DE och HL som ett 32-bitstal / BC, där DE måste vara mindre än BC. Division med återställning, en kvotbit per varv (17 varv). Ut: HL = kvoten, DE = resten, BC = nämnaren, NC; CY om nämnaren är 0 (DIV16, DIVBC). Förstör A. Resten dubblas i 16 bitar, så nämnaren får vara högst `$8000`; heltalsdivisionen delar beloppen (högst 32768), BASSTR med basen och $39CE med 253.*

```
4B2D  42          DIV16:  LD   B,D
4B2E  4B                  LD   C,E
4B2F  78          DIVBC:  LD   A,B            HL = HL / BC, DE = resten; CY om BC = 0
4B30  B1                  OR   C
4B31  37                  SCF                 BC = 0: CY
4B32  C8                  RET  Z
4B33  11 00 00            LD   DE,$0000       DE = resten
4B36  EB          DIV32:  EX   DE,HL          DE:HL / BC
4B37  3E 11               LD   A,$11          17 varv, 16 bitar
4B39  37                  SCF
4B3A  3F          L4B3A:  CCF
4B3B  EB                  EX   DE,HL
4B3C  ED 6A               ADC  HL,HL          kvotbiten in, täljarens bit ut
4B3E  3D                  DEC  A
4B3F  C8                  RET  Z
4B40  EB                  EX   DE,HL
4B41  ED 6A               ADC  HL,HL          ... in i resten
4B43  A7                  AND  A
4B44  ED 42               SBC  HL,BC          resten - nämnaren
4B46  30 F2               JR   NC,L4B3A       gick: kvotbit 1
4B48  09                  ADD  HL,BC          annars tillbaka, kvotbit 0
4B49  18 EF               JR   L4B3A
```

KWSRCH*: sök texten vid HL i nyckelordstabellen vid DE (se $06AA). Bokstäverna ($41-$5E) jämförs utan bit 5, så gemener godtas; annat måste vara exakt lika. Utan EXTEND hoppas blanka i texten och i tabellen över (`"p r i n t"`, ONERRORGOTO); med EXTEND kräver en blank i tabellen minst en blank i texten, och ett namn som slutar med bokstav eller siffra får inte följas av en sådan (ONERROR fel 220). Ordningen i tabellen avgör: `INPUT LINE` står före `INPUT`, så utan EXTEND blir `INPUT LINES$ INPUT LINE S$`. Z: funnet, A = koden, DE pekar på den, HL efter nyckelordet och blanka efter. NZ: inte funnet, HL oförändrad. In: HL = texten, DE = tabellen. Förstör A och DE; BC bevaras.*

```
4B4B  1A          KWSRCH: LD   A,(DE)
```

```
4B4C  FE FF              CP   $FF
4B4E  28 40              JR   Z,L4B90          $FF: tabellen slut
4B50  E5                 PUSH HL               texten och posten sparas
4B51  D5                 PUSH DE
4B52  13                 INC  DE
4B53  1A          L4B53: LD   A,(DE)
4B54  FE 20              CP   $20
4B56  28 0E              JR   Z,L4B66          blank i tabellen
4B58  E6 5F              AND  $5F
4B5A  D6 41              SUB  $41
4B5C  FE 1E              CP   $1E              bokstav ($41-$5E)?
4B5E  1A                 LD   A,(DE)
4B5F  30 14              JR   NC,L4B75
4B61  AE                 XOR  (HL)
4B62  E6 DF              AND  $DF              bokstav: gemen = versal
4B64  18 10              JR   L4B76

4B66  13          L4B66: INC  DE
4B67  FD CB 1D 56        BIT  2,(IY+$1D)
4B6B  28 E6              JR   Z,L4B53          utan EXTEND hoppas den över
4B6D  BE                 CP   (HL)             EXTEND: blank även i texten
4B6E  20 15              JR   NZ,L4B85
4B70  CD 19 4C           CALL S4C19            och fler blanka efter
4B73  18 DE              JR   L4B53

4B75  AE          L4B75: XOR  (HL)             annat: exakt lika
4B76  F5          L4B76: PUSH AF
4B77  CC 10 4C           CALL Z,NXTCHR         lika: nästa i texten
4B7A  F1                 POP  AF
4B7B  1A                 LD   A,(DE)
4B7C  13                 INC  DE
4B7D  28 D4              JR   Z,L4B53
4B7F  A7                 AND  A
4B80  28 24              JR   Z,L4BA6          $00 i tabellen: funnet
4B82  FA 92 4B           JP   M,L4B92          nästa kod: hela namnet lika
4B85  1B          L4B85: DEC  DE               olika: till nästa post
4B86  13          L4B86: INC  DE
4B87  1A                 LD   A,(DE)
4B88  A7                 AND  A
4B89  F2 86 4B           JP   P,L4B86
4B8C  F1                 POP  AF               posten bort, texten tillbaka
4B8D  E1                 POP  HL
4B8E  18 BB              JR   KWSRCH

4B90  A7          L4B90: AND  A                inte funnet: NZ, NC
4B91  C9                 RET

4B92  FD CB 1D 56 L4B92: BIT  2,(IY+$1D)(egenhet)
4B96  28 0E              JR   Z,L4BA6          utan EXTEND: funnet
4B98  7E                 LD   A,(HL)
4B99  CD AF 4B           CALL ALNUM            bokstav/siffra efter, ...
4B9C  38 08              JR   C,L4BA6
4B9E  1B                 DEC  DE
4B9F  1B                 DEC  DE
4BA0  1A                 LD   A,(DE)
4BA1  CD AF 4B           CALL ALNUM            ... och sist i namnet?
4BA4  30 E0              JR   NC,L4B86         båda: inget nyckelord
4BA6  D1          L4BA6: POP  DE               DE = koden
4BA7  33                 INC  SP
4BA8  33                 INC  SP
4BA9  CD 1A 4C           CALL SKIPB0           blanka efter hoppas över
4BAC  1A                 LD   A,(DE)
4BAD  BF                 CP   A                Z, A = koden
4BAE  C9                 RET
```

ALNUM: *NC om A är en siffra eller bokstav (A-Z, Ä Ö Å, även gemener och ‘ $60), annars CY. Tecken från $80 räknas utan bit 7 och 5: alla utom $DE, $DF, $FE och $FF godtas. @ och ^ ($5E) är inte bokstäver här, men ^ är det i KWSRCH. Förstör bara A.*

```
4BAF  FE 30       ALNUM: CP   $30
4BB1  D8                 RET  C                under 0
```

```
4BB2  FE 3A              CP   $3A
4BB4  3F                 CCF
4BB5  D0                 RET  NC              siffra: NC
4BB6  FE 41              CP   $41
4BB8  D8                 RET  C
4BB9  E6 5F              AND  $5F             bit 7 och 5 bort
4BBB  FE 5E              CP   $5E
4BBD  3F                 CCF                  $41-$5D: NC
4BBE  C9                 RET

4BBF  CD 10 4C     S4BBF: CALL NXTCHR
```

GETLNR: *läs ett decimalt heltal (radnummer) vid HL till DE.*

Utan EXTEND *hoppas blanka mellan siffrorna över (*GOTO 1 0 = 10*). NC: HL efter talet och blanka efter det, A = tecknet där. CY: ingen siffra (DE = 0), eller över 65535 (HL tillbaka vid början).*

S4BBF: *först ett tecken framåt (efter + eller -).* GETINT: *heltal med tecken. Utan prov av gränserna: -65535 blir 1 och 40000 blir -25536 (*INPUT *och* READ *till heltalsvariabler). Förstör A; BC bevaras (även i* GETINT*).*

```
4BC2  7E           GETLNR: LD   A,(HL)
4BC3  11 00 00             LD   DE,$0000
4BC6  D6 30                SUB  $30
4BC8  FE 0A                CP   $0A
4BCA  3F                   CCF
4BCB  D8                   RET  C             ingen siffra: CY, DE = 0
4BCC  E5                   PUSH HL
4BCD  7E           L4BCD:  LD   A,(HL)
4BCE  D6 30                SUB  $30
4BD0  FE 0A                CP   $0A
4BD2  38 07                JR   C,L4BDB
4BD4  33                   INC  SP            början bort
4BD5  33                   INC  SP
4BD6  CD 1A 4C             CALL SKIPB0
4BD9  A7                   AND  A
4BDA  C9                   RET

4BDB  E5           L4BDB:  PUSH HL
4BDC  62                   LD   H,D
4BDD  6B                   LD   L,E
4BDE  29                   ADD  HL,HL         DE * 10 ...
4BDF  38 16                JR   C,L4BF7
4BE1  29                   ADD  HL,HL
4BE2  38 13                JR   C,L4BF7
4BE4  19                   ADD  HL,DE
4BE5  38 10                JR   C,L4BF7
4BE7  29                   ADD  HL,HL
4BE8  38 0D                JR   C,L4BF7
4BEA  5F                   LD   E,A
4BEB  16 00                LD   D,$00
4BED  19                   ADD  HL,DE         ... + siffran
4BEE  38 07                JR   C,L4BF7
4BF0  EB                   EX   DE,HL
4BF1  E1                   POP  HL
4BF2  CD 10 4C             CALL NXTCHR        utan EXTEND blanka över
4BF5  18 D6                JR   L4BCD

4BF7  E1           L4BF7:  POP  HL            spill: HL = början, CY
4BF8  E1                   POP  HL
4BF9  C9                   RET

4BFA  CD 1A 4C     GETINT: CALL SKIPB0        heltal med tecken ur text vid HL
4BFD  FE 2B                CP   $2B
4BFF  28 BE                JR   Z,S4BBF       +
4C01  FE 2D                CP   $2D
4C03  20 BD                JR   NZ,GETLNR
4C05  CD BF 4B             CALL S4BBF         -
4C08  D8                   RET  C
4C09  EB                   EX   DE,HL
4C0A  CD 6D 22             CALL NEGHL         DE = -DE, modulo 65536
4C0D  EB                   EX   DE,HL
```

```
4C0E  A7                AND  A
4C0F  C9                RET
```

NXTCHR: nästa tecken. Blanktecken hoppas över utom med EXTEND (IY+$1D bit 2); då skiljer de namn och nyckelord åt. Utan EXTEND Z vid $00, med EXTEND alltid NZ. S4C19: ett steg framåt och SKIPB0; SKIPB0 hoppar över blanka vid HL även med EXTEND. In: HL = texten. Ut: HL vid tecknet, A = tecknet; inget annat ändras.

```
4C10  23          NXTCHR: INC  HL
4C11  FD 7E 1D            LD   A,(IY+$1D)
4C14  E6 04               AND  $04
4C16  7E                  LD   A,(HL)
4C17  C0                  RET  NZ              EXTEND: tecknet, NZ
4C18  2B                  DEC  HL
4C19  23          S4C19:  INC  HL
4C1A  7E          SKIPB0: LD   A,(HL)
4C1B  FE 20               CP   $20
4C1D  28 FA               JR   Z,S4C19         blanka hoppas över
4C1F  A7                  AND  A               Z vid $00
4C20  C9                  RET
```

3.34 Flyttal $4C21

Talen är binära (inte BCD som i ABC80-BASIC:en): exponentbyte (bias $80, 0 = talet noll), sedan mantissan med den mest signifikanta byten först; dess bit 7 är tecknet och den ledande ettan är underförstådd. Värde = 0,1mmm... * 2^(exp-$80). Längden (4 eller 8 bytes, SINGLE eller DOUBLE) står i IY+$1E; bit 2 = 4 bytes.

Under beräkningen ligger talen på Z80-stacken (under returadressen).

I register: E = exponent, D = tecken + mantissans första byte, BC (och för 8 bytes DE', BC') resten.

```
FLOAD  lägg talet vid HL på stacken     FDUP  kopiera översta talet
FPOP   flytta översta talet till DE     FDROP ta bort översta talet
FSTNP  kopiera det till DE (ingen kod i ROM:en hänvisar hit)
FSWAP  byt plats på de två översta      FCHS  byt tecken
FCOMP  jämför de två översta            FEXT8/FRND4  4 <-> 8 bytes
ITOFS  heltal -> flyttal                FTOIR/FTOIT  flyttal -> heltal
                                        (avrundat/avhugget, CY = för stort)
```

Talen ligger direkt på anroparens stack: rutinerna tar bort sin egen returadress, ändrar stacken och lägger tillbaka den. FCOMP: Z = lika, CY = det undre talet är mindre; bara exponenten avgör om ett tal är 0, så -0 (FCHS på 0) är lika med 0. FRND4 avrundar uppåt när bit 7 i femte byten är 1 (inte vid exponent $FF). FNIP (hopp, inte anrop): översta talet ersätter talet under returadressen, sedan RET. FNRM4/8/9: talet vid SP+2 (exponent och 3, 7 eller 8 bytes mantissa med ettan utskriven, positivt) till FNORM/DNORM; FREG8 läser 8 bytes vid HL till BCDE och B'C'D'E'.

FADD, FSUB, FMUL, FDIV tar de två översta talen och lämnar resultatet; CY = 1 vid spill (resultatet är då alltid +största talet), underspill ger 0. 4-bytestalen räknas här, 8-bytestalen i DADD, DMUL, DDIV ($5365-). FNORM normaliserar, avrundar och packar resultatet.

FDUP: kopiera talet vid SP (under returadressen). FLOAD: lägg talet vid HL (IY+$1E bytes) på stacken. In: HL = talet (FLOAD). Ut: talet vid SP, NC. Förstör A (= längden), BC (= 0), DE och HL.

```
4C21  21 02 00    FDUP:   LD   HL,$0002      källa = översta talet
4C24  39                  ADD  HL,SP
4C25  EB          FLOAD:  EX   DE,HL
4C26  C1                  POP  BC             egen returadress
4C27  FD 7E 1E            LD   A,(IY+$1E)
4C2A  ED 44               NEG
4C2C  6F                  LD   L,A
4C2D  26 FF               LD   H,$FF
4C2F  39                  ADD  HL,SP
4C30  F9                  LD   SP,HL          plats för talet
4C31  C5                  PUSH BC
4C32  ED 44               NEG
4C34  4F                  LD   C,A
4C35  06 00               LD   B,$00
4C37  EB                  EX   DE,HL
4C38  ED B0               LDIR                kopiera
4C3A  A7                  AND  A
```

```
4C3B  C9                  RET
```

FPOP: *flytta talet vid SP (under returadressen) till DE och ta bort det från stacken;* FSTNP *kopierar utan att ta bort det (ingen kod hänvisar hit). In: DE = målet. Ut: FPOP NZ och DE = returadressen, FSTNP Z och DE efter kopian; NC. Förstör A, BC (= 0) och HL.*

```
4C3C  AF 18 02     FSTNP:  DEFB $AF,$18,$02      A = 0: talet står kvar
4C3F  3E 01        FPOP:   LD   A,$01            A = 1: tas bort
4C41  21 02 00             LD   HL,$0002
4C44  39                   ADD  HL,SP
4C45  FD 4E 1E             LD   C,(IY+$1E)
4C48  06 00                LD   B,$00
4C4A  ED B0                LDIR                  översta talet till DE
4C4C  A7                   AND  A
4C4D  C8                   RET  Z
4C4E  D1                   POP  DE
4C4F  F9                   LD   SP,HL            SP förbi talet
4C50  D5                   PUSH DE
4C51  C9                   RET
```

FSWAP: *byt plats på de två talen under returadressen, byte för byte. Ut: NC. Förstör A, BC, DE och HL.*

```
4C52  CD 2A 4F     FSWAP:  CALL FARGS
4C55  0F                   RRCA                  B = längden
4C56  47                   LD   B,A
4C57  1A           L4C57:  LD   A,(DE)           byt byte för byte
4C58  4E                   LD   C,(HL)
4C59  77                   LD   (HL),A
4C5A  79                   LD   A,C
4C5B  12                   LD   (DE),A
4C5C  23                   INC  HL
4C5D  13                   INC  DE
4C5E  10 F7                DJNZ L4C57
4C60  C9                   RET
```

FCOMP: *jämför de två talen under returadressen; de står kvar.* FCOMP2: *samma med HL och DE redan satta (NEXT vid* $2879*). Olika tecken avgör direkt; lika tecken jämförs byte för byte från exponenten, för negativa tal omvänt. Två tal med exponent 0 är lika oavsett resten, så* `PRINT -0=0` *ger -1. Ut: Z = lika, CY = talet vid DE (det undre) är mindre. Förstör A, BC, DE och HL.*

```
4C61  CD 2A 4F     FCOMP:  CALL FARGS
4C64  1A           FCOMP2: LD   A,(DE)
4C65  B6                   OR   (HL)
4C66  C8                   RET  Z                båda exponenterna 0: lika
4C67  23                   INC  HL
4C68  13                   INC  DE
4C69  1A                   LD   A,(DE)
4C6A  E6 80                AND  $80              tecknen
4C6C  4F                   LD   C,A
4C6D  7E                   LD   A,(HL)
4C6E  E6 80                AND  $80
4C70  B9                   CP   C
4C71  C0                   RET  NZ               olika tecken: klart
4C72  A7                   AND  A
4C73  28 01                JR   Z,L4C76
4C75  EB                   EX   DE,HL            båda negativa: omvänt
4C76  2B           L4C76:  DEC  HL               tillbaka till exponenterna
4C77  1B                   DEC  DE
4C78  FD 46 1E             LD   B,(IY+$1E)
4C7B  1A           L4C7B:  LD   A,(DE)
4C7C  BE                   CP   (HL)             exponent, sedan mantissan
4C7D  C0                   RET  NZ
4C7E  23                   INC  HL
4C7F  13                   INC  DE
4C80  10 F9                DJNZ L4C7B
4C82  C9                   RET
```

FCHS: *byt tecken på talet vid SP (bit 7 i andra byten, även för 0). Förstör A, DE och HL.*

```
4C83  E1           FCHS:   POP  HL
4C84  D1                   POP  DE
4C85  7A                   LD   A,D
4C86  EE 80                XOR  $80          teckenbiten, även för 0
4C88  57                   LD   D,A
4C89  D5                   PUSH DE
4C8A  E9                   JP   (HL)
```

FEXT8: *gör talet vid SP till 8 bytes med fyra nollbytes efter det; IY+$1E ändras inte. Ut: NC. Förstör HL, BC', DE' och HL' (= 0); A bevaras.*

```
4C8B  E1           FEXT8:  POP  HL
4C8C  D9                   EXX
4C8D  C1                   POP  BC
4C8E  D1                   POP  DE
4C8F  21 00 00             LD   HL,$0000     fyra nollbytes efter talet
4C92  E5                   PUSH HL
4C93  E5                   PUSH HL
4C94  D5                   PUSH DE
4C95  C5                   PUSH BC
4C96  D9                   EXX
4C97  A7                   AND  A
4C98  E9                   JP   (HL)
```

FRND4: *gör 8-bytestalet vid SP till 4 bytes. Avrundar uppåt (på beloppet) när bit 7 i femte byten är satt, utom vid exponent 0 och $FF; blir mantissan full ökar exponenten. Ut: 4-bytestalet vid SP, A = 0, Z, NC. Förstör BC, DE och IX (returadressen hålls i IX); HL bevaras.*

```
4C99  DD E1        FRND4:  POP  IX           IX = returadressen
4C9B  D1                   POP  DE
4C9C  C1                   POP  BC
4C9D  F1                   POP  AF           S = bit 7 i femte byten
4C9E  33                   INC  SP
4C9F  33                   INC  SP
4CA0  F2 BB 4C             JP   P,L4CBB      under 1/2: avhugget
4CA3  7B                   LD   A,E
4CA4  3C                   INC  A
4CA5  E6 FE                AND  $FE
4CA7  28 12                JR   Z,L4CBB      exponent 0 eller $FF: kvar
4CA9  04                   INC  B            mantissan + 1
4CAA  20 0F                JR   NZ,L4CBB
4CAC  0C                   INC  C
4CAD  20 0C                JR   NZ,L4CBB
4CAF  7A                   LD   A,D          A = tecknet, ettan in
4CB0  CB FA                SET  7,D
4CB2  14                   INC  D
4CB3  20 01                JR   NZ,L4CB6
4CB5  1C                   INC  E            spill: exponenten + 1
4CB6  CB 22        L4CB6:  SLA  D            tecknet tillbaka i bit 7
4CB8  07                   RLCA
4CB9  CB 1A                RR   D
4CBB  C5           L4CBB:  PUSH BC
4CBC  D5                   PUSH DE
4CBD  AF                   XOR  A
4CBE  DD E9                JP   (IX)
```

ITOFS: *heltalet vid SP (under returadressen) blir ett flyttal med IY+$1E bytes. Beloppet läggs med exponent $90 (16 bitar) och normaliseras med FNRM4, sedan sätts tecknet i bit 7 (-32768 blir -32768). Ut: flyttalet vid SP, NC. Förstör A, BC, DE, HL och BC', DE', HL'.*

```
4CC0  E1           ITOFS:  POP  HL           heltal på stacken -> flyttal
4CC1  C1                   POP  BC
4CC2  E5                   PUSH HL
4CC3  C5                   PUSH BC
4CC4  CB 78                BIT  7,B          negativt: beloppet
4CC6  28 07                JR   Z,L4CCF
4CC8  78                   LD   A,B
4CC9  2F                   CPL
```

```
4CCA  47                LD   B,A
4CCB  79                LD   A,C
4CCC  2F                CPL
4CCD  4F                LD   C,A
4CCE  03                INC  BC
4CCF  1E 90       L4CCF: LD   E,$90          exponent $90: 16 bitar
4CD1  50                LD   D,B
4CD2  06 00             LD   B,$00
4CD4  C5                PUSH BC
4CD5  D5                PUSH DE
4CD6  CD 53 4D          CALL FNRM4          normalisera
4CD9  D1                POP  DE
4CDA  C1                POP  BC
4CDB  F1                POP  AF             AF = heltalet
4CDC  CB 12             RL   D
4CDE  07                RLCA                dess tecken i bit 7
4CDF  CB 1A             RR   D
4CE1  E1                POP  HL
4CE2  FD CB 1E 56       BIT  2,(IY+$1E)
4CE6  20 07             JR   NZ,L4CEF       4 bytes: klart
4CE8  D9                EXX
4CE9  21 00 00          LD   HL,$0000       annars 4 nollbytes till
4CEC  E5                PUSH HL
4CED  E5                PUSH HL
4CEE  D9                EXX
4CEF  C5          L4CEF: PUSH BC
4CF0  D5                PUSH DE
4CF1  A7                AND  A
4CF2  E9                JP   (HL)
```

FTOIR avrundar till närmaste (halva bort från noll), FTOIT hugger av mot noll: talet vid SP (under returadressen) blir ett heltal på 2 bytes. Beloppet räknas i 16 bitar och tecknet läggs på efteråt, så 32768-65535 slår runt (40000 blir -25536) och 65535.5 avrundas till 65535 (-1); CY först från 65536, och då är heltalet `$7FFF`*. Ut: heltalet vid SP, NC. Förstör A, BC, DE och HL.*

```
4CF3  3E FF       FTOIR: LD   A,$FF          avrunda
4CF5  18 01             JR   L4CF8

4CF7  AF          FTOIT: XOR  A             hugg av
4CF8  E1          L4CF8: POP  HL
4CF9  D1                 POP  DE
4CFA  E3                 EX   (SP),HL
4CFB  F5                 PUSH AF
4CFC  7A                 LD   A,D
4CFD  F6 80              OR   $80
4CFF  47                 LD   B,A
4D00  4D                 LD   C,L
4D01  7B                 LD   A,E
4D02  D6 90              SUB  $90            A = exponent - $90
4D04  28 1F              JR   Z,L4D25
4D06  30 44              JR   NC,L4D4C       exponent > 16 bitar: spill
4D08  FE EF              CP   $EF
4D0A  30 02              JR   NC,L4D0E
4D0C  3E EF              LD   A,$EF          högst 17 steg
4D0E  FE F9       L4D0E: CP   $F9
4D10  30 0A              JR   NC,L4D1C
4D12  61                 LD   H,C            8 bitar åt gången
4D13  48                 LD   C,B
4D14  06 00              LD   B,$00
4D16  C6 08              ADD  A,$08
4D18  28 0B              JR   Z,L4D25
4D1A  18 F2              JR   L4D0E

4D1C  CB 38       L4D1C: SRL  B             sedan en bit åt gången
4D1E  CB 19              RR   C
4D20  3C                 INC  A
4D21  20 F9              JR   NZ,L4D1C
4D23  CB 1C              RR   H              H bit 7 = biten efter
4D25  F1          L4D25: POP  AF             A = $FF (FTOIR) eller 0 (FTOIT)
4D26  A4                 AND  H
```

```
4D27  F2 30 4D            JP   P,L4D30
4D2A  03                  INC  BC             avrundning uppåt, 65536 -> 65535
4D2B  78                  LD   A,B
4D2C  B1                  OR   C
4D2D  20 01               JR   NZ,L4D30
4D2F  0B                  DEC  BC
4D30  14           L4D30: INC  D              tecknet?
4D31  15                  DEC  D
4D32  F2 3C 4D            JP   P,L4D3C
4D35  AF                  XOR  A              negativt: 65536 - belopp
4D36  91                  SUB  C
4D37  4F                  LD   C,A
4D38  3E 00               LD   A,$00
4D3A  98                  SBC  A,B
4D3B  47                  LD   B,A
4D3C  AF           L4D3C: XOR  A
4D3D  E1           L4D3D: POP  HL
4D3E  FD CB 1E 56         BIT  2,(IY+$1E)
4D42  20 02               JR   NZ,L4D46
4D44  D1                  POP  DE             8 bytes: 4 till bort
4D45  D1                  POP  DE
4D46  C5           L4D46: PUSH BC
4D47  E5                  PUSH HL
4D48  A7                  AND  A
4D49  C8                  RET  Z
4D4A  37                  SCF
4D4B  C9                  RET

4D4C  F1           L4D4C: POP  AF
4D4D  01 FF 7F            LD   BC,$7FFF       spill: $7FFF, CY
4D50  79                  LD   A,C
4D51  18 EA               JR   L4D3D
```

FNRM4/FNRM8/FNRM9: *normalisera ett positivt tal vid SP (under returadressen): exponent och 3, 7 eller 8 bytes mantissa med ettan utskriven.* FREG8 *läser mantissan; 4 bytes går till* FNORM *med skyddsbyten E = 0, 8 och 9 bytes till* DNORM *(med 8 är skyddsbyten 0). A = antalet bytes läggs på stacken, och så många tas bort när resultatet (4 eller 8 bytes, positivt) ersätter talet. Ut: CY = spill. Förstör A, BC, DE, HL och de skuggade registren.*

```
4D53  3E 04        FNRM4: LD   A,$04           exponent + 3 bytes
4D55  18 06               JR   L4D5D

4D57  3E 09        FNRM9: LD   A,$09           exponent + 8 bytes
4D59  18 02               JR   L4D5D

4D5B  3E 08        FNRM8: LD   A,$08           exponent + 7 bytes
4D5D  21 02 00     L4D5D: LD   HL,$0002
4D60  44                  LD   B,H
4D61  39                  ADD  HL,SP
4D62  F5                  PUSH AF
4D63  4E                  LD   C,(HL)         C = exponenten, B = 0: plus
4D64  C5                  PUSH BC
4D65  23                  INC  HL
4D66  CD DA 4D            CALL FREG8          BCDE B'C'D'E' = mantissan
4D69  D6 04               SUB  $04
4D6B  28 0A               JR   Z,L4D77
4D6D  D6 04               SUB  $04
4D6F  20 03               JR   NZ,L4D74
4D71  D9                  EXX
4D72  5F                  LD   E,A            E' = 0
4D73  D9                  EXX
4D74  C3 BC 55     L4D74: JP   DNORM

4D77  5F           L4D77: LD   E,A             E = 0
4D78  C3 E6 50            JP   FNORM
```

DECNRM *(hopp, inte anrop): stryk nollorna sist och först i decimaltalet vid SP (under returadressen) som* FTODEC *lämnar det: antal siffror (bit 7 = tecken), exponent och ASCII-siffror. Siffrorna flyttas upp mot slutet, varje nolla först minskar exponenten, och huvudet läggs direkt före den första siffran; bara nollor ger* 0 *med exponent 0 och plus. Sedan* RET*, med SP högre än förut. Ut: NC. BC, DE och HL bevaras (rutinen räknar i de skuggade registren); förstör A och BC', DE', HL'.*

```
4D7B  D9          DECNRM: EXX
4D7C  21 02 00            LD   HL,$0002
4D7F  39                  ADD  HL,SP
4D80  7E                  LD   A,(HL)
4D81  23                  INC  HL
4D82  E6 7F               AND  $7F              antal siffror (bit 7 = tecken)
4D84  4F                  LD   C,A
4D85  06 00               LD   B,$00
4D87  09                  ADD  HL,BC            HL = sista siffran
4D88  E5                  PUSH HL
4D89  7E          L4D89:  LD   A,(HL)
4D8A  FE 30               CP   $30              nollor sist stryks
4D8C  20 0B               JR   NZ,L4D99
4D8E  2B                  DEC  HL
4D8F  0D                  DEC  C
4D90  20 F7               JR   NZ,L4D89
4D92  E1                  POP  HL               bara nollor: talet 0
4D93  D1                  POP  DE
4D94  01 01 00            LD   BC,$0001
4D97  18 24               JR   L4DBD

4D99  79          L4D99:  LD   A,C
4D9A  D1                  POP  DE
4D9B  ED B8               LDDR                  siffrorna upp till slutet
4D9D  EB                  EX   DE,HL
4D9E  23                  INC  HL
4D9F  4F                  LD   C,A
4DA0  06 00               LD   B,$00
4DA2  0D          L4DA2:  DEC  C
4DA3  28 09               JR   Z,L4DAE
4DA5  7E                  LD   A,(HL)
4DA6  FE 30               CP   $30              nollor först stryks ...
4DA8  20 04               JR   NZ,L4DAE
4DAA  23                  INC  HL
4DAB  05                  DEC  B                ... och exponenten minskar
4DAC  18 F4               JR   L4DA2

4DAE  0C          L4DAE:  INC  C
4DAF  EB                  EX   DE,HL
4DB0  E1                  POP  HL
4DB1  E3                  EX   (SP),HL
4DB2  78                  LD   A,B
4DB3  84                  ADD  A,H              ny exponent
4DB4  47                  LD   B,A
4DB5  CB 11               RL   C                tecknet tillbaka i bit 7
4DB7  CB 15               RL   L
4DB9  CB 19               RR   C
4DBB  E1                  POP  HL
4DBC  EB                  EX   DE,HL
4DBD  F9          L4DBD:  LD   SP,HL            nytt huvud under siffrorna
4DBE  C5                  PUSH BC
4DBF  D5                  PUSH DE
4DC0  A7                  AND  A
4DC1  D9                  EXX
4DC2  C9                  RET
```

FDROP: *ta bort talet vid SP (under returadressen). Förstör DE och HL; A och BC bevaras.*

```
4DC3  D1          FDROP:  POP  DE
4DC4  FD 6E 1E            LD   L,(IY+$1E)
4DC7  26 00               LD   H,$00
4DC9  39                  ADD  HL,SP
4DCA  F9                  LD   SP,HL            SP förbi talet
4DCB  EB                  EX   DE,HL
4DCC  E9                  JP   (HL)
```

FNIP (*hopp, inte anrop*): *talet vid SP ersätter talet under returadressen, sedan* RET *till den; de matematiska funktionerna slutar med* JP FNIP. *Förstör A, BC, DE och HL.*

```
4DCD  FD 6E 1E     FNIP:   LD   L,(IY+$1E)
4DD0  26 00                LD   H,$00
4DD2  39                   ADD  HL,SP
4DD3  23                   INC  HL              DE = talet under returadressen
4DD4  23                   INC  HL
4DD5  EB                   EX   DE,HL
4DD6  CD 3F 4C             CALL FPOP            översta talet dit
4DD9  C9                   RET
```

FREG8: *läs 8 bytes vid HL, de fyra första till B, C, D och E, de fyra sista till B', C', D' och E'. Ut: HL vid den sista byten* $(HL + 7)$. *A och flaggorna bevaras.*

```
4DDA  46           FREG8:  LD   B,(HL)
4DDB  23                   INC  HL
4DDC  4E                   LD   C,(HL)
4DDD  23                   INC  HL
4DDE  56                   LD   D,(HL)
4DDF  23                   INC  HL
4DE0  5E                   LD   E,(HL)
4DE1  23                   INC  HL
4DE2  D5                   PUSH DE
4DE3  56                   LD   D,(HL)
4DE4  23                   INC  HL
4DE5  5E                   LD   E,(HL)
4DE6  23                   INC  HL
4DE7  D5                   PUSH DE
4DE8  56                   LD   D,(HL)
4DE9  23                   INC  HL
4DEA  5E                   LD   E,(HL)
4DEB  D5                   PUSH DE
4DEC  D9                   EXX
4DED  D1                   POP  DE
4DEE  C1                   POP  BC
4DEF  D9                   EXX
4DF0  D1                   POP  DE
4DF1  C9                   RET
```

POW10: *10^0 .. 10^38 som 8-bytes flyttal (används vid omvandling mellan text och tal och av matematikrutinerna). 10^0 .. 10^24 är exakta, de större avrundade till närmaste 56-bitarsmantissa.*

```
4DF2  81 00 00 00… POW10:  DEFB $81,$00,$00,$00,$00,$00,$00,$00  1
4DFA  84 20 00 00…         DEFB $84,$20,$00,$00,$00,$00,$00,$00  10
4E02  87 48 00 00…         DEFB $87,$48,$00,$00,$00,$00,$00,$00  100
4E0A  8A 7A 00 00…         DEFB $8A,$7A,$00,$00,$00,$00,$00,$00  1000
4E12  8E 1C 40 00…         DEFB $8E,$1C,$40,$00,$00,$00,$00,$00  10000
4E1A  91 43 50 00…         DEFB $91,$43,$50,$00,$00,$00,$00,$00  100000
4E22  94 74 24 00…         DEFB $94,$74,$24,$00,$00,$00,$00,$00  1000000
4E2A  98 18 96 80…         DEFB $98,$18,$96,$80,$00,$00,$00,$00  10000000
4E32  9B 3E BC 20…         DEFB $9B,$3E,$BC,$20,$00,$00,$00,$00  100000000
4E3A  9E 6E 6B 28…         DEFB $9E,$6E,$6B,$28,$00,$00,$00,$00  1000000000
4E42  A2 15 02 F9…         DEFB $A2,$15,$02,$F9,$00,$00,$00,$00  10000000000
4E4A  A5 3A 43 B7…         DEFB $A5,$3A,$43,$B7,$40,$00,$00,$00  100000000000
4E52  A8 68 D4 A5…         DEFB $A8,$68,$D4,$A5,$10,$00,$00,$00  1000000000000
4E5A  AC 11 84 E7…         DEFB $AC,$11,$84,$E7,$2A,$00,$00,$00  10000000000000
4E62  AF 35 E6 20…         DEFB $AF,$35,$E6,$20,$F4,$80,$00,$00  100000000000000
4E6A  B2 63 5F A9…         DEFB $B2,$63,$5F,$A9,$31,$A0,$00,$00  1000000000000000
4E72  B6 0E 1B C9…         DEFB $B6,$0E,$1B,$C9,$BF,$04,$00,$00  1e+16
4E7A  B9 31 A2 BC…         DEFB $B9,$31,$A2,$BC,$2E,$C5,$00,$00  1e+17
4E82  BC 5E 0B 6B…         DEFB $BC,$5E,$0B,$6B,$3A,$76,$40,$00  1e+18
4E8A  C0 0A C7 23…         DEFB $C0,$0A,$C7,$23,$04,$89,$E8,$00  1e+19
4E92  C3 2D 78 EB…         DEFB $C3,$2D,$78,$EB,$C5,$AC,$62,$00  1e+20
4E9A  C6 58 D7 26…         DEFB $C6,$58,$D7,$26,$B7,$17,$7A,$80  1e+21
4EA2  CA 07 86 78…         DEFB $CA,$07,$86,$78,$32,$6E,$AC,$90  1e+22
4EAA  CD 29 68 16…         DEFB $CD,$29,$68,$16,$3F,$0A,$57,$B4  1e+23
4EB2  D0 53 C2 1B…         DEFB $D0,$53,$C2,$1B,$CE,$CC,$ED,$A1  1e+24
4EBA  D4 04 59 51…         DEFB $D4,$04,$59,$51,$61,$40,$14,$85  1e+25
4EC2  D7 25 6F A5…         DEFB $D7,$25,$6F,$A5,$B9,$90,$19,$A6  1e+26
4ECA  DA 4E CB 8F…         DEFB $DA,$4E,$CB,$8F,$27,$F4,$20,$0F  1e+27
```

```
4ED2  DE 01 3F 39…        DEFB $DE,$01,$3F,$39,$78,$F8,$94,$0A  1e+28
4EDA  E1 21 8F 07…        DEFB $E1,$21,$8F,$07,$D7,$36,$B9,$0C  1e+29
4EE2  E4 49 F2 C9…        DEFB $E4,$49,$F2,$C9,$CD,$04,$67,$4F  1e+30
4EEA  E7 7C 6F 7C…        DEFB $E7,$7C,$6F,$7C,$40,$45,$81,$23  1e+31
4EF2  EB 1D C5 AD…        DEFB $EB,$1D,$C5,$AD,$A8,$2B,$70,$B6  1e+32
4EFA  EE 45 37 19…        DEFB $EE,$45,$37,$19,$12,$36,$4C,$E3  1e+33
4F02  F1 76 84 DF…        DEFB $F1,$76,$84,$DF,$56,$C3,$E0,$1C  1e+34
4F0A  F5 1A 13 0B…        DEFB $F5,$1A,$13,$0B,$96,$3A,$6C,$11  1e+35
4F12  F8 40 97 CE…        DEFB $F8,$40,$97,$CE,$7B,$C9,$07,$16  1e+36
4F1A  FB 70 BD C2…        DEFB $FB,$70,$BD,$C2,$1A,$BB,$48,$DB  1e+37
4F22  FF 16 76 99…        DEFB $FF,$16,$76,$99,$50,$B5,$0D,$89  1e+38
```

FARGS: *HL = översta talet, DE = det under, A = 2 * längden. Talen räknas från SP+4, förbi FARGS egen returadress och anroparens. Ut: NC; BC bevaras.*

```
4F2A  21 04 00     FARGS: LD   HL,$0004       förbi de två returadresserna
4F2D  39                  ADD  HL,SP
4F2E  EB                  EX   DE,HL
4F2F  FD 6E 1E            LD   L,(IY+$1E)     L = längden (4 eller 8)
4F32  26 00               LD   H,$00
4F34  7D                  LD   A,L
4F35  19                  ADD  HL,DE
4F36  EB                  EX   DE,HL          HL = översta, DE = det under
4F37  87                  ADD  A,A            A = båda talens längd
4F38  C9                  RET
```

FSUB *byter tecken på det översta talet och fortsätter i* FADD.

FADD: *flyttal + (4 bytes; 8 i* DADD*). Talet med minst exponent skiftas höger (högst 25 steg) in i BCDE, där E är en skyddsbyte; bitar under den försvinner (se* $4F77*). Lika tecken: addition, och med en minnessiffra ett steg höger. Olika tecken: det mindre dras från det större; blir skillnaden negativ (lika exponenter) byts tecknet. Tecken och exponent ligger under tiden i H och L på stacken: H bit 7 = olika tecken, bit 0 = tecknet. In: två tal under returadressen. Ut (4 bytes): summan i det undres plats, CY = spill. Förstör A, BC, DE, HL och HL'.*

```
4F39  E1           FSUB:  POP  HL
4F3A  D1                  POP  DE             E = exponent, D = tecken ...
4F3B  3E 80               LD   A,$80          byt tecken på det översta
4F3D  AA                  XOR  D
4F3E  57                  LD   D,A
4F3F  D5                  PUSH DE
4F40  E5                  PUSH HL
4F41  FD CB 1E 56  FADD:  BIT  2,(IY+$1E)     flyttal +, 4 bytes (8: DADD)
4F45  CA 65 53            JP   Z,DADD         8 bytes: DADD
4F48  CD 2A 4F            CALL FARGS
4F4B  F5                  PUSH AF             A = 2 * längden, till FPACK
4F4C  1A                  LD   A,(DE)
4F4D  96                  SUB  (HL)           exponentskillnaden
4F4E  30 03               JR   NC,L4F53
4F50  EB                  EX   DE,HL          HL = det mindre talet
4F51  ED 44               NEG
4F53  4F           L4F53: LD   C,A            C = steg att skifta
4F54  7E                  LD   A,(HL)
4F55  A7                  AND  A
4F56  20 07               JR   NZ,L4F5F
4F58  0C                  INC  C
4F59  0D                  DEC  C
4F5A  CA 2B 51            JP   Z,FZERO        båda 0: resultatet 0
4F5D  0E 19               LD   C,$19          det mindre 0: skiftas helt ut
4F5F  D5           L4F5F: PUSH DE             det större talet
4F60  23                  INC  HL
4F61  13                  INC  DE
4F62  1A                  LD   A,(DE)
4F63  AE                  XOR  (HL)           bit 7: olika tecken
4F64  47                  LD   B,A
4F65  1A                  LD   A,(DE)
4F66  17                  RLA                 CY = det störres tecken
4F67  78                  LD   A,B
4F68  1F                  RRA
4F69  07                  RLCA
```

```
4F6A  5F                LD   E,A            E: bit 7 olika, bit 0 tecknet
4F6B  7E                LD   A,(HL)
4F6C  F6 80             OR   $80            ettan in; BCD = det mindre
4F6E  47                LD   B,A
4F6F  79                LD   A,C
4F70  23                INC  HL
4F71  4E                LD   C,(HL)
4F72  23                INC  HL
4F73  56                LD   D,(HL)
4F74  63                LD   H,E            H = teckeninfo, E = skyddsbyte
4F75  1E 00             LD   E,$00
4F77  FE 1A             CP   $1A(gränsfall)
4F79  38 02             JR   C,L4F7D        högst 25 steg
4F7B  3E 19             LD   A,$19
4F7D  D6 08       L4F7D: SUB  $08            hela bytes först
4F7F  38 07             JR   C,L4F88
4F81  5A                LD   E,D
4F82  51                LD   D,C
4F83  48                LD   C,B
4F84  06 00             LD   B,$00
4F86  18 F5             JR   L4F7D

4F88  C6 08       L4F88: ADD  A,$08
4F8A  6F                LD   L,A
4F8B  28 0B             JR   Z,L4F98
4F8D  CB 38       L4F8D: SRL  B              sedan bitvis
4F8F  CB 19             RR   C
4F91  CB 1A             RR   D
4F93  CB 1B             RR   E
4F95  2D                DEC  L
4F96  20 F5             JR   NZ,L4F8D
4F98  24          L4F98: INC  H              S = olika tecken
4F99  25                DEC  H
4F9A  E3                EX   (SP),HL
4F9B  7E                LD   A,(HL)
4F9C  E3                EX   (SP),HL
4F9D  6F                LD   L,A           L = det större exponent
4F9E  E3                EX   (SP),HL       H och L på stacken
4F9F  23                INC  HL
4FA0  23                INC  HL
4FA1  23                INC  HL
4FA2  FA C3 4F          JP   M,L4FC3       olika tecken: subtrahera
4FA5  7A                LD   A,D           addera, sista byten först
4FA6  86                ADD  A,(HL)
4FA7  57                LD   D,A
4FA8  2B                DEC  HL
4FA9  79                LD   A,C
4FAA  8E                ADC  A,(HL)
4FAB  4F                LD   C,A
4FAC  2B                DEC  HL
4FAD  7E                LD   A,(HL)
4FAE  17                RLA                ettan in, CY till nästa
4FAF  37                SCF
4FB0  1F                RRA
4FB1  88                ADC  A,B
4FB2  47                LD   B,A
4FB3  E1                POP  HL
4FB4  D2 11 51          JP   NC,FROUND     ingen minnessiffra: klart
4FB7  CB 18             RR   B             annars ett steg höger ...
4FB9  CB 19             RR   C
4FBB  CB 1A             RR   D
4FBD  CB 1B             RR   E
4FBF  2C                INC  L             ... och exponenten + 1
4FC0  C3 11 51          JP   FROUND

4FC3  AF          L4FC3: XOR  A              det större minus det mindre
4FC4  93                SUB  E
4FC5  5F                LD   E,A
4FC6  7E                LD   A,(HL)
```

```
4FC7  9A                SBC  A,D
4FC8  57                LD   D,A
4FC9  2B                DEC  HL
4FCA  7E                LD   A,(HL)
4FCB  99                SBC  A,C
4FCC  4F                LD   C,A
4FCD  2B                DEC  HL
4FCE  7E                LD   A,(HL)
4FCF  17                RLA
4FD0  37                SCF
4FD1  1F                RRA
4FD2  98                SBC  A,B
4FD3  47                LD   B,A
4FD4  D2 E6 50          JP   NC,FNORM      positivt: normalisera
4FD7  E1                POP  HL
4FD8  24                INC  H             negativt: tecknet byts ...
4FD9  E5                PUSH HL
4FDA  AF                XOR  A             ... och beloppet tas
4FDB  67                LD   H,A
4FDC  93                SUB  E
4FDD  5F                LD   E,A
4FDE  7C                LD   A,H
4FDF  9A                SBC  A,D
4FE0  57                LD   D,A
4FE1  7C                LD   A,H
4FE2  99                SBC  A,C
4FE3  4F                LD   C,A
4FE4  7C                LD   A,H
4FE5  98                SBC  A,B
4FE6  47                LD   B,A
4FE7  C3 E6 50          JP   FNORM
```

FMUL: *24 * 24 bitar med skift och addition, multiplikatorn (det undre talet) en bit i taget från den lägsta. Produktens låga bitar skiftas in i D' i takt med att multiplikatorns bitar skiftas ut; de 32 översta bitarna behålls, den fjärde byten blir skyddsbyte. Exponent = exp1 + exp2 - $80: under 1 ger 0, över 255 spill. In: två tal under returadressen. Ut (4 bytes): produkten i det undres plats, CY = spill. Förstör A, BC, DE, HL och BC', D', HL'.*

```
4FEA  FD CB 1E 56  FMUL:   BIT  2,(IY+$1E)
4FEE  CA 3F 54             JP   Z,DMUL        8 bytes: DMUL
4FF1  CD 2A 4F             CALL FARGS
4FF4  F5                   PUSH AF
4FF5  1A                   LD   A,(DE)        B = det undres exponent
4FF6  47                   LD   B,A
4FF7  4E                   LD   C,(HL)
4FF8  A7                   AND  A
4FF9  CA 2B 51             JP   Z,FZERO       något av talen 0: 0
4FFC  0C                   INC  C
4FFD  0D                   DEC  C
4FFE  CA 2B 51             JP   Z,FZERO
5001  13                   INC  DE
5002  23                   INC  HL
5003  1A                   LD   A,(DE)
5004  AE                   XOR  (HL)
5005  07                   RLCA               bit 0 = produktens tecken
5006  F5                   PUSH AF
5007  78                   LD   A,B
5008  3D                   DEC  A
5009  81                   ADD  A,C           exp1 + exp2 - 1
500A  C1                   POP  BC            B = tecknet
500B  FA 13 50             JP   M,L5013
500E  D2 2B 51             JP   NC,FZERO      för litet: 0
5011  18 03                JR   L5016

5013  DA 23 51     L5013:  JP   C,FOVFL       för stort: spill
5016  C6 81        L5016:  ADD  A,$81         A = exp1 + exp2 - $80
5018  4F                   LD   C,A
5019  C5                   PUSH BC            tecken och exponent på stacken
501A  D5                   PUSH DE
501B  D9                   EXX
```

```
501C  E1                POP  HL
501D  46                LD   B,(HL)          B'C'D' = det undre talet
501E  CB F8             SET  7,B
5020  23                INC  HL
5021  4E                LD   C,(HL)
5022  23                INC  HL
5023  56                LD   D,(HL)
5024  D9                EXX
5025  7E                LD   A,(HL)
5026  F6 80             OR   $80
5028  23                INC  HL
5029  56                LD   D,(HL)
502A  23                INC  HL
502B  5E                LD   E,(HL)
502C  EB                EX   DE,HL           EHL = det översta
502D  5F                LD   E,A
502E  01 00 00          LD   BC,$0000        produkten BCD = 0
5031  50                LD   D,B
5032  D9                EXX
5033  2E 08       L5033: LD   L,$08           8 bitar per byte
5035  CB 1A       L5035: RR   D               bit ut, produktbit in i D'
5037  D9                EXX
5038  78                LD   A,B
5039  30 08             JR   NC,L5043        biten 0: bara skift
503B  7A                LD   A,D             EHL till produkten
503C  85                ADD  A,L
503D  57                LD   D,A
503E  79                LD   A,C
503F  8C                ADC  A,H
5040  4F                LD   C,A
5041  78                LD   A,B
5042  8B                ADC  A,E
5043  1F          L5043: RRA                  produkten ett steg höger
5044  47                LD   B,A
5045  CB 19             RR   C
5047  CB 1A             RR   D
5049  D9                EXX
504A  2D                DEC  L
504B  20 E8             JR   NZ,L5035
504D  CB 1A             RR   D               sista biten in i D'
504F  78                LD   A,B             fler bytes kvar?
5050  B1                OR   C
5051  28 06             JR   Z,L5059
5053  51                LD   D,C             nästa byte
5054  48                LD   C,B
5055  06 00             LD   B,$00
5057  18 DA             JR   L5033

5059  7A          L5059: LD   A,D             D' = fjärde byten, skyddsbyte
505A  D9                EXX
505B  5F                LD   E,A
505C  C3 E6 50          JP   FNORM
```

FDIV: det undre talet / det översta, division med återställning. Kvotbitarna skiftas in i E', där vakt-ettan visar när byten är full, tills första byten B' har värde; sedan två bitar till, som hamnar i bit 7-6 i skyddsbyten E (26 bitar). Nämnaren 0 ger spill, täljaren 0 ger 0. Exponent = skillnaden + $81, - 1 vid normaliseringen; minsta resultatet 2^-128 (exponent $01) går. In: två tal under returadressen. Ut (4 bytes): kvoten i det undres plats, CY = spill. Förstör A, BC, DE, HL och de skuggade BC', DE', HL'; FDIV går in i FNORM med de skuggade registren valda, så registeruppsättningarna är bytta efteråt.

```
505F  FD CB 1E 56 FDIV:  BIT  2,(IY+$1E)
5063  CA F6 54          JP   Z,DDIV          8 bytes: DDIV
5066  CD 2A 4F          CALL FARGS
5069  F5                PUSH AF
506A  1A                LD   A,(DE)
506B  4F                LD   C,A             C = täljarens exponent
506C  13                INC  DE
506D  1A                LD   A,(DE)
506E  07                RLCA
506F  47                LD   B,A
```

```
5070  7E              LD   A,(HL)
5071  A7              AND  A
5072  CA 23 51        JP   Z,FOVFL        nämnaren 0: spill
5075  0C              INC  C
5076  0D              DEC  C
5077  CA 2B 51        JP   Z,FZERO        täljaren 0: 0
507A  47              LD   B,A
507B  1A              LD   A,(DE)
507C  23              INC  HL
507D  AE              XOR  (HL)
507E  07              RLCA                bit 0 = kvotens tecken
507F  F5              PUSH AF
5080  79              LD   A,C
5081  90              SUB  B              exp täljare - exp nämnare
5082  C1              POP  BC
5083  FA 8B 50        JP   M,L508B
5086  DA 2B 51        JP   C,FZERO        för litet: 0
5089  18 03           JR   L508E

508B  D2 23 51  L508B:  JP   NC,FOVFL     för stort: spill
508E  C6 81     L508E:  ADD  A,$81        A = skillnaden + $81
5090  4F              LD   C,A
5091  C5              PUSH BC
5092  D9              EXX
5093  01 00 00        LD   BC,$0000       kvoten B'C'D' = 0, E' = 1
5096  50              LD   D,B
5097  1E 01           LD   E,$01
5099  D9              EXX
509A  E5              PUSH HL
509B  EB              EX   DE,HL
509C  7E              LD   A,(HL)
509D  F6 80           OR   $80
509F  23              INC  HL
50A0  56              LD   D,(HL)
50A1  23              INC  HL
50A2  5E              LD   E,(HL)
50A3  EB              EX   DE,HL
50A4  5F              LD   E,A            EHL = täljaren
50A5  E3              EX   (SP),HL
50A6  7E              LD   A,(HL)
50A7  F6 80           OR   $80
50A9  47              LD   B,A            BCD = nämnaren
50AA  23              INC  HL
50AB  4E              LD   C,(HL)
50AC  23              INC  HL
50AD  56              LD   D,(HL)
50AE  E1              POP  HL
50AF  18 0F           JR   L50C0

50B1  41        L50B1:  LD   B,C          nästa byte i kvoten
50B2  4A                LD   C,D
50B3  53                LD   D,E
50B4  1E 01             LD   E,$01
50B6  78                LD   A,B
50B7  A7                AND  A
50B8  28 02             JR   Z,L50BC
50BA  1E 40             LD   E,$40        första byten klar: 2 bitar till
50BC  D9        L50BC:  EXX
50BD  29                ADD  HL,HL        täljaren ett steg vänster
50BE  CB 13             RL   E
50C0  38 0D     L50C0:  JR   C,L50CF       bit ut: alltid större
50C2  78                LD   A,B          jämför EHL med BCD
50C3  BB                CP   E
50C4  20 07             JR   NZ,L50CD
50C6  79                LD   A,C
50C7  BC                CP   H
50C8  20 03             JR   NZ,L50CD
50CA  7D                LD   A,L
50CB  BA                CP   D
```

```
50CC  3F              CCF
50CD  30 0A     L50CD: JR   NC,L50D9
50CF  7D        L50CF: LD   A,L             dra ifrån, kvotbiten 1
50D0  92               SUB  D
50D1  6F               LD   L,A
50D2  7C               LD   A,H
50D3  99               SBC  A,C
50D4  67               LD   H,A
50D5  7B               LD   A,E
50D6  98               SBC  A,B
50D7  5F               LD   E,A
50D8  37               SCF
50D9  D9        L50D9: EXX
50DA  CB 13            RL   E               kvotbiten in
50DC  30 DE            JR   NC,L50BC        vakt-ettan inte ute: nästa bit
50DE  78               LD   A,B
50DF  A7               AND  A
50E0  28 CF            JR   Z,L50B1         B' = 0: fler bytes
50E2  7B               LD   A,E
50E3  0F               RRCA                 de två sista bitarna i bit 7-6
50E4  0F               RRCA
50E5  5F               LD   E,A
```

*FNORM: BCDE skiftas vänster tills bit 7 i B är satt; varje steg minskar exponenten L, och når den 0 blir resultatet 0 (underspill). FROUND avrundar halva uppåt räknat på beloppet (bit 7 i skyddsbyten E lägger 1 till BCD; 2^24 + 1 blir 2^24 + 2). FPACK lägger tecknet i ettans plats och ersätter de två talen på stacken med resultatet; CY = spill (exponent 255 + 1). In: BCDE = mantissan (E skyddsbyte) och vid SP H = tecknet i bit 0, L = exponenten; under det AF med A = antalet bytes som tas bort (2 * längden från FARGS, 4 från FNRM4), sedan returadressen och talen. Förstör A, BC, DE, HL och HL'.*

```
50E6  E1        FNORM: POP  HL              H = tecken, L = exponenten
50E7  04               INC  B
50E8  05               DEC  B
50E9  20 15            JR   NZ,L5100
50EB  7B               LD   A,E
50EC  B2               OR   D
50ED  B1               OR   C
50EE  28 3B            JR   Z,FZERO         allt 0: resultatet 0
50F0  7D               LD   A,L
50F1  D6 09     L50F1: SUB  $09             en byte vänster, exponent - 8
50F3  38 36            JR   C,FZERO         under 1: 0
50F5  3C               INC  A
50F6  41               LD   B,C
50F7  4A               LD   C,D
50F8  53               LD   D,E
50F9  1E 00            LD   E,$00
50FB  04               INC  B
50FC  05               DEC  B
50FD  28 F2            JR   Z,L50F1
50FF  6F               LD   L,A
5100  FA 11 51  L5100: JP   M,FROUND        bit 7 satt: klar
5103  2D        L5103: DEC  L               en bit vänster, exponent - 1
5104  28 25            JR   Z,FZERO         exponent 0: 0
5106  CB 23            SLA  E
5108  CB 12            RL   D
510A  CB 11            RL   C
510C  CB 10            RL   B
510E  F2 03 51         JP   P,L5103
5111  7B        FROUND: LD   A,E            bit 7 i E: avrunda uppåt
5112  17               RLA
5113  30 1E            JR   NC,FPACK
5115  14               INC  D
5116  20 1B            JR   NZ,FPACK
5118  0C               INC  C
5119  20 18            JR   NZ,FPACK
511B  04               INC  B
511C  20 15            JR   NZ,FPACK
511E  06 80            LD   B,$80           $1000000 = $800000, exponent + 1
5120  2C               INC  L
```

```
5121  18 10              JR    FPACK
```

FOVFL: spill, största talet med CY = 1. Tecknet som läggs i H skrivs över av POP HL *(*$5126*), så spill ger alltid +största talet (i BASIC fel 130).* FZERO*: resultatet 0.*

```
5123  60          FOVFL:  LD    H,B             tecknet (skrivs över vid $5126)
5124  3E FF       L5124:  LD    A,$FF
5126  E1                  POP   HL              H = 2 * längden, L = flaggorna
5127  2C                  INC   L               CY-biten sätts
5128  E5                  PUSH  HL
5129  18 01               JR    L512C

512B  AF          FZERO:  XOR   A               allt 0
512C  47          L512C:  LD    B,A
512D  4F                  LD    C,A
512E  57                  LD    D,A
512F  6F                  LD    L,A
5130  A7                  AND   A
5131  28 0A               JR    Z,L513D         talet 0: packas direkt
5133  2C          FPACK:  INC   L
5134  2D                  DEC   L
5135  28 ED               JR    Z,L5124         exponent 255 + 1: spill
5137  7C                  LD    A,H             tecknet = bit 0 i H ...
5138  3C                  INC   A
5139  0F                  RRCA
513A  E6 80               AND   $80
513C  A8                  XOR   B               ... i ettans plats
513D  42          L513D:  LD    B,D             E = exponent, D = tecken ...
513E  57                  LD    D,A
513F  5D                  LD    E,L
5140  E1                  POP   HL
5141  E3                  EX    (SP),HL         HL = returadressen
5142  D9                  EXX
5143  E1                  POP   HL              H = 2 * längden, L = flaggorna
5144  7D                  LD    A,L
5145  6C                  LD    L,H
5146  26 00               LD    H,$00
5148  39                  ADD   HL,SP
5149  F9                  LD    SP,HL           SP förbi båda talen
514A  D9                  EXX
514B  C5                  PUSH  BC              resultatet i det undres plats
514C  D5                  PUSH  DE
514D  E5                  PUSH  HL
514E  E6 01               AND   $01             CY = spill
5150  C8                  RET   Z
5151  37                  SCF
5152  C9                  RET
```

3.35 FTODEC: flyttal -> decimal form (för PRINT, LIST och NUM$) $5153

Talet (IY+$1E bytes) ligger under returadressen; ett 4-bytestal förlängs med nollor till 8 (men räknas fortfarande med 4 bytes). Ett tal med bråk multipliceras med 10^n (P10EXP, POW10) tills det är ett heltal; ett tal från 2^63 divideras en gång med 10^n ner under 2^64. Heltalsdelen skiftas sedan bit för bit in i 11 bytes BCD (dubblering med DAA) och blir 22 ASCII-siffror. Resultatet ersätter talet på stacken: antal siffror (bit 7 = tecken), exponent (talet = 0.siffrorna * 10^exp) och siffrorna; DECNRM stryker nollorna först och sist (talet 0 blir 0 med exponent 0). Med 4 bytes blir ca 8 siffror rätt, med 8 bytes ca 17. In: talet (IY+$1E bytes) under returadressen. Ut: den decimala formen vid SP efter RET, så att den slutar där talet slutade: antal siffror (bit 7 = tecken), exponent och ASCII-siffror; NC. Förstör A, BC, DE, HL, IX och de skuggade registren; med 4 bytes och ett tal från 2^63 är registeruppsättningarna bytta efteråt (FDIV).

```
5153  C1          FTODEC: POP   BC              talet flyttas 4 bytes ned:
5154  C5                  PUSH  BC
5155  C5                  PUSH  BC
5156  C5                  PUSH  BC
5157  21 02 00            LD    HL,$0002
515A  39                  ADD   HL,SP
```

```
515B  EB                EX   DE,HL
515C  21 04 00          LD   HL,$0004
515F  19                ADD  HL,DE
5160  06 00             LD   B,$00
5162  FD 4E 1E          LD   C,(IY+$1E)
5165  79                LD   A,C
5166  ED B0             LDIR                 talet, 2 lediga bytes, returadressen
5168  C1                POP  BC
5169  2B                DEC  HL
516A  70                LD   (HL),B
516B  2B                DEC  HL
516C  71                LD   (HL),C
516D  FE 04             CP   $04
516F  CC 8B 4C          CALL Z,FEXT8         4 bytes: förläng till 8
5172  21 00 00          LD   HL,$0000
5175  39                ADD  HL,SP
5176  E5                PUSH HL
5177  DD E1             POP  IX
5179  DD 36 08 00       LD   (IX+$08),$00    IX+8 = tiopotensen, 0
517D  DD 7E 00    L517D: LD   A,(IX+$00)      exponent 0: talet 0
5180  A7                AND  A
5181  28 7D             JR   Z,L5200
5183  FE B8             CP   $B8             från 2^55
5185  30 2E             JR   NC,L51B5
5187  FE 81             CP   $81             under 1: bråk
5189  38 2A             JR   C,L51B5
518B  D6 80             SUB  $80             C = antal heltalsbitar
518D  4F                LD   C,A
518E  E6 07             AND  $07
5190  47                LD   B,A
5191  79                LD   A,C
5192  E6 F8             AND  $F8
5194  0F                RRCA
5195  0F                RRCA
5196  0F                RRCA
5197  3C                INC  A               HL = byten med första bråkbiten
5198  6F                LD   L,A
5199  26 00             LD   H,$00
519B  39                ADD  HL,SP
519C  2F                CPL                  C = bytes kvar till slutet
519D  C6 09             ADD  A,$09
519F  4F                LD   C,A
51A0  3E FF             LD   A,$FF           mask för bråkbitarna i byten
51A2  05          L51A2: DEC  B
51A3  FA AA 51          JP   M,L51AA
51A6  A7                AND  A
51A7  1F                RRA
51A8  18 F8             JR   L51A2

51AA  A6          L51AA: AND  (HL)           bråkbit 1: skala om
51AB  20 08             JR   NZ,L51B5
51AD  3E FF             LD   A,$FF
51AF  23                INC  HL
51B0  0D                DEC  C
51B1  20 F7             JR   NZ,L51AA
51B3  18 4B             JR   L5200           heltal: till BCD

51B5  3E B8       L51B5: LD   A,$B8
51B7  DD 96 00          SUB  (IX+$00)
51BA  30 14             JR   NC,L51D0        <= $B8: bråk (eller 2^55-2^56)
51BC  ED 44             NEG                  från 2^63 (exponent $C0) ...
51BE  D6 08             SUB  $08
51C0  38 3E             JR   C,L5200         ... annars heltal ändå
51C2  CD F0 51          CALL P10EXP          ... / 10^n, ner under 2^64
51C5  DD 71 08          LD   (IX+$08),C
51C8  CD 25 4C          CALL FLOAD
51CB  CD 5F 50          CALL FDIV
51CE  18 30             JR   L5200           en gång räcker

51D0  FD 7E 1E    L51D0: LD   A,(IY+$1E)      bitar i mantissan: 24 eller 56
```

```
51D3  3D                DEC  A
51D4  07                RLCA
51D5  07                RLCA
51D6  07                RLCA
51D7  C6 80             ADD  A,$80
51D9  DD 96 00          SUB  (IX+$00)        A = bitar som saknas
51DC  CD F0 51          CALL P10EXP          minsta 10^n som räcker
51DF  79                LD   A,C
51E0  ED 44             NEG
51E2  DD 86 08          ADD  A,(IX+$08)      IX+8 = IX+8 - n
51E5  DD 77 08          LD   (IX+$08),A
51E8  CD 25 4C          CALL FLOAD
51EB  CD EA 4F          CALL FMUL            * 10^n, sedan om igen
51EE  18 8D             JR   L517D
```

P10EXP: minsta n >= 1 där 10^n i POW10 har minst exponenten A + $81. För mycket små tal slår summan runt och n blir 1 (fler varv i FTODEC, samma svar). In: A = antalet bitar som saknas. Ut: C = n, HL vid 10^n i POW10 ($4DF2 + 8n), som FLOAD sedan läser. Förstör A och DE (= 8); B bevaras.

```
51F0  C6 81       P10EXP: ADD  A,$81           sökt exponent (slår runt över $7E)
51F2  21 F2 4D            LD   HL,$4DF2
51F5  11 08 00            LD   DE,$0008
51F8  4A                  LD   C,D
51F9  19          L51F9:  ADD  HL,DE
51FA  0C                  INC  C
51FB  BE                  CP   (HL)            10^C:s exponent >= A?
51FC  C8                  RET  Z
51FD  30 FA               JR   NC,L51F9
51FF  C9                  RET

5200  21 E8 FF    L5200:  LD   HL,$FFE8        24 bytes: huvud, siffror, BCD
5203  39                  ADD  HL,SP
5204  F9                  LD   SP,HL
5205  E5                  PUSH HL
5206  DD 7E 01            LD   A,(IX+$01)      antal 22, bit 7 = tecknet
5209  E6 80               AND  $80
520B  C6 16               ADD  A,$16
520D  77                  LD   (HL),A
520E  23                  INC  HL
520F  DD 7E 08            LD   A,(IX+$08)      exponent = 22 + IX+8
5212  C6 16               ADD  A,$16
5214  77                  LD   (HL),A
5215  DD CB 01 FE         SET  7,(IX+$01)      ettan in i mantissan
5219  DD E1               POP  IX
521B  21 0D 00            LD   HL,$000D
521E  39                  ADD  HL,SP
521F  E5                  PUSH HL
5220  D1                  POP  DE
5221  13                  INC  DE
5222  36 00               LD   (HL),$00        BCD (11 bytes) = 0
5224  01 0A 00            LD   BC,$000A
5227  ED B0               LDIR
5229  23                  INC  HL
522A  23                  INC  HL
522B  1A                  LD   A,(DE)          A = exponenten
522C  01 07 00            LD   BC,$0007        mantissan en byte ned
522F  ED B0               LDIR
5231  A7                  AND  A
5232  28 20               JR   Z,L5254         talet 0: bara nollor
5234  D6 80               SUB  $80             C = antal heltalsbitar
5236  4F                  LD   C,A
5237  DD 77 21            LD   (IX+$21),A
523A  AF                  XOR  A
523B  12                  LD   (DE),A
523C  EB                  EX   DE,HL
523D  E5          L523D:  PUSH HL              mantissan en bit vänster ...
523E  06 08               LD   B,$08
5240  AF                  XOR  A
5241  7E          L5241:  LD   A,(HL)
```

```
5242  17                RLA
5243  77                LD   (HL),A
5244  2B                DEC  HL
5245  10 FA             DJNZ L5241
5247  06 0B             LD   B,$0B          ... in i BCD: * 2 + biten
5249  7E          L5249: LD   A,(HL)
524A  8F                ADC  A,A
524B  27                DAA
524C  77                LD   (HL),A
524D  2B                DEC  HL
524E  10 F9             DJNZ L5249
5250  E1                POP  HL
5251  0D                DEC  C
5252  20 E9             JR   NZ,L523D
5254  21 02 00    L5254: LD   HL,$0002       bitarna efter de C stryks
5257  39                ADD  HL,SP
5258  EB                EX   DE,HL
5259  21 0B 00          LD   HL,$000B
525C  45                LD   B,L            B = 11 bytes BCD
525D  19                ADD  HL,DE
525E  EB                EX   DE,HL
525F  1A          L525F: LD   A,(DE)         två ASCII-siffror per byte
5260  0F                RRCA
5261  0F                RRCA
5262  0F                RRCA
5263  0F                RRCA
5264  E6 0F             AND  $0F
5266  C6 30             ADD  A,$30
5268  77                LD   (HL),A
5269  23                INC  HL
526A  1A                LD   A,(DE)
526B  E6 0F             AND  $0F
526D  C6 30             ADD  A,$30
526F  77                LD   (HL),A
5270  23                INC  HL
5271  13                INC  DE
5272  10 EB             DJNZ L525F
5274  DD 4E 22          LD   C,(IX+$22)     BC = returadressen
5277  DD 46 23          LD   B,(IX+$23)
527A  21 17 00          LD   HL,$0017
527D  39                ADD  HL,SP
527E  EB                EX   DE,HL
527F  21 23 00          LD   HL,$0023
5282  39                ADD  HL,SP
5283  C5                PUSH BC
5284  01 1A 00          LD   BC,$001A
5287  EB                EX   DE,HL
5288  ED B8             LDDR                huvud och siffror upp till slutet
528A  13                INC  DE
528B  EB                EX   DE,HL
528C  F9                LD   SP,HL
528D  C3 7B 4D          JP   DECNRM         nollor bort, tillbaka till anroparen
```

3.36 DECTOF: decimal form -> flyttal (för VAL, INPUT/READ och kompileringen)

$5290

Siffrorna multipliceras in i ett 8-bytes heltal (x*8 + x*2 + siffra) tills det når 2^60, vilket sker efter den 19:e eller 20:e siffran (den första är aldrig 0); resten av siffrorna huggs av och räknas in i exponenten. Heltalet normaliseras (FNRM9, 8 bytes) och multipliceras eller divideras med 10^n ur POW10 (P10LD), n = använda siffror - exponenten. Talet ersätter den decimala formen på stacken; med 4 bytes avrundas det till sist (FRND4). n >= 39 ger CY och den decimala formen tas bort (i BASIC fel 130), även när talet går att lagra: 1E-38 går, men 1.5E-38 och 1.234567E-35 inte. Multiplikationen och divisionen görs i IY+$1E:s längd, med 4 bytes alltså bara på heltalets första fyra bytes, och de fyra sista avgör sedan avrundningen (se $5306). Kompileringen sätter därför IY+$1E = 8 ($1949). In: den decimala formen under returadressen, som TXTDEC och STRDEC lämnar den. Ut: talet (IY+$1E bytes) under returadressen, så att det slutar där den decimala formen slutade, NC; vid spill CY och inget kvar på stacken. Förstör A, BC, DE, HL och de skuggade registren, med 4 bytes även IX (FRND4); efter en division med 4 bytes är registeruppsättningarna bytta (FDIV).

```
5290  21 F7 FF     DECTOF: LD   HL,$FFF7        9 bytes: exponent $C0 ...
5293  39                   ADD  HL,SP
5294  F9                   LD   SP,HL
5295  36 C0                LD   (HL),$C0
5297  23                   INC  HL
5298  E5                   PUSH HL
5299  D1                   POP  DE
529A  13                   INC  DE
529B  36 00                LD   (HL),$00        ... och 8 bytes heltal = 0
529D  01 07 00             LD   BC,$0007
52A0  ED B0                LDIR
52A2  13                   INC  DE
52A3  13                   INC  DE
52A4  EB                   EX   DE,HL
52A5  7E                   LD   A,(HL)          C = antal siffror
52A6  E6 7F                AND  $7F
52A8  4F                   LD   C,A
52A9  23                   INC  HL
52AA  23                   INC  HL
52AB  7E                   LD   A,(HL)          första siffran i lägsta byten
52AC  E6 0F                AND  $0F
52AE  12                   LD   (DE),A
52AF  06 01                LD   B,$01           B = bytes att multiplicera
52B1  23             L52B1: INC  HL
52B2  0D                   DEC  C               inga fler siffror
52B3  28 2F                JR   Z,L52E4
52B5  E5                   PUSH HL
52B6  D5                   PUSH DE
52B7  C5                   PUSH BC
52B8  AF                   XOR  A               heltalet * 10 = * 8 + * 2
52B9  6F                   LD   L,A
52BA  1A             L52BA: LD   A,(DE)
52BB  67                   LD   H,A
52BC  F5                   PUSH AF              CY från förra byten sparas
52BD  29                   ADD  HL,HL
52BE  4C                   LD   C,H
52BF  29                   ADD  HL,HL
52C0  29                   ADD  HL,HL
52C1  F1                   POP  AF
52C2  6F                   LD   L,A
52C3  79                   LD   A,C
52C4  8C                   ADC  A,H             byten * 2 + byten * 8 + CY
52C5  12                   LD   (DE),A
52C6  1B                   DEC  DE
52C7  10 F1                DJNZ L52BA
52C9  C1                   POP  BC
52CA  E1                   POP  HL
52CB  D1                   POP  DE
52CC  1A                   LD   A,(DE)
52CD  E6 0F                AND  $0F
52CF  E5                   PUSH HL
```

```
52D0  06 08              LD   B,$08
52D2  8E          L52D2: ADC  A,(HL)          siffran läggs till
52D3  77                 LD   (HL),A
52D4  2B                 DEC  HL
52D5  3E 00              LD   A,$00
52D7  10 F9              DJNZ L52D2
52D9  23                 INC  HL
52DA  7E                 LD   A,(HL)         övre halvbyten i högsta byten
52DB  E6 F0              AND  $F0
52DD  E1                 POP  HL
52DE  EB                 EX   DE,HL
52DF  06 08              LD   B,$08
52E1  28 CE              JR   Z,L52B1        under 2^60: nästa siffra
52E3  0D                 DEC  C              fullt: resten av siffrorna ...
52E4  21 0C 00    L52E4: LD   HL,$000C
52E7  39                 ADD  HL,SP
52E8  79                 LD   A,C
52E9  86                 ADD  A,(HL)         ... läggs till exponenten
52EA  77                 LD   (HL),A
52EB  CD 57 4D           CALL FNRM9          normalisera, 8 bytes
52EE  21 0A 00           LD   HL,$000A
52F1  39                 ADD  HL,SP
52F2  4E                 LD   C,(HL)         C = antal (tecken), B = exponent
52F3  23                 INC  HL
52F4  46                 LD   B,(HL)
52F5  D1                 POP  DE
52F6  79                 LD   A,C            tecknet in i bit 7
52F7  17                 RLA
52F8  CB 12              RL   D
52FA  CB 0A              RRC  D
52FC  D5                 PUSH DE
52FD  79                 LD   A,C
52FE  E6 7F              AND  $7F
5300  90                 SUB  B              n = antal - exponent
5301  28 13              JR   Z,L5316
5303  FA 0E 53           JP   M,L530E        negativ: multiplicera
5306  CD 3E 53           CALL P10LD          / 10^n (i IY+$1E:s längd) (gränsfall)
5309  CD 5F 50           CALL FDIV
530C  18 08              JR   L5316

530E  ED 44       L530E: NEG
5310  CD 3E 53           CALL P10LD          * 10^n
5313  CD EA 4F           CALL FMUL
5316  38 38       L5316: JR   C,L5350        spill: fel
5318  21 0A 00           LD   HL,$000A
531B  39                 ADD  HL,SP
531C  7E                 LD   A,(HL)         talet ersätter den decimala formen
531D  E6 7F              AND  $7F
531F  5F                 LD   E,A
5320  16 00              LD   D,$00
5322  13                 INC  DE
5323  EB                 EX   DE,HL
5324  19                 ADD  HL,DE
5325  EB                 EX   DE,HL
5326  2B                 DEC  HL
5327  46                 LD   B,(HL)
5328  2B                 DEC  HL
5329  4E                 LD   C,(HL)
532A  2B                 DEC  HL
532B  C5                 PUSH BC
532C  01 08 00           LD   BC,$0008
532F  ED B8              LDDR
5331  13                 INC  DE
5332  EB                 EX   DE,HL
5333  D1                 POP  DE
5334  F9                 LD   SP,HL
5335  D5                 PUSH DE
5336  FD CB 1E 56        BIT  2,(IY+$1E)     4 bytes: avrunda
533A  C2 99 4C           JP   NZ,FRND4
```

```
533D  C9                  RET
```

P10LD: *10^A ur POW10 till stacken, IY+$1E bytes (FLOAD). A >= 39 tar bort returadressen till DECTOF och går till felutgången $5350 (CY, den decimala formen bort). In: A = n, som inte får vara negativt. Ut: talet vid SP. Förstör A, BC, DE och HL.*

```
533E  FE 27       P10LD:  CP   $27            10^A ur POW10; 10^39 och uppåt: spill (gränsfall)
5340  30 0D               JR   NC,L534F
5342  6F                  LD   L,A
5343  26 00               LD   H,$00
5345  29                  ADD  HL,HL          HL = POW10 + 8 * A
5346  29                  ADD  HL,HL
5347  29                  ADD  HL,HL
5348  11 F2 4D            LD   DE,$4DF2
534B  19                  ADD  HL,DE
534C  C3 25 4C            JP   FLOAD          IY+$1E bytes

534F  E1          L534F:  POP  HL              P10LD:s returadress bort
5350  21 08 00    L5350:  LD   HL,$0008       decimala formen bort, CY
5353  39                  ADD  HL,SP
5354  5E                  LD   E,(HL)
5355  23                  INC  HL
5356  56                  LD   D,(HL)
5357  23                  INC  HL
5358  7E                  LD   A,(HL)
5359  E6 7F               AND  $7F
535B  4F                  LD   C,A
535C  06 00               LD   B,$00
535E  03                  INC  BC
535F  03                  INC  BC
5360  09                  ADD  HL,BC
5361  F9                  LD   SP,HL
5362  EB                  EX   DE,HL
5363  37                  SCF
5364  E9                  JP   (HL)
```

DADD (*8 bytes, från FADD*): *som FADD, men mantissan är 56 bitar i BCDE B'C'D' och E' är skyddsbyte. Det mindre talet skiftas höger högst 57 steg; bitar under E' försvinner. Två egenheter (som i FADD med 25 steg, se $4F77): vid subtraktion dras det avhuggna talet ifrån, så att en skillnad strax under en avrundningsgräns kan avrundas uppåt; och ett tal mer än 57 steg ned räknas som 57 steg ned. Ett tal med exponent 0 skiftas också 57 steg, med mantissan (för 0 och -0 utan verkan). Tecken och exponent ligger i H och L på stacken som i FADD. In: två 8-bytestal under returadressen (via FADD). Ut: summan i det undres plats, CY = spill. Förstör A, BC, DE, HL och de skuggade BC', DE', HL'; registeruppsättningarna är inte bytta efteråt.*

```
5365  CD 2A 4F    DADD:   CALL FARGS
5368  F5                  PUSH AF             A = 2 * längden, till DPACK
5369  1A                  LD   A,(DE)
536A  96                  SUB  (HL)           exponentskillnaden
536B  30 03               JR   NC,L5370
536D  EB                  EX   DE,HL          HL = det mindre talet
536E  ED 44               NEG
5370  4F          L5370:  LD   C,A            C = steg att skifta
5371  7E                  LD   A,(HL)
5372  A7                  AND  A
5373  20 07               JR   NZ,L537C
5375  0C                  INC  C
5376  0D                  DEC  C
5377  CA 2B 56            JP   Z,DZERO        båda 0: resultatet 0
537A  0E 39               LD   C,$39          exponent 0: alltid 57 steg
537C  D5          L537C:  PUSH DE             det större talet
537D  23                  INC  HL
537E  13                  INC  DE
537F  1A                  LD   A,(DE)
5380  AE                  XOR  (HL)           bit 7: olika tecken
5381  47                  LD   B,A
5382  1A                  LD   A,(DE)
5383  17                  RLA
5384  78                  LD   A,B
5385  1F                  RRA
5386  07                  RLCA
```

```
5387  47              LD   B,A          B: bit 7 olika, bit 0 tecknet
5388  C5              PUSH BC
5389  CD 63 56        CALL DREG         det mindre i BCDE B'C'D'E'
538C  E1              POP  HL
538D  E5              PUSH HL
538E  7D              LD   A,L
538F  FE 3A           CP   $3A
5391  38 02           JR   C,L5395      högst 57 steg
5393  3E 39           LD   A,$39
5395  D6 08     L5395: SUB  $08         hela bytes först
5397  38 10           JR   C,L53A9
5399  F5              PUSH AF
539A  7B              LD   A,E
539B  5A              LD   E,D
539C  51              LD   D,C
539D  48              LD   C,B
539E  06 00           LD   B,$00
53A0  D9              EXX               E' faller ut
53A1  5A              LD   E,D
53A2  51              LD   D,C
53A3  48              LD   C,B
53A4  47              LD   B,A
53A5  D9              EXX
53A6  F1              POP  AF
53A7  18 EC           JR   L5395

53A9  C6 08     L53A9: ADD  A,$08
53AB  6F              LD   L,A
53AC  28 07           JR   Z,L53B5
53AE  A7        L53AE: AND  A           sedan bitvis
53AF  CD 6D 56        CALL DSHR
53B2  2D              DEC  L
53B3  20 F9           JR   NZ,L53AE
53B5  E1        L53B5: POP  HL
53B6  24              INC  H            S = olika tecken
53B7  25              DEC  H
53B8  E3              EX   (SP),HL
53B9  7E              LD   A,(HL)
53BA  E3              EX   (SP),HL
53BB  6F              LD   L,A          L = det större exponent
53BC  E3              EX   (SP),HL      H och L på stacken
53BD  D5              PUSH DE
53BE  11 07 00        LD   DE,$0007
53C1  19              ADD  HL,DE        HL = det större sista byte
53C2  D1              POP  DE
53C3  FA F4 53        JP   M,L53F4      olika tecken: subtrahera
53C6  E5              PUSH HL
53C7  D9              EXX
53C8  E1              POP  HL
53C9  7A              LD   A,D          addera, sista byten först
53CA  86              ADD  A,(HL)
53CB  57              LD   D,A
53CC  2B              DEC  HL
53CD  79              LD   A,C
53CE  8E              ADC  A,(HL)
53CF  4F              LD   C,A
53D0  2B              DEC  HL
53D1  78              LD   A,B
53D2  8E              ADC  A,(HL)
53D3  47              LD   B,A
53D4  2B              DEC  HL
53D5  E5              PUSH HL
53D6  D9              EXX
53D7  E1              POP  HL
53D8  7B              LD   A,E
53D9  8E              ADC  A,(HL)
53DA  5F              LD   E,A
53DB  2B              DEC  HL
53DC  7A              LD   A,D
```

```
53DD  8E                ADC  A,(HL)
53DE  57                LD   D,A
53DF  2B                DEC  HL
53E0  79                LD   A,C
53E1  8E                ADC  A,(HL)
53E2  4F                LD   C,A
53E3  2B                DEC  HL
53E4  7E                LD   A,(HL)
53E5  CB FF             SET  7,A            ettan in
53E7  88                ADC  A,B
53E8  47                LD   B,A
53E9  E1                POP  HL
53EA  D2 00 56          JP   NC,DROUND      ingen minnessiffra: avrunda
53ED  CD 6D 56          CALL DSHR           annars ett steg höger ...
53F0  2C                INC  L              ... och exponenten + 1
53F1  C3 00 56          JP   DROUND

53F4  E5          L53F4: PUSH HL           det större minus det mindre
53F5  D9                EXX
53F6  E1                POP  HL
53F7  AF                XOR  A              skyddsbyten: 0 - E'
53F8  93                SUB  E
53F9  5F                LD   E,A
53FA  7E                LD   A,(HL)
53FB  9A                SBC  A,D
53FC  57                LD   D,A
53FD  2B                DEC  HL
53FE  7E                LD   A,(HL)
53FF  99                SBC  A,C
5400  4F                LD   C,A
5401  2B                DEC  HL
5402  7E                LD   A,(HL)
5403  98                SBC  A,B
5404  47                LD   B,A
5405  2B                DEC  HL
5406  E5                PUSH HL
5407  D9                EXX
5408  E1                POP  HL
5409  7E                LD   A,(HL)
540A  9B                SBC  A,E
540B  5F                LD   E,A
540C  2B                DEC  HL
540D  7E                LD   A,(HL)
540E  9A                SBC  A,D
540F  57                LD   D,A
5410  2B                DEC  HL
5411  7E                LD   A,(HL)
5412  99                SBC  A,C
5413  4F                LD   C,A
5414  2B                DEC  HL
5415  7E                LD   A,(HL)
5416  CB FF             SET  7,A
5418  98                SBC  A,B
5419  47                LD   B,A
541A  D2 BC 55          JP   NC,DNORM       positivt: normalisera
541D  E1                POP  HL
541E  24                INC  H              negativt: tecknet byts ...
541F  E5                PUSH HL
5420  D9                EXX
5421  AF                XOR  A              ... och beloppet tas
5422  67                LD   H,A
5423  93                SUB  E
5424  5F                LD   E,A
5425  7C                LD   A,H
5426  9A                SBC  A,D
5427  57                LD   D,A
5428  7C                LD   A,H
5429  99                SBC  A,C
542A  4F                LD   C,A
```

```
542B  7C                LD   A,H
542C  98                SBC  A,B
542D  47                LD   B,A
542E  7C                LD   A,H
542F  D9                EXX
5430  67                LD   H,A
5431  9B                SBC  A,E
5432  5F                LD   E,A
5433  7C                LD   A,H
5434  9A                SBC  A,D
5435  57                LD   D,A
5436  7C                LD   A,H
5437  99                SBC  A,C
5438  4F                LD   C,A
5439  7C                LD   A,H
543A  98                SBC  A,B
543B  47                LD   B,A
543C  C3 BC 55          JP   DNORM
```

DMUL: *produkten byggs i BCDE B'C'D'E' med skift och addition. Det tal som har minst antal bytes kvar när nollorna sist är borträknade (DLEN) blir multiplikator; dess bitar skiftas ut ur talet på stacken från den sista byten som inte är 0, och för varje bit skiftas produkten ett steg höger. Bitar under E' försvinner, men resultatet blir ändå korrekt avrundat. Exponent = exp1 + exp2 - $80: under 1 ger 0, över 255 spill (+största talet, CY). In: två 8-bytestal under returadressen (via FMUL). Ut: produkten i det undres plats, CY = spill. Förstör A, BC, DE, HL och de skuggade BC', DE', HL'.*

```
543F  CD 2A 4F    DMUL:  CALL FARGS
5442  F5                PUSH AF            A = 2 * längden, till DPACK
5443  1A                LD   A,(DE)
5444  47                LD   B,A
5445  4E                LD   C,(HL)
5446  A7                AND  A
5447  CA 2B 56          JP   Z,DZERO       en faktor 0: resultatet 0
544A  0C                INC  C
544B  0D                DEC  C
544C  CA 2B 56          JP   Z,DZERO
544F  13                INC  DE
5450  23                INC  HL
5451  1A                LD   A,(DE)
5452  AE                XOR  (HL)
5453  07                RLCA               bit 0 = produktens tecken
5454  F5                PUSH AF
5455  78                LD   A,B
5456  3D                DEC  A
5457  81                ADD  A,C           exp1 - 1 + exp2
5458  C1                POP  BC
5459  FA 61 54          JP   M,L5461       bit 7: $80-$FF eller från $180
545C  D2 2B 56          JP   NC,DZERO      under $80: 0
545F  18 03             JR   L5464         $100-$17F

5461  DA 23 56    L5461: JP   C,DOVFL       från $180: spill
5464  C6 81       L5464: ADD  A,$81         C = exp1 + exp2 - $80
5466  4F                 LD   C,A
5467  C5                 PUSH BC
5468  E5                 PUSH HL
5469  CB FE              SET  7,(HL)        ettan in
546B  CD E6 54           CALL DLEN          A = bytes utom nollorna sist
546E  4F                 LD   C,A
546F  D5                 PUSH DE
5470  EB                 EX   DE,HL
5471  CB FE              SET  7,(HL)
5473  CD E6 54           CALL DLEN
5476  B9                 CP   C             färre bytes blir multiplikator
5477  38 05              JR   C,L547E
5479  EB                 EX   DE,HL
547A  D1                 POP  DE
547B  E3                 EX   (SP),HL       multiplikatorn (sista byten) på stacken
547C  18 04              JR   L5482

547E  D1          L547E: POP  DE
```

```
547F  D1              POP  DE
5480  4F              LD   C,A
5481  E5              PUSH HL
5482  06 09     L5482: LD   B,$09          B = bitar, C = bytes
5484  C5              PUSH BC
5485  21 06 00        LD   HL,$0006       HL' = multiplikandens sista byte
5488  19              ADD  HL,DE
5489  E5              PUSH HL
548A  D9              EXX
548B  E1              POP  HL
548C  11 00 00        LD   DE,$0000       produkten BCDE B'C'D'E' = 0
548F  D5              PUSH DE
5490  D5              PUSH DE
5491  D5              PUSH DE
5492  C1              POP  BC
5493  D9              EXX
5494  D1              POP  DE
5495  C1              POP  BC
5496  A7        L5496: AND  A
5497  E1              POP  HL
5498  25              DEC  H              8 bitar per byte
5499  20 06           JR   NZ,L54A1
549B  26 08           LD   H,$08
549D  2D              DEC  L
549E  28 3F           JR   Z,L54DF        alla bytes: klart
54A0  37              SCF                 CY: nästa byte
54A1  E3        L54A1: EX   (SP),HL
54A2  30 01           JR   NC,L54A5
54A4  2B              DEC  HL
54A5  CB 1E     L54A5: RR   (HL)          lägsta biten ut i CY
54A7  E3              EX   (SP),HL
54A8  E5              PUSH HL
54A9  30 20           JR   NC,L54CB       0: bara skift
54AB  D9              EXX
54AC  E5              PUSH HL
54AD  7A              LD   A,D            multiplikanden läggs till
54AE  86              ADD  A,(HL)
54AF  57              LD   D,A
54B0  2B              DEC  HL
54B1  79              LD   A,C
54B2  8E              ADC  A,(HL)
54B3  4F              LD   C,A
54B4  2B              DEC  HL
54B5  78              LD   A,B
54B6  8E              ADC  A,(HL)
54B7  47              LD   B,A
54B8  2B              DEC  HL
54B9  E3              EX   (SP),HL
54BA  D9              EXX
54BB  E1              POP  HL
54BC  7B              LD   A,E
54BD  8E              ADC  A,(HL)
54BE  5F              LD   E,A
54BF  2B              DEC  HL
54C0  7A              LD   A,D
54C1  8E              ADC  A,(HL)
54C2  57              LD   D,A
54C3  2B              DEC  HL
54C4  79              LD   A,C
54C5  8E              ADC  A,(HL)
54C6  4F              LD   C,A
54C7  2B              DEC  HL
54C8  7E              LD   A,(HL)
54C9  88              ADC  A,B
54CA  47              LD   B,A
54CB  CB 18     L54CB: RR   B              produkten ett steg höger
54CD  CB 19           RR   C
54CF  CB 1A           RR   D
54D1  CB 1B           RR   E
```

```
54D3  D9                EXX
54D4  CB 18             RR   B
54D6  CB 19             RR   C
54D8  CB 1A             RR   D
54DA  CB 1B             RR   E
54DC  D9                EXX
54DD  18 B7             JR   L5496

54DF  E1          L54DF: POP  HL            H = tecken, L = exponent
54E0  E1                 POP  HL
54E1  04                 INC  B
54E2  05                 DEC  B
54E3  C3 E5 55           JP   L55E5         normalisera
```

DLEN: *antalet bytes i mantissan till och med den sista som inte är 0. In: HL vid mantissans första byte (7 bytes, ettan insatt, så att svaret blir minst 1). Ut: A = 1-7, HL vid den byten. BC och DE bevaras.*

```
54E6  C5          DLEN:  PUSH BC
54E7  01 07 00           LD   BC,$0007      från talets slut
54EA  09                 ADD  HL,BC
54EB  2B          L54EB: DEC  HL
54EC  7E                 LD   A,(HL)
54ED  A7                 AND  A
54EE  20 03              JR   NZ,L54F3
54F0  0D                 DEC  C
54F1  20 F8              JR   NZ,L54EB
54F3  79          L54F3: LD   A,C           A = bytes (1-7)
54F4  C1                 POP  BC
54F5  C9                 RET
```

DDIV: *det undre talet / det översta, division med återställning som FDIV. Resten (först täljaren) jämförs byte för byte med nämnaren; är den större eller lika dras nämnaren ifrån och kvotbiten blir 1. Kvotbitarna skiftas in i H med en vakt-etta; en full byte skrivs över täljaren på stacken. 64 bitar räknas, den första är heltalsbiten (kvoten av mantissorna ligger mellan 1/2 och 2), och resultatet blir korrekt avrundat. Nämnaren 0 ger spill, täljaren 0 ger 0. Exponent = skillnaden + $81, - 1 vid normaliseringen. In: två 8-bytestal under returadressen (via FDIV), täljaren underst. Ut: kvoten i täljarens plats, CY = spill. Förstör A, BC, DE, HL och de skuggade BC', DE', HL'; till skillnad från FDIV med 4 bytes är registeruppsättningarna inte bytta efteråt.*

```
54F6  CD 2A 4F    DDIV:  CALL FARGS
54F9  F5                 PUSH AF            A = 2 * längden, till DPACK
54FA  1A                 LD   A,(DE)
54FB  4F                 LD   C,A
54FC  13                 INC  DE
54FD  1A                 LD   A,(DE)
54FE  07                 RLCA
54FF  47                 LD   B,A
5500  7E                 LD   A,(HL)
5501  A7                 AND  A
5502  CA 23 56           JP   Z,DOVFL       nämnaren 0: spill
5505  0C                 INC  C
5506  0D                 DEC  C
5507  CA 2B 56           JP   Z,DZERO       täljaren 0: resultatet 0
550A  47                 LD   B,A
550B  1A                 LD   A,(DE)
550C  23                 INC  HL
550D  AE                 XOR  (HL)
550E  07                 RLCA               bit 0 = kvotens tecken
550F  F5                 PUSH AF
5510  79                 LD   A,C           exponentskillnaden
5511  90                 SUB  B
5512  C1                 POP  BC
5513  FA 1B 55           JP   M,L551B       bit 7: -128 till -1 eller från 128
5516  DA 2B 56           JP   C,DZERO       under -128: 0
5519  18 03              JR   L551E         0 till 127

551B  D2 23 56    L551B: JP   NC,DOVFL      från 128: spill
551E  C6 81       L551E: ADD  A,$81         C = skillnaden + $81
5520  4F                 LD   C,A
5521  C5                 PUSH BC
5522  CB FE              SET  7,(HL)        nämnarens etta in
```

```
5524  E5                PUSH HL
5525  EB                EX   DE,HL
5526  E5                PUSH HL
5527  CD 63 56          CALL DREG            täljaren i BCDE B'C'D'
552A  D9                EXX
552B  E1                POP  HL
552C  2B                DEC  HL              HL' = kvotens plats (täljaren)
552D  D9                EXX
552E  21 08 01          LD   HL,$0108        H = vakt-etta, L = 8 bytes
5531  A7                AND  A
5532  E3          L5532: EX   (SP),HL
5533  38 2C             JR   C,L5561         minnessiffra: resten större
5535  7E                LD   A,(HL)          jämför resten med nämnaren
5536  B8                CP   B
5537  20 26             JR   NZ,L555F
5539  E5                PUSH HL
553A  23                INC  HL
553B  7E                LD   A,(HL)
553C  B9                CP   C
553D  20 1F             JR   NZ,L555E
553F  23                INC  HL
5540  7E                LD   A,(HL)
5541  BA                CP   D
5542  20 1A             JR   NZ,L555E
5544  23                INC  HL
5545  7E                LD   A,(HL)
5546  BB                CP   E
5547  20 15             JR   NZ,L555E
5549  23                INC  HL
554A  E5                PUSH HL
554B  D9                EXX
554C  E3                EX   (SP),HL
554D  7E                LD   A,(HL)
554E  B8                CP   B
554F  20 0B             JR   NZ,L555C
5551  23                INC  HL
5552  7E                LD   A,(HL)
5553  B9                CP   C
5554  20 06             JR   NZ,L555C
5556  23                INC  HL
5557  7E                LD   A,(HL)
5558  BA                CP   D
5559  20 01             JR   NZ,L555C
555B  37                SCF
555C  E1          L555C: POP  HL
555D  D9                EXX
555E  E1          L555E: POP  HL
555F  30 28       L555F: JR   NC,L5589       mindre: kvotbiten 0
5561  D5          L5561: PUSH DE             annars resten - nämnaren ...
5562  11 06 00          LD   DE,$0006
5565  19                ADD  HL,DE
5566  D1                POP  DE
5567  E5                PUSH HL
5568  D9                EXX
5569  E3                EX   (SP),HL
556A  7A                LD   A,D
556B  96                SUB  (HL)
556C  57                LD   D,A
556D  2B                DEC  HL
556E  79                LD   A,C
556F  9E                SBC  A,(HL)
5570  4F                LD   C,A
5571  2B                DEC  HL
5572  78                LD   A,B
5573  9E                SBC  A,(HL)
5574  47                LD   B,A
5575  2B                DEC  HL
5576  E3                EX   (SP),HL
5577  D9                EXX
```

```
5578  E1              POP   HL
5579  7B              LD    A,E
557A  9E              SBC   A,(HL)
557B  5F              LD    E,A
557C  2B              DEC   HL
557D  7A              LD    A,D
557E  9E              SBC   A,(HL)
557F  57              LD    D,A
5580  2B              DEC   HL
5581  79              LD    A,C
5582  9E              SBC   A,(HL)
5583  4F              LD    C,A
5584  2B              DEC   HL
5585  78              LD    A,B
5586  9E              SBC   A,(HL)
5587  47              LD    B,A
5588  37              SCF                   ... och kvotbiten 1
5589  E3        L5589:EX    (SP),HL
558A  7C              LD    A,H
558B  17              RLA                   biten in, vakt-ettan ut?
558C  67              LD    H,A
558D  30 09           JR    NC,L5598
558F  2D              DEC   L               full byte
5590  D9              EXX
5591  28 17           JR    Z,L55AA         8 bytes: klart
5593  77              LD    (HL),A          kvotbyten sparas
5594  23              INC   HL
5595  D9              EXX
5596  26 01           LD    H,$01
5598  D9        L5598:EXX
5599  CB 22           SLA   D               resten ett steg vänster
559B  CB 11           RL    C
559D  CB 10           RL    B
559F  D9              EXX
55A0  CB 13           RL    E
55A2  CB 12           RL    D
55A4  CB 11           RL    C
55A6  CB 10           RL    B
55A8  18 88           JR    L5532

55AA  5F        L55AA:LD    E,A             kvoten (64 bitar) ...
55AB  2B              DEC   HL
55AC  56              LD    D,(HL)
55AD  2B              DEC   HL
55AE  4E              LD    C,(HL)
55AF  2B              DEC   HL
55B0  46              LD    B,(HL)
55B1  2B              DEC   HL
55B2  E3              EX    (SP),HL
55B3  D9              EXX
55B4  E1              POP   HL
55B5  5E              LD    E,(HL)          ... i BCDE B'C'D'E'
55B6  2B              DEC   HL
55B7  56              LD    D,(HL)
55B8  2B              DEC   HL
55B9  4E              LD    C,(HL)
55BA  2B              DEC   HL
55BB  46              LD    B,(HL)
```

DNORM: *BCDE B'C'D'E' skiftas vänster, först hela bytes och sedan bitvis, tills bit 7 i B är satt; exponenten L minskar, och blir den 0 eller mindre blir resultatet 0 (underspill).* DROUND *avrundar halva uppåt räknat på beloppet (bit 7 i skyddsbyten E').* DOVFL *och en exponent som slår runt till 0 ger +största talet med CY; tecknet skrivs över som i* FOVFL. DPACK *lägger tecknet i ettans plats och ersätter de två talen på stacken med resultatet; CY = spill.*

```
55BC  E1        DNORM:POP   HL              H = tecken, L = exponent
55BD  04              INC   B
55BE  05              DEC   B
55BF  20 24           JR    NZ,L55E5        första byten inte 0
55C1  D9              EXX
```

```
55C2  7B                LD   A,E
55C3  B2                OR   D
55C4  B1                OR   C
55C5  B0                OR   B
55C6  D9                EXX
55C7  B1                OR   C
55C8  B2                OR   D
55C9  B3                OR   E
55CA  28 5F             JR   Z,DZERO          alla bytes 0: resultatet 0
55CC  7D                LD   A,L
55CD  D6 09       L55CD: SUB  $09              exponenten - 8 <= 0: 0
55CF  38 5A             JR   C,DZERO
55D1  3C                INC  A
55D2  F5                PUSH AF
55D3  D9                EXX                   en byte vänster
55D4  78                LD   A,B
55D5  41                LD   B,C
55D6  4A                LD   C,D
55D7  53                LD   D,E
55D8  1E 00             LD   E,$00
55DA  D9                EXX
55DB  41                LD   B,C
55DC  4A                LD   C,D
55DD  53                LD   D,E
55DE  5F                LD   E,A
55DF  F1                POP  AF
55E0  04                INC  B
55E1  05                DEC  B
55E2  28 E9             JR   Z,L55CD
55E4  6F                LD   L,A
55E5  FA 00 56    L55E5: JP   M,DROUND         bit 7 satt: avrunda
55E8  2D          L55E8: DEC  L                exponenten - 1, 0: underspill
55E9  28 40             JR   Z,DZERO
55EB  D9                EXX                   ett steg vänster
55EC  CB 23             SLA  E
55EE  CB 12             RL   D
55F0  CB 11             RL   C
55F2  CB 10             RL   B
55F4  D9                EXX
55F5  CB 13             RL   E
55F7  CB 12             RL   D
55F9  CB 11             RL   C
55FB  CB 10             RL   B
55FD  F2 E8 55          JP   P,L55E8
5600  D9          DROUND: EXX
5601  7B                LD   A,E
5602  17                RLA                   bit 7 i skyddsbyten
5603  30 1B             JR   NC,L5620
5605  14                INC  D                1 till mantissan
5606  20 18             JR   NZ,L5620
5608  0C                INC  C
5609  20 15             JR   NZ,L5620
560B  04                INC  B
560C  20 12             JR   NZ,L5620
560E  D9                EXX
560F  1C                INC  E
5610  20 28             JR   NZ,DPACK
5612  14                INC  D
5613  20 25             JR   NZ,DPACK
5615  0C                INC  C
5616  20 22             JR   NZ,DPACK
5618  04                INC  B
5619  20 1F             JR   NZ,DPACK
561B  06 80             LD   B,$80            minnessiffra: 0,1000...
561D  2C                INC  L                och exponenten + 1
561E  18 1A             JR   DPACK

5620  D9          L5620: EXX
5621  18 17             JR   DPACK
```

```
5623  60          DOVFL:  LD   H,B            tecknet till H (skrivs över)
5624  3E FF       L5624:  LD   A,$FF
5626  E1                  POP  HL             CY in i den sparade F
5627  2C                  INC  L
5628  E5                  PUSH HL
5629  18 01               JR   L562C

562B  AF          DZERO:  XOR  A
562C  47          L562C:  LD   B,A            B = $FF eller 0 ...
562D  4F                  LD   C,A
562E  57                  LD   D,A
562F  5F                  LD   E,A
5630  D9                  EXX
5631  47                  LD   B,A
5632  4F                  LD   C,A
5633  57                  LD   D,A
5634  5F                  LD   E,A
5635  D9                  EXX
5636  6F                  LD   L,A            ... L = exponenten
5637  A7                  AND  A
5638  28 0A               JR   Z,L5644
563A  2C          DPACK:  INC  L
563B  2D                  DEC  L
563C  28 E6               JR   Z,L5624        exponent 255 + 1: spill
563E  7C                  LD   A,H
563F  3C                  INC  A
5640  0F                  RRCA                tecknet i ettans plats
5641  E6 80               AND  $80
5643  A8                  XOR  B
5644  42          L5644:  LD   B,D
5645  57                  LD   D,A
5646  7B                  LD   A,E
5647  5D                  LD   E,L            E = exponent, D = tecken ...
5648  E1                  POP  HL
5649  E3                  EX   (SP),HL        returadressen i HL
564A  D9                  EXX
564B  E1                  POP  HL             HL' = sparad A och F
564C  5F                  LD   E,A
564D  7D                  LD   A,L
564E  6C                  LD   L,H
564F  26 00               LD   H,$00
5651  39                  ADD  HL,SP
5652  F9                  LD   SP,HL          de två talen bort
5653  F5                  PUSH AF
5654  7A                  LD   A,D
5655  50                  LD   D,B
5656  47                  LD   B,A
5657  F1                  POP  AF
5658  C5                  PUSH BC
5659  D5                  PUSH DE
565A  D9                  EXX
565B  C5                  PUSH BC
565C  D5                  PUSH DE
565D  E5                  PUSH HL             resultatet och returadressen
565E  E6 01               AND  $01            CY från den sparade F
5660  C8                  RET  Z
5661  37                  SCF
5662  C9                  RET
```

DREG: *mantissan vid HL med ettan till BCDE B'C'D', skyddsbyten E' = 0* (FREG8 *läser 8 bytes, den sista skrivs över*). *In: HL vid mantissans första byte. Ut: HL + 7. A och flaggorna bevaras.*

```
5663  CD DA 4D    DREG:   CALL FREG8          8 bytes vid HL
5666  CB F8               SET  7,B            ettan in
5668  D9                  EXX
5669  1E 00               LD   E,$00          skyddsbyten E' = 0
566B  D9                  EXX
566C  C9                  RET
```

DSHR: *BCDE B'C'D'E' ett steg höger. In: CY = biten som skiftas in i B. Ut: CY = biten som föll ut ur E'. Inget annat ändras.*

```
566D  CB 18        DSHR:   RR   B              BCDE B'C'D'E' ett steg höger
566F  CB 19                RR   C
5671  CB 1A                RR   D
5673  CB 1B                RR   E
5675  D9                   EXX
5676  CB 18                RR   B
5678  CB 19                RR   C
567A  CB 1A                RR   D
567C  CB 1B                RR   E
567E  D9                   EXX
567F  C9                   RET
```

*Matematiska funktioner. Rutinerna nås via hopptabellen $0C5D (index = funktionens kod - $80 + $36): SIN $39, COS $3A, TAN $3B, ATN $3C, SQR $3D, EXP $59, LOG $5A, LOG10 $5B. Precisionen (4 eller 8 bytes) står i IY+$1E. CY tillbaka = fel 142 (FNFLT $2185). SIN/COS/TAN: y = x * 4/pi delas i heltal n (FTOIT, huggs av mot 0) och rest f; n:s låga byte är oktanten, för COS + 2. Är n udda räknas med 1 - f. Med t = tan(f*pi/8) (TANP8) blir sin = 2t/(1+t^2), cos = (1-t^2)/(1+t^2) och tan = 2t/(1-t^2) eller dess invers; oktantens bit 2 byter tecken. Fel 142 när |n| >= 65536 (|x| >= 51472) och för TAN vid pi/2 (1/0). Egenheter (se $56DD, $56BE): negativa x ger f i (1,2) och några siffror sämre, n >= 32768 tas med tecken (|x| >= 25736 ger fel värde), och x >= 2^125 blir 0 i FSCAL. In: x under returadressen (FNFLT2 har sparat DE i PRGPTR). Ut: resultatet i x:s ställe, CY = fel. Förstör alla register utom IY, även IX.*

```
5680  06 01        SIN:    LD   B,$01
5682  18 06                JR   L568A

5684  06 02        COS:    LD   B,$02
5686  18 02                JR   L568A

5688  06 03        TAN:    LD   B,$03
568A  0E 00        L568A:  LD   C,$00           C = oktanten, B = funktionen
568C  C5                   PUSH BC
568D  DD 21 00 00          LD   IX,$0000
5691  DD 39                ADD  IX,SP           (IX+0) = C, (IX+1) = B
5693  21 04 00             LD   HL,$0004
5696  39                   ADD  HL,SP
5697  CD 25 4C             CALL FLOAD           kopia av x
569A  06 02                LD   B,$02
569C  CD A8 57             CALL FSCAL           x * 4 (slår runt från 2^125)
569F  28 3F                JR   Z,L56E0
56A1  21 E0 57             LD   HL,$57E0
56A4  CD 25 4C             CALL FLOAD
56A7  CD 5F 50             CALL FDIV            y = x * 4/pi
56AA  D1                   POP  DE
56AB  D5                   PUSH DE
56AC  3E 80                LD   A,$80
56AE  BB                   CP   E
56AF  30 2F                JR   NC,L56E0        |y| < 1: ingen reduktion
56B1  CD 21 4C             CALL FDUP
56B4  CD F7 4C             CALL FTOIT           n = y huggs av mot 0
56B7  38 0D                JR   C,L56C6         |n| >= 65536: fel 142
56B9  E1                   POP  HL
56BA  E5                   PUSH HL
56BB  DD 75 00             LD   (IX+$00),L      n:s låga byte = oktanten
56BE  CD C0 4C             CALL ITOFS           n med tecken (fel från 32768) (fel)
56C1  CD 39 4F             CALL FSUB            f = y - n
56C4  18 08                JR   L56CE

56C6  D1           L56C6:  POP  DE
56C7  D1                   POP  DE
56C8  1E 00                LD   E,$00           talet 0, CY står kvar
56CA  D5                   PUSH DE
56CB  C3 88 57             JP   TRGEND

56CE  DD 7E 00     L56CE:  LD   A,(IX+$00)
56D1  1F                   RRA
56D2  30 0C                JR   NC,L56E0        n jämnt
56D4  21 F2 4D             LD   HL,$4DF2
56D7  CD 25 4C             CALL FLOAD
56DA  CD 52 4C             CALL FSWAP
```

```
56DD  CD 39 4F           CALL FSUB            1 - f (f < 0 ger 1..2) (gränsfall)
56E0  DD 7E 01     L56E0: LD   A,(IX+$01)
56E3  3D                  DEC  A
56E4  28 4D               JR   Z,L5733         SIN
56E6  3D                  DEC  A
56E7  28 42               JR   Z,L572B         COS
56E9  CD B2 57            CALL TANP8           TAN: t = tan(f*pi/8)
56EC  CD 21 4C            CALL FDUP
56EF  CD 21 4C            CALL FDUP
56F2  CD EA 4F            CALL FMUL
56F5  21 F2 4D            LD   HL,$4DF2
56F8  CD 25 4C            CALL FLOAD
56FB  CD 39 4F            CALL FSUB
56FE  CD 5F 50            CALL FDIV            t / (t^2 - 1)
5701  06 01               LD   B,$01
5703  CD A8 57            CALL FSCAL           * 2 = -tan(f*pi/4)
5706  D1                  POP  DE
5707  3E 80               LD   A,$80
5709  AA                  XOR  D
570A  57                  LD   D,A
570B  D5                  PUSH DE              tecknet bytt
570C  FD 6E 1E            LD   L,(IY+$1E)
570F  26 00               LD   H,$00
5711  39                  ADD  HL,SP
5712  7E                  LD   A,(HL)
5713  CB 16               RL   (HL)            oktanten ett steg vänster
5715  E6 03               AND  $03
5717  28 6F               JR   Z,TRGEND        oktant 0 eller 3: klart
5719  EE 03               XOR  $03
571B  28 6B               JR   Z,TRGEND
571D  21 F2 4D            LD   HL,$4DF2
5720  CD 25 4C            CALL FLOAD
5723  CD 52 4C            CALL FSWAP
5726  CD 5F 50            CALL FDIV            annars 1 / tan; 1/0 ger CY
5729  18 5D               JR   TRGEND

572B  DD 7E 00     L572B: LD   A,(IX+$00)
572E  C6 02               ADD  A,$02           COS: oktanten + 2
5730  DD 77 00            LD   (IX+$00),A
5733  DD 7E 00     L5733: LD   A,(IX+$00)
5736  E6 03               AND  $03
5738  28 30               JR   Z,L576A         oktant 0 eller 3: sin-formeln
573A  EE 03               XOR  $03
573C  28 2C               JR   Z,L576A
573E  CD B2 57            CALL TANP8
5741  CD 21 4C            CALL FDUP
5744  CD EA 4F            CALL FMUL
5747  CD 21 4C            CALL FDUP
574A  21 F2 4D            LD   HL,$4DF2
574D  CD 25 4C            CALL FLOAD
5750  CD 39 4F            CALL FSUB
5753  CD 52 4C            CALL FSWAP
5756  21 F2 4D            LD   HL,$4DF2
5759  CD 25 4C            CALL FLOAD
575C  CD 41 4F            CALL FADD
575F  CD 5F 50            CALL FDIV            (t^2 - 1) / (t^2 + 1)
5762  D1                  POP  DE
5763  3E 80               LD   A,$80
5765  AA                  XOR  D
5766  57                  LD   D,A
5767  D5                  PUSH DE              tecknet bytt: cos
5768  18 1E               JR   TRGEND

576A  CD B2 57     L576A: CALL TANP8
576D  CD 21 4C            CALL FDUP
5770  CD 21 4C            CALL FDUP
5773  CD EA 4F            CALL FMUL
5776  21 F2 4D            LD   HL,$4DF2
5779  CD 25 4C            CALL FLOAD
```

```
577C  CD 41 4F            CALL FADD
577F  CD 5F 50            CALL FDIV            t / (t^2 + 1)
5782  06 01               LD   B,$01
5784  CD A8 57            CALL FSCAL           * 2 = sin(f*pi/4)
5787  A7                  AND  A
5788  CB 10       TRGEND: RL   B               CY (spill) in i B
578A  FD 6E 1E            LD   L,(IY+$1E)
578D  26 00               LD   H,$00
578F  39                  ADD  HL,SP
5790  3E 04               LD   A,$04
5792  A6                  AND  (HL)            oktantens bit 2: byt tecken
5793  70                  LD   (HL),B          B i C:s plats på stacken
5794  28 06               JR   Z,L579C
5796  D1                  POP  DE
5797  3E 80               LD   A,$80
5799  AA                  XOR  D
579A  57                  LD   D,A
579B  D5                  PUSH DE
579C  11 04 00    L579C:  LD   DE,$0004
579F  19                  ADD  HL,DE
57A0  EB                  EX   DE,HL
57A1  CD 3F 4C            CALL FPOP            resultatet över x
57A4  E1                  POP  HL
57A5  CB 1D               RR   L               CY = spill (fel 142)
57A7  C9                  RET
```

FSCAL*: översta talet * 2^B (exponenten + B, inte om talet är 0; ingen kontroll av spill). Z: talet är 0 eller exponenten slog runt till 0. In: B, talet under returadressen. Förstör A, DE och HL.*

```
57A8  E1          FSCAL:  POP  HL
57A9  D1                  POP  DE
57AA  7B                  LD   A,E
57AB  A7                  AND  A
57AC  28 02               JR   Z,L57B0
57AE  80                  ADD  A,B             ingen kontroll av spill
57AF  5F                  LD   E,A
57B0  D5          L57B0:  PUSH DE
57B1  E9                  JP   (HL)
```

TANP8*: t = tan(f*pi/8) för talet f under returadressen, som f * P(f^2) / Q(f^2); f = 0 ger 0 direkt. Ut: t i f:s ställe. Förstör alla register utom IY, även IX (SERIES).*

```
57B2  21 02 00    TANP8:  LD   HL,$0002
57B5  39                  ADD  HL,SP
57B6  7E                  LD   A,(HL)
57B7  A7                  AND  A
57B8  C8                  RET  Z               f = 0: t = 0
57B9  CD 25 4C            CALL FLOAD
57BC  CD 21 4C            CALL FDUP
57BF  CD 21 4C            CALL FDUP
57C2  CD EA 4F            CALL FMUL
57C5  CD 21 4C            CALL FDUP
57C8  21 06 58            LD   HL,$5806
57CB  CD 6B 5C            CALL SERIES          P(f^2)
57CE  CD 52 4C            CALL FSWAP
57D1  21 E8 57            LD   HL,$57E8
57D4  CD 6B 5C            CALL SERIES          Q(f^2)
57D7  CD 5F 50            CALL FDIV
57DA  CD EA 4F            CALL FMUL            t = f * P/Q
57DD  C3 CD 4D            JP   FNIP
```

*pi, sedan TANP8:s tabeller för SERIES: nämnaren Q ($57E8) och täljaren P ($5806), båda i f^2; tan(f*pi/8) = f * P/Q. Q har underförstådd ledande koefficient 1.*

```
57E0  82 49 0F DA… FPI:   DEFB $82,$49,$0F,$DA,$A2,$21,$68,$C2  3.141592653589793
57E8  81                  DEFB $81
57E9  85 81 86 E2         DEFB $85,$81,$86,$E2  -16.19086
57ED  83                  DEFB $83
```

```
57EE  8B A9 BF A7…        DEFB $8B,$A9,$BF,$A7,$67,$75,$D5,$6B   -1357.989185075897
57F6  92 41 5C 09…        DEFB $92,$41,$5C,$09,$0F,$E0,$72,$B6   198000.1415940399
57FE  96 AC 64 16…        DEFB $96,$AC,$64,$16,$47,$58,$FA,$8A   -2824453.569675364
5806  02                  DEFB $02
5807  7D 06 EA D2         DEFB $7D,$06,$EA,$D2   0.06587757
580B  83 CB 75 DD         DEFB $83,$CB,$75,$DD   -6.358138
580F  03                  DEFB $03
5810  86 D5 82 5D…       DEFB $86,$D5,$82,$5D,$DC,$89,$94,$5E   -53.37731117810935
5818  8F 22 05 F8…       DEFB $8F,$22,$05,$F8,$F5,$47,$7B,$C1   20738.98624633203
5820  95 87 65 42…       DEFB $95,$87,$65,$42,$95,$BB,$D9,$3A   -1109160.323112199
```

*LOG: x <= 0 ger CY (fel 142). En tvåpotens 2^k ger k * ln 2 (x = 1 ger 0). Annars x = m * 2^e (1/2 <= m < 1) och ln x = (e - 1/2) * ln 2 + ln(m * sqr 2), där ln(m * sqr 2) = z * P(z^2)/Q(z^2) med z = (m * sqr 2 - 1)/(m * sqr 2 + 1). Nära 1 tar termerna nästan ut varandra, så felet är absolut snarare än relativt (se $5887). In/Ut som för SIN: x under returadressen, resultatet i dess ställe, CY = fel.*

```
5828  DD 21 02 00  LOG:    LD   IX,$0002
582C  DD 39                ADD  IX,SP
582E  DD 7E 00             LD   A,(IX+$00)
5831  A7                   AND  A
5832  28 06                JR   Z,L583A
5834  DD CB 01 7E          BIT  7,(IX+$01)
5838  28 02                JR   Z,L583C
583A  37           L583A:  SCF                  x <= 0: fel 142
583B  C9                   RET

583C  DD E5        L583C:  PUSH IX
583E  E1                   POP  HL
583F  FD 46 1E             LD   B,(IY+$1E)
5842  05                   DEC  B
5843  AF                   XOR  A
5844  23           L5844:  INC  HL
5845  BE                   CP   (HL)
5846  20 24                JR   NZ,L586C          inte en tvåpotens
5848  10 FA                DJNZ L5844
584A  DD 7E 00             LD   A,(IX+$00)
584D  D6 81                SUB  $81               x = 2^k: k = exponent - $81
584F  20 04                JR   NZ,L5855
5851  DD 77 00             LD   (IX+$00),A        x = 1: 0
5854  C9                   RET

5855  6F           L5855:  LD   L,A
5856  26 00                LD   H,$00
5858  07                   RLCA                   k med tecken till HL
5859  30 01                JR   NC,L585C
585B  25                   DEC  H
585C  E5           L585C:  PUSH HL
585D  CD C0 4C             CALL ITOFS
5860  21 27 59             LD   HL,$5927
5863  CD 25 4C             CALL FLOAD
5866  CD EA 4F             CALL FMUL              k * ln 2
5869  C3 CD 4D             JP   FNIP

586C  DD 7E 00     L586C:  LD   A,(IX+$00)        e = exponent - $80
586F  D6 80                SUB  $80
5871  6F                   LD   L,A
5872  26 00                LD   H,$00
5874  07                   RLCA
5875  30 01                JR   NC,L5878
5877  25                   DEC  H
5878  29           L5878:  ADD  HL,HL
5879  2B                   DEC  HL                2e - 1
587A  E5                   PUSH HL
587B  CD C0 4C             CALL ITOFS
587E  E1                   POP  HL
587F  2D                   DEC  L                 exponenten - 1: e - 1/2
5880  E5                   PUSH HL
5881  21 27 59             LD   HL,$5927
5884  CD 25 4C             CALL FLOAD
5887  CD EA 4F             CALL FMUL              (e - 1/2) * ln 2 (gränsfall)
```

```
588A  DD E5              PUSH IX
588C  E1                 POP  HL
588D  CD 25 4C           CALL FLOAD
5890  E1                 POP  HL
5891  2E 80              LD   L,$80          m: 1/2 <= m < 1
5893  E5                 PUSH HL
5894  21 2F 59           LD   HL,$592F
5897  CD 25 4C           CALL FLOAD
589A  CD EA 4F           CALL FMUL           m * sqr 2
589D  CD 21 4C           CALL FDUP
58A0  21 F2 4D           LD   HL,$4DF2
58A3  CD 25 4C           CALL FLOAD
58A6  CD 39 4F           CALL FSUB
58A9  CD 52 4C           CALL FSWAP
58AC  21 F2 4D           LD   HL,$4DF2
58AF  CD 25 4C           CALL FLOAD
58B2  CD 41 4F           CALL FADD
58B5  CD 5F 50           CALL FDIV           z = (m sqr2 - 1)/(m sqr2 + 1)
58B8  CD 21 4C           CALL FDUP
58BB  CD 21 4C           CALL FDUP
58BE  CD EA 4F           CALL FMUL
58C1  CD 21 4C           CALL FDUP
58C4  21 DF 58           LD   HL,$58DF
58C7  CD 6B 5C           CALL SERIES         P(z^2)
58CA  CD 52 4C           CALL FSWAP
58CD  21 09 59           LD   HL,$5909
58D0  CD 6B 5C           CALL SERIES         Q(z^2)
58D3  CD 5F 50           CALL FDIV
58D6  CD EA 4F           CALL FMUL           z * P/Q = ln(m * sqr 2)
58D9  CD 41 4F           CALL FADD           + (e - 1/2) * ln 2
58DC  C3 CD 4D           JP   FNIP
```

Tabellerna för LOG*: täljaren P (*$58DF*) och nämnaren Q (*$5909*, ledande koefficient 1), sedan ln 2 och sqr 2.*

```
58DF  02                 DEFB $02
58E0  80 65 42 26        DEFB $80,$65,$42,$26  0.8955406
58E4  82 D4 11 4E        DEFB $82,$D4,$11,$4E  -3.313556
58E8  04                 DEFB $04
58E9  7F 57 98 C3…       DEFB $7F,$57,$98,$C3,$90,$47,$98,$65  0.4210873712179797
58F1  84 9A 33 FB…       DEFB $84,$9A,$33,$FB,$68,$92,$40,$AD  -9.637690933686865
58F9  85 77 A8 89…       DEFB $85,$77,$A8,$89,$23,$89,$99,$22  30.95729282153765
5901  85 C0 1C 81…       DEFB $85,$C0,$1C,$81,$04,$6C,$10,$8C  -24.01391795592105
5909  81                 DEFB $81
590A  81 D4 11 4D        DEFB $81,$D4,$11,$4D  -1.656778
590E  83                 DEFB $83
590F  84 8E 93 E7…       DEFB $84,$8E,$93,$E7,$0F,$61,$55,$57  -8.911109027937831
5917  85 1B D9 04…       DEFB $85,$1B,$D9,$04,$BD,$2C,$24,$6B  19.48096607008897
591F  84 C0 1C 81…       DEFB $84,$C0,$1C,$81,$04,$6C,$10,$8C  -12.00695897796053
5927  80 31 72 17… FLN2:  DEFB $80,$31,$72,$17,$F7,$D1,$CF,$79  0.6931471805599453
592F  81 35 04 F3… FSQR2: DEFB $81,$35,$04,$F3,$33,$F9,$DE,$63  1.414213562373095
```

EXP*: y = |x| / ln 2 = n + f (n heltal, 0 <= f < 1). r = 2^-g = (Q - gP)/(Q + gP) med g = f, eller g = f/2 och sedan r * r när f >= 1/2. 2^n görs som talet 1 med exponenten n +* $81*; resultatet är 2^n / r för x > 0 och r / 2^n för x < 0. n >= 127 (|x| >= 88.03) ger fel 142 för x > 0 och 0 för x < 0 (fast tal ned till 2.9E-39 går att lagra). Felet växer med |x| (y räknas med samma längd). In/Ut som för* SIN*.*

```
5937  21 02 00     EXP:   LD   HL,$0002
593A  39                  ADD  HL,SP
593B  E5                  PUSH HL
593C  DD E1               POP  IX             IX = x
593E  CD 25 4C            CALL FLOAD
5941  21 27 59           LD   HL,$5927
5944  CD 25 4C           CALL FLOAD
5947  CD 5F 50           CALL FDIV           y = x / ln 2
594A  D1                  POP  DE
594B  CB BA               RES  7,D            |y|
594D  D5                  PUSH DE
594E  DD 36 00 81         LD   (IX+$00),$81   n = 0: 2^n = 1
```

```
5952  3E 80              LD   A,$80
5954  BB                 CP   E
5955  30 1C              JR   NC,L5973          |y| < 1
5957  CD 21 4C           CALL FDUP
595A  CD F7 4C           CALL FTOIT             n = |y| huggs av
595D  38 29              JR   C,L5988
595F  E1                 POP  HL
5960  E5                 PUSH HL
5961  7D                 LD   A,L
5962  C6 81              ADD  A,$81
5964  38 22              JR   C,L5988           n >= 127: utanför
5966  DD 77 00           LD   (IX+$00),A        exponenten för 2^n
5969  24                 INC  H
596A  25                 DEC  H
596B  20 1B              JR   NZ,L5988          n >= 256
596D  CD C0 4C           CALL ITOFS
5970  CD 39 4F           CALL FSUB              f = |y| - n
5973  DD CB 01 B6  L5973: RES  6,(IX+$01)       bit 6: kvadrera till sist
5977  D1                 POP  DE
5978  7B                 LD   A,E
5979  A7                 AND  A
597A  28 1B              JR   Z,L5997           f = 0: 2^n direkt
597C  FE 80              CP   $80
597E  20 05              JR   NZ,L5985
5980  DD CB 01 F6        SET  6,(IX+$01)        f >= 1/2: f/2
5984  1D                 DEC  E
5985  D5           L5985: PUSH DE
5986  18 1C              JR   L59A4

5988  E1           L5988: POP  HL
5989  CD C3 4D           CALL FDROP
598C  AF                 XOR  A
598D  DD 77 00           LD   (IX+$00),A        x = 0 ...
5990  DD CB 01 7E        BIT  7,(IX+$01)
5994  C0                 RET  NZ                x < 0: 0 utan fel
5995  37                 SCF                    x > 0: spill (fel 142)
5996  C9                 RET

5997  D5           L5997: PUSH DE
5998  CD C3 4D           CALL FDROP
599B  21 F2 4D           LD   HL,$4DF2
599E  CD 25 4C           CALL FLOAD             r = 1
59A1  C3 10 5A           JP   L5A10

59A4  CD 21 4C     L59A4: CALL FDUP
59A7  CD 21 4C           CALL FDUP
59AA  CD EA 4F           CALL FMUL              g^2
59AD  CD 52 4C           CALL FSWAP
59B0  FD 6E 1E           LD   L,(IY+$1E)
59B3  26 00              LD   H,$00
59B5  39                 ADD  HL,SP
59B6  CD 25 4C           CALL FLOAD
59B9  21 3E 5A           LD   HL,$5A3E
59BC  CD 6B 5C           CALL SERIES            P(g^2)
59BF  CD EA 4F           CALL FMUL              g * P
59C2  CD 52 4C           CALL FSWAP
59C5  21 28 5A           LD   HL,$5A28
59C8  CD 6B 5C           CALL SERIES            Q(g^2)
59CB  FD 6E 1E           LD   L,(IY+$1E)
59CE  26 00              LD   H,$00
59D0  29                 ADD  HL,HL
59D1  39                 ADD  HL,SP
59D2  23                 INC  HL
59D3  23                 INC  HL
59D4  E5                 PUSH HL
59D5  DD E1              POP  IX                IX = x igen
59D7  CD 21 4C           CALL FDUP
59DA  FD 6E 1E           LD   L,(IY+$1E)
59DD  26 00              LD   H,$00
59DF  29                 ADD  HL,HL
```

```
59E0  39                  ADD  HL,SP
59E1  CD 25 4C            CALL FLOAD
59E4  CD 39 4F            CALL FSUB            Q - gP
59E7  CD 52 4C            CALL FSWAP
59EA  FD 6E 1E            LD   L,(IY+$1E)
59ED  26 00               LD   H,$00
59EF  29                  ADD  HL,HL
59F0  39                  ADD  HL,SP
59F1  CD 25 4C            CALL FLOAD
59F4  CD 41 4F            CALL FADD            Q + gP
59F7  CD 5F 50            CALL FDIV            r = (Q - gP)/(Q + gP)
59FA  FD 6E 1E            LD   L,(IY+$1E)
59FD  26 00               LD   H,$00
59FF  39                  ADD  HL,SP
5A00  EB                  EX   DE,HL
5A01  CD 3F 4C            CALL FPOP            r över gP
5A04  DD CB 01 76         BIT  6,(IX+$01)
5A08  28 06               JR   Z,L5A10
5A0A  CD 21 4C            CALL FDUP
5A0D  CD EA 4F            CALL FMUL            r * r
5A10  21 F2 4D     L5A10: LD   HL,$4DF2
5A13  CD 25 4C            CALL FLOAD
5A16  D1                  POP  DE
5A17  DD 5E 00            LD   E,(IX+$00)      talet 1 med exponenten: 2^n
5A1A  D5                  PUSH DE
5A1B  DD CB 01 7E         BIT  7,(IX+$01)
5A1F  CC 52 4C            CALL Z,FSWAP         x > 0: 2^n / r, annars r / 2^n
5A22  CD 5F 50            CALL FDIV
5A25  C3 CD 4D            JP   FNIP
```

Tabellerna för EXP: nämnaren Q ($5A28, ledande koefficient 1) och täljaren P ($5A3E), i g^2.

```
5A28  81                  DEFB $81
5A29  85 26 8D 28         DEFB $85,$26,$8D,$28   20.81892
5A2D  82                  DEFB $82
5A2E  88 69 2F 28…        DEFB $88,$69,$2F,$28,$7A,$E8,$95,$42   233.1842114274816
5A36  8D 08 81 B1…        DEFB $8D,$08,$81,$B1,$7C,$3A,$65,$2A   4368.211662727558
5A3E  02                  DEFB $02
5A3F  7C 6C 4C 6D         DEFB $7C,$6C,$4C,$6D   0.05769007
5A43  83 66 E3 A6         DEFB $83,$66,$E3,$A6   7.215289
5A47  03                  DEFB $03
5A48  7B 3D 2E 42…        DEFB $7B,$3D,$2E,$42,$AB,$70,$BD,$A9   0.023093347753750 23
5A50  85 21 9D D4…        DEFB $85,$21,$9D,$D4,$98,$9F,$60,$DB   20.20206565128693
5A58  8B 3D 3D 04…        DEFB $8B,$3D,$3D,$04,$7F,$73,$4D,$BD   1513.906799054339
```

LOG10: ln x / ln 10; x <= 0 ger CY (fel 142). In/Ut som för SIN.

```
5A60  21 02 00     LOG10: LD   HL,$0002
5A63  39                  ADD  HL,SP
5A64  CD 25 4C            CALL FLOAD           kopia av x
5A67  CD 28 58            CALL LOG
5A6A  38 0C               JR   C,L5A78         x <= 0: fel 142
5A6C  21 81 5A            LD   HL,$5A81
5A6F  CD 25 4C            CALL FLOAD
5A72  CD 5F 50            CALL FDIV            ln x / ln 10
5A75  C3 CD 4D            JP   FNIP

5A78  FD 6E 1E     L5A78: LD   L,(IY+$1E)
5A7B  26 00               LD   H,$00
5A7D  39                  ADD  HL,SP
5A7E  F9                  LD   SP,HL           kopian bort
5A7F  37                  SCF
5A80  C9                  RET

5A81  82 13 5D 8D… FLN10: DEFB $82,$13,$5D,$8D,$DD,$AA,$A8,$AB   2.302585092994046
```

SQR: kvadratroten bit för bit (som skriftlig division) i en arbetsyta under x: (IX+0) bytes kvar, (IX+1) provbiten, roten R från IX+3. Resten M är x:s mantissa (halverad när exponenten är udda). Roten huggs av, så den blir aldrig för stor och kan vara upp till två enheter i sista biten för liten (SQR(2) = 1.414213419 med 4 bytes). x = 0 ger 0, x < 0 CY (fel 142). Exponenten halveras

med tecken, och exponent `$FF` *(x >= 2^126, ca 8.5E37) slår runt* (`SQR(9E37)` *= 2.78793E-20; se* `$5A9E`*). In/Ut som för* `SIN`*, men arbetsytan tas under stacken och IX pekar på den.*

```
5A89  E1           SQR:    POP   HL
5A8A  D1                   POP   DE
5A8B  D5                   PUSH  DE
5A8C  7B                   LD    A,E
5A8D  A7                   AND   A
5A8E  CA 22 5B             JP    Z,L5B22          x = 0: 0
5A91  CB 7A                BIT   7,D
5A93  37                   SCF
5A94  C2 22 5B             JP    NZ,L5B22         x < 0: CY (fel 142)
5A97  D1                   POP   DE
5A98  CB FA                SET   7,D              ettan utskriven i talet
5A9A  D5                   PUSH  DE
5A9B  7B                   LD    A,E
5A9C  EE 80                XOR   $80
5A9E  3C                   INC   A                e + 1 (e = $7F slår runt) (fel)
5A9F  CB 2F                SRA   A                / 2; CY = e jämnt
5AA1  CB 12                RL    D                CY till bit 0 (A = D)
5AA3  EE 80                XOR   $80
5AA5  5F                   LD    E,A              E = rotens exponent
5AA6  7A                   LD    A,D
5AA7  D9                   EXX                    E och HL (returen) undan
5AA8  FD 4E 1E             LD    C,(IY+$1E)
5AAB  21 00 00             LD    HL,$0000
5AAE  44                   LD    B,H
5AAF  39                   ADD   HL,SP
5AB0  ED 42                SBC   HL,BC
5AB2  2B                   DEC   HL
5AB3  2B                   DEC   HL
5AB4  F9                   LD    SP,HL            arbetsyta n+2 bytes under x
5AB5  E5                   PUSH  HL
5AB6  DD E1                POP   IX
5AB8  71                   LD    (HL),C
5AB9  35                   DEC   (HL)             (IX+0) = bytes kvar
5ABA  23                   INC   HL
5ABB  36 40                LD    (HL),$40         (IX+1) = provbiten
5ABD  23                   INC   HL
5ABE  23                   INC   HL
5ABF  5D                   LD    E,L
5AC0  54                   LD    D,H
5AC1  41                   LD    B,C
5AC2  36 00        L5AC2:  LD    (HL),$00         R = 0 (även x:s exponent)
5AC4  23                   INC   HL
5AC5  10 FB                DJNZ  L5AC2
5AC7  E5                   PUSH  HL
5AC8  1F                   RRA
5AC9  38 09                JR    C,L5AD4          e udda: mantissan / 2
5ACB  41                   LD    B,C
5ACC  05                   DEC   B
5ACD  CB 1E        L5ACD:  RR    (HL)
5ACF  23                   INC   HL
5AD0  10 FB                DJNZ  L5ACD
5AD2  E1                   POP   HL
5AD3  E5                   PUSH  HL
5AD4  EB           L5AD4:  EX    DE,HL
5AD5  36 40                LD    (HL),$40         första provbiten
5AD7  E3                   EX    (SP),HL          HL = M, (SP) = R:s byte
5AD8  E5           L5AD8:  PUSH  HL
5AD9  D5                   PUSH  DE
5ADA  5D                   LD    E,L
5ADB  54                   LD    D,H
5ADC  AF                   XOR   A
5ADD  ED 42                SBC   HL,BC
5ADF  41                   LD    B,C
5AE0  05                   DEC   B
5AE1  1A           L5AE1:  LD    A,(DE)
5AE2  BE                   CP    (HL)             jämför M med R
```

```
5AE3  23              INC  HL
5AE4  13              INC  DE
5AE5  20 02           JR   NZ,L5AE9
5AE7  10 F8           DJNZ L5AE1
5AE9  D1        L5AE9:POP  DE
5AEA  E1              POP  HL
5AEB  E3              EX   (SP),HL
5AEC  28 13           JR   Z,L5B01       lika: klart
5AEE  38 33           JR   C,L5B23       M < R: biten 0
5AF0  79        L5AF0:LD   A,C
5AF1  DD 96 00        SUB  (IX+$00)
5AF4  47              LD   B,A
5AF5  E5              PUSH HL
5AF6  D5              PUSH DE
5AF7  1A        L5AF7:LD   A,(DE)
5AF8  9E              SBC  A,(HL)        M - R
5AF9  12              LD   (DE),A
5AFA  2B              DEC  HL
5AFB  1B              DEC  DE
5AFC  10 F9           DJNZ L5AF7
5AFE  D1              POP  DE
5AFF  E1              POP  HL
5B00  37              SCF                biten 1
5B01  08        L5B01:EX   AF,AF'
5B02  DD 46 01        LD   B,(IX+$01)
5B05  7E              LD   A,(HL)
5B06  80              ADD  A,B           R + provbiten
5B07  77              LD   (HL),A
5B08  30 03           JR   NC,L5B0D
5B0A  2B              DEC  HL
5B0B  34              INC  (HL)
5B0C  23              INC  HL
5B0D  08        L5B0D:EX   AF,AF'
5B0E  38 19           JR   C,L5B29
5B10  21 04 00  L5B10:LD   HL,$0004      klart: R över x
5B13  44              LD   B,H
5B14  39              ADD  HL,SP
5B15  5D              LD   E,L
5B16  54              LD   D,H
5B17  09              ADD  HL,BC
5B18  EB              EX   DE,HL
5B19  ED B0           LDIR
5B1B  F9              LD   SP,HL
5B1C  D9              EXX
5B1D  C1              POP  BC
5B1E  4B              LD   C,E           rotens exponent
5B1F  CB B8           RES  7,B           positivt
5B21  C5              PUSH BC
5B22  E9        L5B22:JP   (HL)

5B23  DD 46 01  L5B23:LD   B,(IX+$01)
5B26  7E              LD   A,(HL)
5B27  90              SUB  B             R - provbiten
5B28  77              LD   (HL),A
5B29  CB 08     L5B29:RRC  B             provbiten ett steg höger
5B2B  30 09           JR   NC,L5B36
5B2D  DD 35 00        DEC  (IX+$00)      nästa byte
5B30  28 DE           JR   Z,L5B10       alla bytes klara
5B32  23              INC  HL
5B33  13              INC  DE
5B34  36 00           LD   (HL),$00
5B36  7E        L5B36:LD   A,(HL)
5B37  80              ADD  A,B
5B38  77              LD   (HL),A
5B39  DD 70 01        LD   (IX+$01),B    ny provbit i R
5B3C  E3              EX   (SP),HL
5B3D  E5              PUSH HL
5B3E  0D              DEC  C
5B3F  06 00           LD   B,$00
```

```
5B41  09                ADD  HL,BC
5B42  41                LD   B,C
5B43  0C                INC  C
5B44  AF                XOR  A
5B45  2B          L5B45: DEC  HL
5B46  CB 16             RL   (HL)          M ett steg vänster
5B48  10 FB             DJNZ L5B45
5B4A  E1                POP  HL
5B4B  30 8B             JR   NC,L5AD8      ingen bit ut: jämför
5B4D  E3                EX   (SP),HL
5B4E  18 A0             JR   L5AF0         bit ut: M > R
```

ATN: *arcus tangens. Tecknet sparas i x:s bit 7 och flaggor i bit 6-5. |x| >= 1 ger x = 1/x (bit 6), x > tan(pi/8) ger x = (x-1)/(x+1) (bit 5); atn x = x * P(x^2)/Q(x^2); sedan + pi/4 (bit 5), pi/2 - (bit 6) och tecknet. In/Ut som för* SIN.

```
5B50  21 02 00    ATN:  LD   HL,$0002
5B53  39                ADD  HL,SP
5B54  E5                PUSH HL
5B55  DD E1             POP  IX
5B57  CD 25 4C          CALL FLOAD         kopia av x
5B5A  DD 7E 01          LD   A,(IX+$01)
5B5D  E6 80             AND  $80
5B5F  DD 77 01          LD   (IX+$01),A    bara tecknet kvar i x
5B62  D1                POP  DE
5B63  CB BA             RES  7,D           |x|
5B65  D5                PUSH DE
5B66  7B                LD   A,E
5B67  FE 81             CP   $81
5B69  38 10             JR   C,L5B7B       |x| < 1
5B6B  DD CB 01 F6       SET  6,(IX+$01)    bit 6: x = 1/x
5B6F  21 F2 4D          LD   HL,$4DF2
5B72  CD 25 4C          CALL FLOAD
5B75  CD 52 4C          CALL FSWAP
5B78  CD 5F 50          CALL FDIV
5B7B  21 00 00    L5B7B: LD   HL,$0000
5B7E  39                ADD  HL,SP
5B7F  11 63 5C          LD   DE,$5C63
5B82  CD 64 4C          CALL FCOMP2        jämför med tan(pi/8)
5B85  30 1F             JR   NC,L5BA6      x <= tan(pi/8)
5B87  DD CB 01 EE       SET  5,(IX+$01)    bit 5: (x-1)/(x+1)
5B8B  CD 21 4C          CALL FDUP
5B8E  21 F2 4D          LD   HL,$4DF2
5B91  CD 25 4C          CALL FLOAD
5B94  CD 39 4F          CALL FSUB
5B97  CD 52 4C          CALL FSWAP
5B9A  21 F2 4D          LD   HL,$4DF2
5B9D  CD 25 4C          CALL FLOAD
5BA0  CD 41 4F          CALL FADD
5BA3  CD 5F 50          CALL FDIV
5BA6  CD 21 4C    L5BA6: CALL FDUP
5BA9  CD 21 4C          CALL FDUP
5BAC  CD EA 4F          CALL FMUL
5BAF  CD 21 4C          CALL FDUP
5BB2  21 31 5C          LD   HL,$5C31
5BB5  CD 6B 5C          CALL SERIES        P(x^2)
5BB8  CD 52 4C          CALL FSWAP
5BBB  21 07 5C          LD   HL,$5C07
5BBE  CD 6B 5C          CALL SERIES        Q(x^2)
5BC1  CD 5F 50          CALL FDIV
5BC4  CD EA 4F          CALL FMUL          x * P/Q
5BC7  FD 6E 1E          LD   L,(IY+$1E)
5BCA  26 00             LD   H,$00
5BCC  39                ADD  HL,SP
5BCD  23                INC  HL
5BCE  23                INC  HL
5BCF  E5                PUSH HL
5BD0  DD E1             POP  IX            IX = x igen
5BD2  DD CB 01 6E       BIT  5,(IX+$01)
```

```
5BD6  28 0D              JR   Z,L5BE5
5BD8  21 E0 57           LD   HL,$57E0
5BDB  CD 25 4C           CALL FLOAD
5BDE  D1                 POP  DE
5BDF  1D                 DEC  E              pi/4 ...
5BE0  1D                 DEC  E
5BE1  D5                 PUSH DE
5BE2  CD 41 4F           CALL FADD           ... + atn
5BE5  DD CB 01 76  L5BE5: BIT  6,(IX+$01)
5BE9  28 0F              JR   Z,L5BFA
5BEB  CD 83 4C           CALL FCHS
5BEE  21 E0 57           LD   HL,$57E0
5BF1  CD 25 4C           CALL FLOAD
5BF4  D1                 POP  DE
5BF5  1D                 DEC  E
5BF6  D5                 PUSH DE
5BF7  CD 41 4F           CALL FADD           pi/2 - atn
5BFA  DD CB 01 7E  L5BFA: BIT  7,(IX+$01)
5BFE  CA CD 4D           JP   Z,FNIP         x >= 0: klart
5C01  CD 83 4C           CALL FCHS
5C04  C3 CD 4D           JP   FNIP
```

Tabellerna för ATN: nämnaren Q ($5C07, ledande koefficient 1) och täljaren P ($5C31), sedan sqr(2)-1 = tan(pi/8) som ATN jämför med.

```
5C07  82                 DEFB $82
5C08  84 29 6D 00        DEFB $84,$29,$6D,$00  10.58911
5C0C  84 4A 8F 4E        DEFB $84,$4A,$8F,$4E  12.65999
5C10  84                 DEFB $84
5C11  84 78 10 4A...     DEFB $84,$78,$10,$4A,$C3,$D4,$67,$4C  15.5039775514219
5C19  86 7B 57 FE...     DEFB $86,$7B,$57,$FE,$2A,$FA,$73,$4C  62.83593051103237
5C21  87 38 A6 4C...     DEFB $87,$38,$A6,$4C,$53,$7E,$7A,$0F  92.32480107230097
5C29  86 32 2A 55...     DEFB $86,$32,$2A,$55,$0C,$6A,$AC,$35  44.54134005929068
5C31  02                 DEFB $02
5C32  83 4B D0 65        DEFB $83,$4B,$D0,$65  6.369189
5C36  84 4A 8F 4E        DEFB $84,$4A,$8F,$4E  12.65999
5C3A  05                 DEFB $05
5C3B  7E 22 CA 16...     DEFB $7E,$22,$CA,$16,$7B,$44,$1B,$91  0.1589740288482307
5C43  83 55 23 76...     DEFB $83,$55,$23,$76,$9B,$4F,$50,$C2  6.660579017009262
5C4B  86 23 E0 86...     DEFB $86,$23,$E0,$86,$F5,$CC,$2B,$22  40.96926483210226
5C53  87 1A F4 93...     DEFB $87,$1A,$F4,$93,$7C,$17,$5D,$A1  77.4776877192042
5C5B  86 32 2A 55...     DEFB $86,$32,$2A,$55,$0C,$6A,$AC,$35  44.54134005929068
5C63  7F 54 13 CC... FTAN8: DEFB $7F,$54,$13,$CC,$CF,$E7,$79,$91  0.414213562373095
```

3.37 SERIES: beräkna ett polynom med Hornerschema $5C6B

HL = tabell, x ligger på stacken. Tabellen har två delar: först en räknarbyte och koefficienterna som 4-bytestal (enkel precision), sedan en räknarbyte och koefficienterna som 8-bytestal (dubbel precision). Med 8 bytes (IY+$1E bit 2 = 0) hoppas första delen över (4 * antal bytes). Räknarbytens bit 7 = underförstådd ledande koefficient 1. In: HL = tabellen, x under returadressen. Ut: polynomets värde i x:s ställe. Förstör alla register utom IY, även IX.

```
5C6B  FD CB 1E 56  SERIES: BIT  2,(IY+$1E)
5C6F  7E           L5C6F: LD   A,(HL)
5C70  23                  INC  HL
5C71  20 09               JR   NZ,L5C7C       8 bytes: 4-bytesdelen över
5C73  87                  ADD  A,A            4 * antal (bit 7 försvinner)
5C74  87                  ADD  A,A
5C75  85                  ADD  A,L
5C76  6F                  LD   L,A
5C77  8C                  ADC  A,H
5C78  95                  SUB  L
5C79  67                  LD   H,A
5C7A  18 F3               JR   L5C6F

5C7C  CB 7F        L5C7C: BIT  7,A            bit 7: ledande koefficient 1
5C7E  CB BF               RES  7,A
```

```
5C80  E5                PUSH HL
5C81  F5                PUSH AF             (IX+1) = antal, (IX+2) = pekare
5C82  DD 21 00 00       LD   IX,$0000
5C86  DD 39             ADD  IX,SP
5C88  20 05             JR   NZ,L5C8F
5C8A  CD 25 4C          CALL FLOAD          summan = första koefficienten
5C8D  18 1B             JR   L5CAA

5C8F  21 06 00    L5C8F: LD   HL,$0006
5C92  39                 ADD  HL,SP
5C93  CD 25 4C           CALL FLOAD          summan = x (* 1)
5C96  18 06              JR   L5C9E

5C98  CD 25 4C    L5C98: CALL FLOAD
5C9B  CD EA 4F           CALL FMUL           summan * x ...
5C9E  DD 6E 02    L5C9E: LD   L,(IX+$02)
5CA1  DD 66 03           LD   H,(IX+$03)
5CA4  CD 25 4C           CALL FLOAD
5CA7  CD 41 4F           CALL FADD           ... + koefficienten
5CAA  FD 7E 1E    L5CAA: LD   A,(IY+$1E)
5CAD  DD 86 02           ADD  A,(IX+$02)
5CB0  DD 77 02           LD   (IX+$02),A     pekaren till nästa
5CB3  30 03              JR   NC,L5CB8
5CB5  DD 34 03           INC  (IX+$03)
5CB8  DD E5       L5CB8: PUSH IX
5CBA  E1                 POP  HL
5CBB  11 06 00           LD   DE,$0006
5CBE  19                 ADD  HL,DE
5CBF  DD 35 01           DEC  (IX+$01)       antal - 1
5CC2  20 D4              JR   NZ,L5C98
5CC4  EB                 EX   DE,HL
5CC5  CD 3F 4C           CALL FPOP           resultatet över x
5CC8  E1                 POP  HL
5CC9  E1                 POP  HL
5CCA  C9                 RET
```

3.38 ADD$, SUB$, MUL$, DIV$: räkning med decimala tal på stacken $5CCB

Talen har formen från STRDEC: antal siffror (bit 7 = tecken), exponent med tecken (talet = 0.siffrorna * 10^exp) och siffrorna i ASCII. b$ ligger överst, a$ under. BCDLEN räknar antalet siffror i resultatet ur n (IY+$21 = -n) och exponenterna, och BCDRES lägger resultatet som nollor under talen innan det räknas fram. BCDEND flyttar resultatet upp över talen och stryker nollorna (DECNRM); vid fel tar BCDERR bort talen och sätter CY (i BASIC fel 133).

SUBS byter tecken på b$ och fortsätter i ADDS.

ADDS: ADD$ (och SUB$). Resultatet får exponenten och de första siffrorna ur talet med störst exponent (se $5FA6). Det andras siffror adderas eller subtraheras (BCDADD, BCDSUB) så långt de når in i resultatet, resten stryks (se $5CDB). En minnessiffra ut ur den första siffran ger 1 först och exponenten + 1 (exponent 128 är fel); ett lån ut ger tiokomplementet med vänt tecken. In (även MULS och DIVS): returadressen vid SP, under den b$ och a$ i decimal form; IY+$21 = -n. Ut: NC och resultatet i decimal form i talens ställe (DECNRM); CY: talen borttagna (BCDERR). Förstör A, BC, DE, HL och IX.

```
5CCB  E1          SUBS:  POP  HL
5CCC  D1                 POP  DE
5CCD  3E 80              LD   A,$80          tecknet på b$ vänds
5CCF  AB                 XOR  E
5CD0  5F                 LD   E,A
5CD1  D5                 PUSH DE
5CD2  E5                 PUSH HL
5CD3  06 01       ADDS:  LD   B,$01          ADD$ (B = 1: addition)
5CD5  CD B4 5F           CALL BCDLEN
5CD8  DA 99 5D           JP   C,BCDERR       för många siffror: fel
5CDB  FD 4E 21           LD   C,(IY+$21)     C = siffror + 1 (n + 2 dec.) (gränsfall)
5CDE  0C                 INC  C
5CDF  CD 71 5E           CALL BCDRES         resultatet: nollor under talen
5CE2  E5                 PUSH HL
5CE3  CD F6 5F           CALL BCDNXT
```

```
5CE6  D1                POP  DE
5CE7  23                INC  HL
5CE8  13                INC  DE
5CE9  1A                LD   A,(DE)
5CEA  BE                CP   (HL)          DE = störst exponent (gränsfall)
5CEB  F2 EF 5C          JP   P,L5CEF
5CEE  EB                EX   DE,HL
5CEF  1B          L5CEF: DEC  DE
5CF0  2B                DEC  HL
5CF1  E5                PUSH HL            HL = det andra
5CF2  21 04 00          LD   HL,$0004
5CF5  39                ADD  HL,SP
5CF6  E5                PUSH HL
5CF7  EB                EX   DE,HL
5CF8  46                LD   B,(HL)
5CF9  3E 80             LD   A,$80         tecknet ur det största
5CFB  A0                AND  B
5CFC  B1                OR   C
5CFD  12                LD   (DE),A
5CFE  23                INC  HL
5CFF  13                INC  DE
5D00  3E 7F             LD   A,$7F
5D02  A0                AND  B
5D03  B9                CP   C             högst dess antal siffror
5D04  30 01             JR   NC,L5D07
5D06  4F                LD   C,A
5D07  0C          L5D07: INC  C
5D08  06 00             LD   B,$00
5D0A  ED B0             LDIR               exponent och siffror kopieras
5D0C  D1                POP  DE
5D0D  E1                POP  HL
5D0E  1A                LD   A,(DE)
5D0F  AE                XOR  (HL)          S = olika tecken
5D10  F5                PUSH AF
5D11  1A                LD   A,(DE)
5D12  E6 7F             AND  $7F
5D14  4F                LD   C,A
5D15  46                LD   B,(HL)
5D16  CB B8             RES  7,B
5D18  23                INC  HL
5D19  13                INC  DE
5D1A  1A                LD   A,(DE)
5D1B  96                SUB  (HL)          A = skiftet
5D1C  F5                PUSH AF
5D1D  80                ADD  A,B
5D1E  B9                CP   C             C = sista platsen som nås
5D1F  30 01             JR   NC,L5D22
5D21  4F                LD   C,A
5D22  F1          L5D22: POP  AF
5D23  91                SUB  C
5D24  F2 74 5D          JP   P,L5D74       inget når in: klart
5D27  ED 44             NEG
5D29  47                LD   B,A           B = siffror ur det andra
5D2A  C5                PUSH BC
5D2B  78                LD   A,B
5D2C  06 00             LD   B,$00
5D2E  EB                EX   DE,HL         DE, HL = sista siffrorna
5D2F  09                ADD  HL,BC
5D30  EB                EX   DE,HL
5D31  4F                LD   C,A
5D32  09                ADD  HL,BC
5D33  C1                POP  BC
5D34  F1                POP  AF
5D35  FA 55 5D          JP   M,L5D55       olika tecken: subtraktion
5D38  CD A9 5D          CALL BCDADD
5D3B  30 38             JR   NC,BCDEND     ingen minnessiffra ut: klart
5D3D  C1                POP  BC            BC = returadr., HL = huvudet
5D3E  E1                POP  HL
5D3F  16 31             LD   D,$31         1 först, exponent + 1, ...
```

```
5D41  5C                LD   E,H
5D42  65                LD   H,L
5D43  1C                INC  E
5D44  24                INC  H
5D45  D5                PUSH DE
5D46  E5                PUSH HL
5D47  33                INC  SP             ... antal + 1 (låga byten bort)
5D48  C5                PUSH BC
5D49  3E 7F             LD   A,$7F
5D4B  A4                AND  H
5D4C  28 47             JR   Z,BCDER3       128 siffror: fel
5D4E  7B                LD   A,E
5D4F  FE 80             CP   $80
5D51  28 42             JR   Z,BCDER3       exponent 128: fel
5D53  18 20             JR   BCDEND

5D55  CD C2 5D    L5D55: CALL BCDSUB
5D58  30 1B             JR   NC,BCDEND      inget lån ut: klart
5D5A  EB                EX   DE,HL
5D5B  2B                DEC  HL
5D5C  7E                LD   A,(HL)
5D5D  EE 80             XOR  $80            tecknet vänds ...
5D5F  77                LD   (HL),A
5D60  23                INC  HL
5D61  E6 7F             AND  $7F
5D63  47                LD   B,A
5D64  5F                LD   E,A
5D65  16 00             LD   D,$00
5D67  19                ADD  HL,DE
5D68  3E 60       L5D68: LD   A,$60          ... och tiokomplement
5D6A  9E                SBC  A,(HL)
5D6B  CD DB 5D          CALL BCDFIX
5D6E  77                LD   (HL),A
5D6F  2B                DEC  HL
5D70  10 F6             DJNZ L5D68
5D72  18 01             JR   BCDEND

5D74  F1          L5D74: POP  AF
```

BCDEND: *resultatet ligger under returadressen och två tal ovanför det; returadressen, huvudet och siffrorna flyttas upp över talen, och* DECNRM *stryker nollorna och återvänder.* BCDERR *tar bort två tal och återvänder med CY,* BCDER3 *tre* (*resultatet och talen, när* ADDS *får 128 siffror eller exponent 128*).

```
5D75  21 02 00    BCDEND: LD   HL,$0002
5D78  45                LD   B,L
5D79  39                ADD  HL,SP
5D7A  7E                LD   A,(HL)
5D7B  E6 7F             AND  $7F
5D7D  C6 04             ADD  A,$04          C = siffror + huvud + returadr.
5D7F  4F                LD   C,A
5D80  CD F6 5F          CALL BCDNXT
5D83  E5                PUSH HL
5D84  CD F6 5F    L5D84: CALL BCDNXT         DE = efter de två talen
5D87  10 FB             DJNZ L5D84
5D89  EB                EX   DE,HL
5D8A  E1                POP  HL
5D8B  2B                DEC  HL
5D8C  1B                DEC  DE
5D8D  ED B8             LDDR                flyttas upp över talen
5D8F  EB                EX   DE,HL
5D90  23                INC  HL
5D91  F9                LD   SP,HL
5D92  C3 7B 4D          JP   DECNRM         nollor bort, tillbaka

5D95  06 03       BCDER3: LD   B,$03        tre tal bort
5D97  18 02             JR   L5D9B

5D99  06 02       BCDERR: LD   B,$02        två tal bort, CY
5D9B  D1          L5D9B: POP  DE
5D9C  21 00 00          LD   HL,$0000
5D9F  39                ADD  HL,SP
```

```
5DA0  CD F6 5F     L5DA0:  CALL BCDNXT
5DA3  10 FB                DJNZ L5DA0
5DA5  F9                   LD   SP,HL
5DA6  EB                   EX   DE,HL
5DA7  37                   SCF
5DA8  E9                   JP   (HL)
```

BCDADD: *B siffror bakifrån från HL adderas till siffrorna bakifrån från DE, med CY in; minnessiffran förs sedan vidare åt vänster så långt C (platserna kvar före DE) räcker. CY ut = minnessiffra ut ur den första platsen.* BCDSUB *likadant med subtraktion och lån.* BCDFIX: *ASCII-siffran i A under* 0 + 10*, över* 9 - 10*, med CY; annars NC. In: HL och DE = de sista siffrorna, B = antal, C = platser kvar, CY in. Ut: CY = minnessiffra eller lån ut, B = 0; förstör A och C, DE flyttas förbi de använda platserna.*

```
5DA9  1A           BCDADD: LD   A,(DE)          siffra + siffra + CY
5DAA  8E                   ADC  A,(HL)
5DAB  D6 30                SUB  $30
5DAD  CD DB 5D             CALL BCDFIX
5DB0  12                   LD   (DE),A
5DB1  2B                   DEC  HL
5DB2  1B                   DEC  DE
5DB3  0D                   DEC  C
5DB4  10 F3                DJNZ BCDADD
5DB6  D0           L5DB6:  RET  NC             minnessiffran åt vänster
5DB7  0D                   DEC  C
5DB8  F8                   RET  M              förbi första siffran: CY ut
5DB9  1A                   LD   A,(DE)
5DBA  3C                   INC  A
5DBB  CD DB 5D             CALL BCDFIX
5DBE  12                   LD   (DE),A
5DBF  1B                   DEC  DE
5DC0  18 F4                JR   L5DB6

5DC2  1A           BCDSUB: LD   A,(DE)          siffra - siffra - CY
5DC3  9E                   SBC  A,(HL)
5DC4  C6 30                ADD  A,$30
5DC6  CD DB 5D             CALL BCDFIX
5DC9  12                   LD   (DE),A
5DCA  1B                   DEC  DE
5DCB  2B                   DEC  HL
5DCC  0D                   DEC  C
5DCD  10 F3                DJNZ BCDSUB
5DCF  D0           L5DCF:  RET  NC
5DD0  0D                   DEC  C
5DD1  F8                   RET  M
5DD2  1A                   LD   A,(DE)
5DD3  3D                   DEC  A
5DD4  CD DB 5D             CALL BCDFIX
5DD7  12                   LD   (DE),A
5DD8  1B                   DEC  DE
5DD9  18 F4                JR   L5DCF

5DDB  FE 30        BCDFIX: CP   $30             under 0: + 10, CY
5DDD  38 08                JR   C,L5DE7
5DDF  FE 3A                CP   $3A             över 9: - 10, CY
5DE1  3F                   CCF
5DE2  D0                   RET  NC
5DE3  D6 0A                SUB  $0A
5DE5  37                   SCF
5DE6  C9                   RET

5DE7  C6 0A        L5DE7:  ADD  A,$0A
5DE9  37                   SCF
5DEA  C9                   RET
```

MULS: MUL$, *multiplikation av tal i text. Tecknet blir olika tecken, exponenten summan (spill: fel). Multiplikatorn är talet med minst siffror (lika: b$), högst C siffror. För varje multiplikatorsiffra adderas multiplikanden (*BCDADD*) siffran gånger, ett steg längre till höger än förra gången; när den når resultatets sista plats stryks i stället dess sista siffra (se* $5E4A*), och utan siffror kvar är det klart.*

(IX+0) räknar additionerna, (IX+1) multiplikatorsiffrorna.

```
5DEB  06 02        MULS:  LD   B,$02          MUL$ (B = 2: multiplikation)
5DED  CD B4 5F            CALL BCDLEN
5DF0  DA 99 5D            JP   C,BCDERR
5DF3  FD 4E 21            LD   C,(IY+$21)
5DF6  0C                  INC  C
5DF7  1A                  LD   A,(DE)
5DF8  AE                  XOR  (HL)            tecken = olika tecken
5DF9  E6 80               AND  $80
5DFB  B1                  OR   C
5DFC  4F                  LD   C,A
5DFD  23                  INC  HL
5DFE  13                  INC  DE
5DFF  1A                  LD   A,(DE)
5E00  86                  ADD  A,(HL)          exponent = summan, spill fel
5E01  EA 99 5D            JP   PE,BCDERR
5E04  47                  LD   B,A
5E05  CD 71 5E            CALL BCDRES          resultatet: nollor
5E08  7E                  LD   A,(HL)
5E09  E6 7F               AND  $7F
5E0B  47                  LD   B,A
5E0C  E5                  PUSH HL
5E0D  CD F6 5F            CALL BCDNXT
5E10  D1                  POP  DE
5E11  7E                  LD   A,(HL)
5E12  E6 7F               AND  $7F
5E14  B8                  CP   B               HL = det med minst siffror
5E15  38 02               JR   C,L5E19
5E17  EB                  EX   DE,HL
5E18  78                  LD   A,B
5E19  CB B9        L5E19: RES  7,C
5E1B  B9                  CP   C               multiplikatorn: högst C siffror
5E1C  38 01               JR   C,L5E1F
5E1E  79                  LD   A,C
5E1F  23           L5E1F: INC  HL
5E20  F5                  PUSH AF              (IX+1) = siffror kvar
5E21  E5                  PUSH HL
5E22  DD 21 02 00         LD   IX,$0002
5E26  DD 39               ADD  IX,SP
5E28  1A                  LD   A,(DE)
5E29  E6 7F               AND  $7F
5E2B  B9                  CP   C               multiplikanden: högst C
5E2C  38 01               JR   C,L5E2F
5E2E  79                  LD   A,C
5E2F  6F           L5E2F: LD   L,A
5E30  26 00               LD   H,$00
5E32  EB                  EX   DE,HL
5E33  23                  INC  HL
5E34  19                  ADD  HL,DE
5E35  EB                  EX   DE,HL
5E36  39                  ADD  HL,SP           DE = plats A i resultatet
5E37  01 07 00            LD   BC,$0007
5E3A  09                  ADD  HL,BC
5E3B  EB                  EX   DE,HL
5E3C  4F                  LD   C,A
5E3D  47                  LD   B,A
5E3E  DD 7E 04     L5E3E: LD   A,(IX+$04)      multiplikanden vid slutet? ...
5E41  E6 7F               AND  $7F
5E43  B9                  CP   C
5E44  28 04               JR   Z,L5E4A
5E46  13                  INC  DE              nej: ett steg höger
5E47  0C                  INC  C
5E48  18 04               JR   L5E4E

5E4A  2B           L5E4A: DEC  HL              ja: sista siffran bort (gränsfall)
5E4B  05                  DEC  B
5E4C  28 1E               JR   Z,L5E6C
5E4E  E3           L5E4E: EX   (SP),HL         nästa multiplikatorsiffra
5E4F  23                  INC  HL
5E50  7E                  LD   A,(HL)
```

```
5E51  E3                EX   (SP),HL
5E52  D6 30             SUB  $30
5E54  28 11             JR   Z,L5E67
5E56  DD 77 00          LD   (IX+$00),A        (IX) = siffran: så många ...
5E59  E5          L5E59: PUSH HL
5E5A  D5                PUSH DE
5E5B  C5                PUSH BC
5E5C  CD A9 5D          CALL BCDADD            ... gånger multiplikanden
5E5F  C1                POP  BC
5E60  D1                POP  DE
5E61  E1                POP  HL
5E62  DD 35 00          DEC  (IX+$00)
5E65  20 F2             JR   NZ,L5E59
5E67  DD 35 01    L5E67: DEC  (IX+$01)         siffror kvar - 1
5E6A  20 D2             JR   NZ,L5E3E
5E6C  E1          L5E6C: POP  HL
5E6D  E1                POP  HL
5E6E  C3 75 5D          JP   BCDEND
```

BCDRES: *C (bit 0-6) nollor ("0") under de två returadresserna, med huvudet C, B (antal, exponent) före. Ut: HL = b$, DE förstört.*

```
5E71  79          BCDRES: LD   A,C
5E72  E6 7F               AND  $7F
5E74  ED 44               NEG
5E76  6F                  LD   L,A
5E77  26 FF               LD   H,$FF
5E79  D1                  POP  DE              returadresserna av
5E7A  DD E1               POP  IX
5E7C  39                  ADD  HL,SP
5E7D  F9                  LD   SP,HL
5E7E  C5                  PUSH BC              huvudet: C antal, B exponent
5E7F  DD E5               PUSH IX
5E81  D5                  PUSH DE
5E82  1E 30               LD   E,$30
5E84  73          L5E84:  LD   (HL),E          nollor
5E85  23                  INC  HL
5E86  3C                  INC  A
5E87  20 FB               JR   NZ,L5E84
5E89  C9                  RET
```

DIVS: *nämnaren 0 är fel, täljaren 0 ger svaret 0. Har täljaren färre siffror än nämnaren flyttas allt ned och täljaren får nollor sist. Täljarens exponentbyte blir en siffra "0" först, så att resten får plats. Kvoten (C siffror, exponent e1 - e2 + 1) räknas med upprepad subtraktion utan tecken: så länge täljaren med "0"-platsen är minst nämnaren dras nämnaren ifrån och kvotsiffran ökas; sedan flyttas täljaren ett steg vänster till nästa siffra. Rest 0 slutar i förtid. Kvoten huggs av vid sista platsen (n + 2 decimaler) och avrundas sedan rätt. (IX+0) = kvotsiffror kvar.*

```
5E8A  06 03       DIVS:   LD   B,$03           B = 3: division
5E8C  CD B4 5F            CALL BCDLEN
5E8F  DA 99 5D            JP   C,BCDERR
5E92  CD 61 5F            CALL BCDZ
5E95  CA 99 5D            JP   Z,BCDERR        nämnaren 0: fel
5E98  EB                  EX   DE,HL
5E99  CD 61 5F            CALL BCDZ
5E9C  20 04               JR   NZ,L5EA2        täljaren 0: svaret 0
5E9E  D1                  POP  DE
5E9F  F9                  LD   SP,HL
5EA0  EB                  EX   DE,HL
5EA1  E9                  JP   (HL)

5EA2  EB          L5EA2:  EX   DE,HL
5EA3  7E                  LD   A,(HL)
5EA4  E6 7F               AND  $7F
5EA6  4F                  LD   C,A
5EA7  1A                  LD   A,(DE)
5EA8  E6 7F               AND  $7F
5EAA  91                  SUB  C
5EAB  30 28               JR   NC,L5ED5        täljaren kortare: ...
5EAD  5F                  LD   E,A
```

```
5EAE  16 FF              LD   D,$FF
5EB0  81                 ADD  A,C
5EB1  81                 ADD  A,C
5EB2  C6 06              ADD  A,$06
5EB4  4F                 LD   C,A
5EB5  3E 00              LD   A,$00
5EB7  17                 RLA
5EB8  47                 LD   B,A
5EB9  2B                 DEC  HL
5EBA  2B                 DEC  HL
5EBB  EB                 EX   DE,HL
5EBC  19                 ADD  HL,DE
5EBD  F9                 LD   SP,HL          ... allt flyttas ned, ...
5EBE  EB                 EX   DE,HL
5EBF  ED B0              LDIR
5EC1  A7                 AND  A
5EC2  ED 52              SBC  HL,DE
5EC4  45                 LD   B,L
5EC5  3E 30              LD   A,$30
5EC7  12           L5EC7: LD  (DE),A         ... och nollor sist
5EC8  13                 INC  DE
5EC9  2D                 DEC  L
5ECA  20 FB              JR   NZ,L5EC7
5ECC  21 04 00           LD   HL,$0004
5ECF  CD EE 5F           CALL BCDARG
5ED2  1A                 LD   A,(DE)
5ED3  80                 ADD  A,B            antal + nollorna
5ED4  12                 LD   (DE),A
5ED5  1A           L5ED5: LD  A,(DE)
5ED6  AE                 XOR  (HL)
5ED7  E6 80              AND  $80            tecken = olika tecken
5ED9  FD 86 21           ADD  A,(IY+$21)
5EDC  3C                 INC  A
5EDD  4F                 LD   C,A
5EDE  23                 INC  HL
5EDF  13                 INC  DE
5EE0  1A                 LD   A,(DE)
5EE1  96                 SUB  (HL)           exponent = e1 - e2 + 1
5EE2  EA 99 5D           JP   PE,BCDERR
5EE5  3C                 INC  A
5EE6  EA 99 5D           JP   PE,BCDERR
5EE9  47                 LD   B,A
5EEA  3E 30              LD   A,$30
5EEC  12                 LD   (DE),A         täljarens exponent blir 0
5EED  CD 71 5E           CALL BCDRES         kvoten: nollor
5EF0  EB                 EX   DE,HL
5EF1  CB B9              RES  7,C
5EF3  C5                 PUSH BC
5EF4  21 06 00           LD   HL,$0006       pekare på kvotsiffran
5EF7  39                 ADD  HL,SP
5EF8  E5                 PUSH HL
5EF9  EB                 EX   DE,HL
5EFA  DD 21 FA FF        LD   IX,$FFFA       (IX) = kvotsiffror kvar
5EFE  DD 19              ADD  IX,DE
5F00  E5                 PUSH HL
5F01  CB BE              RES  7,(HL)         tecknen bort
5F03  CD F6 5F           CALL BCDNXT
5F06  CB BE              RES  7,(HL)
5F08  D1                 POP  DE
5F09  EB                 EX   DE,HL
5F0A  EB           L5F0A: EX  DE,HL
5F0B  E5                 PUSH HL
5F0C  46                 LD   B,(HL)
5F0D  04                 INC  B
5F0E  CD 6B 5F           CALL BCDNUL         resten 0: klart
5F11  E1                 POP  HL
5F12  28 48              JR   Z,L5F5C
5F14  EB                 EX   DE,HL
5F15  E5                 PUSH HL
```

```
5F16  D5              PUSH DE
5F17  46              LD   B,(HL)
5F18  23              INC  HL
5F19  13              INC  DE
5F1A  1A              LD   A,(DE)
5F1B  FE 30           CP   $30
5F1D  20 08           JR   NZ,L5F27        0-platsen inte 0: större
5F1F  23        L5F1F:INC  HL              siffra för siffra
5F20  13              INC  DE
5F21  1A              LD   A,(DE)
5F22  BE              CP   (HL)
5F23  20 02           JR   NZ,L5F27
5F25  10 F8           DJNZ L5F1F
5F27  D1        L5F27:POP  DE
5F28  E1              POP  HL
5F29  38 16           JR   C,L5F41         mindre: nästa kvotsiffra
5F2B  4E              LD   C,(HL)
5F2C  0C              INC  C
5F2D  06 00           LD   B,$00
5F2F  E5              PUSH HL
5F30  D5              PUSH DE
5F31  09              ADD  HL,BC
5F32  EB              EX   DE,HL
5F33  09              ADD  HL,BC
5F34  EB              EX   DE,HL
5F35  41              LD   B,C
5F36  05              DEC  B
5F37  CD C2 5D        CALL BCDSUB          täljaren - nämnaren ...
5F3A  D1              POP  DE
5F3B  E1              POP  HL
5F3C  E3              EX   (SP),HL
5F3D  34              INC  (HL)            ... och kvotsiffran + 1
5F3E  E3              EX   (SP),HL
5F3F  18 C9           JR   L5F0A

5F41  DD 35 00  L5F41:DEC  (IX+$00)        kvotsiffror kvar - 1
5F44  28 16           JR   Z,L5F5C
5F46  C1              POP  BC
5F47  03              INC  BC              pekaren till nästa
5F48  C5              PUSH BC
5F49  E5              PUSH HL
5F4A  D5              PUSH DE
5F4B  EB              EX   DE,HL
5F4C  4E              LD   C,(HL)
5F4D  06 00           LD   B,$00
5F4F  23              INC  HL
5F50  E5              PUSH HL
5F51  D1              POP  DE
5F52  23              INC  HL
5F53  ED B0           LDIR                 täljaren ett steg vänster
5F55  3E 30           LD   A,$30
5F57  12              LD   (DE),A
5F58  D1              POP  DE
5F59  E1              POP  HL
5F5A  18 AE           JR   L5F0A

5F5C  E1        L5F5C:POP  HL
5F5D  E1              POP  HL
5F5E  C3 75 5D        JP   BCDEND
```

BCDZ: *Z om talet vid HL bara har nollor (HL behålls).* BCDNUL: *Z om B tecken efter HL alla är 0. Båda förstör A och B.* BCDNUL: *HL på den första siffran som inte är 0 (eller på den sista).*

```
5F61  E5        BCDZ: PUSH HL
5F62  46              LD   B,(HL)
5F63  CB B8           RES  7,B
5F65  23              INC  HL
5F66  CD 6B 5F        CALL BCDNUL
5F69  E1              POP  HL
```

```
5F6A  C9                RET

5F6B  3E 30       BCDNUL: LD   A,$30
5F6D  23          L5F6D:  INC  HL
5F6E  BE                  CP   (HL)          Z: bara nollor
5F6F  C0                  RET  NZ
5F70  10 FB               DJNZ L5F6D
5F72  C9                  RET
```

BCDCMP (COMP%)*: jämför a$ med b$ under returadressen. Ut: Z = lika, CY = a$ mindre, annars NZ och NC. Är ett tal 0 eller tecknen olika avgör tecknet; annars jämförs exponenten och sedan siffrorna (omvänt för negativa tal), och vid lika början är talet med flest siffror störst. Exponenterna jämförs med* SUB *och teckenflaggan (se* $5FA6*). In: returadressen vid SP, under den b$ och a$. Talen står kvar; förstör A, BC, DE och HL.*

```
5F73  21 04 00    BCDCMP: LD   HL,$0004
5F76  CD EE 5F            CALL BCDARG
5F79  CD 61 5F            CALL BCDZ          b$ = 0? ...
5F7C  EB                  EX   DE,HL
5F7D  20 07               JR   NZ,L5F86
5F7F  CD 61 5F            CALL BCDZ
5F82  C8                  RET  Z             ... båda 0: Z
5F83  7E                  LD   A,(HL)        a$:s tecken avgör
5F84  18 28               JR   L5FAE

5F86  CD 61 5F    L5F86:  CALL BCDZ
5F89  EB                  EX   DE,HL
5F8A  7E                  LD   A,(HL)
5F8B  2F                  CPL                a$ = 0: b$:s tecken vänt
5F8C  28 20               JR   Z,L5FAE
5F8E  1A                  LD   A,(DE)
5F8F  AE                  XOR  (HL)
5F90  1A                  LD   A,(DE)
5F91  FA AE 5F            JP   M,L5FAE       olika tecken: a$:s avgör
5F94  07                  RLCA
5F95  30 01               JR   NC,L5F98      båda negativa: omvänt
5F97  EB                  EX   DE,HL
5F98  1A          L5F98:  LD   A,(DE)
5F99  96                  SUB  (HL)
5F9A  4F                  LD   C,A           C = skillnaden i antal
5F9B  1A                  LD   A,(DE)
5F9C  38 01               JR   C,L5F9F
5F9E  7E                  LD   A,(HL)
5F9F  E6 7F       L5F9F:  AND  $7F
5FA1  47                  LD   B,A
5FA2  04                  INC  B             B = minsta antal + 1
5FA3  23          L5FA3:  INC  HL            exponenten, sedan siffrorna
5FA4  13                  INC  DE
5FA5  1A                  LD   A,(DE)
5FA6  96                  SUB  (HL)(fel)
5FA7  20 05               JR   NZ,L5FAE      skillnadens tecken (fel)
5FA9  10 F8               DJNZ L5FA3
5FAB  79                  LD   A,C           lika: flest siffror störst
5FAC  A7                  AND  A
5FAD  C8                  RET  Z
5FAE  A7          L5FAE:  AND  A
5FAF  37                  SCF                M: CY (mindre), annars NZ NC
5FB0  F8                  RET  M
5FB1  F6 01               OR   $01
5FB3  C9                  RET
```

BCDLEN*: antalet siffror i resultatet till IY+*$21 *(in B = 1 +, 2 *, 3 /). n < 0: -n + 1. Annars den största exponenten (+), e1 + e2 (*) eller e1 - e2 (/), + n + 1, minst 1. Över 125 eller spill: CY.*

In: B = 1, 2 eller 3; vid SP returadressen och den till S2192*, under dem b$ och a$. Ut: HL = b$, DE = a$. Förstör A och BC.*

```
5FB4  21 06 00    BCDLEN: LD   HL,$0006
5FB7  CD EE 5F            CALL BCDARG
5FBA  FD 7E 21            LD   A,(IY+$21)    A = -n
5FBD  4F                  LD   C,A
5FBE  0D                  DEC  C
```

```
5FBF  F2 E1 5F            JP   P,L5FE1          n < 0: -n + 1 siffror
5FC2  4F                  LD   C,A
5FC3  23                  INC  HL
5FC4  13                  INC  DE
5FC5  1A                  LD   A,(DE)           a$:s exponent ...
5FC6  05                  DEC  B
5FC7  28 09               JR   Z,L5FD2
5FC9  05                  DEC  B
5FCA  28 03               JR   Z,L5FCF
5FCC  96                  SUB  (HL)             /: e1 - e2
5FCD  18 08               JR   L5FD7

5FCF  86          L5FCF:  ADD  A,(HL)           *: e1 + e2
5FD0  18 05               JR   L5FD7

5FD2  BE          L5FD2:  CP   (HL)             +: den största (fel)
5FD3  F2 D7 5F            JP   P,L5FD7
5FD6  7E                  LD   A,(HL)
5FD7  2B          L5FD7:  DEC  HL
5FD8  1B                  DEC  DE
5FD9  91                  SUB  C                + n
5FDA  EA E7 5F            JP   PE,L5FE7
5FDD  F2 E1 5F            JP   P,L5FE1
5FE0  AF                  XOR  A                under 0: 0
5FE1  3C          L5FE1:  INC  A                + 1, högst 125
5FE2  FE 7E               CP   $7E
5FE4  3F                  CCF
5FE5  30 03               JR   NC,L5FEA
5FE7  3E 7D       L5FE7:  LD   A,$7D
5FE9  37                  SCF
5FEA  FD 77 21    L5FEA:  LD   (IY+$21),A       IY+$21 = antal siffror
5FED  C9                  RET
```

BCDARG: *HL = HL + SP (räknat med* BCDARG*:s returadress), DE = talet efter det.* BCDNXT*: HL förbi talet vid HL (antal + 2 bytes). Båda förstör A.*

```
5FEE  39          BCDARG: ADD  HL,SP
5FEF  E5                  PUSH HL
5FF0  CD F6 5F            CALL BCDNXT
5FF3  D1                  POP  DE
5FF4  EB                  EX   DE,HL
5FF5  C9                  RET

5FF6  7E          BCDNXT: LD   A,(HL)
5FF7  E6 7F               AND  $7F
5FF9  C6 02               ADD  A,$02            antal + 2 bytes
5FFB  85                  ADD  A,L
5FFC  6F                  LD   L,A
5FFD  D0                  RET  NC
5FFE  24                  INC  H
5FFF  C9                  RET
```

3.39 Tilläggsprogrammet på $6000: DOS (skivminnen via ABC-bussen) $6000

Nästan samma kod som DOS:en på samma adress i ABC800 M, ABC800 C och ABC802 (3426 av 4096 bytes lika). Signatur GN T{by 1984-03-02 vid $6FAC (T{by = Täby). Startas av EXTSCN via JP-tabellen:

```
$6000 DOSINI  start (DOSSTA)        $6030 DWRREC  skriv post
$6003 DLDRUN  ladda fil och hoppa   $6033-$603C   byte i bufferten
$6006 DLOAD   ladda fil (postformat)$603F DPRVRC  föregående post
$6009 DOSFN   katalogfunktion 1-4   $6042 DCOPY   LDI-slinga
$600C DGETB   nästa byte (DBUFP)    $6045 DSETEH  felvektor C := DE
$600F/$6012   läs/skriv sektor rått $6048 DFNJP   funktion >= 8
$6015 DPREP   skapa fil (PREPARE)   $604B-$605A   tomma (JP till sig själv)
$6018 DOPEN   öppna fil             $605D DCLSAL  stäng alla, BYE
$601B DOPENQ  ladda, saknas = CY    $6060 DSELCD  välj kortet
$601E DRUN    ladda och kör filen   $6063/$6066   skriv/läs sektor, B = 0
$6021 DCLOSE  stäng                 $6069-$6076   sju adresser (läses inte)
$6024 DSETEN  sätt filens slut
```

```
    $602A DSETRC  välj post DE          $602D DRDREC  läs post
```

Anropen har B = kanal * 16 (kanal 0-7), DE = namn eller postnummer.

DOSSTA gör följande:

- Om $FFF6 = $A5 är det en varmstart. Annars nollställs DSYSUN ($FFF7/$FFF8).
- Kopierar enhetsposten DOSDEV till $FDD0, tabellhuvudet DOSSTH till $FDD8 och korttabellen CRDTAB till $FDE2 (CRDRAM).
- Länkar in enheterna före de gamla i DEVLST och satserna BYE, KILL och NAME före de gamla i STKWP.
- Sätter SYSOPT = $FB00 och UNSVEC-målet = UNSRUT ($6EFC).
- Lägger stacken på $F500. Sektorbuffertarna ligger över den: $F500 + 256 * kanal (DBUFP, SLTBUF).
- Nollställer kanalerna (DOSRST) och öppnar BASICINI.SYS (OPNINI). Sektorns bytes 6-21 kopieras till INIBUF. Börjar sektorn (byte 3) med JP körs den koden.
- Returnerar HL = $6F00, så EXTSCN fortsätter på $7000.

```
6000  C3 6C 6C     DOSINI: JP   DOSSTA
6003  C3 12 62     DLDRUN: JP   LDRUN
6006  C3 CC 61     DLOAD:  JP   LOADF
6009  C3 33 62     DOSFN:  JP   DOSFN1
600C  C3 28 62     DGETB:  JP   GETBYT
600F  C3 7E 60     DRDSCR: JP   L607E
6012  C3 8B 60     DWRSCR: JP   L608B
6015  C3 16 65     DPREP:  JP   PREPF
6018  C3 12 65     DOPEN:  JP   OPENF
601B  C3 02 65     DOPENQ: JP   OPENQ
601E  C3 0C 65     DRUN:   JP   RUNFIL
6021  C3 33 65     DCLOSE: JP   CLOSEF
6024  C3 6E 65     DSETEN: JP   SETEND
6027  C3 93 65     DNOP:   JP   L6593
602A  C3 57 64     DSETRC: JP   SETREC
602D  C3 95 64     DRDREC: JP   RDREC
6030  C3 CA 64     DWRREC: JP   WRREC
6033  C3 7B 63     DBUFRD: JP   BUFRD
6036  C3 91 63     DBUFR2: JP   BUFRD2
6039  C3 A0 63     DBUFWR: JP   BUFWR
603C  C3 B8 63     DBUFW2: JP   BUFWR2
603F  C3 4E 64     DPRVRC: JP   PRVREC
6042  C3 8C 62     DCOPY:  JP   COPY
6045  C3 3F 64     DSETEH: JP   SETEH
6048  C3 97 65     DFNJP:  JP   DFNCX
604B  C3 4B 60     L604B:  JP   L604B
604E  C3 4E 60     L604E:  JP   L604E
6051  C3 51 60     L6051:  JP   L6051
6054  C3 54 60     L6054:  JP   L6054
6057  C3 57 60     L6057:  JP   L6057
605A  C3 5A 60     L605A:  JP   L605A
605D  C3 37 65     DCLALL: JP   DCLSAL
6060  C3 6F 61     DSELCD: JP   SELCRD
```

```
6063  C3 89 60     DWRSR0: JP   L6089

6066  C3 7C 60     DRDSR0: JP   L607C
```

Sju adresser: CRDTAB ($6EDB), *korttabellen i RAM* ($FDE2), DSYSUN ($FFF7), *versionen 19, kroken* VFE02, $FD3D *och* VFD3F (*kortadressen*). *Ingenting i kretsen läser dem; UFD-DOS:en har samma tabell med versionen 20.*

```
6069  DB 6E              DEFW $6EDB
606B  E2 FD              DEFW $FDE2
606D  F7 FF              DEFW $FFF7
606F  13 00              DEFW $0013          DOS-versionen: 19 ($14 = 20 i ABC800 M)
6071  02 FE              DEFW $FE02
6073  3D FD              DEFW $FD3D
6075  3F FD              DEFW $FD3F
```

Ingångarna till SECIO. RDSEC/WRSEC *läser/skriver med kanal 0:s buffert,* S6079/S6086 *med kanal B:s.* L607C/L6089 (DRDSR0/DWRSR0) *och* L607E/L608B (DRDSCR/DWRSCR) *sätter dessutom B bit 0, som tillsammans med flaggbit 6 i kortposten stänger av omräkningen av sektornumret; ingen post i* CRDTAB *har den biten, så i praktiken gör de samma sak. In: DE = sektorn (och B = kanal * 16 där kanalen inte är 0). Ut: CY = fel, A = statusen (också i* VFD15). *BC, DE och HL behålls.*

```
6077  06 00       RDSEC:  LD   B,$00          läs, B = 0 (kanal 0)
6079  C5          S6079:  PUSH BC             läs, kanal B
607A  18 04              JR   L6080

607C  06 00       L607C:  LD   B,$00          kanal 0, B bit 0 = 1
607E  C5          L607E:  PUSH BC
607F  04                  INC  B             B bit 0 = 1 (se $60AD)
6080  0E 03       L6080:  LD   C,$03          C = 3: läs
6082  18 0B              JR   SECIO

6084  06 00       WRSEC:  LD   B,$00          skriv, kanal 0
6086  C5          S6086:  PUSH BC             skriv, kanal B
6087  18 04              JR   L608D

6089  06 00       L6089:  LD   B,$00          kanal 0, B bit 0 = 1
608B  C5          L608B:  PUSH BC
608C  04                  INC  B             B bit 0 = 1
608D  0E 0C       L608D:  LD   C,$0C          C = $0C: skriv
```

3.40 SECIO: läs eller skriv en sektor via ABC-bussen $608F

In: C = 3 (läs) eller $0C (skriv), B = kanal * 16, DE = sektornummer. B bit 0 = 1 (DRDSR0/DWRSR0) och bit 6 i kortpostens flaggbyte (satt i ingen post i CRDTAB) stänger av omräkningen av sektornumret. Först anropas kroken VFE02, som är RET (sista byten i kopian av CRDTAB, $6EFB).

- DRETRY ($FD18) = antal försök (3).
- SELCRD väljer kortet (OUT $01) enligt CRDTYP. Vid omförsök ges OUT $04 (återställ).
- Sektornumret räknas om efter kortbytens bitar 6-7: $40 och $C0 ger 32 * (n / 4) + n % 4, $80 32 * n, 0 (HD) n.
- Kommandot är OUT $02 och sedan C, B (enhet & 3), D och E på port $00.
- Data (256 bytes) flyttas med INIR/OTIR (kortbyte bit 6-7 = 0 eller $C0) eller byte för byte med IN $01 bit 0 som klarsignal.
- Sist läses status på IN $00. Status <> 0 ger nytt försök. Felet hamnar i VFD15. $80 = inget kort.

S6196 väntar på IN $01 (L = mask, H = önskat värde) med tidsgräns. $FF och $00 betyder att kortet saknas.

```
608F  CD 02 FE    SECIO:  CALL VFE02
6092  D5                  PUSH DE
6093  E5                  PUSH HL
6094  21 18 FD            LD   HL,$FD18
6097  36 03               LD   (HL),$03       tre försök
6099  CD 6F 61    L6099:  CALL SELCRD
609C  28 0D               JR   Z,L60AB        kortet svarar
609E  D3 04               OUT  ($04),A        återställ och försök igen
60A0  21 18 FD            LD   HL,$FD18
60A3  35                  DEC  (HL)
60A4  20 F3               JR   NZ,L6099
```

```
60A6  3E 80              LD   A,$80          $80: kortet svarar inte
60A8  C3 67 61           JP   L6167

60AB  D3 02       L60AB: OUT  ($02),A        nytt kommando
60AD  CB 40              BIT  0,B
60AF  28 05              JR   Z,L60B6
60B1  23                 INC  HL
60B2  CB 76              BIT  6,(HL)         flaggorna bit 6: ingen omräkning
60B4  20 2B              JR   NZ,L60E1
60B6  E6 C0       L60B6: AND  $C0
60B8  28 27              JR   Z,L60E1        kortbyte bit 6-7 = 0 (HD): som det står
60BA  26 04              LD   H,$04          $40/$C0: 4 sektorer per spår
60BC  FE 40              CP   $40
60BE  28 06              JR   Z,L60C6
60C0  FE C0              CP   $C0
60C2  28 02              JR   Z,L60C6
60C4  26 01              LD   H,$01          $80: 1
60C6  7C          L60C6: LD   A,H
60C7  3D                 DEC  A
60C8  A3                 AND  E
60C9  6F                 LD   L,A            L = n & (H - 1)
60CA  7C                 LD   A,H
60CB  FE 20       L60CB: CP   $20
60CD  28 0D              JR   Z,L60DC
60CF  CB 27              SLA  A
60D1  CB 23              SLA  E              skifta DE tills A = 32
60D3  CB 12              RL   D
60D5  30 F4              JR   NC,L60CB
60D7  3E 02              LD   A,$02          för stort sektornummer
60D9  C3 67 61           JP   L6167

60DC  3E E0       L60DC: LD   A,$E0          32 * (n / H) + L
60DE  A3                 AND  E
60DF  B5                 OR   L
60E0  5F                 LD   E,A
60E1  CD BE 61    L60E1: CALL SLTBUF         H = kanalens buffert
60E4  CD B2 61           CALL UNITNR
60E7  E6 03              AND  $03            enheten på kortet
60E9  47                 LD   B,A
60EA  E5          L60EA: PUSH HL
60EB  21 03 01           LD   HL,$0103       vänta tills IN 1 & 3 = 1
60EE  CD 96 61           CALL S6196
60F1  79                 LD   A,C
60F2  D3 00              OUT  ($00),A        C: kommandot
60F4  CD 96 61           CALL S6196
60F7  78                 LD   A,B
60F8  D3 00              OUT  ($00),A        B: enheten
60FA  CD 96 61           CALL S6196
60FD  7A                 LD   A,D
60FE  D3 00              OUT  ($00),A        D, E: sektorn
6100  CD 96 61           CALL S6196
6103  7B                 LD   A,E
6104  D3 00              OUT  ($00),A
6106  E1                 POP  HL
6107  DB 01       L6107: IN   A,($01)
6109  CB 7F              BIT  7,A            bit 7: status direkt
610B  20 49              JR   NZ,L6156
610D  E6 05              AND  $05            bit 0 = 1, bit 2 = 0: data
610F  EE 01              XOR  $01
6111  20 F4              JR   NZ,L6107
6113  C5                 PUSH BC
6114  06 10              LD   B,$10          kort paus
6116  10 FE       L6116: DJNZ L6116
6118  44                 LD   B,H            H undan
6119  CD 84 61           CALL CRDTYP
611C  60                 LD   H,B            H tillbaka
611D  E6 C0              AND  $C0            Z: HD eller $C0 (blockvis)
611F  28 02              JR   Z,L6123
6121  FE C0              CP   $C0
```

```
6123  79          L6123:  LD   A,C
6124  0F                  RRCA                 CY: läs
6125  2E 00               LD   L,$00           från buffertens början
6127  45                  LD   B,L             B = 0: 256 bytes
6128  4D                  LD   C,L
6129  38 13               JR   C,L613E
612B  28 0D               JR   Z,L613A
612D  DB 01         L612D:  IN   A,($01)       skriv byte för byte, bit 0 = klar
612F  0F                  RRCA
6130  D2 2D 61            JP   NC,L612D
6133  ED A3               OUTI
6135  C2 2D 61            JP   NZ,L612D
6138  18 15               JR   L614F

613A  ED B3         L613A:  OTIR                 skriv blockvis
613C  18 11               JR   L614F

613E  28 0D         L613E:  JR   Z,L614D
6140  DB 01         L6140:  IN   A,($01)       läs byte för byte
6142  0F                  RRCA
6143  D2 40 61            JP   NC,L6140
6146  ED A2               INI
6148  C2 40 61            JP   NZ,L6140
614B  18 02               JR   L614F

614D  ED B2         L614D:  INIR                 läs blockvis
614F  25            L614F:  DEC  H               H tillbaka till buffertens sida
6150  C1                  POP  BC
6151  DB 01         L6151:  IN   A,($01)       vänta på bit 7
6153  07                  RLCA
6154  30 FB               JR   NC,L6151
6156  DB 00         L6156:  IN   A,($00)       status, 0 = rätt
6158  B7                  OR   A
6159  28 0D               JR   Z,L6168
615B  E5                  PUSH HL
615C  21 18 FD            LD   HL,$FD18
615F  35                  DEC  (HL)
6160  E1                  POP  HL
6161  28 04               JR   Z,L6167         försöken slut: fel
6163  D3 02               OUT  ($02),A         nytt kommando, försök igen
6165  18 83               JR   L60EA

6167  37            L6167:  SCF
6168  32 15 FD      L6168:  LD   (VFD15),A     status sparas (fel: CY = 1)
616B  E1                  POP  HL
616C  D1                  POP  DE
616D  C1                  POP  BC
616E  C9                  RET
```

SELCRD: *välj kortet för enheten i* DUNIT. *Kortadressen (kortbytens bitar 0-5) går till* VFD3F *och ut på* OUT $01, *och sedan väntar rutinen på bit 7 i* IN $01. *Ut: Z = kortet svarar, A = kortbyten, HL = posten i korttabellen; NZ = inget kort.*

```
616F  CD 84 61      SELCRD: CALL CRDTYP
6172  E6 3F               AND  $3F             kortadressen
6174  32 3F FD            LD   (VFD3F),A       kortet väljs
6177  D3 01               OUT  ($01),A
6179  E5                  PUSH HL
617A  2E 80               LD   L,$80           vänta på IN 1 bit 7
617C  65                  LD   H,L
617D  CD 96 61            CALL S6196
6180  E1                  POP  HL
6181  C0                  RET  NZ              NZ: inget kort
6182  7E                  LD   A,(HL)          Z: A = kortbyten
6183  C9                  RET
```

CRDTYP: *posten för enheten (*DUNIT, UNITNR*) i korttabellen CRDRAM.*

Enhetskoderna är DR0-2 = 0-2, HD0-3 = 4-7, MF0-2 = 8-A, MO0-1 = C-D, SF0-2 = $10-$12 *och UFD =* $1E. *UFD står för systemenheten* $FFF9.

Tabellen har 4 bytes per typ (typ = enhetskod / 4): kortbyte, flaggor och två bokstäver.

Typ	Kortbyte	Kortadress	Flaggor	Anmärkning
DR	08	-	00	hänvisning till MF
HD	$24	36	05	
MF	$6C	44	02	
MO	$AD	45	00	
SF	$6E	46	02	
??	00	-	00	typ 5 och 6, tomma
RM	00	-	02	typ 7 ($1C-$1F, UFD = $1E)

Kortbyte 0 = inget kort; RM har ingen enhetspost i DOSDEV.

Kortadressen är kortbytens bitar 0-5. Bitarna 6-7 anger sektornumreringen och överföringen (se SECIO*) och bitkartans sektor (*S6218*). Flaggornas bitar 0-2: klusterstorleken 2^n sektorer (*S6274 *ger n + 1).*

```
6184  21 E2 FD     CRDTYP: LD   HL,$FDE2
6187  CD B2 61             CALL UNITNR          A = enhetskoden
618A  E6 1C                AND  $1C             typ = enhetskod / 4
618C  20 01                JR   NZ,L618F        DR: hänvisning (byte 0 = 8)
618E  7E                   LD   A,(HL)
618F  85             L618F: ADD  A,L
6190  6F                   LD   L,A
6191  30 01                JR   NC,L6194
6193  24                   INC  H
6194  7E             L6194: LD   A,(HL)
6195  C9                   RET
```

S6196*: vänta tills (*IN $01 AND *L) = H. Högst 65536 varv; läses* $FF *eller 0 finns inget kort. Ut: Z = klart; NZ med A =* $FF *= inget kort eller tiden ute. BC behålls.*

```
6196  C5           S6196: PUSH BC
6197  01 00 00            LD   BC,$0000
619A  DB 01        L619A: IN   A,($01)
619C  3C                  INC  A               $FF: inget kort
619D  28 10               JR   Z,L61AF
619F  3D                  DEC  A
61A0  28 0D               JR   Z,L61AF
61A2  A5                  AND  L               (IN 1 & L) = H?
61A3  AC                  XOR  H
61A4  28 0A               JR   Z,L61B0
61A6  E3                  EX   (SP),HL         fördröjning
61A7  E3                  EX   (SP),HL
61A8  E3                  EX   (SP),HL
61A9  E3                  EX   (SP),HL
61AA  0B                  DEC  BC
61AB  78                  LD   A,B
61AC  B1                  OR   C
61AD  20 EB               JR   NZ,L619A
61AF  3D           L61AF: DEC  A               tiden ute: A = $FF, NZ
61B0  C1           L61B0: POP  BC
61B1  C9                  RET
```

UNITNR*: A = enhetskoden (*DUNIT AND $1F*). För UFD (*$1E*) blir det i stället den verkliga enheten ur* VFFF9*. Ut: Z = UFD.*

```
61B2  3A 01 FD     UNITNR: LD   A,(DUNIT)
61B5  E6 1F                AND  $1F
61B7  FE 1E                CP   $1E             UFD: systemenheten
61B9  C0                   RET  NZ
61BA  3A F9 FF             LD   A,(VFFF9)       Z: den verkliga enheten
61BD  C9                   RET
```

SLTBUF*: H = sidan för kanal B:s sektorbuffert,* DBUFP*:s höga byte + kanalen. In: B = kanal * 16. Ut: H och A = sidan; L behålls.*

```
61BE  E5           SLTBUF: PUSH HL
61BF  2A 12 FD             LD   HL,(DBUFP)      buffertarna börjar på DBUFP:s sida
61C2  78                   LD   A,B
61C3  CB 3F                SRL  A               kanal / 16 ...
61C5  0F                   RRCA
61C6  0F                   RRCA
61C7  0F                   RRCA
61C8  84                   ADD  A,H             ... = buffertens sida
```

```
61C9  E1              POP  HL
61CA  67              LD   H,A
61CB  C9              RET
```

LOADF: *ladda en fil i postformat. Varje post har:*

```
typbyte   $FF = resten av sektorn hoppas över; $00 = post
antal     (C)
(en byte, används inte)
adress hög, hög inverterad
adress låg, låg inverterad
data      C bytes
kontrollsumma
```

Kontrollsumman är summan av databytena; den kontrolleras med XOR. *Antal 0 avslutar filen, och HL = adressen blir startadressen.*

Fel 6 om formatet är felaktigt. LDRUN *hoppar sedan dit* (PUSH *HL*/RET).

```
61CC  5F          LOADF:  LD   E,A          DE = A (namnet)
61CD  16 00               LD   D,$00
61CF  06 00       S61CF:  LD   B,$00
61D1  CD 18 60            CALL DOPEN        öppna på kanal 0
61D4  3E 00               LD   A,$00        flaggorna kvar
61D6  D8                  RET  C            finns inte: CY, A = 0
61D7  06 00       LOADRC: LD   B,$00
61D9  CD 2D 60            CALL DRDREC       nästa post
61DC  2A 12 FD            LD   HL,(DBUFP)
61DF  2E 03               LD   L,$03        data från plats 3
61E1  7E          L61E1:  LD   A,(HL)
61E2  3C                  INC  A
61E3  28 F2               JR   Z,LOADRC     $FF: nästa sektor
61E5  3D                  DEC  A
61E6  20 23               JR   NZ,L620B
61E8  2C                  INC  L
61E9  4E                  LD   C,(HL)       C = antal
61EA  2C                  INC  L            byten efter antalet används inte
61EB  2C                  INC  L
61EC  56                  LD   D,(HL)
61ED  2C                  INC  L
61EE  7E                  LD   A,(HL)
61EF  2F                  CPL               adress hög, inverterad
61F0  BA                  CP   D
61F1  20 18               JR   NZ,L620B
61F3  2C                  INC  L
61F4  5E                  LD   E,(HL)
61F5  2C                  INC  L
61F6  7E                  LD   A,(HL)       adress låg, inverterad
61F7  2F                  CPL
61F8  AB                  XOR  E
61F9  20 10               JR   NZ,L620B
61FB  2C                  INC  L
61FC  0C                  INC  C
61FD  0D                  DEC  C
61FE  28 0F               JR   Z,L620F      antal 0: slut
6200  47                  LD   B,A          B = 0, summan i A
6201  86          L6201:  ADD  A,(HL)
6202  ED A0              LDI
6204  EA 01 62            JP   PE,L6201
6207  AE                  XOR  (HL)         kontrollsumman
6208  23                  INC  HL
6209  28 D6               JR   Z,L61E1
620B  3E 06       L620B:  LD   A,$06        fel 6
620D  37                  SCF
620E  C9                  RET

620F  EB          L620F:  EX   DE,HL        HL = startadressen
6210  AF                  XOR  A
6211  C9                  RET
```

LDRUN: *ladda filen med* LOADF *och hoppa till startadressen. Går det inte blir det kallstart* (RST *0*).

```
6212  CD CC 61     LDRUN:  CALL LOADF
6215  E5                   PUSH HL              hoppa dit
6216  D0                   RET  NC
6217  C7                   RST  $00             fel: kallstart
```

3.41 Skivans format (MF-diskett, 4 sektorer per kluster) $6218

S6218: bitkartan ligger i sektor 14 (6 för korttyp $80). Bytes 0-$EE: en bit per kluster, 1 = upptaget, högsta biten först. Byte $EF + i = antal använda poster i katalogsektor i; DFFIND hoppar över sektorer med 0.

S625E: katalogen är sektor 16-31 (DSYSUN på UFD); katalogsektorns index = DSLOT & $0F. 16 poster à 16 bytes:

```
+0,+1   bit 15-5 = filens första kluster (indexsektorn);
        +0 är den höga byten;
        bit 0 = skrivskyddad, bit 1 = får inte tas bort eller
        kortas
+2,+3   storlek i sektorer (låg först), indexsektorn medräknad
+4-+E   namn (8) och typ (3), +F = $FF
```

Ledig post = 16 bytes $FF.

Filens första sektor är indexsektorn: DSLOT, 0, 0, $FF och sedan utsträckningarna, 2 bytes var (hög byte först): bit 15-5 = första klustret, bit 4-0 = antal kluster - 1; slut $FF $FF. Varje datasektor börjar med DSLOT (postens nummer * 16 + katalogsektorns index) och posten + 1 (låg, hög), som i kanalposten +2/+3; data på plats 3-255.

```
6218  E5           S6218:  PUSH HL
6219  CD 84 61             CALL CRDTYP
621C  E1                   POP  HL
621D  11 0E 00             LD   DE,$000E        bitkartan: sektor 14
6220  E6 C0                AND  $C0
6222  FE 80                CP   $80
6224  C0                   RET  NZ
6225  1E 06                LD   E,$06           korttyp $80: sektor 6
6227  C9                   RET
```

GETBYT: A = byten vid DBUFP, som stegas. Bara den låga byten ökar, så pekaren stannar på samma sida. Övriga register behålls.

```
6228  E5           GETBYT: PUSH HL
6229  2A 12 FD             LD   HL,(DBUFP)      nästa byte i bufferten
622C  7E                   LD   A,(HL)
622D  2C                   INC  L
622E  22 12 FD             LD   (DBUFP),HL
6231  E1                   POP  HL
6232  C9                   RET
```

DOSFN1: katalogfunktion A (1-4, se DOSDSP). A, BC och DE sparas i DFNA, DFNBC och DFNDE, där katalogfunktionerna läser dem, och hämtas tillbaka efteråt. Ut: flaggorna från funktionen (CY = fel).

```
6233  ED 43 1D FD  DOSFN1: LD   (DFNBC),BC       argumenten sparas
6237  ED 53 1F FD          LD   (DFNDE),DE
623B  32 1C FD             LD   (DFNA),A
623E  CD 4D 62             CALL DOSDSP          funktion 1-4
6241  ED 4B 1D FD          LD   BC,(DFNBC)      och tillbaka
6245  ED 5B 1F FD          LD   DE,(DFNDE)
6249  3A 1C FD             LD   A,(DFNA)
624C  C9                   RET
```

DOSDSP: katalogfunktionerna, A = 1-4 (via DOSFN, som sparar A, BC och DE):

```
1  DFCRE   skapa katalogpost, indexsektor och första post;
           katalogsektorn skrivs om (prövar skrivskyddet)
2  DFCLS   stäng: skriv tillbaka katalogposten; storlek 0 = ta
           bort posten och frigör klustren (KILL)
3  DFFIND  sök filen i katalogen; postens byte 1 bit 0/1 blir
           kanalens bit 6/7
4  DFEXT   ta nya kluster ur bitkartan när filen växer
```

Anropen: FLUSH -> 2, LOOKUP -> 3, PREPF -> 1 om filen inte finns, WRREC -> 4.

```
624D  3D           DOSDSP: DEC  A
```

```
624E  CA E5 65            JP   Z,DFCRE          1 skapa
6251  3D                  DEC  A
6252  CA A5 67            JP   Z,DFCLS          2 stäng
6255  3D                  DEC  A
6256  CA BF 69            JP   Z,DFFIND         3 sök
6259  3D                  DEC  A
625A  CA 39 6B            JP   Z,DFEXT          4 förläng
625D  C9                  RET
```

S625E: *DE = katalogsektorn för filen i* DSLOT*: katalogens början (16, för UFD* DSYSUN*) + DSLOT* AND $0F*. Förstör A.*

```
625E  ED 5B F7 FF  S625E:  LD   DE,(DSYSUN)
6262  CD B2 61             CALL UNITNR
6265  28 03                JR   Z,L626A          Z: UFD, katalogen från DSYSUN
6267  11 10 00             LD   DE,$0010         katalogen från sektor 16
626A  3A 00 FD     L626A:  LD   A,(DSLOT)
626D  E6 0F                AND  $0F              + katalogsektorns index
626F  83                   ADD  A,E
6270  5F                   LD   E,A
6271  D0                   RET  NC
6272  14                   INC  D
6273  C9                   RET
```

S6274: *A = n + 1, där ett kluster är 2 upphöjt till n sektorer (flaggornas bitar 0-2 i kortposten). Övriga register behålls.*

```
6274  E5           S6274:  PUSH HL
6275  CD 84 61             CALL CRDTYP
6278  23                   INC  HL
6279  7E                   LD   A,(HL)           flaggorna
627A  E6 07                AND  $07              klusterstorleken
627C  3C                   INC  A
627D  E1                   POP  HL
627E  C9                   RET
```

SLTCPY: *kopiera kanalens 14 bytes i* SLTTAB *($FD40 + 16 * kanal) till* DSLOT *($FD00). Kanalposten:*

```
    +1     $FF = ledig, annars flaggor. Bit 5 = ändrad, bit 6 =
           skrivskyddad (fel 39), bit 7 = får inte tas bort eller
           kortas (fel 40). Låga 5 bitarna = enhetskoden.
    +0     DSLOT: postens nummer * 16 + katalogsektorns index
    +2/+3  aktuell post + 1 (0 = indexsektorn)
    +4/+5  första posten i den aktuella utsträckningen
    +6/+7  utsträckningsordet (hög byte först)
    +8/+9  indexsektorn
    +A/+B  antal dataposter = högsta värdet i +2/+3
    +E     index i bufferten
```

*Posterna räknas så att indexsektorn är 0: post n i BASIC har n + 1 i +2/+3 (*SETREC*), och +A/+B är katalogpostens storlek - 1.* RECSEC *($62B2) letar upp sektorn för den aktuella posten.*

```
627F  78           SLTCPY: LD   A,B
6280  E6 70                AND  $70              kanal * 16 ...
6282  C6 40                ADD  A,$40            ... + SLTTAB
6284  6F                   LD   L,A
6285  26 FD                LD   H,$FD
6287  11 00 FD            LD   DE,$FD00
628A  0E 0E                LD   C,$0E
628C  ED A0        COPY:   LDI
628E  AF                   XOR  A
628F  B1                   OR   C
6290  20 FA                JR   NZ,COPY
6292  C9                   RET
```

S6293: *läs filens indexsektor (*VFD08*) till kanal B:s buffert. Ut: NC och A = 0, eller CY och A = statusen.*

```
6293  ED 5B 08 FD  S6293:  LD   DE,(VFD08)
6297  CD 79 60             CALL S6079            läs indexsektorn
629A  D8                   RET  C                fel: A = statusen
629B  AF                   XOR  A
629C  C9                   RET
```

S629D: *HL = klusterordet (bit 15-5 = klustret) -> HL = klustrets första sektor.* S62AA: *HL := HL * 2 upphöjt till n (antal kluster -> antal sektorer). Båda förstör A.*

```
629D  CD 74 62     S629D:  CALL S6274
62A0  FE 06        L62A0:  CP   $06            HL >> (5 - n): klustret * 2^n
62A2  C8                   RET  Z
62A3  CB 3C                SRL  H
62A5  CB 1D                RR   L
62A7  3C                   INC  A
62A8  18 F6                JR   L62A0

62AA  CD 74 62     S62AA:  CALL S6274
62AD  3D           L62AD:  DEC  A
62AE  C8                   RET  Z
62AF  29                   ADD  HL,HL          HL << n
62B0  18 FB                JR   L62AD
```

RECSEC: *sektorn för kanal B:s aktuella post (kanalposten +2/+3). Kanalposten kopieras först till* DSLOT (SLTCPY). *Ligger posten inom den aktuella utsträckningen räknas sektorn ut direkt, kanalens storlek ökas till posten om den är större* (S62FF) *och indexet i bufferten nollas. Annars letar* L630E *i indexsektorn. In: B = kanal * 16. Ut: NC, DE = sektorn, H =* $FD*; eller CY = posten ligger efter den sista utsträckningen. Läsfel i indexsektorn går till fel 35/42.*

```
62B2  78           RECSEC: LD   A,B
62B3  32 14 FD             LD   (VFD14),A      kanalen undan
62B6  CD 7F 62             CALL SLTCPY         kanalposten till DSLOT
62B9  21 14 FD     L62B9:  LD   HL,$FD14
62BC  46                   LD   B,(HL)         B = kanal * 16
62BD  2A 02 FD             LD   HL,(VFD02)
62C0  ED 5B 04 FD          LD   DE,(VFD04)
62C4  B7                   OR   A
62C5  ED 52                SBC  HL,DE          post - utsträckningens första
62C7  EB                   EX   DE,HL
62C8  38 44                JR   C,L630E        före: sök i indexsektorn
62CA  3A 07 FD             LD   A,(VFD07)      kluster i utsträckningen
62CD  E6 1F                AND  $1F
62CF  3C                   INC  A
62D0  6F                   LD   L,A
62D1  26 00                LD   H,$00
62D3  CD AA 62             CALL S62AA          * sektorer per kluster
62D6  B7                   OR   A
62D7  2B                   DEC  HL
62D8  ED 52                SBC  HL,DE
62DA  38 32                JR   C,L630E        efter: sök i indexsektorn
62DC  D5                   PUSH DE
62DD  ED 5B 02 FD          LD   DE,(VFD02)
62E1  3E 4B                LD   A,$4B
62E3  CD FF 62             CALL S62FF          storleken := posten om större
62E6  3E 4E                LD   A,$4E
62E8  80                   ADD  A,B            SLTTAB+$E: index
62E9  6F                   LD   L,A
62EA  AF                   XOR  A
62EB  77                   LD   (HL),A         index i bufferten := 0
62EC  2E 06                LD   L,$06          utsträckningsordet (hög byte först)
62EE  56                   LD   D,(HL)
62EF  23                   INC  HL
62F0  7E                   LD   A,(HL)
62F1  E6 E0                AND  $E0            klustret ...
62F3  5F                   LD   E,A
62F4  EB                   EX   DE,HL
62F5  CD 9D 62             CALL S629D          ... som sektor
62F8  D1                   POP  DE
62F9  19                   ADD  HL,DE          + platsen i utsträckningen
62FA  EB                   EX   DE,HL
62FB  26 FD                LD   H,$FD
62FD  AF                   XOR  A
62FE  C9                   RET
```

S62FF: *ordet på* $FD00 *+ A + B - 1 (låg byte) och + A + B (hög) := DE, om DE är större eller lika. Förstör A, C och L; H =* $FD.

```
62FF  80           S62FF:  ADD  A,B
6300  6F                   LD   L,A
6301  26 FD                LD   H,$FD
6303  4E                   LD   C,(HL)          C = ordets höga byte
6304  7B                   LD   A,E
6305  2B                   DEC  HL
6306  96                   SUB  (HL)
6307  7A                   LD   A,D
6308  99                   SBC  A,C
6309  D8                   RET  C               DE mindre: kvar
630A  73                   LD   (HL),E          ordet := DE
630B  23                   INC  HL
630C  72                   LD   (HL),D
630D  C9                   RET
```

L630E: posten ligger utanför utsträckningen i +4..+7. Indexsektorn läses till kanal 0:s buffert, och utsträckningarna (plats 4 och framåt, 2 bytes var) räknas igenom tills posten hittas. Utsträckningen och dess första post läggs i DSLOT, som kopieras tillbaka till SLTTAB, och RECSEC räknar om. VFD15 lånas för den första posten.

```
630E  ED 5B 08 FD  L630E:  LD   DE,(VFD08)
6312  CD 77 60             CALL RDSEC           indexsektorn, kanal 0:s buffert
6315  DA C1 64             JP   C,L64C1         fel: 35 eller 42
6318  21 12 FD             LD   HL,$FD12
631B  36 04                LD   (HL),$04        utsträckningarna från plats 4
631D  ED 5B 02 FD          LD   DE,(VFD02)
6321  21 00 00             LD   HL,$0000        HL = första posten
6324  3A 12 FD   L6324:    LD   A,(DBUFP)
6327  B7                   OR   A
6328  28 49                JR   Z,L6373         slut på sektorn
632A  CD 0C 60             CALL DGETB           hög byte
632D  3C                   INC  A
632E  28 43                JR   Z,L6373         $FF: slut
6330  CD 0C 60             CALL DGETB           låg byte
6333  E6 1F                AND  $1F             antal kluster - 1
6335  3C                   INC  A
6336  D5                   PUSH DE
6337  5F                   LD   E,A
6338  16 00                LD   D,$00
633A  EB                   EX   DE,HL
633B  CD AA 62             CALL S62AA           * sektorer per kluster
633E  EB                   EX   DE,HL
633F  22 15 FD             LD   (VFD15),HL      (VFD15 lånas)
6342  19                   ADD  HL,DE           + sektorerna = nästas första post
6343  D1                   POP  DE
6344  7B                   LD   A,E
6345  95                   SUB  L
6346  7A                   LD   A,D
6347  9C                   SBC  A,H
6348  30 DA                JR   NC,L6324        posten ligger längre fram
634A  2A 15 FD             LD   HL,(VFD15)
634D  22 04 FD             LD   (VFD04),HL      första posten i utsträckningen
6350  21 12 FD             LD   HL,$FD12
6353  35                   DEC  (HL)            DBUFP tillbaka till ordet
6354  35                   DEC  (HL)
6355  CD 0C 60             CALL DGETB           utsträckningsordet ...
6358  2E 06                LD   L,$06
635A  77                   LD   (HL),A
635B  CD 0C 60             CALL DGETB
635E  2C                   INC  L
635F  77                   LD   (HL),A          ... i kanalens +6/+7
6360  3A 14 FD             LD   A,(VFD14)       kanalen
6363  C6 40                ADD  A,$40
6365  5F                   LD   E,A
6366  16 FD                LD   D,$FD
6368  21 00 FD             LD   HL,$FD00
636B  0E 0E                LD   C,$0E
636D  CD 8C 62             CALL COPY            tillbaka till SLTTAB
6370  C3 B9 62             JP   L62B9           räkna om
```

```
6373  21 14 FD     L6373:  LD   HL,$FD14
6376  46                   LD   B,(HL)
6377  AF                   XOR  A
6378  3F                   CCF                  CY = 1: posten finns inte
6379  C9                   RET

637A  44                   DEFB $44
```

Bytevis åtkomst i kanalens sektorbuffert (SLTBUF). Indexet står i SLTTAB+$E. BUFRD/BUFWR stegar indexet; BUFRD2/BUFWR2 använder C. Mellan rutinerna står fyra ensamma bytes som aldrig nås: $637A D, $6390 i, $639F a och $63B7 b = "Diab", en dold signatur (DIAB, Dataindustrier AB, konstruerade ABC800; DN 1981).

```
637B  E5           BUFRD:  PUSH HL
637C  CD BE 61             CALL SLTBUF
637F  E5                   PUSH HL
6380  78                   LD   A,B
6381  C6 4E                ADD  A,$4E           SLTTAB+$E: index
6383  6F                   LD   L,A
6384  26 FD                LD   H,$FD
6386  34                   INC  (HL)
6387  7E                   LD   A,(HL)
6388  E1                   POP  HL
6389  6F                   LD   L,A             byten vid det gamla indexet
638A  2D                   DEC  L
638B  FE 03                CP   $03             CY: bufferten slut
638D  7E                   LD   A,(HL)
638E  E1                   POP  HL
638F  C9                   RET

6390  69                   DEFB $69
6391  E5           BUFRD2: PUSH HL
6392  CD BE 61             CALL SLTBUF
6395  79                   LD   A,C
6396  C6 03                ADD  A,$03           plats C + 3
6398  6F                   LD   L,A
6399  DE 02                SBC  A,$02           C := C + 1
639B  4F                   LD   C,A
639C  7E                   LD   A,(HL)
639D  E1                   POP  HL
639E  C9                   RET

639F  61                   DEFB $61
63A0  E5           BUFWR:  PUSH HL
63A1  F5                   PUSH AF
63A2  78                   LD   A,B
63A3  E6 70                AND  $70
63A5  C6 4E                ADD  A,$4E
63A7  6F                   LD   L,A
63A8  26 FD                LD   H,$FD
63AA  34                   INC  (HL)
63AB  6E                   LD   L,(HL)          L = index + 1
63AC  CD BE 61             CALL SLTBUF
63AF  2D                   DEC  L
63B0  F1                   POP  AF
63B1  77                   LD   (HL),A
63B2  7D                   LD   A,L
63B3  C6 01                ADD  A,$01           Z, CY: sista platsen skriven
63B5  E1                   POP  HL
63B6  C9                   RET

63B7  62                   DEFB $62
63B8  E5           BUFWR2: PUSH HL
63B9  6F                   LD   L,A
63BA  CD BE 61             CALL SLTBUF
63BD  79                   LD   A,C
63BE  C6 03                ADD  A,$03           plats C + 3
63C0  4D                   LD   C,L
63C1  6F                   LD   L,A
63C2  71                   LD   (HL),C
63C3  DE 02                SBC  A,$02           C := C + 1
```

```
63C5  4F              LD   C,A
63C6  E1              POP  HL
63C7  C9              RET
```

NXTREC: *öka kanal B:s aktuella post (+2/+3). Förstör A och HL.*

```
63C8  78        NXTREC: LD   A,B
63C9  E6 70             AND  $70
63CB  C6 42             ADD  A,$42          SLTTAB+2/3: posten + 1
63CD  6F                LD   L,A
63CE  26 FD             LD   H,$FD
63D0  34                INC  (HL)
63D1  23                INC  HL
63D2  C0                RET  NZ
63D3  34                INC  (HL)
63D4  C9                RET
```

Ingångar till DERR med felindexet i C. De nås med JP från rutiner som anropats direkt via hopptabellen, så det enda på stacken är anroparens returadress. L63E9 tar först bort en returadress till (från S6473). De ensamma DEFB $21 gör att nästa LD C blir ett LD HL och hoppas över.

```
63D5  0E 00     L63D5:  LD   C,$00          fel 35
63D7  18 1C             JR   DERR

63D9  0E 10     L63D9:  LD   C,$10          fel 43 om skrivskyddad
63DB  3A 15 FD          LD   A,(VFD15)
63DE  E6 40             AND  $40
63E0  20 13             JR   NZ,DERR
63E2  0E 02             LD   C,$02          annars 36
63E4  21                DEFB $21            LD HL,$040E (överlappar nästa)
63E5  0E 04     L63E5:  LD   C,$04          fel 37
63E7  18 0C             JR   DERR

63E9  E1        L63E9:  POP  HL
63EA  0E 06     L63EA:  LD   C,$06          fel 38
63EC  21                DEFB $21            LD HL,$080E (överlappar nästa)
63ED  0E 08     L63ED:  LD   C,$08          fel 39
63EF  21                DEFB $21            LD HL,$0A0E (överlappar nästa)
63F0  0E 0A     L63F0:  LD   C,$0A          fel 40
63F2  21                DEFB $21            LD HL,$0E0E (överlappar nästa)
63F3  0E 0E     L63F3:  LD   C,$0E          fel 42
```

3.42 DOS-fel $63F5

C = felindex (jämnt). För C < $12 prövas först felvektorn DERRV + C (sätts med DSETEH; nollas när den används). Är den satt fortsätter koden där. Annars hoppar koden till DERRX (VFD33) med A = $23 + C / 2, alltså BASIC-felen 35-48:

```
35 (C=0)   läsfel (status <> 0 efter tre försök)
36 (C=2)   skrivfel
37 (C=4)   sektorns huvud stämmer inte med kanal och post
38 (C=6)   posten finns inte (bortom filens slut)
39 (C=8)   filen är skrivskyddad (kanalens bit 6)
40 (C=$A)  filen får inte tas bort eller kortas (bit 7)
42 (C=$E)  kortet svarar inte ($80)
43 (C=$10) skrivskydd (bit 6 i statusen VFD15)
44 (C=$12) kanalen är inte öppen
45 (C=$14) felaktigt kanalnummer
47 (C=$18) felvektorns index >= 9
```

DERRX är normalt DERRDV ($6CFE): stacken sätts tillbaka till DSAVSP, och drivrutinen returnerar med CY = 1. $6422 (LD C,$1A) nås inte; fel 48 kommer från katalogfunktionerna (DFNBC = $1A vid läsfel i katalogen eller bitkartan, $6669).

Felvägarna tar bort returadresserna inne i DOS:en (L6425 två, L6426 en) och sedan anroparens, så att felvektorn och DERRX får SP som före anropet till DOS:en. DERRX får dessutom anroparens returadress i DE.

```
63F5  79        DERR:   LD   A,C
63F6  C6 21             ADD  A,$21          felvektorn DERRV + C
```

```
63F8  6F                LD   L,A
63F9  26 FD             LD   H,$FD
63FB  AF                XOR  A
63FC  5E                LD   E,(HL)
63FD  77                LD   (HL),A          nollas när den hämtas
63FE  2C                INC  L
63FF  56                LD   D,(HL)
6400  77                LD   (HL),A
6401  7B                LD   A,E
6402  B2                OR   D
6403  E1                POP  HL              anroparens returadress bort
6404  28 30             JR   Z,L6436         ingen vektor
6406  D5                PUSH DE              hoppa till vektorn
6407  C9                RET
```

SLTCHK: *pröva kanalen. In: B = kanal * 16. Ut: A = kanalpostens flaggor (+1), HL pekar på dem. Fel 45 om B har andra bitar än kanalens, fel 44 om kanalen inte är öppen* ($FF).

```
6408  78          SLTCHK: LD   A,B
6409  E6 70               AND  $70
640B  B8                  CP   B             kanal * 16?
640C  20 0C               JR   NZ,L641A
640E  C6 41               ADD  A,$41         SLTTAB+1: flaggorna
6410  6F                  LD   L,A
6411  26 FD               LD   H,$FD
6413  7E                  LD   A,(HL)
6414  FE FF               CP   $FF           $FF: inte öppen
6416  C0                  RET  NZ            öppen
6417  0E 12               LD   C,$12         fel 44
6419  21                  DEFB $21           LD HL,$140E (överlappar nästa)
641A  0E 14       L641A:  LD   C,$14         fel 45
641C  18 08               JR   L6426

641E  0E 18       L641E:  LD   C,$18         fel 47
6420  18 05               JR   L6427

6422  0E 1A 2E            DEFB $0E,$1A,$2E   LD C,$1A (fel 48), nås inte
6425  E1          L6425:  POP  HL            två returadresser bort
6426  E1          L6426:  POP  HL            en returadress bort
6427  3A 15 FD    L6427:  LD   A,(VFD15)
642A  E6 40               AND  $40           statusbit 6: fel 43
642C  28 02               JR   Z,L6430
642E  0E 10               LD   C,$10
6430  79          L6430:  LD   A,C
6431  FE 12               CP   $12           fel 44-: utan felvektor
6433  38 C0               JR   C,DERR
6435  E1                  POP  HL            anroparens returadress
6436  EB          L6436:  EX   DE,HL         DE = returadressen
6437  2A 33 FD            LD   HL,(DERRX)    felutgången
643A  79                  LD   A,C           felindexet
643B  0F                  RRCA               C / 2
643C  C6 23               ADD  A,$23         A = $23 + C / 2 = 35 + C / 2
643E  E9                  JP   (HL)          DERRX
```

SETEH (DSETEH): *felvektor C (0-8) := DE. Vektor C gäller felindexet 2 * C, alltså fel 35 + C. Index 9 och större ger fel 47.*

```
643F  79          SETEH:  LD   A,C
6440  FE 09               CP   $09
6442  30 DA               JR   NC,L641E      index >= 9
6444  87                  ADD  A,A           DERRV + 2 * C
6445  C6 21               ADD  A,$21
6447  6F                  LD   L,A
6448  26 FD               LD   H,$FD
644A  73                  LD   (HL),E
644B  2C                  INC  L
644C  72                  LD   (HL),D
644D  C9                  RET
```

PRVREC: *välj posten före den aktuella.* SETREC: *välj post DE. Båda prövar kanalen B (kanal * 16) med* SLTCHK *(fel 45 för ett*

felaktigt B, 44 för en stängd kanal), lägger postnumret + 1 i kanalposten +2/+3, sätter indexet i bufferten till 3 och letar upp postens sektor med RECLFT *och* RECSEC*. In: B = kanal * 16, DE = posten (*SETREC*). Ut: CY om posten ligger efter filens slut.*

```
644E  CD 08 64     PRVREC: CALL SLTCHK
6451  2C                 INC  L              aktuell post ...
6452  5E                 LD   E,(HL)
6453  2C                 INC  L
6454  56                 LD   D,(HL)
6455  1B                 DEC  DE             ... - 1
6456  1B                 DEC  DE
6457  CD 08 64    SETREC: CALL SLTCHK         pröva kanalen
645A  13                 INC  DE             post 0 är indexsektorn med klusterlistan
645B  3E 42              LD   A,$42
645D  80                 ADD  A,B
645E  6F                 LD   L,A
645F  73                 LD   (HL),E         posten i kanalens +2/+3
6460  2C                 INC  L
6461  72                 LD   (HL),D
6462  CD 80 64           CALL RECLFT         saknas i ABC 800 M:s DOS (v.20)
6465  D4 B2 62           CALL NC,RECSEC      inom filen: sektorn
6468  F5                 PUSH AF             carry undan
6469  3E 4E              LD   A,$4E
646B  80                 ADD  A,B
646C  6F                 LD   L,A
646D  26 FD              LD   H,$FD
646F  36 03              LD   (HL),$03       index 3: förbi huvudet, första data
6471  F1                 POP  AF             carry från RECLFT eller RECSEC
6472  C9                 RET
```

S6473*: jämför DE med ordet på* $FD00 *+ A + B - 1 / + A + B. Är DE större eller lika blir det fel 38 (returadressen tas bort först), annars DE := DE + 1. Förstör A, C och L.*

```
6473  80          S6473: ADD  A,B
6474  6F                 LD   L,A
6475  4E                 LD   C,(HL)         C = ordets höga byte
6476  2D                 DEC  L
6477  7B                 LD   A,E
6478  96                 SUB  (HL)
6479  7A                 LD   A,D
647A  99                 SBC  A,C
647B  D2 E9 63           JP   NC,L63E9       för stort: fel 38
647E  13                 INC  DE             posten + 1
647F  C9                 RET
```

RECLFT*: In: B = kanal * 16. Ut: HL = storleken (+A/+B) - posten (+2/+3); CY = posten ligger efter filens slut. Förstör A och DE.*

```
6480  3E 42       RECLFT: LD   A,$42
6482  80                 ADD  A,B
6483  6F                 LD   L,A
6484  5E                 LD   E,(HL)         posten
6485  2C                 INC  L
6486  56                 LD   D,(HL)
6487  D5                 PUSH DE
6488  3E 4A              LD   A,$4A          storleken
648A  80                 ADD  A,B
648B  6F                 LD   L,A
648C  5E                 LD   E,(HL)
648D  2C                 INC  L
648E  56                 LD   D,(HL)
648F  E1                 POP  HL
6490  EB                 EX   DE,HL
6491  B7                 OR   A
6492  ED 52              SBC  HL,DE          CY: posten > storleken
6494  C9                 RET
```

RDREC*: läs kanal B:s aktuella post till kanalens buffert. Sektorns huvud (bytes 0-2) ska vara* DSLOT *och posten; annars fel 37. Sedan ökas posten (*NXTREC*), och indexet i bufferten står på 3, första databyten. Fel: 38 efter filens slut, 35 läsfel, 42 inget kort, 44*

och 45 från SLTCHK.

```
6495  CD 08 64     RDREC:  CALL SLTCHK
6498  CD 80 64             CALL RECLFT
649B  D4 B2 62             CALL NC,RECSEC
649E  DA EA 63             JP   C,L63EA          finns inte: fel 38
64A1  CD 79 60             CALL S6079            läs sektorn
64A4  38 1C                JR   C,L64C2          fel: 35 eller 42
64A6  CD 7B 63             CALL BUFRD            byte 0 (RECSEC nollade indexet)
64A9  2E 00                LD   L,$00
64AB  AE                   XOR  (HL)             huvudet: DSLOT ...
64AC  4F                   LD   C,A
64AD  CD 7B 63             CALL BUFRD
64B0  2E 02                LD   L,$02
64B2  AE                   XOR  (HL)             ... och posten
64B3  B1                   OR   C
64B4  4F                   LD   C,A
64B5  CD 7B 63             CALL BUFRD
64B8  2C                   INC  L
64B9  AE                   XOR  (HL)
64BA  B1                   OR   C
64BB  C2 E5 63             JP   NZ,L63E5         stämmer inte: fel 37
64BE  C3 C8 63             JP   NXTREC           nästa post

64C1  E1           L64C1:  POP  HL
64C2  E6 80        L64C2:  AND  $80              $80: fel 42, annars 35
64C4  CA D5 63             JP   Z,L63D5
64C7  C3 F3 63             JP   L63F3
```

WRREC*: skriv kanal B:s buffert (data på plats 3-255) som den aktuella posten. Kanalen märks som ändrad (bit 5), huvudet (*DSLOT *och posten) läggs i bytes 0-2, och sektorn skrivs. Sedan ökas posten. Ligger posten efter den sista utsträckningen tar* DFEXT *nya kluster, och skrivningen görs om. Fel: 39 skrivskyddad fil, 36 skrivfel, 43 skrivskyddad skiva, 42 inget kort, 44 och 45 från* SLTCHK *och felen från* DFEXT.

```
64CA  CD 08 64     WRREC:  CALL SLTCHK
64CD  07                   RLCA                  bit 6 till CY
64CE  07                   RLCA
64CF  DA ED 63             JP   C,L63ED          bit 6: fel 39
64D2  CB EE                SET  5,(HL)           bit 5: ändrad
64D4  CD B2 62     L64D4:  CALL RECSEC
64D7  38 1F                JR   C,L64F8          utanför: ta nytt kluster
64D9  2E 00                LD   L,$00
64DB  7E                   LD   A,(HL)           huvudet: DSLOT och posten
64DC  CD A0 63             CALL BUFWR
64DF  2E 02                LD   L,$02
64E1  7E                   LD   A,(HL)
64E2  CD A0 63             CALL BUFWR
64E5  2C                   INC  L
64E6  7E                   LD   A,(HL)
64E7  CD A0 63             CALL BUFWR
64EA  CD 86 60             CALL S6086            skriv sektorn
64ED  D2 C8 63             JP   NC,NXTREC        rätt: posten + 1
64F0  E6 80                AND  $80              $80: fel 42, annars 43/36
64F2  CA D9 63             JP   Z,L63D9
64F5  C3 F3 63             JP   L63F3

64F8  3E 04        L64F8:  LD   A,$04
64FA  CD 09 60             CALL DOSFN            DFEXT
64FD  DA 27 64             JP   C,L6427          fel från DFEXT
6500  18 D2                JR   L64D4
```

OPENQ *(*DOPENQ*) och* RUNFIL *(*DRUN*): ladda filen DE med* S61CF*, en ingång i* LOADF *som öppnar på kanal 0. In: DE = namnet, C = enheten (*$FF *= ingen angiven).* OPENQ*: Ut: NC, HL = startadressen; CY och A = 0 om filen inte finns. Felaktigt format ger fel 37.* RUNFIL *hoppar till startadressen (*XBYE *kör CMDINT.SYS så).*

```
6502  CD CF 61     OPENQ:  CALL S61CF
6505  D0                   RET  NC               laddad: HL = startadressen
6506  A7           L6506:  AND  A
6507  37                   SCF
6508  C8                   RET  Z                A = 0: CY
```

```
6509  C3 E5 63        JP   L63E5          A = 6 (formatet): fel 37

650C  CD CF 61  RUNFIL: CALL S61CF
650F  38 F5             JR   C,L6506       som OPENQ
6511  E9                JP   (HL)          kör filen
```

*OPENF (DOPEN), PREPF (DPREP) och CLOSEF (DCLOSE). In: B = kanal * 16, DE = namnet, C = enheten ($FF = ingen angiven). OPENF letar upp filen (LOOKUP); Ut: NC = öppnad, CY och Z = finns inte. PREPF väljer UFD om ingen enhet är angiven och DSYSUN är satt; finns filen öppnas den som den är, annars skapas den (DFCRE). CLOSEF stänger kanalen (FLUSH). Rutinerna gör CALL + RET i stället för JP, så att FLUSH alltid har samma antal returadresser över sig: felvägen L6425 tar bort två innan DERR tar anroparens.*

```
6512  CD 45 65  OPENF:  CALL LOOKUP
6515  C9                RET

6516  79        PREPF:  LD   A,C
6517  3C                INC  A
6518  20 09             JR   NZ,L6523      enhet angiven
651A  2A F7 FF          LD   HL,(DSYSUN)
651D  7C                LD   A,H
651E  B5                OR   L
651F  28 02             JR   Z,L6523       ingen UFD
6521  0E 1E             LD   C,$1E         UFD
6523  CD 45 65  L6523:  CALL LOOKUP
6526  D0                RET  NC            finns: öppnas, storleken kvar
6527  C2 27 64          JP   NZ,L6427      nås inte: LOOKUP ger bara NC eller Z
652A  3E 01             LD   A,$01
652C  CD 09 60          CALL DOSFN         DFCRE
652F  D0                RET  NC            skapad
6530  C3 27 64          JP   L6427

6533  CD 54 65  CLOSEF: CALL S6554
6536  C9                RET
```

DCLSAL (DCLALL): stäng kanal 7 till 1 och gå till XBYE, som kör CMDINT.SYS. S6554 ger samma stackdjup som i CLOSEF.

```
6537  06 70     DCLSAL: LD   B,$70         kanal 7 ...
6539  CD 54 65  L6539:  CALL S6554
653C  78                LD   A,B
653D  D6 10             SUB  $10           ... till 1
653F  47                LD   B,A
6540  20 F7             JR   NZ,L6539
6542  C3 37 6D          JP   XBYE          sedan BYE
```

LOOKUP: stäng kanal B (FLUSH) och sök filen DE (DFFIND). Hittas den öppnas den på kanalen. Ut: NC = öppnad, H = $FD; CY och Z = finns inte. Andra fel går till DERR.

```
6545  CD 58 65  LOOKUP: CALL FLUSH         stäng kanalen först
6548  3E 03             LD   A,$03
654A  CD 09 60          CALL DOSFN         DFFIND
654D  26 FD             LD   H,$FD
654F  D0                RET  NC
6550  C8                RET  Z             Z: finns inte
6551  C3 26 64          JP   L6426         annat fel

6554  CD 58 65  S6554:  CALL FLUSH
6557  C9                RET
```

*FLUSH: stäng kanal B (DFCLS). Kanal 0 och lediga kanaler hoppas över. Ändrade filer får katalogposten tillbaka; storlek 0 tar bort filen. In: B = kanal * 16 (andra bitar tas bort).*

```
6558  78        FLUSH:  LD   A,B
6559  E6 70             AND  $70
655B  C8                RET  Z             kanal 0: inget
655C  47                LD   B,A           B = kanal * 16
655D  C6 41             ADD  A,$41
655F  6F                LD   L,A
6560  26 FD             LD   H,$FD
6562  7E                LD   A,(HL)        flaggorna
6563  3C                INC  A
```

```
6564  C8              RET  Z             ledig
6565  3E 02           LD   A,$02
6567  CD 09 60        CALL DOSFN         DFCLS
656A  D0              RET  NC
656B  C3 25 64        JP   L6425         fel från DFCLS
```

SETEND (DSETEN): *gör post DE till filens sista, alltså storleken = DE + 1. D = $FF ger storleken 0. Posten måste ligga inom filen (fel 38); filen får inte vara skyddad (fel 40 och 39). Ut: DE = 0. Klustren frigörs när filen stängs.*

```
656E  CD 08 64  SETEND: CALL SLTCHK
6571  07                RLCA
6572  DA F0 63          JP   C,L63F0       bit 7: fel 40
6575  07                RLCA
6576  DA ED 63          JP   C,L63ED       bit 6: fel 39
6579  7A                LD   A,D
657A  3C                INC  A
657B  CC 90 65          CALL Z,S6590       D = $FF: DE := 0, storlek 0
657E  3E 4B             LD   A,$4B
6580  C4 73 64          CALL NZ,S6473      post >= storleken: fel 38
6583  3E 4A             LD   A,$4A
6585  80                ADD  A,B
6586  6F                LD   L,A
6587  73                LD   (HL),E        ny storlek: posten + 1
6588  2C                INC  L
6589  72                LD   (HL),D
658A  3E 41             LD   A,$41
658C  80                ADD  A,B
658D  6F                LD   L,A
658E  CB EE             SET  5,(HL)        ändrad
6590  11 00 00  S6590:  LD   DE,$0000      DE = 0
6593  C9        L6593:  RET                DNOP
```

L6594: NAME (*från* $6DA9): *B = kanalen för den gamla filen ur FCB+$15, sedan* DFNCX.

```
6594  DD 46 15  L6594:  LD   B,(IX+$15)
```

DFNCX (*DOS-funktion 8 och uppåt via* DFNJP, NAME): *byt namn på en fil. In: B = kanal * 16 för den gamla filen, DE = det nya namnet (11 tecken). Det nya namnet söks först med* OPENF *på kanal 0 och samma enhet; finns det redan blir det CY (BASIC ger fel 64). Annars läses den gamla filens katalogsektor, de 11 tecknen på postens +4 byts och sektorn skrivs. Skyddsbitarna prövas inte. Ut: NC = klart. Vid fel går bit 7 i statusen till* L63F3 *(fel 42), annat till* L63D9 *(43 eller 36, också vid läsfel). Förstör A, C, DE, HL.*

```
6597  D5        DFNCX:  PUSH DE
6598  C5                PUSH BC
6599  CD 08 64          CALL SLTCHK        kanalen med det gamla namnet
659C  E6 1F             AND  $1F
659E  4F                LD   C,A
659F  06 00             LD   B,$00
65A1  CD 12 65          CALL OPENF         sök det nya namnet på kanal 0
65A4  C1                POP  BC
65A5  3E FF             LD   A,$FF
65A7  32 41 FD          LD   (VFD41),A     kanal 0 ledig igen
65AA  38 03             JR   C,L65AF
65AC  D1                POP  DE            finns redan
65AD  37                SCF
65AE  C9                RET

65AF  CD 08 64  L65AF:  CALL SLTCHK
65B2  E6 1F             AND  $1F
65B4  32 01 FD          LD   (DUNIT),A
65B7  2B                DEC  HL
65B8  7E                LD   A,(HL)        den gamla filens DSLOT
65B9  32 00 FD          LD   (DSLOT),A
65BC  CD 5E 62          CALL S625E
65BF  CD 77 60          CALL RDSEC         läs katalogsektorn
65C2  38 17             JR   C,L65DB
65C4  7E                LD   A,(HL)
65C5  E6 F0             AND  $F0           postens plats
65C7  EB                EX   DE,HL
```

```
65C8  E3              EX   (SP),HL
65C9  EB              EX   DE,HL
65CA  2A 12 FD        LD   HL,(DBUFP)
65CD  C6 04           ADD  A,$04          namnet på +4
65CF  6F              LD   L,A
65D0  0E 0B           LD   C,$0B
65D2  EB              EX   DE,HL
65D3  CD 42 60        CALL DCOPY          11 tecken
65D6  D1              POP  DE
65D7  D5              PUSH DE
65D8  CD 84 60        CALL WRSEC          skriv tillbaka
65DB  D1        L65DB: POP  DE
65DC  D0              RET  NC
65DD  E6 80           AND  $80            $80: fel 42, annars 43/36
65DF  CA D9 63        JP   Z,L63D9
65E2  C3 F3 63        JP   L63F3
```

DFCRE (DOSFN 1, PREPARE *av en ny fil*): *C = enhet,* $FF *= den första som går, från UFD (om* DSYSUN *finns) och enhet 4 och uppåt till* $1C. *På varje enhet läses katalogsektorerna och den första skrivs tillbaka på prov; den första lediga posten (byte 0 =* $FF*) ger* DSLOT. *Antalet poster på* $EF*+i i bitkartan ökas,* S6BD1 *tar den längsta lediga följden, högst 16 kluster (minst 2 sektorer), som* DFCLS *sedan kortar till filens storlek. Sedan skrivs bitkartan, indexsektorn (*DSLOT*, 0, 0,* $FF*, utsträckningen,* $FF $FF*), den första datasektorn (*DSLOT*, 1, 0, sex nollor och ETX på plats 9, som när en tom fil stängs (*BUFXFR*, funktion 0)) och katalogposten (första klustret, storlek 0, namn och typ från* DFNDE*). Kanalposten får post 1, storlek 1 och buffertindex 3. Fel (A till* DFNBC*, CY): 46 ingen systemenhet, 41 alla kataloger fulla, 42 inget kort svarar, 43 skrivskydd, 36 skrivfel, 48 läsfel.*

```
65E5  CD F3 6A  DFCRE: CALL S6AF3          enhet (C = $FF: vilken som helst)
65E8  21 3E FD         LD   HL,$FD3E
65EB  36 00            LD   (HL),$00       bit 0: läst, bit 1: skrivbar
65ED  3E 16            LD   A,$16
65EF  DA 75 66         JP   C,L6675        ingen enhet: fel 46
65F2  AF               XOR  A
65F3  32 00 FD         LD   (DSLOT),A
65F6  CD 77 60  L65F6: CALL RDSEC          katalogsektor 16 (UFD: DSYSUN)
65F9  38 28            JR   C,L6623
65FB  CB C6            SET  0,(HL)
65FD  CD 86 60         CALL S6086          provskriv sektorn
6600  38 21            JR   C,L6623
6602  CB CE            SET  1,(HL)
6604  06 10            LD   B,$10          16 katalogsektorer
6606  18 07            JR   L660F

6608  C5        L6608: PUSH BC
6609  CD 77 60         CALL RDSEC
660C  C1               POP  BC
660D  38 55            JR   C,L6664
660F  0E 00     L660F: LD   C,$00
6611  CD BF 6B  L6611: CALL S6BBF          postens första byte
6614  3C               INC  A
6615  CA 7C 66         JP   Z,L667C        $FF: ledig post
6618  3E 0F            LD   A,$0F          nästa post
661A  81               ADD  A,C
661B  4F               LD   C,A
661C  30 F3            JR   NC,L6611
661E  13               INC  DE
661F  10 E7            DJNZ L6608
6621  18 0A            JR   L662D

6623  E6 C0     L6623: AND  $C0            skrivskydd, inget kort:
6625  28 3D            JR   Z,L6664
6627  3A 1D FD         LD   A,(DFNBC)      bestämd enhet: fel
662A  3C               INC  A
662B  20 37            JR   NZ,L6664
662D  CD B2 61  L662D: CALL UNITNR         nästa enhet
6630  3A 1D FD         LD   A,(DFNBC)
6633  20 05            JR   NZ,L663A
6635  3C               INC  A
6636  3E 04            LD   A,$04          efter UFD: från enhet 4
6638  28 07            JR   Z,L6641
663A  3C        L663A: INC  A
```

```
663B  20 1D              JR   NZ,L665A        bestämd enhet: fel 41
663D  3A 01 FD           LD   A,(DUNIT)
6640  3C                 INC  A
6641  11 10 00    L6641:  LD   DE,$0010
6644  FE 1D              CP   $1D             till och med $1C
6646  32 01 FD           LD   (DUNIT),A
6649  20 AB              JR   NZ,L65F6
664B  21 3E FD           LD   HL,$FD3E
664E  CB 46              BIT  0,(HL)
6650  3E 0E              LD   A,$0E           ingen läsbar: fel 42
6652  28 21              JR   Z,L6675
6654  CB 4E              BIT  1,(HL)
6656  3E 10              LD   A,$10           ingen skrivbar: fel 43
6658  28 1B              JR   Z,L6675
665A  3E 0C        L665A:  LD   A,$0C         katalogerna fulla: fel 41
665C  18 17              JR   L6675

665E  E6 80        L665E:  AND  $80
6660  3E 02              LD   A,$02           skrivfel: 36
6662  28 11              JR   Z,L6675
6664  3A 15 FD    L6664:  LD   A,(VFD15)
6667  E6 C0              AND  $C0
6669  3E 1A              LD   A,$1A           läsfel: 48
666B  28 08              JR   Z,L6675
666D  E6 40              AND  $40
666F  3E 10              LD   A,$10           skrivskydd: 43
6671  20 02              JR   NZ,L6675
6673  3E 0E              LD   A,$0E           svarar inte: 42
6675  32 1D FD    L6675:  LD   (DFNBC),A       felet, CY
6678  3E 81              LD   A,$81
667A  87                 ADD  A,A
667B  C9                 RET

667C  0D            L667C:  DEC  C            postens plats
667D  3E 10              LD   A,$10
667F  90                 SUB  B
6680  B1                 OR   C
6681  32 00 FD           LD   (DSLOT),A       | katalogsektorns index
6684  E6 0F              AND  $0F
6686  C6 EF              ADD  A,$EF           antalet poster i bitkartan
6688  4F                 LD   C,A
6689  C5                 PUSH BC
668A  06 01              LD   B,$01           räkna de använda + den nya
668C  0E 00              LD   C,$00
668E  CD BF 6B    L668E:  CALL S6BBF
6691  3C                 INC  A
6692  28 01              JR   Z,L6695
6694  04                 INC  B
6695  3E 0F       L6695:  LD   A,$0F
6697  81                 ADD  A,C
6698  4F                 LD   C,A
6699  30 F3              JR   NC,L668E
669B  60                 LD   H,B
669C  C1                 POP  BC
669D  CD 18 62           CALL S6218           bitkartan
66A0  CD B2 61           CALL UNITNR
66A3  20 05              JR   NZ,L66AA
66A5  ED 5B F7 FF        LD   DE,(DSYSUN)     UFD: sektorn före DSYSUN
66A9  1B                 DEC  DE
66AA  CD 77 60    L66AA:  CALL RDSEC
66AD  44                 LD   B,H
66AE  38 B4              JR   C,L6664
66B0  CD C8 6B           CALL S6BC8           nytt antal poster
66B3  CD B2 61           CALL UNITNR
66B6  20 0D              JR   NZ,L66C5
66B8  CD 84 60           CALL WRSEC           UFD: skriv, läs bitkartan
66BB  38 A7              JR   C,L6664
66BD  CD 18 62           CALL S6218
66C0  CD 77 60           CALL RDSEC
```

```
66C3  38 9F              JR   C,L6664
66C5  CD D1 6B     L66C5: CALL S6BD1          lediga kluster: VFD06, VFD08
66C8  DA 2D 66           JP   C,L662D         skivan full: nästa enhet
66CB  CD 74 62           CALL S6274           klusterstorlek 1:
66CE  3D                 DEC  A
66CF  20 08              JR   NZ,L66D9
66D1  3A 07 FD           LD   A,(VFD07)       minst 2 kluster
66D4  E6 1F              AND  $1F
66D6  CA 2D 66           JP   Z,L662D
66D9  CD 84 60     L66D9: CALL WRSEC          skriv bitkartan
66DC  DA AC 6B           JP   C,L6BAC
66DF  16 00              LD   D,$00           indexsektorn:
66E1  4A                 LD   C,D
66E2  21 00 FD           LD   HL,$FD00
66E5  46                 LD   B,(HL)          DSLOT, 0, 0, $FF
66E6  CD C8 6B           CALL S6BC8
66E9  42                 LD   B,D
66EA  CD C8 6B           CALL S6BC8
66ED  06 00              LD   B,$00
66EF  CD C8 6B           CALL S6BC8
66F2  05                 DEC  B
66F3  CD C8 6B           CALL S6BC8
66F6  2E 06              LD   L,$06           utsträckningen
66F8  46                 LD   B,(HL)
66F9  CD C8 6B           CALL S6BC8
66FC  2C                 INC  L
66FD  46                 LD   B,(HL)
66FE  CD C8 6B           CALL S6BC8
6701  06 FF              LD   B,$FF           $FF $FF: slut
6703  CD C8 6B           CALL S6BC8
6706  CD C8 6B           CALL S6BC8
6709  ED 5B 08 FD        LD   DE,(VFD08)      indexsektorn
670D  CD 84 60           CALL WRSEC
6710  DA 5E 66           JP   C,L665E
6713  0E 00              LD   C,$00           första datasektorn:
6715  2E 00              LD   L,$00
6717  46                 LD   B,(HL)          DSLOT, post 1
6718  CD C8 6B           CALL S6BC8
671B  06 01              LD   B,$01
671D  CD C8 6B           CALL S6BC8
6720  06 00              LD   B,$00
6722  CD C8 6B     L6722: CALL S6BC8          nollor till plats 8
6725  79                 LD   A,C
6726  FE 09              CP   $09
6728  20 F8              JR   NZ,L6722
672A  06 03              LD   B,$03           ETX på plats 9: tom fil, som från L49C3
672C  CD C8 6B           CALL S6BC8
672F  13                 INC  DE              sektorn efter indexsektorn
6730  CD 84 60           CALL WRSEC
6733  DA 5E 66           JP   C,L665E
6736  CD 5E 62           CALL S625E           katalogsektorn igen
6739  CD 77 60           CALL RDSEC
673C  DA 64 66           JP   C,L6664
673F  D5                 PUSH DE
6740  2E 00              LD   L,$00
6742  7E                 LD   A,(HL)
6743  E6 F0              AND  $F0             postens plats
6745  4F                 LD   C,A
6746  2A 06 FD           LD   HL,(VFD06)      första klustret,
6749  45                 LD   B,L
674A  CD C8 6B           CALL S6BC8
674D  3E E0              LD   A,$E0           utan flaggor
674F  A4                 AND  H
6750  47                 LD   B,A
6751  CD C8 6B           CALL S6BC8
6754  06 00              LD   B,$00           storlek 0
6756  CD C8 6B           CALL S6BC8
6759  CD C8 6B           CALL S6BC8
675C  2A 1F FD           LD   HL,(DFNDE)      namn och typ
```

```
675F  3E 0B              LD   A,$0B
6761  CD 99 67           CALL S6799
6764  06 FF              LD   B,$FF          $FF sist
6766  CD C8 6B           CALL S6BC8
6769  D1                 POP  DE
676A  CD 84 60           CALL WRSEC
676D  DA 64 66           JP   C,L6664
6770  21 00 00           LD   HL,$0000       kanalposten:
6773  22 04 FD           LD   (VFD04),HL
6776  22 0C FD           LD   (VFD0C),HL
6779  23                 INC  HL             storlek 1, post 1
677A  22 0A FD           LD   (VFD0A),HL
677D  22 02 FD           LD   (VFD02),HL
6780  21 0E FD           LD   HL,$FD0E
6783  36 03              LD   (HL),$03       buffertindex 3
6785  2E 01              LD   L,$01
6787  CB EE              SET  5,(HL)         ändrad
6789  3A 1E FD           LD   A,(VFD1E)      till kanalens post
678C  C6 40              ADD  A,$40
678E  5F                 LD   E,A
678F  16 FD              LD   D,$FD
6791  2D                 DEC  L
6792  0E 0F              LD   C,$0F          15 bytes
6794  CD 8C 62           CALL COPY
6797  AF                 XOR  A
6798  C9                 RET
```

S6799: *skriv A bytes från HL till bufferten från plats C med* S6BC8. *Ut: C ökad med A, HL efter de skrivna. Förstör A, B, E.*

```
6799  81           S6799: ADD  A,C            C+A: slut
679A  5F                  LD   E,A
679B  46           L679B: LD   B,(HL)         A bytes från HL
679C  CD C8 6B            CALL S6BC8
679F  23                  INC  HL
67A0  79                  LD   A,C
67A1  BB                  CP   E
67A2  20 F7               JR   NZ,L679B
67A4  C9                  RET
```

DFCLS (*DOS-funktion 2, stäng*): *In: B = kanal * 16. Kanalen märks ledig; är filen inte ändrad (bit 5 i +1) är det klart. Med* VFD0A *= 0 (*KILL *sätter det) tas filen bort: posten blir 16* $FF *och* S6900 *frigör alla kluster och minskar antalet poster på* $EF*+i. Annars läses indexsektorn i kanalens buffert och utsträckningarna summeras i sektorer (*S62AA*) tills de räcker till* VFD0A *+ 1. Den sista kortas till det som behövs, överskottet skrivs som en egen utsträckning efter den och frigörs med* S6900 *tillsammans med resten av indexet, och den kortade läggs tillbaka. Sedan skrivs* VFD0A *+ 1 som storlek i katalogposten (*S68E2*) och indexsektorn skrivs. Tar indexet slut innan blir summan storleken (*S68E6*). Prov på en MO-skiva (ett kluster per sektor): en fil med 42 dataposter fick 16, 16 och 16 kluster medan den skrevs och 16, 16 och 11 efter stängningen, med storleken 43. Felen: se* L68CE.

```
67A5  CD 7F 62     DFCLS: CALL SLTCPY         kanalposten till $FD00
67A8  11 F3 FF            LD   DE,$FFF3
67AB  19                  ADD  HL,DE
67AC  36 FF               LD   (HL),$FF       kanalen ledig
67AE  2E 01               LD   L,$01
67B0  7E                  LD   A,(HL)
67B1  E6 20               AND  $20            inte ändrad: klart
67B3  C8                  RET  Z
67B4  2E 0A               LD   L,$0A
67B6  7E                  LD   A,(HL)
67B7  2C                  INC  L
67B8  B6                  OR   (HL)
67B9  CA 99 68            JP   Z,L6899        storlek 0: ta bort filen
67BC  CD 93 62            CALL S6293          indexsektorn i kanalens buffert
67BF  DA D7 68            JP   C,L68D7
67C2  ED 5B 0A FD         LD   DE,(VFD0A)
67C6  0E 00               LD   C,$00
67C8  3E FF               LD   A,$FF          $FF på plats 3
67CA  CD B8 63            CALL BUFWR2
67CD  AF                  XOR  A
67CE  2E 0C               LD   L,$0C          VFD0C: sektorer hittills
```

```
67D0  77                LD   (HL),A
67D1  2C                INC  L
67D2  77                LD   (HL),A
67D3  CD 91 63    L67D3: CALL BUFRD2          utsträckningens höga byte
67D6  DA 84 68           JP   C,L6884
67D9  3C                INC  A
67DA  CA 84 68           JP   Z,L6884         $FF: slut
67DD  CD 91 63           CALL BUFRD2
67E0  E6 1F              AND  $1F             antal kluster
67E2  3C                INC  A
67E3  26 00              LD   H,$00
67E5  6F                LD   L,A
67E6  CD AA 62           CALL S62AA           i sektorer
67E9  E5                PUSH HL
67EA  D5                PUSH DE
67EB  ED 5B 0C FD        LD   DE,(VFD0C)
67EF  19                ADD  HL,DE
67F0  22 0C FD           LD   (VFD0C),HL
67F3  D1                POP  DE
67F4  37                SCF                  räcker till storleken + 1?
67F5  ED 52              SBC  HL,DE
67F7  E1                POP  HL
67F8  38 D9              JR   C,L67D3
67FA  EB                EX   DE,HL           sista utsträckningen:
67FB  2A 0C FD           LD   HL,(VFD0C)
67FE  B7                OR   A
67FF  ED 52              SBC  HL,DE
6801  22 0C FD           LD   (VFD0C),HL      sektorerna före den
6804  0D                DEC  C
6805  0D                DEC  C
6806  21 0E FD           LD   HL,$FD0E        dess plats i indexet
6809  71                LD   (HL),C
680A  CD 91 63           CALL BUFRD2
680D  2E 06              LD   L,$06           utsträckningsordet
680F  77                LD   (HL),A
6810  CD 91 63           CALL BUFRD2
6813  2C                INC  L
6814  77                LD   (HL),A
6815  2A 0A FD           LD   HL,(VFD0A)      sektorer som behövs - 1
6818  ED 5B 0C FD        LD   DE,(VFD0C)
681C  B7                OR   A
681D  ED 52              SBC  HL,DE
681F  CD 74 62           CALL S6274
6822  3D          L6822: DEC  A               / klusterstorleken
6823  28 06              JR   Z,L682B
6825  CB 3C              SRL  H
6827  CB 1D              RR   L
6829  18 F7              JR   L6822

682B  4D          L682B: LD   C,L
682C  21 07 FD           LD   HL,$FD07
682F  7E                LD   A,(HL)
6830  E6 1F              AND  $1F             kluster - 1 mot de behövda
6832  91                SUB  C
6833  28 59              JR   Z,L688E         lika: inget att korta
6835  57                LD   D,A             D = överskottet
6836  7E                LD   A,(HL)
6837  E6 E0              AND  $E0
6839  B1                OR   C               kortad utsträckning
683A  77                LD   (HL),A
683B  E6 E0              AND  $E0
683D  5F                LD   E,A
683E  2D                DEC  L
683F  7E                LD   A,(HL)
6840  2E 10              LD   L,$10           $FD10 = höga byten
6842  77                LD   (HL),A
6843  3E 20       L6843: LD   A,$20           första överblivna klustret
6845  83                ADD  A,E
6846  5F                LD   E,A
```

```
6847  3E 00              LD   A,$00
6849  8E                 ADC  A,(HL)
684A  77                 LD   (HL),A
684B  79                 LD   A,C
684C  D6 01              SUB  $01
684E  4F                 LD   C,A
684F  30 F2              JR   NC,L6843
6851  7A                 LD   A,D              överskottet som utsträckning
6852  B3                 OR   E
6853  3D                 DEC  A
6854  5F                 LD   E,A
6855  7E                 LD   A,(HL)
6856  2E 0E              LD   L,$0E
6858  4E                 LD   C,(HL)
6859  CD B8 63           CALL BUFWR2           i indexet
685C  7B                 LD   A,E
685D  CD B8 63           CALL BUFWR2
6860  CD 00 69           CALL S6900            frigör det
6863  DA D7 68           JP   C,L68D7
6866  2E 0E              LD   L,$0E            den kortade utsträckningen
6868  4E                 LD   C,(HL)
6869  2E 06              LD   L,$06
686B  7E                 LD   A,(HL)
686C  CD B8 63           CALL BUFWR2
686F  2C                 INC  L
6870  7E                 LD   A,(HL)
6871  CD B8 63           CALL BUFWR2
6874  C5           L6874: PUSH BC
6875  CD E2 68           CALL S68E2            storleken + 1 i katalogen
6878  C1                 POP  BC
6879  ED 5B 08 FD        LD   DE,(VFD08)       skriv indexsektorn
687D  D4 86 60           CALL NC,S6086
6880  38 55              JR   C,L68D7
6882  AF                 XOR  A
6883  C9                 RET

6884  2A 0C FD     L6884: LD   HL,(VFD0C)       indexet slut: alla sektorer
6887  CD E6 68           CALL S68E6
688A  38 4B              JR   C,L68D7
688C  AF                 XOR  A
688D  C9                 RET

688E  2E 0E        L688E: LD   L,$0E           nästa utsträckning
6890  34                 INC  (HL)
6891  34                 INC  (HL)
6892  CD 00 69           CALL S6900            frigör resten
6895  38 40              JR   C,L68D7
6897  18 DB              JR   L6874

6899  CD 5E 62     L6899: CALL S625E           ta bort: katalogsektorn
689C  CD 79 60           CALL S6079
689F  38 36              JR   C,L68D7
68A1  3A 01 FD           LD   A,(DUNIT)        skrivskyddad, får inte tas bort
68A4  E6 C0              AND  $C0
68A6  20 26              JR   NZ,L68CE
68A8  CD BE 61           CALL SLTBUF
68AB  3A 00 FD           LD   A,(DSLOT)        posten
68AE  E6 F0              AND  $F0
68B0  6F                 LD   L,A
68B1  3E 10              LD   A,$10
68B3  36 FF        L68B3: LD   (HL),$FF        16 $FF: ledig
68B5  2C                 INC  L
68B6  3D                 DEC  A
68B7  20 FA              JR   NZ,L68B3
68B9  CD 86 60           CALL S6086            skriv katalogsektorn
68BC  38 19              JR   C,L68D7
68BE  CD 93 62           CALL S6293            indexsektorn
68C1  38 14              JR   C,L68D7
68C3  3E 01              LD   A,$01            från första utsträckningen
68C5  32 0E FD           LD   (VFD0E),A
```

```
68C8  CD 00 69            CALL S6900            frigör alla kluster
68CB  38 0A               JR   C,L68D7
68CD  C9                  RET
```

Felutgångarna i DFCLS (L68CE, L68D2, L68D7 *och* $68DD) *lägger felindexet i C, men* L6675 *lagrar A i* DFNBC. *Felet blir därför det som A är: vid* L68D7 *statusen och* $80, *alltså index 0 (fel 35, också vid skrivfel) eller* $80 *(99;* FLUSH *ger ändå 43 för en skrivskyddad skiva, eftersom* L6425 *prövar det först). ABC 800 M:s DOS (v.20) laddar A.*

```
68CE  0E 0A       L68CE:  LD   C,$0A            fel 40 avsett, men C används inte
68D0  18 0D               JR   L68DF

68D2  E1          L68D2:  POP  HL
68D3  0E 14               LD   C,$14            fel 45 avsett (kanal 0)
68D5  18 08               JR   L68DF

68D7  E6 80       L68D7:  AND  $80              A = statusen från SECIO och $80 (fel)
68D9  0E 0E               LD   C,$0E            fel 42 avsett
68DB  20 02               JR   NZ,L68DF         bit 7: kortet svarar inte
68DD  0E 1C               LD   C,$1C            fel 49 avsett (ABC 800 M: LD A,$1A, 48)
68DF  C3 75 66    L68DF:  JP   L6675            DFNBC = A, inte C
```

S68E2: *skriv* VFD0A + 1 *som storlek i katalogpostens +2/+3;* S68E6 *skriver HL. Katalogsektorn läses (*S625E, RDSEC*) och skrivs tillbaka. Ut: CY = fel. Förstör A, BC, DE.*

```
68E2  2A 0A FD    S68E2:  LD   HL,(VFD0A)       storleken + 1
68E5  23                  INC  HL
68E6  CD 5E 62    S68E6:  CALL S625E            katalogsektorn
68E9  CD 77 60            CALL RDSEC
68EC  D8                  RET  C
68ED  3A 00 FD            LD   A,(DSLOT)
68F0  E6 F0               AND  $F0
68F2  C6 02               ADD  A,$02            postens +2,+3
68F4  4F                  LD   C,A
68F5  45                  LD   B,L
68F6  CD C8 6B            CALL S6BC8
68F9  44                  LD   B,H
68FA  CD C8 6B            CALL S6BC8
68FD  C3 84 60            JP   WRSEC            skriv
```

S6900: *frigör utsträckningarna i kanal B:s indexsektor från platsen i* VFD0E *(räknad som i* BUFRD2*) till slutet (*$FF*) och skriv* $FF $FF *i deras ställe. Bitarna nollställs i bitkartan i kanal 0:s buffert, som sedan skrivs. Anropad från* $68C8 (KILL *i* DFCLS, *returadressen* $68CB*) minskas först antalet poster på* $EF+*i (för UFD i sektorn före* DSYSUN*). Kanal 0 går till* L68D2. *Ut: CY = fel, B = kanalen, H =* $FD. *Förstör A, C, DE, L.*

```
6900  CD 18 62    S6900:  CALL S6218            bitkartan
6903  3A 0E FD            LD   A,(VFD0E)        plats i indexet
6906  32 12 FD            LD   (DBUFP),A
6909  E1                  POP  HL               returadressen: anropad från $68C8?
690A  E5                  PUSH HL
690B  7D                  LD   A,L
690C  FE CB               CP   $CB              retur $68CB? (ABC 800 M: $0D)
690E  20 2B               JR   NZ,L693B         inte bort: bara bitkartan
6910  CD B2 61            CALL UNITNR           ta bort: minska antalet poster
6913  20 05               JR   NZ,L691A
6915  ED 5B F7 FF         LD   DE,(DSYSUN)
6919  1B                  DEC  DE
691A  CD 77 60    L691A:  CALL RDSEC
691D  D8                  RET  C
691E  21 00 FD            LD   HL,$FD00
6921  7E                  LD   A,(HL)
6922  E6 0F               AND  $0F
6924  C6 EC               ADD  A,$EC            $EF + katalogsektorns index
6926  4F                  LD   C,A
6927  CD 91 63            CALL BUFRD2
692A  3D                  DEC  A
692B  0D                  DEC  C
692C  CD B8 63            CALL BUFWR2
692F  CD B2 61            CALL UNITNR
6932  20 0B               JR   NZ,L693F
```

```
6934  CD 84 60           CALL WRSEC          UFD: skriv, läs bitkartan
6937  D8                 RET  C
6938  CD 18 62           CALL S6218
693B  CD 77 60     L693B:  CALL RDSEC          bitkartan till kanal 0
693E  D8                 RET  C
693F  21 1E FD     L693F:  LD   HL,$FD1E       kanalen
6942  46                 LD   B,(HL)
6943  78                 LD   A,B
6944  B7                 OR   A
6945  28 8B              JR   Z,L68D2
6947  21 12 FD           LD   HL,$FD12
694A  4E                 LD   C,(HL)
694B  CD 91 63           CALL BUFRD2         utsträckningen
694E  38 5C              JR   C,L69AC
6950  3C                 INC  A
6951  28 59              JR   Z,L69AC        $FF: slut
6953  D6 04              SUB  $04            bitkartans byte - 3 (BUFRD2 läser plats C + 3)
6955  2E 10              LD   L,$10
6957  77                 LD   (HL),A
6958  CD 91 63           CALL BUFRD2
695B  2C                 INC  L
695C  77                 LD   (HL),A
695D  E6 1F              AND  $1F
695F  2E 0F              LD   L,$0F          $FD0F: kluster - 1
6961  77                 LD   (HL),A
6962  2E 12              LD   L,$12
6964  4E                 LD   C,(HL)
6965  3E FF              LD   A,$FF          $FF $FF i stället
6967  CD B8 63           CALL BUFWR2
696A  3E FF              LD   A,$FF
696C  CD B8 63           CALL BUFWR2
696F  71                 LD   (HL),C
6970  06 00              LD   B,$00
6972  2E 11              LD   L,$11
6974  7E                 LD   A,(HL)         bitens nummer
6975  E6 E0              AND  $E0
6977  07                 RLCA
6978  07                 RLCA
6979  07                 RLCA
697A  4F                 LD   C,A
697B  16 7F              LD   D,$7F          masken
697D  79           L697D:  LD   A,C
697E  D6 01              SUB  $01
6980  38 0C              JR   C,L698E
6982  4F                 LD   C,A
6983  7A                 LD   A,D
6984  0F                 RRCA
6985  57                 LD   D,A
6986  18 F5              JR   L697D

6988  CD B9 69     L6988:  CALL S69B9          nästa byte
698B  34                 INC  (HL)
698C  16 7F              LD   D,$7F
698E  2E 10        L698E:  LD   L,$10
6990  4E                 LD   C,(HL)
6991  CD 91 63           CALL BUFRD2
6994  A2           L6994:  AND  D              nollställ biten
6995  5F                 LD   E,A
6996  2E 0F              LD   L,$0F
6998  7E                 LD   A,(HL)
6999  D6 01              SUB  $01            fler kluster?
699B  38 09              JR   C,L69A6
699D  77                 LD   (HL),A
699E  7A                 LD   A,D
699F  0F                 RRCA
69A0  57                 LD   D,A
69A1  7B                 LD   A,E
69A2  30 E4              JR   NC,L6988
69A4  18 EE              JR   L6994
```

```
69A6  7B          L69A6:  LD   A,E
69A7  CD B9 69            CALL S69B9
69AA  18 93               JR   L693F

69AC  CD 18 62    L69AC:  CALL S6218          skriv bitkartan
69AF  CD 84 60            CALL WRSEC
69B2  21 1E FD            LD   HL,$FD1E
69B5  46                  LD   B,(HL)
69B6  26 FD               LD   H,$FD
69B8  C9                  RET
```

S69B9: *skriv A tillbaka till bitkartans byte* ($FD10) *med* BUFWR2. *Förstör C.*

```
69B9  2E 10       S69B9:  LD   L,$10          byten tillbaka
69BB  4E                  LD   C,(HL)
69BC  C3 B8 63            JP   BUFWR2
```

DFFIND (*DOS-funktion 3, öppna en befintlig fil*): *C = enhet* ($FF = *vilken som helst*, S6AF3), DFNDE *pekar på namnet* (*11 tecken*). *Är H = 0 i* DFNDE *är L en* DSLOT *och posten tas direkt; det kräver en bestämd enhet, annars fel 46. Annars söks UFD först* (*om* DSYSUN *finns*) *och sedan enhet 4 till och med* $1B. *Antalen på* $EF+*i kopieras till* $FDC0, *och katalogsektorer med 0 poster hoppas över. Svarar en enhet inte* (*bit 7 i statusen*) *prövas nästa; andra fel ger 48. När posten hittas kopieras 15 bytes till kanalens post:* DSLOT, DUNIT *med skyddsbitarna* (*postens +1 bit 1-0 till bit 7-6*), VFD06 = *postens +0*, VFD07 = *första utsträckningens låga byte ur indexet*, VFD08 = *indexsektorn*, VFD0A = *storleken - 1, post 1 och buffertindex 3. Stämmer inte indexsektorns* DSLOT *blir det fel 48. Ut: NC och A = 0 = öppnad; CY och Z = finns inte* (*också för ett tomt namn och en ledig post*); *fel 46 och 48 med A i* DFNBC *och CY.*

```
69BF  CD F3 6A    DFFIND: CALL S6AF3          enhet (C = $FF: vilken som helst)
69C2  DA 60 6A            JP   C,L6A60        ingen enhet: fel 46
69C5  2A 1F FD            LD   HL,(DFNDE)
69C8  7C                  LD   A,H
69C9  B7                  OR   A
69CA  CA 68 6A            JP   Z,L6A68        H = 0: DSLOT i L
69CD  7E                  LD   A,(HL)
69CE  C6 01               ADD  A,$01
69D0  D8                  RET  C              tomt namn: CY
69D1  D5          L69D1:  PUSH DE
69D2  CD 18 62            CALL S6218          bitkartan (inte på UFD)
69D5  CD B2 61            CALL UNITNR
69D8  C4 77 60            CALL NZ,RDSEC
69DB  D1                  POP  DE
69DC  38 56               JR   C,L6A34
69DE  21 C0 FD            LD   HL,$FDC0       antalet poster per katalogsektor
69E1  01 EF 10            LD   BC,$10EF       till $FDC0
69E4  CD BF 6B    L69E4:  CALL S6BBF
69E7  77                  LD   (HL),A
69E8  23                  INC  HL
69E9  10 F9               DJNZ L69E4
69EB  AF                  XOR  A
69EC  32 00 FD            LD   (DSLOT),A
69EF  21 C0 FD    L69EF:  LD   HL,$FDC0       katalogsektorns antal
69F2  E6 0F               AND  $0F
69F4  85                  ADD  A,L
69F5  6F                  LD   L,A
69F6  30 01               JR   NC,L69F9
69F8  24                  INC  H
69F9  7E          L69F9:  LD   A,(HL)
69FA  B7                  OR   A
69FB  28 2C               JR   Z,L6A29        0: hoppa över
69FD  CD 77 60            CALL RDSEC
6A00  38 32               JR   C,L6A34
6A02  AF                  XOR  A
6A03  E6 F0       L6A03:  AND  $F0             postens namn ...
6A05  C6 04               ADD  A,$04
6A07  4F                  LD   C,A
6A08  C6 0B               ADD  A,$0B
6A0A  47                  LD   B,A
6A0B  2A 1F FD            LD   HL,(DFNDE)
6A0E  CD BF 6B    L6A0E:  CALL S6BBF          ... mot DFNDE, 11 tecken
```

```
6A11  BE                CP   (HL)
6A12  20 0C             JR   NZ,L6A20
6A14  23                INC  HL
6A15  79                LD   A,C
6A16  B8                CP   B
6A17  20 F5             JR   NZ,L6A0E
6A19  79                LD   A,C            lika: postens plats
6A1A  3D                DEC  A
6A1B  E6 F0             AND  $F0
6A1D  4F                LD   C,A
6A1E  18 68             JR   L6A88

6A20  21 00 FD    L6A20: LD   HL,$FD00     nästa post
6A23  3E 10             LD   A,$10
6A25  86                ADD  A,(HL)
6A26  77                LD   (HL),A
6A27  30 DA             JR   NC,L6A03
6A29  13          L6A29: INC  DE           nästa katalogsektor
6A2A  21 00 FD          LD   HL,$FD00
6A2D  34                INC  (HL)
6A2E  7E                LD   A,(HL)         16 sektorer?
6A2F  C6 F0             ADD  A,$F0
6A31  30 BC             JR   NC,L69EF
6A33  B7                OR   A
6A34  3A 15 FD    L6A34: LD   A,(VFD15)    inte funnen eller fel:
6A37  B7                OR   A
6A38  28 04             JR   Z,L6A3E
6A3A  E6 80             AND  $80            bit 7 (inte redo): nästa enhet
6A3C  28 25             JR   Z,L6A63        annat fel: 48
6A3E  CD B2 61    L6A3E: CALL UNITNR
6A41  3A 1D FD          LD   A,(DFNBC)
6A44  20 05             JR   NZ,L6A4B
6A46  3C                INC  A
6A47  3E 04             LD   A,$04          efter UFD: från enhet 4
6A49  28 07             JR   Z,L6A52
6A4B  3C          L6A4B: INC  A
6A4C  20 0F             JR   NZ,L6A5D       bestämd enhet: finns inte
6A4E  3A 01 FD          LD   A,(DUNIT)
6A51  3C                INC  A
6A52  FE 1C       L6A52: CP   $1C           till och med $1B (DFCRE: $1C)
6A54  32 01 FD          LD   (DUNIT),A
6A57  11 10 00          LD   DE,$0010
6A5A  C2 D1 69          JP   NZ,L69D1
6A5D  AF          L6A5D: XOR  A             Z, CY: finns inte
6A5E  37                SCF
6A5F  C9                RET
```

L6A60*: fel 46 (ingen enhet),* L6A63*: fel 48 (läsfel eller fel indexsektor), båda till* L6675*. Byten* $01 *på* $6A62 *gör nästa* LD A *till en* LD BC *som hoppas över.*

```
6A60  3E 16       L6A60: LD   A,$16
6A62  01                 DEFB $01           LD BC,$1A3E (överlappar nästa)
6A63  3E 1A       L6A63: LD   A,$1A
6A65  C3 75 66           JP   L6675

6A68  3E 01       L6A68: LD   A,$01         med DSLOT:
6A6A  80                 ADD  A,B
6A6B  38 F3              JR   C,L6A60       vilken enhet som helst: fel 46
6A6D  3A 1F FD           LD   A,(DFNDE)
6A70  21 00 FD           LD   HL,$FD00
6A73  77                 LD   (HL),A        DSLOT := L
6A74  CD 5E 62           CALL S625E
6A77  CD 77 60           CALL RDSEC         katalogsektorn
6A7A  38 B8              JR   C,L6A34
6A7C  7E                 LD   A,(HL)
6A7D  E6 F0              AND  $F0
6A7F  4F                 LD   C,A
6A80  5F                 LD   E,A
6A81  CD BF 6B           CALL S6BBF
```

```
6A84  C6 01              ADD  A,$01          ledig: CY
6A86  D8                 RET  C
6A87  4B                 LD   C,E
6A88  CD BF 6B     L6A88: CALL S6BBF         katalogpostens +0
6A8B  57                 LD   D,A
6A8C  21 06 FD           LD   HL,$FD06
6A8F  77                 LD   (HL),A
6A90  CD BF 6B           CALL S6BBF          +1
6A93  5F                 LD   E,A
6A94  0F                 RRCA                bit 1-0 ...
6A95  0F                 RRCA
6A96  E6 C0              AND  $C0
6A98  2E 01              LD   L,$01          ... till DUNIT bit 7-6
6A9A  B6                 OR   (HL)
6A9B  77                 LD   (HL),A
6A9C  3E E0              LD   A,$E0          första klustret
6A9E  A3                 AND  E
6A9F  5F                 LD   E,A
6AA0  EB                 EX   DE,HL
6AA1  CD 9D 62           CALL S629D          i sektorer
6AA4  22 08 FD           LD   (VFD08),HL     indexsektorn
6AA7  CD BF 6B           CALL S6BBF          storleken - 1
6AAA  6F                 LD   L,A
6AAB  CD BF 6B           CALL S6BBF
6AAE  67                 LD   H,A
6AAF  2B                 DEC  HL
6AB0  22 0A FD           LD   (VFD0A),HL
6AB3  EB                 EX   DE,HL
6AB4  21 00 00           LD   HL,$0000       post 1
6AB7  22 04 FD           LD   (VFD04),HL
6ABA  23                 INC  HL
6ABB  22 02 FD           LD   (VFD02),HL
6ABE  21 08 FD           LD   HL,$FD08
6AC1  5E                 LD   E,(HL)
6AC2  2C                 INC  L
6AC3  56                 LD   D,(HL)
6AC4  CD 77 60           CALL RDSEC          läs indexsektorn
6AC7  DA 34 6A           JP   C,L6A34
6ACA  2E 00              LD   L,$00
6ACC  0E 00              LD   C,$00
6ACE  CD BF 6B           CALL S6BBF          DSLOT stämmer?
6AD1  BE                 CP   (HL)
6AD2  20 8F              JR   NZ,L6A63       nej: fel 48
6AD4  0E 05              LD   C,$05          första utsträckningen:
6AD6  CD BF 6B           CALL S6BBF
6AD9  2E 07              LD   L,$07          VFD07 (VFD06 = katalogens +0)
6ADB  77                 LD   (HL),A
6ADC  3E 03              LD   A,$03          buffertindex 3
6ADE  2E 0E              LD   L,$0E
6AE0  77                 LD   (HL),A
6AE1  3A 1E FD           LD   A,(VFD1E)      till kanalens post
6AE4  C6 40              ADD  A,$40
6AE6  5F                 LD   E,A
6AE7  16 FD              LD   D,$FD
6AE9  21 00 FD           LD   HL,$FD00
6AEC  0E 0F              LD   C,$0F
6AEE  CD 8C 62           CALL COPY
6AF1  AF                 XOR  A
6AF2  C9                 RET
```

S6AF3: *välj den första enheten för* DFCRE *och* DFFIND. *In: C = enhet,* $FF *= vilken som helst,* $1E *= UFD. Ut: B = det givna C,* DUNIT *satt, DE = första katalogsektorn (16), NC. Med* $FF *börjar sökningen på UFD om* DSYSUN *finns, annars på enhet 4. För UFD läses sektorn före* DSYSUN *(huvudet med antalen) till kanal 0:s buffert, och DE blir* DSYSUN*; huvudet prövas (bytes 0, 5 och 6 ska tillsammans ge 0 med* XOR*). CY bara för UFD, när* DSYSUN *= 0 eller huvudet är fel. Ett läsfel ger inte CY (*CCF, RET NC*) utan NC med DE =* DSYSUN *- 1; felet kommer när anroparen läser den sektorn. Förstör A.*

```
6AF3  41           S6AF3: LD   B,C           B = C
6AF4  0C                  INC  C
```

```
6AF5  28 05              JR   Z,L6AFC          $FF: vilken som helst
6AF7  0D                 DEC  C
6AF8  79                 LD   A,C
6AF9  E6 1F              AND  $1F              annars enheten
6AFB  4F                 LD   C,A
6AFC  79           L6AFC: LD   A,C
6AFD  ED 5B F7 FF        LD   DE,(DSYSUN)
6B01  FE 1E              CP   $1E              UFD?
6B03  28 0B              JR   Z,L6B10
6B05  78                 LD   A,B
6B06  3C                 INC  A
6B07  79                 LD   A,C
6B08  20 27              JR   NZ,L6B31         bestämd enhet
6B0A  7A                 LD   A,D
6B0B  B3                 OR   E
6B0C  3E 04              LD   A,$04            ingen UFD: från enhet 4
6B0E  28 21              JR   Z,L6B31
6B10  7A           L6B10: LD   A,D
6B11  B3                 OR   E
6B12  37                 SCF                   DSYSUN = 0: CY
6B13  C8                 RET  Z
6B14  3E 1E              LD   A,$1E
6B16  32 01 FD           LD   (DUNIT),A
6B19  1B                 DEC  DE               sektorn före katalogen
6B1A  CD 77 60           CALL RDSEC
6B1D  3F                 CCF                   läsfel: NC, DE = DSYSUN - 1
6B1E  D0                 RET  NC
6B1F  13                 INC  DE
6B20  2A 12 FD           LD   HL,(DBUFP)
6B23  2E 00              LD   L,$00            byte 0 ^ 5 ^ 6 = 0?
6B25  7E                 LD   A,(HL)
6B26  2C                 INC  L
6B27  2C                 INC  L
6B28  2C                 INC  L
6B29  2C                 INC  L
6B2A  2C                 INC  L
6B2B  AE                 XOR  (HL)
6B2C  2C                 INC  L
6B2D  AE                 XOR  (HL)
6B2E  C8                 RET  Z
6B2F  37                 SCF                   nej: CY
6B30  C9                 RET

6B31  11 10 00     L6B31: LD   DE,$0010        katalogen från sektor 16
6B34  32 01 FD           LD   (DUNIT),A
6B37  B7                 OR   A
6B38  C9                 RET
```

DFEXT (*DOS-funktion 4, från* WRREC *när filen växer*): *In: B = kanal * 16. Indexsektorn läses och slutet* ($FF) *söks från plats 4; går platsen runt till 0 är indexet fullt (fel 41).* S6BD1 *tar den längsta lediga följden, högst 16 kluster, bitkartan skrivs, och den nya utsträckningen skrivs i indexet följd av* $FF $FF (*inte på den sista platsen*). *Kanalen märks ändrad (bit 5). Ut: NC = klart; fel med A i* DFNBC *och CY: 41 (indexet eller skivan full), 35 läsfel och 36 skrivfel i indexsektorn, 43, 48 eller 42 för bitkartan* (L6BAC).

```
6B39  CD 7F 62     DFEXT: CALL SLTCPY          kanalposten till $FD00
6B3C  21 09 FD           LD   HL,$FD09
6B3F  56                 LD   D,(HL)
6B40  2D                 DEC  L
6B41  5E                 LD   E,(HL)
6B42  CD 77 60           CALL RDSEC            indexsektorn
6B45  38 5E              JR   C,L6BA5
6B47  0E 04              LD   C,$04            utsträckningarna från plats 4
6B49  CD BF 6B     L6B49: CALL S6BBF
6B4C  FE FF              CP   $FF              $FF: slut
6B4E  28 07              JR   Z,L6B57
6B50  CD BF 6B           CALL S6BBF
6B53  20 F4              JR   NZ,L6B49         plats 0: indexet fullt
6B55  18 4B              JR   L6BA2

6B57  0D           L6B57: DEC  C               slutets plats
```

```
6B58  2E 0E              LD   L,$0E
6B5A  71                 LD   (HL),C
6B5B  CD 18 62           CALL S6218          bitkartan
6B5E  CD 77 60           CALL RDSEC
6B61  38 49              JR   C,L6BAC
6B63  CD D1 6B           CALL S6BD1          lediga kluster: VFD06
6B66  38 3A              JR   C,L6BA2        skivan full: fel 41
6B68  CD 84 60           CALL WRSEC          skriv bitkartan
6B6B  38 3F              JR   C,L6BAC
6B6D  21 08 FD           LD   HL,$FD08
6B70  5E                 LD   E,(HL)
6B71  2C                 INC  L
6B72  56                 LD   D,(HL)
6B73  CD 77 60           CALL RDSEC          indexsektorn igen
6B76  38 2D              JR   C,L6BA5
6B78  2E 0E              LD   L,$0E
6B7A  4E                 LD   C,(HL)
6B7B  2E 06              LD   L,$06          den nya utsträckningen
6B7D  46                 LD   B,(HL)
6B7E  CD C8 6B           CALL S6BC8
6B81  2C                 INC  L
6B82  46                 LD   B,(HL)
6B83  CD C8 6B           CALL S6BC8
6B86  28 08              JR   Z,L6B90        sista platsen: utan slut
6B88  06 FF              LD   B,$FF          $FF $FF
6B8A  CD C8 6B           CALL S6BC8
6B8D  CD C8 6B           CALL S6BC8
6B90  CD 84 60     L6B90: CALL WRSEC         skriv indexsektorn
6B93  38 13              JR   C,L6BA8
6B95  3A 1E FD           LD   A,(VFD1E)      kanalen ändrad
6B98  C6 41              ADD  A,$41
6B9A  6F                 LD   L,A
6B9B  26 FD              LD   H,$FD
6B9D  7E                 LD   A,(HL)
6B9E  F6 20              OR   $20
6BA0  77                 LD   (HL),A
6BA1  C9                 RET

6BA2  3E 0C        L6BA2: LD   A,$0C         fel 41
6BA4  01                 DEFB $01            LD BC,$003E (överlappar nästa)
6BA5  3E 00        L6BA5: LD   A,$00         läsfel: 35
6BA7  01                 DEFB $01            LD BC,$023E (överlappar nästa)
6BA8  3E 02        L6BA8: LD   A,$02         skrivfel: 36
6BAA  18 10              JR   L6BBC

6BAC  47           L6BAC: LD   B,A           bitkartan:
6BAD  E6 40              AND  $40
6BAF  3E 10              LD   A,$10          skrivskydd: 43
6BB1  20 09              JR   NZ,L6BBC
6BB3  78                 LD   A,B
6BB4  E6 80              AND  $80
6BB6  3E 1A              LD   A,$1A          läsfel: 48
6BB8  28 02              JR   Z,L6BBC
6BBA  3E 0E              LD   A,$0E          svarar inte: 42
6BBC  C3 75 66     L6BBC: JP   L6675
```

S6BBF: *A = byte C i bufferten som* DBUFP *pekar ut;* S6BC8: *byte C = B. Platsen räknas från buffertens början (inte + 3 som i* BUFRD2*), och C ökas; Z när den går runt till 0. Andra register orörda.*

```
6BBF  E5           S6BBF: PUSH HL            A := buffert[C], C+1
6BC0  2A 12 FD            LD   HL,(DBUFP)
6BC3  69                  LD   L,C
6BC4  7E                  LD   A,(HL)
6BC5  0C                  INC  C
6BC6  E1                  POP  HL
6BC7  C9                  RET

6BC8  E5           S6BC8: PUSH HL            buffert[C] := B, C+1
6BC9  2A 12 FD            LD   HL,(DBUFP)
6BCC  69                  LD   L,C
```

```
6BCD  70                LD   (HL),B
6BCE  0C                INC  C
6BCF  E1                POP  HL
6BD0  C9                RET
```

S6BD1: *hitta lediga kluster i bitkartan (kanal 0:s buffert, bytes 0-$EE, 0 = ledig, byte 0 bit 7 = kluster 0). Den första längsta lediga följden tas, högst 16 kluster; en följd på 16 tas genast. Utsträckningen läggs i VFD06 (byten) och VFD07 (biten * 32 + antalet - 1) och bitarna sätts; med DFNA = 1 (DFCRE) blir VFD08 indexsektorn (det första klustret i sektorer, S629D). Slingan över en följd prövar inte slutet $EF, men skivorna har $FF där (MO-skivan i bytes $50-$EE). Ut: CY = inget ledigt; annars A = 0 och DE = bitkartans sektor (S6218). Förstör BC, HL.*

```
6BD1  1E 00       S6BD1:  LD   E,$00        första längsta lediga följd
6BD3  26 FF               LD   H,$FF        H = $FF: ingen än
6BD5  06 80               LD   B,$80        från byte 0, bit 7
6BD7  0E 01               LD   C,$01
6BD9  E5                  PUSH HL
6BDA  E5                  PUSH HL
6BDB  E1          L6BDB:  POP  HL
6BDC  E1                  POP  HL
6BDD  0D          L6BDD:  DEC  C
6BDE  79                  LD   A,C
6BDF  FE EF               CP   $EF          byte $EF: slut
6BE1  30 3B               JR   NC,L6C1E
6BE3  E5                  PUSH HL
6BE4  CD BF 6B    L6BE4:  CALL S6BBF
6BE7  6F                  LD   L,A
6BE8  7D          L6BE8:  LD   A,L
6BE9  A0                  AND  B            ledig bit?
6BEA  28 0E               JR   Z,L6BFA
6BEC  0F                  RRCA
6BED  47                  LD   B,A
6BEE  30 F8               JR   NC,L6BE8
6BF0  06 80               LD   B,$80
6BF2  79                  LD   A,C
6BF3  FE EF               CP   $EF
6BF5  20 ED               JR   NZ,L6BE4
6BF7  E1                  POP  HL
6BF8  18 24               JR   L6C1E

6BFA  7D          L6BFA:  LD   A,L
6BFB  C5                  PUSH BC
6BFC  16 00               LD   D,$00        D = följdens längd
6BFE  6F          L6BFE:  LD   L,A
6BFF  7D          L6BFF:  LD   A,L
6C00  A0                  AND  B
6C01  20 11               JR   NZ,L6C14
6C03  14                  INC  D            ledig: räkna
6C04  7A                  LD   A,D
6C05  FE 10               CP   $10          högst 16 kluster
6C07  28 0F               JR   Z,L6C18
6C09  CB 08               RRC  B
6C0B  30 F2               JR   NC,L6BFF
6C0D  06 80               LD   B,$80
6C0F  CD BF 6B            CALL S6BBF
6C12  18 EA               JR   L6BFE

6C14  7B          L6C14:  LD   A,E          längre än den bästa?
6C15  BA                  CP   D
6C16  30 C3               JR   NC,L6BDB
6C18  5A          L6C18:  LD   E,D
6C19  E1                  POP  HL
6C1A  E3                  EX   (SP),HL
6C1B  E1                  POP  HL
6C1C  20 BF               JR   NZ,L6BDD
6C1E  3E FF       L6C1E:  LD   A,$FF        ingen ledig: CY
6C20  BC                  CP   H
6C21  37                  SCF
6C22  C8                  RET  Z
6C23  85                  ADD  A,L
```

```
6C24  6F              LD   L,A
6C25  47              LD   B,A
6C26  E5              PUSH HL
6C27  0E 00           LD   C,$00
6C29  7C        L6C29: LD  A,H          bitens nummer * 32
6C2A  07              RLCA
6C2B  38 07           JR   C,L6C34
6C2D  67              LD   H,A
6C2E  3E 20           LD   A,$20
6C30  81              ADD  A,C
6C31  4F              LD   C,A
6C32  18 F5           JR   L6C29

6C34  7B        L6C34: LD  A,E          kluster - 1
6C35  3D              DEC  A
6C36  B1              OR   C
6C37  21 06 FD        LD   HL,$FD06     utsträckningen till VFD06
6C3A  70              LD   (HL),B
6C3B  2C              INC  L
6C3C  77              LD   (HL),A
6C3D  3A 1C FD        LD   A,(DFNA)
6C40  FE 01           CP   $01          DFCRE:
6C42  20 08           JR   NZ,L6C4C
6C44  60              LD   H,B
6C45  69              LD   L,C
6C46  CD 9D 62        CALL S629D        indexsektorn
6C49  22 08 FD        LD   (VFD08),HL
6C4C  E1        L6C4C: POP HL
6C4D  54              LD   D,H
6C4E  4D              LD   C,L
6C4F  CD BF 6B  L6C4F: CALL S6BBF       märk följden upptagen
6C52  47              LD   B,A
6C53  0D              DEC  C
6C54  7A        L6C54: LD  A,D
6C55  B0              OR   B
6C56  47              LD   B,A
6C57  1D              DEC  E
6C58  28 06           JR   Z,L6C60
6C5A  CB 0A           RRC  D
6C5C  30 F6           JR   NC,L6C54
6C5E  16 80           LD   D,$80
6C60  CD C8 6B  L6C60: CALL S6BC8
6C63  7B              LD   A,E
6C64  B7              OR   A
6C65  20 E8           JR   NZ,L6C4F
6C67  CD 18 62        CALL S6218        DE = bitkartan
6C6A  AF              XOR  A
6C6B  C9              RET
```

DOSSTA (DOSINI): *DOS:ens start, anropad av* EXTSCN *vid RESET (se texten vid* $6000*). Kopierar den blanka enhetsposten, satstabellens huvud och* CRDTAB *till RAM (*$FDD0-$FE02*), länkar in enheterna före de gamla i* DEVLST *och satserna före de gamla i* STKWP*, lägger stacken under buffertarna (SP =* $F500*) och nollställer DOS:en (*DOSRST*). Finns BASICINI.SYS läses dess post 0: bytes 6-21 blir felnumren i* INIBUF*, och börjar sektorn med* JP *på byte 3 körs den på* $F503 *innan RET går tillbaka. Ut: HL =* $6F00*. Förstör alla.*

```
6C6C  21 F6 FF  DOSSTA: LD  HL,$FFF6
6C6F  7E              LD   A,(HL)
6C70  FE A5           CP   $A5          $A5: varmstart
6C72  36 00           LD   (HL),$00     $FFF6 nollas alltid
6C74  28 05           JR   Z,L6C7B
6C76  23              INC  HL           kallstart: DSYSUN := 0
6C77  AF              XOR  A
6C78  77              LD   (HL),A
6C79  23              INC  HL
6C7A  77              LD   (HL),A       $FFF8 := 0
6C7B  11 D0 FD  L6C7B: LD  DE,$FDD0     den blanka enhetsposten till DEVLNK2
6C7E  01 08 00        LD   BC,$0008     8 bytes
6C81  21 35 6E        LD   HL,$6E35
6C84  ED B0           LDIR
```

```
6C86  21 A2 6F            LD   HL,$6FA2        DOSSTH till DOSSTR
6C89  0E 0A               LD   C,$0A           10 bytes
6C8B  ED B0               LDIR
6C8D  21 DB 6E            LD   HL,$6EDB        CRDTAB till $FDE2
6C90  0E 21               LD   C,$21           33 bytes, sist $C9 = kroken VFE02
6C92  ED B0               LDIR
6C94  2A 7B FF            LD   HL,(DEVLST)     den gamla listan ...
6C97  22 D0 FD            LD   (DEVLNK2),HL    ... efter $FDD0
6C9A  2A 35 6E            LD   HL,(DOSDEV)
6C9D  22 7B FF            LD   (DEVLST),HL     DEVLST := DR0 ($6E3D)
6CA0  2A 7D FF            LD   HL,(STKWP)      de gamla satserna ...
6CA3  22 D8 FD            LD   (DOSSTR),HL     ... efter DOSSTR
6CA6  21 D8 FD            LD   HL,$FDD8
6CA9  22 7D FF            LD   (STKWP),HL      STKWP := DOSSTR
6CAC  21 00 FB            LD   HL,$FB00
6CAF  22 81 FF            LD   (SYSOPT),HL     SYSOPT := $FB00
6CB2  21 FC 6E            LD   HL,$6EFC        UNSVEC-målet
6CB5  22 88 FF            LD   (VFF88),HL
6CB8  21 00 F5            LD   HL,$F500
6CBB  D1                  POP  DE              återhoppet
6CBC  F9                  LD   SP,HL           stacken under buffertarna
6CBD  22 12 FD            LD   (DBUFP),HL      DBUFP := $F500: kanal 0:s buffert
6CC0  D5                  PUSH DE
6CC1  CD E6 6C            CALL DOSRST          DERRX := $F500 tills OPNINI sätter sin
6CC4  11 BD 6E            LD   DE,$6EBD        BASICINI.SYS
6CC7  01 FF 00            LD   BC,$00FF        C = $FF: vilken enhet som helst
6CCA  AF                  XOR  A               A = 0: post 0
6CCB  CD FA 6D            CALL OPNINI
6CCE  20 12               JR   NZ,L6CE2        ingen fil: inga feltexter
6CD0  E5                  PUSH HL
6CD1  01 10 00            LD   BC,$0010
6CD4  11 09 FE            LD   DE,$FE09        bytes 6-21 till INIBUF
6CD7  2E 06               LD   L,$06           HL = $F506, sektorns byte 6
6CD9  ED B0               LDIR
6CDB  E1                  POP  HL              HL = $F503, byte 3
6CDC  7E                  LD   A,(HL)
6CDD  FE C3               CP   $C3
6CDF  20 01               JR   NZ,L6CE2        börjar BASICINI med JP?
6CE1  E5                  PUSH HL              JP: $F503 körs vid RET
6CE2  21 00 6F     L6CE2: LD   HL,$6F00        EXTSCN fortsätter på $7000
6CE5  C9                  RET                  till EXTSCN, eller först till $F503
```

DOSRST: *nollställ DOS:en.* DERRX *:= HL, alla kanaler lediga* ($FF *i* $FD41-$FDB1*, från kanal 0:s +1 till kanal 7:s +1*) *och felvektorerna 0-8 := 0* (DSETEH)*. In: HL = felrutinen. Förstör A, BC, DE, HL.*

```
6CE6  22 33 FD     DOSRST: LD   (DERRX),HL
6CE9  21 41 FD             LD   HL,$FD41        SLTTAB+1 ...
6CEC  06 71                LD   B,$71           113 bytes, till kanal 7:s +1
6CEE  36 FF        L6CEE:  LD   (HL),$FF        ... $FF: lediga
6CF0  23                   INC  HL
6CF1  10 FB                DJNZ L6CEE
6CF3  0E 08                LD   C,$08           felvektorerna 8-0 := 0
6CF5  50                   LD   D,B             DE = 0
6CF6  58                   LD   E,B
6CF7  CD 45 60     L6CF7:  CALL DSETEH
6CFA  0D                   DEC  C
6CFB  F8                   RET  M
6CFC  18 F9                JR   L6CF7
```

DERRDV: *DOS:ens felrutin* (DERRX) *medan* DOSDRV *arbetar. SP sätts till* DSAVSP*, så att RET går direkt till den som anropade* DOSDRV*, med CY och A = felet. Är kanalen inte ledig och felet inte 37 tas bit 5 (ändrad) bort ur kanalens flaggbyte, så att filen inte skrivs tillbaka när den stängs. In: A = felet, IX = FCB.*

```
6CFE  DD 5E 15     DERRDV: LD   E,(IX+$15)      kanal * 16
6D01  16 00                LD   D,$00
6D03  21 41 FD             LD   HL,$FD41
6D06  19                   ADD  HL,DE
6D07  56                   LD   D,(HL)          flaggbyten, $FF = ledig
```

```
6D08  14                INC  D
6D09  ED 7B 07 FE       LD   SP,(DSAVSP)     SP som när DOSDRV anropades
6D0D  37                SCF
6D0E  C8                RET  Z               ledig: CY till DOSDRV:s anropare
6D0F  FE 25             CP   $25             fel 37: CY
6D11  37                SCF
6D12  C8                RET  Z
6D13  CB AE             RES  5,(HL)          annars: inte ändrad
6D15  C9                RET                  CY från SCF
```

ABORT: felrutinen efter XBYE. Skriver CR LF Abort och felnumret med två siffror (A & $7F; 100-127 får :, ; eller < som tiotal) och går sedan in i XBYE utan att ta bort siffrorna från stacken. In: A = felet.

```
6D16  F5          ABORT:  PUSH AF
6D17  21 D3 6E            LD   HL,$6ED3       CR LF Abort
6D1A  01 08 00            LD   BC,$0008
6D1D  CD 0B 00            CALL PRTXT
6D20  F1                  POP  AF
6D21  0E 2F               LD   C,$2F          tiotal i C
6D23  E6 7F               AND  $7F
6D25  0C          L6D25:  INC  C
6D26  D6 0A               SUB  $0A
6D28  30 FB               JR   NC,L6D25
6D2A  C6 3A               ADD  A,$3A          ental i B
6D2C  47                  LD   B,A
6D2D  C5                  PUSH BC             tiotal vid SP, ental vid SP+1
6D2E  01 02 00            LD   BC,$0002
6D31  60                  LD   H,B
6D32  68                  LD   L,B
6D33  39                  ADD  HL,SP          HL = SP: de två siffrorna
6D34  CD 0B 00            CALL PRTXT
```

XBYE (BYE): nollställer DOS:en med ABORT som felrutin och kör CMDINT.SYS (DRUN, DE = $6EC8). Finns filen inte blir det RST 0; ett skivfel går till ABORT, som anropar XBYE igen.

```
6D37  21 16 6D    XBYE:   LD   HL,ABORT(egenhet)
6D3A  CD E6 6C            CALL DOSRST
6D3D  11 C8 6E            LD   DE,$6EC8       CMDINT.SYS
6D40  CD 1E 60            CALL DRUN
6D43  C7                  RST  $00            finns inte: kallstart
```

*FRESLT: sök en ledig kanal 1-7 (flaggbyte $FF). Kanalen * 16 hamnar i FCB+$15. Finns ingen blir det fel 19 (ERRJP), och sökningen görs om. In: IX = FCB. Ut: B = kanal * 16, C = enhetskoden, A = 0, Z. Förstör HL.*

```
6D44  21 51 FD    FRESLT: LD   HL,$FD51       SLTTAB+1 för kanal 1
6D47  01 10 00            LD   BC,$0010
6D4A  7E          L6D4A:  LD   A,(HL)
6D4B  3C                  INC  A
6D4C  28 0D               JR   Z,L6D5B        $FF: ledig
6D4E  09                  ADD  HL,BC
6D4F  7D                  LD   A,L
6D50  FE C1               CP   $C1            efter kanal 7
6D52  20 F6               JR   NZ,L6D4A
6D54  3E 13               LD   A,$13          fel 19: ingen ledig kanal
6D56  CD 33 00            CALL ERRJP
6D59  18 E9               JR   FRESLT

6D5B  7D          L6D5B:  LD   A,L            kanal * 16
6D5C  D6 41               SUB  $41
6D5E  DD 77 15            LD   (IX+$15),A
6D61  47                  LD   B,A
6D62  DD 6E 04            LD   L,(IX+$04)     FCB+4: enhetspostens drivrutinsadress
6D65  DD 66 05            LD   H,(IX+$05)
6D68  23                  INC  HL
6D69  23                  INC  HL
6D6A  4E                  LD   C,(HL)         C = enhetskoden (adressen + 2)
6D6B  AF                  XOR  A
6D6C  C9                  RET
```

```
6D6D  9F           L6D6D:  SBC  A,A             A = $FF, CY kvar
6D6E  C9                   RET
```

OPEN (funktion 0) och PREPARE (1): en ledig kanal med FRESLT, sedan DOPEN eller DPREP. In: IX = FCB, DE = namnet. Ut: D = kanalens buffertsida (DBUFP + kanalen), C = $84 (radlängd 132); för OPEN CY och A = 21 om filen inte finns.

```
6D6F  CD 44 6D     L6D6F:  CALL FRESLT
6D72  CD 18 60             CALL DOPEN
6D75  3E 15                LD   A,$15           fel 21: filen finns inte
6D77  D8                   RET  C
6D78  18 06                JR   L6D80

6D7A  CD 44 6D     L6D7A:  CALL FRESLT
6D7D  CD 15 60             CALL DPREP
6D80  78           L6D80:  LD   A,B             kanal ...
6D81  0F                   RRCA
6D82  0F                   RRCA
6D83  0F                   RRCA
6D84  0F                   RRCA
6D85  2A 12 FD             LD   HL,(DBUFP)      ... + buffertsidan
6D88  84                   ADD  A,H
6D89  57                   LD   D,A
6D8A  0E 84                LD   C,$84           radlängd 132
6D8C  C9                   RET
```

3.43 DOSDRV: drivrutinen för alla DOS-enheter $6D8D

SP sparas i DSAVSP, så att DERRDV kan avbryta. A = funktion:

```
0 OPEN     FRESLT och DOPEN. Fel 21 om filen inte finns.
           C = $84 (radlängd 132) och D = buffertsidan tillbaka.
1 PREPARE  FRESLT och DPREP, sedan som OPEN
2 CLOSE    FCB+3 := 0 och DCLOSE
3-6        CY = 1, A = $FF: INPUT, PRINT, GET och PUT sköter
           BASIC:en i bufferten, med funktion 7 och 8
7          läs post (DSETRC och DRDREC)
8          skriv post (DSETRC och DWRREC)
9 KILL     DSETEN(DE = $FFFF) och stäng; DFCLS tar bort posten
10- NAME   L6594 (DFNCX)
```

Kanalen står i FCB+$15.

```
6D8D  ED 73 07 FE  DOSDRV: LD   (DSAVSP),SP
6D91  CD 95 6D             CALL DOSDR1          SP sparad för DERRDV
6D94  C9                   RET

6D95  A7           DOSDR1: AND  A
6D96  28 D7                JR   Z,L6D6F         0 OPEN
6D98  3D                   DEC  A
6D99  28 DF                JR   Z,L6D7A         1 PREPARE
6D9B  3D                   DEC  A
6D9C  28 17                JR   Z,L6DB5         2 CLOSE
6D9E  D6 05                SUB  $05
6DA0  DA 6D 6D             JP   C,L6D6D         3-6: CY
6DA3  28 1A                JR   Z,L6DBF         7 läs post
6DA5  3D                   DEC  A
6DA6  28 20                JR   Z,L6DC8         8 skriv post
6DA8  3D                   DEC  A
6DA9  C2 94 65             JP   NZ,L6594        >= 10 NAME
6DAC  DD 46 15             LD   B,(IX+$15)      9 KILL: storlek 0
6DAF  11 FF FF             LD   DE,$FFFF
6DB2  CD 24 60             CALL DSETEN
6DB5  DD 36 03 00  L6DB5:  LD   (IX+$03),$00    FCB+3 := 0: stängd
6DB9  DD 46 15             LD   B,(IX+$15)
6DBC  C3 21 60             JP   DCLOSE

6DBF  DD 46 15     L6DBF:  LD   B,(IX+$15)
6DC2  CD 2A 60             CALL DSETRC
6DC5  C3 2D 60             JP   DRDREC
```

```
6DC8  DD 46 15     L6DC8:  LD   B,(IX+$15)
6DCB  CD 2A 60             CALL DSETRC
6DCE  C3 30 60             JP   DWRREC
```

INIERR: kroken för feltexter (ERRTXV) när BASICINI.SYS finns. Felnumret A söks i den sorterade tabellen INIBUF, och posten i filen blir 1 + antalet nummer <= felet. Posten läses med OPNINI från enheten i DSVHL, och texten efter byten A | $80 kopieras till DE, fram till en byte med bit 7 satt. In: A = felet, DE = platsen för texten. Ut: NC och DE efter texten, eller CY (ingen fil; texten blir Error n). Förstör BC, HL.

```
6DD1  D5           INIERR: PUSH DE(fel)
6DD2  F5                   PUSH AF
6DD3  21 09 FE             LD   HL,$FE09        INIBUF: första felnumret per post
6DD6  BE           L6DD6:  CP   (HL)            felet mot tabellen
6DD7  23                   INC  HL
6DD8  28 02                JR   Z,L6DDC         lika: NC
6DDA  30 FA                JR   NC,L6DD6        större: nästa nummer
6DDC  7D           L6DDC:  LD   A,L             L = efter numret som stoppade
6DDD  DE 08                SBC  A,$08           post = 1 + antalet nummer <= felet
6DDF  ED 5B 05 FE          LD   DE,(DSVHL)      DSVHL: katalogposten och enheten
6DE3  4A                   LD   C,D             C = enhetsbyten
6DE4  16 00                LD   D,$00           D = 0: DE = katalogposten
6DE6  42                   LD   B,D
6DE7  CD FA 6D             CALL OPNINI          läs posten
6DEA  C1                   POP  BC              B = felet, C = flaggorna
6DEB  D1                   POP  DE
6DEC  78                   LD   A,B
6DED  37                   SCF
6DEE  C0                   RET  NZ              ingen BASICINI.SYS: CY, texten blir "Error n"
6DEF  F6 80                OR   $80             A | $80: bit 7 som i posten
6DF1  ED B1                CPIR                 söker A, högst BC = felet * 256 + flaggorna bytes (fel)
6DF3  7E           L6DF3:  LD   A,(HL)
6DF4  A7                   AND  A
6DF5  F8                   RET  M               bit 7: slut, NC
6DF6  ED A0                LDI                  texten till DE
6DF8  18 F9                JR   L6DF3
```

OPNINI: öppna filen DE (BASICINI.SYS) på kanal 0 och läs post A. In: A = posten, B = 0, C = enheten ($FF = vilken som helst). Under läsningen är ERRTXV = $00pp, alltså av, och DERRX = $6E1C. Ut: Z om filen fanns, och då ERRTXV := INIERR; HL = sektorns byte 3 (DBUFP + 3); DSVHL = kanal 0:s +0/+1 (katalogposten och enhetsbyten); kanal 0 ledig och DERRX := DERRDV. Förstör A, BC, DE.

```
6DFA  21 1C 6E     OPNINI: LD   HL,$6E1C        fel: tillbaka hit
6DFD  22 33 FD             LD   (DERRX),HL
6E00  60                   LD   H,B
6E01  6F                   LD   L,A
6E02  E5                   PUSH HL
6E03  22 AC FF             LD   (ERRTXV),HL     ERRTXV := $00rr under läsningen: av (ERRMSG prövar H)
6E06  CD 18 60             CALL DOPEN
6E09  48                   LD   C,B             C = 0 (B): inte funnen än
6E0A  38 10                JR   C,L6E1C         finns inte
6E0C  D1                   POP  DE
6E0D  D5                   PUSH DE
6E0E  CD 2A 60             CALL DSETRC          post A
6E11  CD 2D 60             CALL DRDREC
6E14  21 D1 6D             LD   HL,INIERR       feltexterna ur BASICINI.SYS
6E17  22 AC FF             LD   (ERRTXV),HL
6E1A  0E FF                LD   C,$FF           C = $FF: funnen
6E1C  21 FE 6C     L6E1C:  LD   HL,DERRDV       DERRX := DERRDV igen (egenhet)
6E1F  22 33 FD             LD   (DERRX),HL
6E22  2A 40 FD             LD   HL,(SLTTAB)     kanal 0:s +0/+1: katalogposten och enhetsbyten
6E25  22 05 FE             LD   (DSVHL),HL
6E28  3E FF                LD   A,$FF           kanal 0 ledig
6E2A  32 41 FD             LD   (VFD41),A
6E2D  2A 12 FD             LD   HL,(DBUFP)      HL = sektorns byte 3
6E30  2E 03                LD   L,$03
6E32  D1                   POP  DE
6E33  0C                   INC  C               Z: funnen
6E34  C9                   RET
```

DOSDEV: *enhetsposter à 8 bytes: länk, namn (3 tecken), drivrutin (DOSDRV) och enhetskod. FCB+4 pekar på postens drivrutinsadress (+5), och enhetskoden står på +7. Den första posten (blankt namn, enhet $FF) kopieras till DEVLNK2 ($FDD0) och får den gamla DEVLST som länk. DEVLST blir DR0, och listan går DR0 DR1 DR2 UFD MF0 MF1 MF2 MO0 MO1 SF0 SF1 SF2 HD0 HD1 HD2 HD3, $FDD0 och sedan de gamla enheterna.*

```
6E35  3D 6E 20 20… DOSDEV: DEFB $3D,$6E,"    "
6E3A  8D 6D FF             DEFB $8D,$6D,$FF      DOSDRV och enheten $FF
6E3D  45 6E 44 52…         DEFB $45,$6E,"DR0"
6E42  8D 6D 00             DEFB $8D,$6D,$00
6E45  4D 6E 44 52…         DEFB $4D,$6E,"DR1"
6E4A  8D 6D 01             DEFB $8D,$6D,$01
6E4D  55 6E 44 52…         DEFB $55,$6E,"DR2"
6E52  8D 6D 02             DEFB $8D,$6D,$02
6E55  5D 6E 55 46…         DEFB $5D,$6E,"UFD"
6E5A  8D 6D 1E             DEFB $8D,$6D,$1E
6E5D  65 6E 4D 46…         DEFB $65,$6E,"MF0"
6E62  8D 6D 08             DEFB $8D,$6D,$08
6E65  6D 6E 4D 46…         DEFB $6D,$6E,"MF1"
6E6A  8D 6D 09             DEFB $8D,$6D,$09
6E6D  75 6E 4D 46…         DEFB $75,$6E,"MF2"
6E72  8D 6D 0A             DEFB $8D,$6D,$0A
6E75  7D 6E 4D 4F…         DEFB $7D,$6E,"MO0"
6E7A  8D 6D 0C             DEFB $8D,$6D,$0C
6E7D  85 6E 4D 4F…         DEFB $85,$6E,"MO1"
6E82  8D 6D 0D             DEFB $8D,$6D,$0D
6E85  8D 6E 53 46…         DEFB $8D,$6E,"SF0"
6E8A  8D 6D 10             DEFB $8D,$6D,$10
6E8D  95 6E 53 46…         DEFB $95,$6E,"SF1"
6E92  8D 6D 11             DEFB $8D,$6D,$11
6E95  9D 6E 53 46…         DEFB $9D,$6E,"SF2"
6E9A  8D 6D 12             DEFB $8D,$6D,$12
6E9D  A5 6E 48 44…         DEFB $A5,$6E,"HD0"
6EA2  8D 6D 04             DEFB $8D,$6D,$04
6EA5  AD 6E 48 44…         DEFB $AD,$6E,"HD1"
6EAA  8D 6D 05             DEFB $8D,$6D,$05
6EAD  B5 6E 48 44…         DEFB $B5,$6E,"HD2"
6EB2  8D 6D 06             DEFB $8D,$6D,$06
6EB5  D0 FD 48 44…         DEFB $D0,$FD,"HD3"
6EBA  8D 6D 07             DEFB $8D,$6D,$07      HD3; länken går till $FDD0
```

INITXT: `BASICINI SYS`, `CMDINT  SYS` (*för* `BYE`) *och CR LF* `Abort .`

```
6EBD  42 41 53 49… INITXT: DEFB "BASICINISYSCMDINT  SYS",$0D
6ED4  0A 41 62 6F…         DEFB $0A,"Abort "
```

CRDTAB: *korttyperna, 4 bytes var: kortbyte, flaggor och två bokstäver. DR, HD, MF, MO, SF, två tomma (??) och RM. Kopieras av DOSSTA till $FDE2; byten efter, $C9 (RET), blir kroken VFE02.*

```
6EDB  08 00 44 52… CRDTAB: DEFB $08,$00,$44,$52,$24,$05,$48,$44  DR och HD
6EE3  6C 02 4D 46…         DEFB $6C,$02,$4D,$46,$AD,$00,$4D,$4F  MF och MO
6EEB  6E 02 53 46…         DEFB $6E,$02,$53,$46,$00,$00,$3F,$3F  SF och typ 5 (tom)
6EF3  00 00 3F 3F…         DEFB $00,$00,$3F,$3F,$00,$02,$52,$4D  typ 6 (tom) och RM
6EFB  C9                   DEFB $C9              RET: kroken VFE02
```

UNSRUT: *UNSAVE (CMDTAB -> UNSVEC, utan DOS fel 200). Stänger programfilen (PRGCLS), öppnar namnet som LOAD (OPEN med A = 2, standardtyp BAS) och tar bort filen som KILL (funktion 9). Fel från funktion 9 kommer tillbaka med CY och A = felet; UNSAVE "PR:" ger 52.*

```
6EFC  CD 69 00     UNSRUT: CALL S0069            PRGCLS
6EFF  3E 02                LD   A,$02            standardtyp BAS
6F01  CD 5D 00             CALL S005D
6F04  18 0D                JR   L6F13            funktion 9
```

XKILL (`KILL`): *filnamnet beräknas (FNAMEX, CEXPS-uttrycket) och drivrutinens funktion 9 anropas. Fel från funktion 9 går till ERRA;* `KILL "NUL:"` *ger inget fel,* `KILL "PR:"` *fel 52.*

FNAMEX: *beräknar strängen, tar den från stacken och öppnar filen med FCB $FF63 (OPENIX, A = 0). OPENIX ger själv felen vid öppningen (fel 21 om filen inte finns), så FNAMEX kommer bara tillbaka när filen är öppen. Ut: IX = FCB, DE efter uttrycket.*

XNAME (`NAME` *a* `AS` *b*): *öppnar den gamla filen med FNAMEX, tolkar det nya namnet med PARSFN (14 bytes på stacken) och anropar*

funktion 10 med det i DE. Kanalen stängs med CLOSEF *även efter ett fel, och varje fel från* PARSFN *eller funktion 10 blir fel 64: också* NAME "CON:" AS "X" *och* NAME "PR:" AS "X"*, som med* KILL *ger 52.*

CNAME *kompilerar* NAME*: uttryck,* AS *(tabellen* $6F9E*), uttryck. Fel 231 om* AS *saknas.*

```
6F06  CD 0D 6F     XKILL:  CALL S6F0D
6F09  D0                   RET  NC
6F0A  C3 12 00             JP   ERRA            fel: ERRA

6F0D  CD 69 00     S6F0D:  CALL S0069           PRGCLS
6F10  CD 1B 6F             CALL FNAMEX
6F13  D5           L6F13:  PUSH DE
6F14  3E 09                LD   A,$09           funktion 9: KILL
6F16  CD 5A 00             CALL S005A
6F19  D1                   POP  DE
6F1A  C9                   RET

6F1B  E7           FNAMEX: RST  $20             beräkna namnet (sträng)
6F1C  DD E1                POP  IX              IX = strängens plats på stacken
6F1E  E1                   POP  HL              HL = texten
6F1F  C1                   POP  BC              BC = längden
6F20  DD 39                ADD  IX,SP           IX = ovanför strängen
6F22  D5                   PUSH DE
6F23  DD E5                PUSH IX
6F25  DD 21 63 FF          LD   IX,$FF63        FCB $FF63
6F29  AF                   XOR  A
6F2A  CD 60 00             CALL S0060           öppna (OPENIX, A = 0); fel direkt till ERRA
6F2D  E1           L6F2D:  POP  HL              strängen bort från stacken
6F2E  D1                   POP  DE
6F2F  F9                   LD   SP,HL
6F30  C9                   RET

6F31  CD 69 00     XNAME:  CALL S0069
6F34  CD 1B 6F             CALL FNAMEX
6F37  13                   INC  DE              över $86 $A3
6F38  13                   INC  DE
6F39  E7                   RST  $20             det nya namnet
6F3A  DD E1                POP  IX              IX = strängens plats på stacken
6F3C  E1                   POP  HL              HL = texten
6F3D  C1                   POP  BC              BC = längden
6F3E  DD 39                ADD  IX,SP           IX = ovanför strängen
6F40  D5                   PUSH DE
6F41  DD E5                PUSH IX
6F43  EB                   EX   DE,HL
6F44  21 F2 FF             LD   HL,$FFF2        14 bytes för det tolkade
6F47  39                   ADD  HL,SP
6F48  F9                   LD   SP,HL
6F49  EB                   EX   DE,HL
6F4A  D5                   PUSH DE
6F4B  CD 63 00             CALL S0063           PARSFN: det nya namnet
6F4E  D1                   POP  DE
6F4F  DD 21 63 FF          LD   IX,$FF63
6F53  3E 0A                LD   A,$0A           funktion 10: NAME
6F55  D4 5A 00             CALL NC,S005A        bara om PARSFN gav NC
6F58  F5                   PUSH AF
6F59  CD 33 65             CALL CLOSEF          stäng den gamla filen (B från DFNCX)
6F5C  F1                   POP  AF
6F5D  38 07                JR   C,L6F66         varje fel blir 64
6F5F  21 0E 00             LD   HL,$000E        stacken tillbaka
6F62  39                   ADD  HL,SP
6F63  F9                   LD   SP,HL
6F64  18 C7                JR   L6F2D

6F66  D7           L6F66:  RST  $10             fel 64, vilket fel det än var
6F67  40                   DEFB $40

6F68  CD 1B 00     CNAME:  CALL CEXPS           första namnet
6F6B  D8                   RET  C
6F6C  3E 86                LD   A,$86           $86 $A3 mellan namnen
6F6E  12                   LD   (DE),A
6F6F  13                   INC  DE
6F70  3E A3                LD   A,$A3
```

```
6F72  12                LD   (DE),A
6F73  13                INC  DE
6F74  D5                PUSH DE
6F75  11 9E 6F          LD   DE,$6F9E          AS
6F78  CD 23 00          CALL S0023
6F7B  D1                POP  DE
6F7C  3E E7             LD   A,$E7             fel 231: AS saknas
6F7E  37                SCF
6F7F  C0                RET  NZ
6F80  C3 1B 00    L6F80: JP   CEXPS            andra namnet

6F83  59 00       DOSCMP: DEFW $0059           kompilering: BYE (RET), KILL, NAME
6F85  80 6F               DEFW $6F80
6F87  68 6F               DEFW $6F68
6F89  37 6D       DOSRUN: DEFW $6D37           körning
6F8B  06 6F               DEFW $6F06
6F8D  31 6F               DEFW $6F31
6F8F  80 42 59 45 DOSKW:  DEFB $80,"BYE"       satsorden
6F93  81 4B 49 4C…        DEFB $81,"KILL"
6F98  82 4E 41 4D…        DEFB $82,"NAME"
6F9D  FF                  DEFB $FF
6F9E  83 41 53            DEFB $83,"AS"        AS för CNAME
6FA1  FF                  DEFB $FF
```

DOSSTH: tabellhuvud för satserna (kopieras till DOSSTR $FDD8): länk, första kod $A0, antal 4 och tabellerna DOSRUN/DOSKW/DOSCMP. Koderna är BYE $A0, KILL $A1, NAME $A2 (i programmet $86 $A0 ...; XSTM 160-162 ger samma satser). Tabellerna har bara tre poster: AS ($83 efter $FF i DOSKW) är inget satsord, och kod $A3 tar ordet efter DOSCMP, alltså början av DOSRUN ($6D37 = XBYE), som kompileringsrutin. XSTM 163 kör därför BYE redan när raden matas in. Samma i ABC800 M.

```
6FA2  A2 6F       DOSSTH: DEFW $6FA2           länken, ersätts i RAM (fel)
6FA4  A0 04               DEFW $04A0           första koden $A0, antal 4
6FA6  89 6F               DEFW $6F89           körning
6FA8  8F 6F               DEFW $6F8F           satsorden
6FAA  83 6F               DEFW $6F83           kompilering
6FAC  47 4E 20 54…        DEFM "GN T{by 1984-03-02"
6FBE  07 07 07 07…        DEFB $07,$07,$07,$07,$07,$07,$00,$00  fyllnad, som i ABC800 M
6FC6  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FCE  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FD6  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FDE  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FE6  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FEE  00 00 00 00…        DEFB $00,$00,$00,$00,$00,$00,$00,$00
6FF6  00 00 00 00…        DEFB $00,$00,$00,$00,$D7,$C8,$D7,$C8
6FFE  D7 C8               DEFB $D7,$C8
```

3.44 Tilläggsprogrammet på $7000: enheten PR: (skrivare) $7000

Startas av EXTSCN vid RESET. PRINIT länkar in enhetsposten PRDEV ("PR ", drivrutin PRDRV) i DEVLST, reserverar ett arbetsområde under stacken (PRAREA) och kopierar standardinställningen PRDEF till PRDEFR ($FFC0, i den oanvända delen av IM 2-tabellen). HL = $7600 tillbaka, så att EXTSCN fortsätter på $7700: sidorna $7100-$76FF hör till programmet och undersöks inte.

Skrivaren kan vara seriell (V24, bitvis via PIO B, port $3A) eller parallell (Centronics-kort på ABC-bussen, portarna $00-$03).

Filnamnet efter PR: är en inställning, t.ex. PR:VSA30D72.4. Namnet och tillägget (8 + 3 tecken, se S4AB6) läses som en följd:

```
[0]    C = Centronics, V = V24
[1]    paritet: S (oförändrad), M (bit 7 = 1), E (jämn), O (udda)
[2]    A-Z: antal NUL efter varje LF = 2 * (bokstav - "A")
[3]    1-7: radlängd 40, 72, 80, 120, 132, 158, 255 (PRWTAB)
[4]    0-9: marginal; sidmatning när högst så många rader är kvar
[5]    A-D: två bitar. Bit 0 = 1: sidmatning görs med LF, annars
       skickas FF. Bit 1 = 1: CR LF när raden är full, annars
       räknas kolumnen bara om från 1.
[6,7]  sidlängd i rader, 01-99
[8]    bara V (första tecknet i tillägget): hastighet 1-7 = 110, 300, 600, 1200, 2400, 4800,
```

```
          9600 baud (härlett ur fördröjningarna i BAUDTB och 3 MHz;
          110 baud får två stoppbitar)
```

Tomt filnamn: förra inställningen används (radräknaren behålls). Finns ingen: PRDEFR, med C om det finns ett skrivarkort, annars V.

Fel 21 vid felaktig inställning, fel 52 för INPUT, GET och PUT.

```
                  EXT7000:
7000  C3 03 70           JP   PRINIT
```

PRINIT: anropas av EXTSCN genom $7000 vid RESET. Länkar in PR: i DEVLST (DEVLNK) och reserverar arbetsområdet (PRAREA). Ut: HL = $7600, SP 34 bytes lägre (10 för enhetsposten och 24 för blocken). Förstör alla, också de andra registren (EXX).

```
7003  21 64 70     PRINIT: LD   HL,$7064
7006  01 7B FF             LD   BC,$FF7B        listhuvudet DEVLST
7009  CD 0F 70             CALL DEVLNK
700C  C3 3F 70             JP   PRAREA          två block under stacken
```

DEVLNK: HL = kedja av enhetsposter i ROM (+0,+1 länk), BC = listhuvudet. Varje post kopieras till 10 bytes under stacken: länken (gamla huvudet) och 8 bytes från posten +2. Huvudet pekar sedan på kopian. Under kopieringen ligger de två returadresserna (PRINIT:s och EXTSCN:s) i BC′ och DE′, och de läggs tillbaka under kopian; registersatserna byter plats varje varv. Här finns bara en post ($7064). En enhetspost i RAM är 8 bytes som i DOSDEV (namnet på +2, drivrutinen på +5, enhetskoden på +7); PR:s post i ROM slutar efter drivrutinsadressen, så de tre sista av de 8 är de första bytena i PRDRV ($A7 $28 $10). Kopians enhetskod blir $A7, och den läser PRDRV inte. In: HL, BC. Ut: HL = 0, SP 10 bytes lägre per post. Förstör alla, också de andra registren.

```
700F  7C           DEVLNK: LD   A,H
7010  B5                   OR   L
7011  C8                   RET  Z
7012  5E                   LD   E,(HL)
7013  23                   INC  HL
7014  56                   LD   D,(HL)
7015  23                   INC  HL
7016  D5                   PUSH DE              nästa post
7017  EB                   EX   DE,HL
7018  D9                   EXX                  byt registersats
7019  E1                   POP  HL              HL′ = nästa post
701A  C1                   POP  BC              BC′ = PRINIT:s retur ($700C)
701B  D1                   POP  DE              DE′ = EXTSCN:s retur ($00EA)
701C  D9                   EXX
701D  21 F6 FF             LD   HL,$FFF6        10 bytes ned
7020  39                   ADD  HL,SP
7021  F9                   LD   SP,HL
7022  EB                   EX   DE,HL
7023  D5                   PUSH DE              kopians adress
7024  C5                   PUSH BC              listhuvudet
7025  01 02 00             LD   BC,$0002
7028  E3                   EX   (SP),HL
7029  ED B0                LDIR                 +0,+1: gamla huvudet
702B  E3                   EX   (SP),HL
702C  0E 08                LD   C,$08
702E  ED B0                LDIR                 +2..+9: 8 bytes ur posten
7030  E1                   POP  HL
7031  D1                   POP  DE
7032  2B                   DEC  HL              huvudet pekar på kopian
7033  72                   LD   (HL),D
7034  2B                   DEC  HL
7035  73                   LD   (HL),E
7036  D9                   EXX
7037  D5                   PUSH DE              returadresserna under kopian
7038  C5                   PUSH BC
7039  D9                   EXX
703A  E5                   PUSH HL
703B  D9                   EXX
703C  C1                   POP  BC              BC = huvudet, HL = nästa post (i den andra satsen)
703D  18 D0                JR   DEVLNK
```

PRAREA: *två block à 12 bytes under stacken, första byten 0 (ingen sparad inställning).* PRUNIT *pekar på det sista. Blocket: 11 bytes inställning,* +$0B *radräknare (rader kvar på sidan, 0 = sidans början). Det andra blocket används inte. ABC800 har två enheter, PR: och V24:, men reserverar ett enda område på 220 bytes* (LD HL,$FF24 / ADD HL,SP, $70E9 *i ABC800 M). Sedan kopieras* PRDEF *till* PRDEFR. *In: returadressen vid SP. Ut: HL =* $7600, *B = 0, DE = returadressen, SP 24 bytes lägre. Förstör AF.*

```
703F  D1           PRAREA: POP  DE              returadressen
7040  06 02                LD   B,$02
7042  21 F4 FF     L7042:  LD   HL,$FFF4
7045  39                    ADD  HL,SP
7046  F9                    LD   SP,HL
7047  22 DC FF              LD   (PRUNIT),HL
704A  36 00                 LD   (HL),$00        ingen sparad inställning
704C  10 F4                 DJNZ L7042
704E  D5                    PUSH DE
704F  21 00 76              LD   HL,$7600        EXTSCN fortsätter efter $76FF
7052  C5                    PUSH BC
7053  D5                    PUSH DE
7054  E5                    PUSH HL
7055  21 35 73              LD   HL,$7335        PRDEF -> PRDEFR
7058  11 C0 FF              LD   DE,$FFC0        PRDEFR
705B  01 0B 00              LD   BC,$000B
705E  ED B0                 LDIR
7060  E1                    POP  HL
7061  D1                    POP  DE
7062  C1                    POP  BC
7063  C9                    RET
```

PRDEV: *enhetsposten i ROM: länk 0, namnet "PR " och drivrutinen* PRDRV. *Posten har ingen enhetskod;* DEVLNK *kopierar ändå 8 bytes efter länken och tar med de tre första i* PRDRV.

```
7064  00 00        PRDEV:  DEFB $00,$00
7066  50 52 20 6B…        DEFM "PR kp"
```

3.45 PRDRV: drivrutinen för PR:. A = funktion, IX = FCB. $706B

0 OPEN, 1 PREPARE -> PROPEN, 2 CLOSE -> PRCLOS, 4 PRINT -> PRPRT, annars fel 52. FCB: +$0D sidlängd, +$0E antal NUL, +$0F paritets-bokstaven, +$10 marginal, +$12,+$13 = PRUNIT, +$15 funktionen vid öppningen. Efter DEC A är den alltid 0, också för PREPARE (se PRPEND).

```
706B  A7           PRDRV:  AND  A               OPEN
706C  28 10                JR   Z,PROPEN        A = 0
706E  3D                   DEC  A               PREPARE (A blir 0) (fel)
706F  28 0D                JR   Z,PROPEN        också här A = 0
7071  3D                   DEC  A               CLOSE
7072  CA C8 72             JP   Z,PRCLOS
7075  D6 02                SUB  $02             PRINT
7077  CA 9D 71             JP   Z,PRPRT
707A  3E 34                LD   A,$34           okänd funktion
707C  37                   SCF
707D  C9                   RET
```

PROPEN: *OPEN och PREPARE. In: A = 0, DE = filnamnet efter PR:, IX = FCB. Är första tecknet ett blanktecken (prövas utan att bit 7 tas bort) är ingen inställning angiven: då används den sparade i* PRUNIT*-blocket, direkt i* PRSPEC *så att radräknaren behålls, och finns ingen sparad tar* PRDFLT *standarden. Annars* PRNEW. *Ut som* PRSPEC.

```
707E  2A DC FF     PROPEN: LD   HL,(PRUNIT)
7081  01 C0 FF             LD   BC,$FFC0        BC = PRDEFR
7084  DD 77 15             LD   (IX+$15),A      funktionen (alltid 0)
7087  DD 75 12             LD   (IX+$12),L      PRUNIT i FCB
708A  DD 74 13             LD   (IX+$13),H
708D  1A                   LD   A,(DE)
708E  13                   INC  DE
708F  FE 20                CP   $20             blanktecken: ingen inställning
7091  20 28                JR   NZ,PRNEW
7093  7E                   LD   A,(HL)          sparad inställning?
7094  A7                   AND  A
7095  28 06                JR   Z,PRDFLT
```

```
7097  EB                EX   DE,HL
7098  CD 77 71          CALL SPCHR          första tecknet (C/V)
709B  18 28             JR   PRSPEC
```

PRDFLT: *ingen sparad inställning.* PRDEFR+1 *(10 tecken) kopieras in efter blanktecknet i filnamnet, och blanktecknet byts mot C eller V: kortval på ABC-bussen; ger* IN ($01) $FF *finns inget kort (V). Fortsätter i* PRNEW *med A = C eller V och DE på nästa tecken.*

```
709D  D5          PRDFLT: PUSH DE
709E  69                  LD   L,C          HL = PRDEFR+1
709F  60                  LD   H,B
70A0  23                  INC  HL
70A1  01 0A 00            LD   BC,$000A
70A4  ED B0               LDIR
70A6  D1                  POP  DE
70A7  0E 56               LD   C,$56        V
70A9  3E BC               LD   A,$BC        kortval 60 med bit 7 satt
70AB  D3 01               OUT  ($01),A
70AD  D3 03               OUT  ($03),A
70AF  DB 01               IN   A,($01)      $FF: inget kort
70B1  2F                  CPL
70B2  A7                  AND  A
70B3  28 02               JR   Z,L70B7
70B5  0E 43               LD   C,$43        C: det finns ett skrivarkort
70B7  79          L70B7:  LD   A,C
70B8  1B                  DEC  DE           först i filnamnet
70B9  12                  LD   (DE),A
70BA  13                  INC  DE
```

PRNEW: *ny inställning, radräknaren nollställs (ny sida). In: A = första tecknet, DE efter det, IX = FCB. Fortsätter i* PRSPEC.

```
70BB  21 0B 00    PRNEW:  LD   HL,$000B     radräknaren = 0
70BE  ED 4B DC FF         LD   BC,(PRUNIT)
70C2  09                  ADD  HL,BC
70C3  36 00               LD   (HL),$00
```

PRSPEC: *tolka inställningen (se* $7000*). In: A = första tecknet, DE = nästa, IX = FCB. [0] prövas mot* PRTYPT*, [1] blir FCB+*$0F*, [2] FCB+*$0E*, [3] ger radlängden ur* PRWTAB*, [4] blir FCB+*$10*, [5] prövas bara mot* FORMTB *och [6,7] blir FCB+*$0D*. Sedan kopieras de 11 tecknen från DE - 8 till* PRUNIT*-blocket (med den sparade inställningen är det blocket självt), och för V läses [8]. Ut: C = radlängden, D =* $FF *(ingen buffert), A = 0, CY = 0. Fel 21 (*SPERR*) vid ett felaktigt tecken. Förstör B, E och HL.*

```
70C5  21 28 73    PRSPEC: LD   HL,$7328     C eller V (egenhet)
70C8  CD 6F 71            CALL SPTAB
70CB  06 FF               LD   B,$FF        B = $FF (blir D tillbaka)
70CD  C5                  PUSH BC
70CE  CD 77 71            CALL SPCHR
70D1  21 2B 73            LD   HL,$732B     paritet
70D4  CD 6F 71            CALL SPTAB
70D7  DD 77 0F            LD   (IX+$0F),A
70DA  CD 77 71            CALL SPCHR
70DD  D6 41               SUB  $41          antal NUL
70DF  FE 1A               CP   $1A          A-Z
70E1  D2 75 71            JP   NC,SPERR
70E4  87                  ADD  A,A          * 2
70E5  DD 77 0E            LD   (IX+$0E),A
70E8  CD 77 71            CALL SPCHR
70EB  FE 31               CP   $31          radlängd 1-7
70ED  DA 75 71            JP   C,SPERR
70F0  FE 38               CP   $38
70F2  D2 75 71            JP   NC,SPERR
70F5  E6 0F               AND  $0F          index i PRWTAB
70F7  21 20 73            LD   HL,$7320
70FA  85                  ADD  A,L
70FB  6F                  LD   L,A
70FC  7C                  LD   A,H
70FD  CE 00               ADC  A,$00
70FF  67                  LD   H,A
```

```
7100  C1              POP  BC
7101  4E              LD   C,(HL)          C = radlängden
7102  C5              PUSH BC
7103  CD 77 71        CALL SPCHR
7106  CD 80 71        CALL SPDIG           marginal
7109  DD 77 10        LD   (IX+$10),A
710C  CD 77 71        CALL SPCHR
710F  21 30 73        LD   HL,$7330        A-D
7112  CD 6F 71        CALL SPTAB
7115  CD 77 71        CALL SPCHR           sidlängd, två siffror
7118  CD 80 71        CALL SPDIG
711B  4F              LD   C,A             tiotal
711C  CD 77 71        CALL SPCHR
711F  CD 80 71        CALL SPDIG
7122  D5              PUSH DE
7123  69              LD   L,C
7124  26 00           LD   H,$00
7126  29              ADD  HL,HL           10 * tiotal + ental
7127  54              LD   D,H
7128  5D              LD   E,L
7129  29              ADD  HL,HL
712A  29              ADD  HL,HL
712B  19              ADD  HL,DE
712C  85              ADD  A,L
712D  28 46           JR   Z,SPERR         0 rader: fel
712F  D1              POP  DE
7130  DD 77 0D        LD   (IX+$0D),A      sidlängden
7133  D5              PUSH DE
7134  21 F8 FF        LD   HL,$FFF8        första tecknet i inställningen
7137  19              ADD  HL,DE
7138  7E              LD   A,(HL)
7139  DD 5E 12        LD   E,(IX+$12)      11 tecken till PRUNIT-blocket
713C  DD 56 13        LD   D,(IX+$13)
713F  01 0B 00        LD   BC,$000B
7142  ED B0           LDIR(egenhet)
7144  D1              POP  DE
```

V24: hastighetssiffran [8] sparas som tecken (PRBAUD) och fördröjningen för en bit tas ur BAUDTB (PRBDLY). Siffran 0 godtas men ger index $FE och läser ordet på $743E, utanför tabellen. I den här ROM:en är ordet $FFFF, och en bit tar omkring 0,57 sekunder (knappt 2 baud). I abc80sim:s basicii80.rom står där $312C ur en rest av äldre kod, omkring 0,1 sekund och 9 baud. För C läses [8] inte.

```
7145  FE 56           CP   $56             V24?
7147  20 22           JR   NZ,L716B
7149  CD 77 71        CALL SPCHR
714C  CD 80 71        CALL SPDIG
714F  FE 08           CP   $08             hastighet 0-7 (fel)
7151  30 22           JR   NC,SPERR
7153  11 E4 FF        LD   DE,$FFE4
7156  F5              PUSH AF
7157  F6 30           OR   $30             som siffertecken
7159  12              LD   (DE),A
715A  F1              POP  AF
715B  3D              DEC  A               (siffran - 1) * 2
715C  87              ADD  A,A
715D  5F              LD   E,A
715E  16 00           LD   D,$00
7160  21 40 73        LD   HL,$7340        BAUDTB
7163  19              ADD  HL,DE
7164  7E              LD   A,(HL)
7165  23              INC  HL
7166  66              LD   H,(HL)
7167  6F              LD   L,A
7168  22 E5 FF        LD   (PRBDLY),HL
716B  C1        L716B:POP  BC              C = radlängd, D = $FF
716C  50              LD   D,B
716D  AF              XOR  A               CY = 0
716E  C9              RET
```

SPTAB: *In: A = tecknet, HL = tabellen (antal, tecken ...). Ut: Z om A finns med, annars fel 21. Förstör B och HL.* SPCHR: *In: DE. Ut: A = nästa tecken utan bit 7, DE + 1; fel 21 för blanktecken.* SPDIG: *In: A = tecknet. Ut: A = siffrans värde; fel 21 om A inte är 0-9.*

```
716F  46          SPTAB:  LD   B,(HL)
7170  23          L7170:  INC  HL
7171  BE                  CP   (HL)
7172  C8                  RET  Z              finns: Z
7173  10 FB               DJNZ L7170
7175  D7          SPERR:  RST  $10             fel 21
7176  15                  DEFB $15

7177  1A          SPCHR:  LD   A,(DE)
7178  13                  INC  DE
7179  E6 7F               AND  $7F             bit 7 bort
717B  FE 20               CP   $20
717D  C0                  RET  NZ
717E  18 F5               JR   SPERR

7180  FE 30       SPDIG:  CP   $30
7182  38 F1               JR   C,SPERR
7184  FE 3A               CP   $3A
7186  30 ED               JR   NC,SPERR
7188  E6 0F               AND  $0F             siffrans värde
718A  C9                  RET
```

PRBRK: *anropas under väntan på skrivaren. BREAK (IY+$23 bit 0) ger fel 0 genom* ERRJP*, som kommer tillbaka vid CON; då väntar rutinen vidare. Ut: Z (ingen BREAK). Förstör A och F.*

```
718B  FD CB 23 46 PRBRK:  BIT  0,(IY+$23)
718F  C8                  RET  Z              inte BREAK
7190  AF                  XOR  A              fel 0
7191  C3 33 00            JP   ERRJP
```

BITDLY: *en bittid,* PRBDLY *varv à 26 T-cykler. Förstör A, F och HL.*

```
7194  2A E5 FF  BITDLY: LD   HL,(PRBDLY)
7197  2B          L7197:  DEC  HL             26 T-cykler per varv
7198  7C                  LD   A,H
7199  B5                  OR   L
719A  20 FB               JR   NZ,L7197
719C  C9                  RET
```

3.46 PRPRT: skriv BC tecken från HL. Efter varje tecken PRMARG. $719D

PRMARG: sidmatning om radräknaren - 1 < marginalen.

PRCHR: ett tecken (E). FF: med bit 0 i bokstaven [5] skickas LF tills radräknaren är 0 (PRFFLF), annars nollställs räknaren och FF skickas. Skrivbara tecken räknas i kolumnen (FCB+6) mot radlängden (FCB+8); är raden full blir det CR LF eller bara kolumn 1 (bit 1 i [5]).

Efter utskriften (PROUT): CR ger kolumn 0, BS en kolumn tillbaka, LF räknar ned radräknaren (från sidlängden när den är 0) och skickar FCB+$0E st NUL.

Två egenheter, som i skrivarkretsen 9704. Efter CR LF vid full rad står kolumnen på 0, och tecknet som sedan skrivs räknas inte; med bit 1 i [5] får därför raderna efter den första radlängden + 1 tecken (prövat i emulatorn med radlängd 40: 40, 41, 41). BS ändrar inte kolumnen när den är 129 eller mer, eftersom JP P på $7215 tar kolumnen − 1 som ett negativt tal (prövat med radlängd 158: tio BS efter 100 tecken ger kolumn 90, efter 135 tecken ingen ändring).

PRPRT: In: HL = texten, BC = längden, IX = FCB. Ut: A = 0, BC = 0, HL efter texten, Z och NC. Förstör DE. PRCHR: In: E = tecknet, IX = FCB. Förstör A, F och E. PRMARG: förstör A, F och E.

```
719D  CD EA 72    PRPRT:  CALL UNTYPE
71A0  79                  LD   A,C              BC = 0: klart
71A1  B0                  OR   B
71A2  C8                  RET  Z
71A3  5E                  LD   E,(HL)
71A4  23                  INC  HL
```

```
71A5  0B                DEC  BC
71A6  CD BB 71          CALL PRCHR
71A9  CD AE 71          CALL PRMARG
71AC  18 EF             JR   PRPRT

71AE  CD 00 73    PRMARG: CALL UNLCNT
71B1  3D                DEC  A             rader kvar - 1
71B2  DD BE 10          CP   (IX+$10)
71B5  D0                RET  NC
71B6  1E 0C             LD   E,$0C         FF
71B8  C3 BB 71          JP   PRCHR

71BB  3E 0C       PRCHR:  LD   A,$0C
71BD  BB                CP   E
71BE  20 0C             JR   NZ,L71CC      inte FF
71C0  CD F5 72          CALL UNFORM
71C3  3D                DEC  A             bit 0 i [5]: sidmatning med LF
71C4  CB 47             BIT  0,A
71C6  20 73             JR   NZ,PRFFLF
71C8  AF                XOR  A             FF: räknaren = 0
71C9  CD 0B 73          CALL UNSCNT
71CC  7B          L71CC:  LD   A,E
71CD  E6 7F             AND  $7F
71CF  FE 20             CP   $20           styrtecken räknas inte
71D1  38 2B             JR   C,L71FE
71D3  DD 34 06          INC  (IX+$06)      kolumnen
71D6  28 09             JR   Z,L71E1
71D8  DD 7E 06          LD   A,(IX+$06)
71DB  3D                DEC  A
71DC  DD BE 08          CP   (IX+$08)      mot radlängden
71DF  38 1D             JR   C,L71FE
71E1  CD F5 72    L71E1:  CALL UNFORM
71E4  3D                DEC  A
71E5  CB 4F             BIT  1,A           bit 1 i [5]: CR LF vid full rad
71E7  20 06             JR   NZ,L71EF
71E9  DD 36 06 01       LD   (IX+$06),$01  bara kolumn 1 (egenhet)
71ED  18 0F             JR   L71FE

71EF  D5          L71EF:  PUSH DE            CR LF (fel)
71F0  1E 0D             LD   E,$0D
71F2  CD BB 71          CALL PRCHR
71F5  1E 0A             LD   E,$0A
71F7  CD BB 71          CALL PRCHR
71FA  CD AE 71          CALL PRMARG
71FD  D1                POP  DE
71FE  D5          L71FE:  PUSH DE            skicka tecknet
71FF  CD 47 72          CALL PROUT
7202  D1                POP  DE
7203  7B                LD   A,E
7204  E6 7F             AND  $7F
7206  FE 0D             CP   $0D
7208  20 04             JR   NZ,L720E
720A  DD 36 06 00       LD   (IX+$06),$00  CR: kolumn 0
720E  FE 08       L720E:  CP   $08
7210  20 09             JR   NZ,L721B
7212  DD 35 06          DEC  (IX+$06)      BS: en kolumn tillbaka
7215  F2 1B 72          JP   P,L721B       kolumnen negativ: 0 eller över 128 (fel)
7218  DD 34 06          INC  (IX+$06)      inte under 0
721B  FE 0A       L721B:  CP   $0A
721D  C0                RET  NZ
721E  CD 00 73          CALL UNLCNT        LF: rader kvar
7221  A7                AND  A
7222  20 03             JR   NZ,L7227
7224  DD 7E 0D          LD   A,(IX+$0D)    0: ny sida
7227  3D          L7227:  DEC  A
7228  CD 0B 73          CALL UNSCNT        räkna ned
722B  DD 7E 0E          LD   A,(IX+$0E)    antal NUL
722E  A7                AND  A
722F  C8                RET  Z
7230  F5          L7230:  PUSH AF            NUL
```

```
7231  1E 00              LD   E,$00
7233  CD BB 71           CALL PRCHR
7236  F1                 POP  AF
7237  3D                 DEC  A
7238  20 F6              JR   NZ,L7230
723A  C9                 RET

723B  1E 0A       PRFFLF: LD   E,$0A
723D  CD BB 71           CALL PRCHR
7240  CD 00 73           CALL UNLCNT          tills räknaren är 0
7243  A7                 AND  A
7244  20 F5              JR   NZ,PRFFLF
7246  C9                 RET
```

3.47 PROUT: skicka tecknet E (med paritet enligt PARITY). $7247

V24 via PIO B (port $3A): bit 3 = utgående data (1 = vila), bit 4 sätts till 0 under sändningen och till 1 efteråt, bit 1 = 1 betyder att skrivaren inte är klar. Startbit 0, 8 databitar (minst signifikant först), stoppbit(ar) 1; varje bit hålls BITDLY. Efteråt en paus på 10 bittider.

Centronics (CENOUT): OUT $BC till port $01 och $03 (kortval och C2 på ABC-bussen; Luxors skrivarkort jämför bara bit 0-5 i kortvalet och har ingen ingång för C2, Westh 1980, s. 55, bild 33), vänta tills IN ($01) har bit 4-6 = 0, sedan tecknet till port $00.

In: E = tecknet, IX = FCB. Förstör A och F; BC, DE och HL bevaras.

```
7247  CD EA 72    PROUT:  CALL UNTYPE
724A  FE 56                CP   $56             V24?
724C  20 45                JR   NZ,CENOUT
724E  C5                   PUSH BC
724F  E5                   PUSH HL
7250  CD A8 72             CALL PARITY
7253  4F                   LD   C,A             C = tecknet
7254  06 0A                LD   B,$0A           start + 8 bitar + stopp
7256  3A E4 FF             LD   A,(PRBAUD)      hastighetssiffran
7259  FE 31                CP   $31             110 baud: två stoppbitar
725B  20 01                JR   NZ,L725E
725D  04                   INC  B
725E  DB 3A       L725E:   IN   A,($3A)
7260  F6 08                OR   $08             vila (1), bit 4 = 0
7262  E6 EF                AND  $EF
7264  D3 3A                OUT  ($3A),A
7266  CD 8B 71    L7266:   CALL PRBRK           BREAK?
7269  DB 3A                IN   A,($3A)         PIO B
726B  E6 02                AND  $02             skrivaren klar (CTS)? AND ger CY = 0
726D  20 F7                JR   NZ,L7266        vänta på CTS
726F  DB 3A       L726F:   IN   A,($3A)
7271  38 04                JR   C,L7277         biten i CY (först 0: startbiten)
7273  E6 F7                AND  $F7             TxD = 0 (bit 3)
7275  18 02                JR   L7279

7277  F6 08       L7277:   OR   $08             TxD = 1 (bit 3)
7279  D3 3A       L7279:   OUT  ($3A),A         biten ut
727B  CD 94 71             CALL BITDLY          en bittid
727E  37                   SCF
727F  CB 19                RR   C               nästa bit, ettor in (stoppbitar)
7281  10 EC                DJNZ L726F
7283  06 0A                LD   B,$0A           paus 10 bittider
7285  CD 94 71    L7285:   CALL BITDLY
7288  10 FB                DJNZ L7285
728A  DB 3A                IN   A,($3A)
728C  F6 10                OR   $10             bit 4 = 1
728E  D3 3A                OUT  ($3A),A
7290  E1                   POP  HL
7291  C1                   POP  BC
7292  C9                   RET

7293  CD 8B 71    CENOUT:  CALL PRBRK
7296  3E BC                LD   A,$BC           kortval och port $03
```

```
7298  D3 01           OUT  ($01),A
729A  D3 03           OUT  ($03),A
729C  DB 01           IN   A,($01)
729E  E6 70           AND  $70          Centronics klar?
72A0  20 F1           JR   NZ,CENOUT
72A2  CD A8 72        CALL PARITY
72A5  D3 00           OUT  ($00),A      tecknet till port $00
72A7  C9              RET
```

PARITY: *A = E med bit 7 enligt paritetsbokstaven: S oförändrad, M bit 7 = 1, O udda och E jämn paritet över alla 8 bitar. In: E, IX = FCB. Förstör F.*

```
72A8  DD 7E 0F     PARITY: LD   A,(IX+$0F)
72AB  FE 53                CP   $53          S: oförändrad
72AD  20 02                JR   NZ,L72B1
72AF  7B                   LD   A,E
72B0  C9                   RET

72B1  FE 4D        L72B1:  CP   $4D          M
72B3  20 04                JR   NZ,L72B9
72B5  3E 80                LD   A,$80
72B7  B3                   OR   E
72B8  C9                   RET

72B9  FE 4F        L72B9:  CP   $4F          O
72BB  7B                   LD   A,E
72BC  20 05                JR   NZ,L72C3
72BE  A7                   AND  A            redan udda: klart
72BF  E0                   RET  PO
72C0  EE 80                XOR  $80
72C2  C9                   RET

72C3  A7           L72C3:  AND  A            E
72C4  E8                   RET  PE
72C5  EE 80                XOR  $80
72C7  C9                   RET
```

PRCLOS: *sidslut* (PRPEND). *För Centronics sedan* OUT $3C *till port* $01 *och* $03*: kort 60 utan bit 7, som på Luxors skrivarkort är samma kort (Westh 1980, s. 55, bild 33).* PRPEND *ska ge sidmatning om filen öppnades med PREPARE och skrivaren inte står i början av en sida. FCB+*$15 *är alltid 0* (PRDRV)*, så* PRPEND *gör aldrig något och lämnar A = 1. In: IX = FCB. Ut: CY = 0. Förstör A och F.*

```
72C8  CD EA 72     PRCLOS: CALL UNTYPE
72CB  FE 56                CP   $56          V24: bara PRPEND
72CD  28 0A                JR   Z,PRPEND
72CF  CD D9 72             CALL PRPEND
72D2  3E 3C                LD   A,$3C        Centronics: kort 60 utan bit 7
72D4  D3 01                OUT  ($01),A
72D6  D3 03                OUT  ($03),A
72D8  C9                   RET

72D9  DD 7E 15     PRPEND: LD   A,(IX+$15)
72DC  EE 01                XOR  $01          PREPARE? (aldrig)
72DE  C0                   RET  NZ
72DF  CD 00 73             CALL UNLCNT       sidans början: inget
72E2  A7                   AND  A
72E3  1E 0C                LD   E,$0C
72E5  C4 BB 71             CALL NZ,PRCHR
72E8  AF                   XOR  A
72E9  C9                   RET
```

Läs och skriv i PRUNIT*-blocket.* UNITIX*: IX = blockets adress ur FCB+*$12/$13*, HL bevaras.* UNTYPE*: A =* [0] (C/V)*,* UNFORM*: A =* [5] (A-D)*,* UNLCNT*: A = radräknaren* (+$0B)*,* UNSCNT*: radräknaren = A. In: IX = FCB, som de fyra lämnar orörd; de förstör inget annat än A* (UNSCNT *inte ens det*).

```
72EA  DD E5        UNTYPE: PUSH IX
72EC  CD 16 73             CALL UNITIX
72EF  DD 7E 00             LD   A,(IX+$00)   C eller V
72F2  DD E1                POP  IX
72F4  C9                   RET
```

```
72F5  DD E5        UNFORM: PUSH IX
72F7  CD 16 73             CALL UNITIX
72FA  DD 7E 05             LD   A,(IX+$05)
72FD  DD E1                POP  IX
72FF  C9                   RET

7300  DD E5        UNLCNT: PUSH IX
7302  CD 16 73             CALL UNITIX
7305  DD 7E 0B             LD   A,(IX+$0B)
7308  DD E1                POP  IX
730A  C9                   RET

730B  DD E5        UNSCNT: PUSH IX
730D  CD 16 73             CALL UNITIX
7310  DD 77 0B             LD   (IX+$0B),A
7313  DD E1                POP  IX
7315  C9                   RET

7316  E5           UNITIX: PUSH HL
7317  DD 6E 12             LD   L,(IX+$12)       PRUNIT
731A  DD 66 13             LD   H,(IX+$13)
731D  E3                   EX   (SP),HL
731E  DD E1                POP  IX
7320  C9                   RET
```

PRWTAB*: radlängder för siffrorna 1-7 (index = siffran,* $7320 *är* RET*-instruktionen). Sedan tabeller för* SPTAB *(antal + tecken):* PRTYPT CV, PARTAB MSEO, FORMTB ABCD. PRDEF *= standard-inställningen (V, S, A, 80 tecken, marginal 0, D, 72 rader, 1200 baud).* BAUDTB*: fördröjning per bit för hastighet 1-7.*

```
7321  28 48 50 78  PRWTAB: DEFM "(HPx"
7325  84 9E FF             DEFB $84,$9E,$FF
7328  02           PRTYPT: DEFB $02
7329  43 56                DEFB "CV"
732B  04           PARTAB: DEFB $04
732C  4D 53 45 4F            DEFB "MSEO"
7330  04           FORMTB: DEFB $04
7331  41 42 43 44          DEFB "ABCD"
7335  56 53 41 33… PRDEF:  DEFB "VSA30D724  "
7340  12 04        BAUDTB: DEFW $0412
7342  7B 01                DEFW $017B
7344  BB 00                DEFW $00BB
7346  5C 00                DEFW $005C
7348  2C 00                DEFW $002C
734A  14 00                DEFW $0014
734C  08 00                DEFW $0008
```

Signatur (ingen referens): BEL FF BEL BEL BEL JL Eskilstuna 1986 *och tio BEL. Texten ABCDVSA30D724 är tabellerna* FORMTB *och* PRDEF*.*

```
734E  07 0C 07 07…         DEFB $07,$0C,$07,$07,$07
7353  4A 4C 20 45…         DEFB "JL Eskilstuna 1986"
7365  07 07 07 07…         DEFB $07,$07,$07,$07,$07,$07,$07,$07
736D  07 07 FF             DEFB $07,$07,$FF
```

Resten av sidorna $7370*-*$77FF *används inte och är* $FF*. (I basicii80.rom ur abc80sim ligger där i stället rester av äldre kod.)*

```
7370  FF FF FF FF…         DEFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
```

$7378–$7FFF: bara $FF (3208 bytes), utelämnas här.

Felkoder

Felnumret är hela byten, 1–255 (0 = BREAK/Stop). Det ges med `RST $10` följt av numret, med ett hopp till en felingång (en `RST $10` med numret efter), med numret i A och ett hopp till ERRJP eller ERRSP, eller (kompilatorn, länkningen och drivrutinerna) med CY = 1 och numret i A. ROM:en har inga feltexter; kolumnen *Betydelse* är tolkningen ur koden (se filhuvudet). *Ges i* är de namngivna rutiner där koden ger felet (enligt kommentarerna och hoppen till felingångarna); listan är inte nödvändigtvis fullständig.

Nr	Hex	Betydelse	Ingång	Ges i
6	$06	–		LOADRC
19	$13	DOS: ingen ledig kanal		FRESLT
20	$14	–		LINLNG, EMIT, SOPEN, DECOMP, DEMIT, BUFXFR, BINPUT
21	$15	felaktigt filnamn / okänd enhet; DOS: filen finns inte	L1103, SPERR	OPENIX, RUN, FRESLT, PRSPEC, SPTAB, SPCHR, SPDIG
32	$20	filen är inte öppen	L1194	CLSPRG, FNDFIL, TRACE
34	$22	–		NULDEV, MERGE, SOPEN, BUFXFR, BINPUT
35	$23	DOS: läsfel		NXTREC
36	$24	DOS: skrivfel		
37	$25	DOS: sektorns huvud stämmer inte		NXTREC, RDREC, DERRDV
38	$26	POSIT: för stor position; DOS: posten finns inte	L39C7	SOPEN, POSIT, NXTREC, SETREC, RDREC, SETEND
39	$27	DOS: filen är skrivskyddad		SLTCPY, NXTREC, WRREC, SETEND
40	$28	DOS: filen får inte tas bort		SLTCPY, NXTREC, SETEND
41	$29	DOS: katalogen är full		DFCRE, DFEXT
42	$2A	DOS: kortet eller enheten svarar inte		NXTREC, RDREC, WRREC, DFNCX, DFCRE
43	$2B	DOS: skrivskydd		NXTREC, SLTCHK, DFCRE
44	$2C	DOS: kanalen är inte öppen		SLTCHK
45	$2D	DOS: felaktigt kanalnummer		SLTCHK, DFCLS
46	$2E	DOS: systemenheten (UFD) saknas		DFCRE, DFFIND
47	$2F	DOS: felvektorns index >= 9		SLTCHK
48	$30	DOS: läsfel i katalogen eller bitkartan		DERR, SLTCHK, DFFIND
52	$34	funktionen finns inte för enheten (CON:, PR:)		CONDRV, EXT7000, PRDRV
53	$35	–		CONINP
58	$3A	–		LOAD, MERGE, BUFXFR, BINPUT
64	$40	NAME: det nya namnet finns (DOS)	L6F66	XNAME
130	$82	spill i flyttalsräkning		CARGS, FARITH, IPOW, FPOW, TXTNUM, FOVFL, DECTOF
131	$83	index utanför fältet		ARRELM, AUTODM
132	$84	heltalsspill		THEN, FTOI, MOD, IMULDV, SPACE, FPOWI, HEX, INP, POKE
133	$85	ADD$ m.fl.: division med 0, för många siffror		FNSTR1, SUBS
134	$86	fel position/längd i LEFT$, RIGHT$, MID$	L2552	MIDF, LEFT, RIGHT, RIGHT1, MID
135	$87	negativt antal i STRING$/SPACE$; TAB med 0 eller negativt,		TAB, SPACE, STRNG
136	$88	MID$-sats: texten längre än fönstret; CHAIN: namnet för långt	L2C11	MID, CHAIN
137	$89	strängen längre än variabelns maxlängd		STSVAR, MKARR, SVALOC, SALLOC, SOPEN, GET
138	$8A	ON-index utanför listan	L2C91	ON
140	$8C	RETURN utan GOSUB / fel typ		FNAMRK, FRETF, FNEND, GOSUB, RETURN
141	$8D	READ: slut på DATA		READ
142	$8E	–		FPOW, SIN, TAN, TRGEND, LOG, EXP, LOG10, SQR
143	$8F	SYS med okänt nummer		SYS
144	$90	felrad körd ($8B)	ERR90	JPTABL, DOLINE

Nr	Hex	Betydelse	Ingång	Ges i
145	$91	FNEND utan RETURN		FNEND
146	$92	PRINT USING: tom mall, mall utan fält, tal till strängfält,	L1E29, L1F3E	PRINT, TAB
147	$93	READ/INPUT: , väntas		RDVALS
148	$94	INPUT: för få värden		RDVALS
149	$95	RESTORE till rad utan DATA		RESTR0, RESTOR
150	$96	INPUT: för många värden		RDVALS
151	$97	RESUME utan fel		ON, ERRHND, RESUME, RESCHK
180	$B4	raden finns inte		LLINE, ERRHND, CMDGO, RENUM
181	$B5	hopp in i/ut ur DEF FN		LLINE, ERRHND, CMDGO
182	$B6	FOR/WHILE utan NEXT/WEND		LINK, LDEF
183	$B7	NEXT vars FOR inte har körts (som vid länkningen); NEXT/WEND utan FOR/WHILE		NEXT, BLKPOP
184	$B8	NEXT/WEND passar inte		BLKPOP
185	$B9	FOR med öppen styrvariabel		LFOR
186	$BA	NEXT med lokal variabel		LNEXT
187	$BB	FN utan DEF		LFNCAL
188	$BC	DEF FN två gånger		LINK
189	$BD	DEF/FNEND i fel ordning		LINK, LDEF
190	$BE	fält med fel antal index		VARLNK
191	$BF	strängparameter tilldelas		VARLNK
200	$C8	odefinierad kod	UNSDEF	RAMINI, JPTABL, FNTAB, CXSTM, XSTM, FNDTAB, LNKTAB, UNSRUT
201	$C9	minnet räcker inte		DOLINE, INSLIN, NEWSTK, LNRNG, MKARR, DIMERR, SCAT, SPACE, FNCALL, GOSUB, CEXLVL, MEMERR
202	$CA	programmet förstört/skyddat		DOLINE, SAVE, PRGCHK
203	$CB	–		LNRNG
204	$CC	–		LOAD, MERGE
205	$CD	COMMON passar inte (ALLOCC)		REFADR, LNRNG
206	$CE	FN i direktläge före RUN		LINK1
207	$CF	CON utan avbrott/fel		ERROR, ERRA, ERRHND, CONCHK
208	$D0	otillåtet i direktläge		CON, LINK1, LNKONE
209	$D1	fel radintervall / RENUMBER	L15C0	LNRNG, RENUM
210	$D2	felaktigt tal eller radnummer		CARGS, CDEF, PRINT, VAL, RDVALS, CMDGO, FNBCD, FNSTR1, TXTNUM, STRDEC
211	$D3	SINGLE/DOUBLE: flyttal finns		CLRALL, LNRNG, INTEGR, DOUBLE
220	$DC	okänd sats		CMPSTM, CPRINT, SHNAME, CEXPR, KWSRCH
221	$DD	radslut väntas (CR, !)		CLET, CDIM, CARGS, CIF, CEND, CFOR, CXY, CINLIN, TRACE, SHNAME
222	$DE	satsen måste stå först på raden (END, DATA, DEF, FNEND)		CMPST2, STMEND
223	$DF	fel antal/typ av argument till FN; sträng väntas, fel antal		CEXPI, CARGS, PRINT, MIDF, FNCALL, LFNCAL, CTYPE
224	$E0	tal väntas (eller strängvariabel)		CCOMMN, CARGS, CINLIN, EXECA, SPACE, WHILE, OCT, CEXLVL, INP, CTOINT
225	$E1	variabel väntas		CCOMMN
226	$E2	fel sats efter ON		CON
227	$E3	, väntas		CXY, CXY2
228	$E4	= väntas		CMPSTM, CEQUAL, CDEF
229	$E5	) väntas		CFACT, CARGS, CDEF, MKARR
230	$E6	AS FILE väntas		CINLIN, COPEN
231	$E7	AS saknas i NAME (DOS)		CNAME
232	$E8	TO väntas		CFOR
233	$E9	radnummer väntas		DOLINE, LOAD, CLNR, CON, CDEF
234	$EA	bokstav väntas (operand saknas: A=, (), A=1:)		DOLINE, CCOMMN, CFOR, CHR, SHNAME, VNAME, VARPTR, CEXPR

Tokens och tabeller

Tabellerna här är lästa ur ROM-filen med etiketternas adresser. Nyckelordslistorna har formen kod, namn, ..., `$FF`; ett `$00` efter ett namn betyder att nästa tecken inte kontrolleras. Satserna och funktionerna letas upp via tabellhuvudena (`STHDR`, `FNHDR`), så att ett tilläggsprogram kan lägga in egna ord först (DOS:en gör det för `BYE`, `KILL` och `NAME`).

Direktkommandon

Ur `CMDKW` och `CMDTAB`; koden − $80 är index i CMDTAB.

Kommando	Kod	Rutin
RUN	$80	RUN
SCR	$81	NEW
NEW	$81	NEW
LOAD	$82	LOAD
CON	$83	CONT
LIST	$84	LIST
CLEAR	$85	CLEAR
MERGE	$86	MERGE
RENUMBER	$87	RENUM
REN	$87	RENUM
ERASE	$88	ERASE
AUTO	$89	AUTO
ED	$8A	ED
SAVE	$8B	SAVE
UNSAVE	$8C	UNSVEC
GOTO	$8D	CMDGO
RESUME	$8E	CMDRES

Satser

Grupp 1 ur `STKW`: koden $k = \$80$–$\$A4$ står först i satsens interna kod. Kompileringen står i `JPTABL` med index $k - \$80$, körningen med index $\$3D + k - \$80$. Grupp 2 ur `STKW2` skrivs som `$86` (`XSTM`) följt av koden; index $\$25 + k - \$80$ respektive $\$62 + k - \$80$. Satser utan kompileringsrutin (0) har ingen argumentdel eller kompileras av en annan sats.

Kod	Ord	Kompilering	Körning
$80	GOTO	CLNR	GOTO
$81	–	CLNR	GOTO
$82	GOSUB	CLNR	GOSUB
$83	RESTORE	CLNR0	RESTOR
$84	RESUME	CLNR0	RESUME
$85	ON ERROR GOTO	CLNR0	ONERR
$86	XSTM	CXSTM	XSTM
$87	–	0	EOL
$88	–	0	NOHOOK
$89	–	0	READY
$8A	–	0	SKIP
$8B	–	0	ERR90
$8C	DEF FN	CDEF	DEF
$8D	FNEND	NOHOOK	FNEND
$8E	DATA	CDATA	SKIP2
$8F	REM	CTEXT	SKIP
$90	!	CTEXT	SKIP

Kod	Ord	Kompilering	Körning
$91	LET	CLET	EXECOD
$92	END	CEND	ENDST
$93	PRINT	CPRINT	PRINT
$94	;	CPRINT	PRINT
$95	IF	CIF	EXECOD
$96	INPUT	CINPUT	PRINT
$97	INPUT LINE	CINLIN	INPLIN
$98	FOR	CFOR	FOR
$99	NEXT	CNEXT	NEXT
$9A	WHILE	CWHILE	WHILE
$9B	WEND	CWEND	GOTO
$9C	READ	CREAD	READ
$9D	–	0	RESTR0
$9E	–	0	RESUM0
$9F	–	0	ONERR0
$A0	RETURN	CRETRN	RETURN
$A1	ON	CON	ON
$A2	MID$, MID	CMID	MID
$A3	SINGLE	SINGLE	NOHOOK
$A4	DOUBLE	DOUBLE	NOHOOK

Satser, grupp 2

Kod		Ord	Kompilering	Körning
$86	$80	DIM	CDIM	EXECOD
$86	$81	COMMON	CCOMMN	EXECOD
$86	$82	POKE	CPOKE	POKE
$86	$83	OUT	COUT	OUTST
$86	$84	OPEN	COPEN	SOPEN
$86	$85	PREPARE	COPEN	PREPAR
$86	$86	CLOSE	CCLOSE	SCLOSE
$86	$87	RANDOMIZE	NOHOOK	RANDOM
$86	$88	STOP	NOHOOK	STOP
$86	$89	TXPOINT	CTXPNT	UNSDEF
$86	$8A	GET	CGET	GET
$86	$8B	CHAIN	CEXPS	CHAIN
$86	$8C	TRACE	CFILNR	TRACE
$86	$8D	NO TRACE	NOHOOK	NOTRAC
$86	$8E	PUT	CPUT	PUT
$86	$8F	INTEGER	INTEGR	NOHOOK
$86	$90	FLOAT	FLOAT	NOHOOK
$86	$91	EXTEND	EXTEND	NOHOOK
$86	$92	NO EXTEND	NOEXT	NOHOOK
$86	$93	DIGITS	CEXPI	DIGITS
$86	$94	OPTION BASE	CEXPI	OPTBAS
$86	$95	POSIT	CPOSIT	POSIT
$86	$96	SET DOT	CXY	UNSDEF
$86	$97	CLR DOT	CXY	UNSDEF

Satser i DOS:en

Tabellhuvudet DOSSTH (första kod $\$A0$, antal 4) kopieras till RAM och läggs först i kedjan. Här står rutinerna i egna tabeller: körningen på $+4$, kompileringen på $+8$, med ordets kod $-$ \$80 som index. Hur koden $\$A0$ hänger ihop med ordens koder är inte utrett (?).

Kod	Ord	Kompilering	Körning
$80	BYE	$0059	XBYE
$81	KILL	L6F80	XKILL
$82	NAME	CNAME	XNAME

Övriga nyckelord

Små tabeller efter STKW, som kompileringsrutinerna söker i med KWSRCH. Koden skrivs i den interna koden.

Tabell	Kod	Ord
$09E8	$8B	?
$09E8	$88	:
$09E8	$8A	ELSE
$09F3	$BE	STEP
$09F9	$BD	TO
$09FD	$BB	AS FILE
$0A06	$B9	COUNT
$0A0D	$B8	USING
$0A14	$B6	LOCAL
$0A1B	$BD	THEN

Operatorer

Grupperna i OPGRP (pekare till nyckelorden och prioritet), i tabellens ordning. Jämförelser, + och - har tre koder (flyttal, heltal, sträng), * och / två (flyttal, heltal). Körrutinen står i JPTABL med index $\$7D + kod - \$80$. NOT är unär och ligger i en egen grupp.

Grupp	Prio	Operator	Kod	Körning
$06C8	$03	EQV	$84	IEQV
$06CD	$03	IMP	$85	IIMP
$06D2	$03	OR	$86	IOR
$06D2	$03	XOR	$87	IXOR
$06DA	$03	AND	$88	IAND
$06E4	$82	<=	$98 $99 $9A	FCMP ILE SCMP
$06E4	$82	<>	$8C $8D $8E	FCMP INE SCMP
$06E4	$82	<	$8F $90 $91	FCMP ILT SCMP
$06E4	$82	>=	$92 $93 $94	FCMP IGE SCMP
$06E4	$82	>	$95 $96 $97	FCMP IGT SCMP
$06E4	$82	=	$89 $8A $8B	FCMP IEQ SCMP
$06F4	$81	+	$9B $9C $9D	FARITH IADD SCAT
$06F4	$81	-	$9E $9F	FARITH ISUB
$06F9	$01	*	$A0 $A1	FARITH IMUL
$06F9	$01	/	$A2 $A3	FARITH IDIV
$06FE	$04	^, **	$A4	FPOW

Funktioner

Ur FNKW; koden $-\$80$ är index i FNTAB. Formen och typerna är ur FNARG som CFUNC och CARGS läser den: x flyttal, i heltal, s$ sträng, a valfri typ, v variabel, [ ] får utelämnas, , . . . får upprepas (tolkningen av bitarna är delvis osäker (?)).

Kod	Form	Resultat	Rutin
$80	FNnamn(...)	–	FNCALL
$81	FIX(x)	flyttal	FIX
$82	INT(x)	flyttal	INT
$83	SIN(x)	flyttal	FNFLT
$84	COS(x)	flyttal	FNFLT
$85	TAN(x)	flyttal	FNFLT
$86	ATN(x)	flyttal	FNFLT
$87	SQR(x)	flyttal	FNFLT
$88	PI	flyttal	PI
$89	RND[(x)]	flyttal	RND
$8A	SGN(x)	heltal	SGN
$8B	ASCII(s$), ASC	heltal	ASC
$8C	CHR$(i,...)	sträng	CHR
$8D	LEFT$(s$,i), LEFT	sträng	LEFT

Kod	Form	Resultat	Rutin
$8E	RIGHT$(s$,i),RIGHT	sträng	RIGHT
$8F	MID$(s$,i,i),MID	sträng	MIDF
$90	LEN(s$)	heltal	LEN
$91	INSTR(i,s$,s$)	heltal	INSTR
$92	SPACE$(i)	sträng	SPACE
$93	STRING$(i,i)	sträng	STRNG
$94	SWAP%(i)	heltal	SWAP
$95	PEEK(i)	heltal	PEEK
$96	INP(i)	heltal	INP
$97	CALL(i[,i])	heltal	FCALL
$98	ERRCODE	heltal	ERRCOD
$99	SYS(i)	heltal	SYS
$9A	ABS(x)	flyttal	ABS
$9B	ADD$(s$,s$,i)	sträng	FNBCD
$9C	NUM$(x)	sträng	FNFLT
$9D	TAB(i)	sträng	TAB
$9E	SUB$(s$,s$,i)	sträng	FNBCD
$9F	CUR(i,i)	sträng	CUR
$A0	MUL$(s$,s$,i)	sträng	FNBCD
$A1	TXPOINT(i,i)	heltal	UNSDEF
$A2	DIV$(s$,s$,i)	sträng	FNBCD
$A3	EXP(x)	flyttal	FNFLT
$A4	LOG(x)	flyttal	FNFLT
$A5	LOG10(x)	flyttal	FNFLT
$A6	COMP%(s$,s$)	heltal	FNSTR1
$A7	VAL(s$)	flyttal	FNSTR1
$A9	TIME$	sträng	CLKSTR
$AA	POSIT(i)	flyttal	FPOSIT
$AB	VARPTR(va)	heltal	VARPTR
$AC	VAROOT(va)	heltal	VAROOT
$AD	PEEK2(i)	heltal	PEEK2
$AE	MOD(i,i)	heltal	MOD
$AF	CVT%$(i)	sträng	CVTIS
$B0	CVT$%(s$)	heltal	CVTSI
$B1	CVTF$(x)	sträng	CVTFS
$B2	CVT$F(s$)	flyttal	CVTSF
$B3	HEX$(i)	sträng	HEX
$B4	OCT$(i)	sträng	OCT
$B5	DOT(i,i)	heltal	UNSDEF

Färgord

Ur COLKW (koderna $80–$9F i huvudet $0704); de ger ett heltal, koden $-$ $80 (styrtecknet i teletextens teckenuppsättning (?)).

$81	RED	$88	FLSH	$94	GBLU	$9C	BLBG
$82	GRN	$89	STDY	$95	GMAG	$9D	NWBG
$83	YEL	$8C	NRML	$96	GCYA	$9E	GHOL
$84	BLU	$8D	DBLE	$97	GWHT	$9F	GREL
$85	MAG	$91	GRED	$98	HIDE		
$86	CYA	$92	GGRN	$99	GCON		
$87	WHT	$93	GYEL	$9A	GSEP		

Uttryckskod

Koderna $80–$$CE$ i den kompilerade uttryckskoden, körrutinen i JPTABL (index 7D + kod - $80) och, för $A7–$$CF$, rutinen i DECTAB som LIST använder. Vad varje kod betyder beskrivs vid JPTABL i listningen.

Kod	Operator	Körning	LIST
$80		L2326	
$81		FTOI	

Kod	Operator	Körning	LIST
$82		ITOF	
$83		STRNUM	
$84	EQV	IEQV	
$85	IMP	IIMP	
$86	OR	IOR	
$87	XOR	IXOR	
$88	AND	IAND	
$89	=	FCMP	
$8A	=	IEQ	
$8B	=	SCMP	
$8C	<>	FCMP	
$8D	<>	INE	
$8E	<>	SCMP	
$8F	<	FCMP	
$90	<	ILT	
$91	<	SCMP	
$92	>=	FCMP	
$93	>=	IGE	
$94	>=	SCMP	
$95	>	FCMP	
$96	>	IGT	
$97	>	SCMP	
$98	<=	FCMP	
$99	<=	ILE	
$9A	<=	SCMP	
$9B	+	FARITH	
$9C	+	IADD	
$9D	+	SCAT	
$9E	-	FARITH	
$9F	-	ISUB	
$A0	*	FARITH	
$A1	*	IMUL	
$A2	/	FARITH	
$A3	/	IDIV	
$A4	^,**	FPOW	
$A5		FPOWI	
$A6		IPOW	
$A7		EXECOD	DPAREN
$A8		FNEG	DNEG
$A9		INEG	DNEG
$AA		FNAMRK	DSKIP
$AB		FNAMRK	DSKIP
$AC		FNAMRK	DSKIP
$AD		FTOI2	DSKIP
$AE		ITOF2	DSKIP
$AF		INOT	DNOT
$B0		COLARG	DEMPTY
$B1		SFARR	DASGN1
$B2		SIARR	DASGN1
$B3		SSARR	DASGN1
$B4		STFVAR	DASGN
$B5		STIVAR	DASGN
$B6		STSVAR	DASGN
$B7		FRETF	DDROP
$B8		FRETI	DDROP
$B9		FRETS	DDROP
$BA		PTRF	DDROP
$BB		PTRI	DDROP
$BC		PTRS	DDROP
$BD		THEN	DTHEN
$BE		THEN	L3D95
$BF		NOHOOK	DDROP
$C0		LDFVAR	DVAR
$C1		LDIVAR	DVAR
$C2		LDSVAR	DVAR

Kod	Operator	Körning	LIST
$C3		L232D	DEMPTY
$C4		VARADR	DVAR
$C5		VARADR	DVAR
$C6		VARADR	DVAR
$C7		INTCON	DINT
$C8		ARRELM	DARR
$C9		ARRELM	DARR
$CA		ARRELM	DARR
$CB		STRCON	DSTR
$CC		STRCON	DSTR2
$CD		XDIM	DDIM
$CE		XCOMMN	DCOMMN

Fel och egenheter

Det som inte fungerar som man kan vänta sig, ur bilagan Fel och egenheter i BASIC II i band 2 (där finns hela resonemanget). Adressen är kodraden där det händer och är en länk dit; kodraden har i sin tur en länk hit. Exemplen är prövade i emulatorn med BASIC II (64K); posterna om PR:, klockan och BASICINI.SYS bygger på koden. Felen finns likadant i ABC 800 M:s BASIC II, utom där posten säger något annat.

Fel

Adress	Rutin	Vad som händer
$41FF	DEOL	**Djupt indrag klipper raden i LIST.** Indraget, två blanka per FOR-, WHILE- och DEF-nivå, läggs in sist utan att platsen i bufferten (159 tecken) prövas. För varje blank som inte ryms försvinner ett tecken i slutet, först CR LF: efter tio FOR listas en lång PRINT-rad utan sitt sista citattecken och ihop med nästa rad. Till en fil ger MERGE sedan fel 20.
$0938	STHDR	**XSTM 24–36 hoppar via ROM:ens början.** Huvudtabellens huvud har ingen hopptabell (0), så en kod som inte finns i grupp 2 tas som ordet på adress 2n: XSTM 26 ger Error 26., andra n hoppar ut i RAM.
$1863	CFUNC	**XFN 128–159 tar signaturen ur ROM:ens början.** Huvudets +8 ($0080 för färgorden) tas som signaturtabell: A$=XFN 130 ger fel 223, A%=XFN 130 fel 224, och 10 PRINT XFN 130 listas PRINT GRN. XFN 3(2) är SIN(2).
$2E9C	LINK	**Dubbel DEF FN anges på fel rad.** Rutinen byter den aktuella raden mot den funna posten innan felet lämnas, så fel 188 får den första DEF:ens radnummer.
$6FA2	DOSSTH	**XSTM 163 kör BYE.** Huvudet anger fyra DOS-satser men tabellerna har tre. 10 XSTM 163 startar om datorn redan när raden matas in; XSTM 160–162 listas BYE, KILL och NAME.
$169B	CMID	**Fel i MID$-satsen ger fel felnummer.** Tecknet vid HL skrivs över felnumret: MID$(A$,1)="XY" ger fel 61, MID$(A$,1)+"XY" fel 43, MID$(A$,1,2 fel 13 och MID$(A,1,2)="X" fel 44. MID$(A$,1,2)=5 ger rätt fel 223.
$0D7E	LSTWK2	**TRACE till en fil ger fel 32.** Efter TRACE #1 och STOP ger RUN (likaså CLEAR, NEW och en ny rad) fel 32: filen tas ur FCB-listan innan TRACE skriver sitt radslut, och den stängs aldrig. Andra försöket går.
$2C8E	ON	**ON n ON ERROR GOTO blir ON n RESUME.** Satsen godtas och listas, men körningen skiljer bara ut GOTO, GOSUB och RESTORE; 10 ON 1 ON ERROR GOTO 20 ger fel 151.
$43BE	AUTO	**AUTO vid 65535 ger ett falskt fel.** Spillet lämnar det första inskrivna tecknet minus $30 som felnummer: efter AUTO 65530 ger PRINT 7 Error 32. och REM X Error 34.
$435A	AUTO	**AUTO efter rad 65530 föreslår rad 0.** (sista / 10 + 1) * 10 spiller utan prövning. Med sista raden 65530 föreslår AUTO rad 0, och det som skrivs där körs direkt (0 PRINT 8 skriver 8).
$5FA6	BCDCMP	**Exponenter som skiljer 128 eller mer jämförs fel.** Skillnaden prövas med teckenflaggan: COMP%("1E100","1E-100") ger -1, ADD$("1E70","1E-60",0) ger "0".
$48F3	TXTDEC	**Exponenter över 127 i en talsträng slår runt.** Exponenten är en byte med tecken: PRINT 1E-220 skriver 1.E+36, och 1E200 är 1E-56. Gäller konstanter, VAL, INPUT och ADD$ med flera (LEN(ADD$("1E-200","0",0)) = 57). Över 255 ger fel 210.
$4A13	BSPACE	**Fler än 255 blanka i följd blir för få i en fil.** Antalet packade blanka lagras i en byte och slår runt: PRINT #1,"A";SPACE$(300);"B" läses tillbaka med 44 blanka. 256 blir 0, som läses som 256.
$49E6	BINPUT	**INPUT läser förbi blockets slut utan ETX.** INPUT väntar på ETX i stället för att pröva platsen. PUT kan fylla blocket ända ut, och då läses buffertens början igen.
$56BE	TAN	**SIN, COS och TAN fel för stora x.** x * 4/pi blir ett 16-bitars heltal med tecken: från |x| = 25736 blir värdet fel (SIN(30000) = 1, rätt -.80267), från 51472 fel 142. Från 2^125 slår exponenten runt: SIN(5E37) = 0.
$5A9E	SQR	**SQR fel för x >= 2^126.** Exponenten halveras med tecken, och e + 1 slår runt vid e = 127: SQR(1E38) = 2.93874E-20 i stället för 1E19.
$68D7	DFCLS	**Skrivfel i katalogen ger fel 35.** Felindexet ska ligga i C men A lagras: ett skrivfel i katalogen eller bitkartan vid CLOSE eller KILL ger Error 35. (läsfel), med bit 7 i status Error 99. ABC 800 M ger fel 48.
$706E	PRDRV	**PREPARE "PR:" ger ingen sidmatning vid CLOSE.** A minskas till 0 innan PROPEN sparar det, och PRPEND kräver 1. ABC 800 M sparar A först.

Adress	Rutin	Vad som händer
$714F	PRSPEC	**PR: med hastighet 0 ger under 10 baud.** Siffran prövas bara mot 8; 0 ger index $FE och ordet på $743E utanför BAUDTB.
$71EF	PRCHR	**PR: med AUTO LF gör raderna efter den första ett tecken för långa.** Tecknet som inte fick plats före CR LF räknas inte: med radlängd 40 får alla rader efter den första 41 tecken. Som i skrivarkretsen 9704.
$7215	PRCHR	**PR: BS verkar inte från kolumn 129.** JP P efter DEC (IX+6) tar kolumnen som negativ, och BS gör ingenting. Likadant i ABC 800 M.
$3AB3	NMICLK	**Klockan går knappt en minut per dygn för fort.** Klockan stegas efter 50 avbrott, men efter 49 när sekunden är 0 eller 32: 2998 avbrott per minut. Talen är bytta från ABC 800 M:s utan att räkningen stämmer.
$6DF1	INIERR	**Fel som saknas i BASICINI.SYS ger skräp.** CPIR räknar med det som råkar stå i AF och söker vidare efter bufferten; det som följer den första träffen skrivs som text.
$6DD1	INIERR	**Efter ett läsfel i BASICINI.SYS skrivs alla fel som nummer.** Kroken ERRTXV är avstängd medan posten läses och sätts inte tillbaka efter ett läsfel.

Gränsfall i talen

Adress	Rutin	Vad som händer
$2E09	FPOW	**Negativ bas med stor heltalsexponent ger fel 130.** Från y = 16384 räknas x^y som EXP(y*LOG(x)): (-1)^16383 = -1, (-1)^16384 ger fel 130.
$222F	IADD	**Heltalsräkning slår runt utan fel.** 32767%+1% = -32768 och 200%*200% = -25536; multiplikationen ger fel 132 först över 65535.
$2162	FTOI	**Heltal slår runt vid tilldelning.** Avrundat modulo 65536 under 65536: A%=40000 ger -25536, A%=32767.6 -32768, A%=-40000 25536.
$5CDB	ADDS	**ADD$ och SUB$ stryker siffror före avrundningen.** Siffror efter n + 2 decimaler stryks utan avrundning: SUB$("1","0.005001",2) ger "1.00" (rätt 0.99).
$5E4A	MULS	**MUL$ hugger av delprodukterna.** Den sista siffran stryks när multiplikanden når resultatets sista plats: MUL$("0.7","0.767",0) ger "0" (rätt 1).
$56DD	TAN	**SIN, COS och TAN sämre för negativa x.** Heltalsdelen huggs mot 0, så resten blir negativ och 1 - resten hamnar utanför seriens område: SIN(-3) = -.141181, -SIN(3) = -.14112.
$5887	LOG	**LOG nära 1 tappar siffror.** (e - 1/2) * ln 2 och ln(m * sqr 2) tar nästan ut varandra: LOG(.9999) = -1.00076E-04, rätt -1.00022E-04.
$5306	DECTOF	**VAL, INPUT och READ avrundar fel i enkel precision.** Bara heltalets fyra första byte räknas med tiopotensen: VAL("1.23456789") blir 1.19209E-07 större än konstanten 1.23456789.
$533E	P10LD	**Små tal går inte att skriva in.** Division med 10^n, n >= 39, ger fel 130: 1E-38 går, 1.5E-38 och 1E-40 ger fel 130, men PRINT 1.5E-8*1E-30 skriver 1.5E-38.
$4F77	FADD	**Subtraktion avrundar ibland fel.** Bitarna under skyddsbyten försvinner innan talen dras ifrån: med A = 2^24 ger (A+2)-1.0000001-A 2 och A-(A-0.375) 1, båda i stället för 0.

Egenheter i språket

Adress	Rutin	Vad som händer
$1C51	CINPUT	**INPUT A;B listas INPUT A,B.** Skiljetecknet mellan variablerna sparas inte.
$31E0	LNEXT	**FOR med lokal variabel går inte att avsluta.** FOR med en parameter eller LOCAL-variabel i en DEF FN godtas, men NEXT ger fel 186.
$2FB6	VARLNK	**Strängparametrar kan inte tilldelas.** J$="X" med parametern J$ ger fel 191; talparametrar och LOCAL-strängar går.
$37B0	CEXPR	**Unärt minus bara först eller efter en jämförelse.** 2*-3 ger fel 234, -NOT 0 fel 220; 2*(-3) och 1=-1 går. NOT 2+3 = -6, 2=NOT 1 ger fel 220.
$2CC7	TRCHOK	**TRACE skriver 0 vid programmets slut.** Utan END skrivs slutmärkets radnummer 0.
$2B99	FRETS	**RETURN-värdet måste lämnas på funktionens egen nivå.** RETURN 9 i en subrutin som funktionen anropat ger fel 140, liksom RETURN "A" i en talfunktion.
$2283	NEGHL	**Heltalsdivision avrundar nedåt.** 7%/(-2%) = -4, MOD(-7%,2%) = 1, men -7%/2% = -3 (minus gäller hela termen).
$1FEA	TAB	**PRINT USING med färre värden än fält ger fel 146.** PRINT USING "X##Y##Z";1 skriver X 1Y och ger fel 146.
$11DC	DOLINE	**Ny rad efter en RUN som fyllt minnet ger fel 201.** Raden kompileras innan variablerna tas bort: efter DIM A%(6352) på 16 KB ger 20 PRINT 1 fel 201, 30 REM går in och därefter går allt.

Adress	Rutin	Vad som händer
$1213	DOLINE	**Felrader sparas som text.** En rad som inte går att kompilera när ett textprogram läses sparas som text och listas med ? efter numret; RUN ger fel 144 när raden nås.
$133A	CMPST2	**END går inte efter THEN.** IF 1 THEN END ger fel 222; STOP eller ett hopp till en END-rad går.
$15C2	LNRNG	**COMMON jämförs bara efter storlek.** Värdena förs över efter plats: COMMON B tar emot COMMON A, COMMON A% mot COMMON A ger fel 205.
$15C2	LNRNG	**Misslyckad LOAD eller CHAIN lämnar inget program.** Programmet raderas innan filen läses; fel 201, 203 eller 205 lämnar ett tomt minne.
$23D0	MKARR	**DIM igen: större ger fel 137.** DIM A(5), DIM A(3) går, DIM A(9) ger fel 137; likaså DIM A$=5 och DIM A$=10.
$23BD	MKARR	**DIM A(-1) är ett odimensionerat fält.** Fältet får 0 element och blir (10) vid första användningen (A(5)=1 går); DIM A(-2) ger fel 201.
$2575	SCAT	**Sammanfogning tar dubbla platsen.** + bygger resultatet under båda operanderna på stacken: på 16 KB ger STRING$(3200,65)+STRING$(3200,65) fel 201; med variabler som operander går det.
$2839	NEXT	**Heltals-FOR slår runt.** FOR I%=32760 TO 32766 STEP 5 ger 32760, 32765, -32766 och tar aldrig slut.
$28D7	RESTOR	**RESTORE till en rad utan DATA ger fel 149.** Raden måste börja med DATA; ABC80-BASIC:en gick vidare till nästa.
$29C9	RDVALS	**INPUT frågar inte om.** För få värden ger fel 148, för många 150 och fel tecken (1X,2) 147; variablerna är orörda.
$3BBB	ERRHND	**ON ERROR gäller en gång om programmet saknar RESUME.** Utan RESUME någonstans stängs ON ERROR av när felrutinen nås; felrutinen kan sätta det igen.
$2C5D	RETURN	**Vissa fel fångas inte av ON ERROR.** RETURN utan GOSUB (140), full stack vid GOSUB (201) och RESUME utanför felrutinen (151) avbryter alltid; CON ger sedan fel 207.
$3C28	RESUME	**RESUME efter vissa fel ger ett nytt fel.** RESUME fortsätter bara efter fel i räkningen (1/0 ger 1.70141E+38); efter MID$ utanför strängen (134) ger det fel 207, som samma felrutin fångar om och om igen.
$33E3	NAMTX1	**LIST slår på EXTEND.** Efter NO EXTEND ger 20 PRINT DELTA fel 220, men efter en LIST som skriver ett långt namn godtas samma rad.
$3416	NAMPAK	**Långa namn försvinner inte.** Namnen står kvar i namnlistan, också från direktläge, och sparas med programmet; bara NEW tömmer den.
$4B92	KWSRCH	**Nyckelord kräver blanka med EXTEND.** PRINTALPHA ger fel 220, FOR I=1TO2 fel 221 och NOEXTEND fel 220; utan EXTEND listas FOR I=1TO2 som FOR I=1 TO 2.
$3731	CVTSI	**CVT$% och CVT$F prövar inte längden.** CVT$%("") ger ett skräpvärde, inget fel.
$3A59	INTEGR	**INTEGER och EXTEND gäller från inmatningen.** 20 INTEGER gör också rad 5, som skrivs in efter den, till en heltalsrad, och NEW behåller läget.
$3AA8	DIGITS	**DIGITS, OPTION BASE, CHR$, STRING$ och CUR tar låga byten.** DIGITS -1 godtas, CHR$(321) = "A", STRING$(3,322) = "BBB".
$399A	POSIT	**POSIT räknar 253 byte per block, PRINT skriver 252.** PRINT # lämnar blockets sista plats åt ETX, så POSIT 253 och framåt motsvarar inte textens byte.
$3C1A	RESUM0	**Misslyckat RESUME i direktläge tar bort CON.** Efter STOP ger RESUME fel 151 och CON sedan fel 207; GOTO till en rad som inte finns (180) behåller CON.
$4250	DCHK	**En godtagen rad kan vara för lång för LIST.** 1E6 listas 1.E+06: en rad med 23 1E6 godtas men ger fel 20 vid LIST.
$3E2B	DOPR	**Mellanslag före NOT efter ; och (.** Tecknet före i bufferten avgör, och en vänsterparentes räknas inte: A;NOT B listas A; NOT B och A*(NOT B) A*(NOT B), men PRINT (NOT A) står kvar.
$42C8	RENUM	**RENUMBER prövar numret efter sista raden.** Med raderna 65531, 65533 och 65535 ger RENUMBER 65531,2 fel 209, RENUMBER 65529,2 går.
$4723	FMRND	**Avrundning till 0 behåller ibland minustecknet.** PRINT USING "##.#";-0.04 ger -0.0, med "#.#" -.0; -0.0004 ger 0.0.
$4457	NUMS	**NUM$ skriver talet utan exponent där PRINT har en när DIGITS är över 7.** NUM$ och ADD$ m.fl. tar högst 7 siffror (17 i DOUBLE) men väljer formen efter DIGITS platser, medan PRINT väljer den efter siffrorna. Med DIGITS 10 ger NUM$(12345678) 12345680 men PRINT 12345678 1.234568E+07, och ADD$("12345678","0",-2) ger 12000000 (med DIGITS 7 1.2E+07). I DOUBLE med DIGITS 20 ger NUM$(1E18) 100000000000000000 men PRINT 1E18 1.E+18.
$4AB6	PARSFN	**Filnamnen tolkas fritt.** Enhetsnamnet huggs av efter tredje tecknet ("DR0XYZ:H" går), små bokstäver blir versaler och ‘ blir @, men @ själv ger fel 21.
$6D37	XBYE	**BYE vid skivfel skriver Abort utan slut.** ABORT faller in i XBYE igen, så Abort nn skrivs om och om igen; utan CMDINT.SYS blir det en kallstart.
$71E9	PRCHR	**PR: räknar inte en rad som skrivaren själv bryter.** Utan AUTO LF hamnar sidmatningen en rad längre ned för varje sådan rad. ABC 800 M räknar raden.
$7142	PRSPEC	**PR: en felaktig hastighet förstör den sparade inställningen.** Tecknen kopieras innan hastigheten prövas: efter OPEN "PR:VMA20C70.9" ger också OPEN "PR:" fel 21.

Adress	Rutin	Vad som händer
$70C5	PRSPEC	**PR:-filerna har samma typ.** Inställningen står på ett enda ställe: öppnas en andra PR:-fil som V går också den första via V24.
$4982	DVEND	**PRINT mitt i en fil avslutar den.** 60 rader, sedan `POSIT #1,0`, `PRINT #1,"Y"` och `CLOSE 1`: bara Y går att läsa, nästa `INPUT LINE` ger fel 34. Med `PUT` är alla raderna kvar.
$6E1C	INIERR	**BASICINI.SYS hittas genom katalogposten.** Filen öppnas igen med postens nummer, inte med namnet: efter `NAME` finns texterna kvar.
$1D92	THEN	**IF-villkoret avrundas till närmaste heltal.** `IF .5` och `IF -.5` är sanna, `IF .4` falskt, `IF 65536` ger fel 132.
$11FE	DOLINE	**Radnummer 0 och över 65535 körs direkt.** `0 PRINT 7` skriver 7; `65536 PRINT 6` ger fel 234.

Oklara ställen

Ställen i listningen där tolkningen är osäker, markerade med (?) i kommentarerna, och det som enligt kommentarerna inte används. Adressen är kodraden som kommentaren hör till (för en blockkommentar raden efter den) och är en länk dit.

Osäkra tolkningar

Adress	Rutin	Kommentar
$13C5	STMEND	$8C = DEF, $8D = FNEND, $8E = DATA; END ger också fel 222 ($92 (?)); $00 (?).
$2F3A	VARLNK	Namnet är två bytes: B = bokstaven (1-29), C = siffran * 8 eller $78 utan siffra; de låga bitarna ger sorten: bit 0 heltal (%), bit 1 sträng ($), bit 2 fält (?).
$2F3A	VARLNK	Saknas variabeln skapas en post med ALLOC: +0,+1 länk, +2,+3 namn, +4.. värdet (heltal 2 bytes, flyttal 4/8 bytes enligt IY+$1E, sträng 6 bytes, fält 6 bytes per dimension (?)).
$34B1	RND	frö = frö * $00BB40E64D + $1440B60CA0 (mod 2^40) (?)

Oanvänt

Adress	Rutin	Kommentar
$0170	IM2INI	Bytes $0170-$017A används inte.
$3552	PSCAN	variabeln före går som $0A med IY+$1D bit 5 satt Sort 4 används inte.
$61CC	LOADF	typbyte $FF = resten av sektorn hoppas över; $00 = post antal (C) (en byte, används inte) adress hög, hög inverterad adress låg, låg inverterad data C bytes kontrollsumma
$63F5	DERR	$6422 (LD C,$1A) nås inte; fel 48 kommer från katalogfunktionerna (DFNBC = $1A vid läsfel i katalogen eller bitkartan, $6669).
$6422	SLTCHK	DEFB $0E,$1A,$2E: LD C,$1A (fel 48), nås inte
$68CE	DFCLS	LD C,$0A: ABC800 M: LD A; L6675 lagrar A, C används inte
$7000	EXT7000	PRINIT länkar in enhetsposten PRDEV ("PR ", drivrutin PRDRV) i DEVLST, reserverar ett arbetsområde under stacken (PRAREA) och kopierar standardinställningen PRDEF till PRDEFR ($FFC0, i den oanvända delen av IM 2-tabellen).
$703F	PRAREA	Det andra blocket används inte.
$7145	PRSPEC	Siffran 0 godtas men ger index $FE och läser ordet på $743E, utanför tabellen: $312C, tecknen ,1 ur BASIC-texten i den oanvända delen.
$7370	BAUDTB	Resten av sidorna $7370-$77FF används inte och är $FF.

Källor

Arne Stockman: *Programvaran i ABC80. Disassemblering med kommentarer*. ABC-klubben, Rapport nr 1, april 1981, 172 s.

Kommenterad disassemblering av ABC80:s BASIC-ROM, handskrivna kommentarer på A3-uppslag. Arbetet med ABC80-ROM:en (../dis) bygger delvis på den.

Inskannad: `https://www.abc80.net/archive/luxor/ABC80/`

Johan Finnved: *Programvaran i persondatorn ABC80*. Examensarbete, Institutionen för tillämpad elektronik, KTH, godkänt 1979-01-29 (utfört vid Dataindustrier AB, Täby).

Beskrivning av ABC80-BASIC:en av den som skrev den: kompilering till postfix, länkning vid `RUN`, `LIST`, enheter. Citeras `(Finnved 1979)`.

Inskannad: `https://dflund.se/~triad/diab/archive/ABC%2080/`

Gunnar Markesjö: *Mikrodatorns ABC. Elektroniken i ett mikrodatorsystem*. Esselte Studium, 1978. ISBN 91-24-29008-4.

Lärobok som bygger på ABC80: elektroniken, PIO:ns inkoppling, grafiken, kassetten och realtidsklockan.

H. Peter Anvin m.fl.: *abc80sim*. Emulator för ABC80.

Dess ROM-filer för ABC80-BASIC:en är identiska med versionerna 11273 och 9913. Uppgifter därifrån, främst om portarna, är markerade ”(abc80sim)”.

`https://github.com/sasq64/abc80sim`

Symbolförteckning

ABORT $6D16 284
ABS $20F6 80
ADDS $5CD3 241
ADRREF $0F49 31
ALLOC $0FD3 33
ALLOCC $0FAE 33
ALNUM $4BAF 199
ARRELM $2332 87
ASC $2523 93
ATN $5B50 239
AUTO $4336 175
AUTODM $237E 87
AUTOTX $002E 6
BASDIG $4517 181
BASSTR $4515 181
BAUDTB $7340 298
BCDADD $5DA9 244
BCDARG $5FEE 250
BCDCMP $5F73 249
BCDEND $5D75 243
BCDER3 $5D95 243
BCDERR $5D99 243
BCDFIX $5DDB 244
BCDLEN $5FB4 249
BCDNUL $5F6B 249
BCDNXT $5FF6 250
BCDRES $5E71 246
BCDSUB $5DC2 244
BCDZ $5F61 248
BEEP $022A 12
BGET $49D7 194
BINPUT $49E3 194
BITDLY $7194 294
BLANK $02C0 13
BLKPOP $31E6 129
BLKPSH $31CC 128
BPRINT $4A31 195
BPUT $49CD 194
BSPACE $4A13 195
BUFADR $0051 7
BUFBLK $4A5F 196
BUFFLU $4A5C 196
BUFRD $637B 261
BUFRD2 $6391 261
BUFWR $63A0 261
BUFWR2 $63B8 261
BUFXFR $49A6 194
CARGS $188D 57
CCLOSE $1CD7 69
CCOMMN $16EB 52
CDATA $1A70 62
CDEF $1AFD 64
CDIM $16E8 52
CEND $1A21 61
CENOUT $7293 296
CEQUAL $17A5 54
CEXLVL $37B4 145
CEXPA $1680 51
CEXPF $16A8 51
CEXPI $16A4 51
CEXPR $37B0 145
CEXPS $001B 6
CEXPT $16AA 51
CFACT $17AF 54
CFILNR $1CA7 69
CFOR $1A88 63
CFUNC $1822 56
CGET $1C8B 69
CHAIN $2C93 114
CHKWLK $34A8 137
CHR $25E2 95
CIF $19D4 61
CINLIN $1C7E 69
CINPUT $1C16 67
CLEAR $0EE7 31
CLET $16B7 52
CLKSEP $3B63 155
CLKSTR $3B1F 154
CLKTIK $3AD5 153
CLNR $19BC 60
CLNR0 $1A7D 63
CLOSEF $6533 266
CLRALL $0D27 26
CLRV1 $0D8F 27
CLRVAR $0D34 26
CLS $02B2 13
CLSALL $0D50 26
CLSERR $1141 37
CLSF1 $0DF9 28
CLSPRG $110F 37
CMDGO $3C5A 157
CMDKW $403F 168
CMDLP $0120 9
CMDRES $3C0C 157
CMDTAB $409B 168
CMID $1693 51
CMPLIN $132A 42
CMPST2 $133A 43
CMPSTM $1351 43
CNAME $6F68 288
CNEXT $1ABC 63
COLARG $2509 92
COLKW $08B7 19
COMPS $448D 179
CON $1A33 62
CONCHK $3BF4 156
CONDEV $0180 9
CONDRV $0190 10
CONGET $036D 15
CONIN2 $02FD 14
CONINP $02FA 14
CONLIN $494D 193
CONNAM $017B 9
CONOPN $01A7 10
CONPRT $01BB 10
CONT $3BDD 156
COPEN $1CBC 69
COPY $628C 258
COS $5684 230
COUT $1AE9 64
CPOKE $1ADF 64
CPOSIT $1CA0 69
CPRINT $1BBA 66
CPUT $1C84 69
CR $0235 12
CRDTAB $6EDB 287
CRDTYP $6184 255
CREAD $1C67 68
CRETRN $19CD 60
CRLF $3CED 159
CRLFC $3CE8 159
CRLFTX $0026 6
CTEXT $1A72 62
CTOFLT $43E2 177
CTOINT $43D1 177
CTXPNT $1AF6 64
CTYPE $43C1 177
CUNARY $3796 144
CUR $2715 98
CURADR $02D2 13
CURDEF $315D 127
CURLFT $0257 12
CVAR $1793 54
CVREF $1770 54
CVREFX $1785 54
CVTFS $3740 143
CVTIS $3744 143
CVTSF $371B 143
CVTSI $371F 143
CWEND $1ABF 63
CWHILE $1AC6 63
CXSTM $1B8A 66
CXY $1ACB 64
CXY1 $1AD7 64
CXY2 $1ACF 64
DADD $5365 220
DARR $3DAE 161
DASGN $3DBF 161
DASGN1 $3D98 161
DBLANK $423C 173
DBUFR2 $6036 251
DBUFRD $6033 251
DBUFW2 $603C 251
DBUFWR $6039 251
DCHK $4250 173
DCLALL $605D 251
DCLOSE $6021 251
DCLSAL $6537 266
DCOMMA $3F78 166
DCOMMN $3F01 164
DCOPY $6042 251
DDEDNT $4143 170
DDEFFN $41B6 171
DDIM $3F00 164
DDIV $54F6 225
DDROP $3D9D 161
DECEND $484D 190
DECNRM $4D7B 206
DECOMP $3D60 160
DECSTR $4513 181
DECTAB $3FED 167
DECTOF $5290 218
DEF $1D99 72
DEFEX1 $1160 37
DEFEXT $115E 37
DEMIT $4249 173
DEMPTY $3F68 165
DEOL $41EB 172
DERR $63F5 262
DERRDV $6CFE 283
DEVLNK $700F 290
DFCLS $67A5 271
DFCRE $65E5 268
DFEXT $6B39 279
DFFIND $69BF 276

DFN $3DF4 162
DFNAME $3FA6 166
DFNCX $6597 267
DFNJP $6048 251
DFUNC $3DD8 162
DGETB $600C 251
DIGCNT $4936 192
DIGITS $3AA8 153
DIMERR $2402 89
DINT $3EBF 164
DIV16 $4B2D 198
DIV32 $4B36 198
DIVBC $4B2F 198
DIVS $5E8A 246
DKWTXT $3FB8 166
DLDRUN $6003 251
DLEN $54E6 225
DLINE $40BB 169
DLNUMS $41A4 171
DLOAD $6006 251
DMERGE $3FCF 167
DMUL $543F 223
DNAMES $4213 172
DNEG $3EF5 164
DNOP $6027 251
DNORM $55BC 227
DNOT $3E16 162
DNUM $3E65 163
DOCMD $13AB 44
DOLINE $11DC 39
DOP $3E24 162
DOPEN $6018 251
DOPENQ $601B 251
DOPR $3E28 162
DOPSP $3E1D 162
DOPTXT $3F89 166
DORETI $03DD 16
DOSCMP $6F83 289
DOSDEV $6E35 287
DOSDR1 $6D95 285
DOSDRV $6D8D 285
DOSDSP $624D 257
DOSFN $6009 251
DOSFN1 $6233 257
DOSINI $6000 251
DOSKW $6F8F 289
DOSRST $6CE6 283
DOSRUN $6F89 289
DOSSTA $6C6C 282
DOSSTH $6FA2 289
DOUBLE $3A76 152
DOVFL $5623 229
DPACK $563A 229
DPAREN $3D9F 161
DPREP $6015 251
DPRVRC $603F 251
DRDREC $602D 251
DRDSCR $600F 251
DRDSR0 $6066 252
DREG $5663 229
DROUND $5600 228
DRUN $601E 251
DRVJP $1152 37
DSELCD $6060 251
DSEP $4247 173
DSETEH $6045 251
DSETEN $6024 251
DSETRC $602A 251
DSHR $566D 230
DSKIP $3D5F 160
DSMALL $3EB6 164
DSPJP $4262 173
DSPTAB $4255 173
DSTMT $40D2 169
DSTR $3ED6 164
DSTR2 $3ED8 164
DTEXT $4164 170
DTHEN $3D8A 161
DVAR $3DB9 161
DVARS $4180 171
DVBUF $495D 193
DVCALL $4953 193
DVEND $4982 193
DVGET $11A0 38
DVINP $4951 193
DVNAME $3DCD 161
DVPRT $119A 38
DVPUT $119D 38
DWRREC $6030 251
DWRSCR $6012 251
DWRSR0 $6063 252
DZERO $562B 229
ED $13CA 44
EMIT $1687 51
ENDST $1D6B 71
EOL $1D64 71
EOSTM $1C01 67
ERASE $1653 50
ERR90 $1E00 73
ERRA $0012 6
ERRCOD $21A2 82
ERRHND $3B80 155
ERRJP $0033 6
ERRMSG $3C84 158
ERROR $0010 6
ERRSP $3B6B 155
ERRSUM $1145 37
ESCAN $357C 139
ESCARG $026B 12
ESCPOS $023A 12
ESTEP $3586 139
EXEC $0020 6
EXECA $1DAA 72
EXECNX $0028 6
EXECOD $1DA9 72
EXP $5937 234
EXT7000 $7000 290
EXTBAC $1173 37
EXTEND $3A69 152
EXTSCN $00E4 8
FADD $4F41 208
FALSE $21FE 83
FARGS $4F2A 208
FARITH $2178 81
FCALL $3A18 151
FCBSRC $11C4 38
FCHS $4C83 203
FCLOS1 $1119 37
FCLOSE $1115 37
FCMP $2119 81
FCOMP $4C61 202
FCOMP2 $4C64 202
FDISP $212E 81
FDIV $505F 211
FDROP $4DC3 206
FDUP $4C21 201
FEXT8 $4C8B 203
FFRAC $2D20 116
FILSPC $4B0E 198
FIX $2D02 116
FLN10 $5A81 236
FLN2 $5927 234
FLOAD $4C25 201
FLOAT $3A5D 152
FLUSH $6558 266
FMDIGS $4692 185
FMEXP $45D1 183
FMFIX $4625 184
FMFREE $4545 182
FMRDEC $46E9 186
FMRFIX $4622 184
FMRMAX $46F7 186
FMRND $46FC 186
FMSEP $46D7 186
FMSGN $458F 182
FMUL $4FEA 210
FMUSE $459F 183
FMZERO $46C9 185
FNAMEX $6F1B 288
FNAMRK $2B0A 110
FNARG $0841 19
FNBCD $43F3 178
FNCALL $2A1B 107
FNCHAR $493C 192
FNDFIL $1188 38
FNDLIN $328C 131
FNDTAB $1DEB 73
FNEG $20FB 80
FNEND $2BE4 112
FNFLT $2185 81
FNFLT2 $2197 82
FNHDR $0704 18
FNIP $4DCD 207
FNKSRC $348F 137
FNKW $0718 18
FNORM $50E6 213
FNPART $4AF0 197
FNPRT1 $4AF5 197
FNRM4 $4D53 205
FNRM8 $4D5B 205
FNRM9 $4D57 205
FNSRC $324B 130
FNSTR1 $43FC 178
FNTAB $0C5D 24
FOR $2783 100
FORMTB $7330 298
FOVFL $5123 214
FPACK $5133 214
FPI $57E0 232
FPOP $4C3F 202
FPOSIT $39E6 150
FPOW $2E01 118
FPOWI $2D61 117
FPUSH $22ED 86
FREG8 $4DDA 207
FRESLT $6D44 284
FRETF $2B56 111
FRETI $2B84 111
FRETS $2B99 111
FRND4 $4C99 203
FROUND $5111 213
FSCAL $57A8 232
FSQR2 $592F 234
FSTLIN $0044 7
FSTNP $4C3C 202
FSUB $4F39 208
FSWAP $4C52 202
FTAN8 $5C63 240
FTODEC $5153 214
FTOI $2162 81
FTOI2 $2161 81
FTOIR $4CF3 204
FTOIT $4CF7 204
FTRUNC $2D4F 117
FZERO $512B 214
GET $3956 149
GETBYT $6228 257
GETINT $4BFA 200
GETKEY $0002 6
GETLNR $4BC2 200
GETPOS $117A 38
GETPS0 $1179 38
GOSUB $2C34 113
GOTO $1D9D 72
HEX $374F 144
IADD $222F 84
IAND $21E0 83
IDIV $223D 84

IEQ $21F8 83
IEQV $21BE 82
IGE $221B 83
IGT $222B 83
IIMP $21CA 82
ILE $2227 83
ILT $220F 83
IM2INI $0166 9
IMUL $223E 84
IMULDV $224F 84
INCOMN $0E10 29
INDEF $3101 126
INE $2203 83
INEG $21B8 82
INIERR $6DD1 286
INIT $0072 7
INITXT $6EBD 287
INLIN2 $018D 10
INLINE $0005 6
INOT $21EA 83
INP $3A0C 151
INPLIN $391D 148
INSCHR $389D 147
INSLIN $12C1 41
INSTR $266D 96
INT $2D06 116
INTCON $21B0 82
INTEGR $3A59 152
INTSTR $4500 181
IOR $21D6 83
IPOW $2DAB 118
ISUB $2234 84
ITOF $2170 81
ITOF2 $216F 81
ITOFS $4CC0 203
IXBC0 $272C 99
IXBC4 $2733 99
IXBC6 $273A 99
IXBC8 $2741 99
IXBCHL $275D 99
IXDE2 $2748 99
IXDE4 $274F 99
IXHL0 $2756 99
IXHL2 $2760 99
IXHL4 $2767 99
IXHL6 $276E 99
IXHLA $2775 99
IXOR $21EE 83
IXST0 $2425 89
IXST2 $256E 93
IXST4 $277C 99
JPTAB $13B7 44
JPTABL $0AC5 21
KBINIT $0416 17
KEYINT $03C1 16
KEYSET $0187 10
KEYWT $0378 15
KWSRCH $4B4B 198
LCOMMN $2F20 121
LDBCHL $30E0 126
LDDEHL $3233 130
LDEF $312B 127
LDFVAR $22E5 86
LDIVAR $22A6 85
LDRUN $6212 257
LDSVAR $2322 87
LEFT $253C 93
LEN $251C 92
LFNCAL $2FE2 123
LFNEND $3142 127
LFOR $3171 127
LINFD $027B 13
LINK $2E27 119
LINK1 $2F0B 121
LINLNG $167D 51
LINNR $3D03 159
LIST $13CE 44
LLINE $302C 124
LNEXT $31DC 129
LNKCLR $314B 127
LNKJP $32C2 132
LNKONE $32C8 132
LNKTAB $3305 132
LNRNG $158B 48
LOAD $1499 46
LOADF $61CC 256
LOADRC $61D7 256
LOG $5828 233
LOG10 $5A60 236
LOOKUP $6545 266
LSTLIN $152B 48
LSTSRC $3256 130
LSTWK2 $0D68 27
LSTWLK $0D65 27
LVAR $2F19 121
LWEND $31C7 128
LWHILE $31B9 128
MEMCHK $0FB3 33
MEMERR $3B69 155
MERGE $149D 46
MID $2BEC 112
MIDF $2532 93
MKARR $23A3 88
MOD $223A 84
MUL16 $4B19 198
MULS $5DEB 245
NAMCNT $3474 137
NAMHD $3447 136
NAMOFS $347F 137
NAMPAK $3416 135
NAMSET $3442 136
NAMSRC $3457 136
NAMTX1 $33C3 135
NAMTXT $33C0 135
NAMWLK $34A0 137
NEGHL $226D 84
NEW $0EC6 30
NEWLIN $0278 12
NEWSTK $1375 43
NEXT $2809 102
NMI $0066 7
NMICLK $3AB3 153
NOEXT $3A6D 152
NOHOOK $19CC 60
NOLINK $3129 127
NOTRAC $2003 78
NULDEV $11B2 38
NULDRV $11B9 38
NUMCON $20EE 80
NUMPSH $4800 189
NUMS $4439 179
NXTADD $2885 102
NXTCHR $4C10 201
NXTREC $63C8 262
OCT $3752 144
ON $2C65 114
ONERR $2C13 113
ONERR0 $2C28 113
OPEN $1035 34
OPENF $6512 266
OPENIX $105F 35
OPENQ $6502 265
OPGRP $06AA 17
OPKW $06C8 17
OPNBLK $3166 127
OPNINI $6DFA 286
OPTBAS $3AAE 153
OUTST $3A3F 152
P10EXP $51F0 216
P10LD $533E 220
PARITY $72A8 297
PARSFN $4AB6 197
PARSRC $362D 141
PARTAB $732B 298
PEEK $21A6 82
PEEK2 $22AA 85
PI $22E9 86
PIOINT $03DF 16
POKE $3A2D 151
POSIT $399A 150
POSLD $499C 194
POSST $4992 194
POW10 $4DF2 207
PR2DIG $3B56 155
PRAREA $703F 291
PRBRK $718B 294
PRCHR $71BB 295
PRCLOS $72C8 297
PRCON $1196 38
PRDEF $7335 298
PRDEV $7064 291
PRDFLT $709D 292
PRDRV $706B 291
PREPAR $38B2 147
PREPF $6516 266
PRFFLF $723B 296
PRFILE $1197 38
PRFREE $452E 181
PRGCHK $32B5 132
PRGCLS $1105 36
PRGINI $1018 34
PRGSIZ $1009 34
PRINIT $7003 290
PRINT $1E02 73
PRMARG $71AE 295
PRNEW $70BB 292
PROMPT $3D46 160
PROPEN $707E 291
PROUT $7247 296
PRPEND $72D9 297
PRPRT $719D 294
PRSPEC $70C5 292
PRTXT $000B 6
PRTYPT $7328 298
PRVREC $644E 264
PRWTAB $7321 298
PSCAN $3552 139
PSCAN1 $3555 139
PSTEP $355F 139
PTRF $22C8 86
PTRI $22CE 86
PTRS $22CB 86
PUT $397E 149
RAMINI $0147 9
RANDOM $3537 138
RDREC $6495 265
RDSEC $6077 252
RDVALS $2912 104
READ $28EE 104
READY $0110 9
READY0 $010C 9
RECLFT $6480 264
RECSEC $62B2 259
REFADR $0F68 32
RENUM $427C 173
RESCHK $3C49 157
RESET $0000 6
RESTOR $28C6 103
RESTR0 $28B7 103
RESUM0 $3C13 157
RESUME $3C22 157
RETIS $00C2 8
RETURN $2C46 113
RIGHT $2554 93
RIGHT1 $255D 93
RNARGS $430D 175
RND $34B1 137
RNDADD $354D 138

RNDCLR $353A 138
RNDMUL $3548 138
RNLINE $305A 124
RNUNLK $32D9 132
ROWTAB $03E6 16
RST08 $0008 6
RUBOUT $035E 15
RUN $1CE9 70
RUN0 $1CF3 70
RUNCLR $0E26 29
RUNFIL $650C 266
RUNLP $1D07 70
SALLOC $242C 90
SAVE $1423 45
SCAT $2575 94
SCLOSE $38FB 148
SCMP $260A 95
SCNCB $35A8 140
SCNCOD $35D8 140
SCNEJP $36AD 142
SCNREF $35A5 140
SCNSJP $3689 142
SCNSTM $3657 142
SCOMMN $3618 141
SCRCHR $01D6 11
SCROLL $0287 13
SCRPUT $0215 11
SCRTXT $01C3 10
SDEF $35E7 140
SECIO $608F 252
SELCRD $616F 254
SERIES $5C6B 240
SETBLK $4AA8 196
SETCNT $4AAF 196
SETDEF $3159 127
SETEH $643F 263
SETEND $656E 267
SETREC $6457 264
SETREL $327C 131
SFARR $22D5 86
SFOR $35FA 141
SGN $2102 80
SHNAME $331F 133
SIARR $22A2 85
SIN $5680 230
SINGLE $3A73 152
SKIP $1D8D 71
SKIP2 $1D8B 71
SKIPB0 $4C1A 201
SKIPBL $0018 6
SLINE $35BE 140
SLTBUF $61BE 255
SLTCHK $6408 263
SLTCPY $627F 258
SMLINT $21A9 82
SONGO $35C8 140
SOPEN $38B0 147
SPACE $26D4 98
SPCHK $0FF6 33
SPCHK0 $0FEF 33
SPCHR $7177 294
SPDIG $7180 294
SPERR $7175 294
SPTAB $716F 294
SQR $5A89 237
SSARR $2316 86
STBCHL $323A 130
STDEHL $323F 130
STFVAR $22B0 85
STHDR $092E 20
STIVAR $2298 85
STKW $0942 20
STKW2 $0A21 21
STMEND $13C5 44
STMLP $1D56 71
STOP $3B7B 155
STRCON $2510 92
STRDEC $485E 190
STRNG $26D9 98
STRNUM $4482 179
STSVAR $22F4 86
SUBS $5CCB 241
SVALOC $2408 89
SVSET $2450 90
SWAP $2248 84
SYS $44BF 180
TAB $1FB7 77
TAN $5688 230
TANP8 $57B2 232
THEN $1D92 72
TRACE $2CEA 115
TRCCOD $2CE1 115
TRCHOK $2CC7 115
TRCLNR $2CFC 116
TRGEND $5788 232
TRUE $220A 83
TXTDEC $486D 190
TXTNUM $481D 189
UFOR $30E8 126
ULKJP $32E6 132
ULKPRT $32DD 132
ULKTAB $32EB 132
ULREF $3067 124
ULVAL $3080 125
UNFORM $72F5 298
UNITIX $7316 298
UNITNR $61B2 255
UNLCNT $7300 298
UNLINK $2EFD 120
UNLNK0 $2F05 120
UNSCNT $730B 298
UNSDEF $0036 6
UNSRUT $6EFC 287
UNTYPE $72EA 297
USEDEC $4867 190
USEFLD $473D 187
VAL $218C 82
VARADR $2293 85
VARLNK $2F3A 121
VARNAM $3343 133
VAROOT $3706 143
VARPTR $36F9 143
VARREF $0F63 32
VARSRC $3251 130
VDSC $245D 90
VDSC0 $245C 90
VNAME $333E 133
WHILE $28A7 103
WRREC $64CA 265
WRSEC $6084 252
XBYE $6D37 284
XCOMMN $24AD 91
XDIM $24AC 91
XKILL $6F06 288
XNAME $6F31 288
XNUM $1B95 66
XSTM $1D71 71
ZERO $0FD6 33